GLOSSO

Language reference

Syntax, semantics, and the compilation model

A printable reference edition of the Glosso Manual, with the complete language chapters, code examples, grammar, and diagnostic reference.

146 sections / 16 topic groups / 1310 examples and transcripts

Finding your way

This edition follows the chapter order and content of the manual in this repository. It preserves the prose, examples, output, notes, and reference tables. The contents and the PDF bookmarks link to the language sections.

Examples include both successful programs and deliberately invalid programs followed by their diagnostics. Target-specific output and paths are reproduced as written in the manual. The final topic group contains the standard library map, grammar reference, diagnostic reference, and style guidelines; the website's generated per-symbol standard library API is a separate reference.

Quick lookup

LOOK UP	START HERE
Program entry and the compiler	Program entry · Compiler options · Targets
Imports and visibility	Imports and loads · Visibility and ownership units
Declarations	Binding forms · Scope and shadowing
Literals and primitive types	Integers · Strings · Primitive types
Conversions and precedence	Implicit conversions · Casts · Precedence
Aggregates	Structs · Enums · Unions
Pointers and collections	Pointers · Arrays and views · Slicing
Functions and generics	Functions · Generics · Constraints
Typeclasses and operators	Typeclasses · Instances · Operators
Options and error handling	Options · Results · Try · Propagation
Branching and patterns	Switches · ifx · Patterns
Iteration	while · for · Custom iteration
Compile-time programming	Comptime · Macros · Reflection
Memory and resource ownership	Allocators · Drop · Moves · Lifetimes
C interop and builds	Calling C · Bindgen · Build scripts
Grammar and diagnostics	Grammar · Diagnostics · Style

Contents

1 Getting started 5
1.1 Hello World and Program Entry 5
1.2 Using the Compiler 8
1.3 Backends, Artifacts, and Optimization Levels 11
1.4 The Compilation Pipeline 14
1.5 Source Files, Imports, and Loads 17
1.6 The Implicit Prelude 20
2 Lexical structure 24
2.1 Source Text, Whitespace, and Comments 24
2.2 Identifiers and Keywords 26
2.3 Integer Literals 28
2.4 Float Literals 30
2.5 String Literals and Escapes 32
2.6 `#string` Multi-Line Blocks 35
2.7 `u8`, `rune`, and `#char` Literals 38
2.8 Punctuation and the Token Table 40
2.9 Reserved Operators and Operator Names 42
2.10 The Operator Prescan 44
3 Declarations and scope 47
3.1 Values, Constants, and Variables 47
3.2 Top-Level Declarations and Thread-Local Storage 50
3.3 Assignment and Update Assignment 52
3.4 Blocks, Scope, and Shadowing 55
3.5 Visibility and Ownership Units 57
3.6 `defer` and `#try defer` 59
4 Primitive types 63
4.1 The Primitive Types 63
4.2 Integers 67
4.3 Floating Point 70
4.4 Complex Numbers 73
4.5 Booleans and Truthiness 75
4.6 Built-in Strings 77
4.7 SIMD Vectors and Matrices 81
4.8 Types as Values 84
4.9 Compile-Time Numbers and Literal Defaulting 87
4.10 Conversions: What Is Implicit 89
4.11 `cast` and `acast` 91
4.12 `distinct` Nominal Types 94
4.13 Relaxing Strictness with `lossy_numerics` 96
4.14 Overflow, Traps, and the `-O` Level 97
4.15 Operator Precedence and the 0-20 Scale 99
5 Structs, unions, and enums 102
5.1 Struct Declarations 102
5.2 Struct Memory Layout 105
5.3 `using`, `#empty`, and `#as` 108
5.4 Inline Aggregate Field Types 110
5.5 Tuples 113
5.6 Enums and Flag Enums 115
5.7 Tagged Unions 118
5.8 Reading a Union Payload 120
5.9 `#from` Variants 123
5.10 Raw Unions 125
5.11 The Option Type `?T` 128
5.12 The Result Type `E!T` 131
6 Pointers and collections 135
6.1 Pointers 135
6.2 Many-Pointers and Null 135
6.3 Fixed Arrays, Views, and Dynamic Arrays 136
6.4 Ranges and Slicing 137
6.5 Structure-of-Arrays and Array-of-Structs 140
6.6 Aggregate and Collection Literals 143
6.7 Member Access, Calls, and Argument Expansion 146
7 Functions and generics 150
7.1 Function Declarations 150
7.2 Default and Named Parameters 153
7.3 Variadic Parameters 156
7.4 `noalias` Pointer Parameters 159
7.5 Lazy Parameters and Short-Circuit Operators 159
7.6 Compile-Time and Type-Only Parameters 163
7.7 Generic Functions 165
7.8 Constraints with `where` 168
7.9 Structural Constraints with `#meaningful` 171
7.10 Lambdas 174
7.11 Function Pointers 177
7.12 Function and Aggregate Directives 179
8 Typeclasses 184
8.1 Typeclasses: Declaring a Contract 184
8.2 `#minimal` and Required Methods 187
8.3 Instances: Heads, Bodies, and `where` 190
8.4 Associated Types 193
8.5 Evidence, Constraints, and Negative Instances 197
8.6 How an Instance Is Chosen 200
8.7 Higher-Kinded Typeclasses 203
8.8 Deriving Instances with `#derive` 206
8.9 Writing a Derive Macro 209
8.10 Equality and Ordering 213
8.11 `Show`, `Formatter`, and Text Conversion 216
8.12 `Try`, `?`, and `#try` 220
8.13 `Propagate` and Multi-Error Programs 224
8.14 `Error`, `Termination`, and `main` 226
8.15 Typeclass-Constrained Variadics 229
9 Operators 232
9.1 The Operator System 232
9.2 Assignment Operators 235
9.3 Indexing and Slicing Operators 237
9.4 Iteration and Range Typeclasses 239
10 Control flow 243
10.1 Statements and Semicolons 243
10.2 `if`, `else`, and Truthiness 246
10.3 Switching on a Value 248
10.4 `ifx` Expressions 250
10.5 Pattern Matching with `#pattern` 253
10.6 The Pattern Vocabulary 256
10.7 Ranges in Patterns and `ToPattern` 259
10.8 Exhaustiveness, Guards, and Reachability 262
10.9 `case A, B;` Groups and `#falling` 264
10.10 Bind or Diverge: `#pattern P := subject else` 267
10.11 Patterns as Compile-Time Values 269
10.12 Static Branching with `#if` 272
10.13 `while` and `#inline while` 275
10.14 `for` Loops 277
10.15 Custom Iteration with `for_expansion` 280
10.16 `#inline for` 283
10.17 Labels and `goto` 286
10.18 Explicit Control Transfer with the Jump Library 290
11 Compile-time programming 295
11.1 What Runs When 295
11.2 `#comptime` 298
11.3 Compile-Time Constants 300
11.4 `#assert` and `#compile_error` 303
11.5 Code Values: `#code` and Backticks 307
11.6 `#insert` 308
11.7 Macros with `#expand` 310
11.8 Reflection 313
11.9 Pattern Reflection 316
11.10 Build Scripts 320
11.11 The Compile-Time Command Line and Artifact Cache 325
12 Memory and context 329
12.1 The Implicit Context 329
12.2 Allocators 331
12.3 Temporary Allocation 334
12.4 The Debug Allocator 336
12.5 Automatic Destruction with `Drop` 339
12.6 Moves and Managed Values 339
12.7 Dynamic Array and Container Ownership 340
12.8 Memory Contracts and Storage Lifetimes 340
12.9 Owning Foreign Resources 342
12.10 Manual Destruction with `ManuallyDrop` 343
12.11 Cleanup Errors and `#on_drop_error` 343
12.12 Feature Gates: `#enable` and `#disable` 344
13 Runtime behaviour 347
13.1 Runtime Safety Checks and Panics 347
13.2 Sanitizers 350
13.3 Environment, Toolchain, and Exit Codes 352
14 Interop and low-level code 356
14.1 Calling C from Glosso 356
14.2 Being Called from C 358
14.3 C Data Across the Boundary 361
14.4 Generating Bindings with Bindgen 365
14.5 Inline Assembly and Inline Bytes 367
14.6 `#magic` and the Intrinsic Table 371
14.7 Embedding Lua 374
15 Targets and build 378
15.1 Targets and Cross-Compilation 378
15.2 Freestanding, Bare Metal, UEFI, and WebAssembly 381

15.3 Diagnostics: Reading and Configuring Them 384
15.4 The Build Cache and Generated Sources 386
16 Reference 389
16.1 The Standard Library Map 389
16.2 Grammar Reference 394
16.3 Diagnostic Reference 400
16.4 Style Guidelines for Glosso Code 409

1 Getting started

1.1 Hello World and Program Entry

A Glosso program is one source file handed to the compiler. Execution begins at the procedure named `main`, which every hosted program must declare. The smallest complete program declares it and nothing else.

GLOSSO
```
main :: () {}
```

`::` binds a name to a compile-time constant, and the constant here is a procedure: an empty parameter list, no return type, an empty body. Compiling that file links an executable that starts, returns immediately, and reports status 0. Nothing is printed because nothing asked for output.

Printing

`print` is an ordinary procedure, not a statement or a keyword, and it is not one of the names the implicit prelude supplies. It comes from the `Io` module, so a program that prints begins with an import. The Implicit Prelude lists what does arrive with no import at all.

GLOSSO
```
#import "Io";

main :: () {
    print("hello, world\n");
}
```

CONSOLE / OUTPUT
```
$ glosso --run hello.glo
glosso: compile hello.glo (target=x86_64-pc-windows-msvc)
glosso: link hello.exe (target=x86_64-pc-windows-msvc, -O1)
glosso: done hello.exe  (backend=llvm, target=x86_64-pc-windows-msvc, -O1)
hello, world
```

Omitting the import is the first mistake a new program makes. The checker does not know `print`, and the name it suggests instead is `tprint`, the temp-allocated formatting helper `Format` supplies. That one is not in scope either; the suggestion reaches it because the Prelude imports `Format` for its own use.

CONSOLE / OUTPUT
```
// main :: () { print("hi\n"); }
error: unknown function print
help: did you mean 'tprint'?
```

What the build prints

Every line the driver writes about its own progress goes to stderr and is prefixed `glosso: `. Stdout belongs to the program and to compile-time execution, so the two streams can be separated without filtering text. A default build prints three lines; the rest of the vocabulary appears only for particular backends and options.

Progress line	When it appears
`glosso: compile <source> (target=<triple>)`	Once, before checking begins, on every build.
`glosso: link <output> (target=<triple>, <-O flag>)`	For the backends that invoke a linker or an archiver: `llvm`, `static-lib`, `shared-lib`.
`glosso: done <output>  (backend=…, target=…, <-O flag>)`	After a linked artifact has been written.
`glosso: wrote <output>  (backend=…, target=…, <-O flag>)`	In place of `link` and `done`, for the unlinked backends `object` and `llvm-ir`.
`glosso: wrote import library <path>`	`shared-lib` only, on the targets that produce one.
`glosso: restored <output> from the native artifact cache`	In place of every other line, when an unchanged build is served from the cache.
`glosso: program exited with status <status>`	`--run` only, and only when the program's status is not 0.

CONSOLE / OUTPUT
```
$ glosso --run hello.glo 2>/dev/null
hello, world
```

The signature of `main`

`main` takes no parameters. There is no argument-vector form of the declaration on a hosted target; command-line arguments are read with `get_cmdline_args()`, which the prelude supplies. The one exception is the freestanding WebAssembly entry point, described in Freestanding, Bare Metal, UEFI, and WebAssembly.

CONSOLE / OUTPUT
```
// main :: (args: []string) {}
error: main must not take parameters; use get_cmdline_args() for command-line arguments
```

The return type is optional. A `main` with no return type, or with an explicit `-> void`, always reports status 0. Any other return type is accepted only if it implements the `Termination` typeclass, which is the protocol that turns a returned value into a process status.

GLOSSO
```
Termination :: typeclass (T: type) #minimal(termination_status) {
    termination_status :: (value: T) -> s32;
}

$T ::: instance Termination where Is_Integer(T) {
    termination_status :: (value: T) -> s32 {
        return cast(s32)value;
    }
}
```

That instance is why every integer type works as a return type directly. The value is narrowed to `s32` and handed to the host, which then applies its own rules for what a status may be.

GLOSSO
```
#import "Io";

main :: () -> ssize {
    print("working\n");
    return 3;
}
```

CONSOLE / OUTPUT
```
$ glosso -o status.exe status.glo
glosso: compile status.glo (target=x86_64-pc-windows-msvc)
glosso: link status.exe (target=x86_64-pc-windows-msvc, -O1)
glosso: done status.exe  (backend=llvm, target=x86_64-pc-windows-msvc, -O1)
$ ./status.exe
working
$ echo $?
3
```

The protocol is open. Declaring an instance for an application's own result type makes it a legal return type for `main` and centralizes the conversion to a process status. `Termination` is exported through the Prelude; the class itself requires no explicit import.

GLOSSO
```
#import "Io";
#import "Typeclass";
#import "Prelude/Termination";

Exit_Reason :: enum { Success; Bad_Input; No_Space; }

Exit_Reason ::: instance Termination {
    termination_status :: (value: Exit_Reason) -> s32 {
        if value == {
        case .Success;   return 0;
        case .Bad_Input; return 64;
        case .No_Space;  return 70;
        }
    }
}

main :: () -> Exit_Reason {
    print("checking input\n");
    return .Bad_Input;
}
```

CONSOLE / OUTPUT

```
$ ./reason.exe
checking input
$ echo $?
64
```

The prelude also gives `E!void` a `Termination` instance for every error type `E` that implements `Error`. `.Ok` is status 0; `.Err` calls `report_error` on the payload and then reports status 1. A `main` declared that way can use `?` on any failing call in its body. The reporting protocol itself belongs to `Error`, `Termination`, and `main`.

GLOSSO

```
#import "Io";
#import "Typeclass";
#import "Typeclass/Error";

App_Error :: struct { message: string; }

App_Error ::: instance Error {
    report_error :: (error: App_Error) {
        ("app").error_report_line(error.message);
    }
}

step :: (ok: bool) -> App_Error!s64 {
    if !ok return .Err(.{ .message = "step failed" });
    return .Ok(7);
}

main :: () -> App_Error!void {
    value := step(false)?;
    print("value = %\n", value);
    return .Ok;
}
```

CONSOLE / OUTPUT

```
$ ./apperr.exe
app: step failed
$ echo $?
1
```

Declaration of `main`	Process status
`main :: () { … }`, `main :: () -> void { … }`	Always 0.
`main :: () -> ssize { … }` and any other integer type	The returned value, narrowed to `s32`.
`main :: () -> E!void { … }` where `E` implements `Error`	0 for `.Ok`; 1 for `.Err`, after `report_error` has run.
`main :: () -> T { … }` for any `T` with a `Termination` instance	Whatever that instance's `termination_status` returns.
Any other return type	Rejected while checking, for example `error: main return type bool does not implement Termination`.

NOTE

`--run` reports the program's status but does not adopt it. A program that exits 3 makes the compiler print `glosso: program exited with status exit code: 3` and then exit 1 itself. An exact status is observable only by building the executable and running it directly.

`exit(status)` ends the process immediately from anywhere in the program. It is also supplied by the prelude, so it needs no import, and unlike a `return` from `main` it does not run the remaining `defer` statements.

GLOSSO

```
#import "Io";

main :: () {
    print("before exit\n");
    exit(9);
    print("never printed\n");
}
```

CONSOLE / OUTPUT

```
$ ./exitp.exe
before exit
$ echo $?
9
```

A file with no `main` is not a compile error. Checking succeeds, the backend emits a module with no entry point, and the platform linker is the component that complains. Building a library rather than a program is what `--backend static-lib`, `--backend shared-lib`, and `--no-main` are for; they are covered in Using the Compiler and in Backends, Artifacts, and Optimization Levels.

CONSOLE / OUTPUT

```
$ glosso helper.glo
glosso: compile helper.glo (target=x86_64-pc-windows-msvc)
glosso: link helper.exe (target=x86_64-pc-windows-msvc, -O1)
error: lld-link: error: undefined symbol: wmain
```

1.2 Using the Compiler

The driver compiles exactly one source file: `glosso [options] <file.glo>`. There is no project file, no source list, and no separate link step. Everything else a program needs enters through `#import` and `#load` from inside that one file, and the whole graph reached that way is a single compilation unit.

GLOSSO

```
// app.glo
#import "Io";
#load "parts/geometry.glo";

main :: () {
    print("area = %\n", area(6, 7));
}
```

CONSOLE / OUTPUT

```
$ glosso --run app.glo
glosso: compile app.glo (target=x86_64-pc-windows-msvc)
glosso: link app.exe (target=x86_64-pc-windows-msvc, -O1)
glosso: done app.exe  (backend=llvm, target=x86_64-pc-windows-msvc, -O1)
area = 42
```

`parts/geometry.glo` is part of the program but is not a program: naming it on the command line instead compiles it on its own, and it then has no `main` to link against. Which file is named therefore decides what is built. If more than one positional argument is given the last one silently wins; `glosso a.glo b.glo` builds `b.exe` from `b.glo` and never mentions `a.glo`.

The option list

`glosso --help` prints the complete list and exits 0. It is printed before the backend, the target, and the source file are validated, so it works alongside otherwise invalid options. There is no `--version` option: `glosso --version` is `glosso: unknown option --version`.

Every option that takes a value accepts both the separated and the `=`-joined spelling, so `--backend=object`, `--color=never`, `--std=<dir>`, `--target=wasm32-wasi`, `--cache-dir=<dir>`, `--linker-arg=<arg>`, and `--temp-allocator-size=8192` are equivalent to the two-token forms. `-o` is the exception: it has no `=` form, and `-o=out.obj` is rejected with `glosso: unknown option -o=out.obj`.

Option	Value	Default
`--backend`	`llvm`, `object`, `static-lib`, `shared-lib`, `llvm-ir`	`llvm`
`--target`	an `os-cpu-abi` or `cpu-os-abi` triple	the host
`--print-targets`, `--targets`	aliases; print the target list and exit	not applicable
`-O0 -O1 -O2 -O3`	none	`-O1`
`-Os -Oz -Ozz`	none	no size mode
`-o`	an output path, separated form only	derived from the source stem
`--linker-arg`	one raw linker argument; repeatable	none
`--memory-sanitizer`	none	off
`--thread-sanitizer`	none	off
`--borrow-check`	`off`, `warning`, `error`	`warning`

Option	Value	Default
`--borrow-check-std`	none; include standard-library bodies	off
`--memory-signatures`, `--memory-axioms`, `--memory-report`	none; inferred contracts, external contracts, inventory	off
`--temp-allocator-size`	a positive byte count	`4194304` (4 MiB)
`--no-main`	none	off
`--use-std`	none; imports every bulk standard module unqualified	off
`--run`	none; `--backend llvm` and the host target only	off
`--cache-dir`	an absolute directory	the per-user cache root
`--print-cache-dir`	none; prints and exits	not applicable
`--show-generated`	none	off
`--color`	`auto`, `always`, `never`	`auto`
`--std`	a directory holding `Prelude.glo` or `Prelude/`	auto-discovered
`--module-path`	an existing directory; repeatable	none
`--`	everything after it, as compile-time arguments	empty
`-h`, `--help`	none; prints and exits	not applicable

Without `-o` the output name is derived from the source file's stem and the chosen backend and target; the full set of default names is in Backends, Artifacts, and Optimization Levels. `-o` with a path whose parent directory does not exist creates the directory. `-o ""` is not an error; the empty string falls back to the backend default.

`--std`

`--std <dir>` names the standard-library root. The directory must contain either a `Prelude.glo` file or a `Prelude/` directory, and anything else is `glosso: --std directory must contain Prelude.glo or Prelude/`. Without the option the driver searches, in order, `$GLOSSO_STD`, `<cwd>/std`, and then `std` beside the compiler executable and one and two directories above it, followed by the per-user install locations; the first candidate that holds a prelude wins. Finding none is `glosso: could not find the project-local or installed Glosso standard library`.

`--use-std`

`--use-std` loads every top-level single-file module and every directory module marked by `module.glo`, unqualified, after the Prelude. It is a conformance convenience rather than the normal import model. Namespace-only directories such as `File_System` are not bulk modules, so their components still need explicit imports.

`--run`

`--run` builds the program and then executes it with no arguments, inheriting stdout and stderr. It is accepted only for the default `llvm` backend, because no other backend produces something that can be executed, and the target must be the host.

CONSOLE / OUTPUT

```
$ glosso --run --backend object hello.glo
glosso: --run is available only for --backend llvm

$ glosso --run --target x86_64-linux-musl hello.glo
glosso: --run can only execute the host target (got x86_64-pc-linux-musl)
```

> **NOTE**
> The first of those two is a command-line error and exits 2; the second is discovered after the options parse and exits 1. `--run` never adopts the program's own status either, so it is a convenience for development rather than a way to script a build.

`--no-main`

Reachability normally starts at `main`. `--no-main` starts it at every `#c_call` function that has a body instead, which is what a library wants; the `static-lib` and `shared-lib` backends do this on their own and do not need the flag. The effect is visible in the emitted module: with the flag, the allocators, the formatters, and the hosted entry wrapper are all gone.

GLOSSO
```
add  :: (a: s32, b: s32) -> s32 #c_call { return a + b; }
main :: () #c_call { }
```

CONSOLE / OUTPUT
```
$ glosso --no-main --backend llvm-ir -o roots.ll roots.glo
$ grep '^define' roots.ll
define i32 @glo_add(i32 %0, i32 %1) {
define void @glo_main() {

$ glosso --backend llvm-ir -o roots2.ll roots.glo
$ grep -c '^define' roots2.ll
28
```

Combining `--no-main` with the executable backend on a hosted target is a mistake: the flag suppresses the platform entry wrapper as well, so the link fails with `error: lld-link: error: undefined symbol: wmain` on Windows and the equivalent elsewhere. The flag belongs with an object, library, or bare-metal build.

`--color`

`--color auto` is the default and enables colored diagnostics only when stderr is a terminal. `--color always` and `--color never` force the choice, and any other value is `glosso: --color must be auto, always, or never`. Setting `NO_COLOR` in the environment turns color off whatever the flag says, including `--color always`.

Compile-time arguments after `--`

`get_cmdline_args()` returns the process argument vector when the program runs, and the arguments written after `--` when the same call is made during compile-time execution. That is how a build script written in Glosso is parameterized without an extra configuration file.

GLOSSO
```
#import "Io";

show_args :: (label: string) {
    args := get_cmdline_args();
    print("% count = %\n", label, args.count);
    for arg: args print("%   %\n", label, arg);
}

#comptime { show_args("[comptime]"); }

main :: () {
    show_args("[runtime] ");
}
```

CONSOLE / OUTPUT
```
$ glosso -o cmdargs.exe cmdargs.glo -- build --release "two words"
glosso: compile cmdargs.glo (target=x86_64-pc-windows-msvc)
[comptime] count = 3
[comptime]   build
[comptime]   --release
[comptime]   two words
glosso: link cmdargs.exe (target=x86_64-pc-windows-msvc, -O1)
glosso: done cmdargs.exe  (backend=llvm, target=x86_64-pc-windows-msvc, -O1)
$ ./cmdargs.exe
[runtime]  count = 1
[runtime]    C:\Users\almag\...\cmdargs.exe
```

The two vectors differ in more than their contents: the runtime vector begins with the executable path, and the compile-time one does not. `--` also swallows the whole rest of the command line, so the source file and every driver option have to appear before it. For a nested `add_build_file` or `add_build_string` call, the elements of `Build_Options.compile_time_command_line` occupy this same tail, one argument per element and after an implicit `--`.

Exit codes

Situation	Exit code
A successful build, and the options that print and stop: `--help`, `--print-targets`/`--targets`, `--print-cache-dir`	`0`
A compile or type error, an unreadable source file, a link failure, `--run` on a non-host target, or a program that `--run` executed and that exited nonzero	`1`
A top-level `#comptime` that called `exit(n)`	`n`, and no artifact is written

Command-line errors are plain `glosso:` lines with no source frame, because there is no source to frame yet. An unknown option is followed by the full usage block.

- `-O` accepts a single digit 0 through 3 and nothing else. `-O4` is not a clamped level, it is `glosso: unknown option -O4`.

The size modes -Os, -Oz, and -Ozz produce runnable artifacts. They combine LLVM optimization with optsize/minsize attributes and target-specific linker size policies. Runtime checks are disabled in these modes; use -O0 or -O1 for checked builds. -Ozz also strips symbols and metadata.

- `--memory-sanitizer` and `--thread-sanitizer` are mutually exclusive rather than last-wins; asking for both is a command-line error.

`--unit-source UNIT=FILE`	exact source ownership mapping; repeatable; enables explicit mapping for all non-standard files	inferred from project and module directories

1.3 Backends, Artifacts, and Optimization Levels

`--backend` chooses what a build produces. All five backends go through LLVM, and the one that produces an executable is spelled `llvm`; `llvm-ir` is the one that stops at textual IR instead of generating machine code.

`--backend`	Produces	Accepts `--run`
`llvm` (default)	A linked native executable	yes
`object`	One relocatable object file, not linked	no
`static-lib`	A static archive, assembled by the archiver	no
`shared-lib`	A shared library, plus an import library where the target has one	no
`llvm-ir`	Textual LLVM IR, not linked	no

`exe` and `executable` are not spellings of the first one. An unrecognized name is a command-line error, reported before the source file is read: `glosso: unknown backend 'exe' (choose llvm, object, static-lib, shared-lib, or llvm-ir)`. The library backends root reachability at the defined `#c_call` functions on their own, so they do not need `--no-main`.

CONSOLE / OUTPUT

```
$ glosso --backend object lib.glo
glosso: compile lib.glo (target=x86_64-pc-windows-msvc)
glosso: wrote lib.obj  (backend=object, target=x86_64-pc-windows-msvc, -O1)

$ glosso --backend static-lib lib.glo
glosso: compile lib.glo (target=x86_64-pc-windows-msvc)
glosso: link lib.lib (target=x86_64-pc-windows-msvc, -O1)
glosso: done lib.lib  (backend=static-lib, target=x86_64-pc-windows-msvc, -O1)

$ glosso --backend shared-lib lib.glo
glosso: compile lib.glo (target=x86_64-pc-windows-msvc)
glosso: link lib.dll (target=x86_64-pc-windows-msvc, -O1)
glosso: wrote import library lib.dll.lib
glosso: done lib.dll  (backend=shared-lib, target=x86_64-pc-windows-msvc, -O1)

$ glosso --backend llvm-ir lib.glo
glosso: compile lib.glo (target=x86_64-pc-windows-msvc)
glosso: wrote lib.ll  (backend=llvm-ir, target=x86_64-pc-windows-msvc, -O1)
```

Default artifact names

Without `-o`, the output name is the source file's stem plus whatever naming convention the target uses for that kind of artifact. The stem is taken from the file, not from any declaration inside it, so `lib.glo` gives the names below.

Backend	Default output for `lib.glo`
`llvm`	`lib.exe` on Windows, `lib.wasm` on a WebAssembly target, `lib` with no extension on ELF and Mach-O targets.
`object`	`lib.obj` on `*-windows-msvc`, `lib.o` on every other target including `*-windows-gnu`.
`static-lib`	`lib.lib` on Windows targets that are not `-gnu`, `liblib.a` everywhere else.
`shared-lib`	`lib.dll` on Windows, `liblib.dylib` on Darwin, `liblib.wasm` on WebAssembly, `liblib.so` elsewhere.
`llvm-ir`	`lib.ll` on every target.

A Windows `shared-lib` build writes a second file, the import library that a consumer links against: `lib.dll.lib` under the MSVC ABI and `liblib.dll.a` under the GNU one. It is announced by its own progress line, and it is the file to name in a `#library` declaration on the consuming side, not the `.dll` itself.

CONSOLE / OUTPUT

```
# all four library backends, one target per line
x86_64-windows-msvc:  lib.obj  lib.lib   lib.dll  lib.dll.lib   lib.ll
x86_64-windows-gnu:   lib.o    liblib.a  lib.dll  liblib.dll.a  lib.ll
x86_64-linux-gnu:     lib.o    liblib.a  liblib.so              lib.ll
wasm32-wasi:          lib.o    liblib.a  liblib.wasm            lib.ll
```

Only `llvm`, `object`, and `static-lib` participate in the native artifact cache; a `shared-lib` or `llvm-ir` build recompiles every time. Building the same `llvm-ir` file twice therefore prints `glosso: wrote lib.ll` twice, where the object build prints `glosso: restored lib.obj from the native artifact cache` the second time. The cache itself is described in The Build Cache and Generated Sources.

Optimization levels

The size modes -Os, -Oz, and -Ozz produce runnable artifacts. They combine LLVM optimization with optsize/minsize attributes and target-specific linker size policies. Runtime checks are disabled in these modes; use -O0 or -O1 for checked builds. -Ozz also strips symbols and metadata.

Flag	Request	Current result
`-O0`	level 0	builds; runtime checks on; the default allocator remains the heap allocator
`-O1` (default)	level 1	builds; runtime checks on
`-O2`	level 2	builds; runtime checks off
`-O3`	level 3	builds; runtime checks off
`-Os`	level 2 plus `Small`	LLVM O2 with `optsize`; removes unused sections
`-Oz`	level 3 plus `VerySmall`	LLVM O3 with `optsize` and `minsize`; folds identical code when supported
`-Ozz`	level 3 plus `Tiny`	the `-Oz` size request plus stripping symbols and metadata

Runtime checks are configured exactly at underlying level 0 or 1. Levels 2 and 3 remove overflow, bounds, tag, and null checks, so the three size modes are configured without them. Today that distinction does not become observable program behaviour because LLVM stops before emitting the object.

> **NOTE**
> The size modes -Os, -Oz, and -Ozz produce runnable artifacts. They combine LLVM optimization with optsize/minsize attributes and target-specific linker size policies. Runtime checks are disabled in these modes; use -O0 or -O1 for checked builds. -Ozz also strips symbols and metadata.

What the checks cover

- Signed and unsigned integer overflow on `+`, `-`, and `*`.
- Array bounds on any index into something with a known element count. A many-pointer `[*]T` has no count and is never checked.
- Union tag mismatch, on the unchecked payload reads that `#enable(allow_union_unwrap)` permits.
- A null address when an integer or an enum is converted to `*T`, `[*]T`, or a `#fn_ptr` type.

A failing check writes a category and terminates immediately. Arithmetic and slice failures have labels such as `arithmetic.overflow`; array bounds and union-tag failures also name their source location. Windows and Linux x86-64 use status 134, WASI uses 125, and the remaining targets use their platform `abort()` path. Runtime Safety Checks and Panics covers the complete family.

GLOSSO

```
#import "Io";

main :: () {
    a: s32 = 2147483647;
    b: s32 = 1;
    print("sum = %\n", a + b);
    print("survived\n");
}
```

CONSOLE / OUTPUT

```
-O0    runtime check failed: arithmetic.overflow   status 134 on Windows
-O1    runtime check failed: arithmetic.overflow   status 134 on Windows
-O2    sum = -2147483648 / survived      status 0
-O3    sum = -2147483648 / survived      status 0
```

The wrap-around result at `-O2` is not undefined behaviour; it is the defined two's-complement result of the same instruction, with the trap removed. The bounds check behaves the same way: the read that aborts at `-O1` returns whatever the address held at `-O2`.

GLOSSO

```
#import "Io";

element :: (values: []mut s32, index: ssize) -> s32 {
    return values[index];
}

main :: () {
    storage: [3]s32;
    storage[0] = 10; storage[1] = 20; storage[2] = 30;
    view := storage.view();
    print("in range: %\n", element(view, 1));
    print("out of range: %\n", element(view, 7));
    print("survived\n");
}
```

CONSOLE / OUTPUT

```
$ glosso -O1 -o bounds.exe bounds.glo && ./bounds.exe
in range: 20
<path>:<line>:<column>: runtime check failed: array.bounds
$ echo $?
134

$ glosso -O2 -o bounds.exe bounds.glo && ./bounds.exe
in range: 20
out of range: 0
survived
```

GLOSSO

```
#import "Io";
#enable(allow_union_unwrap);

main :: () {
    r: string!s32 = .Err("nope");
    print("value = %\n", r.Ok);
}
```

CONSOLE / OUTPUT

```
$ glosso -O1 -o unionfail.exe unionfail.glo && ./unionfail.exe
...\unionfail.glo:6:28: union access: expected .Ok, but the value holds one of .Ok | .Err
$ echo $?
134

$ glosso -O2 -o unionfail.exe unionfail.glo && ./unionfail.exe
value = 4
```

`4` is the length of `"nope"`: the `.Err` payload is a string descriptor, and reading it as an `s32` picks up the low half of its count field. That is the cost of the missing check stated plainly. At `-O1` the program stops at the line that is wrong; at `-O2` it continues with a value that means nothing.

`GLOSSO_DEBUG_ALLOCATOR`

Optimization no longer selects the default allocator. With `GLOSSO_DEBUG_ALLOCATOR` absent or set to `0`, every optimization level starts with the plain heap allocator. Set it to any other value to install the debug allocator, which records each allocation with its source location and prints a leak report when the process ends. The allocator itself is described in The Debug Allocator.

GLOSSO

```
#import "Io";
#import "Context";

main :: () {
    p := alloc(64);
    print("allocated\n");
}
```

CONSOLE / OUTPUT

```
$ GLOSSO_DEBUG_ALLOCATOR=1 glosso -O1 -o leak.exe leak.glo && ./leak.exe
allocated
glosso debug allocator: leaked 1 allocation(s), 64 byte(s)
  leak: 64 byte(s)
    allocated at:
      ...\leak.glo:5:10

$ glosso -O0 -o leak.exe leak.glo && ./leak.exe
allocated
```

NOTE
The environment choice is part of the native artifact-cache identity, so a cached heap build is never restored for a debug-allocator build or the reverse. The switch is independent of the runtime checks: `-O0` and `-O1` still control those exactly as the table above says.

Size optimization

The size modes -Os, -Oz, and -Ozz produce runnable artifacts. They combine LLVM optimization with optsize/minsize attributes and target-specific linker size policies. Runtime checks are disabled in these modes; use -O0 or -O1 for checked builds. -Ozz also strips symbols and metadata.

Size modes apply target-specific linker policies: dead-section removal, identical-code folding where supported, and symbol stripping at -Ozz. These modes produce artifacts with the current installed toolchain.

NOTE
Nested builds select the same size modes through Optimization.Optimized_Small, Optimized_Very_Small, and Optimized_Tiny. Native_Compile_Output.compile_c_archive accepts the same Optimization enum.

- A bare-metal target cannot use `-O0` or `-O1` without supplying an `abort`, because that is what a failing runtime check calls. See Freestanding, Bare Metal, UEFI, and WebAssembly.

1.4 The Compilation Pipeline

A build is one compiler process running a fixed sequence of phases over the whole program. Each phase consumes what the phase before it produced, so every failure is reported in the vocabulary of the phase that found it: the lexer talks about characters, the parser about tokens, the checker about types, the linker about symbols.

The order is not configurable and no option skips a phase. `--backend`, `--target` and the `-O` level change what the last two phases do; everything up to Glosso IR is the same work for every build.

Phase	Timing label	Produces
Lexing	`prepare`	One token stream per source file.
Operator prescan	`prepare`	The spellings, fixities and precedences the parser will use.
Parsing	`prepare`	Declaration nodes, each stamped with the feature gates active where it was written.
Resolution	`prepare`	One flat declaration list: built-ins, the Prelude, imports, loads, root file.
Checking	`check`	Types, instance selection, managed-value moves and cleanup, monomorphized bodies, constant-evaluated `#comptime`.
Lowering	`lower`	Glosso IR, one function per reachable declaration.
Compile-time execution	`consume`	The side effects of top-level `#comptime`, run in the bytecode VM.
Backend emission	`consume`	The LLVM module, and whatever the backend links out of it.

Setting the environment variable `GLOSSO_TIMINGS` to any value prints one line per phase to stderr. It is the cheapest way to see where a build spends its time, and where compile-time output falls relative to the rest of the work.

GLOSSO

```
#import "Io";

announce :: (tag: string) -> ssize {
    print("[%] evaluated during checking\n", tag);
    return 1;
}

CONSTANT :: #comptime announce("const");

#comptime { print("[vm] evaluated after lowering\n"); }

main :: () { print("runtime %\n", CONSTANT); }
```

CONSOLE / OUTPUT

```
glosso: compile engines.glo (target=x86_64-pc-windows-msvc)
glosso: timing prepare 237.568 ms
[const] evaluated during checking
[const] evaluated during checking
glosso: timing check 570.978 ms
glosso: timing lower 304.605 ms
[vm] evaluated after lowering
glosso: link engines.exe (target=x86_64-pc-windows-msvc, -O1)
glosso: done engines.exe  (backend=llvm, target=x86_64-pc-windows-msvc, -O1)
glosso: timing consume 306.244 ms
runtime 1
```

The transcript places the two compile-time engines. The `[const]` lines come from the constant evaluator, which folds `CONSTANT` while the checker is running, and which visits the initializer more than once. The `[vm]` line comes from the bytecode VM, which runs top-level `#comptime` exactly once after lowering and before the backend emits. What Runs When treats the difference in full.

Lexing

The lexer turns a file's bytes into tokens. It has no recovery: the first lexical error ends the build, and no diagnostic from any later phase for that file is printed beside it. The file below contains an illegal declaration name on line 1, which is a parse error, and an unterminated literal on line 2, which is a lex error. Only the second is reported.

GLOSSO

```
1bad :: 3;
main :: () { s := "unterminated
}
```

CONSOLE / OUTPUT

```
error: unterminated string literal
  --> …\scan.glo:2:19
  |
2 | main :: () { s := "unterminated
  |                   ^^^^^^^^^^^^^
```

The operator prescan

Every operator in Glosso is a typeclass method carrying an `#operator` directive, so a token like `<^>` means nothing until such a declaration has been read. Parsing cannot discover the fixity as it goes, because the declaration may come after the use. Before parsing starts, the prescan records the `#operator` modes and levels in the source and its relative `#load` and `#export` dependency graph, starting with the Prelude's operator table.

GLOSSO

```
#import "Io";

main :: () {
    print("2 <^> 3 <^> 2 = %\n", 2 <^> 3 <^> 2);
}

Power :: typeclass (T: type) {
    '<^>' :: (a: T, b: T) -> T #operator(right, 17);
}

ssize ::: instance Power {
    '<^>' :: (a: ssize, b: ssize) -> ssize #operator(right, 17) {
        r := 1;
        i := 0;
        while i < b { r *= a; i += 1; }
        return r;
    }
}
```

CONSOLE / OUTPUT
```
2 <^> 3 <^> 2 = 512
```

The declaration sits below its own use and the expression still groups to the right: `2 <^> (3 <^> 2)` is 512. Changing both directives to `#operator(left, 17)` prints 64 instead. An operator declared in a relative `#load`ed dependency can also be used by its loader: the dependency graph is scanned before the loader's expressions are parsed. This does not make private declarations public or give one sibling module another sibling's dependencies. The Operator Prescan gives the complete rules.

Parsing and feature gates

The parser recovers. It records a diagnostic, skips to the next plausible declaration boundary and continues, so one run reports every top-level form it could not read and finishes with a count.

GLOSSO
```
1bad :: 3;
Foo : struct { x: ssize; }
main :: () {}
```

CONSOLE / OUTPUT
```
error: bad decl name '1'
  --> …\decls.glo:1:1
   |
1 | 1bad :: 3;
   | ^

error: a top-level constant binding uses '::'; the second ':' is missing
  --> …\decls.glo:2:5
   |
2 | Foo : struct { x: ssize; }
   |     ^

error: aborting due to 2 previous errors
```

Feature gates are resolved here rather than by the checker. `#enable` and `#disable` are sticky from the directive to the end of the file, and the parser stamps the gates that were active onto each declaration it builds; the checker reads the stamp back when it checks that declaration. That is why wrapping one procedure in `#enable(lossy_numerics)` relaxes conversions in that procedure alone. An unrecognised name is rejected immediately with `error: unknown feature 'bogus_feature'`. Feature Gates: `#enable` and `#disable` lists the three supported gates.

Resolution

Resolution assembles the program: it installs the compiler-owned declarations, loads `std/Prelude`, expands `#load`, resolves `#import`, applies declaration visibility, and appends the root file, producing a single declaration list with no ordering rule inside it. A module that cannot be found is reported here, as is a `#load` path that does not exist. Resolution failures do not stop the checker: both sets of diagnostics are printed in one batch sorted by line and column, so a file with one bad import and one bad conversion reports both and ends with `error: aborting due to 2 previous errors`. Source Files, Imports, and Loads covers what resolution accepts.

Checking and monomorphization

Checking gives every expression a type, selects a typeclass instance for every operator and method call, and checks each generic body once per instantiation. A generic declaration on its own proves nothing: the body is checked against the types it is actually called with, which is why the diagnostic points into the generic body while the reason for it is at the call site.

GLOSSO
```
#import "Io";

Pair :: struct { a: ssize; b: ssize; }

twice :: (x: $T) -> T { return x + x; }

main :: () {
    p: Pair;
    print("%\n", twice(p).a);
}
```

CONSOLE / OUTPUT
```
error: no operator '+' for Pair, Pair
  --> …\twice.glo:5:34
   |
5 | twice :: (x: $T) -> T { return x + x; }
   |                                  ^
```

A `where` clause moves the check back to the call and names the callee instead: `error: constraint failed for double`. Constraints with `where` explains when to prefer one over the other.

Compile-time execution

Compile-time code runs in two places. Values, which means `X :: #comptime f()`, `#comptime` parameters, `#if`, `#assert` and `#insert`, are produced by the constant evaluator during checking. A top-level `#comptime` block or call is lowered to an IR function and executed by the bytecode VM after lowering, once each and in declaration order, whichever backend was selected.

A VM failure ends the build with the phase named in the message, for example `error: #comptime stopped: VM: unknown #magic intrinsic 'ctx_alloc'`. Calling `exit(n)` from compile time is the one path that hands an arbitrary status to the shell: the compiler stops after this phase, writes no artifact, and exits with `n`.

GLOSSO

```
make    :: () -> *mut u8 #foreign "c" "make" ;
release :: (p: *mut u8)  #foreign "c" "release" ;

main :: () {
    p := make();
    release(p);
    _ := p.*;
}
```

Lowering, emission, and linking

Lowering turns checked declarations into Glosso IR, and rejects the constructs that survive checking but have no runtime form, such as `error: recursive call to lazy function 'rec' is not supported`. Backend emission then translates IR into an LLVM module; an unregistered intrinsic fails here with `error: native LLVM lowering has no classification for #magic 'no_such_intrinsic'`. The linker is last and speaks only about symbols, which is why a file with no `main` compiles without complaint and then fails.

CONSOLE / OUTPUT

```
glosso: compile helper.glo (target=x86_64-pc-windows-msvc)
glosso: link helper.exe (target=x86_64-pc-windows-msvc, -O1)
error: lld-link: error: undefined symbol: wmain
>>> referenced by exe_common.inl:90
>>>               msvcrt.lib(exe_wmain.obj):(int __cdecl invoke_main(void))
```

Reading a diagnostic by its phase

Matching a message to the phase that produced it says what the compiler already knew when it complained, and therefore what can and cannot be the cause. Every message below is real output from this compiler.

Message	Phase
`unterminated string literal`	Lexing
`unexpected token '<^>'`	Parsing, for an operator the prescan never saw
`unknown feature 'bogus_feature'`	Parsing
`cannot find module "X" (searched: <roots>)`	Resolution
`no operator '+' for Pair, Pair`	Checking, at one instantiation
`recursive call to lazy function 'rec' is not supported`	Lowering
`#comptime stopped: VM: unknown #magic intrinsic 'ctx_alloc'`	Compile-time execution
`lld-link: error: undefined symbol: wmain`	Linking

1.5 Source Files, Imports, and Loads

A Glosso source file is a flat sequence of top-level declarations with no ordering rule. `main` may call a procedure declared below it, a type may be used above its declaration, and `#import` and `#load` may sit anywhere in the file, including on the last line.

GLOSSO

```
main :: () {
    p: Point = .{ .x = 3, .y = 4 };
    print("manhattan = %\n", manhattan(p));
}
```

```
#load "parts/geometry.glo";
#import "Io";
```

CONSOLE / OUTPUT

```
manhattan = 7
```

The resolver assembles declarations before ordinary name checking, so forward references in one module work. Each dependency still has its own imports and visibility. Module-level `#if` selects declarations using compile-time conditions; those conditions need already available imports. Top-level `#comptime` steps retain their declaration order.

`#import` and `#load` are declarations, not statements. Written inside a procedure body, `#import "Io";` is `error: unexpected directive 'import'`.

#import

`#import` names a module in the standard-library search path and brings its public declarations into the importing file. Five forms are accepted. `only` and `hide` are the only modifiers; anything else is `error: #import modifier must be only or hide`.

Form	Effect
`#import "Io";`	Public names enter the importing file's scope unqualified.
`M :: #import "Math/Constants";`	Names are reachable through the namespace, such as `M.S32_MAX`.
`#import,only(S32_MAX) "Math/Constants";`	Exactly the listed public names enter scope.
`#import,hide(S32_MAX) "Math/Constants";`	Everything except the listed names enters scope.
`M :: #import,only(S32_MAX) "Math/Constants";`	A namespace holding a selection.

GLOSSO

```
#import "Io";
M :: #import "Math/Constants";
#import,only(S32_MAX) "Math/Constants";

main :: () {
    print("qualified=% selected=%\n", M.S32_MAX == S32_MAX, S32_MAX > 0);
}
```

CONSOLE / OUTPUT

```
qualified=1 selected=1
```

A name in an `only` or `hide` list must be a public module-level declaration. A typeclass method is not one, so `#import,only(sqrt) "Math/Core";` fails with `error: #import,only names unknown public declaration 'sqrt' in module "Math/Core"` followed by a `note: public names:` list of what is selectable. Repeating a name is `error: duplicate name 'print' in #import,only`. An empty list, `#import,only() "Io";`, is legal and imports nothing at all.

`hide` removes the selected declaration from this import's unqualified surface. Repeating an import is harmless. Visibility controls selection, not a name's spelling: an explicitly public `_helper` can be named in `only` and `hide`, while a private declaration cannot.

> **NOTE**
> Use the explicit `Math/Core` module name for scalar mathematics and `Math/Constants` for constants. A `Math` compatibility shorthand exists only in full-standard-library conformance mode; ordinary source imports should name the component directly.

How a module name resolves

A module name is a path with `/` separators. It is validated first: `..`, backslashes, absolute paths and empty components are rejected by `error: invalid standard module name`, which then quotes the spelling it refused. The resolver looks for two things in the standard-library directory.

On disk	Result
`std/Name.glo`	That one file is the module.
`std/Name/module.glo` exists	That entry file is the module; its `#export` directives select its public components.
both exist	`error: ambiguous module "Name": both <file> and <dir>/module.glo exist`
neither exists	`error: cannot find module "Name" (searched: <roots>)`, where the roots are the standard-library directory followed by every `--module-path` directory

Only `module.glo` is loaded automatically. It normally contains `#export "Component.glo";` directives that assemble an umbrella module. A sibling file is included only through an explicit dependency. Nested modules can be named by path or reexported by their parent.

Both shapes coexist in the standard library, and a name may have a file and a same-named directory without being ambiguous when the directory has no `module.glo`. `std/Io.glo` and `std/Io/` are that shape, so `#import "Io"` resolves to the file. `std/File_System/` now has neither a sibling `File_System.glo` nor a marker: it is a namespace only, and each component is imported by its full path.

GLOSSO

```
#import "Io";
#import "File_System/Metadata";

main :: () {
    if #pattern .Ok(found) = exists("lookup.glo") {
        print("exists = %\n", found);
    }
}
```

CONSOLE / OUTPUT

```
exists = 1
```

Namespace-only directories

A directory without `module.glo` is a namespace, not a module. `#import "Container";` is `error: cannot find module "Container" (searched: <std>)`, which lists the standard-library root and every `--module-path` directory. Its contents are imported one component at a time, by path. The six are `Container`, `Crypto`, `File_System`, `Math`, `Net`, and `Serialization`. `Strings` has an umbrella entry point exporting `Memory`, `Utf8`, and `Utf16`.

GLOSSO

```
#import "Io";
#import "Strings/Utf8";

main :: () {
    print("%\n", ("compiler").substring(0, 4));
}
```

CONSOLE / OUTPUT

```
comp
```

The same path form also names one file of a real module, so `#import "Io/Core";` imports a single component of the `Io` directory. Each component declares its own dependencies. Umbrella modules expose the components their `#export` directives select; importing a component directly exposes that component's public declarations and reexports.

`#load`

`#load` loads an isolated module by file path. The path is relative to the file containing the directive, not the working directory, and may contain `..`. Public declarations become available in the loading file; an alias makes them available through a namespace. The loaded file resolves names using its own explicit dependencies.

GLOSSO

```
#import "Io";
G :: #load "parts/geometry.glo";

main :: () {
    p: G.Point = .{ .x = 3, .y = -4 };
    print("manhattan = %\n", G.manhattan(p));
}
```

CONSOLE / OUTPUT

```
manhattan = 1
```

- Each distinct path is loaded once; a repeated `#load` of the same path is a no-op rather than a redeclaration error.
- With an alias the names are reachable only through it: writing `Point` alone gives `error: unknown type 'Point'`.
- A missing file is `error: #load: cannot find file "parts/nope.glo" (looked for <absolute path>)`, and the absolute path the compiler tried is printed in full.
- Dependencies stay local: loading a helper that imports `Io` does not make `print` available to the caller. The helper also cannot use a dependency loaded only by its caller or by another sibling module.
- Declarations are private by default. A loaded file must mark its API `#public`, or use `#public(unit)` for callers in the same ownership unit. Visibility and Ownership Units explains both audiences.

> **NOTE**
> The operator prescan follows relative `#load` and `#export` dependencies, including embedded loads, before parsing expressions. The loader can therefore use an operator declared by its loaded module. Ordinary module visibility still governs whether the owning typeclass and implementation can be used; a spelling in the parser's table is not an implicit import.

`#export`: umbrella modules

`#export "path";` loads a relative module and exposes its public declarations to this module's users. If `A.glo` exports `B.glo`, a file that loads `A.glo` can use `B.glo` too. Reexports are transitive; ordinary loads and imports inside a dependency remain local.

```
GLOSSO
// shapes/module.glo
#export "point.glo";
#export "rectangle.glo";

// app.glo
#load "shapes/module.glo";
// Public declarations from both components are available here.
```

`#load,embed`: shared source fragments

`#load,embed "path";` places a source fragment in its containing logical module and shares the module's dependencies. Each fragment retains its physical source identity, so cross-file declarations still need `#public(unit)` or `#public`. This manual uses unit-visible chapter and UI fragments. Nested paths remain relative to the directive's physical file.

```
GLOSSO
#import "Io";
#load,embed "parts/driver.glo";
// The embedded driver can use this file's Io dependency.
```

1.6 The Implicit Prelude

Every compilation begins with names already in scope. Nothing in the program declares them and no import produces them. They arrive in three layers, in this order: type spellings the parser recognises, declarations the compiler injects, and the thirteen public components of `std/Prelude`.

The Prelude is loaded unqualified for every program, before any `#import` in the program is resolved. It is the only module treated this way, and it cannot be turned off.

Built-in type names

A fixed set of names, the `Simd` and `Matrix` type constructors, and two declaration-annotation forms are recognised by the compiler itself. They are not library declarations and need no import.

Group	Spellings
Sized integers	`s8`, `s16`, `s32`, `s64`, `s128`, `u8`, `u16`, `u32`, `u64`, `u128`
Native-size integers	`ssize`, `usize`
Floating point	`f16`, `f32`, `f64`, `f80`, `f128`
Complex	`c32`, `c64`, `c128`, `c160`, `c256`
Characters and text	`rune`, `string`, `cstring`, `string16`, `cstring16`
Other	`bool`, `void`, `#Never`, `any`, `type`, `label`, `Code`, `Pattern`, `Namespace`, `Library`
Type constructors	`Simd(T, N)`, `Matrix(T, Rows, Cols)`
Declaration annotations	`Typeclass(T, …)`, `Macro(signature)`

`ssize` and `usize` are signed and unsigned pointer-width integers: 32 bits on a 32-bit output target and 64 bits on a 64-bit one. An integer literal with no other context defaults to `ssize`; collection counts and indices use it too. The checker additionally accepts `sN` and `uN` for every unpadded decimal width `N` from 1 through 65535. `rune` is a Unicode scalar in four bytes, distinct from integers. An ASCII `#char` literal defaults to `u8`, while a non-ASCII literal defaults to `rune`; there is no built-in `char` type. Primitive Types covers the full primitive set and its representations. `Typeclass(...)` and `Macro(...)` describe explicitly typed declarations; they are not general-purpose type-value constructors.

Declarations the compiler injects

Before the Prelude is read the compiler installs two source fragments of its own, under the pseudo-paths `<builtin:Types>` and `<builtin:Context>`. The first declares the two unions that `?T` and `E!T` resolve to.

GLOSSO

```
Option :: union #magic "option" {
    None: void;
    Some: $T;
}

Result :: union #magic "result" {
    Ok: $T;
    Err: $E;
}
```

Those names are internal and the parser rejects them wherever a program could write one: `Option(ssize)` is `error: the built-in option type is written '?T'; its variants are '.Some(x)' and '.None'`, and `Result(string, ssize)` is `error: the built-in result type is written 'E!T'; its variants are '.Ok(x)' and '.Err(e)'`. They are built in rather than declared in `std` so that a program cannot reshape the type that `?`, `#try` and pattern lowering resolve to. `std/Prelude/Option.glo` and `std/Prelude/Result.glo` supply everything else about them.

The second fragment declares the implicit context's type. Its spelling carries the `#`: a parameter typed `Context` is `error: the built-in Context type is written #Context`.

GLOSSO

```
Context :: struct {
    allocator:        Allocator;
    temp_allocator:   Allocator;
    logger:           Logger;
    random_generator: Random_Generator;
    user_ptr:         ?*mut void;
    user_index:       ssize;
}
```

GLOSSO

```
#import "Io";
depth :: (c: #Context) -> ssize { return c.user_index; }
main :: () { print("user_index = %\n", depth(context)); }
```

CONSOLE / OUTPUT

```
user_index = 0
```

The struct is always in scope but its field types are gated. Naming `Allocator`, `Logger` or `Random_Generator` in a program that has not imported `Context` gives `error: 'Allocator' is a #Context field type`, and the rest of the message asks for `#import "Context";`. The Implicit Context and Allocators cover what the fields do.

The thirteen Prelude components

`std/Prelude/module.glo` explicitly assembles the Prelude. Everything its public components declare is in scope for every program.

File	Supplies
`Show_and_Read.glo`	`Show`, the protocol behind `%` and `print`; `FromString`; the `derive_show` macro.
`Formatters.glo`	The default, integer-radix, and character `Formatter` instances. The `Formatter` protocol itself lives in `Format`.
`Primitive_Typeclasses.glo`	`AsView`, `PartialEq`, `Eq`, `PartialOrd`, `Ordering`, `Num`, `Iterable`, and `PointerIterable`.
`Primitive_Operators.glo`	The operator typeclasses and their primitive instances, plus `Indexable`, `Step`, the range classes, and `ToPattern`.
`Option.glo`	`?T` constructors and methods: `make_some`, `make_none`, `is_some`, `is_none`, `unwrap_or`, `map`, `and_then`, and `Option_Residual`.
`Result.glo`	`E!T` constructors and methods: `make_ok`, `make_err`, `is_ok`, `is_err`, `unwrap_or`, `map`, `E1!T.map_error`, `E1!T.bind_error`, `((E1) -> E2)!T.ap_error`, and `Result_Residual`.
`Try.glo`	`Try`, `Try_Branch`, and `Propagate`, the protocols behind postfix `?` and residual conversion.
`Termination.glo`	`Termination(T)`, which converts a value returned by `main` into a process status.
`Array.glo`	Array methods `push`, `append`, `reserve`, `reset`, `free`, `deinit`, and `view`.

File	Supplies
`Assembly.glo`	`Asm_Constraint`, `Asm_Operand_Flags`, `Asm_Clobber`, the vocabulary `#asm` operands are written in.
`Source_Location.glo`	`Source_Location`, with `filename: string`, `line: ssize`, and `column: ssize`; its `Show` instance renders `filename:line:column`.
`Utilities.glo`	`Target_OS`, `Target_CPU`, `Target_ABI`, `Target_Endian`, `Output_Target`, `Host_Target`, `target_endian`, `get_cmdline_args`, `exit`.

Prelude components import dependencies such as `Meta`, `Typeclass`, `String_Builder` and `Format`. Those imports serve the Prelude and do not reach the program: a name that arrives only that way is `error: unknown function tprint` with `note: 'tprint' belongs to a transitive dependency; import its module directly`. A program that wants those names writes the imports itself.

GLOSSO

```
#import "Io";
#import "Meta";
#import "String_Builder";
#import "Format";
main :: () {
    print("type_name = %\n", type_name(s32));
    print("size_of   = %\n", size_of(f64));
    print("joined    = %\n", "a" + "b");
    print("format    = %\n", (255).format_int(16));
    print("tprint    = %\n", tprint("% + %", 1, 2));
}
```

CONSOLE / OUTPUT

```
type_name = s32
size_of   = 8
joined    = ab
format    = ff
tprint    = 1 + 2
```

`OUT` and `HOST` come from no file: the checker installs them as constants once `Output_Target` and `Host_Target` exist, which is why `Utilities.glo` declares the shapes but not the values. Targets and Cross-Compilation covers what they hold.

> **NOTE**
> `#source_location` needs no import either: it is a compiler expression that constructs the Prelude's `Source_Location` at the directive's lexical position. Pass it explicitly to a diagnostic helper when you want the caller's filename, line, and column. The chapter `#assert` and `#compile_error` explains its ownership and diagnostic formatting.

GLOSSO

```
#import "Io";

main :: () {
    maybe: ?ssize = .Some(21);
    outcome: string!ssize = .Ok(42);
    print("% %\n", maybe.is_some(), outcome.is_ok());

    xs: [..]ssize;
    xs.push(3);
    xs.push(4);
    print("count=% sum=%\n", xs.count, xs[0] + xs[1]);
    xs.free();

    print("ordering=% show=%\n", 1 <=> 2, show(7));
    print("os=% endian=% args=%\n", OUT.os, target_endian(), get_cmdline_args().count);
}
```

CONSOLE / OUTPUT

```
1 1
count=2 sum=7
ordering=0 show=7
os=3 endian=0 args=1
```

The operator typeclasses

Every operator in the language is a method of a typeclass declared in `Primitive_Operators.glo` or `Primitive_Typeclasses.glo`, with its fixity and precedence written on the requirement. That is what the operator prescan reads out of the Prelude, and it is why `a + b` needs no import while a library's own operator does.

Class	Operator	Fixity and level
`Add`, `Sub`	`+`, `-`	left, 17
`Mul`, `Div`, `Rem`	`*`, `/`, `%`	left, 18
`Neg`, `BitNot`, `LogicalNot`	`-`, `~`, `!`	prefix
`ShiftLeft`, `ShiftRight`	`<<`, `>>`	left, 16
`BitAnd`	`&`	left, 14
`BitXor`	`^`	left, 13
`BitOr`	`|`	left, 12
`PartialOrd`	`<=>`, `<`, `<=`, `>`, `>=`	left, 10
`PartialEq`	`==`, `!=`	left, 8
`LogicalAnd`	`&&`	left, 6
`LogicalOr`	`||`	left, 4
`RangeExclusiveOp`, `RangeInclusiveOp`	`..`, `..=`	left, 2
`AddAssign` and the rest of the assign family	`+=`, `-=`, `*=`, `/=`, `%=`	assign, 0

The wrapping and saturating families (`WrapAdd`, `SaturatingMul` and their assigning forms) follow the same shape, and `Indexable` and `IndexSet` own `[]` and `[]=` without a precedence. The Operator System and Operator Precedence and the 0-20 Scale give the complete table.

`LogicalAnd` and `LogicalOr` declare their right operand `#lazy`, which is the whole mechanism behind short-circuiting `&&` and `||`; Lazy Parameters and Short-Circuit Operators explains it.

The command line and exit

`Utilities.glo` also supplies the two process-level procedures every program can reach without an import.

GLOSSO

```
get_cmdline_args :: () -> []string #magic "get_cmdline_args"
exit :: (status: ssize) -> #Never #magic "exit";
```

`get_cmdline_args` returns the process argument vector at run time, including the program path at index 0. During compile-time execution it returns the arguments written after `--` on the compiler's own command line, without a leading program path. `exit` ends the process, and called from a top-level `#comptime` it ends the build with that status and writes no artifact.

What is not in the Prelude

`print` is not. It belongs to `Io`, so the smallest program that writes anything needs one import. The help text points at `tprint`, which is no more in scope: it belongs to `Format`, which the Prelude imports for itself, and naming it takes `#import "Format";`.

GLOSSO

```
main :: () { print("hello from glosso\n"); }
```

CONSOLE / OUTPUT

```
error: unknown function print
  --> …\hello.glo:1:14
   |
1 | main :: () { print("hello from glosso\n"); }
   |              ^^^^^
help: did you mean 'tprint'?
```

> **NOTE**
> Numeric constants are not supplied by the Prelude. `PI`, `S64_MAX`, `F64_MAX`, and the other numeric limits belong to `Math/Constants`. Import that component directly; a dependency's private import does not bring its names into the caller's scope.

`Drop.glo`	`Drop(T)` and its associated `Error`, the protocol for automatic resource cleanup.

2 Lexical structure

2.1 Source Text, Whitespace, and Comments

A source file is a sequence of bytes that the compiler reads as UTF-8 text, and nothing else is accepted. A root file whose bytes are not valid UTF-8 is rejected before lexing begins, with `glosso: cannot read` followed by the file name; a file reached through `#load` fails with `could not read`, the path, and `stream did not contain valid UTF-8`. An encoding mistake is therefore always a read failure, never a stray token halfway down the file.

Past that point the scanner works on decoded code points rather than bytes. The line, column and length recorded on every token are counted in code points from the start of the line, which is what keeps a caret span the same width as the token it underlines when the line holds non-Latin text.

```
GLOSSO
main :: () {
    안녕 := 1;
    λμν := 안녕 + oops;
}
```

That program fails with `error: unbound 'oops'`, reported at `3:17`. Column 17 counts the sixteen code points before `oops`; a byte count would have said 24, because `λ`, `μ` and `ν` occupy two bytes each and `안` and `녕` occupy three.

Whitespace

Exactly four code points are skipped as whitespace. The scanner tests for them before anything else, and only for them.

Code point	How the scanner treats it
U+000A LF	Ends the line and advances the line counter.
U+000D CR	Skipped, which is why a CRLF file lexes exactly like an LF file.
U+0020 SP	Skipped.
U+0009 HT	Skipped, and counted as a single column.

Nothing else is whitespace. Any other code point falls through to token dispatch, and one that starts no token becomes a stray-character error whose entire message is the offending character in single quotes. Three invisible characters reach that path often enough to be worth naming.

Character	Diagnostic	Usual source
U+FEFF	`error: ''`, with the byte-order mark itself between the quotes, so nothing is visible	Saving as UTF-8 with BOM
U+000C	`error: ''`, likewise empty-looking	A form feed used as a page break
U+00A0	`error: ' '`, which looks exactly like a space	Pasting from a web page

> **NOTE**
> A byte-order mark sits at 1:1, so a BOM-prefixed file does not compile at all: the first thing the scanner sees is a character it cannot use. An editor that offers both `UTF-8 with BOM` and `UTF-8` produces a working file only with the second.

Because a carriage return is whitespace, line endings do not matter to ordinary tokens. Marked multi-line-string lines follow the same rule: their scanner removes a trailing carriage return before storing the line, so LF and CRLF source files produce the same literal text.

Line comments

`//` begins a comment that runs to the next newline, or to the end of input if there is none. Everything after it on that line is invisible to the scanner, including quotes, `/*` and `*/`.

```
GLOSSO
#import "Io";

main :: () {
    print("%\n", 10 // 2 is never divided
```

```
    );
}
```

CONSOLE / OUTPUT

```
10
```

Comments are recognised before token dispatch, so a `/` immediately followed by `/` or by `*` can never begin an operator. A library may declare an operator named with those characters, but no use site can ever spell it; see Reserved Operators and Operator Names.

Block comments

`/*` opens a comment that `*/` closes, and the pair nests. The scanner keeps a depth counter rather than searching for the first `*/`, so a region that already contains comments can be commented out as a whole.

GLOSSO

```
#import "Io";

/* outer
   /* inner nested comment */
   still inside the outer comment
*/

main :: () {
    x := 1 /* inline */ + /* /* nested */ */ 2;
    print("x = %\n", x);
    print("[%]\n", "http://example.com /* not a comment */");
}
```

CONSOLE / OUTPUT

```
x = 3
[http://example.com /* not a comment */]
```

Two further facts are visible there. A newline inside a block comment still advances the line counter, so diagnostics after a large commented-out region report the right line. An unterminated `/*` is a lexical error instead: the scanner reports `error: unterminated block comment` at the opening delimiter rather than emitting end-of-input.

> **NOTE**
> The depth counter reacts to `/*` and `*/` everywhere inside the comment, including inside text that looks like a string literal. A comment is scanned as characters, not as tokens, so nothing protects a `*/` written between quotes.

GLOSSO

```
main :: () {
    /* the pattern "*/" ends this comment early */
    x := 1;
}
```

The comment ends at the `*/` inside the quotes. What remains of the line opens a string literal that no quote closes, and the file fails with `error: unterminated string literal`, underlining from that quote to the end of the line.

- A `/*` written inside a `//` comment opens nothing; the line comment already owns the rest of the line.
- A `//` written inside a block comment does not close it, and neither does a newline.
- A `/*` or `*/` written inside a string literal is ordinary text: the scanner looks for comments only where a token could start.

Reading a source position

`#source_location` is an expression whose value has the Prelude type `Source_Location`. It records the position of its own `#` token, with one-based `line` and `column` fields. Columns count Unicode scalars, not UTF-8 bytes, display cells, or tab stops: a tab is one column and a combining mark is another scalar. Comments and whitespace before the directive still contribute to its position.

GLOSSO

```
#import "Io";
main :: () {
    text := "한"; location := #source_location;
    print("% %:%\n", text, location.line, location.column);
}
```

CONSOLE / OUTPUT
```
한 3:30
```

The `filename` field is the compiler's source path, absolute for loaded source files, or `<unknown>` when no filename is available. On Windows the compiler removes extended-length path prefixes for editor-friendly diagnostics; a path's backslashes remain literal characters, not string escapes. The chapter `#assert` and `#compile_error` shows how to print and pass these values without losing their origin.

2.2 Identifiers and Keywords

An identifier begins with _ or an alphabetic code point, and continues with _ or an alphanumeric one. There is no length limit and no restriction to ASCII.

The two classes come from Unicode tables generated into the compiler and built to match CPython's single-character `str.isalpha` and `str.isalnum`. Carrying the tables rather than borrowing the host language's own predicates means the accepted identifier set does not shift when the toolchain that built the compiler updates its Unicode version.

Position	Class	Also accepted
First character	Alphabetic, `isalpha`	_
Later characters	Alphanumeric, `isalnum`	_

`isalnum` is the union of the alphabetic, decimal, digit and numeric classes, so it is strictly wider than `isalpha`. Some code points can therefore continue an identifier without being able to start one.

GLOSSO
```
#import "Io";

main :: () {
    λ := 1;
    안녕 := 2;
    _x := 3;
    Ω_1 := 4;
    a² := 5;
    x9 := 6;
    ᵃ := 7;
    Δθ := 8;
    print("% % % % % % % %\n", λ, 안녕, _x, Ω_1, a², x9, ᵃ, Δθ);
}
```

CONSOLE / OUTPUT
```
1 2 3 4 5 6 7 8
```

`λ`, `Ω` and `Δθ` are Greek letters and `안녕` is Hangul; all are alphabetic and may start a name. `ᵃ` is a modifier letter, which is alphabetic too. `a²` ends in a superscript two, which is a digit but not a letter, so it may appear only after the first character.

Marks, emoji, and superscripts

A combining mark belongs to neither class. An identifier written in decomposed form therefore ends at its base letter, and the mark that follows becomes a stray character.

GLOSSO
```
main :: () {
    café := 1;
}
```

With `é` written as `e` plus U+0301 that file fails, and the stray-character message quotes the combining mark, which renders on top of whatever precedes it. With `é` written precomposed as U+00E9 the same source compiles: one alphabetic code point, one identifier. No normalisation is performed in either direction, so the two spellings are simply different source text.

Emoji are not alphabetic, and there is no separate rule for them. `🙂 := 1;` fails with `error: '🙂'`, the ordinary stray-character message.

Superscripts and other non-decimal digits satisfy `isalnum` but not `isalpha`. Inside an identifier they are accepted; at the start they do not produce an identifier at all, because the scanner dispatches on the digit class and begins a number. A leading `²` therefore fails with `error: number literals must use ASCII digits`.

> **NOTE**
> There is no diagnostic saying an identifier cannot start with a digit, because no such rule exists at the token level. `2abc := 1;` lexes as the integer `2` followed by the identifier `abc`. The parser reads `2` as the start of an expression statement, then requires its `;` and reports `error: unexpected token 'abc'` immediately.

GLOSSO

```
#import "Io";

main :: () {
    2abc := 1;
    print("abc = %\n", abc);
}
```

CONSOLE / OUTPUT

```
error: unexpected token 'abc'
  --> digit-name.glo:4:6
   |
4 |     2abc := 1;
   |      ^^^
```

The keywords

Nineteen words are reserved. The scanner reads a whole word first and then looks it up, so a word is a keyword only when it is the entire identifier.

Group	Keywords
Conditions and loops	`if`, `ifx`, `else`, `while`, `for`, `case`
Leaving a body	`return`, `break`, `continue`
Aggregates	`struct`, `union`, `enum`
Modifiers	`using`, `defer`, `context`, `noalias`, `where`
Conversions	`cast`, `acast`

Every one of them is rejected as a binding name, though the wording depends on where the parser gives up: `if := 1;` is `error: unexpected token ':='`, `struct := 1;` is `error: unexpected token 'struct'`, and `cast := 1;` is `error: cast requires a type in parentheses; use acast for contextual casts`.

Whole-word matching means a longer word that merely begins or ends with a keyword is an ordinary identifier.

GLOSSO

```
#import "Io";

main :: () {
    iffy := 1;
    structure := 2;
    forward := 3;
    casing := 4;
    _if := 5;
    if_ := 6;
    print("% % % % % %\n", iffy, structure, forward, casing, _if, if_);
}
```

CONSOLE / OUTPUT

```
1 2 3 4 5 6
```

Words that only look reserved

`typeclass`, `instance`, `distinct`, `type`, `ssize` and `string` are plain identifier tokens. Their meaning comes from the position they appear in, which is why each of them can also be bound as a local.

GLOSSO

```
#import "Io";

main :: () {
    typeclass := 1;
    instance := 2;
    distinct := 3;
    type := 4;
    ssize := 5;
    string := 6;
    print("% % % % % %\n", typeclass, instance, distinct, type, ssize, string);
}
```

CONSOLE / OUTPUT
```
1 2 3 4 5 6
```

`distinct` is the clearest case: it is recognised only directly after `::` in type position, so one file can declare a nominal type and also bind a local of that name.

GLOSSO
```
#import "Io";

Handle :: distinct u32;

main :: () {
    distinct := 7;
    h : Handle = cast(Handle) 3;
    print("% %\n", distinct, cast(u32) h);
}
```

CONSOLE / OUTPUT
```
7 3
```

> **NOTE**
> `true`, `false` and `null` are not keyword tokens either, but the parser resolves those spellings to literals wherever an expression is expected. A local named `true` can be declared and then never read: the program below binds `false` to `2` and still prints the boolean, which shows as `0`.

GLOSSO
```
#import "Io";

main :: () {
    true := 1;
    false := 2;
    print("% %\n", true, false);
}
```

CONSOLE / OUTPUT
```
1 0
```

- `_` on its own is a legal identifier and binds like any other name. Its separate role in patterns is described in The Pattern Vocabulary.
- Identifier comparison is exact: neither case folding nor Unicode normalisation is applied to a name before it is looked up.

2.3 Integer Literals

A decimal integer literal is a run of ASCII digits with optional `_` separators. The separators carry no meaning; they are removed before the value is parsed.

GLOSSO
```
#import "Io";

main :: () {
    a := 12_345;
    b := 1_032_000;
    c := 1000000;
    print("% % %\n", a, b, c);
}
```

CONSOLE / OUTPUT
```
12345 1032000 1000000
```

A run of digits may not be empty, may not begin or end with `_`, and may not contain two adjacent separators. That rule is applied independently to four different parts of a number, each with its own message, so the diagnostic names the part that is malformed.

Part	Diagnostic
Integer digits	`invalid '_' separator in number literal`

Part	Diagnostic
Fraction digits	`invalid '_' separator in fractional literal`
Exponent digits	`invalid '_' separator in scientific exponent`
Base digits	`invalid '_' separator in base integer literal`

Because the parts are checked separately, a separator next to a `.` or an `e` is legal on both sides: `1_0.2_5` and `1_000e1_0` are accepted. A leading `_` is not a number error at all, since `_1` starts an identifier; that spelling fails later with `error: unbound '_1'`.

Explicit base

A non-decimal integer is written as a base, a `#`, and digits. The base is a decimal number between 2 and 36, and the digits are `0` to `9` and `a` to `z` read case-insensitively. Both the base and the digits may carry `_` separators.

GLOSSO

```
#import "Io";

main :: () {
    bin := 2#1011;
    oct := 8#777;
    hex := 16#ff;
    hexcase := 16#Ff;
    b36 := 36#Z;
    wide := 16#4865_316F_774F_6C64;
    print("% % % % % %\n", bin, oct, hex, hexcase, b36, wide);
}
```

CONSOLE / OUTPUT

```
11 511 255 255 35 5216630098191412324
```

One spelling covers every radix, which is why there is no separate hexadecimal or binary prefix to remember and why base 36 costs nothing extra. The base is scanned as a number in its own right, so `1_6#ff` is the same literal as `16#ff`.

Written	Diagnostic
`1#1`	`integer literal base must be between 2 and 36, got 1`
`37#z`	`integer literal base must be between 2 and 36, got 37`
`2#102`	`digit '2' is not valid for base 2`
`16#`	`base-16 integer literal needs at least one digit`
`16 #ff`	`unexpected directive 'ff'`

The last row is the one that surprises. The `#` is absorbed by the number scanner only when it immediately follows the digits. With a space in between, `16` is a complete integer literal and `#ff` is an ordinary directive token, which the parser then rejects by name.

A based literal is a bit pattern

A based literal is recorded as a bit pattern rather than as a magnitude, and the checker measures it against the width of the target type instead of against that type's signed range. A mask is written for its bits, so `16#ff` is a legal `s8` and denotes `-1` there.

GLOSSO

```
#import "Io";

main :: () {
    a : s8  = -128;
    b : s8  = 16#80;
    c : s8  = 16#ff;
    d : u8  = 16#ff;
    e : s64 = 16#8000_0000_0000_0000;
    f : u64 = 18_446_744_073_709_551_615;
    g : s16 = 16#ffff;
    print("% % % % % % %\n", a, b, c, d, e, f, g);
}
```

CONSOLE / OUTPUT

```
-128 -128 -1 255 -9223372036854775808 18446744073709551615 -1
```

The relaxation is about width, not about range: a pattern wider than the target is still rejected. `a : s8 = 16#1ff;` gives `error: 511 does not fit s8, whose range is -128..=127` with the note `a based literal is a bit pattern, so it may use every bit of s8 - this one is wider than that`. A decimal `255` at `s8` is rejected as well, since it never had the exemption.

A leading `-` withdraws the exemption, because writing a sign says the literal is meant as a magnitude again. `a : s8 = -16#ff;` gives `error: -255 does not fit s8, whose range is -128..=127`, while `a : s8 = -16#7f;` is the ordinary `-127`.

Decimal integers have arbitrary precision

A decimal integer token has no 128-bit ceiling. While its type is still undecided it is a `comptime_int`; if its magnitude is too large for the token's 128-bit payload, the lexer keeps the separator-free digits and the checker materialises the exact value as a `BigInt`. A destination such as `u129` can therefore receive a decimal value above `u128` with no truncation.

GLOSSO

```
#import "Io";

id :: (x: u129) -> u129 { return x; }
FOLD :: #comptime id(340_282_366_920_938_463_463_374_607_431_768_211_455) + 1;

main :: () {
    a: [1]u129 = .[340_282_366_920_938_463_463_374_607_431_768_211_455];
    direct: u129 = 340_282_366_920_938_463_463_374_607_431_768_211_456;
    print("% % %\n", a[0] + 1, FOLD, direct);
}
```

CONSOLE / OUTPUT

```
340282366920938463463374607431768211456 340282366920938463463374607431768211456 340282366920938463463374607431768211456
```

Context still performs the ordinary exact fitting check. A decimal value that is outside its destination reports `does not fit T, whose range is ...`; it is not a lexical error. For example, changing `direct` above to `u128` reports that its exact value does not fit the `u128` range. Based literals are different: they are stored as bit patterns, and that representation remains limited to 128 bits. For example, `16#1_0000_0000_0000_0000_0000_0000_0000_0000` reports `error: base-16 integer literal does not fit in 128 bits`, even with a `u129` destination. Write the value in decimal when more than 128 bits are required.

> **NOTE**
> The remaining 128-bit limit belongs to compile-time VM execution, not to decimal literals, constant folding, or native code. A top-level `#comptime { ... }` step that tries to execute arithmetic on `u129` reports `compile-time execution does not support u129, an integer wider than 128 bits`. Foldable `#comptime` constants such as `FOLD` above and compiled `u129` arithmetic remain exact.

Rejected spellings

The C prefixes are not merely unsupported; they are detected and named, so a constant copied out of a C header fails with an instruction rather than with a confusing split into a zero and an identifier.

Written	Diagnostic
`0xff`	`legacy '0x' integer literals are not supported; use '16#digits'`
`0B10`	`legacy '0b' integer literals are not supported; use '2#digits'`

Both letter cases are caught, for both prefixes. There is no `0o` case, so an octal constant in the C spelling does split: `0o7` lexes as the integer `0` followed by the identifier `o7`, and fails with `error: unbound 'o7'`.

2.4 Float Literals

A number token becomes a float when either of two shapes appears, and one literal may use both at once.

- A `.` followed by at least one digit, which opens the fraction: `1.25`, `1_234.333_333`.
- An `e` or `E`, optionally followed by `+` or `-`, followed by at least one digit: `1e3`, `1E10`, `1e+2`, `1.2e-3`.

The exponent is looked for whether or not a fraction was seen, so an integer-looking literal with an exponent is still a float. `_` separators are allowed in the integer part, the fraction and the exponent digits, under the same rules and with the same per-part diagnostics given in Integer Literals.

GLOSSO

```
#import "Io";

main :: () {
    a := 1.25;
    b := 1_234.333_333;
    c := 1e3;
    d := 1.2e-3;
    e := 1E10;
    f := 1e+2;
```

```
    g := 2.5e-3;
    print("% % % % % % %\n", a, b, c, d, e, f, g);
}
```

CONSOLE / OUTPUT

```
1.25 1234.333333 1000 0.0012 10000000000 100 0.0025
```

Anything outside those two shapes is not a float literal. The four spellings below all look like numbers and none of them is one.

Written	Result
`.5`	A bare `.`, then `5`: `error: unexpected token '5'`
`1.`	`error: float literal '1.' needs a digit after the '.'; write '1.0'`
`1.e3`	`1`, then `.`, then the name `e3`: `error: member of non-struct`
`1.5e`	An exponent with no digits: `error: scientific number literal needs exponent digits after 'e' or 'E'`

Requiring a digit on each side of the `.` is what keeps a dot next to a number unambiguous. `.` is member access, `..` is a range and `.*` is a dereference, so a fraction allowed to start or end with nothing would collide with all three.

Literal precision and expected types

The scanner strips the separators and parses the remaining text once, at double precision. The token also keeps its source digits, and a literal with no annotation defaults to `f64`: `x := 1.5;` followed by `y : string = x;` reports `error: got f64, expected string`.

For `f16` and `f32`, the checker rounds those original digits directly to the expected format, avoiding a second rounding through `f64`. A literal that overflows that format or rounds a nonzero value to zero is rejected. A wider annotation still changes where the value lands, not how it was read: An `f80` or `f128` literal has already been rounded to double precision by the time the type is known, so digits beyond roughly the seventeenth are gone before the annotation is ever consulted.

GLOSSO

```
#import "Io";

main :: () {
    a : f80  = 1.000_000_000_000_000_000_1;
    b : f80  = 1.0;
    c : f128 = 1.000_000_000_000_000_000_1;
    d : f128 = 1.0;
    print("f80  literal == 1.0: %\n", a == b);
    print("f128 literal == 1.0: %\n", c == d);
}
```

CONSOLE / OUTPUT

```
f80  literal == 1.0: 1
f128 literal == 1.0: 1
```

> **NOTE**
> Both comparisons are true. `f80` and `f128` are real storage widths with real arithmetic; only the literal is limited, and a constant more precise than a double cannot be written down as one token.

The lexer's `f64` range check

Every float token is parsed as `f64` before an expected type is applied. The scanner rejects a non-finite result, and it also rejects a nonzero mantissa that underflows to zero. This check is lexical, so an `f80` or `f128` annotation cannot make a token outside the host `f64` range legal.

Written	Result
`x : f64 = 1e400;`	`error: float literal '1e400' is out of range for f64`
`x : f64 = 1e-400;`	`error: float literal '1e-400' is out of range for f64`
`x : f64 = 0e-400;`	Zero; its mantissa was already zero

That token check is separate from conversion to a narrower destination. A value that is finite as `f64` can still overflow or underflow when the annotation narrows it. For example, `x : f16 = 100000.0;` is accepted and stores positive infinity because `f16` stops at 65504.

> **NOTE**
> The range check concerns the token's `f64` payload, not the eventual storage width. It prevents a literal itself from silently becoming infinity or nonzero-to-zero during lexing; it does not add a general checked float-conversion rule.

A float literal never reaches an integer target. `x : s64 = 1.5;` is `error: cannot implicitly convert comptime_float to s64`, and `x : u8 = 1.0;` is `error: cannot implicitly convert comptime_float to u8` even though the value is whole. That rule and its escape hatches are in Conversions: What Is Implicit.

2.5 String Literals and Escapes

A string literal is written between double quotes. The scanner walks forward to the closing quote, stepping two characters over any backslash pair so that an escaped quote does not end the literal. It then validates every escape and stores the raw source text found between the quotes.

Decoding is deferred: the stored text still holds the backslashes as written, and escapes become characters later, when the literal turns into a value. Validation is not deferred. An unknown escape, a malformed hexadecimal escape, or an invalid Unicode scalar is a lexical error.

GLOSSO

```
#import "Io";

main :: () {
    controls := "\a\b\f\n\r\t\v\0";
    print("count %\n", controls.count);
    print("% % % % % % % %\n",
        controls[0], controls[1], controls[2], controls[3],
        controls[4], controls[5], controls[6], controls[7]);
    print("[%]\n", "\\ \" \' \? \x41 \u{03bb}");
}
```

CONSOLE / OUTPUT

```
count 8
7 8 12 10 13 9 11 0
[\ " ' ? A λ]
```

The escape table

Escape	Result
`\a`	U+0007, alert
`\b`	U+0008, backspace
`\f`	U+000C, form feed
`\n`	U+000A, line feed
`\r`	U+000D, carriage return
`\t`	U+0009, tab
`\v`	U+000B, vertical tab
`\0`	U+0000; retained by a counted string, but terminates a C-string view
`\\`	One backslash
`\"`	A double quote
`\'`	An apostrophe
`\?`	A question mark
`\xHH`	The code point U+00HH; exactly two hexadecimal digits
`\u{H...}`	One Unicode scalar; one to six hexadecimal digits
`\%`	Nothing; the backslash and the per-cent sign both survive

The table is closed. `\x` requires exactly two hexadecimal digits. `\u` requires braces, one to six hexadecimal digits, and a value that is a Unicode scalar rather than a surrogate or a value above U+10FFFF. Every other character after a backslash is rejected by name.

CONSOLE / OUTPUT
```
main :: () { _ := "\q"; }
error: unknown string escape '\q'

main :: () { _ := "\xA"; }
error: \x escape needs exactly two hexadecimal digits

main :: () { _ := "\u{}"; }
error: \u escape needs hexadecimal digits and a closing '}'

main :: () { _ := "\u{d800}"; }
error: \u escape is not a Unicode scalar value
```

`\xHH` and `\u{...}` write code points, not bytes

The hexadecimal digits name a code point, not a storage byte. The character they select is then stored in the string's UTF-8 encoding, so an escape above `\x7f` may occupy several bytes and `.count`, which counts bytes, reports the encoded length.

GLOSSO
```
#import "Io";

main :: () {
    ascii := "\x41\x7f";
    high  := "\xff";
    print("ascii count % bytes % %\n", ascii.count, ascii[0], ascii[1]);
    print("high  count % bytes % %\n", high.count, high[0], high[1]);
}
```

CONSOLE / OUTPUT
```
ascii count 2 bytes 65 127
high  count 2 bytes 195 191
```

> **NOTE**
> `\xff` is U+00FF encoded as the two bytes 195 and 191, not the single byte 255. An ordinary string literal produces valid UTF-8. To write arbitrary bytes, use a byte-array literal such as `b"\xff"`, whose hexadecimal escape contributes the single byte 255.

`\%` and the runtime formatter

`%` is the placeholder in a format string, so a literal per-cent sign written in a format is counted as a placeholder and the call fails.

GLOSSO
```
#import "Io";

main :: () {
    print("50% off\n");
}
```

CONSOLE / OUTPUT
```
error: print: format has 1 '%' placeholder(s) but 0 argument(s) given
```

`\%` is the answer, and it is the one escape the decoder deliberately leaves alone: both characters survive into the stored value. What turns them into a single `%` is the runtime formatter, which recognises `\%` while scanning the format and emits a per-cent sign without consuming an argument.

GLOSSO
```
#import "Io";

main :: () {
    print("100\% done\n");
    print("[%]\n", "pct=\%");
}
```

CONSOLE / OUTPUT
```
100% done
[pct=\%]
```

The second line shows the division of labour. That `\%` sits in an argument rather than in the format, nothing scans it as a format, and both characters print.

Line breaks and termination

A literal may span physical lines. The newline is part of the text, and the line counter is corrected afterwards so that later diagnostics still report the right line. A backslash immediately before a physical newline is an explicitly accepted escape: decoding drops the backslash and keeps the newline, so it is not a line continuation.

GLOSSO

```
#import "Io";

main :: () {
    plain := "a
b";
    slashed := "a\
b";
    print("% %\n", plain.count, slashed.count);
    print("% %\n", plain[1], slashed[1]);
}
```

CONSOLE / OUTPUT

```
3 3
10 10
```

Both strings are three bytes long and both carry a line feed, byte 10, in the middle. A literal with no closing quote is an error rather than a run to end of file: `s := "abc;` alone on its line gives `error: unterminated string literal`, underlining from the opening quote to the end of the line.

The four string types

One literal syntax serves all four built-in string types; the annotation decides which is built, and whether a terminating NUL and a UTF-16 re-encoding are added. `.count` is the element count in every case and never includes a terminator.

GLOSSO

```
#import "Io";

main :: () {
    plain : string    = "hello";
    c     : cstring   = "hello";
    wide  : string16  = "hello";
    cwide : cstring16 = "hello";
    print("% % % %\n", plain.count, c.count, wide.count, cwide.count);
}
```

CONSOLE / OUTPUT

```
5 5 5 5
```

All four string types support the default `%` rendering through `Show`. The C forms stop at their first zero unit; counted strings keep an explicit length, including any embedded zeros. Their view layouts and storage lifetimes are covered in Built-in Strings.

C-string and byte-array literals

The adjacent lowercase prefix `c` makes a UTF-8 C-string literal: `c"hello"` has type `cstring` without an annotation. Its static storage has a trailing zero and is borrowed; never free it. An embedded zero ends its visible length. The prefix fixes the type: it does not adapt to `cstring16`. Use `cast(string)` to make a counted view of its visible bytes, or use an ordinary literal for contextual UTF-16 conversion.

The prefix `b` instead makes a fixed `[N]u8` array, with no added terminator. Text and Unicode escapes are encoded as UTF-8, while each `\xNN` contributes exactly one byte. A local array can be modified; copying it copies its bytes. `b""` has type `[0]u8`. The element type stays `u8`, and an explicit length must match. `[_]u8` infers the length from the initializer; `[*]u8` remains a many-pointer type.

GLOSSO

```
#import "Io";
#import "Meta";

main :: () {
    text := c"A\0tail";
    bytes: [_]u8 = b"é\xFF\0";
    copy := bytes;
    bytes[0] = 65;
    print("C: % count=% terminator=%\n", type_name(type_of(text)), text.count, text.data[text.count]);
    print("bytes: % count=% values=% % % %\n", type_name(type_of(bytes)), bytes.count,
        bytes[0], bytes[1], bytes[2], bytes[3]);
    print("copy first=% empty count=%\n", copy[0], b"".count);
}
```

CONSOLE / OUTPUT

```
C: cstring count=1 terminator=0
bytes: [4]u8 count=4 values=65 169 255 0
copy first=195 empty count=0
```

> **NOTE**
> The prefix must be one lowercase letter immediately before the quote. `c "text"`, `B"text"`, and `name"text"` are an identifier followed by an ordinary string, not prefixed literals. Both prefixed forms use the same escape validation as ordinary strings.

2.6 `#string` Multi-Line Blocks

`#string` builds a string literal out of a run of marked lines. Each line of the block begins with the marker `\\`, and everything after those two backslashes, to the end of that line, is captured. The directive collects every consecutive marked line and joins them into one literal.

GLOSSO

```
#import "Io";

BANNER :: #string
\\alpha
\\beta
;

main :: () {
    print("[%] count=%\n", BANNER, BANNER.count);
}
```

CONSOLE / OUTPUT

```
[alpha
beta] count=10
```

The marker is a token in its own right, produced by the scanner wherever two backslashes appear. A single backslash is not a token at all: it is the stray-character error `error: '\'`. A marked line is consumed only by `#string` and by the structured form of `#asm`, described in the assembly chapter; anywhere else the parser reports the captured text as the offending token, so a stray `bare := \\line one` gives `error: unexpected token 'line one'`.

What the marker captures

Capture starts immediately after the second backslash and stops at the newline, which is not part of the text. Whitespace before the marker is ordinary whitespace and is discarded, so a block can be indented to match its surroundings without changing its value. Whitespace after the marker is text. Nothing inside the captured span is interpreted: quotation marks, backslashes, `//` and `/* */` are all ordinary characters.

- Indentation written before the marker positions the block in the source and never reaches the string; indentation written after the marker does reach it.
- A marked line with no text after the marker contributes an empty line.
- Comments are removed before the directive sees the token stream, so a `//` or `/* */` comment written between two marked lines does not interrupt the run: the lines on either side still join into one literal.
- No escape decoding runs unless `,escape` is given, so a `\t` written after a marker is a backslash followed by the letter `t`, two characters.

GLOSSO

```
#import "Io";

INDENTED :: #string
    \\    four spaces survive
    \\next
;

VERBATIM :: #string
\\he said "hi" // not a comment /* nor this */ \n \t \\
;

WITH_BLANK :: #string
\\a
\\
\\b
;

COMMENTED :: #string
\\alpha
// this comment never reaches the directive
\\beta
```

```
;

main :: () {
    print("INDENTED   [%] count=%\n", INDENTED, INDENTED.count);
    print("VERBATIM   [%] count=%\n", VERBATIM, VERBATIM.count);
    print("WITH_BLANK [%] count=%\n", WITH_BLANK, WITH_BLANK.count);
    print("COMMENTED  [%] count=%\n", COMMENTED, COMMENTED.count);
}
```

CONSOLE / OUTPUT

```
INDENTED   [    four spaces survive
next] count=28
VERBATIM   [he said "hi" // not a comment /* nor this */ \n \t \\] count=53
WITH_BLANK [a

b] count=4
COMMENTED  [alpha
beta] count=10
```

NOTE

Because capture runs to the end of the line, the semicolon that ends the declaration must sit on a line of its own. `SAME :: #string \\text ;` is a legal declaration whose value is the six characters `text ;`, not a two-word string.

The joining rule

The consecutive marked lines are collected in source order and joined with a single separator: a newline by default, a single space under `,oneline`. The separator goes between lines only, so a block never ends with one. At least one marked line is required; `V :: #string;` reports the error below.

CONSOLE / OUTPUT

```
error: #string expects one or more '\\' lines
```

`#string` is an expression, not only a declaration form, so a block may appear anywhere a string is expected: a local initialiser, an argument, a struct field.

The two modifiers

Two modifiers exist and no others. They are written as comma-separated identifiers directly after the directive, may be given in either order, and may each be given at most once. `,oneline` changes the separator from a newline to a single space. `,escape` changes how the joined text is stored: without it the text is already final and backslashes stay literal, with it the escape decoder described in the chapter on string literals is applied to the joined text.

Form	Separator	Escape decoding
`#string`	newline	none: backslashes are literal
`#string,oneline`	one space	none: backslashes are literal
`#string,escape`	newline	string-literal escape decoding
`#string,escape,oneline`	one space	string-literal escape decoding

NOTE

Marked `#string` lines are tokenized separately and do not pass through the quoted-literal escape validator. Use the documented escape forms with `,escape`; malformed and unknown sequences are not guaranteed the same clean lexical diagnostics as an ordinary quoted string.

The four forms differ on exactly two axes, so the same two marked lines produce four different strings. This program reports each result by length and by the byte codes at offsets three, four and five, which is where the two axes show up.

GLOSSO

```
#import "Io";

RAW :: #string
\\bar\t
\\foo
;

ONELINE :: #string,oneline
\\bar\t
\\foo
;

ESCAPED :: #string,escape
```

```
\\bar\t
\\foo
;

BOTH :: #string,escape,oneline
\\bar\t
\\foo
;

report :: (name: string, value: string) {
    print("% count=% bytes 3..5 = % % %\n", name, value.count,
        value[3], value[4], value[5]);
}

main :: () {
    report("RAW                 ", RAW);
    report("ONELINE             ", ONELINE);
    report("ESCAPED             ", ESCAPED);
    report("ESCAPE + ONELINE    ", BOTH);
}
```

CONSOLE / OUTPUT

```
RAW                 count=9 bytes 3..5 = 92 116 10
ONELINE             count=9 bytes 3..5 = 92 116 32
ESCAPED             count=8 bytes 3..5 = 9 10 102
ESCAPE + ONELINE    count=8 bytes 3..5 = 9 32 102
```

Byte 3 is `92` and byte 4 is `116` in the two undecoded forms: a backslash and a `t`, nine characters in all. Under `,escape` the pair has become the single tab `9` and the string is one character shorter. Byte 5 shows the separator: `10` is the newline of the default join, `32` the space of `,oneline`.

- Under `,oneline` a blank marked line still contributes a segment, so the three lines `a`, empty, `b` join as `a  b`, four characters with two spaces.
- `,escape` decodes escapes written inside the captured text; it has no effect on the marker, which is consumed by the scanner before any of the text exists.
- `\%` survives escape decoding as the two characters `\` and `%`, in a marked block exactly as in a quoted literal, because it is the runtime formatter and not the lexer that turns it into a per-cent sign.

That last rule is why an undecoded block still works as a `print` format. The block below carries a literal `\%` and a `%` placeholder, and both behave as they would in a quoted format string.

GLOSSO

```
#import "Io";

FORMAT_RAW :: #string
\\raw format: 100\% and a value %
;

main :: () {
    print(FORMAT_RAW, 7);
    print("\n");
    inline_use := #string,oneline
        \\an expression, not only a declaration,
        \\joined across two marked lines
    ;
    print("[%]\n", inline_use);
}
```

CONSOLE / OUTPUT

```
raw format: 100% and a value 7
[an expression, not only a declaration, joined across two marked lines]
```

Modifier errors

An identifier after the comma that is neither `oneline` nor `escape` is rejected by name, and so is a repeat of either one. Both diagnostics quote the modifier back.

CONSOLE / OUTPUT

```
V :: #string,wrapped
error: unknown #string modifier 'wrapped'; expected 'oneline' or 'escape'

V :: #string,oneline,oneline
error: duplicate #string modifier 'oneline'

V :: #string,escape,escape
error: duplicate #string modifier 'escape'
```

NOTE
Capture excludes the line ending. When the scanner finds LF it also removes a CR immediately before it, so a two-line `alpha` / `beta` block has `count == 10` in both LF and CRLF source files.

2.7 `u8`, `rune`, and `#char` Literals

`#char` writes one character value. The scanner handles the form inside directive scanning and emits a marked integer token; the parser keeps that marker, and the checker defaults ASCII literals to `u8` and other literals to `rune`. The old `char` type has been removed; `#char` remains the spelling of the literal directive.

GLOSSO
```
#import "Io";
#import "Meta";

main :: () {
    a := #char "A";
    l := #char "λ";
    print("A      = % (%)\n", a, type_name(type_of(a)));
    print("lambda = % (%)\n", l, type_name(type_of(l)));
    contextual: rune = #char "A";
    print("contextual A = % (%)\n", cast(u32)contextual,
        type_name(type_of(contextual)));
}
```

CONSOLE / OUTPUT
```
A      = 65 (u8)
lambda = 955 (rune)
contextual A = 65 (rune)
```

The form is the directive, then optional spaces and tabs, then a quoted string holding exactly one direct code point, one fixed two-character escape, one exact `\xHH` escape, or one `\u{...}` scalar escape. The separator may be empty, so `#char"A"` is the same literal as `#char "A"`. It may not contain a newline: a string on the following line is `error: #char expects a string literal`.

Bytes and Unicode scalars

`u8` is an ordinary integer byte; `rune` is a four-byte Unicode scalar. Only `rune` is outside `Is_Integer` and `Is_Numeric`: it has equality and ordering but no arithmetic, bitwise, shift, integer-range, or SIMD-lane operations. Direct string indexing yields `u8` or `u16` code units. An ASCII `#char` literal compares directly with a byte; a UTF-8 character above ASCII may occupy several bytes.

GLOSSO
```
#import "Io";

is_digit_byte :: (byte: u8) -> bool {
    return byte >= #char "0" && byte <= #char "9";
}

main :: () {
    text : string = "h7";
    print("first is h: %\n", text[0] == #char "h");
    print("second is a digit: %\n", is_digit_byte(text[1]));
    c := #char "A";
    r := c.char_to_rune();
    print("char_to_rune(A): %\n", cast(u32)r);
}
```

CONSOLE / OUTPUT
```
first is h: 1
second is a digit: 1
char_to_rune(A): 65
```

The accepted escapes

The scanner decodes the escape itself rather than deferring to the string machinery, so `#char` has its own closed table. It uses the fixed C escapes, an exact two-digit hexadecimal form, and a Unicode scalar form with one to six hexadecimal digits inside braces.

Escape	Value	Character
`\a`	7	alert
`\b`	8	backspace

Escape	Value	Character
`\f`	12	form feed
`\n`	10	line feed
`\r`	13	carriage return
`\t`	9	tab
`\v`	11	vertical tab
`\0`	0	NUL
`\\`	92	backslash
`\"`	34	quotation mark
`\'`	39	apostrophe
`\?`	63	question mark
`\xHH`	0 through 255	exactly two hexadecimal digits
`\u{H...}`	Unicode scalar	one to six hexadecimal digits; no surrogates or values above U+10FFFF

GLOSSO

```
#import "Io";

main :: () {
    print("a=% b=% f=% n=% r=% t=% v=%\n",
        #char "\a", #char "\b", #char "\f", #char "\n",
        #char "\r", #char "\t", #char "\v");
    print("0=% backslash=% quote=% apostrophe=% question=% hex=%\n",
        #char "\0", #char "\\", #char "\"", #char "\'", #char "\?",
        #char "\x41");
    print("unicode=%\n", cast(u32)(#char "\u{1F600}"));
}
```

CONSOLE / OUTPUT

```
a=7 b=8 f=12 n=10 r=13 t=9 v=11
0=0 backslash=92 quote=34 apostrophe=39 question=63 hex=65
unicode=128512
```

An escape outside the table is rejected instead of dropping its backslash. Thus `#char "\q"` reports `error: unknown #char escape '\q'`. Escaping a direct non-ASCII character is rejected too rather than truncating the code point.

`\xHH` requires exactly two hexadecimal digits. `\u{...}` accepts one to six hexadecimal digits and must name a Unicode scalar, excluding surrogates and values above U+10FFFF. A non-ASCII result has type `rune`, just as if that code point had been written directly.

CONSOLE / OUTPUT

```
main :: () { c := #char "\x41"; }
accepted: c is 65

main :: () { c := #char "\x4"; }
error: #char \x escape needs exactly two hexadecimal digits

main :: () { c := #char "\q"; }
error: unknown #char escape '\q'

main :: () { c := #char "ab"; }
error: #char string literal must contain exactly one character

main :: () { c := #char ""; }
error: #char expects one character

main :: () { c := #char 5; }
error: #char expects a string literal

main :: () { c := #char "a; }
error: unterminated #char string literal
```

A code point above ASCII is written directly instead: `#char "λ"` has scalar value `955`, because the source is decoded to code points before the scan and the literal takes one of them.

The synthesised type

Left alone, an ASCII `#char` synthesises as `u8`; every other code point synthesises as `rune`. Context can select `s8` or `rune` for an ASCII literal. Untyped constants preserve that literal choice, but a stored byte does not: use `u8.char_to_rune` or an explicit cast to widen a byte variable. Where several overloads match an ASCII literal, `u8` is preferred to `s8`, and `s8` to `rune`.

Code point	Default type	Types it accepts
at most `0x7f`	`u8`	`u8`, `s8`, `rune`
above `0x7f`	`rune`	`rune`

CONSOLE / OUTPUT

```
main :: () { c : u32 = #char "A"; }
error: #char literal can infer only as u8, s8, or rune, not u32

main :: () { c : u8 = #char "λ"; }
error: #char literal can infer only as rune, not u8

main :: () { c := #char "A"; r : rune = c; }
error: got u8, expected rune
```

NOTE

The contextual list is closed: other integer targets are refused even when their representation is wide enough. Low-level conversion remains explicit: `cast(u8)(#char "A")` is `65`, and `cast(u32)(#char "λ")` is `955`. Conversely, an integer cast can forge a `rune` outside its scalar domain, so validate external data before casting. `%c` renders bytes or runes as UTF-8; bare `%` renders their decimal value.

2.8 Punctuation and the Token Table

The scanner produces tokens of a fixed set of kinds. Punctuation and operators are drawn from the same eighteen characters, and which one a run of those characters becomes is decided by a table of special cases first and a maximal-munch rule afterwards. Every spelling in the punctuation table below is a token kind of its own, which is why none of them can be redeclared as an operator.

The punctuation table

The middle column names the token kind the scanner emits. Some of these spellings, including `.*`, `.&`, `?`, and `@`, are operator tokens rather than dedicated kinds, but they are punctuation in practice and are listed here with the rest.

Spelling	Token	Meaning
`;`	Semicolon	statement and declaration terminator; grammar delimiter
`,`	Comma	separator in lists and literals
`,,`	CommaComma	nonlocal label, as in `main,,'entry`
`.`	Dot	member access
`.{ .[`	DotBrace, DotBracket	aggregate literal, array literal
`.*`	Op	pointer dereference, written after the pointer
`.&`	Op	address of the preceding value
`.. ..=`	Range, RangeInclusive	exclusive and inclusive ranges
`...`	Ellipsis	variadic parameter and the rest pattern
`:`	Colon	type annotation
`::`	ColonColon	constant binding
`:::`	TripleColon	separates an instance head from `instance`
`:=`	ColonEqual	variable binding
`::=`	ColonColonEqual	compile-time binding
`=`	Equal	assignment

Spelling	Token	Meaning
`->`	Arrow	return type
`$`	Dollar	generic parameter introducer
`?`	Op	optional type former and the propagation suffix
`@`	Op	pattern binder, as in `whole @ subpattern`
`( ) { } [ ]`	LParen through RBracket	grouping, blocks, brackets

Two absences are worth naming. `.(`, which opens a tuple literal, is not one token: it is `Dot` followed by `LParen`. And `x : := e` is not a third form; the scanner emits `Colon` and then `ColonEqual`, and the parser reads that pair as `::=`. The remaining token kinds carry text rather than punctuation: `Ident`, `Int`, `Float`, `Str`, `CStr`, `BStr`, `MlStr`, `Directive`, `Op`, `OpName`, `Backtick`, `DoubleBacktick` and `Label`, each covered by the chapter that owns its construct.

GLOSSO

```
#import "Io";

Point :: struct { x: ssize; y: ssize; }

sum_all :: (values: ...ssize) -> ssize {
    total := 0;
    for v : values total += v;
    return total;
}

main :: () {
    K :: 10;                        // ::   constant binding
    n := 3;                         // :=   variable binding
    m ::= 6 * 7;                    // ::=  compile-time binding
    q : ssize = 5;                    // :    type annotation
    q = 6;                          // =    assignment
    p : Point = .{ .x = 1, .y = 2 };  // .{   aggregate literal
    a : [5]ssize = .[10, 20, 30, 40, 50];
    t := .(1, "two");               // .(   Dot then LParen
    print("K=% n=% m=% q=% p=% a=% t=%\n", K, n, m, q, p.x, a[2], t.0);
    print("full=% tail=% head=% mid=%\n",
        a[..].count, a[2..].count, a[..2].count, a[1..=3].count);
    print("varargs=%\n", sum_all(1, 2, 3));
    r := &p;
    print("deref x=%\n", r.*.x);
}
```

CONSOLE / OUTPUT

```
K=10 n=3 m=42 q=6 p=1 a=30 t=1
full=5 tail=3 head=2 mid=3
varargs=6
deref x=1
```

The operator character set

Exactly eighteen ASCII characters may appear in an operator token. Any run of them that is not caught by one of the special cases below is scanned as a single operator token.

CONSOLE / OUTPUT

```
+ - * / < > ! @ % ^ & | ~ ? . , : =
```

What is absent matters as much as what is present. `$`, `#`, `\`, `'`, backtick, `"`, `;` and the six bracket characters are not operator characters, so they always end an operator run. That is why `x: []ssize` and `Err![]T` lex the way they read, and why an operator whose name contains a bracket can only be written in the quoted form described in the chapter on reserved operators.

Maximal munch and its exceptions

The scanner takes the longest run of operator characters it can, which it must: a library may introduce any spelling, so the scanner cannot know that `+|` is one operator and `+ |` two without preferring the longer read. Five characters carry extra rules on top of that. `?` never continues an operator, so `*?T` is three tokens and a propagation suffix composes with whatever follows it. A lone `@` is taken by itself, but only when no operator character follows, which keeps `@=` and `@>` available as user spellings. After `:` the scanner checks the fixed forms `:::`, `::=`, `::` and `:=` first, then falls into the general scan unless the next character is `*`. The order is load-bearing: `T ::: instance C` is one `TripleColon` token, not a constant-binding token followed by a colon. After `=` it falls into the general scan whenever another operator character follows. A trailing `.` is handed back when the character after it could start a non-operator token. The pointer suffixes `.*` and `.&` are

recognized as separate tokens, so `p.*.field.&` chains without spaces. If a token still ends in `.` it is `error: operator token cannot end with '.'`.

Written	Lexes as	Result
`a = -1`	`= - 1`	assignment of a negated literal
`a =-1`	`=-`	`error: unexpected token '=-'`
`1 ++2`	`++`	`error: unexpected token '++'`
`o: ?ssize`	`:` then `?ssize`	an optional annotation
`o:?ssize`	`:` alone	`error: reserved operator token ':'`
`p:*ssize`	`: *`	works, by the `*` exception
`Err! *ssize`	`! *`	a result of a pointer
`Err!*ssize`	`!*`	`error: unexpected token '!*'`
`2 * p.*.b`	`2 * p .* . b`	explicit dereference before the field
`2 *. 3`	`*.`	`error: operator token cannot end with '.'`

NOTE
Whitespace between two operator characters is load-bearing. The two pairs a program is most likely to write by accident are `x:?T`, which loses the `?` and reports a reserved `:`, and `E!*T`, which glues `!` and `*` into one unknown operator. Both read perfectly well and both fail; a space fixes each.

2.9 Reserved Operators and Operator Names

The punctuation spellings below are reserved. The check lives in the scanner and fires on the token, not on the declaration, so it catches a reserved spelling written as a quoted operator name and a reserved spelling that reaches operator position by accident with the same message. The message always names the operator twice, once for the token and once for the rule.

GLOSSO
```
X :: '=';
main :: () {}
```

CONSOLE / OUTPUT
```
error: reserved operator token '=' (operator '=' cannot be overloaded)
```

The reserved spellings

Each of these carries syntax the parser must be able to recognise without consulting any operator table, which is what makes it unavailable for overloading.

Spelling	What it is
`=`	assignment
`.`	member access
`,`	list separator
`,,`	nonlocal label constructor
`:=`	variable binding
`::`	constant binding
`:::`	instance declaration separator
`:`	type annotation
`->`	return-type arrow
`::=`	compile-time binding

Spelling	What it is
`.*`	pointer dereference
`.&`	address of the preceding value
`...`	varargs and the rest pattern
`@`	pattern binder
backtick	hygiene escape
double backtick	code splice

Because the check is on the token, a mistyped expression can produce it. Writing `. {` with a space reaches the operator scanner and reports `error: reserved operator token '.' (operator '.' cannot be overloaded)`. Write `.{` without a space to begin a struct literal. A trailing decimal point such as `1.` instead receives a float-literal diagnostic.

> **NOTE**
> The instance separator is reserved all the way down in the lexer. Writing `':::'` as an operator name reports `reserved operator token ':::' (operator ':::' cannot be overloaded)`, and the former `T :: instance C` form no longer parses. Write `T ::: instance C`.

Why `..`, `..=` and `?` are not on the list

All three have dedicated token kinds and dedicated parser handling, so they look like candidates for the list. They are absent on purpose. A range and a propagation are both defined by typeclasses in the library rather than built into the compiler, and a user type joins in by writing an instance. Reserving the spellings would make that impossible.

GLOSSO

```
// std/Prelude/Primitive_Operators.glo
RangeExclusiveOp :: typeclass (L: type, R: type) {
    Output :: type;
    '..' :: (start: L, end: R) -> Output #operator(left, 2);
}

// Method of Try in std/Prelude/Try.glo (body omitted).
'?' :: (#empty: $Target, value: Carrier) -> Try_Branch(Output, Target)
    #operator(suffix, try(Target))
    where Try(Target) &&
        (Try(Carrier).Residual == Try(Target).Residual ||
         Propagate(Try(Carrier).Residual, Try(Target).Residual));
```

The proof is in the diagnostic a competing declaration gets. Declaring `'..'` in a new typeclass is not rejected as reserved; it is rejected because the spelling already belongs to a class, which is the ordinary rule for every operator.

CONSOLE / OUTPUT

```
MyRange :: typeclass (L: type, R: type) {
    Output :: type;
    '..' :: (a: L, b: R) -> Output #operator(left, 2);
}
error: operator '..' is already owned by typeclass 'RangeExclusiveOp'
```

The quoted operator-name form

An operator is named by writing its characters between single quotes. After a quote the scanner looks ahead: if what follows is a letter or _ and the run is not closed by a second quote, the token is a label such as `'again`; otherwise it scans to the next quote and produces an operator name. So `'+'` names an operator and `'done` declares a label, with no ambiguity between them.

The text between the quotes is taken as written. The scanner applies only the reserved-spelling check and the trailing-dot check, so a name may contain characters that could never be lexed as an operator run, including letters and brackets. The declaration then succeeds even when the name can never be written at a use site.

- A letters-only name such as `'plus'` declares cleanly, but at a use site the letters lex as an identifier: `2 plus 3` is `error: unexpected token 'plus'`.
- `''` is a legal empty name, and using it produces `error: unbound ''`.
- `'[]'` and `'[]='` are the indexing and index-assignment operators. They are the only names allowed to contain a bracket; any other gives `error: operator '<[>' cannot contain '[' or ']'` with the offending name quoted back.

Beyond the spelling, the checker constrains what an operator declaration may say. These rules are enforced when the declaration is checked, not when it is scanned.

Rule	Diagnostic
one to three parameters	`operator functions need one, two, or three parameters`
must belong to a typeclass	`operator '<~>' must be declared by a typeclass and implemented by its instances`
one owning class per spelling	`operator '+' is already owned by typeclass 'Add'`
`'[]'` takes exactly two	`operator '[]' must have arity 2`
an all-stars or all-ampersands name is never prefix	`operator '**' cannot be overloaded as a prefix operator`

NOTE
Prefix `*`, `**`, and longer all-stars spellings are built-in dereferences, or pointer constructors when their operand is a type. Prefix `&`, `&&`, and longer all-ampersands spellings are built-in address operations. They cannot be overloaded as prefix operators, including through macros; binary uses remain available. The suffixes `.*` and `.&` are reserved and cannot be overloaded in any fixity.

2.10 The Operator Prescan

Before a file is parsed, its whole token vector is walked once by the prescan, which looks for operator declarations and records their fixity, precedence and associativity. Only then does parsing begin.

The prescan exists because Glosso lets a library introduce operator spellings and precedences. Parsing cannot decide how `2 * 3 %% 4` groups, or even whether `%%` is an operator at all, until every `#operator` declaration available to the file's prescan has been read. A single-pass parser would have to see the declaration first; the prescan removes that ordering requirement by reading the declarations out of the token stream before any of them is parsed.

What the prescan recognises

The pattern is narrow and purely lexical: an operator name, then `::`, then an opening parenthesis. From the matching close parenthesis the scan runs forward to the first `{` or `;`, which is to say across the signature and its directives and no further, looking for `#operator`.

GLOSSO
```
'%%' :: (a: L, b: R) -> Output #operator(left, 19);
'~~' :: (value: T) -> Output #operator(prefix);
'!!' :: (value: T) -> Output #operator(suffix);
```

`#operator(prefix)` and `#operator(suffix)` register unary fixity. `#operator(left, prec)`, `#operator(right, prec)` and `#operator(assign, prec)` register precedence and associativity. Use these same forms on the owning typeclass's method signature; there is no separate signature-only precedence directive.

Mode	Registers	Meaning
`left`	precedence, left associativity	ordinary binary operator
`right`	precedence, right associativity	right-grouping binary operator
`assign`	precedence, right associativity	assignment-shaped operator
`prefix`	prefix fixity	written before its operand
`suffix`	suffix fixity	written after its operand

A prefix or suffix mode carries no precedence. Supplying a level, as in `#operator(prefix, 19)`, is `error: unexpected token ','`. A name made only of asterisks is never registered as prefix, because those spellings are the pointer sigils.

An operator may be used before it is declared

That is the whole visible consequence. The declaration below sits after `main`, and `main` uses the operator anyway, at the precedence the later declaration asks for.

GLOSSO
```
#import "Io";

main :: () {
    print("3 \%\% 4       = %\n", 3 %% 4);
    print("2 * 3 \%\% 4   = %\n", 2 * 3 %% 4);
}

#public Mod :: typeclass (L: type, R: type) {
```

```
    Output :: type;
    '%%' :: (a: L, b: R) -> Output #operator(left, 19);
}

(ssize, ssize) ::: instance Mod {
    Output :: ssize;
    '%%' :: (a: ssize, b: ssize) -> Output { return a * 10 + b; }
}
```

CONSOLE / OUTPUT

```
3 %% 4       = 34
2 * 3 %% 4   = 68
```

Precedence 19 binds tighter than `*` at 18, so the second line is `2 * (3 %% 4)`, which is `2 * 34`. Changing the one number in the declaration changes how the expression above it groups: at 17 the operator binds looser than `*` and the same source reads `(2 * 3) %% 4`.

CONSOLE / OUTPUT

```
3 %% 4       = 34
2 * 3 %% 4   = 64
```

A new binary operator with no `#operator(left|right|assign, n)` anywhere in the prescanned declarations has no registered precedence, so its use site fails as an unknown token. The declaration is reported too, with the directive it is missing. An instance can omit the directive when its owning class already supplies the level.

CONSOLE / OUTPUT

```
error: unexpected token '%%'
error: binary operator '%%' needs #operator(left|right|assign, n)
```

> **NOTE**
> The precedence must be a written integer literal. The prescan reads it straight out of the token stream, before any name means anything, so a named constant cannot be used: `#operator(left, P)` with `P :: 19;` is `error: unexpected token 'P'`, reported at the constant in the directive, and the declaration is rejected.

Operator syntax across relative modules

The resolver seeds the operator table from the Prelude and prescans relative `#load` and `#export` dependencies before parsing the caller. An operator declared in a loaded file can therefore be written in the loading file. The source files still have separate name visibility: learning an operator's syntax does not grant access to hidden methods or to an unrelated module's instances.

GLOSSO

```
// Mod2.glo: combine uses %% on its first line, before the declaration
#public combine :: (a: ssize, b: ssize) -> ssize { return a %% b; }

#public Mod :: typeclass (L: type, R: type) {
    Output :: type;
    '%%' :: (a: L, b: R) -> Output #operator(left, 19);
}

(ssize, ssize) ::: instance Mod {
    Output :: ssize;
    '%%' :: (a: ssize, b: ssize) -> Output { return a * 10 + b; }
}
```

GLOSSO

```
#import "Io";
#load "Mod2.glo";

main :: () {
    print("combine(3, 4) = %\n", combine(3, 4));   // fine
    print("operator = %\n", 3 %% 4);                // also fine
}
```

CONSOLE / OUTPUT

```
combine(3, 4) = 34
operator = 34
```

Both calls compile. The dependency prescan supplies `%%`'s precedence before the caller is parsed, and the explicit load supplies the public class and instance needed when the expression is checked.

- The early dependency walk follows relative `#load` and `#export` paths recursively. Directory imports have their own entrypoint and dependency traversal; use an explicit relative dependency when an operator's syntax must be available while parsing the caller.
- The Prelude is prescanned before anything else and its table seeds every file, which is why the arithmetic, comparison, bitwise and range spellings are available with no import and no declaration.
- A directory import reads `module.glo`; it does not discover all other files in the directory automatically. The entrypoint must load, export, or embed every required component explicitly.

A declaration may follow its use within a file because the prescan runs first. Across files, make the module dependency explicit and export the declarations that callers need.

3 Declarations and scope

3.1 Values, Constants, and Variables

Glosso has no `let`, `var`, or `const` keyword. A declaration is a name, an optional type annotation, and an initialiser, and the punctuation between them carries the whole meaning: whether the name is immutable, whether its value is required at compile time, and whether the type is written or inferred.

A colon introduces the type annotation. When the right side is an ordinary value expression, a single `:` or a `::` before it binds a constant; an `=`, `:=`, or `::=` binds something that can be written to. A declaration with a type and no initialiser is zero-filled. A definition-shaped right side—such as `struct`, `#import`, or a procedure—is the named-definition exception described below.

GLOSSO

```
#import "Io";

main :: () {
    LIMIT :: 64;
    step ::= 2;
    total := 0;
    scale: f64 = 1.5;
    seen: s64;
    total = LIMIT / step;
    print("total=% scale=% seen=%\n", total, scale, seen);
}
```

CONSOLE / OUTPUT

```
total=32 scale=1.5 seen=0
```

The declaration forms

The same eight value-binding spellings are accepted at file scope and inside a procedure body, and two of them mean different things in the two places. The difference is that only a declaration inside a body is entered into the checker's compile-time environment, so only a local value binding can be a compile-time binding. In the first eight rows, `v` means an ordinary value expression.

Spelling	At file scope	In a body
`x :: v`	Constant.	Constant.
`x : T : v`	Typed constant.	Typed constant.
`x : T :: v`	Typed constant.	Typed constant.
`x ::= v`	Mutable global.	Compile-time binding.
`x : T := v`	Mutable global.	Typed compile-time binding.
`x := v`	Mutable global.	Mutable local.
`x : T = v`	Mutable global.	Mutable local.
`x : T;`	Zero-filled global.	Zero-filled local.
`x :: definition`	Named definition.	Named definition in the enclosing lexical scope.
`x := definition`, `x ::= definition`, `x : D : definition`, or `x : D :: definition`	Named definition; its declaration type is inferred, or checked against `D`.	Not the named-definition grammar; `:=` and `::=` take the ordinary local-binding path.

Both `:` and `::` are accepted after a type annotation, so `MAX: ssize : 100` and `MAX: ssize :: 100` declare the same typed constant. Every ordinary value-binding form in the first eight rows ends with `;`, both at file scope and in an ordinary procedure body. A named definition instead follows its own grammar: a braced definition ends at `}`, while a definition such as `distinct` or `#import` carries its own `;`. Inside a `#code` quotation, the parser also lets a binding immediately before any closing `}` end there, including at the end of a nested block. Statements and Semicolons gives the complete rule.

> **NOTE**
> For an ordinary value expression, `::=` is the one spelling whose meaning flips between the two scopes. A file-scope `G ::= 7;` is an ordinary mutable global, not a compile-time constant, and a `#if G == 7` that follows it fails with `error: unknown const G`. A definition-shaped right side takes the named-definition path instead. Use `::` for a file-scope value constant.

Constants

`::` means an immutable binding. It comes in two flavours, and the compiler chooses between them without any help from the source: if the initialiser can be evaluated during checking the name becomes a compile-time constant, and if it cannot the name becomes a runtime-frozen constant instead. A `::` whose initialiser is not compile-time evaluable is not an error.

A runtime-frozen constant is lowered like an ordinary local: the initialiser is evaluated once, at the declaration, and the name is bound to that result for the rest of the scope. Later changes to whatever the initialiser read do not reach it.

GLOSSO

```
#import "Io";

main :: () {
    n := 7;
    FROZEN :: n * 2;
    print("FROZEN=%\n", FROZEN);
    n = 100;
    print("FROZEN=%\n", FROZEN);
}
```

CONSOLE / OUTPUT

```
FROZEN=14
FROZEN=14
```

The two flavours differ in exactly one visible way: a frozen constant is not a compile-time value, so it cannot drive a construct that demands one. Adding `#if FROZEN == 14` to the program above gives `error: unknown const FROZEN`, `#assert #comptime FROZEN == 14` gives `error: #assert #comptime condition is not compile-time evaluable`, and passing it to a `#comptime` parameter gives `error: argument for #comptime parameter 'v' must be known at compile time`. Each of those carries `note: unknown const FROZEN`.

Neither flavour has storage. `p := *FROZEN` is `error: cannot take a pointer to a non-addressable expression: 'FROZEN'`, and the same holds for a file-scope constant.

Compile-time bindings

Inside a body, `x ::= v` and `x: T := v` declare a compile-time binding. The initialiser must be compile-time evaluable, the name is registered in the compile-time environment, and the binding may be updated by ordinary assignment during checking. This is what drives `#inline while` counters and static branching.

GLOSSO

```
#import "Io";

main :: () {
    width ::= 4;
    width = width * 2;
    #if width == 8 print("the static branch saw %\n", width);
}
```

CONSOLE / OUTPUT

```
the static branch saw 8
```

An initialiser the checker cannot fold is rejected rather than frozen: `K ::= n * 2` over a runtime `n` is `error: comptime variable initializer must be compile-time evaluable`. A compile-time binding also has no runtime storage, so its address cannot be taken: `p := &width` is `error: cannot take pointer to width`. Values with no runtime representation must be bound this way or with `::`. That includes values of type `type`, `Namespace`, `Library`, `Code`, and `Pattern`, as well as typeclass and macro declarations.

Variables

`x := v` and `x: T = v` declare a mutable binding with storage. `:=` infers the type from the initialiser; the annotated form fixes it and checks the initialiser against it. `x: T;` declares a mutable binding with no initialiser, and the storage is zero-filled rather than left holding whatever was on the stack. A declaration with neither a type nor an initialiser has nothing to work from and is rejected.

Assigning to a constant

Assignment to a constant is always rejected, but the diagnostic depends on where the constant was declared and how it was spelled. A file-scope constant is not in the assignable namespace at all, so name resolution fails before the assignment rule is ever consulted; a local constant is found and then refused, with a help line naming the spelling that would have made it mutable.

Binding	Assigning to it
`K :: 7;` at file scope	`error: unbound 'K'`
`MAX: ssize : 100;` at file scope	`error: unbound 'MAX'`
`c :: 9;` in a body	`error: constant cannot be modified`, with `help: change '::' to ':=' to make the binding mutable`
`c: s64 : 9;` in a body	the same error, with `help: change ':' to '=' to make the binding mutable`
`c: s64 :: 9;` in a body	the same error, with `help: change '::' to '=' to make the binding mutable`
`g := 0;` at file scope	Accepted; a global is an ordinary mutable binding.

The refusal reaches through members and indexes, not just through the bare name. For `p :: Point.{ .x = 1, .y = 2 };` the statement `p.x = 5;` is the same `error: constant cannot be modified`, reported at the field, and for `a: [3]s64 :: .[1, 2, 3];` so is `a[0] = 9;`.

`#undefined` initialisers

`x: T = #undefined;` declares a binding whose initial contents are not defined. It exists to skip the zero fill for a buffer that is about to be overwritten in full, which is why the standard library's binary readers open with `bytes: [16]u8 = #undefined;` before reading into the slice. The spelling is the directive; a bare `undefined` is `error: unbound 'undefined'`.

GLOSSO

```
zeroed :: () -> u8 {
    b: [64]u8;
    return b[3];
}

scratch :: () -> u8 {
    b: [64]u8 = #undefined;
    return b[3];
}
```

CONSOLE / OUTPUT

```
define i8 @glo_zeroed() {
entry:
  %r0 = alloca i64, align 8
  %r1 = alloca [64 x i8], align 1
  %r2 = alloca i64, align 8
  %r3 = alloca i8, align 1
  %temp.mark = load i64, ptr @glo_temp_used, align 8
  store i64 %temp.mark, ptr %r0, align 8
  store [64 x i8] zeroinitializer, ptr %r1, align 1
  store i64 3, ptr %r2, align 8

define i8 @glo_scratch() {
entry:
  %r0 = alloca i64, align 8
  %r1 = alloca [64 x i8], align 1
  %r2 = alloca i64, align 8
  %r3 = alloca i8, align 1
  %temp.mark = load i64, ptr @glo_temp_used, align 8
  store i64 %temp.mark, ptr %r0, align 8
  store i64 3, ptr %r2, align 8
```

Under `--backend llvm-ir -O0` the two bodies differ by exactly one instruction: `zeroed` emits `store [64 x i8] zeroinitializer, ptr %r1` and `scratch` does not. `#undefined` has no standalone type of its own. It takes the type of the declaration, parameter, or field that receives it; bound alone with `x := #undefined` it falls back to the `label` type used for an absent jump target.

> **NOTE**
> `#undefined` changes nothing at file scope. A global lives in the program image and has to be given some pattern of bytes, so `buffer: [8]u8 = #undefined;` still emits `@glo_global_buffer = internal global [8 x i8] zeroinitializer`. The directive only pays for itself on a local.

3.2 Top-Level Declarations and Thread-Local Storage

A Glosso source file is a flat sequence of top-level declarations. There is no package statement, no header, and no forward-declaration form, because there is nothing to declare forward: the whole file is read before anything in it is checked.

Order is therefore free. `main` may be the first declaration in the file and call a procedure written below it, a global may be typed by a struct declared later, and `#import` and `#load` may sit at the very bottom.

GLOSSO

```
main :: () {
    print("later() = %\n", later());
    print("LIMIT = % origin.x = %\n", LIMIT, origin.x);
}

LIMIT :: 64;
Point :: struct { x: ssize; y: ssize; }
origin: Point = .{ .x = 3, .y = 4 };
later :: () -> ssize { return LIMIT / 2; }

#import "Io";
```

CONSOLE / OUTPUT

```
later() = 32
LIMIT = 64 origin.x = 3
```

What may appear at file scope

Form	Declares
`name :: expr;`	A constant.
`name : T : expr;`, `name : T :: expr;`	A typed constant.
`name := expr;`, `name ::= expr;`	A mutable global, type inferred, when the right side is an ordinary expression.
`name : T = expr;`, `name : T := expr;`	A mutable global with a written type.
`name : T;`	A zero-filled global.
`name :: (params) -> R { … }`	A procedure.
`name :: struct { … }`, `union { … }`, `enum { … }`, `enum_flags T { … }`	An aggregate type.
`name :: distinct T;`	A nominal type.
`name :: typeclass (…) { … }`	A typeclass.
`head ::: instance Class { … }`	An instance; `:::` is its dedicated separator.
`name := definition`, `name ::= definition`	A named type, procedure, namespace, library, or macro with its declaration type inferred.
`name : D : definition`, `name : D :: definition`	The same named definitions with declaration-value type `D` checked explicitly.
`name :: #fn_ptr(…) -> R;`	A function-pointer type.
`name :: #library[,kind] "file";`	A foreign library handle.
`#import "M";`, `name :: #import "M";`	A module import.
`#load "path";`, `name :: #load "path";`	An isolated relative module; use `#load,embed` for a shared source fragment.
`#thread_local <global declaration>`	A thread-local global.
`#comptime { … }`, `#comptime call();`	A compile-time step; the expression form requires `;`.
`#insert <Code-or-string expression>;`	Declarations generated during checking.
`#enable(feature);`, `#disable(feature);`	A feature gate, sticky to the end of the file.

`#if`, `else #if`, and `else` also select declarations at module scope. The condition must be a compile-time boolean. Only the chosen branch adds declarations or loads dependencies, so an inactive platform branch need not have its files or foreign libraries installed. Custom operator declarations and dependencies that provide operator syntax must stay unconditional, because parsing precedes branch selection. `#assert` remains a statement; use a `#comptime` block for a file-level assertion.

Named definitions are values too

At file scope, a definition may use the traditional `name :: definition` spelling, an inferred `name := definition` or `name ::= definition`, or an explicit declaration-value annotation written `name : D : definition` (with `::` also accepted for the second separator). The parser recognises a definition-shaped right side before deciding that `:=` means a mutable global. Structs, unions, enums, flag enums, distinct types, typeclasses, procedures, imports, loads, libraries, and function-pointer definitions all take this route. Inside a procedure body, only the canonical `name :: definition` form enters the nested definition path; the alternate punctuation above is file-scope syntax.

GLOSSO

```
Point : type : struct { x: ssize; y: ssize; }
double : (ssize) -> ssize : (value: ssize) -> ssize { return value * 2; }
Io : Namespace : #import "Io";
kernel32 : Library : #library,system "kernel32";
Show_One : Typeclass(type) : typeclass(T: type) {
    show_one :: (value: T) -> string;
}
trace : Macro((ssize) -> void) : (value: ssize) #expand { _ := value; }
Auto_Point := struct { x: ssize; }
auto_main ::= () {}
```

The annotation describes the value introduced by the declaration, not the values stored inside it. An aggregate, distinct type, or function-pointer definition has type `type`; `#import` and `#load` produce `Namespace`; `#library` produces `Library`; a typeclass uses the parameterized declaration annotation `Typeclass(parameter-kinds...)`; a macro uses `Macro((parameter-types...) -> result)`; and an ordinary procedure has its procedure type. `Typeclass(...)` and `Macro(...)` are recognized in this declaration-annotation position; they are not generally callable type constructors in ordinary expressions. These names are case-sensitive. The former lowercase `namespace`, `library`, and `pattern` spellings are ordinary identifiers, not builtin types.

> **NOTE**
> The checker verifies an explicit declaration-value annotation. For example, `Wrong : Library : struct {}` reports `declaration 'Wrong' has type type, not declared type Library`. In an explicit declaration annotation, `Typeclass` requires at least one parameter kind and `Macro` requires exactly one procedure signature, such as `Macro((ssize) -> void)`.

Procedures and types may also be declared inside a procedure body. Their names belong to the containing lexical block; other top-level declarations cannot reach them. Separate blocks may use the same local name. Function Declarations covers local procedures and their capture rules.

Globals and their initialisers

A global is a mutable binding with static storage. Its name is private by default; use `#public` or `#public(unit)` when another source file needs it. Its initializer is evaluated during compilation and emitted as the symbol's static initial value. Visibility and Ownership Units covers which callers can reach the binding.

That is a hard requirement, not an optimisation. An initialiser the checker cannot fold is an error, and the most common way to hit it is to read another global. A global variable is not a compile-time value, so `second: s64 = first + 1;` gives `error: unknown const first`. Anything that needs the running program is rejected the same way: `buffer: ?*void = alloc(64);` is `error: #magic intrinsic 'ctx_alloc' is not available during compile-time evaluation`.

Anything the compile-time evaluator can run is allowed, including calls to procedures declared later in the file and constants declared later in the file.

GLOSSO

```
compute :: () -> s64 { return 6 * 7; }

answer: s64 = compute();
table: [4]s64 = .[1, 2, 3, 4];
blank: s64;

main :: () { x := answer + table[0] + blank; }
```

CONSOLE / OUTPUT

```
@glo_global_answer = internal global i64 42
@glo_global_blank = internal global i64 0
@glo_global_table = internal global [4 x i64] [i64 1, i64 2, i64 3, i64 4]
```

Initialisation order

Because every global initial value is a folded constant, there is no runtime global-initialisation pass at all. Nothing runs before `main` to fill globals in, so there is no initialisation order to specify and no order-of-initialisation hazard between files. The guarantee the compiler makes is that every global holds its declared initial value, or zero, from the first instruction of the program.

The visible consequence is that a side effect in a global initialiser happens during the build, not during the run.

GLOSSO

```
#import "Io";

noisy :: () -> s64 { print("initializer ran\n"); return 1; }

value: s64 = noisy();

main :: () { print("value=%\n", value); }
```

CONSOLE / OUTPUT

```
glosso: compile program.glo (target=x86_64-pc-windows-msvc)
initializer ran
glosso: link program.exe (target=x86_64-pc-windows-msvc, -O1)
glosso: done program.exe  (backend=llvm, target=x86_64-pc-windows-msvc, -O1)
value=1
```

NOTE

`initializer ran` is printed by the compiler, on the compiler's stdout, between the `compile` and `link` progress lines. Work that has to happen once at program start belongs at the top of `main` or behind an explicit initialisation procedure, not in a global initialiser.

Thread-local storage

`#thread_local` prefixes a global variable declaration and gives each thread its own copy of the storage. Everything else about the declaration is unchanged: the initialiser is still folded at compile time, and an omitted initialiser still means zero.

GLOSSO

```
#import "Io";

#thread_local depth: s64 = 0;
#thread_local scratch: [4]u8;

main :: () {
    depth += 1;
    scratch[0] = 7;
    print("depth=% scratch[0]=%\n", depth, scratch[0]);
}
```

CONSOLE / OUTPUT

```
depth=1 scratch[0]=7
```

The directive prefixes a variable and nothing else. `#thread_local K :: 5;` and `#thread_local f :: () { }` are both `error: #thread_local can only prefix a global variable declaration`, and inside a body it is not a statement at all: `error: unexpected directive 'thread_local'`.

NOTE

Whether `#thread_local` reaches the object file depends on the target. On a hosted target the global is emitted as `@glo_global_depth = internal thread_local global i64 0`; on Windows, on freestanding Linux x86-64, and on bare metal the backend drops the `thread_local` qualifier and emits an ordinary internal global, because those targets do not use the ELF thread-local model the runtime is built around. Targets and Cross-Compilation covers the target list.

`Type ::: [#public] { ... }`	An inherent method or associated-function block.
`#public`, `#public(unit)`, `#private`	A declaration visibility prefix or transparent group.
`#if condition { ... } else { ... }`	Compile-time selection of module declarations.

3.3 Assignment and Update Assignment

`place = value;` is a statement, not an expression. It has no result and it cannot be nested: `y := (x = 2);` is `error: unexpected token '='`, and so is `if x = 1 { }`. An assignment written where a comparison was meant is a parse error rather than a silent truth test. The update-assignment operators described below are a separate mechanism and do produce values.

The type rule is the ordinary conversion rule, with no relaxation for assignment. The right side must already be, or implicitly convert to, the type of the left side. Widening between integer types is implicit; narrowing is not, so `small: s32 = 1; big: s64 = 2; small = big;` is `error: assign s64 to s32`. An unrelated type is the same shape of message: assigning a `string` to an `s32` is `error: assign string to s32`. Conversions: What Is Implicit gives the full rule, and Relaxing Strictness with `lossy_numerics` gives the gate that turns the narrowing case back on.

What can be assigned to

The left side must denote storage. Four forms do: a mutable name, a field of one, an element of one, and the target of a pointer dereference. They compose, and an element reached through a view writes through to the array the view was taken from.

GLOSSO

```
#import "Io";

main :: () {
    arr: [4]s64 = .[1, 2, 3, 4];
    arr[1] += 10;

    x := 5;
    p := &x;
    p.* += 2;

    view := arr[0..2];
    view[0] += 100;

    print("arr = % % % %   x = %\n", arr[0], arr[1], arr[2], arr[3], x);
}
```

CONSOLE / OUTPUT

```
arr = 101 12 3 4   x = 7
```

Anything else is `error: left side of assignment is not assignable`. That covers a literal (`5 = 3;`), the result of a call (`f() = 3;`), and any computed expression (`(a + b) = 3;`). Constants get their own diagnostics, listed in Values, Constants, and Variables. Parameters, by contrast, are ordinary mutable bindings and may be assigned to; the caller's value is unaffected because arguments are passed by value.

Assignment copies

Assignment of an aggregate copies it. A struct or fixed array bound from another one is an independent value, and writing through the copy does not disturb the original. Views and pointers are the exception, and only because the thing being copied is the reference, not what it refers to.

GLOSSO

```
#import "Io";

Point :: struct { x: ssize; y: ssize; }

main :: () {
    a: Point = .{ .x = 1, .y = 2 };
    b := a;
    b.x = 99;
    print("a.x=% b.x=%\n", a.x, b.x);

    c: [3]s64 = .[1, 2, 3];
    d := c;
    d[0] = 99;
    print("c[0]=% d[0]=%\n", c[0], d[0]);
}
```

CONSOLE / OUTPUT

```
a.x=1 b.x=99
c[0]=1 d[0]=99
```

Update assignment

Every `op=` spelling is a genuine operator, declared in the Prelude as a typeclass method carrying `#operator(assign, 0)`. Level 0 is the loosest tier on the precedence scale, so everything on the right of it groups first, and the result is stored back into the left operand. Because it is an operator rather than statement syntax, `y := (x += 1);` compiles and binds `y` to the updated value.

GLOSSO

```
#import "Io";

main :: () {
    total := 10;
    total += 3;
    total <<= 2;

    saturated: u8 = 250;
    saturated +|= 10;

    wrapped: u8 = 250;
    wrapped +%= 10;
```

```
    ready := false;
    ready ||= total > 0;

    print("total=% saturated=% wrapped=% ready=%\n",
          total, saturated, wrapped, ready);
}
```

CONSOLE / OUTPUT

```
total=52 saturated=255 wrapped=4 ready=1
```

There are nineteen of them, and each has its own two-parameter typeclass with an `Output` associated type. None of those classes has to be implemented directly: the Prelude ships a blanket instance for every one, conditioned on the corresponding binary operator class, so a type that implements `Add` gets `+=` for free.

Operator	Typeclass	Blanket instance requires
`+=`	`AddAssign`	`Add(L, R)`
`-=`	`SubAssign`	`Sub(L, R)`
`*=`	`MulAssign`	`Mul(L, R)`
`/=`	`DivAssign`	`Div(L, R)`
`%=`	`RemAssign`	`Rem(L, R)`
`+%=`	`WrapAddAssign`	`WrapAdd(L, R)`
`-%=`	`WrapSubAssign`	`WrapSub(L, R)`
`*%=`	`WrapMulAssign`	`WrapMul(L, R)`
`+|=`	`SaturatingAddAssign`	`SaturatingAdd(L, R)`
`-|=`	`SaturatingSubAssign`	`SaturatingSub(L, R)`
`*|=`	`SaturatingMulAssign`	`SaturatingMul(L, R)`
`<<=`	`ShiftLeftAssign`	`ShiftLeft(L, R)`
`<<|=`	`SaturatingShiftLeftAssign`	`SaturatingShiftLeft(L, R)`
`>>=`	`ShiftRightAssign`	`ShiftRight(L, R)`
`&=`	`BitAndAssign`	`BitAnd(L, R)`
`|=`	`BitOrAssign`	`BitOr(L, R)`
`^=`	`BitXorAssign`	`BitXor(L, R)`
`&&=`	`LogicalAndAssign`	`LogicalAnd(L, R)`
`||=`	`LogicalOrAssign`	`LogicalOr(L, R)`

A type with no instance reports the missing operator rather than a type mismatch: `a: Point; b: Point; a += b;` is `error: no operator '+=' for Point, Point`. The Operator System and Assignment Operators cover declaring new ones and what `#operator(assign, 0)` means for parsing.

> **NOTE**
> The wrapping, saturating, and shifting families exist because the plain arithmetic operators trap on overflow at `-O0` and `-O1`. `+%=` wraps and `+|=` clamps, which is why `250 +%= 10` is `4` and `250 +|= 10` is `255` on a `u8`. Overflow, Traps, and the `-O` Level explains when the trap is emitted.

Binding several names at once

A declaration may take a comma-separated list of names on the left, which destructures a multiple-result call or a tuple. `_` in that list discards a component. This is a declaration form, not an assignment form: `::`, `:=` and `::=` all accept a name list.

GLOSSO

```
#import "Io";

divmod :: (a: ssize, b: ssize) -> (ssize, ssize) { return a / b, a % b; }

main :: () {
    q, r := divmod(7, 2);
    print("q=% r=%\n", q, r);

    _, remainder := divmod(9, 4);
```

```
    print("remainder=%\n", remainder);

    pair := .(1, "two");
    n, s := pair;
    print("n=% s=%\n", n, s);
}
```

CONSOLE / OUTPUT

```
q=3 r=1
remainder=1
n=1 s=two
```

There is no multiple-assignment statement. Once the names exist, `a, b = divmod(9, 4);` is `error: unexpected token '='`, and so is the swap idiom `x, y = y, x;`. The right side of a destructuring declaration is one expression, not a list: `a, b := 1, 2;` is `error: unexpected token ','`, and the tuple literal `a, b := .(1, 2);` is the spelling that works.

The name count must match what the right side produces: `a, b, c := divmod(7, 2);` is `error: destructuring binds 3 names from a 2-value result`. Function Declarations covers multiple and named results, and Tuples covers the values themselves.

3.4 Blocks, Scope, and Shadowing

A block is a brace-delimited statement list used as a statement in its own right. It introduces a scope: a name declared inside it is unreachable after the closing brace. Scoping is lexical and sequential, so a name is visible from its declaration to the end of the scope that holds it, and not before.

GLOSSO

```
#import "Io";

main :: () {
    a := 1;
    {
        print("inner sees outer a=%\n", a);
        a := 99;
        print("inner a=%\n", a);
    }
    print("outer a=%\n", a);
}
```

CONSOLE / OUTPUT

```
inner sees outer a=1
inner a=99
outer a=1
```

The first `print` resolves `a` to the outer binding because the inner declaration has not been reached yet, and the inner `a := 99;` declares a second variable rather than assigning to the first, so the outer one is unchanged once the block ends. Naming a binding after its scope has closed is `error: unbound 'hidden'`; in call position the same failure reads `error: unknown function hidden`, because the checker resolves callees through a lookup separate from the one it uses for values.

What introduces a scope

Every construct with a body carries its own scope, and the checker clones the environment when it enters one. A body written without braces is still a body: `if true  a := 5;` followed by a use of `a` on the next line is `error: unbound 'a'`.

Construct	Scope
`{ ... }` as a statement	A fresh scope, closed by the `}`.
A procedure body	A fresh scope; the parameters are declared in it.
An `if` or `else` body	A fresh scope, whether or not it is braced.
A `while` or `for` body	A fresh scope, re-entered on every iteration.
A switch or pattern arm body	A fresh scope per arm, holding that arm's captures.
A `#push_context` or `#push_allocator` body	A fresh scope.
A lambda body	A fresh scope that cannot reach enclosing locals.

Local declarations and lexical scope

A named function or type declared inside a body belongs to its containing lexical block. Other procedures cannot reach that local name. Type aliases can also be used in local annotations. Constants and variables follow the ordinary block scope rules.

GLOSSO
```
#import "Io";

collect :: () {
    Sample :: struct { v: ssize; }
    s: Sample = .{ .v = 3 };
    print("collect %\n", s.v);
}

report :: () {
    Sample :: struct { v: ssize; tag: bool; }
    t: Sample = .{ .v = 9 };
    print("report %\n", t.v);
}

main :: () { collect(); report(); }
```

CONSOLE / OUTPUT
```
collect 3
report 9
```

collect and report each declare their own Sample type. Their names are independent and their fields may differ. A function declared in one block cannot be called from a separate top-level procedure; pass a callback explicitly when that behavior is needed.

> **NOTE**
> A local function or type inside a braced #if branch stays in that lexical block. At module scope, #if selects complete declaration blocks; only the active branch contributes declarations and dependencies.

Captures and pattern bindings

A pattern capture is a binding whose scope is exactly the branch that proved the pattern. It does not exist before the arm and it does not survive it.

GLOSSO
```
#import "Io";

Res :: union { Ok: ssize; Err: string; }

main :: () {
    r := Res.Ok(5);
    if r == {
    case .Ok(v);  print("ok %\n", v);
    case .Err(e); print("err %\n", e);
    }
    if #pattern .Ok(v) = r print("tested v=%\n", v);
}
```

CONSOLE / OUTPUT
```
ok 5
tested v=5
```

Naming `v` after the switch is `error: unbound 'v'`, with `help: did you mean 'r'?`. The `else` of an `if #pattern` test cannot see the captures either: on that path the pattern did not match, so there is nothing for them to name.

The one form that binds outward is `#pattern P := subject else { ... }`, which places the captures in the enclosing scope so a procedure is not indented one level deeper per optional. That is sound only because the `else` block is checked without the bindings and must not fall through; the form is covered in Bind or Diverge.

GLOSSO
```
#import "Io";

Res :: union { Ok: ssize; Err: string; }

take :: (r: Res) -> ssize {
    #pattern .Ok(v) := r else { return -1; }
    return v * 10;
}

main :: () {
    print("%\n", take(Res.Ok(4)));
    print("%\n", take(Res.Err("no")));
}
```

CONSOLE / OUTPUT

```
40
-1
```

`using p;` inserts a struct's field names into the current scope for the remainder of it, and they disappear with the scope: reading `x` after the block that said `using p;` is `error: unbound x`. The statement is covered in `using`, `#empty`, and `#as`.

A lambda body is a scope that deliberately cannot reach outward. Reading an enclosing local gives `error: lambda captures local 'base', but captured closures are not supported`, with `help: pass the value as an explicit callback or context argument`. Globals and constants remain readable; see Lambdas.

Shadowing

Inside a procedure, shadowing is unrestricted. A local may hide a parameter, a global variable, a top-level constant, a type name or a procedure name, and a nested block may hide anything the enclosing block declared.

GLOSSO

```
#import "Io";

LIMIT :: 100;
counter := 10;

scale :: (x: ssize) -> ssize {
    x := x * 2;
    LIMIT := 3;
    counter := 0;
    return x + LIMIT + counter;
}

main :: () {
    print("scale=% LIMIT=% counter=%\n", scale(4), LIMIT, counter);
}
```

CONSOLE / OUTPUT

```
scale=11 LIMIT=100 counter=10
```

Redeclaring a name in the same block is accepted as well: `a := 1; a := 2;` is two declarations rather than a declaration and an assignment, and the second binding wins from that point on. Call position and value position are resolved separately, so shadowing a procedure name does not make the procedure uncallable: after `print := 3;` the call `print("%\n", print);` still resolves and prints `3`.

Shadowing is refused within one file's top-level namespace. A second declaration there is an error, including declarations brought into the same unit by #load,embed. Local functions and types stay in their lexical blocks, so their names do not collide across procedure bodies. Ordinary #load preserves the loaded module's identity. Qualified loads distinguish equally named declarations in separate modules.

Duplicate at file scope	Diagnostic
`K :: 1;` then `K :: 2;`	`error: constant 'K' is already declared`
Two procedures named `helper`	`error: function 'helper' is already declared`
Two structs named `Point`	`error: type 'Point' is already declared`

> **NOTE**
> The standard library's public names occupy that same namespace. A top-level `PI :: 3.14;` in a user file is `error: constant 'PI' is already declared`, because the Prelude already owns the name. Local bindings are free to shadow it.

- Labels are function-scoped rather than block-scoped: they are collected in a pre-pass over the whole body, so a jump may go forward, and may enter a nested block from outside it. See Labels and `goto`.
- A `defer` belongs to the scope it is written in and runs when that scope is left. See `defer` and `#try defer`.
- Visibility groups are transparent and restore the enclosing audience when their braces close. See Visibility and Ownership Units.

3.5 Visibility and Ownership Units

Functions, types, constants, globals, methods, and named module bindings are private by default. A private declaration is accessible only in its defining physical source file and logical module. Struct fields and enum or union variants start public, subject to the visibility of their enclosing type.

Use `#public` to expose an API, `#public(unit)` to share declarations within one ownership unit, and `#private` to make file privacy explicit. Each directive prefixes one named declaration or a transparent group. Groups introduce no namespace and restore the enclosing default when they end.

GLOSSO
```
#public Counter :: struct {
    value: s64;
    #private cached: s64;
}

Counter ::: #public {
    make :: () -> #Self_Type { return .{ .value = 0, .cached = 0 }; }
    read :: (#self) -> s64 { return #self.value; }
    #private reset_cache :: (*mut #self) { #self.*.cached = 0; }
    #public(unit) refresh_cache :: (*mut #self) { #self.*.cached = #self.*.value; }
}

#public(unit) {
    refresh :: () {}
    #private scratch: s64;
}
```

The visibility after `:::` sets an inherent block's default member audience. The header accepts `#public`, `#public(unit)`, or `#private`. `Counter ::: #public { ... }` is equivalent to `Counter ::: { #public { ... } }`. A method without an explicit audience in a plain `Counter ::: { ... }` block remains private, even if the type or surrounding declaration group is public. Put a block visibility before its optional `where` clause, as in `Box($T) ::: #public where true { ... }`.

Visibility on locals or parameters is rejected. An inherent block takes its visibility after `:::`, not before the owner type name. Typeclass requirements and instance slots follow the class contract. A public name beginning with _ is still public; its spelling does not hide it from imports.

The former `#enable(private_section)`, `#disable(private_section)`, `private_sections`, and `siblings` forms have been removed and produce a migration diagnostic. Replace them with declaration prefixes or visibility groups. Other feature gates keep their file-scoped behavior.

Imports, loads, and reexports never widen an item's audience. Each module still declares its own dependencies. `#load,embed` shares a logical module but preserves each physical file's private identity: declarations used by another embedded file need `#public(unit)` or `#public`.

GLOSSO
```
// parts/store.glo
#public STORE_PUBLIC :: 1;
#private key :: 2;
#public(unit) STORE_SHARED :: 3;
#public read_key :: () -> s64 { return key; }

// app.glo
#import "Io";
#load "parts/store.glo";
main :: () {
    print("% % %\n", STORE_PUBLIC, read_key(), STORE_SHARED);
}
```

In this example both files are in the entry file's project directory and therefore share an ownership unit by default. The caller can use `STORE_SHARED` through its explicit load, while `key` remains accessible only inside `store.glo`. Loading a file alone never makes its private declarations callable.

Ownership controls who may add inherent methods and use unit-visible names. It is separate from module scope. Without explicit mappings, the entry file and other files under its source directory form a project unit. A configured `--module-path` directory creates dependency boundaries by its first relative path component. Other external files get separate file units.

Repeated `--unit-source UNIT=FILE` options configure ownership explicitly. Once any mapping is given, every loaded non-standard source, including the entry file, requires an exact mapping. Canonical paths keep their unit across aliases, reexports, and embedded loads. Standard sources belong to the reserved `builtin:std` unit, and names starting with `builtin:` are reserved.

GLOSSO
```
// app and its implementation share a unit; the dependency has its own.
// glosso --unit-source app=app.glo --unit-source app=parts/store.glo app.glo
```

Private fields cannot be read by a method in another file, even in the same unit. An inaccessible field also blocks external aggregate literals, spread construction, and recursive default initialization. Expose a factory such as `Counter.make()` to construct the value. Copying an existing value and updating its accessible fields remain permitted.

Public signatures cannot expose less-visible types. Generic bodies and default expressions retain the access rights of their defining file; specialization does not grant access to the caller's private members. Reflection omits inaccessible fields and variants without changing layout or discriminants. Matching a type with hidden variants requires a catch-all.

NOTE
Language visibility is independent of native symbol export. A defined `#c_call` function can retain its external linker name while remaining private to Glosso callers. An ordinary private `main` is a valid entry point; it does not need a public annotation.

3.6 `defer` and `#try defer`

`defer` schedules a statement to run when the enclosing scope is left. Two forms are accepted: `defer <statement>;` and `defer { ... }`. The statement is not executed where it is written; it is pushed onto the scope's pending list and run on the way out.

GLOSSO
```
#import "Io";

main :: () {
    defer print("end of main\n");
    {
        defer print("A\n");
        defer print("B\n");
        defer { print("C1\n"); print("C2\n"); }
        print("block body\n");
    }
    print("after block\n");
}
```

CONSOLE / OUTPUT
```
block body
C1
C2
B
A
after block
end of main
```

Pending statements run last-in, first-out, so cleanup unwinds in the reverse of the order it was set up, which is the order that matches acquisition. The block form is a single entry: `C1` and `C2` run together, and together they run before `B`.

Each entry belongs to the scope it was written in, not to the procedure. The inner block's three entries run at its closing brace; the entry in `main` waits until `main` returns.

Which exits run the pending statements

GLOSSO
```
#import "Io";

main :: () {
    i := 0;
    while i < 4 {
        defer print("  defer sees i=%\n", i);
        i += 1;
        if i == 2 continue;
        if i == 4 break;
        print("  body i=%\n", i);
    }
    print("done i=%\n", i);
}
```

CONSOLE / OUTPUT
```
  body i=1
  defer sees i=1
  defer sees i=2
  body i=3
  defer sees i=3
  defer sees i=4
done i=4
```

The loop body is a scope, so the entry is registered and discharged once per iteration: four times, including the iteration cut short by `continue` and the one cut short by `break`. The printed value is equally informative. `i` is incremented after the `defer` line, and the trace shows the incremented value, because a deferred statement is evaluated when it runs and not when it is written.

Leaving the scope by	Pending statements
Reaching the closing brace	Run.
`return`	Run, for every scope between the `return` and the procedure body.

Leaving the scope by	Pending statements
`break` or `continue`	Run, for every scope up to and including the loop body.
The end of a loop iteration	Run, once per iteration.
`goto`	Not run.
`exit()`	Not run; the process ends inside the call.

`goto` transfers to a label rather than unwinding scopes, so it steps over the pending list entirely. That is why `defer` cannot carry cleanup in a procedure that uses the jump library for control transfer; see Explicit Control Transfer with the Jump Library.

GLOSSO

```
#import "Io";
#import "Jump";

main :: () {
    {
        defer print("inner defer\n");
        print("before goto\n");
        goto('out);
    }
    'out;
    print("after goto\n");
}
```

CONSOLE / OUTPUT

```
before goto
after goto
```

NOTE

Pending statements run before the caller observes the result, and a result named by a bare local is read after they have run. In `by_name` below the caller sees `99`; writing `return x + 0;` materializes a temporary first and yields `1`. Do not assign in a deferred statement to a local that the same scope returns by name.

GLOSSO

```
#import "Io";

by_name  :: () -> ssize { x := 1; defer x = 99; return x; }
by_value :: () -> ssize { x := 1; defer x = 99; return x + 0; }

main :: () { print("by_name=% by_value=%\n", by_name(), by_value()); }
```

CONSOLE / OUTPUT

```
by_name=99 by_value=1
```

`#try defer`

`#try defer <statement>` registers an entry that runs only when the scope is left along an error path. It is the rollback half of a two-part cleanup: `defer` releases what was acquired either way, and `#try defer` undoes work that only a successful completion should keep.

An error path means propagation through postfix `?`, so the form requires a boundary that `?` can return through. Without one it is `error: #try defer requires a Try-returning function or an enclosing #try block`. Propagation itself is covered in `Try`, `?`, and `#try`.

GLOSSO

```
#import "Io";

step :: (fail: bool) -> string!ssize {
    if fail return .Err("step failed");
    return .Ok(1);
}

run :: (fail: bool) -> string!ssize {
    print("  acquire\n");
    #try defer print("  rollback\n");
    defer print("  release\n");
    v := step(fail)?;
    print("  commit\n");
    return .Ok(v + 10);
}
```

```
report :: (r: string!ssize) {
    if r == { case .Ok(v); print("  -> ok %\n", v); case .Err(e); print("  -> err %\n", e); }
}

main :: () {
    print("success path:\n");
    report(run(false));
    print("failure path:\n");
    report(run(true));
}
```

CONSOLE / OUTPUT

```
success path:
  acquire
  commit
  release
  -> ok 11
failure path:
  acquire
  release
  rollback
  -> err step failed
```

On the success path `run` reaches `commit`, the plain entry runs, and the rollback is discarded unexecuted. On the failure path `step` returns `.Err`, the `?` carries it out of `run` without reaching `commit`, and both entries run. The order is worth reading: the rollback was registered first, so it runs last.

Both kinds share one last-in, first-out list. An error-only entry is skipped on the success path but keeps its position, so the interleaved order of the two kinds is the reverse of the order they were written in.

GLOSSO

```
#import "Io";

step :: (fail: bool) -> string!ssize {
    if fail return .Err("boom");
    return .Ok(1);
}

run :: (fail: bool) -> string!ssize {
    #try defer { print("  undo 1\n"); }
    defer print("  always 1\n");
    #try defer print("  undo 2\n");
    defer print("  always 2\n");
    v := step(fail)?;
    return .Ok(v);
}

main :: () {
    print("ok:\n");
    _ := run(false);
    print("err:\n");
    _ := run(true);
}
```

CONSOLE / OUTPUT

```
ok:
  always 2
  always 1
err:
  always 2
  undo 2
  always 1
  undo 1
```

NOTE

Only propagation through `?` counts as an error path. A hand-written `return .Err(...);` is an ordinary return: the plain entries run and the `#try defer` entries do not. A procedure that mixes explicit error returns with `#try defer` rollback silently skips the rollback on exactly the paths it was written for.

GLOSSO

```
#import "Io";

run :: (fail: bool) -> string!ssize {
    #try defer print("  rollback\n");
    defer print("  release\n");
    if fail return .Err("explicit");
    return .Ok(1);
}

main :: () {
```

```
    print("explicit .Err:\n");
    _ := run(true);
}
```

CONSOLE / OUTPUT

```
explicit .Err:
  release
```

The boundary may be any `Try`-returning result, including the option type: in a procedure returning `?ssize`, a `?` applied to a `.None` runs the error-only entries exactly as an `.Err` does.

GLOSSO

```
#import "Io";
#import "Context";

Fault :: enum { Bad; }

work :: (fail: bool) -> Fault!ssize {
    p := alloc(64);
    #try defer p.free();
    if fail return .Err(.Bad);
    p.free();
    return .Ok(1);
}

main :: () { _ := work(false); }
```

The warning is accurate here. `return .Err(.Bad);` is not a `?` propagation, so `#try defer p.free()` never runs on that path and the allocation really does leak. Replacing it with a plain `defer p.free();` and dropping the later `p.free();` makes the procedure correct and the warning disappear. Release obligations are covered in Ownership and Release Obligations.

- `defer` accepts any statement, including a call, an assignment or a block; the block form counts as one entry and its statements run in written order.
- A deferred statement reads its operands at the moment it runs, so it observes every mutation made after it was written.
- A `#try defer` written inside a nested block belongs to that block. It is discarded when the block is left normally, so a failure that happens after the block does not run it.

4 Primitive types

4.1 The Primitive Types

A primitive type is one the compiler knows without a declaration. The spellings below are recognised directly, and the checker also accepts the fixed C integer names, the arbitrary-width families `sN` and `uN`, and the primitive type constructors `Simd` and `Matrix`. In the integer families `N` is an unpadded decimal width from 1 through 65535. Everything else in the language is assembled out of these primitives.

A primitive spelling is not a keyword. The lexer produces an ordinary identifier and the name is resolved during checking, after locals and globals have been consulted. A binding therefore shadows a primitive: `ssize := 5;` declares a local named `ssize`, and `Pattern := 5;` declares one named `Pattern`. The lowercase name `pattern` is just an ordinary identifier; the compile-time pattern type is spelled with a capital `P`.

Family	Spellings	Detailed in
Integers	`s8 s16 s32 s64 s128 u8 u16 u32 u64 u128 ssize usize`	Integers
Arbitrary-width integers	`sN uN`, with decimal `N` from 1 through 65535	Integers
Unicode scalars	`rune`	Character Types and `#char` Character Literals
Floating point	`f16 f32 f64 f80 f128`	Floating Point
Complex	`c32 c64 c128 c160 c256`	Complex Numbers
SIMD vector	`Simd(T, N)`	SIMD and Matrices
Matrix	`Matrix(T, Rows, Cols)`	SIMD and Matrices
Boolean	`bool`	Booleans and Truthiness
Strings	`string cstring string16 cstring16`	Built-in Strings
Definition declarations	`Namespace Library Typeclass(...) Macro(...)`	this chapter
Other	`void #Never type any label Code Pattern`	this chapter

Sizes and alignments

Both numbers come from `type_info(T)`, whose result carries `.size` and `.alignment`. There is no separate alignment intrinsic: `align_of(s64)` is `error: unknown function align_of`. The figures below are for `x86_64-pc-windows-msvc`. Pointer-derived sizes change with the target, and scalar alignment can change too: never infer a portable record layout from this host table.

```
GLOSSO
#import "Io";
#import "Meta";

row :: (name: string, size: ssize, alignment: ssize) {
    print("%   size %   align %\n", name, size, alignment);
}

// Resolve compile-time-only type metadata before runtime code.
type_size :: type_info(type).size;
type_alignment :: type_info(type).alignment;
Code_size :: type_info(Code).size;
Code_alignment :: type_info(Code).alignment;
Pattern_size :: type_info(Pattern).size;
Pattern_alignment :: type_info(Pattern).alignment;
Namespace_size :: type_info(Namespace).size;
Namespace_alignment :: type_info(Namespace).alignment;
Library_size :: type_info(Library).size;
Library_alignment :: type_info(Library).alignment;

main :: () {
    row("s8", type_info(s8).size, type_info(s8).alignment);
    row("s16", type_info(s16).size, type_info(s16).alignment);
    row("s32", type_info(s32).size, type_info(s32).alignment);
    row("s64", type_info(s64).size, type_info(s64).alignment);
    row("s128", type_info(s128).size, type_info(s128).alignment);
    row("ssize", type_info(ssize).size, type_info(ssize).alignment);
    row("u8", type_info(u8).size, type_info(u8).alignment);
    row("u16", type_info(u16).size, type_info(u16).alignment);
    row("u32", type_info(u32).size, type_info(u32).alignment);
    row("u64", type_info(u64).size, type_info(u64).alignment);
```

```
    row("u128", type_info(u128).size, type_info(u128).alignment);
    row("usize", type_info(usize).size, type_info(usize).alignment);
    row("rune", type_info(rune).size, type_info(rune).alignment);
    row("f16", type_info(f16).size, type_info(f16).alignment);
    row("f32", type_info(f32).size, type_info(f32).alignment);
    row("f64", type_info(f64).size, type_info(f64).alignment);
    row("f80", type_info(f80).size, type_info(f80).alignment);
    row("f128", type_info(f128).size, type_info(f128).alignment);
    row("c32", type_info(c32).size, type_info(c32).alignment);
    row("c64", type_info(c64).size, type_info(c64).alignment);
    row("c128", type_info(c128).size, type_info(c128).alignment);
    row("c160", type_info(c160).size, type_info(c160).alignment);
    row("c256", type_info(c256).size, type_info(c256).alignment);
    row("bool", type_info(bool).size, type_info(bool).alignment);
    row("string", type_info(string).size, type_info(string).alignment);
    row("cstring", type_info(cstring).size, type_info(cstring).alignment);
    row("string16", type_info(string16).size, type_info(string16).alignment);
    row("cstring16", type_info(cstring16).size, type_info(cstring16).alignment);
    row("void", type_info(void).size, type_info(void).alignment);
    row("type", type_size, type_alignment);
    row("any", type_info(any).size, type_info(any).alignment);
    row("label", type_info(label).size, type_info(label).alignment);
    row("Code", Code_size, Code_alignment);
    row("Pattern", Pattern_size, Pattern_alignment);
    row("Namespace", Namespace_size, Namespace_alignment);
    row("Library", Library_size, Library_alignment);
}
```

CONSOLE / OUTPUT

```
s8   size 1   align 1
s16   size 2   align 2
s32   size 4   align 4
s64   size 8   align 8
s128   size 16   align 16
ssize   size 8   align 8
u8   size 1   align 1
u16   size 2   align 2
u32   size 4   align 4
u64   size 8   align 8
u128   size 16   align 16
usize   size 8   align 8
rune   size 4   align 4
f16   size 2   align 2
f32   size 4   align 4
f64   size 8   align 8
f80   size 16   align 16
f128   size 16   align 16
c32   size 4   align 2
c64   size 8   align 4
c128   size 16   align 8
c160   size 32   align 16
c256   size 32   align 16
bool   size 1   align 1
string   size 16   align 8
cstring   size 8   align 8
string16   size 16   align 8
cstring16   size 8   align 8
void   size 0   align 1
type   size 0   align 1
any   size 16   align 8
label   size 8   align 8
Code   size 0   align 1
Pattern   size 0   align 1
Namespace   size 0   align 1
Library   size 0   align 1
```

- Size equals alignment for the listed integers and floats on this target. That is not a language-wide identity: on `x86-linux-musl`, `f64` is size 8 / alignment 4; on Linux x86, `f80` is 12 / 4; wide integers on 32-bit Arm, MIPS, and PowerPC have alignment 8 rather than 16.
- `s128`, `u128`, `f80` and `f128` all occupy sixteen bytes. `f80` is the x87 80-bit extended format, with a 64-bit significand; its in-memory representation is padded to sixteen bytes.
- A complex type is aligned to its component, not to the pair, so `c32` is four bytes with alignment two and `c128` is sixteen bytes with alignment eight.
- `void`, `type`, `Code`, `Pattern`, `Namespace`, and `Library` are zero-sized. The annotation-only `Typeclass(...)` and `Macro(...)` declaration types have no runtime representation. `any` is two pointers wide and `label` is one.

Native-size integers and character types

`ssize` and `usize` are target-sized signed and unsigned integers. They are 64 bits on x86-64 and wasm64, and 32 bits on 32-bit targets. They remain distinct type values from fixed-width `s32`, `s64`, `u32`, and `u64`, even when their layouts agree. An unconstrained integer literal defaults to `ssize`; counts, indices, tuple counts, string counts, capacities, and `it_index` use it as well. The old source spellings `isize`, `int`, `uint`, and `char` are not accepted type names.

GLOSSO

```
#import "Io";
#import "Meta";

main :: () {
    print("type_name(ssize)  %\n", type_name(ssize));
    print("type_name(s64)  %\n", type_name(s64));
    print("type_name(usize) %\n", type_name(usize));
    print("type_name(u64)   %\n", type_name(u64));
    print("ssize == s64      %\n", ssize == s64);
    print("usize == u64     %\n", usize == u64);
    print("s64 == int64_t    %\n", s64 == int64_t);
    print("ASCII literal == u8 %\n", type_of(#char "A") == u8);
    print("rune == u32       %\n", rune == u32);
}
```

CONSOLE / OUTPUT

```
type_name(ssize)  ssize
type_name(s64)  s64
type_name(usize) usize
type_name(u64)   u64
ssize == s64      0
usize == u64     0
s64 == int64_t    1
ASCII literal == u8 1
rune == u32       0
```

NOTE

The fixed C spellings are normalized to their corresponding fixed Glosso types, so `int64_t` and `s64` are one type value. Native-size spellings are different: diagnostics preserve `ssize`, `usize`, `s64`, and `u64`. `rune` likewise retains its identity instead of becoming its `u32` storage type. ASCII character literals default to ordinary `u8`.

Bytes and `rune`

`u8` stores bytes and is the default type of an ASCII `#char` literal. It has the ordinary integer operations. `rune` stores a Unicode scalar in four unsigned bytes and has equality, total-equality, ordering, and `Show` instances. A rune is not an integer: `Is_Integer` and `Is_Numeric` return false, and arithmetic, bitwise operations, shifts, integer ranges, and `Simd(rune, N)` are rejected.

An ASCII `#char` literal may adopt `u8`, `s8`, or `rune`; a non-ASCII literal may adopt only `rune`. A stored byte does not convert implicitly to a rune. Use `c.char_to_rune()` for the safe widening, while `cast` is the explicit low-level bridge to integer storage. The literal rules and exact diagnostics are in `#char` Character Literals.

The accepted C spellings

These names exist because Bindgen emits them. They are resolved by the checker rather than listed as builtin names in the parser, which is why they can appear in a type annotation without being reserved words. Each is a named integer type of the stated width and converts to its Glosso counterpart implicitly, both being the same width and signedness.

C spelling	Width and signedness	Prints as
`int8_t int16_t int32_t int64_t`	signed 8, 16, 32, 64	`s8 s16 s32 s64`
`__int128`	signed 128	`s128`
`uint8_t uint16_t uint32_t uint64_t`	unsigned 8, 16, 32, 64	`u8 u16 u32 u64`
`unsigned __int128`	unsigned 128	`u128`

The last row has no source spelling: the name contains a space, so it is reachable only from inside the compiler. Write `u128`.

GLOSSO

```
#import "Io";
#import "Meta";

main :: () {
    a: int8_t = -1;
    b: uint32_t = 7;
    c: __int128 = 170141183460469231731687303715884105727;
    print("int8_t   % %\n", type_name(int8_t), a);
    print("uint32_t % %\n", type_name(uint32_t), b);
    print("__int128 % %\n", type_name(__int128), c);
}
```

CONSOLE / OUTPUT
```
int8_t   s8 -1
uint32_t u32 7
__int128 s128 170141183460469231731687303715884105727
```

`void`, `type`, `any`, `label`, `Code`, `Pattern`, and definition types

`void` is the no-value type. It has exactly one value, written `.()`, and zero bytes. A procedure with no return annotation returns it, and a union variant with no payload carries it. `(void, void)` has `PartialEq` and `PartialOrd` instances that always answer equal.

GLOSSO
```
#import "Io";
#import "Meta";

main :: () {
    a: void = .();
    b: void = .();
    print("void == void  %\n", a == b);
    print("size of void  %\n", size_of(void));
    print("show of void  %\n", show(a));
}
```

CONSOLE / OUTPUT
```
void == void  1
size of void  0
show of void  .()
```

Spelling	Size	What it holds
`void`	0	the single value `.()`
`type`	0	a type, as a first-class value
`any`	2 pointers	any value, boxed with its type
`label`	1 pointer	a jump target within a procedure
`Code`	0	an unexpanded fragment of program text
`Pattern`	0	a match pattern as a compile-time value
`Namespace`	0	a namespace produced by `#import` or `#load`
`Library`	0	a native library declaration
`Typeclass(Kinds...)`	no runtime representation	annotation-only type of a typeclass declaration
`Macro(Signature)`	no runtime representation	annotation-only type of a macro declaration

`any` is the boxing target: the coercion rule accepts every type into it, which is how variadic arguments are collected. A compile-time number takes its default type on the way in. `type` values should be bound as compile-time constants for reflection and formatting; a mutable local is not a compile-time constant; `Types as Values` covers that boundary. `Code`, `Pattern`, `Namespace`, `Library`, `Typeclass(...)` and `Macro(...)` exist only during compilation. `type_kind` reports `0` for the four ordinary type expressions in that list because `Type_Info_Tag` has no dedicated tags for them; the two parameterised annotation-only forms have the same internal fallback.

GLOSSO
```
Foo : type : struct { x: ssize; }
main : () -> void : () {}
IO : Namespace : #import "Io";
Loaded : Namespace : #load "foo.glo";
kernel32 : Library : #library,system "kernel32";
Showable : Typeclass(type) : typeclass(T: type) {}
trace : Macro((ssize) -> void) : (x: ssize) #expand {}
```

These annotations describe the compile-time value produced by each definition. Aggregate definitions have type `type`; an ordinary procedure definition has its function type; imported and loaded modules produce `Namespace`; `#library` produces `Library`; and typeclass and macro definitions carry parameterised declaration types. In this role, `Typeclass` requires at least

one parameter-kind argument and `Macro` takes exactly one function-signature argument. These two forms are accepted in declaration type annotations; they are not callable type constructors or general type-value expressions. The checker validates the annotation against the definition it introduces.

> **NOTE**
> A `void` value can be passed directly to `print` and renders as `.()`. It is also supported as a zero-sized struct, tuple, or array member; the members after it keep their declaration-order indexes.

`#Never` is the uninhabited type: no expression can produce a value of that type and complete normally. Declare a nonreturning function with `-> #Never`; the old `#noreturn` directive is rejected. `#Never!T` is an infallible result, whose `?` needs no fallible enclosing boundary. It is also the `Drop.Error` type for cleanup that cannot fail.

4.2 Integers

Glosso has signed and unsigned integer types for every width from one through 65535 bits. Write `sN` for a signed N-bit type and `uN` for an unsigned one. The fixed spellings below are the conventional widths. `ssize` and `usize` instead follow the target's pointer width and retain identities distinct from the fixed-width spellings. Integer arithmetic is homogeneous, so two operands must already share a type before an operator will accept them.

Spelling	Range	Size
`s8`	-128..=127	1
`s16`	-32768..=32767	2
`s32`	-2147483648..=2147483647	4
`s64`	-9223372036854775808..=9223372036854775807	8
`s128`	-170141183460469231731687303715884105728..=170141183460469231731687303715884105727	16
`u8`	0..=255	1
`u16`	0..=65535	2
`u32`	0..=4294967295	4
`u64`	0..=18446744073709551615	8
`u128`	0..=340282366920938463463374607431768211455	16
`ssize`	target signed pointer-width range	4 or 8
`usize`	target unsigned pointer-width range	4 or 8

Arbitrary-width integers

In `sN` and `uN`, `N` must be an unpadded decimal integer from 1 through 65535. Thus `s7` is a signed seven-bit integer and `u3` an unsigned three-bit integer. `s0`, `u65536`, and `s08` are not type spellings. The fixed-width entries above are the convenient names with established machine and C-ABI representations.

The ranges above are the compiler's own: a written literal that leaves them is rejected with the range quoted back, as in `error: 300 does not fit u8, whose range is 0..=255`. The rule and its exceptions belong to `Compile-Time Numbers and Literal Defaulting`.

GLOSSO

```
#import "Io";

main :: () {
    s8lo:  s8   = -128;                    s8hi:  s8   = 127;
    s64lo: s64  = -9223372036854775808;    s64hi: s64  = 9223372036854775807;
    u8hi:  u8   = 255;                     u64hi: u64  = 18446744073709551615;
    print("s8   % .. %\n", s8lo, s8hi);
    print("s64  % .. %\n", s64lo, s64hi);
    print("u8   0 .. %\n", u8hi);
    print("u64  0 .. %\n", u64hi);
}
```

CONSOLE / OUTPUT

```
s8   -128 .. 127
s64  -9223372036854775808 .. 9223372036854775807
u8   0 .. 255
u64  0 .. 18446744073709551615
```

Two's complement

Every integer is a two's-complement bit pattern, at compile time and at run time alike. The compile-time evaluator carries values in a 128-bit signed payload and re-widens at each operation, so every width up to 128 bits computes exactly. A cast between a signed and an unsigned type of the same width therefore reinterprets rather than converts, and `~0` is all ones at every width.

GLOSSO

```
#import "Io";

main :: () {
    hi: s8 = 127;
    one: s8 = 1;
    print("s8 127 wrap-add 1  = %\n", hi +% one);
    lo: s8 = -128;
    print("s8 -128 wrap-sub 1 = %\n", lo -% one);
    print("s8 -128 as u8 = %\n", cast(u8) lo);
    print("u8 255 as s8  = %\n", cast(s8) cast(u8) 255);
    z: s8 = 0;
    print("~0 at s8      = %\n", ~z);
    zu: u8 = 0;
    print("~0 at u8      = %\n", ~zu);
}
```

CONSOLE / OUTPUT

```
s8 127 wrap-add 1  = -128
s8 -128 wrap-sub 1 = 127
s8 -128 as u8 = 128
u8 255 as s8  = -1
~0 at s8      = -1
~0 at u8      = 255
```

Plain `+`, `-`, `*` and unary `-` do not wrap silently: they are checked at `-O0` and `-O1` and abort on overflow. `+%` and `-%`, used above, are the operators that ask for the wrap explicitly; `*%` completes that family and `+|`, `-|`, `*|` saturate instead. See `Overflow, Traps, and the -O Level` for the checks and `The Operator System` for the full family.

Division and remainder

Both truncate toward zero. The quotient of a negative dividend is rounded up toward zero rather than down, and the remainder takes the sign of the dividend, so `(a / b) * b + (a % b)` reproduces `a` for every pair that does not overflow.

GLOSSO

```
#import "Io";

main :: () {
    a: s64 = 7;   b: s64 = 2;
    c: s64 = -7;  d: s64 = -2;
    print(" 7 / 2 = %   7 rem 2 = %\n", a / b, a % b);
    print("-7 / 2 = %  -7 rem 2 = %\n", c / b, c % b);
    print(" 7 / -2 = %   7 rem -2 = %\n", a / d, a % d);
    print("-7 / -2 = %  -7 rem -2 = %\n", c / d, c % d);
    print("quotient times divisor plus remainder: %\n", (c / b) * b + (c % b));
}
```

CONSOLE / OUTPUT

```
 7 / 2 = 3   7 rem 2 = 1
-7 / 2 = -3  -7 rem 2 = -1
 7 / -2 = -3   7 rem -2 = 1
-7 / -2 = 3  -7 rem -2 = -1
quotient times divisor plus remainder: -7
```

- `%` is integer-only. There is no float instance of `Rem`, so `a % b` on two `f64` values is `error: no operator '%' for f64, f64`.

> **NOTE**
> At `-O0` and `-O1`, `/`, `//`, and `%` check for a zero divisor and abort before reaching the hardware. Signed division also checks the one unrepresentable quotient, `T.MIN / -1`. Signed remainder is different: `T.MIN % -1` is well-defined as zero and produces zero at every optimisation level. At `-O2` and above the division guards are omitted.

The three shifts

There are three shift operators, each owned by its own typeclass. `>>` is arithmetic on a signed operand and logical on an unsigned one; the operand type decides, because the instance is selected on it. There is no saturating right shift, since a right shift cannot leave the range it started in.

Operator	Class	Meaning
`<<`	`ShiftLeft`	shift left, discarding the bits that leave the width
`>>`	`ShiftRight`	shift right, sign-filling when signed and zero-filling when unsigned
`<<|`	`SaturatingShiftLeft`	shift left, clamping to the extreme value instead of discarding

GLOSSO

```
#import "Io";

main :: () {
    a: u8 = 1;
    print("u8   1 << 7  = %\n", a << 7);
    print("u8   1 <<| 9 = %\n", a <<| 9);
    b: s8 = -8;
    print("s8  -8 >> 2  = %\n", b >> 2);
    c: u8 = 248;
    print("u8 248 >> 2  = %\n", c >> 2);
    d: s8 = 64;
    print("s8  64 << 1  = %\n", d << 1);
    print("s8  64 <<| 1 = %\n", d <<| 1);
}
```

CONSOLE / OUTPUT

```
u8   1 << 7  = 128
u8   1 <<| 9 = 255
s8  -8 >> 2  = -2
u8 248 >> 2  = 62
s8  64 << 1  = -128
s8  64 <<| 1 = 127
```

The shift count has the same type as the shifted value, and a written literal count is checked against the width rather than against the value range. A literal count at least as large as the width is rejected. A count known only at run time is defined instead: over-shifting left or logically right yields zero, while an arithmetic right shift yields the sign (`0` or `-1`). A negative signed count has a large unsigned bit pattern and follows the same over-shift rule. LLVM lowering clamps or selects the count so none of these cases produces poison.

GLOSSO

```
main :: () {
    a: u8 = 1;
    b: u8 = a << 8;
    _ := b;
}
```

CONSOLE / OUTPUT

```
error: shift count 8 is not less than the 8 bits of u8
  --> shift.glo:3:18
  |
3 |     b: u8 = a << 8;
  |                  ^
note: a shift by the width or more has no defined result
```

128-bit and wider integers

`s128` and `u128` are ordinary integer types, not a library emulation. Decimal literals are kept at arbitrary precision while the checker finds their context, so a magnitude beyond 128 bits is accepted when its target is a wider `sN` or `uN`. Native arithmetic and `print` use that declared width exactly, up to the 65535-bit type limit.

GLOSSO

```
#import "Io";

main :: () {
    hi:  s128 = 170141183460469231731687303715884105727;
    lo:  s128 = -170141183460469231731687303715884105728;
    umx: u128 = 340282366920938463463374607431768211455;
    print("s128 max %\n", hi);
    print("s128 min %\n", lo);
    print("u128 max %\n", umx);
    a: u128 = 18446744073709551615;
    print("u64max squared %\n", a * a);
    one: u128 = 1;
```

```
    print("1 << 127 %\n", one << 127);
    mask: u128 = 16#ffff_ffff_ffff_ffff_ffff_ffff_ffff_ffff;
    print("hex mask %\n", mask);
    p: s128 = -7;  q: s128 = 2;
    print("-7 / 2 = %  -7 rem 2 = %\n", p / q, p % q);
    print("cast s128 -1 to u128 %\n", cast(u128) cast(s128) -1);
    print("cast u128 max to u8  %\n", cast(u8) umx);
    wider: u129 = 340282366920938463463374607431768211456;
    print("u129 two^128 %\n", wider);
}
```

CONSOLE / OUTPUT

```
s128 max 170141183460469231731687303715884105727
s128 min -170141183460469231731687303715884105728
u128 max 340282366920938463463374607431768211455
u64max squared 340282366920938463426481119284349108225
1 << 127 170141183460469231731687303715884105728
hex mask 340282366920938463463374607431768211455
-7 / 2 = -3  -7 rem 2 = -1
cast s128 -1 to u128 340282366920938463463374607431768211455
cast u128 max to u8  255
u129 two^128 340282366920938463463374607431768211456
```

NOTE

A 128-bit value needs an annotation. An unannotated literal defaults to `ssize`, so `big := 170141183460469231731687303715884105727;` is `error: 170141183460469231731687303715884105727 does not fit ssize, whose range is -9223372036854775808..=9223372036854775807`. Write `: s128` or `: u128`, or cast.

NOTE

The constant folder and generated native code handle integers wider than 128 bits, but the compile-time register VM does not: its registers hold `i128`. Trying to execute an operation on `u129` in a `#comptime` block reports `compile-time execution does not support u129, an integer wider than 128 bits` instead of producing a truncated answer. A directly foldable constant expression can still be exact at that width.

4.3 Floating Point

Five float spellings are recognised. Each names a semantic width, which is how many bits the arithmetic actually uses, and a storage size, which is how many bytes the value occupies. The two differ for `f80`: on x86, the x87 80-bit extended format, with a 64-bit significand, is stored in sixteen bytes. The table shows x86-64 Windows layout; other targets can use different alignments and an alternative carrier for `f80`.

Spelling	Format	Size / alignment
`f16`	IEEE binary16	2 / 2
`f32`	IEEE binary32	4 / 4
`f64`	IEEE binary64	8 / 8
`f80`	x87 80-bit extended	16 / 16
`f128`	IEEE binary128	16 / 16

`f64` is the default: an unannotated float literal becomes one, and the unnamed float type prints as `f64`. The backend lowers the five to the LLVM types `half`, `float`, `double`, `x86_fp80` and `fp128`. Literal defaulting itself is covered in `Compile-Time Numbers and Literal Defaulting`.

GLOSSO

```
#import "Io";

main :: () {
    a: f16 = 1.5;  b: f32 = 1.5;  c: f64 = 1.5;
    d: f80 = 1.5;  e: f128 = 1.5;
    print("f16 %  f32 %  f64 %  f80 %  f128 %\n", a, b, c, d, e);

    t: f16 = 1.0 / 3.0;
    u: f32 = 1.0 / 3.0;
    v: f64 = 1.0 / 3.0;
    print("one third at f16 %  f32 %  f64 %\n", cast(f64) t, u, v);

    one: f16 = 1.0;
    tick: f16 = 0.001;
    print("f16 next above 1.0 is %\n", cast(f64)(one + tick));
}
```

CONSOLE / OUTPUT
```
f16 1.5  f32 1.5  f64 1.5  f80 1.5  f128 1.5
one third at f16 0.333252  f32 0.333333  f64 0.333333
f16 next above 1.0 is 1.000977
```

binary16 is real

`f16` is not a storage-only format that widens for arithmetic. The compile-time evaluator rounds through a binary16 encode and decode with round-to-nearest, ties-to-even, so a compile-time result and a run-time result agree. One third is `0.333252` at `f16` against `0.333333` at `f32`, and the next representable value above `1.0` is `1.000977`, which is one plus two to the minus tenth. The largest finite `f16` is `65504`.

`f80` is x87 only

`f80` is the x87 extended format, and only an x86 target has it in hardware. The backend asks the target: for cpu `x86_64`, `x86` or `i386` it emits LLVM's `x86_fp80`, and on every other target it carries the value in an `fp128` instead. The storage size stays sixteen bytes either way, but alignment is target-dependent. In particular, 32-bit x86 with a non-Microsoft C ABI aligns `f80` to four bytes, so a containing record can change layout in a cross-build.

GLOSSO
```
widen :: (a: f80, b: f80) -> f80 {
    return a + b;
}

main :: () {
    x: f80 = 1.5;
    y: f80 = 2.5;
    z := widen(x, y);
    _ := z;
}
```

CONSOLE / OUTPUT
```
$ glosso --backend llvm-ir -o f80_x86.ll f80.glo
define x86_fp80 @glo_widen(x86_fp80 %0, x86_fp80 %1) {

$ glosso --backend llvm-ir --target aarch64-linux-musl -o f80_arm.ll f80.glo
define fp128 @glo_widen(fp128 %0, fp128 %1) {
```

NOTE
The two carriers are not the same bit image. On an x86 target the storage holds an x87 extended value; its byte size follows the target ABI. Elsewhere f80 uses an IEEE binary128. An `f80` written to a file by one build is not readable as an `f80` by the other. `f80` is for reaching x87 precision, not for portable storage.

NOTE
On x86-64 Windows, generated programs widen the x87 precision-control setting at startup. Runtime `f80` arithmetic therefore keeps the format's full 64-bit significand instead of being rounded to the legacy 53-bit control setting.

Float literals and runtime overflow

A finite float literal must fit the target type. For example, over: f16 = 100000.0 reports float literal '100000.0' is out of range for f16. A nonzero literal that rounds to zero is also rejected. Runtime narrowing and arithmetic can still produce infinity; very small values may round to zero. The example below narrows an ordinary f32 variable at runtime rather than an out-of-range literal.

GLOSSO
```
#import "Io";

main :: () {
    source: f32 = 100000.0;
    over := cast(f16)source;
    tiny: f32 = 0.00000001;
    under := cast(f16)tiny;
    limit: f16 = 65504.0;
    print("over <= f16 max : %\n", over <= limit);
    print("over == over    : %\n", over == over);
    print("over + over     : %\n", over + over == over);
    print("over / over     : %\n", over / over == over / over);
    print("under == 0.0    : %\n", cast(f64) under == 0.0);
}
```

CONSOLE / OUTPUT
```
over <= f16 max : 0
over == over    : 1
over + over     : 1
over / over     : 0
under == 0.0    : 1
```

That output is the signature of a positive infinity. `over` is not below the largest finite `f16`, it equals itself so it is not a NaN, and adding it to itself leaves it unchanged. Dividing it by itself does produce a NaN, which is why the fourth comparison fails. `under` has become an exact zero.

> **NOTE**
> `print` names the infinity directly: a positive one renders as `inf`, a negative one as `-inf`, and a NaN as `nan`. Float rendering is described in `Show, FromString, and print Formatting`; comparisons remain the reliable test of which special value a magnitude became.

GLOSSO
```
#import "Io";

main :: () {
    source: f32 = 100000.0;
    over := cast(f16)source;
    print("f16 100000.0 prints as %\n", cast(f64) over);
}
```

CONSOLE / OUTPUT
```
f16 100000.0 prints as inf
```

Conversions and operators

No float converts implicitly to another float, in either direction, and no runtime integer converts implicitly to a float. Widening is rejected as firmly as narrowing: the conversion rule has a lossless-widening branch for integers and none at all for floats. `#enable(lossy_numerics)` does not change this, because the gate is consulted only in the integer-to-integer branch; `f32` to `f64` still fails under it.

GLOSSO
```
main :: () {
    x: f32 = 1.5;
    y: f64 = x;
    _ := y;
}
```

CONSOLE / OUTPUT
```
error: got f32, expected f64
  --> widen.glo:3:14
  |
3 |     y: f64 = x;
  |              ^
```

- `cast(T)` is the bridge in both directions, and it truncates toward zero going to an integer. `cast` and `acast` covers it.
- Operators are homogeneous. `a + b` with `a: f32` and `b: f64` is `error: got f64, expected f32`, not a promotion.
- `%` has no float instance: `error: no operator '%' for f64, f64`.
- Floats are `PartialEq` but explicitly not `Eq`. The Prelude declares a negative instance, `$T ::: instance !Eq where Is_Float(T)`, because NaN is not equal to itself.

> **NOTE**
> On `x86_64-pc-windows-msvc`, the driver builds and links a bundled quad-precision helper archive. `f128` arithmetic, comparison, conversion, boxing, and printing therefore work when that archive is available. If the helper build cannot be produced, the driver warns and the eventual link can still report unresolved compiler-runtime symbols; other targets depend on their target runtime and toolchain. Linux musl and WebAssembly links also include target-specific bundled floating-point helpers; an emitted object alone does not supply them.

Wide conversions and display

A binary128 significand holds every bit of a `u64` or `s64` exactly. Convert the value, not an intermediate `f64`, if those bits matter. The reverse cast succeeds exactly for these in-range integers; more general float-to-integer casts still truncate and saturate as described in `cast` and `acast`.

GLOSSO
```
#import "Io";

round_trip :: (value: u64) -> bool {
    wide := cast(f128)value;
    return cast(u64)wide == value;
}

main :: () {
    print("% %\n", round_trip(1), round_trip(18446744073709551615));
}
```

CONSOLE / OUTPUT
```
1 1
```

NOTE
Default float display narrows to binary64; printing is not a way to recover every significant bit of `f80` or `f128`. On 32-bit Arm, MIPS, and PowerPC musl targets, the binary128-to-binary64 path uses an integer-word helper because the C target has no binary128 calling convention. That conversion rounds to nearest, ties to even, and also handles subnormals, overflow, infinities, and NaNs.

4.4 Complex Numbers

A complex type is a primitive, not a two-field struct. Five widths exist, each a pair of same-width floats laid out as real part then imaginary part. The name gives the total number of semantic bits, so `c128` is a pair of `f64` and `c32` a pair of `f16`. Alignment follows the component, not the pair.

Spelling	Components	Size / alignment
`c32`	two `f16`	4 / 2
`c64`	two `f32`	8 / 4
`c128`	two `f64`	16 / 8
`c160`	two `f80`	32 / 16
`c256`	two `f128`	32 / 16

`c160` and `c256` are thirty-two bytes rather than twenty and thirty-two respectively, because each component uses the sixteen byte storage its scalar type uses. The components round to their own widths: one third is `0.333252` in a `c32` and `0.333333` in a `c64`, exactly as it is in `f16` and `f32`. The alignments shown are for x86-64 Windows. Query the corresponding float on another target: `type_info(c160).alignment == type_info(f80).alignment`, including alignment 4 on `x86-linux-musl` and alignment 8 on `s390x-linux-musl`.

`type_kind` reports `15` for a complex type, its own tag. `Is_Complex(T)` answers true; `Is_Numeric(T)` does not, because `Is_Numeric` is defined as integer or float. That definition is the reason for everything in the next section.

Why the arithmetic is not in the Prelude

Every primitive operator instance in the Prelude is keyed on `Is_Integer(T)` or on `Is_Float(T)`. A complex type satisfies neither, so none of those instances reaches it. `Add`, `Sub`, `Mul`, `Div`, `Neg`, `Num` and `PartialEq` for `Is_Complex(T)` are declared in `std/Math/Complex.glo` instead, together with the constructors and accessors, which are `#magic` functions. Nothing about complex arithmetic is in scope until that file is imported.

GLOSSO
```
#import "Io";

main :: () {
    a := complex128(1.0, 2.0);
    _ := a;
}
```

CONSOLE / OUTPUT
```
error: unknown function complex128
  --> nocomplex.glo:4:10
   |
4 |     a := complex128(1.0, 2.0);
   |          ^^^^^^^^^^
```

Importing `Math/Complex` supplies the operators and inherent methods. Constructors such as `complex128(real, imaginary)` keep their width-specific free-function names; accessors use receiver calls such as `z.real_part()` and infer the matching component width from `z`.

Function	Widths	Result
`complex32` … `complex256`	32, 64, 128, 160, 256	a complex value from a real and an imaginary component
`value.real_part()`	every complex width	the real component, as the matching float type
`value.imaginary_part()`	every complex width	the imaginary component
`value.conjugate()`	every complex width	the complex conjugate
`value.norm()`	every complex width	the Euclidean magnitude, as the matching float type

GLOSSO

```
#import "Io";
#import "Math/Complex";

main :: () {
    a := complex128(1.0, 2.0);
    b := complex128(3.0, -4.0);
    sum := a + b;
    prod := a * b;
    print("sum  re % im %\n", sum.real_part(), sum.imaginary_part());
    print("prod re % im %\n", prod.real_part(), prod.imaginary_part());
    print("conj re % im %\n", (a.conjugate()).real_part(), (a.conjugate()).imaginary_part());
    print("norm b  %\n", b.norm());
    print("a == a  %\n", a == a);
    z: c128;
    print("default re % im %\n", z.real_part(), z.imaginary_part());
}
```

CONSOLE / OUTPUT

```
sum  re 4 im -2
prod re 11 im 2
conj re 1 im -2
norm b  5
a == a  1
default re 0 im 0
```

NOTE

Complex accessors belong to their owning type. They are not namespace functions such as `Math.real_part128`: call `z.real_part()`, `z.norm()`, or the explicit receiver form `c128.real_part(z)`.

There is no imaginary literal

Construction is explicit. Glosso has no suffix or directive for an imaginary part; `#I` in particular is not a token, and writing it is `error: unexpected directive 'I'`. The only way to make a complex value with a non-zero imaginary component is a constructor call.

GLOSSO

```
#import "Math/Complex";

main :: () {
    a: c128 = 2.0 + 3.0#I;
    _ := a;
}
```

CONSOLE / OUTPUT

```
error: unexpected directive 'I'
  --> imaginary.glo:4:24
   |
4 |     a: c128 = 2.0 + 3.0#I;
   |                        ^^
```

A compile-time integer does reach a complex target and lands in the real part, since the compile-time number rules let an integer family widen into the complex family. That is the whole of the implicit path: there is no conversion from a runtime scalar.

GLOSSO

```
#import "Io";
#import "Math/Complex";

main :: () {
    z: c128 = 3;
    print("from comptime_int re % im %\n", z.real_part(), z.imaginary_part());
    r: f64 = 1.5;
    w := complex128(r, 0.0);
    print("built re % im %\n", w.real_part(), w.imaginary_part());
}
```

CONSOLE / OUTPUT

```
from comptime_int re 3 im 0
built re 1.5 im 0
```

- Mixing a complex and a scalar in one operator is rejected: with `a: c128` and `r: f64`, `a + r` is `error: no operator '+' for c128, f64`. Widen the scalar with a constructor first.
- A complex type does not implement `Show`, so a bare `%` has no matching `Formatter` instance for `c128`. Print the components or supply a custom `Formatter` instance.
- `c256` components are `f128` and use that type's compiler-runtime helpers when printed; the driver supplies target-specific archives for x86-64 Windows, musl, and WebAssembly. `c160` components are `f80` and print directly as well. Neither component display preserves all the precision of a wide float: the display path narrows to `f64`.
- `c160` uses x87 arithmetic on x86 targets and an `fp128` carrier elsewhere, for the same reason `f80` does. `Floating Point` has the detail.

4.5 Booleans and Truthiness

`bool` is one byte with two values, `true` and `false`. It is its own type kind, not a narrow integer: `Is_Bool(bool)` is true and `Is_Integer(bool)` is false, so none of the integer arithmetic instances reaches it and `true + false` is `error: no operator '+' for bool, bool`.

Nothing converts to `bool` implicitly and `bool` converts to nothing. `b: bool = 1;` is `error: got ssize, expected bool`, and `x: s64 = true;` is `error: got bool, expected s64`. The strictness is deliberate: an integer used where a truth value was meant is the most common way for a condition to be silently wrong, and Glosso answers that question in the condition position only, where it is visible.

`cast` is the only bridge

GLOSSO

```
#import "Io";
#import "Meta";

main :: () {
    t := true;
    f := false;
    print("show true/false  % %\n", t, f);
    print("cast(ssize) true   %\n", cast(ssize) t);
    print("cast(bool) 0     %\n", cast(bool) 0);
    print("cast(bool) 5     %\n", cast(bool) 5);
    print("cast(bool) -1    %\n", cast(bool) -1);
    print("size / kind      % %\n", size_of(bool), type_kind(bool));
    z: bool;
    print("default          %\n", z);
    print("and / or / not   % % %\n", t && f, t || f, !t);
}
```

CONSOLE / OUTPUT

```
show true/false  1 0
cast(ssize) true   1
cast(bool) 0     0
cast(bool) 5     1
cast(bool) -1    1
size / kind      1 3
default          0
and / or / not   0 1 0
```

- `show` renders a `bool` as `1` or `0`, not as `true` or `false`. The Prelude's shower emits the C strings `"1"` and `"0"`.
- `cast(bool)` of an integer is a not-equal-to-zero test, so `5` and `-1` both become `true`.
- An uninitialised `bool` local is `false`; the default value of every primitive is its zero.
- `&&` and `||` have a single instance each, `(bool, bool)`, with a `#lazy` right operand. A non-`bool` operand is rejected rather than tested: `n && b` with `n: s64` is `error: no operator '&&' for s64, bool`.

Truthiness

A condition does not have to be a `bool`. `if` and `while` accept any type for which the backend has a truth rule, and the rule is chosen by the type's kind. The table below is the complete set; anything outside it is rejected, not guessed at.

Type	The condition is true when
`bool`	the value is `true`
integer, `enum`	the value is not zero
float	the value is ordered-unequal to zero, so `-0.0` and NaN are both false
`*T`, `[*]T`, function pointer, `label`	the address is not null
`?*T`, `?[*]T`, optional function pointer	the optional is present, which is the same null test
`string`, `cstring`, `string16`, `cstring16`	the count is not zero
`[]T`, `[..]T`	the count is not zero

GLOSSO

```
#import "Io";

main :: () {
    zero: s64 = 0;
    if zero print("0        taken\n"); else print("0        skipped\n");
    neg: s64 = -2;
    if neg print("-2       taken\n"); else print("-2       skipped\n");
    minus: f64 = -0.0;
    if minus print("-0.0     taken\n"); else print("-0.0     skipped\n");
    nan := minus / minus;
    if nan print("nan      taken\n"); else print("nan      skipped\n");
    empty: string = "";
    if empty print("\"\"       taken\n"); else print("\"\"       skipped\n");
    text: string = "x";
    if text print("\"x\"      taken\n"); else print("\"x\"      skipped\n");
    n: s64 = 7;
    p: *mut s64 = &n;
    if p print("*mut s64     taken\n"); else print("*mut s64     skipped\n");
    none: ?*mut s64 = .None;
    if none print("?*mut s64 .None  taken\n"); else print("?*mut s64 .None  skipped\n");
}
```

CONSOLE / OUTPUT

```
0        skipped
-2       taken
-0.0     skipped
nan      skipped
""       skipped
"x"      taken
*s64     taken
?*s64 .None  skipped
```

> **NOTE**
> A NaN is false. The float test is an ordered comparison against zero, and an ordered comparison involving a NaN is false whatever the other operand is. So `if x` and `if x != 0.0` disagree on a NaN, and neither `if x` nor `if !x` will be entered by one.

A pointer optional is truthy exactly when it is present because it has no tag to consult: `?*T` is laid out as the bare pointer and `.None` is address zero, so the presence test and the null test are the same instruction. An optional over a value type is a real tagged union, and the rule does not extend to it.

GLOSSO

```
#import "Io";

main :: () {
    o: ?s64 = .Some(5);
    if o print("taken\n"); else print("skipped\n");
}
```

CONSOLE / OUTPUT

```
error: values of type Option_s64 have no native truthiness rule [opcode Jz]
  while lowering 'main' at IR instruction 4
```

The same refusal covers structs, tuples, unions and every other aggregate: `if p` on a struct `P` is `error: values of type P have no native truthiness rule`. Every entry in the table above rests on a word the value already carries: a tag-free null, or a count at offset zero. A fixed `[N]T` carries neither, so write `arr[..]` or `arr.count` and test that.

`!` is a typeclass operator

`!` is not built into the language. It is owned by the `LogicalNot` typeclass, whose declaration lives in `std/Prelude/Primitive_Operators.glo`, and it does only what an instance says it does. The Prelude ships three instances, and no instance covers integers or pointers.

GLOSSO
```
LogicalNot :: typeclass (T: type) {
    Output :: type;
    '!' :: (value: T) -> Output #operator(prefix);
}
```

Instance	`!value` is true when
`bool`	the value is `false`; the body is `#magic "bool_not"`
`?$T`	the optional is `.None`, for every `T`, niche or not
`$E!$T`	the result is an `.Err`

GLOSSO
```
#import "Io";

Fault :: enum { Bad; }

main :: () {
    b: bool = false;
    print("!false     %\n", !b);

    o: ?s64 = .Some(5);
    n: ?s64 = .None;
    print("!.Some(5)  %\n", !o);
    print("!.None     %\n", !n);

    ok: Fault!s64 = .Ok(1);
    bad: Fault!s64 = .Err(.Bad);
    print("!.Ok(1)    %\n", !ok);
    print("!.Err      %\n", !bad);
}
```

CONSOLE / OUTPUT
```
!false     1
!.Some(5)  0
!.None     1
!.Ok(1)    0
!.Err      1
```

The two mechanisms answer different questions and cover different types. `!o` works on a `?s64` because `LogicalNot` has an instance for every `?T`, while `if o` on the same value does not, because the truth rule stops at the niche layouts. In the other direction, `if n` on an integer works and `!n` does not: `error: no prefix operator '!' for ssize`.

> **NOTE**
> Truthiness belongs to `if` and `while` alone. `#if` requires a `bool` outright: `#if 3` is `error: #if condition must be bool, got comptime_int`. `ifx` requires one too, reporting it as an ordinary conversion failure: `ifx 3 "a" else "b"` is `error: got ssize, expected bool`. See `Static Branching with #if` and `ifx Expressions`.

4.6 Built-in Strings

Glosso has four built-in string view types. They borrow existing storage and differ in code-unit width and how they determine its length. Counted strings store a pointer and count; C strings store a zero-terminated pointer. No import is needed to use any of them.

Spelling	Code unit	Trailing zero
`string`	`u8`, UTF-8	no
`cstring`	`u8`, UTF-8	yes
`string16`	`u16`, UTF-16	no
`cstring16`	`u16`, UTF-16	yes

All four are one kind to the compiler: `type_kind` answers `4` (`Type_Info_Tag.String`) for every one of them. `string` and `string16` have the layout `{ data, count }`: two native words, or 16 bytes on a 64-bit target and 8 on a 32-bit target. `cstring` and

`cstring16` are one pointer wide. Passing any of them by value copies the view, never the character data, and does not transfer ownership of the storage.

Field	Type	Holds
`.count`	`ssize`	stored count, or a scan to the first zero for C strings
`.data`	`[*]u8` or `[*]u16`	the first code unit
`.capacity`	unavailable	a string view does not track an allocation's capacity

`.count` is measured in code units, not in bytes and not in Unicode scalars: a `string16` containing four BMP scalars has `.count` four, but a non-BMP scalar needs two surrogate units. `.data` is a many-pointer to the unit type; `type_name` prints `[*]u8` for the UTF-8 types and `[*]u16` for the UTF-16 ones. The descriptor fields are read-only: assigning `.count` or `.data`, taking their addresses, and assigning `text[index]` are rejected.

GLOSSO

```
#import "Io";
#import "String_Builder";

main :: () {
    s: string = "hello";
    print("count %\n", s.count);
    print("s[0] %  s[1..3] %\n", s[0], s[1..3]);
    builder := String_Builder.builder_from_string(s);
    builder.write_string(" world");
    print("concat %\n", builder.to_string());
    print("equal %  differ %\n", s == "hello", s != "hello");
}
```

CONSOLE / OUTPUT

```
count 5
s[0] 104  s[1..3] el
concat hello world
equal 1  differ 0
```

Indexing yields one code unit as an integer, so `s[0]` on a `string` is a `u8` and prints as the byte value `104`, not as the character `h`. Indexing `string16` or `cstring16` yields a `u16` UTF-16 code unit, including during constant folding; a non-BMP character therefore occupies the lead and trail surrogate at two indexes. Slicing with a range yields a borrowed counted string: `string` for either UTF-8 input and `string16` for either UTF-16 input. It does not preserve C termination. `+` concatenates constant strings only. Build runtime text with `String_Builder`, or use `format` for an owned allocation and `tprint` for temporary text. Repetition uses a loop that writes to a builder. The former string `cat` and `repeat` helpers have been removed.

> **NOTE**
> Iteration is scalar-oriented even though indexing is code-unit-oriented. A `for` over any of the four string types decodes and yields `rune`; malformed input yields U+FFFD, and `it_index` is a scalar ordinal. Iteration and `for <` are detailed in `for` Loops and Iterable.

GLOSSO

```
#import "Io";
#import "Strings/Utf8";

main :: () {
    text: string = "Aé😀";
    wide: string16 = "Aé😀";
    print("units % %\n", text.count, wide.count);
    for value: text print("%:% ", it_index, cast(u32)value);
    print("\n");
    for value: text.bytes() print("% ", value);
    print("\n");
    for < value: wide print("%:% ", it_index, cast(u32)value);
    print("\n");
}
```

CONSOLE / OUTPUT

```
units 7 4
0:65 1:233 2:128512
65 195 169 240 159 152 128
0:128512 1:233 2:65
```

Use `string.bytes` when the algorithm operates on encoded bytes, and iterate the string itself when it operates on Unicode scalars. The explicit rune-to-`u32` cast above asks the formatter for numeric code points instead of displaying the characters. A reverse string walk also counts iterations from zero; its index is not a byte offset or the original forward scalar index.

> **NOTE**
> `.count` counts code units, so it is not a character count for anything outside ASCII. The four-character string `café` has `.count` five as a `string` and `.count` four as a `string16`, because the `é` occupies two UTF-8 bytes and one UTF-16 unit.

GLOSSO

```
#import "Io";

main :: () {
    s: string = "café";
    print("bytes % first % last %\n", s.count, s[0], s[4]);
    w: string16 = "café";
    print("utf-16 units %\n", w.count);
    c: cstring = "café";
    print("cstring count % terminator %\n", c.count, c.data[c.count]);
}
```

CONSOLE / OUTPUT

```
bytes 5 first 99 last 169
utf-16 units 4
cstring count 5 terminator 0
```

The trailing zero

A non-null `cstring` or `cstring16` points at zero-terminated storage. Its `.count` scans up to the first zero and excludes that terminator; there is no stored length or capacity. Thus `c: cstring = "A\0BC"` has count 1. A default-initialised C string is null and has count zero. Do not dereference its `.data` or assume an empty view is non-null.

> **NOTE**
> `c"text"` directly infers `cstring`, with borrowed static storage and a terminating zero. In contrast, `b"text"` is a fixed `u8` array without an added terminator. Its bytes can be copied or modified like any other fixed array; String Literals covers the prefixes and their escape rules.

`string` and `string16` make no such promise. A string literal happens to carry one, because a literal is emitted as a static constant that is one unit longer than its count. A borrowed slice or an allocating library result need not have a readable zero at `data[count]`. The counted view permits embedded zeros without shortening its count.

> **NOTE**
> Never assume a `string` is terminated. Convert with `string.cstring_from_string` or `string.c_string` from `Strings/Utf8` when a terminator is required; the first takes an allocator and must be released with `cstring.free`, the second allocates from the temporary allocator.

Reaching C

C strings and string literals can supply a read-only pointer of the matching code-unit type. A runtime counted string cannot: its view does not promise termination. For a C API that scans to zero, first construct a C string. For an API that accepts a separate length, pass `.data` and `.count` explicitly.

Source	Reaches
`cstring` or UTF-8 literal	matching `*u8` or `*s8` pointer
`cstring16` or UTF-16 literal	matching `*u16` or `*s16` pointer

GLOSSO

```
#import "Io";
#import "Meta";

W16 : string16 : "hi";
B8 : string : "hi";
Bytes :: type_of(B8.data);
Units :: type_of(W16.data);

first_byte :: (p: [*]u8) -> u8 { return p[0]; }
first_char :: (p: [*]s8) -> s8 { return p[0]; }
first_unit :: (p: [*]u16) -> u16 { return p[0]; }

main :: () {
    print(".data is % for string, % for string16\n",
```

```
        type_name(Bytes), type_name(Units));
    s: cstring = "hi";
    w: cstring16 = "hi";
    print("% % %\n", first_byte(s), first_char(s), first_unit(w));
}
```

CONSOLE / OUTPUT

```
.data is [*]u8 for string, [*]u16 for string16
104 104 104
```

The reverse direction is refused by name, because a pointer has no length to recover: `back: string = p;` reports `cannot implicitly convert [*]u8 to string, which carries a count the pointer does not; make a slice of known length and use byte_slice_to_string`. Scanning for a terminator would be a hidden traversal of a buffer that never promised to have one, so the compiler asks for the length instead of inventing it.

Explicit view casts preserve matching code units: `cast(string)bytes` accepts `[]u8`, and `cast([]u8)text` exposes a counted UTF-8 view. The corresponding UTF-16 pair is `string16` and `[]u16`. `cast(string)c_text` and `c_text.to_string()` scan a C string and borrow its storage. None of these operations copies or validates the contents. A runtime counted view still cannot cast to a C string: use an explicit terminating conversion. `cast(cstring)pointer` trusts the caller's termination guarantee; transcoding between UTF-8 and UTF-16 allocates.

GLOSSO

```
#import "Io";

#import "Strings/Utf8";

main :: () {
    s: string = "hi";
    c := s.c_string();
    print("cstring count % terminator %\n", c.count, c.data[c.count]);
    print("back to string %\n", c.to_string());
    bytes := s.bytes();
    print("view count % first %\n", bytes.count, bytes[0]);
    print("round trip %\n", bytes.byte_slice_to_string());
}
```

CONSOLE / OUTPUT

```
cstring count 2 terminator 0
back to string hi
view count 2 first 104
round trip hi
```

Comparison

`==` and `!=` compare contents, not descriptors: the Prelude declares `($T, T) ::: instance PartialEq where Is_String(T)` with the `str_eq` and `str_ne` intrinsics, and `$T ::: instance Eq where Is_String(T)` makes the relation a full equality. Counted strings compare equal when their counts and code units agree. C strings additionally distinguish a null pointer from a non-null pointer to an empty string, although both have count zero. Two null C strings compare equal.

- All four types have lexicographic `PartialOrd` instances. `<=>` uses the `str_cmp` intrinsic; `<`, `<=`, `>` and `>=` are the class defaults written over that result. UTF-8 byte order agrees with code-point order for valid UTF-8, and length breaks a shared-prefix tie. `string16` and `cstring16` compare unsigned UTF-16 code units rather than raw bytes, so their order is independent of the target's endianness.
- Both operands must still be the same string type. Ordering is declared for pairs of the same type, so comparing a `string` against a `cstring` requires an explicit conversion to a common view type.

> **NOTE**
> All four types implement `Show` and support the default `%` selector. `string` renders its view unchanged; `cstring` first scans for the terminator and borrows the resulting bytes. Both UTF-16 forms decode into temporary UTF-8 text for display. Rendering does not transfer ownership of the input or make its backing storage live longer.

Backing storage and ownership

Slicing, `substring`, trimming, byte views, and split pieces borrow the input. `split` allocates an array of views, so free that array without freeing its pieces. A view cannot outlive its backing storage or survive an operation that invalidates it. A literal's storage is static.

`String_Builder` owns a growable buffer and remembers its allocator. Scope exit releases it through `Drop`; `to_string()` borrows the buffer until the next mutation or release. `clone()` creates an independent allocation, while consuming `into_string()` transfers the buffer. Those returned string views do not remember an allocator or free themselves: release an allocated result once with its producing allocator. Import `Strings/Memory` to use `text.free(allocator)` on owned UTF-8 buffers.

`Strings/Utf8` and the `Strings` umbrella also export it; `String_Builder` imports it for its own implementation. UTF-16 and C-string cleanup comes from the matching `Strings/Utf16` or `Strings/Utf8` component. A freed descriptor and all its aliases become invalid. Never free a literal, a borrowed substring, or a temporary-arena result with the ordinary heap allocator. For an owned arena allocation, text.into_temporary() transfers its release obligation to that arena. It preserves the bytes only until the next arena reset or restore.

GLOSSO

```
#import "Io";
#import "String_Builder";
#import "Strings/Utf8";

main :: () {
    builder := String_Builder.builder_from_string("hello world");
    {
        view := builder.to_string();
        print("view %\n", view.substring(6, 11));
    }
    independent := builder.clone();
    defer independent.free();
    builder.reset();
    builder.write_string("changed");
    print("copy %; builder %\n", independent, builder.to_string());
}
```

CONSOLE / OUTPUT

```
view world
copy hello world; builder changed
```

GLOSSO

```
#import "Io";

main :: () {
    missing: cstring;
    empty: cstring = "";
    c_text: cstring = "A\0tail";
    wide: cstring16 = "한";
    earlier: string16 = "A";
    later: string16 = "B";
    print("counts % %; equal %\n", missing.count, empty.count, missing == empty);
    print("C views % %; ordered %\n", c_text, wide, earlier < later);
}
```

CONSOLE / OUTPUT

```
counts 0 0; equal 0
C views A 한; ordered 1
```

4.7 SIMD Vectors and Matrices

`Simd(T, N)` is a first-class primitive type constructor holding exactly `N` inline lanes of type `T`; it needs no import to spell. `std/Simd.glo` supplies its helpers and operator instances. The compile-time evaluator executes each lane scalarly; the LLVM backend receives a fixed-width vector and selects native instructions where the target has them.

GLOSSO

```
#import "Io";

#import "Simd";
#import "Meta";

V4 :: Simd(f32, 4);

main :: () {
    a: V4 = #simd .[1.0, 2.0, 3.0, 4.0];
    b: V4 = #simd .[10.0, 20.0, 30.0, 40.0];
    sum := a + b;
    print("sum % % % %\n", sum[0], sum[1], sum[2], sum[3]);
    print("lanes % element % size % kind %\n",
        simd_lane_count(V4), type_name(simd_element_type(V4)),
        size_of(V4), cast(s64) type_kind(V4));
    print("count % name %\n", a.count, type_name(V4));
    a[0] = 99.0;
    print("after lane store %\n", a[0]);
}
```

CONSOLE / OUTPUT

```
sum 11 22 33 44
lanes 4 element f32 size 16 kind 16
```

```
count 4 name Simd(f32, 4)
after lane store 99
```

A `Simd` is not an array. Internally it is an array type carrying a vector flag, so `Is_Simd(V4)` is `1` while `Is_Array(V4)` is `0`, and `type_kind` answers `16` rather than `6`. `Type::repr` renders it `Simd(f32, 4)`, which is what every diagnostic naming one prints.

Lane counts and lane types

The lane count must be a literal in `1..64`, and the lane type must be an integer of at most 64 bits, `f32`, or `f64`. Both limits are checked where the type is written, so a bad shape is a type error and never a backend surprise.

Written	Result
`Simd(u64, 64)`	accepted, 512 bits
`Simd(f64, 65)`	`Simd lane count must be a literal in 1..64`
`Simd(f64, 0)`	`Simd lane count must be a literal in 1..64`
`Simd(s128, 2)`	`unsupported Simd lane type s128`
`Simd(f16, 4)`	`unsupported Simd lane type f16`
`Simd(bool, 4)`	`unsupported Simd lane type bool`

`#simd` literals

A vector literal is `#simd` followed by a dot-bracket literal. The prefix is required and the context must already be a `Simd(T, N)`: an unprefixed `.[...]` builds an ordinary fixed array and is never reinterpreted, which is what keeps the two shapes from silently swapping places. The element count must equal the lane count exactly.

Written	Diagnostic
`a: V4 = .[1, 2, 3, 4];`	`ordinary array literal .[...] cannot initialize Simd(s32, 4); use #simd .[...]`
`a := #simd .[1, 2, 3, 4];`	`#simd literal requires a Simd(T, N) context`
`a: [4]s32 = #simd .[1, 2, 3, 4];`	`#simd literal requires a Simd(T, N) context, got [4]s32`
`a: V4 = #simd .[1, 2, 3];`	`SIMD literal has 3 lane(s), expected 4`

Lane access, and the missing `.data`

`value[i]` and `value[i] = lane` go through the ordinary `[]` and `[]=` operators and lower to LLVM `extractelement` and `insertelement`; the store replaces a lane in the value, it does not write through a pointer. `.count` reports the lane count. There is deliberately no `.data` (`Simd has no stable .data pointer`) and no slicing (`slice constructor does not support SIMD arrays`), because the register layout of a vector is a target decision and exposing an address would make it a promise.

> **NOTE**
> The internal vector ABI is not a C ABI. Cross a foreign boundary with ordinary scalar memory and move lanes with `simd_load` and `simd_store`, which accept unaligned addresses and lower with alignment 1.

GLOSSO

```
#import "Io";

#import "Simd";

V4 :: Simd(s32, 4);

main :: () {
    lanes: [4]s32 = .[5, 6, 7, 8];
    v := cast(V4) lanes;
    w: V4 = acast lanes;
    print("cast % acast %\n", v[3], w[3]);
    back := cast([4]s32) (v * w);
    print("back % %\n", back[0], back[3]);

    loaded := simd_load(V4, lanes.data);
    out: [4]s32;
    simd_store(out.data, loaded + loaded);
    print("loaded % stored %\n", loaded[1], out[1]);
    print("reduce % reversed %\n",
        loaded.simd_reduce_add(cast(s32) 0), loaded.simd_reverse()[0]);
}
```

CONSOLE / OUTPUT
```
cast 8 acast 8
back 25 64
loaded 6 stored 12
reduce 26 reversed 8
```

The array bridge

A `Simd(T, N)` and a fixed `[N]T` of the same element type convert in both directions, but only through `cast` or `acast`, and only at exactly matching shape. There is no implicit conversion: `v: V4 = src;` reports `cannot implicitly convert [4]f32 to Simd(f32, 4); use cast(Simd(f32, 4))value or acast`. A mismatched shape is not castable either, so `cast(Simd(f32, 4)) src` on a `[8]f32` is `cannot cast [8]f32 to Simd(f32, 4)`.

Helpers and operators

`std/Simd.glo` declares `($V, V)` instances of `Add`, `Sub`, `Mul`, `Div`, `BitAnd`, `BitOr`, `BitXor`, `ShiftLeft` and `ShiftRight`, plus `Neg` and `BitNot` over `Is_Simd(V)`, so vectors use the ordinary operator spellings lane-wise. Both operands must be the same vector type; there is no scalar-against-vector instance, which is what `simd_splat` exists for.

Helper	Meaning
`simd_zero(V)`	all lanes zero
`simd_splat(V, value)`	every lane set to `value`, which must already be the lane type
`simd_load(V, source)` / `simd_store(destination, value)`	move every lane to or from contiguous scalar memory
`value.simd_reverse()`	lanes in reverse order
`value.simd_reduce_add(initial)` and `simd_reduce_mul`, `simd_reduce_xor`, `simd_reduce_or` methods	fold every lane into one scalar
`simd_lane_count(V)` / `simd_element_type(V)`	compile-time shape queries

Matrices

`Matrix(T, Rows, Cols)` is the other primitive flagged array type: fixed inline row-major storage, dimensions part of the type, no allocator and no view state. `std/Math/Matrix.glo` supplies its helpers. The flat index of `(row, column)` is `row * Cols + column`, and that flat index is what `[]` takes. A matrix value copies like any other value, so pass a pointer when a caller should mutate one or when the value is large.

GLOSSO
```
#import "Io";

#import "Math/Matrix";
#import "Meta";

M23 :: Matrix(s64, 2, 3);

main :: () {
    m: M23 = .[1, 2, 3,
               4, 5, 6];
    print("rows % columns % elements % size %\n",
        matrix_row_count(M23), matrix_column_count(M23),
        matrix_element_count(M23), size_of(M23));
    print("flat m[4] % kind %\n", m[4], cast(s64) type_kind(M23));
    value, ok := m.get(1, 2);
    print("get(1, 2) % ok %\n", value, ok);
    missing, found := m.get(5, 0);
    print("get(5, 0) % ok %\n", missing, found);
    n, stored := m.set(0, 0, cast(s64)99);
    print("set -> % original % ok %\n", n[0], m[0], stored);
    print("name % transpose %\n",
        type_name(M23), type_name(matrix_transpose_type(M23)));
}
```

CONSOLE / OUTPUT
```
rows 2 columns 3 elements 6 size 48
flat m[4] 5 kind 17
get(1, 2) 6 ok 1
get(5, 0) 0 ok 0
set -> 99 original 1 ok 1
name Matrix(ssize, 2, 3) transpose Matrix(ssize, 3, 2)
```

`value.get` returns a `(value, ok)` tuple rather than trapping, and `value.set` is functional: it returns a new matrix and leaves its argument alone, which is what keeps matrix values ordinary values in both the compile-time evaluator and native

code. The shape intrinsics `matrix_transpose_type`, `matrix_product_type` and `matrix_matvec_result_type` compute result types at compile time and reject mismatched shapes there; the vector argument of the last one is a fixed `[Cols]T`, not a `Simd`.

> **NOTE**
> A type alias may be declared inside a procedure. For example, `M :: Matrix(s64, 2, 3);` followed by `m: M = .[1, 2, 3, 4, 5, 6];` uses that alias within its lexical scope.

4.8 Types as Values

`type` is a type whose values are types. A type value is obtained by naming a type, and it can be bound to a constant, compared, passed as a parameter and switched on. It is a compile-time value: `size_of(type)` is `0`, because a type carries no runtime representation of its own.

The introspection intrinsics live in `std/Meta/Utilities.glo` and the classifying predicates in `std/Meta/Types.glo`; both come in with `#import "Meta";`. Every one of them is folded to a constant during checking, so they belong in `#comptime`, `#if`, `where`, constant initialisers and `#insert`.

GLOSSO

```
#import "Io";

#import "Meta";

Vec2 :: struct { x: f64; y: f64; }
Small :: s32;

main :: () {
    print("% is % bytes, kind %\n",
        type_name(Small), size_of(Small), cast(s64) type_kind(Small));
    print("% is % bytes, kind %\n",
        type_name(Vec2), size_of(Vec2), cast(s64) type_kind(Vec2));
    x: u16 = 3;
    X :: type_of(x);
    print("type_of(x) is %\n", type_name(X));
    info :: type_info(f80);
    print("f80: name % size % alignment % tag %\n",
        info.name, info.size, info.alignment, cast(s64) info.tag);
}
```

CONSOLE / OUTPUT

```
s32 is 4 bytes, kind 1
Vec2 is 16 bytes, kind 7
type_of(x) is u16
f80: name f80 size 16 alignment 16 tag 2
```

`type_of(value)` is the static type of an expression, `type_name(T)` its printed spelling, `size_of(T)` its size in bytes under natural C layout, `type_kind(T)` its `Type_Info_Tag`, and `type_info(T)` a record holding all of that at once. There is no `align_of`: writing one is `unknown function align_of`, and alignment is read from `type_info(T).alignment`. Field, variant and enum reflection is a larger surface and is covered in Reflection.

`type_kind` returns an enum, and `show` prints an enum's numeric value rather than its name, so `print("%", type_kind(s8))` prints `1`. The tag set is closed and is declared as `Type_Info_Tag` in `std/Meta/Types.glo`.

Value	Name	Covers
0	`Void`	`void`
1	`Integer`	`sN`, `uN` for `N` in 1..65535, plus `ssize`, `usize`; excludes `rune`
2	`Float`	`f16`..`f128`
3	`Bool`	`bool`
4	`String`	`string`, `cstring`, `string16`, `cstring16`
5	`Pointer`	`*T` and `[*]T`
6	`Array`	fixed arrays, views, dynamic arrays
7	`Struct`	`struct`
8	`Enum`	`enum` and `enum_flags`
9	`Union`	tagged and raw unions

Value	Name	Covers
10	`Procedure`	procedure and function-pointer types
11	`Tuple`	multiple-return tuple types
12	`Type`	`type`
13	`Any`	`any`
14	`Label`	`label`
15	`Complex`	`c32`..`c256`
16	`Simd`	`Simd(T, N)`
17	`Matrix`	`Matrix(T, Rows, Cols)`
18	reserved	the removed `char` tag; no current type uses it
19	`Rune`	`rune`

`Type_Info` itself is an ordinary struct with fields `tag`, `name`, `size`, `alignment` and `shower`, so a single `type_info(T)` call answers every shape question about `T` at once.

> **NOTE**
> The tag values are `0` through `17` and `19`; `18` remains reserved for ABI compatibility. `Code`, `Pattern`, `Namespace`, and `Library` currently fall back to `Void`/`0` in `type_kind`; the annotation-only `Typeclass(...)` and `Macro(...)` forms have the same internal fallback. That classification does not make any of them the `void` type.

Types of named definitions

A declaration can state the type of the compile-time definition value between its name and initializer. Aggregate and distinct-type definitions produce `type`; imports and loads produce `Namespace`; a `#library` definition produces `Library`; typeclasses produce `Typeclass(parameter kinds...)`; and macros produce `Macro(function signature)`. An ordinary procedure definition instead has its function type.

```
GLOSSO
Point : type : struct { x: ssize; y: ssize; }
twice : (ssize) -> ssize : (x: ssize) -> ssize { return x * 2; }
IO : Namespace : #import "Io";
kernel32 : Library : #library,system "kernel32";
Render : Typeclass(type) : typeclass(T: type) {}
emit : Macro((ssize) -> void) : (x: ssize) #expand {}
```

`Typeclass` needs at least one type argument, each describing a parameter kind. `Macro` needs exactly one function-signature argument. Both parameterised forms are valid only in this declaration-annotation position: they are not callable constructors or ordinary first-class type expressions. They describe compile-time definitions, not runtime payloads, and the checker rejects a mismatched annotation.

The `Is_*` predicates

`std/Meta/Types.glo` wraps the tag comparison in a family of named predicates: `Is_Integer`, `Is_Rune`, `Is_Character`, `Is_Float`, `Is_Complex`, `Is_Numeric`, `Is_Bool`, `Is_String`, `Is_Pointer`, `Is_Procedure`, `Is_Addressable`, `Is_Enum`, `Is_Struct`, `Is_Union`, `Is_Array`, `Is_Simd`, `Is_Matrix` and `Is_Label`. They are what `where` clauses and instance heads are written against, so they are the working vocabulary rather than `type_kind` comparisons.

- `Is_Numeric` is `Is_Integer(T) || Is_Float(T)` and therefore excludes complex numbers and runes: `Is_Numeric(c128)` and `Is_Numeric(rune)` are both `0`. Bytes remain integers, so `Is_Numeric(u8)` is `1`.
- `Is_Character(T)` is `Is_Rune(T)`. The former `Is_Char` predicate has been removed with the `char` type. A rune has integer-width storage internally, but its dedicated tag keeps it out of generic integer and numeric instances.
- `Is_Addressable` is `Is_Pointer(T) || Is_Procedure(T)`, which is the predicate the pointer-equality instance is keyed on.
- The checker answers these for the compile-time number kinds too: `Is_Integer(comptime_int)` is true, which is what lets `1 + 2` resolve against instances declared over `Is_Integer(T)`.

> **NOTE**
> The predicates are capitalised. `is_integer(s32)` is `unknown function is_integer` with `help: did you mean 'Is_Integer'?`.

GLOSSO
```
#import "Io";

#import "Meta";

describe :: (value: $T) -> string {
    #if Is_Integer(T) return "an integer";
    else #if Is_Float(T) return "a float";
    else #if Is_String(T) return "a string";
    else return "something else";
}

size_of_integer :: (#comptime Q: type) -> ssize where Is_Integer(Q) {
    return size_of(Q);
}

main :: () {
    print("% / % / % / %\n",
        describe(1), describe(1.5), describe("hi"), describe(true));
    print("u8 % s128 %\n", size_of_integer(u8), size_of_integer(s128));
    print("numeric: c128 % u8 % rune %\n",
        Is_Numeric(c128), Is_Numeric(u8), Is_Numeric(rune));
    print("characters: % % %\n",
        Is_Character(u8), Is_Rune(rune), Is_Character(rune));
}
```

CONSOLE / OUTPUT
```
an integer / a float / a string / something else
u8 1 s128 16
numeric: c128 0 u8 1 rune 0
characters: 0 1 1
```

A parameter declared `Q: type` receives a type as its argument. For a `where` clause to see it, the parameter has to be a binder the signature already depends on or be marked `#comptime`, as above; a bare `Q: type` that appears nowhere else in the signature leaves the constraint unevaluated and every call fails with `constraint failed`. Compile-Time and Type-Only Parameters covers the distinction.

Equality and switching

`(type, type)` has a `PartialEq` instance, so `T == s32` is an ordinary comparison folded at compile time, and a `case` list of types is the natural way to branch on one. Equality is over the type value, so native and fixed widths remain distinguishable: `ssize != s64` and `usize != u64`, even on a 64-bit target. C spellings normalize to the fixed-width type instead, so `int64_t == s64`.

GLOSSO
```
#import "Io";

#import "Meta";

width :: (Q: type) -> string {
    if #partial Q == {
        case s8, u8;   return "one byte";
        case s16, u16; return "two bytes";
        case s32, u32; return "four bytes";
        case s64, u64; return "eight bytes";
    }
    return "something else";
}

main :: () {
    print("% / % / % / %\n", width(u8), width(s32), width(u64), width(f64));
    print("ssize == s64 %  s64 == int64_t %  usize == u64 %\n",
        ssize == s64, s64 == int64_t, usize == u64);
    t :: s32;
    print("a type constant shows as %\n", t);
}
```

CONSOLE / OUTPUT
```
one byte / four bytes / eight bytes / something else
ssize == s64 0  s64 == int64_t 1  usize == u64 0
a type constant shows as s32
```

Type queries fold during checking. Bind a query to a constant, such as `X :: type_of(x);`, when its answer is needed by another intrinsic. A mutable local holding a type value is not a compile-time constant: `type_kind(t)` on such a local reports `unknown const t`.

Unevaluated type queries

`type_of(value)` is a free function imported from Meta. Its argument is typechecked but never evaluated, so a call inside it has no side effects and a managed argument does not move. It reports the expression's static type: an any value reports any, rather than its boxed payload type.

GLOSSO

```
#import "Io";
#import "Meta";

calls: ssize;
next :: () -> s64 { calls += 1; return 7; }
main :: () {
    T :: type_of(next());
    print("type=% calls=%\n", type_name(T), calls);
}
```

CONSOLE / OUTPUT

```
type=s64 calls=0
```

4.9 Compile-Time Numbers and Literal Defaulting

A written numeric literal does not have `ssize` or `f64`. It has one of three compile-time kinds whose exact value the checker knows and whose runtime type a context chooses: `comptime_int`, `comptime_float` and `comptime_complex`. Keeping the value exact until something concrete decides it is what makes `x: u8 = 200` and `x: s128 = 200` both mean the number two hundred rather than a widening of a 64-bit payload.

Kind	Produced by	Defaults to
`comptime_int`	an integer literal, and integer arithmetic over compile-time numbers	`ssize`
`comptime_float`	a float literal, and arithmetic that mixes one in	`f64`
`comptime_complex`	nothing, today	`c128`

The three kinds have no source spelling. `x: comptime_int = 1;` is `unknown type 'comptime_int'`; the names exist so that a diagnostic can report an undecided value honestly, which is where a reader meets them. Annotating a declaration with a numeric constant rather than a type produces one: with `M :: 4;`, writing `m: M = .[1, 2, 3];` reports `got [3]comptime_int, expected comptime_int`. `comptime_complex` is currently unreachable, because there is no imaginary literal; `#I` is `unexpected directive 'I'`. A `#char` literal is a separate case rather than a `comptime_int`: ASCII defaults to `u8`, and non-ASCII defaults to `rune`. `#char` Character Literals covers its contextual rule.

Defaulting

When nothing decides a compile-time number, it takes its default. That happens at a `:=` binding, which has no annotation to consult, and at a generic parameter that binds the literal directly, so `describe(5)` on `describe :: (value: $T)` instantiates `T` as `ssize`. The defaults are the target's native signed width for integers and `f64` for floats.

GLOSSO

```
#import "Io";

#import "Meta";

main :: () {
    a := 5;
    b := 5.0;
    c := 1 + 2;
    A :: type_of(a); B :: type_of(b); C :: type_of(c);
    print("5 -> % / 5.0 -> % / 1 + 2 -> % (%)\n",
        type_name(A), type_name(B), type_name(C), c);
    f: f64 = 5;
    z: c128 = 3;
    Z :: type_of(z);
    print("f64 from an integer literal % ; c128 from one is %\n",
        f, type_name(Z));
}
```

CONSOLE / OUTPUT

```
5 -> ssize / 5.0 -> f64 / 1 + 2 -> ssize (3)
f64 from an integer literal 5 ; c128 from one is c128
```

Arithmetic over two compile-time numbers stays compile-time and meets at the wider family: integer is narrower than float, which is narrower than complex. `+`, `-`, `*` and `/` keep the joined kind; `%`, `&`, `|`, `^`, `<<` and `>>` are integer-only, and `1.5 % 2.0` falls out of the compile-time rule and then fails as `no operator '%' for f64, f64`; comparisons produce `bool`. When one operand is concrete the concrete side decides outright, which is why `x + 1` on a `u8` is `u8` arithmetic and `f * 2.0` on an `f32` is `f32` arithmetic, with no promotion to the default width.

Which types a compile-time number reaches

A compile-time number may become any type in its own family or a wider one. The relation is one-directional and stops at `distinct` types, which convert in neither direction without a `cast`.

Kind	Reaches
`comptime_int`	any integer, any float, any complex
`comptime_float`	any float, any complex
`comptime_complex`	any complex

This is a rule about literals, not about values. `f: f64 = 5;` compiles and a named `K :: 5; b: f64 = K;` compiles, because both are still compile-time integers. Once a value has a runtime type the families are sealed: `x: s64 = 5; y: f64 = x;` is `got s64, expected f64`, and the reverse is `got f64, expected s64`.

Going the other way across the families is refused with the kind named: `x: s64 = 1.5;` is `cannot implicitly convert comptime_float to s64` and `x: u8 = 1.0;` is `cannot implicitly convert comptime_float to u8`. A `distinct` target refuses a cross-family literal for the same reason: with `Meters :: distinct f64;`, `m: Meters = 3;` is `cannot implicitly convert comptime_int to Meters`. `cast` and `acast` covers the explicit spelling, and `distinct` Nominal Types covers the nominal rule.

Joint defaulting across a call

When several instances could take the same call because every argument is still undecided, the tie is broken by defaulting: the candidate whose parameter types are exactly what those arguments would default to wins. The rule fires only when a tie exists and only when some argument was undecided, so an already unambiguous call is untouched.

GLOSSO

```
#import "Io";

Pairable :: typeclass (L: type, R: type) #minimal(pair_name) {
    pair_name :: (a: L, b: R) -> string;
}

(ssize, ssize) ::: instance Pairable {
    pair_name :: (a: ssize, b: ssize) -> string { return "ssize, ssize"; }
}
(f64, f64) ::: instance Pairable {
    pair_name :: (a: f64, b: f64) -> string { return "f64, f64"; }
}
(f32, f32) ::: instance Pairable {
    pair_name :: (a: f32, b: f32) -> string { return "f32, f32"; }
}

main :: () {
    print("(1, 2)     chose %\n", pair_name(1, 2));
    print("(1.5, 2.5) chose %\n", pair_name(1.5, 2.5));
}
```

CONSOLE / OUTPUT

```
(1, 2)     chose ssize, ssize
(1.5, 2.5) chose f64, f64
```

All three instances accept `(1, 2)`, since a `comptime_int` reaches `ssize`, `f64` and `f32` alike; the joint default `(ssize, ssize)` picks one. The narrowing is joint and exact: adding `pair_name(1, 2.5)` to the program above is `ambiguous typeclass operation 'pair_name' for these argument types`, because the joint default is `(ssize, f64)` and no instance has that shape. Delete the `(f32, f32)` instance and the same call resolves to `(f64, f64)`, as the only remaining candidate every argument can reach.

The value-fitting rule

Reaching a type is a question about families; fitting it is a question about the value. At an integer context the checker asks whether the magnitude and sign the programmer wrote fit the target's range, and answers from the written syntax rather than from a wrapped payload. That is why a value at the exact edge of a type is accepted and one past it is a compile error rather than a runtime abort.

Written	Result
`x: s8 = -128;`	accepted; the sign folds into the literal, so no negation runs
`x: s8 = -129;`	`-129 does not fit s8, whose range is -128..=127`
`x: u8 = 300;`	`300 does not fit u8, whose range is 0..=255`
`x: u8 = -1;`	`-1 does not fit u8, whose range is 0..=255`
`x: u64 = 16#ffff_ffff_ffff_ffff;`	accepted; a based literal is a bit pattern

Written	Result
`x: s8 = 16#1ff;`	`511 does not fit s8, whose range is -128..=127`
`x: s8 = - ~127;`	`128 does not fit s8, whose range is -128..=127`

A based literal is read as a bit pattern, so it may use every bit of the target width and `16#ffff_ffff_ffff_ffff` is `18446744073709551615` at `u64` and `-1` at `s64`. Over-wide ones say so: the `s8` row above carries `note: a based literal is a bit pattern, so it may use every bit of s8 - this one is wider than that`. A leading `-` drops the relaxation, because it means a magnitude again. `~` is re-evaluated at the target width, so `~127` is `-128` at `s8` and `128` at `u8`, and negating it at `s8` asks for `+128`, which does not fit.

GLOSSO

```
#import "Io";

main :: () {
    mask: u64 = 16#ffff_ffff_ffff_ffff;
    same: s64 = 16#ffff_ffff_ffff_ffff;
    low:  s8  = -128;
    all:  u8  = ~127;
    print("% % % %\n", mask, same, low, all);
}
```

CONSOLE / OUTPUT

```
18446744073709551615 -1 -128 128
```

The check recurses into the operands of integer arithmetic but does not fold the result: each written operand must fit, so `x: u8 = 255 + 0;` compiles and prints `255`, while `x: u8 = 300 + 0;` is rejected on the `300`. An expression whose operands each fit but whose result does not is a runtime question, and `x: u8 = 200 + 100;` compiles and aborts under the checked arithmetic of `-O0` and `-O1`. A shift count is exempted from this rule and judged against the shifted type's width instead.

> **NOTE**
> Named compile-time integers and explicit `#comptime` results must fit the type they receive too. With `K :: 300;`, `x: u8 = K;` is rejected just as `x: u8 = 300;` is. A provably fitting typed constant can narrow without a cast; that exception does not permit an out-of-range value.

4.10 Conversions: What Is Implicit

Coercion is the conversion the compiler performs without being asked: at a declaration's initialiser, at an assignment, at a call argument, at a `return`, and at every aggregate-literal field. Glosso's default is strict. C's rule that any arithmetic type converts to any other is gone, because it silently truncates and silently reinterprets signs; what remains implicit is the set of conversions that cannot lose information. `cast` and `acast` is the explicit way to do the rest.

The order the checker asks in

`Checker::coercible` tries the rules below in order and stops at the first that accepts. The order matters at one point in particular: the `distinct` guard runs before every structural rule, so a nominal type never slips through the integer or pointer cases.

Rule	Accepts
`any` target	every source; the value is boxed
`distinct` guard	nothing across a nominal boundary, in either direction
structural equality	the same type, and `label` to `label`
integer to integer	only a lossless widening, or anything under `lossy_numerics`
function pointer to function pointer	matching calling convention, varargs and `noalias`, with every parameter and the return coercing
compile-time number	a target its family reaches, if the value fits
pointer-like into a niche optional	`*T` into `?*T`, and likewise for `[*]T` and `#fn_ptr`
`null`	only a niche optional pointer, never a bare one
slice to slice	the same element type with compatible mutability and `volatile`
pointer to a fixed-array literal	`&.[...]` to `[]T` when its element type and qualifiers are compatible

Rule	Accepts
pointer to pointer	the same pointer kind, compatible pointees, never read-only to mutable
C string or string literal to pointer	read-only pointer of the matching code-unit type

Everything else is rejected, and there are two shapes of message. A rejected assignment or return reports `got <actual>, expected <expected>`; a rejected argument reports `arg type <actual> != <expected> in call <name>`. A handful of cases have their own wording where the generic form would not say what to do instead.

Lossless integer widening

Integer to integer is accepted when no value of the source is unrepresentable in the target. Same signedness needs only a target at least as wide. An unsigned source reaching a signed target needs a strictly wider target, because the sign bit costs a bit. A signed source never reaches an unsigned target at any width, because negatives have nowhere to go.

From	To	Implicit
`u8`	`u16`, `s16`, `s64`; also `ssize` when native width exceeds 8	yes
`s8`	`s16`	yes
`u32`	`s64`	yes
`u64`	`s128`, `u128`	yes
`u64`	`s64`	no: `got u64, expected s64`
`s8`	`u16`	no: `got s8, expected u16`
`u8`	`s8`	no: `got u8, expected s8`
`s64`	`s8`	no: `got s64, expected s8`

The rule is asked of a width-and-signedness profile. Native integers use the target width, while fixed and normalized C spellings keep their written widths. On x86-64, `s64` and `ssize` therefore have the same profile but remain different type values and print under their own names. They may widen to the same destinations without becoming interchangeable. Relaxing Strictness with `lossy_numerics` covers the section-scoped gate that turns the whole integer rule off.

Floats never convert

No float type converts implicitly to any other, in either direction: `y: f64 = x;` on an `f32` is `got f32, expected f64`, and the narrowing is `got f64, expected f32`. Widening is arguably lossless, but making it implicit would make the narrowing look symmetrical, and a mixed-precision expression would silently pick a width. Cross-family conversion is likewise closed once a value has a runtime type: `y: f64 = x;` on an `s64` is `got s64, expected f64`, and `y: s64 = x;` on an `f64` is `got f64, expected s64`. `bool` participates in neither direction: `b: bool = 1;` is `got ssize, expected bool` because the literal defaults, while `x: s64 = true;` is `got bool, expected s64`.

> **NOTE**
> `#enable(lossy_numerics)` does not change any of this. The gate is consulted only in the integer-to-integer branch, so both directions between float types stay errors under it.

Pointers, slices, and optionals

A pointer reaches a pointer of the same kind with a compatible pointee, and writable reaches read-only but never the reverse. `*void` is the deliberate any-pointer spelling and matches every pointee. A pointer-like value reaches the matching niche optional for free, because the representations are identical; `null` reaches only such an optional, and a bare pointer excludes zero precisely so that the niche can put `.None` there.

GLOSSO

```
#import "Io";

read_one   :: (p: *s64) -> s64 { return p.*; }
count_them :: (view: []s64) -> ssize { return view.count; }
maybe      :: (p: ?*mut s64) -> s64 {
    if #pattern .Some(q) = p return q.*;
    return -1;
}

main :: () {
    v: s64 = 7;
    print("mutable pointer at a read-only parameter %\n", read_one(&v));
    print("pointer at an optional pointer %\n", maybe(&v));
    print("null at an optional pointer %\n", maybe(null));
    storage: [3]s64 = .[1, 2, 3];
```

```
    print("mutable slice at a read-only slice %\n", count_them(storage[..]));
    print("temporary fixed-array pointer at a slice %\n", count_them(&.[cast(s64)4, cast(s64)5, cast(s64)6]));
}
```

CONSOLE / OUTPUT

```
mutable pointer at a read-only parameter 7
pointer at an optional pointer 7
null at an optional pointer -1
mutable slice at a read-only slice 3
temporary fixed-array pointer at a slice 3
```

NOTE

`&.[...]` materialises anonymous fixed-array storage and points at it; an expected borrowed-slice type recovers the element pointer and count. The storage lasts only for the surrounding expression, which makes an immediate call safe. Do not retain or return that slice; give the array a name before slicing it when the borrow must live longer.

The rejections are as narrow as the acceptances. Passing a read-only pointer where writable access is required is `arg type *s64 != *mut s64`; `*T` and `[*]T` describe different things, one element against a run of them, and do not interconvert, so passing a `*s64` where `[*]s64` is wanted is `arg type *s64 != [*]s64`; a differing pointee is `arg type *s64 != *s32`. `p: *s64 = null;` is `'null' is not a value of *s64`, with `note: a plain pointer excludes zero - write ?*s64 for a pointer that may be absent` and `note: its absent value is spelled '.None'. #enable(pointer_punning)`, described in Feature Gates, relaxes the pointer rules.

Strings

A `cstring` reaches a `*u8` or `*s8` pointer, and a `cstring16` reaches a `*u16` or `*s16` pointer. Literals also support these conversions. Counted `string` and `string16` variables do not promise C termination, so pass their data and count explicitly or construct a C string. Explicit matching-code-unit view casts are covered in Built-in Strings.

Compile-time native integers and collection elements

A compile-time scalar of the native/default `ssize` type may narrow where an ordinary fixed-width value cannot, provided its value is provably in range; an explicit `: ssize :` annotation still uses that native route. Elements read from immutable compile-time collections receive the same value-based exception even when the collection element type is fixed, which keeps byte tables such as `[N]s64` convenient. A scalar declared as `A : s64 : 100` retains `s64` and does not take either route.

GLOSSO

```
#import "Io";

Limit : ssize : 100;

main :: () {
    narrow: u8 = Limit;
    print("%\n", narrow);
}
```

CONSOLE / OUTPUT

```
100
```

A value that is not foldable gets nothing: `runtime: s64 = 100; narrow: u8 = runtime;` is `got s64, expected u8`, because a local is not a constant the checker may evaluate. The fit check still applies to an annotated native constant: `Limit : ssize : 300;` makes the same declaration `got ssize, expected u8`. Collection lookup never bypasses it either: `VALUES : [1]s64 : .[300]; byte: u8 = VALUES[0];` is `got s64, expected u8`. The earlier literal-fitting chapter records the current scalar-name exception for an out-of-range `::` constant.

4.11 `cast` and `acast`

A cast is the explicit conversion. Everything Glosso converts on its own is described in the chapter on conversions; a cast is what the program writes when the conversion loses information, changes the interpretation of a bit pattern, or crosses a nominal boundary the checker will not cross silently.

There are two spellings and they differ in exactly one respect: where the target type comes from. `cast(T)` writes the target down; `acast` takes the type the surrounding context expects. Both produce the same node in the checker, so every rule below applies to both.

GLOSSO

```
#import "Io";
```

```
take_u8 :: (v: u8) -> u8 { return v; }
Point   :: struct { x: u8; }
to_f32  :: (v: ssize) -> f32 { return acast v; }

main :: () {
    big: ssize = 300;

    a := cast(u8) big;          // prefix, explicit target
    b := big.cast(u8);          // suffix, explicit target
    c: u8 = acast big;          // prefix, contextual target
    d: u8 = big.acast;          // suffix, contextual target

    print("a % b % c % d %\n", a, b, c, d);

    p: Point = .{ .x = acast big };
    print("field %\n", p.x);
    print("arg %\n", take_u8(acast big));
    print("ret %\n", to_f32(7));
}
```

CONSOLE / OUTPUT

```
a 44 b 44 c 44 d 44
field 44
arg 44
ret 7
```

Form	Where the target type comes from
`cast(T) expr`	the parentheses
`expr.cast(T)`	the parentheses
`acast expr`	the surrounding context
`expr.acast`	the surrounding context

The parentheses on `cast` are mandatory and the diagnostic points at the other form: `cast ssize 5` is `error: cast requires a type in parentheses; use acast for contextual casts`. `acast` is the mirror image and never takes a parenthesised type at all; `acast(f64) 3` is not a form, and the compiler reads it as an `acast` with no context followed by a parenthesised expression.

Where `acast` finds a type

`acast` resolves in any position that already carries an expected type. That is the same set of positions a literal is checked against, and it is a closed set.

- The initialiser of a declaration with a type annotation.
- The right side of a plain assignment to a typed place.
- An argument at a parameter with a declared type.
- A field of an aggregate literal at a known field type.
- A `return` in a procedure with a declared return type.

Everywhere else there is nothing to read the target from, and the compiler says so rather than guessing: `a := acast 5`, an operand of a binary operator, and a variadic argument all produce `error: acast needs a target type from context or an explicit cast type`. In those positions `cast(T)` is the only spelling.

What a cast accepts

A cast is legal when the source and target are semantically the same type, or when they fall into one of the pairs below. The list is small on purpose: a cast reinterprets or converts a value, it does not restructure one, so there is no cast between aggregates and no cast that builds a `string`.

Source	Target	What happens
`Int`, `Float`, `bool`, `enum`	`Int`, `Float`, `bool`, `enum`	numeric conversion: truncate, extend, or round toward zero
`*T`, `[*]T`, `#fn_ptr`, `null`	`*T`, `[*]T`, `#fn_ptr`	reinterpretation; no pointee check
`*T`, `[*]T`, `#fn_ptr`	`Int` or `enum`	the address as a number
`Int` or `enum`	`*T`, `[*]T`, `#fn_ptr`	the number as an address, asserted non-zero
`Int`, `enum`, or a pointer	`?*T`, `?[*]T`	total; zero yields `.None`

Source	Target	What happens
`[N]T`	`Simd(T, N)`, and back	exact shape only; the lane type and count must match

Everything else is `cannot cast <source> to <target>`, with both types printed under their own names. `cast(ssize) s` on a `string` is `error: cannot cast string to ssize`; `cast(ssize) p` on a struct `P` is `error: cannot cast P to ssize`; `cast(any) 5` is `error: cannot cast ssize to any`, because boxing into `any` is an implicit conversion and not a cast. A fixed array casts only to a `Simd` of its exact shape, so `cast([2]f32) a` on a `[4]f32` is `error: cannot cast [4]f32 to [2]f32`.

What a cast does that no implicit conversion will

GLOSSO

```
#import "Io";

Color :: enum { Red; Green; Blue; }

main :: () {
    wide: u64 = 300;
    f:    f64 = 3.9;
    print("narrow      %\n", cast(s8) wide);
    print("wrap neg    %\n", cast(u8) -1);
    print("trunc f64   %\n", cast(ssize) f);
    print("lit to f32  %\n", cast(f32) 7);
    print("integer literal to u8   %\n", cast(u8) 300);
    print("float lit to u8 %\n", cast(u8)(300.0));
    print("float lit to s8 %\n", cast(s8)(300.0));
    print("enum to ssize %\n", cast(ssize) Color.Green);
    print("ssize to enum %\n", cast(Color) 2);
    print("bool to ssize %\n", cast(ssize) true);
    print("ssize to bool %\n", cast(bool) 5);
    print("f64 to f32  %\n", cast(f32) f);
}
```

CONSOLE / OUTPUT

```
narrow      44
wrap neg    255
trunc f64   3
lit to f32  7
integer literal to u8   44
float lit to u8 255
float lit to s8 127
enum to ssize 1
ssize to enum 2
bool to ssize 1
ssize to bool 1
f64 to f32  3.9
```

- Narrowing truncates instead of erroring, and signedness is reinterpreted rather than rejected. `b: s8 = wide` on a `u64` is `error: got u64, expected s8`; `cast(s8) wide` keeps the low bits, and `cast(u8) -1` is `255`.
- Float to integer truncates toward zero while the value is in range and saturates to the target's nearest bound when it is not. This applies to a float literal directly under `cast` as well as a runtime value, so `cast(u8)(300.0)` is `255` and `cast(s8)(300.0)` is `127`. Integer to float and `f64` to `f32` also convert. Negative values cast to an unsigned integer clamp to zero, and NaN converts to zero. These are the conversion's semantics, not optional `-O1` safety checks. None of these runtime numeric conversions is implicit.
- A compile-time integer reaches any integer target without the fitting check: `cast(u8) 300` keeps the low bits and is `44`, where `x: u8 = 300` is `error: 300 does not fit u8, whose range is 0..=255`. A compile-time float follows the saturating float-to-integer rule instead.
- An `enum` crosses to and from its base integer in both directions, and a `distinct` type crosses to and from its representation in both directions. Nothing else in the language crosses both ways; the `distinct` case is covered in the chapter on `distinct` nominal types.

A cast supplies the target context while preserving the exact compile-time magnitude, which matters for literals wider than 64 bits. Thus `cast(u128) 340282366920938463463374607431768211455` prints the number that was written, while the same literal bound with `:=` is `error: 340282366920938463463374607431768211455 does not fit ssize, whose range is -9223372036854775808..=9223372036854775807`.

`cast(*T)` asserts, `cast(?*T)` is total

Every pointer flavour in Glosso excludes zero. That is what gives `?*T` a free niche, where `.None` is address 0: a zero `*T` would be indistinguishable from `.None`, and `.Some(p)` would match `.None`. A cast from an address integer to a bare pointer therefore asserts the address is not zero. When the compiler can already fold the address to zero, `cast(*T)` is a compile error rather than a value, and the diagnostic points at the spelling that is total.

CONSOLE / OUTPUT
```
error: cast to *ssize of an address that is zero at compile time
note: a plain pointer excludes zero - that is what gives ?T its niche, where .None IS address 0
note: cast to ?*ssize instead, which is total and yields .None here
```

An address that is not known until run time carries the assertion as a safety check instead. That check covers `*T`, `[*]T`, and `#fn_ptr` alike, because all three take the same niche; it reports the target type and aborts, and it is emitted at `-O0` and `-O1` only, like every other runtime check.

GLOSSO
```
#import "Io";

address :: (value: usize) -> usize { return value; }

main :: () {
    print("start\n");
    p := cast([*]mut ssize) address(0);
    print("survived %\n", cast(usize)p);
}
```

CONSOLE / OUTPUT
```
start
cast to [*]ssize of an address that is zero
glosso: program exited with status exit code: 134
```

`cast(?*T) addr` is the same reinterpretation with the assertion dropped. It is total: zero becomes `.None`, anything else becomes `.Some`, and no check is emitted because no case is excluded.

GLOSSO
```
#import "Io";

main :: () {
    x: ssize = 7;
    addr := cast(u64) &x;

    none := cast(?*mut ssize) 0;
    if #pattern .Some(r) = none { print("some %\n", r.*); } else { print("none\n"); }

    some := cast(?*mut ssize) addr;
    if #pattern .Some(r) = some { print("some %\n", r.*); } else { print("none\n"); }
}
```

CONSOLE / OUTPUT
```
none
some 7
```

NOTE
The reverse direction is deliberately absent. `cast(*T) opt` on a `?*T` is `error: cannot cast Option__ssize to *ssize`, even though the niche layout would make it free. An optional already has a total spelling that the compiler verifies rather than asserts, so a cast that unwrapped one would be a second escape hatch with no gate and no check above `-O1`.

4.12 `distinct` Nominal Types

`distinct T` declares a type with the representation of `T` and an identity of its own. Nothing about the layout changes: the size, the alignment, and the machine operations are those of `T`. What changes is that the checker treats the two as unrelated, so a length cannot be assigned to a duration and a file handle cannot be added to a socket handle.

GLOSSO
```
Meters :: distinct f64 #derive(Add, Sub, Neg, PartialEq, PartialOrd, Mul(Meters, f64), Show);
Feet   :: distinct f64;
Count  :: distinct s64 #derive,all;
Handle :: distinct u32;
```

`distinct` is a contextual keyword. It is recognised only in type position directly after `::`, so it remains available as an ordinary identifier everywhere else.

GLOSSO
```
#import "Io";

main :: () {
    distinct := 5;
```

```
    print("identifier still works: %\n", distinct);
}
```

CONSOLE / OUTPUT

```
identifier still works: 5
```

Representation and identity

A distinct type carries the representation's kind and name untouched and adds a nominal tag. `size_of(Handle)` is `4` because `u32` is 4 bytes, and `type_name(Handle)` is `Handle` because the tag is what names the type. The same tag is what diagnostics print, so a mismatch reads `got Meters, expected f64` rather than the useless `got f64, expected f64`.

The representation may be any scalar, pointer, `bool`, or `enum`. It may not be a struct: `distinct P` over a struct `P` declares fine but has no way to construct a value, because `cast(Q) p` is `error: cannot cast P to Q`. The same is true of `distinct string`, which is therefore unusable in practice.

Crossing the boundary

There is no implicit conversion in either direction. This rule is checked before every other coercion rule, which is why it applies even where the representations are identical: `x: f64 = m` on a `Meters` is `error: got Meters, expected f64`, `f: Feet = m` is `error: got Meters, expected Feet`, and `c: Count = x` on an `s64` is `error: got s64, expected Count`.

A cast crosses in both directions, and a distinct type is the only thing in the language a cast will cross both ways.

GLOSSO

```
#import "Io";

Meters :: distinct f64 #derive(Add, PartialOrd);

main :: () {
    a: Meters = 3.0;
    b: Meters = cast(Meters) 4.0;
    print("a+b = % a<b = %\n", cast(f64)(a + b), a < b);
}
```

CONSOLE / OUTPUT

```
a+b = 7 a<b = 1
```

> **NOTE**
> A literal of the representation's own family is stamped with the distinct type directly, so `a: Meters = 3.0` compiles without a cast. A literal of a different family is not: `m: Meters = 3` is `error: cannot implicitly convert comptime_int to Meters`. The rule that refuses a compile-time number at a distinct target runs after the one that stamps a matching literal kind, and the asymmetry is visible in nothing else.

`#derive` and the operations a distinct type has

A distinct type gets only the classes it names. Without a `#derive` it has no operators at all, and the diagnostic names the fix: `error: no operator '+' for Meters and Meters: a distinct type gets only the classes it derives; add '#derive(...)' or '#derive,all' on the declaration of 'Meters'`.

This is the point of the feature rather than an inconvenience. A type that inherited everything from its representation would be a type alias with a different spelling; naming the classes is how a distinct type says which of the representation's operations are meaningful for it. Adding two lengths is meaningful, multiplying two lengths is not, and scaling a length by a plain number is.

Spelling	Meaning
`#derive(Add)`	the same-type instance: every operand and the result are the nominal type
`#derive(Mul(Meters, f64))`	spells the operand types out where they differ
`#derive(Add, Sub, PartialEq)`	a comma-separated list; the two forms mix freely
`#derive,all`	every operator the representation has
no `#derive`	no operators

No code is generated. The operator is resolved on the representation and the nominal tag is put back when the representation's result is the representation, so `Meters + Meters` is `Meters` and `Meters * f64` is `Meters`, while `Meters == Meters` stays `bool` and `Meters <=> Meters` stays `Ordering`.

GLOSSO
```
#import "Io";

Meters :: distinct f64 #derive(Mul(Meters, f64));
Secs   :: distinct f64 #derive,all;

main :: () {
    m: Meters = 3.0;
    scale: f64 = 2.0;
    out: Meters = m * scale;         // Mul(Meters, f64); result is Meters
    print("scaled = %\n", cast(f64) out);

    s: Secs = 7.0;
    t: Secs = 2.0;
    print("s/t = % s<t = %\n", cast(f64)(s / t), s < t);
}
```

CONSOLE / OUTPUT
```
scaled = 6
s/t = 3.5 s<t = 0
```

Each entry grants exactly what it names and nothing adjacent. `#derive(Mul(Meters, f64))` does not give `Meters * Meters`, and `#derive(Add)` does not give `Meters + f64`; both are the same `no operator` error. `#derive,all` waves every operator through, but the operation still has to exist on the representation: `s % t` on a `distinct f64` with `#derive,all` falls through to `error: no operator '%' for f64, f64`.

Two spellings are rejected outright. `#derive` with neither a list nor a mode is `error: '#derive' on a distinct type needs a class list or ',all'`, and any other mode is `error: unknown '#derive' mode 'every'; the only one is 'all'`.

> **NOTE**
> The derive list gates operators only. A named typeclass method still resolves through the ordinary generic instances, because the representation's predicate is still true of the distinct type: `Is_Float(Meters)` holds, so `show(a)` on a `Meters :: distinct f64` with no `#derive` at all prints `3.5`.

4.13 Relaxing Strictness with `lossy_numerics`

Glosso converts between numeric types implicitly only when the conversion cannot lose anything: an integer widens to a type that can hold every one of its values, and nothing else. Floats never convert implicitly in either direction, and neither does `bool`. `#enable(lossy_numerics)` relaxes exactly one clause of that rule.

Under the gate, any integer type coerces to any other integer type. Narrowing truncates and a signedness change reinterprets the bits, both silently, at every position where a coercion happens: assignment, argument passing, and `return`.

GLOSSO
```
#import "Io";

#enable(lossy_numerics);

truncating :: (a: u64) -> s8  { return a; }   // narrowing, implicit
resign     :: (a: s64) -> u64 { return a; }   // reinterpreted sign

#disable(lossy_numerics);

main :: () {
    print("truncating(300) = %\n", truncating(300));
    print("resign(-1)      = %\n", resign(-1));
}
```

CONSOLE / OUTPUT
```
truncating(300) = 44
resign(-1)      = 18446744073709551615
```

Section scoping

`#enable` and `#disable` are parser directives, not statements. They may appear only between top-level declarations, and the gate is stamped on every declaration parsed while it is on. Writing `#enable(lossy_numerics)` inside a procedure body is `error: unexpected directive 'enable'`.

The stamp is per declaration and covers the whole of it, body included. A declaration after the matching `#disable` is unaffected: in the fragment below `inside` compiles and `outside` is `error: got u64, expected s8`.

GLOSSO
```
#enable(lossy_numerics);
inside  :: (a: u64) -> s8 { return a; }
```

```
#disable(lossy_numerics);
outside :: (a: u64) -> s8 { return a; }
```

Because the gate belongs to the declaration that performs the coercion, it does not travel with a procedure to its call sites. A `takes :: (v: s8)` declared under the gate does not accept a `u64` argument from an ungated caller: the call is `error: arg type u64 != s8 in call takes`. The parser state also ends with the file, so the gate never reaches a `#load`ed or `#import`ed source.

What the gate does not restore

The gate is consulted in one branch of one function: the integer to integer case of the coercion rule. Every other rejection stands, and each of the following still fails with the gate on.

Written under the gate	Diagnostic
`narrow :: (a: f64) -> f32 { return a; }`	`error: got f64, expected f32`
`widen :: (a: f32) -> f64 { return a; }`	`error: got f32, expected f64`
`f :: (a: f64) -> ssize { return a; }`	`error: got f64, expected ssize`
`g :: (a: ssize) -> f64 { return a; }`	`error: got ssize, expected f64`
`f :: (a: bool) -> ssize { return a; }`	`error: got bool, expected ssize`
`f :: (a: Meters) -> f64 { return a; }`	`error: got Meters, expected f64`
`x: u8 = 300;`	`error: 300 does not fit u8, whose range is 0..=255`

Two of those are worth spelling out. The distinct-type rule runs before the numeric rules and is not a numeric rule at all, so no gate reaches it; `cast` remains the only bridge. And the fitting check on a written literal is a question about the literal, not about a conversion, so it survives too: the gate makes a `u64` variable narrow to `u8`, it does not make `300` fit in one.

The gate also does not touch operator resolution. Every primitive operator instance is homogeneous, keyed on `($T, T)`, and mixed operand widths are an instance-selection failure rather than a coercion failure. With the gate on, `a + b` for `a: u8` and `b: u16` is still `error: no operator '+' for u8, u16`.

> **NOTE**
> Feature names are checked: `#enable(lossy_stuff)` is `error: unknown feature 'lossy_stuff'`. The siblings of this gate are `pointer_punning` and `allow_union_unwrap`, described in the chapter on feature gates.

Why the default is strict

C's conversion rules are the reason integer bugs are hard to see: the truncation and the sign flip look exactly like the code that does not convert at all. Making them explicit costs a `cast` at each site and buys three things.

- Every place a value can change is written down, so a narrowing is found by reading rather than by running.
- A diagnostic can name both types, which is only useful when the program did not ask for the conversion. `got usize, expected ssize` is a question about intent; a silent truncation is not.
- Overflow checking has something to check. The runtime checks described in the chapter on overflow assume a value's type is the type the program chose for it, which is false as soon as widths convert on their own.

`lossy_numerics` exists for the code where that trade is wrong: hand-translated C, a hash or checksum written in terms of deliberate truncation, or a block of bit manipulation where every line would otherwise carry a cast. Bracketing such a block with `#enable` and `#disable` keeps the relaxation visible and bounded, which a command-line flag would not.

4.14 Overflow, Traps, and the `-O` Level

Signed and unsigned integer overflow is a checked failure in Glosso, not wraparound and not undefined behaviour. The check is emitted by the backend, so whether it is present is a property of the build, and the property is exactly the optimisation level.

Runtime checks are on when the optimisation level is 0 or 1. `-O1` is the default, so a plain `glosso --run bump.glo` is a checked build; the line the driver prints names the level it used.

CONSOLE / OUTPUT
```
glosso: link bump.exe (target=x86_64-pc-windows-msvc, -O1)
```

Flag	Optimization policy	Runtime checks in an artifact
`-O0`	available, level 0	on
`-O1` (default)	available, level 1	on
`-O2`	available, level 2	off
`-O3`	available, level 3	off
`-Os`	available, level 2 with `optsize`	off
`-Oz`	available, level 3 with `minsize`	off
`-Ozz`	available, minimum size	off

The size modes -Os, -Oz, and -Ozz produce runnable artifacts. They combine LLVM optimization with optsize/minsize attributes and target-specific linker size policies. Runtime checks are disabled in these modes; use -O0 or -O1 for checked builds. -Ozz also strips symbols and metadata.

What is checked

The arithmetic check covers `+`, `-`, and `*` on integers, and unary `-`, which is lowered as a checked `0 - x`. Each is emitted as an LLVM overflow intrinsic in the operand's own signedness, and the overflow bit branches to an abort.

GLOSSO

```
#import "Io";

bump :: (v: s8) -> s8 { return v + 1; }

main :: () {
    print("before\n");
    print("after: %\n", bump(127));
}
```

CONSOLE / OUTPUT

```
$ glosso --run bump.glo
glosso: compile bump.glo (target=x86_64-pc-windows-msvc)
glosso: link bump.exe (target=x86_64-pc-windows-msvc, -O1)
glosso: done bump.exe  (backend=llvm, target=x86_64-pc-windows-msvc, -O1)
before
runtime check failed: arithmetic.overflow
glosso: program exited with status 134

$ glosso --run -O2 bump.glo
glosso: compile bump.glo (target=x86_64-pc-windows-msvc)
glosso: link bump.exe (target=x86_64-pc-windows-msvc, -O2)
glosso: done bump.exe  (backend=llvm, target=x86_64-pc-windows-msvc, -O2)
before
after: -128
```

The same flag gates the complete backend check family, so an unchecked build drops all of them together.

Check	Failure at `-O0` or `-O1`
`+` `-` `*` and unary `-` on integers	`runtime check failed: arithmetic.overflow`, then abort
integer `/`, `//`, `%` by zero, and signed division `T.MIN / -1`	`runtime check failed: divide.by.zero` or `divide.overflow`, then abort
array and view indexing	when available, a source position followed by `runtime check failed: array.bounds`, then abort
slicing outside the source count	`runtime check failed: slice.bounds`, then abort
`cast` of a runtime-zero address to a pointer	`cast to *ssize of an address that is zero`, then abort
union payload access under `#enable(allow_union_unwrap)`	`union access: expected .Square, but the value holds one of .Circle | .Square`, then abort
integer formatting with a base outside 2..=16	`integer formatting base is outside 2..=16`, then abort

Every check prints a category before terminating. Array bounds and a union mismatch carry a source position when available; arithmetic, division, and slice bounds name the failed operation without a source position or stack trace.

What is not checked

At `-O2` and above, division checks are omitted. Division by zero and signed `T.MIN / -1` then reach LLVM and the target hardware with no language-level result. Remainder by zero is likewise invalid in an unchecked build. Signed `T.MIN % -1` is not an overflow case: the backend substitutes a divisor of `1`, so the result is reliably `0` at every optimisation level.

- The wrapping operators `+%`, `-%`, `*%`, and prefix `-%` are never checked; wrapping is what they mean.
- The saturating operators `+|`, `-|`, `*|`, and `<<|` are never checked; clamping is what they mean.
- Shifts do not trap at run time. A written literal count at least as large as the operand width is rejected, but a dynamic over-shift has a defined result: left and logical-right shifts yield zero, and arithmetic-right shift yields the sign. A negative signed count is treated as out of range by the same rule. The backend avoids LLVM poison for every case.
- Float arithmetic is never checked. IEEE 754 produces an infinity or a NaN and the program continues.
- A many-pointer index carries no length, so it cannot be bounds checked at any level.

Compile-time integer arithmetic is checked

Ordinary constant integer arithmetic must fit its target type. Overflow in a compile-time expression is rejected with the operands, result, and allowed range. Use explicit wrapping operators such as +% when wrapping is intended. Runtime optimization levels do not permit overflowing constant expressions. Invalid constant division is also rejected.

GLOSSO

```
#import "Io";

compute :: () -> u8 { a: u8 = 200; b: u8 = 100; return a + b; }

K :: #comptime compute();

main :: () {
    print("compile time: %\n", K);
    print("run time:     %\n", compute());
}
```

CONSOLE / OUTPUT

```
error: constant arithmetic overflow: 200 + 100 = 300 does not fit u8, whose range is 0..=255
```

> **NOTE**
> At runtime, -O0 and -O1 check ordinary integer arithmetic and counted array access. Optimized builds may omit those runtime checks. Explicit wrapping and saturating operators retain their specified behavior.

4.15 Operator Precedence and the 0-20 Scale

Binary operator precedence in Glosso is an integer from 0 to 20. Higher binds tighter. The scale is deliberately sparse: odd and unused tiers are left free so a library can introduce an operator that lands between two existing families without renumbering anything.

A level is not a property of a spelling the compiler knows in advance. It comes from the `#operator` directive on the typeclass method that declares the operator, and the prescan reads every visible declaration before parsing begins. That mechanism is described in the chapter on the operator prescan; this chapter is the table it produces for the prelude, and the rules a new entry must follow.

Level	Operators	Owning classes
0	`=` and every compound form: `+= -= *= /= %= &= |= ^= <<= >>= &&= ||= +%= -%= *%= +|= -|= *|= <<|=`	the `*Assign` classes
2	`.. ..=`	`RangeExclusiveOp`, `RangeInclusiveOp`
4	`||`	`LogicalOr`
6	`&&`	`LogicalAnd`
8	`== !=`	`PartialEq`
10	`< <= > >= <=>`	`PartialOrd`
12	`|`	`BitOr`
13	`^`	`BitXor`
14	`&`	`BitAnd`

Level	Operators	Owning classes
16	`<< >> <<|`	`ShiftLeft`, `ShiftRight`, `SaturatingShiftLeft`
17	`+ - +% -% +| -|`	`Add`, `Sub`, `WrapAdd`, `WrapSub`, `SaturatingAdd`, `SaturatingSub`
18	`* / % *% *|`	`Mul`, `Div`, `Rem`, `WrapMul`, `SaturatingMul`
19	unoccupied	reserved for exponentiation-class user operators
20	unoccupied	type formers

Tiers 1, 3, 5, 7, 9, 11, and 15 are unoccupied as well. The prelude uses twelve of the twenty-one levels, and a user-declared operator may claim any level from 0 to 1000; the top of that range binds tighter than everything in the table.

Unary operators have no tier. A prefix operator binds tighter than every infix operator, and a suffix operator binds to the primary expression it follows, so `-a + b` is `(-a) + b` and `~a & b` is `(~a) & b`.

Bitwise binds tighter than comparison

This is the one place the table departs from C. In C, and in Glosso before the rescale, `& ^ |` sit below the comparisons, so `flags & MASK == 0` parses as `flags & (MASK == 0)`. That grouping has been a standing source of bugs since it was chosen; Rust and Go both corrected it and the 0-20 table does the same. Relative order is preserved everywhere else.

GLOSSO

```
#import "Io";

main :: () {
    flags: s64 = 6;
    mask:  s64 = 4;
    print("flags & mask == 4  -> %\n", (flags & mask) == 4);
    print("1 | 2 ^ 3 & 6      -> %\n", 1 | 2 ^ 3 & 6);
    print("1 << 2 + 3         -> %\n", 1 << 2 + 3);
    print("2 * 3 + 4          -> %\n", 2 * 3 + 4);
    print("true || false && false -> %\n", true || false && false);
}
```

CONSOLE / OUTPUT

```
flags & mask == 4  -> 1
1 | 2 ^ 3 & 6      -> 1
1 << 2 + 3         -> 32
2 * 3 + 4          -> 10
true || false && false -> 1
```

Because exactly one family moved, the expressions whose meaning changed are exactly those that mix the two families without parentheses, and the source does not say which reading its author meant. The compiler lints them.

CONSOLE / OUTPUT

```
warning: this expression changes meaning under the 0-20 precedence
  table: bitwise operators now bind tighter than comparisons
note: add parentheses to say which reading you mean
```

The lint fires when an unparenthesised bitwise expression is an operand of a comparison, or an unparenthesised comparison is an operand of a bitwise operator, and in no other case. `flags & mask == 4` and `a < b & c` both warn; `(a & b) == 4` does not, and neither does `a & b | c`, where both operators belong to the same family. Adding the parentheses is what silences it, and the parenthesised form reads the same under either table.

Associativity

Every binary operator declares its associativity alongside its level. Every entry in the table above is declared `left` except the assignment tier, which uses the `assign` mode and groups to the right. Two operators that share a level but disagree on associativity cannot be mixed without parentheses, and the parse fails rather than picking one: `error: operators '**' and '~>' both have precedence 19 but different associativity, so '** ~>' is ambiguous; add parentheses`.

How a user-declared operator takes a level

The level is written on the class method, in the `#operator` directive, as `#operator(mode, level)`. The instance that implements the operator does not repeat the level; the class has already registered it, and an instance may omit the directive entirely for a binary operator.

GLOSSO

```
#import "Io";

Pow :: typeclass (L: type, R: type) {
    Output :: type;
    '**' :: (a: L, b: R) -> Output #operator(right, 19);
}
Chain :: typeclass (L: type, R: type) {
    Output :: type;
    '~>' :: (a: L, b: R) -> Output #operator(left, 19);
}

(s64, s64) ::: instance Pow {
    Output :: s64;
    '**' :: (a: s64, b: s64) -> s64 {
        r: s64 = 1;
        i: s64 = 0;
        while i < b { r = r * a; i = i + 1; }
        return r;
    }
}
(s64, s64) ::: instance Chain {
    Output :: s64;
    '~>' :: (a: s64, b: s64) -> s64 { return a * 10 + b; }
}

main :: () {
    print("2 ** 3 ** 2 = %\n", 2 ** 3 ** 2);
    print("1 ~> 2 ~> 3 = %\n", 1 ~> 2 ~> 3);
    print("2 ** 3 * 2  = %\n", 2 ** 3 * 2);
}
```

CONSOLE / OUTPUT

```
2 ** 3 ** 2 = 512
1 ~> 2 ~> 3 = 123
2 ** 3 * 2  = 16
```

`#operator` is the only directive for declaring operator syntax. Older code using `#precedence(left, n)`, `#precedence(right, n)` or `#precedence(assign, n)` must use `#operator` with the same arguments; the old spelling is no longer accepted or registered by the prescan. Unary operators use `#operator(prefix)` or `#operator(suffix)` without a level.

Mistake	Diagnostic
a binary operator with no level anywhere	`error: binary operator '<+>' needs #operator(left|right|assign, n)`
a level outside 0 to 1000	`error: operator precedence level must be between 0 and 1000`
a level on `prefix` or `suffix`	`error: unexpected token ','`
a mode that is not one of the five	`error: #operator expects left, right, prefix, suffix, or assign`
a second class claiming an owned spelling	`error: operator '+' is already owned by typeclass 'Add'`

The remaining rules about operator declaration, including which spellings are reserved and how ownership is keyed by name, arity, and fixity, belong to the chapter on the operator system.

5 Structs, unions, and enums

5.1 Struct Declarations

A struct is a nominal record type: a fixed set of named fields, with an identity that comes from its name rather than from its shape. It is introduced by an ordinary constant declaration whose value is a `struct` body, so the declaration form is `Name :: struct { ... }`.

GLOSSO

```
Vec2 :: struct {
    x: s64 = 1;
    y: s64 = 2;
}
```

Every member ends with a semicolon, and the diagnostic names the offending member: a comma gives `error: struct member 'x' must end with ';'`. A member is a terminated declaration rather than an element of a list because it may carry a default expression, and that expression can itself contain commas. `union` and `enum` members are terminated the same way, with the same wording.

The closing brace ends the declaration and nothing may follow it. A trailing semicolon at file scope is `error: expected a declaration, found ';'`, with help to remove it because a declaration whose body is braced ends at `}`. A struct may be declared at file scope or inside a procedure body, and a procedure-local declaration keeps its plain name, so `type_name` on a local `Local` reports `Local`.

Field defaults

A field may carry `= expr` after its annotation. The expression is an ordinary runtime expression, not a compile-time constant, and it is evaluated afresh on every default initialization of the struct. A field with no default is zeroed.

GLOSSO

```
#import "Io";

counter: s64 = 0;
next_id :: () -> s64 { counter += 1; return counter; }

Entry :: struct {
    id:   s64 = next_id();
    name: string = "anon";
}

Slot :: struct { x: s64 = 7; y: s64; }

main :: () {
    a: Entry;
    b: Entry;
    print("fresh defaults: % %\n", a.id, b.id);
    print("string default: %\n", a.name);

    s: Slot;
    print("declared: % %\n", s.x, s.y);

    u: Slot = #undefined;
    u.x = 1;
    u.y = 2;
    print("undefined then set: % %\n", u.x, u.y);

    row: [2]Slot;
    print("array element: % %\n", row[0].x, row[0].y);
}
```

CONSOLE / OUTPUT

```
fresh defaults: 1 2
string default: anon
declared: 7 0
undefined then set: 1 2
array element: 0 0
```

`a` and `b` receive different identifiers because `next_id()` runs twice. `u: Slot = #undefined;` skips initialization entirely, so neither the default nor the zeroing happens. A default cannot name a sibling field: `b: s64 = a + 1;` is `error: unbound 'a'`, because the defaults are checked in the enclosing scope and a field name is not a binding there.

> **NOTE**
> Field defaults are not applied to the elements of an array. `row: [2]Slot;` gives `row[0].x == 0`, not `7`. An array of structs is zeroed as a block. Where the defaults matter, initialize the elements from a literal.

Generic fields

A field annotation containing `$T` introduces an inferred type parameter for the whole struct. Later fields spell it as plain `T`. The struct is then instantiated by applying it to type arguments, and each instantiation is a separate type with its own mangled name.

GLOSSO

```
#import "Io";
#import "Meta";

Pair :: struct {
    first:  $T;
    second: T;
    label:  string = "pair";
}

main :: () {
    p: Pair(s64) = .{ .first = 1, .second = 2 };
    q: Pair(f64) = .{ 1.5, 2.5 };
    print("% % %\n", p.first, p.second, p.label);
    print("% %\n", q.first, q.second);
    print("%\n", type_name(type_of(p)));
    print("%\n", type_name(type_of(q)));
}
```

CONSOLE / OUTPUT

```
1 2 pair
1.5 2.5
Pair_s64
Pair_f64
```

`Pair_s64` and `Pair_f64` are unrelated types: `q: Pair(f64) = p;` is `error: got Pair_s64, expected Pair_f64`. An applied generic cannot carry a qualified literal, because the qualified form only accepts a bare identifier: `Pair(f64).{ 1.5, 2.5 }` is `error: struct literal needs a type`. Annotate the binding and use the inferred form instead.

The two literal forms

A struct value is written with a brace literal. The qualified form names the type; the inferred form leaves the type to context. Both accept positional elements, named elements `.field = value`, or a mix, and both accept fewer elements than the struct has fields, in which case the remaining fields take their defaults.

Form	Type comes from	Restriction
`T.{ ... }`	The written name `T`.	`T` must be a bare identifier.
`.{ ... }`	The expected type at that position.	There must be one.
`T(A).{ ... }`	Nothing; rejected.	`error: struct literal needs a type`

GLOSSO

```
#import "Io";

Vec2 :: struct { x: s64 = 1; y: s64 = 2; }

take :: (v: Vec2) -> s64 { return v.x + v.y; }
make :: () -> Vec2 { return .{ 6, 7 }; }

main :: () {
    w := Vec2.{ 10, 20 };
    u := Vec2.{ .y = 7 };
    m := Vec2.{ 3, .y = 4 };
    z: Vec2 = .{ .x = 5, .y = 6 };
    print("qualified positional: % %\n", w.x, w.y);
    print("qualified named:      % %\n", u.x, u.y);
    print("mixed:                % %\n", m.x, m.y);
    print("inferred:             % %\n", z.x, z.y);

    z = .{ 8, 9 };
    print("assigned:  % %\n", z.x, z.y);
    print("argument:  %\n", take(.{ 3, 4 }));
    r := make();
    print("returned:  % %\n", r.x, r.y);
    grid: [2]Vec2 = .[ .{ 1, 1 }, .{ 2, 2 } ];
    print("in array:  % %\n", grid[0].x, grid[1].y);
}
```

CONSOLE / OUTPUT

```
qualified positional: 10 20
qualified named:      1 7
mixed:                3 4
inferred:             5 6
assigned:  8 9
argument:  7
returned:  6 7
in array:  1 2
```

The inferred form works wherever the checker already has a concrete expectation: an annotated declaration, the left side of an assignment, an argument position, a `return`, and an element of an array literal whose element type is known. Without an expectation it is `error: struct literal needs a type`, which is what `v := .{ 1, 2 };` produces.

Positional elements follow declaration order, which is not the order the fields are stored in. The layout rule is covered in the chapter on struct memory layout; the point here is that a literal is written against the source, never against the record.

GLOSSO

```
#import "Io";

Plain :: struct { a: u8; b: s64; c: u16; }

offset :: (base: *mut $T, field: *mut $U) -> u64 {
    return cast(u64)field - cast(u64)base;
}

main :: () {
    p := Plain.{ 1, 2, 3 };
    print("positional: a=% b=% c=%\n", p.a, p.b, p.c);
    print("offsets:    a=% b=% c=%\n",
          offset(&p, &p.a), offset(&p, &p.b), offset(&p, &p.c));
}
```

CONSOLE / OUTPUT

```
positional: a=1 b=2 c=3
offsets:    a=10 b=0 c=8
```

Two literal mistakes have their own wording. Supplying more positional elements than the struct has fields is `error: no field '' in struct literal for V` (the empty name is the missing field it tried to fill), and a named element that does not match any field is `error: no field 'z' in struct literal for V`. Reading an absent field off a value is a different diagnostic, `error: no field 'z' in 'V'`, and it carries a `help: did you mean 'x'?` suggestion.

Recursive structs

A struct is a value, so it cannot contain itself by value: the size would not be finite. Recursion goes through an indirection whose size does not depend on the pointee, which means `*T`, `?*T`, `[*]T`, a view, or a dynamic array.

GLOSSO

```
#import "Io";
#import "Meta";

Node :: struct {
    value: s64;
    next:  ?*mut Node;
}

main :: () {
    tail: Node = .{ .value = 2, .next = .None };
    head: Node = .{ .value = 1, .next = &tail };
    #pattern .Some(p) := head.next else { print("end\n"); return; };
    print("% %\n", head.value, p.*.value);
    print("size_of(Node)=%\n", size_of(Node));
}
```

CONSOLE / OUTPUT

```
1 2
size_of(Node)=16
```

> **NOTE**
> Direct or indirect by-value recursion has no finite layout and is rejected. For `Node :: struct { next: Node; }`, the diagnostic identifies the self-containing struct and its field path. A pointer breaks the layout cycle, which is why the linked-node example is valid.

Field restrictions and zero-sized values

- A nested named type declaration. `Inner :: struct { ... }` written inside a struct body is `error: unexpected token '::'`. An anonymous aggregate written directly as a field type is the supported spelling; see the chapter on inline aggregate field types.
- A `void` field is allowed and occupies zero bytes. It remains a member for reflection and declaration-order indexing, so a later field keeps its index. The default value of that field is `.()`.
- Nothing at all is fine: `Empty :: struct { }` is a legal type of size `0`.

Directives written between `struct` and the opening brace change layout, generate instances, or rewrite type parameters. `#c_call` and `#packed` are covered in the chapter on struct memory layout, `#derive` in the chapter on deriving instances, and `#modify` in the chapter on function and aggregate directives.

5.2 Struct Memory Layout

An undecorated struct does not store its fields in declaration order. The compiler sorts them before assigning offsets, so that a record needs no internal padding beyond what the field alignments themselves require. Declaration order still governs everything a program writes: positional literals and the field indices reflection reports are both in source order, and a directive can pin the record to source order as well.

The sort is a single stable ordering with three keys, applied to the stored fields only. Placing the strictest alignments first means every following field lands on a boundary it already satisfies, and ordering by decreasing size within one alignment class packs the tail densely.

Key	Direction
Field alignment	Descending.
Field size	Descending.
Declaration index	Ascending, as the tiebreak.

GLOSSO

```
#import "Io";
#import "Meta";

Bytes :: struct { a: u8; b: [3]u8; c: u8; }
Tie   :: struct { a: u32; b: u32; }
Mixed :: struct { flag: bool; id: u64; code: u16; ratio: f32; }

offset :: (base: *mut $T, field: *mut $U) -> u64 {
    return cast(u64)field - cast(u64)base;
}

main :: () {
    x: Bytes; y: Tie; m: Mixed;
    print("Bytes size=% a=% b=% c=%\n", size_of(Bytes),
          offset(&x, &x.a), offset(&x, &x.b), offset(&x, &x.c));
    print("Tie   size=% a=% b=%\n", size_of(Tie),
          offset(&y, &y.a), offset(&y, &y.b));
    print("Mixed size=% id=% ratio=% code=% flag=%\n", size_of(Mixed),
          offset(&m, &m.id), offset(&m, &m.ratio),
          offset(&m, &m.code), offset(&m, &m.flag));
}
```

CONSOLE / OUTPUT

```
Bytes size=5 a=3 b=0 c=4
Tie   size=8 a=0 b=4
Mixed size=16 id=0 ratio=8 code=12 flag=14
```

`Bytes` shows the size key on its own: every field has alignment `1`, so the three-byte array goes first and `a` and `c` follow in declaration order. `Tie` shows the tiebreak: identical alignment and identical size, so the declared order is kept. `Mixed` shows the whole rule: `8`, `4`, `2`, `1`, giving `16` bytes with a single padding byte at the end, against the `24` bytes the declared order would have needed.

Without #align, the record's alignment is the largest field alignment, and its size is the end of the last field rounded up to that alignment. A struct with no fields has size `0`.

Measuring a layout

There is no `offset_of` and no `align_of`. Offsets are obtained by subtracting the address of the value from the address of the field, and alignment is read from the type's `Type_Info` record. A generic helper keeps this to one line per field.

GLOSSO
```
offset :: (base: *mut $T, field: *mut $U) -> u64 {
    return cast(u64)field - cast(u64)base;
}
```

Spelling	Result
`size_of(T)`	The record size in bytes.
`type_info(T).alignment`	The record alignment in bytes.
`align_of(T)`	`error: unknown function align_of`
`offset_of(T, "b")`	`error: unknown function offset_of`
`struct #align(16) { ... }`	Raises record alignment to at least 16 bytes.

#align(N) requests an explicit record alignment. N must be a literal positive power of two no greater than 268435456. Ordinary structs keep at least their natural field alignment. The record size rounds up to its alignment, including when it has no fields. #packed removes internal field padding; with #align(N), its record alignment is N and its size still rounds up to N. Match any corresponding C declaration exactly.

`#c_call`: declaration order and the C ABI

`struct #c_call { ... }` suppresses the sort. The fields are laid out in declaration order with the padding the target's C compiler would insert, which is what a record shared with C requires: the two sides must agree on offsets, and only the declared order is common to both languages.

The directive also makes the compiler check that every field type has a C ABI at all. `V :: struct #c_call { r: c64; }` is `error: primitive c64 has no declared C ABI; pass its real and imaginary components separately`, and the check recurses into nested aggregates. The rest of the boundary is covered in the chapter on C data across the boundary.

`#packed`: no field padding

`struct #packed { ... }` also suppresses the sort, and additionally removes internal padding: without #align, the size is the sum of the field sizes and the record alignment is `1`. Writing both directives is accepted, and `#packed` wins, because it is the stronger constraint.

GLOSSO
```
#import "Io";
#import "Meta";

Plain  :: struct         { a: u8; b: u64; c: u8; }
C_Aln  :: struct #c_call { a: u8; b: u64; c: u8; }
Packed :: struct #packed { a: u8; b: u64; c: u8; }

offset :: (base: *mut $T, field: *mut $U) -> u64 {
    return cast(u64)field - cast(u64)base;
}

main :: () {
    p: Plain; q: C_Aln; r: Packed;
    print("Plain  size=% align=% a=% b=% c=%\n", size_of(Plain),
          type_info(Plain).alignment,
          offset(&p, &p.a), offset(&p, &p.b), offset(&p, &p.c));
    print("C_Aln  size=% align=% a=% b=% c=%\n", size_of(C_Aln),
          type_info(C_Aln).alignment,
          offset(&q, &q.a), offset(&q, &q.b), offset(&q, &q.c));
    print("Packed size=% align=% a=% b=% c=%\n", size_of(Packed),
          type_info(Packed).alignment,
          offset(&r, &r.a), offset(&r, &r.b), offset(&r, &r.c));
}
```

CONSOLE / OUTPUT
```
Plain  size=16 align=8 a=8 b=0 c=9
C_Aln  size=24 align=8 a=0 b=8 c=16
Packed size=10 align=1 a=0 b=1 c=9
```

Three layouts of one declaration. `Plain` stores `b` at offset `0` although it is written second, and fits in `16` bytes. `C_Aln` keeps the written order and costs `8` bytes more for it. `Packed` keeps the written order and costs nothing, at the price of a `u64` sitting at offset `1`. Neither directive exists on a union: `union #packed { ... }` is `error: unexpected token 'packed'`, since a union's size is fixed by its tag and its largest payload.

GLOSSO
```
#import "Io";
#import "Meta";
```

```
Header :: struct #packed {
    magic:   u32;
    version: u8;
    length:  u32;
}

Inner  :: struct { a: u8; b: u64; }
Nested :: struct #packed { tag: u8; inner: Inner; }

offset :: (base: *mut $T, field: *mut $U) -> u64 {
    return cast(u64)field - cast(u64)base;
}

main :: () {
    h: Header;
    print("Header size=% magic=% version=% length=%\n", size_of(Header),
          offset(&h, &h.magic), offset(&h, &h.version), offset(&h, &h.length));
    h.magic = 1; h.version = 2; h.length = 3;
    print("read back: % % %\n", h.magic, h.version, h.length);
    print("Inner=% Nested=%\n", size_of(Inner), size_of(Nested));
}
```

CONSOLE / OUTPUT

```
Header size=9 magic=0 version=4 length=5
read back: 1 2 3
Inner=16 Nested=17
```

NOTE

`#packed` is not contagious. It removes the padding this record would have inserted between its own fields; a field whose type is another aggregate keeps that aggregate's own size, padding included. `Nested` is `1 + 16`, not `1 + 9`. A wire format made of nested records needs `#packed` on every level.

Reflection reports declaration order

GLOSSO

```
#import "Io";
#import "Meta";

Plain :: struct { a: u8; b: u64; c: u8; }

main :: () {
    print("fields: % % %\n",
          field_name(Plain, 0), field_name(Plain, 1), field_name(Plain, 2));
    print("count=%\n", field_count(Plain));
    print("types: % %\n",
          type_name(field_type(Plain, 0)), type_name(field_type(Plain, 1)));
}
```

CONSOLE / OUTPUT

```
fields: a b c
count=3
types: u8 u64
```

`field_count`, `field_name`, and `field_type` walk the storage fields in declaration order. The sorted order is used only to compute sizes and to emit code, and is not exposed. The index argument must be a compile-time constant: neither `for i: 0..field_count(T)` nor `#inline for` over the same range works, and both are `error: unknown const i`. A traversal is written as a recursive procedure whose index is a parameter, evaluated by `#comptime`.

GLOSSO

```
#import "Format";
#import "Io";
#import "Meta";
#import "String_Builder";

Plain :: struct { a: u8; b: u64; c: u8; }

names :: (#comptime T: type, #comptime i: ssize) -> string {
    #if i >= field_count(T) { return ""; } else {
        return tprint("% %", field_name(T, i), names(T, i + 1));
    }
}

PLAIN_FIELDS :: #comptime names(Plain, 0);

main :: () { print("%\n", PLAIN_FIELDS); }
```

CONSOLE / OUTPUT
```
a b c
```

Explicit record alignment

GLOSSO
```
#import "Io";
#import "Meta";

Aligned :: struct #align(32) { value: s32; }
Packed :: struct #packed #align(2) { tag: u8; value: s32; }

main :: () {
    print("Aligned size=% alignment=%\n", size_of(Aligned), type_info(Aligned).alignment);
    print("Packed size=% alignment=%\n", size_of(Packed), type_info(Packed).alignment);
}
```

CONSOLE / OUTPUT
```
Aligned size=32 alignment=32
Packed size=6 alignment=2
```

5.3 `using`, `#empty`, and `#as`

Three directives may decorate a struct field. They look alike and are often confused; they do three unrelated things, and one of them currently does nothing at all.

Marker	Written	Effect on the struct
`using`	`using base: Base;`	None. Parsed and discarded.
`#empty`	`#empty: $T;`	Declares a field with no storage.
`#as`	`#as value: s64;`	Marks the implicit conversion field.

`using` on a field promotes nothing

A `using` field is an ordinary field. The parser consumes the keyword and records nothing about it, so the struct has the same fields, the same size, and the same member namespace it would have had without it. There is no member promotion and no inheritance.

GLOSSO
```
#import "Io";
#import "Meta";

Base    :: struct { x: s64 = 1; y: s64 = 2; }
Derived :: struct { using base: Base; z: s64 = 3; }

main :: () {
    d: Derived;
    print("through the field: % % %\n", d.base.x, d.base.y, d.z);
    print("size=% field_count=% names=% %\n", size_of(Derived),
          field_count(Derived), field_name(Derived, 0), field_name(Derived, 1));
}
```

CONSOLE / OUTPUT
```
through the field: 1 2 3
size=24 field_count=2 names=base z
```

Reaching a base field by its bare name fails: `d.x` is `error: no field 'x' in 'Derived'` with `help: did you mean 'z'?`, and the suggestion naming a sibling rather than a base member is the clearest sign that the marker had no effect. `d.base.x` is the spelling that works.

`using` as a statement and as a parameter modifier

The two forms that do work are both about a binding rather than about a type. `using b;` opens a struct-typed local so its fields are readable as bare names; `(using b: Base)` on a parameter is rewritten by the parser into exactly that statement at the top of the body.

GLOSSO
```
#import "Io";

Base :: struct { x: s64 = 1; y: s64 = 2; }
```

```
sum :: (using b: Base) -> s64 { return x + y; }

main :: () {
    print("parameter: %\n", sum(Base.{ 4, 5 }));

    local := Base.{ 10, 20 };
    using local;
    print("promoted:  % %\n", x, y);
    local.x = 99;
    print("re-read:   %\n", x);
}
```

CONSOLE / OUTPUT

```
parameter: 9
promoted:  10 20
re-read:   99
```

The promoted names read the original storage, not a snapshot: writing `local.x` afterwards changes what the bare `x` reports. The promotion is block-scoped and ends with the block that opened it, so a `using` inside a nested block leaves nothing behind: reading the promoted name after that block is `error: unbound x`.

Promoted names are read-only. `x = 5;` after `using local;` is `error: unbound 'x'`, because the promotion lives in expression resolution and never installs an assignable binding. Write through the original name.

The operand must be a struct value. A pointer is `error: using expects a struct, got *Base`, and there is no automatic dereference. It must also be a plain name, not a path: `using d.base;` stops at the identifier and the remaining `.base` is read as a variant shorthand, giving `error: enum shorthand needs an enum context`.

A promoted name is the last thing an identifier lookup consults. A local, a global, or a constant of the same name wins, so opening a struct that happens to have an `x` cannot capture an existing `x`. Among several open `using` statements the earliest one wins, not the most recent, because the open list is searched in the order the statements were executed.

`#empty`: fields with no storage

`#empty: T;` declares a marker field. It occupies no bytes, has no name a program can write, and is filtered out of the storage fields, so it is invisible to `size_of`, `field_count`, and every literal form. Several markers may share one directive, separated by commas and terminated by a semicolon; omitting the terminator is `error: missing ';' after #empty field marker`.

GLOSSO

```
#import "Io";
#import "Meta";

Phantom :: struct { #empty: $T; raw: s64; }
Marker  :: struct { #empty: u8, u16; v: s64 = 9; }

main :: () {
    p: Phantom(f64) = .{ .raw = 5 };
    q: Phantom(s8)  = .{ .raw = 6 };
    m: Marker;
    print("phantom size=% fields=% raw=% name=%\n", size_of(type_of(p)),
          field_count(type_of(p)), p.raw, type_name(type_of(p)));
    print("other instance name=%\n", type_name(type_of(q)));
    print("marker size=% fields=% first=% v=%\n", size_of(Marker),
          field_count(Marker), field_name(Marker, 0), m.v);
}
```

CONSOLE / OUTPUT

```
phantom size=8 fields=1 raw=5 name=Phantom_f64
other instance name=Phantom_s8
marker size=8 fields=1 first=v v=9
```

The generic form is the one that earns its keep. Type-parameter collection still walks the annotation of an `#empty` field, so `#empty: $T;` makes the struct generic over a type it does not store. `Phantom(f64)` and `Phantom(s8)` are then distinct types of eight bytes each, and assigning one to the other is `error: got Phantom_f64, expected Phantom_s8`. The standard library uses this for containers that keep their elements behind an indirection, such as `LinkedList` in `std/Container/LinkedList.glo` and `RangeFull` in the prelude.

> **NOTE**
> `#empty:` in a parameter list is a different feature with the same spelling. `f :: (#empty: $T, ...)` declares a type-witness parameter that carries a type and no value; it is described in the chapter on compile-time and type-only parameters. Nothing about the field form applies to it.

`#as`: the conversion field

`#as` marks one stored field as the struct's implicit conversion target. A value of the struct is then accepted anywhere that field's type is expected, and the compiler substitutes the field. This is how a wrapper stays a distinct type while remaining usable as the thing it wraps.

GLOSSO

```
#import "Io";

Id :: struct { tag: u8; #as value: s64; }

double :: (v: s64) -> s64 { return v * 2; }
give   :: (i: Id) -> s64 { return i; }

main :: () {
    i: Id = .{ .tag = 3, .value = 77 };
    n: s64 = i;
    m: s64;
    m = i;
    print("declaration: %\n", n);
    print("assignment:  %\n", m);
    print("argument:    %\n", double(i));
    print("return:      %\n", give(i));
}
```

CONSOLE / OUTPUT

```
declaration: 77
assignment:  77
argument:    154
return:      77
```

The conversion is outward only. There is no route from the field's type back into the struct: with `f :: (i: Id)`, the call `f(5)` is `error: arg type ssize != Id in call f`. The struct is still a distinct type; `#as` only widens where it may be spent.

It fires only where a concrete expected type already exists, which is the same set of positions the inferred literal `.{ ... }` works in. Operator resolution is not one of them: with the `Id` above, `i + 1` is `error: no operator '+' for Id, ssize`. Operators are chosen by the operand types, before any coercion is attempted.

The conversion is one step deep. If `Outer` has `#as i: Inner` and `Inner` has `#as v: s64`, then `n: s64 = o;` is `error: got Outer, expected s64`; the compiler tests the marked field's type against the expectation and does not recur.

A struct can mark only one field with `#as`; a second marker is rejected and the diagnostic names the existing conversion field. `#as` and `using` combine on one field in either order, `#as using inner: Base;` or `using #as inner: Base;`, though the `using` half still does nothing. `#as` is a struct-only directive: on a union variant it is `error: unexpected token 'as'`.

`Path` in `std/File_System/Path.glo` is deliberately not an example of `#as` any more. It owns a root anchor and separator-free components, has no `.value` text field, and does not convert implicitly to `string`. Path-taking APIs instead accept a generic `P` constrained by `AsView(P, Path_View)`. The standard library supplies that zero-copy view for `string`, pointers to `Path`, and `Path_View`. An owning path's `as_view()` method borrows its components without moving it.

GLOSSO

```
#import "Io";
#import "File_System/Path";

is_component_path :: (value: $P) -> bool
     where AsView(P, Path_View) {
    return path_like_view(value).kind == Path_View_Kind.Components;
}

main :: () {
    path := Path.path_from_string("src/./manual/../main.glo", Path_Style.Posix);
    // Path is a managed owner; scope exit releases its storage.

    text := path.to_string(Path_Style.Posix, context.temp_allocator);
    print("path=% components=%\n", text, path.components.count);
    print("views: text=% components=%\n",
        path_like_view("src/main.glo").kind == Path_View_Kind.Text,
        is_component_path(path.as_view()));
}
```

CONSOLE / OUTPUT

```
path=src/main.glo components=2
views: text=1 components=1
```

5.4 Inline Aggregate Field Types

A `struct`, `union`, `enum`, or `enum_flags` body may be written directly where a field's type annotation goes. The aggregate is not anonymous: the compiler gives it a nominal name built from the field it was written on, hoists the declaration out

to the enclosing scope, and puts that name in the annotation. This is the only nesting a struct body supports; a named `Inner :: struct { ... }` inside a struct body is `error: unexpected token '::'`.

Written as a field type	Declares
`struct { ... }`	A record, with the full struct directive set.
`union { ... }`	A tagged union; `union #raw { ... }` for an untagged one.
`enum { ... }` or `enum u8 { ... }`	An enum, with an optional backing type.
`enum_flags u8 { ... }`	A flag enum; the backing type stays mandatory.

GLOSSO

```
#import "Io";
#import "Meta";

Outer :: struct {
    inner: struct { a: s64; b: s64; };
    kind:  enum { One; Two; };
    perm:  enum_flags u8 { Read :: 1; Write :: 2; };
    tag:   union { L: s64; R: bool; };
}

sum :: (v: Outer.inner) -> s64 { return v.a + v.b; }

main :: () {
    o: Outer;
    o.inner = .{ 1, 2 };
    o.kind  = .Two;
    o.perm  = .Write;
    o.tag.L = 5;
    print("values: % % % %\n", o.inner.a, o.inner.b, o.kind, o.perm);
    print("inner name: %\n", type_name(type_of(o.inner)));
    print("kind name:  %\n", type_name(type_of(o.kind)));
    print("perm name:  %\n", type_name(type_of(o.perm)));
    print("tag name:   %\n", type_name(type_of(o.tag)));
    if o.tag == {
        case .L(v); print("tag L %\n", v);
        case .R(b); print("tag R %\n", b);
    }
    standalone: Outer.inner = .{ 3, 4 };
    print("named annotation: % %\n", sum(o.inner), sum(standalone));
}
```

CONSOLE / OUTPUT

```
values: 1 2 1 2
inner name: Outer.inner
kind name:  Outer.kind
perm name:  Outer.perm
tag name:   Outer.tag
tag L 5
named annotation: 3 7
```

The name is `Outer.field`: the owning aggregate, a dot, and the field name. `type_name` reports it verbatim, and it is a real type name, so it may be written in any annotation, including a parameter, a return type, and a declaration of a value that is not stored in an `Outer` at all. The inline form is therefore a placement convenience, not a scoping mechanism: the type is as visible as the outer declaration is.

The dotted spelling is annotation syntax only. In expression position the dot is member access, so `size_of(Outer.inner)` is `error: member of non-struct`. Where a type value is needed, take it from a binding with `type_of(value)`.

Nesting and composition

The name hint follows the field being parsed, so an inline aggregate may appear anywhere inside that field's type: under a pointer, under an array or view, under an option, and inside a further inline aggregate. A union variant's payload is an aggregate field too, so the same spelling works there and yields `Union.Variant`.

GLOSSO

```
#import "Io";
#import "Meta";

Outer :: struct {
    inner: struct {
        deep: struct { n: s64; };
        m: s64;
    };
    items: []struct { v: s64; };
    slot:  ?*struct { w: s64; };
}

Msg :: union {
    Header: struct { id: s64; len: s64; };
```

```
    Eof:    void;
}

main :: () {
    o: Outer;
    o.inner.deep.n = 3;
    print("deep=% name=%\n", o.inner.deep.n, type_name(type_of(o.inner.deep)));
    print("items elem name=%\n", type_name(type_of(o.items)));
    print("slot name=%\n", type_name(type_of(o.slot)));
    m := Msg.Header(.{ .id = 1, .len = 2 });
    if m == {
        case .Header(h); print("header % %  name=%\n", h.id, h.len,
                               type_name(type_of(h)));
        case .Eof;       print("eof\n");
    }
}
```

CONSOLE / OUTPUT

```
deep=3 name=Outer.inner.deep
items elem name=[]Outer.items
slot name=Option__Outer_slot
header 1 2  name=Msg.Header
```

A nested inline aggregate composes its name from the path it was reached by, giving `Outer.inner.deep`. Under a type constructor the element name is still `Outer.items`, and the constructor's own spelling wraps it, which is why the view prints as `[]Outer.items` and the option as `Option__Outer_slot`.

> **NOTE**
> One inline aggregate per field. The name is derived from the field alone, so a field whose type mentions two of them collides with itself: `both: (struct { p: s64; }, struct { q: s64; });` is `error: type 'Outer.both' is already declared`. Give at least one of them a name of its own.

Directives on an inline aggregate

An inline body is parsed by the same procedure as a named one, so it takes the same directives. A field may be a packed record, a C-layout record, a derived record, a narrow enum, or a raw union, without any of them needing a top-level declaration.

GLOSSO

```
#import "Io";
#import "Meta";

Outer :: struct {
    head:  struct #packed { a: u8; b: u64; };
    body:  struct #c_call { a: u8; b: u64; };
    shown: struct { n: s64; };
    small: enum u8 { A; B; };
    bits:  union #raw { i: s64; f: f64; };
}

main :: () {
    o: Outer;
    o.shown.n = 4;
    o.small = .B;
    o.bits.f = 1.0;
    print("head=% body=% small=% bits=%\n", size_of(type_of(o.head)),
          size_of(type_of(o.body)), size_of(type_of(o.small)),
          size_of(type_of(o.bits)));
    print("shown.n=%\n", o.shown.n);
    print("raw read: %\n", o.bits.i);
    print("Outer=%\n", size_of(Outer));
}
```

CONSOLE / OUTPUT

```
head=9 body=16 small=1 bits=8
shown.n=4
raw read: 4607182418800017408
Outer=48
```

Where an inline aggregate is refused

Everywhere else. A type body written outside an aggregate field is `error: inline aggregate types are only allowed in aggregate fields`, with the span on the `struct`, `union`, or `enum` keyword.

Position	Accepted
Struct field type	Yes, including under `*`, `[]`, `[N]`, `?`, and tuples.
Union variant payload	Yes, named `Union.Variant`.

Position	Accepted
Local declaration	No.
Parameter or return type	No.
`cast` target	No.

The rule is a naming rule rather than a policy. The parser can only mint a nominal name when it already knows an owner and a field, and it knows both only while it is parsing a struct field or a union variant. Elsewhere there is nothing to call the type, so it is rejected rather than made anonymous. A type that is needed in a signature is declared at the top level and named once.

5.5 Tuples

A tuple is an anonymous product type written `(A, B, C)`. It has no declaration and no name of its own: its identity is its element list, so two procedures that independently write `(s64, string)` agree on the type. The elements are positional and unnamed, which is the whole difference between a tuple and a `struct`.

The literal is `.(a, b, c)` and element access is `.0`, `.1`, and so on. The index is a decimal literal that the parser reads as part of the access, not an expression, so a named constant does not work there: `t.i` is `error: tuple member must be a numeric field like .0`.

GLOSSO

```
#import "Io";

main :: () {
    t: (s64, string, bool) = .(7, "hi", true);
    print("% % %\n", t.0, t.1, t.2);
}
```

CONSOLE / OUTPUT

```
7 hi 1
```

An annotation supplies the element types and the element count. With no annotation the element types are inferred from the values, and an integer literal stays a compile-time number: `.(5)` has the element type `comptime_int`. A literal whose length disagrees with the annotation is `error: tuple literal has 2 value(s), expected 3`.

Form	Meaning
`(A, B, C)`	The tuple type. Element order is significant and there are no names.
`.(a, b, c)`	The tuple literal. Its length must match the expected type exactly.
`t.0`, `t.1`	Element access. The index is a literal, never an expression.
`a, b := expr`	Destructuring declaration, one name per element.
`.()`	The empty tuple, whose type is `void`.

Returning more than one value

There is no separate multiple-return mechanism. A procedure that produces several values declares a tuple return type and returns a tuple literal; the caller destructures it in a declaration.

GLOSSO

```
#import "Io";

divmod :: (a: s64, b: s64) -> (s64, s64) {
    return .(a / b, a % b);
}

main :: () {
    q, r := divmod(17, 5);
    print("% remainder %\n", q, r);
    _, only := divmod(9, 2);
    print("%\n", only);
}
```

CONSOLE / OUTPUT

```
3 remainder 2
1
```

- Destructuring reads any tuple value, not only a call result. With `t: (s64, string)` in scope, `a, b := t;` binds both elements.
- `_` in a destructuring position discards that element.
- Destructuring is a declaration form only. Assigning into existing variables, `x, y = divmod(9, 2);`, is `error: unexpected token '='`.
- Binding the wrong number of names is `error: destructuring binds 3 names from a 2-value result`, and an index past the end is `error: tuple index .2 is out of range for 2-value tuple`.

Tuples are not field-reordered

An undecorated `struct` is laid out by sorting its fields by descending alignment, so a record can be smaller than the sum of its declared members. A tuple keeps declaration order, because the program names element positions directly with `.0` and `.1` and a reordering would silently renumber them.

GLOSSO

```
#import "Io";
#import "Meta";

Record :: struct { a: u8; b: s64; c: u8; }

main :: () {
    t: (u8, s64, u8);
    print("struct % tuple %\n", size_of(Record), size_of(type_of(t)));
}
```

CONSOLE / OUTPUT

```
struct 16 tuple 24
```

> **NOTE**
> The same three members cost 16 bytes as a struct and 24 as a tuple. A tuple is the right shape for a short-lived group of values, such as a multiple return; a type that will be stored in quantity should be a struct. Layout is covered in the chapter on struct memory layout.

`void` is the empty tuple

`.()` is the zero-element tuple literal and its type is `void`, the same `void` a procedure with no return annotation produces and the same `void` that spells a payload-less union variant. It occupies no storage.

GLOSSO

```
#import "Io";
#import "Meta";

main :: () {
    e := .();
    print("% %\n", type_name(type_of(e)), size_of(type_of(e)));
}
```

CONSOLE / OUTPUT

```
void 0
```

- A `void` element is allowed inside a larger tuple. It occupies no storage but remains an element for `.count` and indexing, so `pair: (void, s64) = .(.(), 7);` has two elements and `pair.1` is 7.

Current limits

- Tuples have no `Show` instance, so the default `%` has no matching `Formatter` for a tuple. Print the elements individually or provide a custom rendering.
- An element cannot be assigned. On a `(s64, s64)` value, `t.0 = 10;` reaches lowering and fails with `error: internal error: glo_tup_long_long has no storage field '0'`. Build a replacement tuple instead.
- Nested tuple indexing composes directly: `n.0.0` means `(n.0).0`. After a member-access dot the digits are a tuple index, so the lexer does not combine successive indexes into a floating-point literal.
- `(T)` in type position is the parenthesized type `T`, not a one-element tuple: `one: (s64) = .(5);` is `error: got tuple, expected s64`.
- There is no tuple pattern. `#pattern .(a, b) = t` is `error: unexpected token '('`; destructure with `a, b := t;`.

GLOSSO
```
#import "Io";

main :: () {
    nested := .(.(1, 2), 3);
    pair: (void, s64) = .(.(), 7);
    print("nested %; void %; members %\n", nested.0.1, pair.0, pair.count);
}
```

CONSOLE / OUTPUT
```
nested 2; void .(); members 2
```

5.6 Enums and Flag Enums

An enum is a named set of integer constants that share a type of their own. Each member is written on its own line and, like every other aggregate member, ends with a semicolon; a comma is `error: enum member 'Red' must end with ';'`.

An explicit value is attached with `::`, never with `=`. `=` is the assignment and field-default spelling everywhere else in the language, while `::` is how a constant is bound, and an enum member is a constant. Writing `=` is rejected outright with `error: enum variant integer values use '::', not '='` rather than being quietly accepted as a second spelling.

GLOSSO
```
#import "Io";
#import "Meta";

Color :: enum { Red; Green :: 5; Blue; }

main :: () {
    print("% % %\n", cast(s64)Color.Red, cast(s64)Color.Green, cast(s64)Color.Blue);
    print("size % count %\n", size_of(Color), enum_count(Color));
}
```

CONSOLE / OUTPUT
```
0 5 6
size 8 count 3
```

Numbering starts at zero and each member takes one more than the member before it. An explicit value resets that counter, so `Blue` above is 6 and not 2. The value after `::` is a single integer literal token, optionally preceded by `-`, not an expression: naming a constant, `Red :: K;`, is rejected. Negative values such as `Low :: -1;` require a signed backing type. Hexadecimal literals such as `16#1` are accepted, and two members may carry the same value.

Backing type and size

The backing type is written between `enum` and the brace. It fixes the size and the alignment of the type and must resolve to an integer type. With no backing type an enum uses `ssize`, so its size follows the target's native word rather than always being eight bytes.

GLOSSO
```
#import "Io";
#import "Meta";

Byte_Code :: enum u8  { Push; Pop; Halt; }
Wide_Code :: enum s32 { Push; Pop; Halt; }
Signed_Code :: enum s8 { Negative :: -1; Zero; Positive; }

main :: () {
    print("u8 % s32 %\n", size_of(Byte_Code), size_of(Wide_Code));
    print("signed % % %\n", cast(s64)Signed_Code.Negative,
        cast(s64)Signed_Code.Zero, cast(s64)Signed_Code.Positive);
}
```

CONSOLE / OUTPUT
```
u8 1 s32 4
signed -1 0 1
```

Form	Meaning
`Name :: enum { A; B; }`	Default `ssize` backing type, one native word.
`Name :: enum u8 { A; B; }`	Size and alignment follow the named backing type.
`Name :: enum_flags u32 { ... }`	Flag enum. The backing type is mandatory here.

Form	Meaning
`A :: 5;` in the body	Explicit value; the next member continues from 6.

Explicit variant values and automatically incremented values must fit the backing type. `enum u8 { Red :: 300; }` is rejected, as is a member following value 255 in a `u8` enum. A cast can still produce an integer that is not a declared member, such as `cast(Color)9` for a three-member enum; the type alone does not prove membership in the declared set.

Naming a member

The qualified form `Color.Blue` works anywhere. The dotted shorthand `.Blue` works wherever an enum type is already expected: an annotated declaration, an argument position, the other side of a comparison, a `return`, and a `case` label.

GLOSSO

```
#import "Io";

Color :: enum { Red; Green; Blue; }

pick :: (c: Color) -> Color { if c == .Red return .Blue; return .Red; }

main :: () {
    c: Color = .Green;
    print("annotation: %\n", cast(s64)c);
    print("argument: %\n", cast(s64)pick(.Red));
    print("comparison: %\n", c == .Green);
    if c == {
        case .Green; print("case: green\n");
        case .Red;   print("case: red\n");
        case .Blue;  print("case: blue\n");
    }
}
```

CONSOLE / OUTPUT

```
annotation: 1
argument: 2
comparison: 1
case: green
```

- A shorthand with no expected type is `error: enum shorthand needs an enum context`, and an unknown member is `error: no enum variant Color.Purple`.
- A plain `if x == { ... }` switch over an enum is exhaustiveness-checked, which is why the example above lists `Blue`. Dropping that arm is `error: non-exhaustive switch` with `note: missing case: Color.Blue` and `note: cover them, or add 'else' or '#partial'`; `if #partial x == { ... }` opts out. The chapter on exhaustiveness, guards, and reachability has the whole rule.

Reflection

`enum_count(T)` is the number of members; `enum_name(T, i)` and `enum_value(T, i)` report the member at declaration index `i`. The index has to be a compile-time constant, so an ordinary `for i: 0..enum_count(T)` loop over them is `error: unknown const i`.

GLOSSO

```
#import "Io";
#import "Meta";

Color :: enum { Red; Green :: 5; Blue; }

name_of :: (c: Color) -> string {
    if c == {
        case .Red;   return "Red";
        case .Green; return "Green";
        case .Blue;  return "Blue";
    }
    return "?";
}

main :: () {
    c: Color = .Blue;
    print("% is %\n", name_of(c), cast(s64)c);
    print("% raw prints as %\n", enum_name(Color, 2), c);
    print("declared: % = %, % = %, % = %\n",
          enum_name(Color, 0), enum_value(Color, 0),
          enum_name(Color, 1), enum_value(Color, 1),
          enum_name(Color, 2), enum_value(Color, 2));
}
```

CONSOLE / OUTPUT
```
Blue is 6
Blue raw prints as 6
declared: Red = 0, Green = 5, Blue = 6
```

NOTE
The `Show` instance for enums is the integer one, so `print("%", c)` prints the numeric value and never the member name. Printing a name means either a switch, as in `name_of` above, or `enum_name` with a literal index.

Flag enums

`enum_flags` declares an enum whose members are intended to be combined bitwise. The backing type is not optional here: `enum_flags { Read; }` is `error: unexpected token '{'`, because the parser reads a type immediately after the keyword.

`enum_flags` assigns no bit patterns. Members are numbered sequentially exactly as in a plain enum, and the marker is recorded on the declaration and then never read again by the checker: it documents intent and nothing else. A flag set written without explicit values is therefore wrong in a way that still compiles and still runs.

GLOSSO
```
#import "Io";

Perm :: enum_flags u32 { Read; Write; Exec; }

main :: () {
    print("Read=% Write=% Exec=%\n", cast(u32)Perm.Read, cast(u32)Perm.Write, cast(u32)Perm.Exec);
    p := Perm.Read | Perm.Exec;
    print("Read|Exec raw=% holds Write=%\n", cast(u32)p, (p & Perm.Write) == Perm.Write);
}
```

CONSOLE / OUTPUT
```
Read=0 Write=1 Exec=2
Read|Exec raw=2 holds Write=0
```

NOTE
`Read` is 0, so `Read | Exec` is `Exec` alone, and the set claims not to hold `Write` only because `Write` happens to be 1 and `Exec` is 2. Every member of a flag enum must carry an explicit power-of-two value.

GLOSSO
```
#import "Io";

Perm :: enum_flags u32 {
    None  :: 0;
    Read  :: 16#1;
    Write :: 16#2;
    Exec  :: 16#4;
}

holds :: (set: Perm, flag: Perm) -> bool { return (set & flag) == flag; }

main :: () {
    p := Perm.Read | Perm.Exec;
    print("raw=%\n", cast(u32)p);
    print("read=% write=% exec=%\n", holds(p, Perm.Read), holds(p, Perm.Write), holds(p, Perm.Exec));
    p = p | Perm.Write;
    print("after add write: raw=%\n", cast(u32)p);
    p = p ^ Perm.Exec;
    print("after clear exec: raw=%\n", cast(u32)p);
}
```

CONSOLE / OUTPUT
```
raw=5
read=1 write=0 exec=1
after add write: raw=7
after clear exec: raw=3
```

Four operators are defined directly on enum operands by the compiler rather than through a typeclass instance. The three binary ones take two values of the same enum type; all four produce that enum type again, so a combination stays typed and does not decay to an integer.

Expression	Result
`a | b`	Set union: the bits of both operands.

Expression	Result
`a & b`	Set intersection, the usual membership test when compared back to `b`.
`a ^ b`	Symmetric difference; clears a flag that is known to be present.
`~a`	Complement over the backing type's width.

- Both operands must be the same enum type. `Perm.Read | 4` is `error: no operator '|' for Perm, ssize`; convert the enum with `cast` if raw bits are wanted.
- Arithmetic is not defined: `Perm.Read + Perm.Write` is `error: no operator '+' for Perm, Perm`.
- The compound forms are not defined either. `p |= Perm.Write;` is `error: no operator '|=' for Perm, Perm`; write `p = p | Perm.Write;`.
- Because the operators are attached to the enum kind and not to `enum_flags`, they work on a plain `enum` as well. `enum_flags` is the signal to a reader that combination is intended.

5.7 Tagged Unions

A `union` holds exactly one of several named variants at a time, together with a tag that records which one. Each variant is written `Name: Type;` and, like a struct member, ends with a semicolon; a comma is `error: union member 'A' must end with ';'`. A variant that carries nothing is declared with the payload type `void`.

GLOSSO

```
#import "Io";
#import "Meta";

Vec :: struct { x: s64; y: s64; }

Shape :: union {
    Circle: s64;
    Box:    Vec;
    Empty:  void;
}

area :: (s: Shape) -> s64 {
    if s == {
        case .Circle(r); return 3 * r * r;
        case .Box(q);    return q.x * q.y;
        case .Empty;     return 0;
    }
    return -1;
}

main :: () {
    s := Shape.Circle(5);
    print("% %\n", area(s), size_of(Shape));
}
```

CONSOLE / OUTPUT

```
75 24
```

Variant names are capitalised by convention and nothing enforces it. A payload is never read by writing `s.Circle`; the only way in is a pattern capture, which the chapter on reading a union payload covers in full. Like an enum, a union subject makes a plain `if x == { ... }` switch exhaustiveness-checked, so a missing arm is `error: non-exhaustive switch` with one `note: missing case:` line per uncovered variant; the chapter on exhaustiveness, guards, and reachability has the details.

The tag, the size, and the alignment

A tagged union is laid out as a target-width `ssize` tag followed by the largest payload, and the record takes the larger of the tag's alignment and the largest payload alignment. `void` variants contribute nothing to either figure. The tag is independent of the variant count: it is eight bytes on a 64-bit target and four on a 32-bit target. The two-variant `union { A: u8; B: u8; }` is therefore 16 bytes on x86-64 Windows, but 8 bytes on `x86-linux-musl` or `wasm32`.

GLOSSO

```
#import "Io";
#import "Meta";

Tiny :: union { A: u8;  B: u8; }
Big  :: union { A: [24]u8; B: u8; }
Wide :: union { A: Simd(f32, 4); B: u8; }

main :: () {
    print("Tiny % / %\n", size_of(Tiny), type_info(Tiny).alignment);
    print("Big  % / %\n", size_of(Big),  type_info(Big).alignment);
```

```
    print("Wide % / %\n", size_of(Wide), type_info(Wide).alignment);
}
```

CONSOLE / OUTPUT

```
Tiny 16 / 8
Big  32 / 8
Wide 32 / 16
```

NOTE

That output is for x86-64 Windows. On `x86-linux-musl` and `wasm32-freestanding`, `Tiny` is 8 / 4 and `Big` is 28 / 4. The payload offset is the tag size rounded up to the payload alignment, not an unconditional eight-byte prefix. A union holding `string` or `s64` has offset 4 on `x86-linux-musl` but offset 8 on `wasm32`, whose `s64` alignment is 8. Compile-time execution uses the output target's layout too.

- `Wide` shows the alignment rule: a 16-byte SIMD payload raises the record's alignment to 16, so the payload starts at offset 16 and the whole union is 32 bytes rather than 24.
- A union declared in a program never gets a niche. `union { A: void; B: *u8; }` is 16 bytes on x86-64 even though the pointer has a spare null representation. Only the compiler's own `?T` and `E!T` are laid out that way; see the chapters on the option and result types.
- There is no layout control. `union #packed { ... }` is `error: unexpected token 'packed'` and `union #c_call { ... }` is `error: unexpected token 'c_call'`. A union that must match a C layout is a raw union, covered in its own chapter.
- A default-initialized union, `s: Shape;`, holds the first declared variant with a zeroed payload.

Constructing a value

Six forms build a union value. Two name the union explicitly, two go through a variable that already carries the type, and two rely on an expected type to supply it.

GLOSSO

```
#import "Io";

Shape :: union { Circle: s64; Empty: void; }

tag_of :: (s: Shape) -> string {
    if s == { case .Circle; return "Circle"; case .Empty; return "Empty"; }
    return "?";
}

main :: () {
    a := Shape.Circle(1);          // qualified call
    b: Shape = .Circle(2);         // dotted call, type from the annotation
    c: Shape;  c.Circle = 3;       // variant assignment
    d: Shape;  d.Empty;            // bare variant statement
    e := Shape.Empty();            // qualified void variant, parentheses required
    f: Shape = .Empty;             // dotted void variant, no parentheses
    print("% % % % % %\n", tag_of(a), tag_of(b), tag_of(c), tag_of(d), tag_of(e), tag_of(f));
}
```

CONSOLE / OUTPUT

```
Circle Circle Circle Empty Empty Empty
```

Form	What it does
`Shape.Circle(1)`	Qualified call. Works anywhere; needs no expected type.
`.Circle(2)`	Dotted call. The expected type supplies the union.
`c.Circle = 3;`	Selects the variant and stores the payload in one step.
`d.Empty;`	Bare statement. Stores the tag of a `void` variant.
`Shape.Empty()`	Qualified `void` variant. The empty parentheses are required.
`f: Shape = .Empty;`	Dotted `void` variant. No parentheses.

The parentheses rule falls out of what the qualified form is. `Shape` is a type, and `Shape.Circle` asks for a member of it, which a type does not have; only the call `Shape.Circle(1)` is recognised as construction. So `x := Shape.Empty;` is `error: member of non-struct`, and so is `x := Shape.Circle;`. The dotted form is not a call at all, which is why `.Empty` stands alone. The same applies to the built-in option: write `(?s64).None()`, not `(?s64).None`.

- A dotted variant with no expected type has nothing to resolve against: `x := .Empty;` is `error: enum shorthand needs an enum context`.

- An unknown variant is `error: 'Q' is not a variant of union S` in the dotted call form and `error: no union field Q` in the assignment form.
- A `void` variant has no payload to assign: `s.Empty = 1;` is `error: assign ssize to void`. Use the bare statement `s.Empty;`.

Generic unions

A `$T` written in a variant's payload type introduces a type parameter, exactly as it does in a struct. The union is then applied like any other generic type, and it may refer to itself through a pointer.

GLOSSO

```
#import "Io";
#import "Meta";

Tree :: union {
    Leaf: $T;
    Pair: *mut Tree(T);
}

main :: () {
    t: Tree(s64) = .Leaf(4);
    n := 0;
    if t == { case .Leaf(v); n = v; case .Pair(p); n = -1; }
    print("% %\n", type_name(type_of(t)), n);
}
```

CONSOLE / OUTPUT

```
Tree_s64 4
```

- A parameter is introduced once with `$` and spelled as plain `T` everywhere after that, including in the union's own name. The instantiated type is named by mangling the arguments onto the declaration name, which is why `Tree(s64)` reports as `Tree_s64`.

Variant reflection

Three compile-time queries describe a union's variants in declaration order.

Call	Result
`variant_count(T)`	Number of variants, as an `ssize`.
`variant_name(T, i)`	The variant's name, as a `string`.
`variant_type(T, i)`	The payload type; `void` for a payload-less variant.

GLOSSO

```
#import "Io";
#import "Meta";

Vec :: struct { x: s64; y: s64; }

Shape :: union { Circle: s64; Box: Vec; Empty: void; }

main :: () {
    print("count %\n", variant_count(Shape));
    print("0: % %\n", variant_name(Shape, 0), type_name(variant_type(Shape, 0)));
    print("1: % %\n", variant_name(Shape, 1), type_name(variant_type(Shape, 1)));
    print("2: % %\n", variant_name(Shape, 2), type_name(variant_type(Shape, 2)));
}
```

CONSOLE / OUTPUT

```
count 3
0: Circle s64
1: Box Vec
2: Empty void
```

- The index must be a compile-time constant, so a runtime `for` over `variant_count` is `error: unknown const i`. An index past the last variant is `error: variant index 5 is out of range`.
- Three directives may follow the `union` keyword: `#raw`, `#derive(...)`, and the compiler-internal `#magic`. A single variant may additionally carry `#from`. Each has its own chapter.

5.8 Reading a Union Payload

A tagged union's payload is only readable through a pattern match. Writing `s.Circle` as an expression is a compile error even when the preceding line stored that very variant.

GLOSSO
```
#import "Io";

Shape :: union { Circle: s64; Box: s64; Empty: void; }

main :: () {
    s := Shape.Circle(5);
    print("%\n", s.Circle);
}
```

CONSOLE / OUTPUT
```
error: tagged-union payload '.Circle' may only be bound by a pattern match
note: match on the union and use that arm's payload capture, or enable #enable(allow_union_unwrap) for unsafe access
```

The tag is the only evidence that a payload is the one being read, and a member access has nowhere to put the test that consults it. A pattern does: the capture exists inside a branch that the match has already proved is the right one. Requiring the pattern makes every payload read dominated by a tag test by construction, instead of by the programmer's discipline.

The three legal readers

A switch arm, a pattern test, and a bind-or-diverge binding are the three shapes that produce a capture. All three bind the payload to a fresh name that is in scope only where the tag is known.

GLOSSO
```
#import "Io";

Shape :: union { Circle: s64; Box: s64; Empty: void; }

area :: (s: Shape) -> s64 {
    if s == {
        case .Circle(r); return 3 * r * r;
        case .Box(w);    return w * w;
        case .Empty;     return 0;
    }
    return -1;
}

is_unit :: (s: Shape) -> bool {
    if #pattern .Circle(r) = s  return r == 1;
    return false;
}

radius :: (s: Shape) -> s64 {
    #pattern .Circle(r) := s else { return -1; };
    return r;
}

main :: () {
    print("% % %\n", area(Shape.Circle(5)), is_unit(Shape.Circle(1)), radius(Shape.Box(9)));
}
```

CONSOLE / OUTPUT
```
75 1 -1
```

Reader	Shape
Switch arm	`case .Circle(r);` inside `if s == { ... }`, or the `if #pattern s == { ... }` and `ifx` forms.
Pattern test	`if #pattern .Circle(r) = s`, binding `r` for the branch taken.
Bind or diverge	`#pattern .Circle(r) := s else { ... };`, binding `r` for the rest of the block.

- A qualified pattern is accepted as well as a dotted one: `if #pattern Shape.Box(q) = s` reads the same as `if #pattern .Box(q) = s`, because a variant name resolves in the subject's namespace.
- An arm that only needs the tag omits the capture: `case .Circle;` is legal and binds nothing. In a pattern test, _ serves as the capture name when the shape must be written but the payload is unwanted: `if #pattern .Circle(_) = s`.
- The `else` of a `:=` binding must not fall through. Ending it without a jump is `error: the 'else' of a '#pattern ... :=' binding must not return normally` The failure path must end with `return`, `break`, `continue`, or a call that returns `#Never`.

The rule is a property of tagged unions, not of user declarations, so it reaches the built-in option and result types too. `o.Some` on a `?s64` is `error: tagged-union payload '.Some' may only be bound by a pattern match`, and `r.Ok` on an `E!s64` reports the same thing for `.Ok`.

What is exempt

Three cases are outside the rule, and each is exempt for a reason that is visible in the code.

GLOSSO

```
#import "Io";

Shape :: union { Circle: s64; Empty: void; }
Word  :: union #raw { i: s64; bytes: [8]u8; }

tag_of :: (s: Shape) -> string {
    if s == { case .Circle; return "Circle"; case .Empty; return "Empty"; }
    return "?";
}

main :: () {
    s: Shape;
    s.Circle = 7;        // write: selects the variant and stores the payload
    print("%\n", tag_of(s));
    s.Empty;             // bare statement: stores the tag
    print("%\n", tag_of(s));

    w: Word;
    w.i = 1;
    print("% %\n", w.i, w.bytes[0]);   // raw union: reads are unrestricted
}
```

CONSOLE / OUTPUT

```
Circle
Empty
1 1
```

- A write is variant selection, not a read. `s.Circle = 7;` sets the tag and the payload together, so there is no earlier tag to prove.
- A bare `s.Empty;` statement is a tag store for a `void` variant, and is flagged as a setter before the rule runs.
- A `#raw` union is exempt entirely. It has no tag, and it exists precisely for C-style reinterpretation, so there is nothing to prove and nothing to protect.
- A `void` payload gets no exemption of its own. The statement `s.Empty;` is a write, but `v := s.Empty;` is a read and is rejected like any other.
- Compound assignment is a read followed by a write, so it is rejected: `s.Circle += 1;` reports `error: tagged-union payload '.Circle' may only be bound by a pattern match`.

`#enable(allow_union_unwrap)`

The gate suspends the requirement to prove the tag statically. It is a section gate written at the top level of a file: `#enable` applies to every declaration that starts after it, and `#disable` restores the rule for everything that follows. Because it is stamped on whole declarations rather than on statements, it also reaches constant initializers and top-level `#comptime` blocks, which have no other escape hatch.

GLOSSO

```
#import "Io";

Shape :: union { Circle: s64; Box: s64; Empty: void; }

#enable(allow_union_unwrap);
peek :: (s: Shape) -> s64 { return s.Circle; }
#disable(allow_union_unwrap);

main :: () {
    print("%\n", peek(Shape.Circle(5)));
    print("wrong tag read: %\n", peek(Shape.Box(9)));
}
```

What the gate suspends is the compile-time proof, not the check. At `-O0` and `-O1` the read lowers to a tag test; the program above prints `5`, then prints the source position of the read followed by `union access: expected .Circle, but the value holds one of .Circle | .Box | .Empty`, and aborts. Hosted Windows and Linux x86-64 use exit status 134; WASI uses 125.

At `-O2` and above the check and its message are omitted. The same binary reads the payload bits of whatever variant is actually stored and exits normally.

CONSOLE / OUTPUT

```
5
wrong tag read: 9
```

> **NOTE**
> This is the sharpest edge in the area: a program that aborts loudly during development returns a wrong answer silently in a release build. The gate is for interoperability and for generated code, not for convenience; a pattern match is checked at every optimization level.

- `#enable` is a top-level directive. Inside a procedure body it is `error: unexpected directive 'enable'`.
- `#disable(allow_union_unwrap)` genuinely restores the rule: a declaration written after it that reads a payload is rejected again with the ordinary message.

When the tag is already known

If the compiler can see which variant the subject holds, the gate escalates to a compile error rather than emitting a check that could never pass. A check whose outcome is already decided is not a check; it is a fault that would fire on every execution.

GLOSSO

```
#import "Io";

Shape :: union { Circle: s64; Box: s64; Empty: void; }

A :: Shape.Circle(5);

#enable(allow_union_unwrap);
main :: () { print("%\n", A.Box); }
#disable(allow_union_unwrap);
```

CONSOLE / OUTPUT

```
error: this holds .Circle; reading .Box is never valid
note: the tag is known at compile time, so the gate's runtime check would fail on every execution
```

- The escalation only applies when the tag is a compile-time fact. A value that arrives at runtime keeps the runtime check, which is what the `peek` example above demonstrates.

5.9 `#from` Variants

`#from` is a per-variant directive on a tagged union. It marks one variant as the landing place for its own payload type, and the compiler generates a `Propagate` instance from that payload type into the union. A `Disk_Error` can then be widened into the union that carries it without a hand-written conversion.

The instance exists for `?`. When `?` breaks out of a function whose carrier residual differs from the operand's, the inherited operator makes a `propagate` call between the two residual types, and the Prelude lifts a `Propagate` between two error types to a `Propagate` between the matching `Result_Residual` types. `#from` is therefore what lets a function returning `App_Error!s64` call one returning `Disk_Error!s64` and write `?` after it. The propagation machinery itself is described in `Try`, `?`, and `#try` and in `Propagate` and Multi-Error Programs.

GLOSSO

```
App_Error :: union {
    #from parse: Parse_Error;
    #from disk:  Disk_Error;
    Refused: void;
}
```

The directive is written before the variant name and does not change the variant itself: `parse` and `disk` are ordinary variants with ordinary payloads, matched as `.parse(inner)` and `.disk(inner)`, and a union may mix `#from` variants with plain ones as `Refused` shows. What changes is that one extra declaration is injected per marked variant. The union above generates the equivalent of this, twice:

GLOSSO

```
(Disk_Error, App_Error) ::: instance Propagate {
    propagate :: (#empty: App_Error, value: Disk_Error) -> App_Error {
        out: App_Error;
        out.disk = value;
        return out;
    }
}
```

The generated body writes the variant rather than reading it. `out.disk = value;` is variant selection, which the payload rule exempts because an assignment chooses the tag instead of trusting it; see Reading a Union Payload. Writing this instance by hand instead of marking the variant works and is what `#from` saves.

A worked example

GLOSSO

```
#import "Io";

Parse_Error :: struct { line: s64; }
Disk_Error  :: struct { code: s64; }
```

```
App_Error :: union {
    #from parse: Parse_Error;
    #from disk:  Disk_Error;
    Refused: void;
}

fetch :: (ok: bool) -> Disk_Error!string {
    if !ok return .Err(.{ .code = 5 });
    return .Ok("42");
}

decode :: (text: string) -> Parse_Error!s64 {
    if text == "42" return .Ok(42);
    return .Err(.{ .line = 1 });
}

load :: (ok: bool) -> App_Error!s64 {
    text := fetch(ok)?;
    return .Ok(decode(text)?);
}

describe :: (value: App_Error!s64) -> string {
    if value == {
        case .Ok(n); return "ok";
        case .Err(e);
            if e == {
                case .parse(inner); return "parse";
                case .disk(inner);  return "disk";
                case .Refused;      return "refused";
            }
    }
}

main :: () {
    print("% %\n", describe(load(true)), describe(load(false)));
    widened := propagate(App_Error, Disk_Error.{ .code = 9 });
    print("%\n", describe((App_Error!s64).Err(widened)));
}
```

CONSOLE / OUTPUT

```
ok disk
disk
```

`load` propagates two different error types with the same operator. `fetch(ok)?` yields a `Disk_Error` residual and `decode(text)?` a `Parse_Error` residual; both are converted to an `App_Error` residual by the generated instances before the early return builds the `App_Error!s64` that `load` declares.

The generated instance is an ordinary one, so `propagate` may also be called directly. The witness parameter carries the target type, so the call is written `propagate(App_Error, value)`, as the last two lines of `main` show.

Without the directive

Dropping `#from` leaves the variants intact and removes the conversion. The `?` in `load` then fails its residual-conversion constraint, so no inherited operator overload accepts the call:

CONSOLE / OUTPUT

```
error: no overload '?' accepts these arguments
```

What a source type may be

The payload may be any type with a non-zero size: a struct, a scalar, a `string`. Nothing requires it to implement `Error`; that requirement belongs to `#derive(Error)`, which is a separate directive.

GLOSSO

```
Small :: union {
    #from n: s64;
    #from s: string;
    Other: void;
}

give :: (ok: bool) -> s64!bool {
    if !ok return .Err(7);
    return .Ok(true);
}

wrap :: (ok: bool) -> Small!bool {
    v := give(ok)?;
    return .Ok(v);
}
```

Calling `wrap(false)` produces `Small.n(7)` inside the `.Err` arm; calling `wrap(true)` produces `.Ok(true)`.

Rejections

Four shapes are refused outright, each with its own message. They are reported at the union declaration, not at the `?` that would have used the instance.

Declaration	Diagnostic
`union #raw { #from a: E1; }`	`error: #from is not defined for #raw union`
`union { #from a: void; }`	`error: #from variant 'a' must have a payload`
`union { #from a: E1; #from b: E1; }`	`error: #from variant 'b' duplicates the source type E1`
`union { #from a: $T; None: void; }`	`error: #from does not yet support generic unions`

A raw union has no tag, so there is no variant for the generated body to select; see Raw Unions. A `void` payload has nothing to convert from. Two variants with the same source type would produce two instances with identical heads, and instance selection keys on the type pair alone, so the pair would be ambiguous at every use.

`#from` is a union directive. On a struct field it does not reach the checker at all: `S :: struct { #from a: s64; }` is `error: unexpected token 'from'`.

Propagation is one hop

`Propagate` is not transitive. A union whose `#from` variant carries another `#from` union gets an instance for that one step only. A `?` that would need both conversions fails its operator constraint: the two generated instances cannot substitute for a direct one.

CONSOLE / OUTPUT

```
error: no overload '?' accepts these arguments
```

> **NOTE**
> Chaining requires a hand-written `Propagate` instance for the pair that is actually needed, or a `#from` variant on the outer union that names the innermost payload type directly. Adding intermediate unions does not compose.

With a derived `Error`

`#from` and `#derive(Error)` are the pair a multi-error program usually wants: the first makes `?` widen into the union, the second gives the union the `Error` instance that a `main` returning `App_Error!void` needs. `#derive(Error)` requires every variant to carry a payload that itself implements `Error`, which is why the union below has no payload-less variant.

GLOSSO

```
App_Error :: union #derive(Error) {
    #from missing: Missing;
    #from number:  Bad_Number;
}

main :: () -> App_Error!void {
    text := read_config(true)?;
    n := parse(text)?;
    print("n = %\n", n);
    bad := parse("nope")?;
    print("unreachable %\n", bad);
    return .Ok();
}
```

CONSOLE / OUTPUT

```
n = 17
bad number: nope
glosso: program exited with status exit code: 1
```

The second `?` breaks with a `Bad_Number`, the generated instance widens it to `App_Error`, and the `Termination` instance on `E!void` reports it through `Error` before turning it into status 1. That chain is covered in `Error`, `Termination`, and `main`.

5.10 Raw Unions

`union #raw { ... }` is the untagged union: the C `union`, with C's rules. It carries no tag word, every variant begins at offset zero, and any variant may be read at any time regardless of which one was written last. It exists so that reinterpretation of a value's bits and foreign declarations that use a C union can be expressed.

`#raw` is written after the `union` keyword and before the brace, in any order with the other union directives. Members follow the same rule as a tagged union's: each ends with `;`.

GLOSSO

```
#import "Io";
#import "Meta";

Word :: union #raw {
    i: s64;
    f: f64;
    bytes: [8]u8;
}

main :: () {
    print("size=% align=%\n", size_of(Word), type_info(Word).alignment);

    w: Word;
    w.f = 1.0;
    print("f=% i=%\n", w.f, w.i);
    print("bytes=% % % % % % % %\n",
        w.bytes[0], w.bytes[1], w.bytes[2], w.bytes[3],
        w.bytes[4], w.bytes[5], w.bytes[6], w.bytes[7]);

    q := Word.i(258);
    print("q.i=% q.bytes[0]=% q.bytes[1]=%\n", q.i, q.bytes[0], q.bytes[1]);
}
```

CONSOLE / OUTPUT

```
size=8 align=8
f=1 i=4607182418800017408
bytes=0 0 0 0 0 0 240 63
q.i=258 q.bytes[0]=2 q.bytes[1]=1
```

`w.i` after `w.f = 1.0` is the IEEE-754 bit pattern of `1.0` read as an integer, and `w.bytes` is the same eight bytes in memory order, little end first. Nothing about the read is a conversion: the three members name three ways of spelling the same eight bytes.

Size and alignment

The size of a raw union is the size of its largest member, rounded up to the union's alignment; the alignment is the largest member alignment. A `void` member contributes nothing. The tagged union with the same members costs an extra target-width tag and takes that tag's alignment.

GLOSSO

```
#import "Io";
#import "Meta";

Mixed  :: union #raw { a: u8; b: [3]u16; c: u32; }
Tagged :: union      { a: u8; b: [3]u16; c: u32; }

main :: () {
    print("raw    size=% align=%\n", size_of(Mixed), type_info(Mixed).alignment);
    print("tagged size=% align=%\n", size_of(Tagged), type_info(Tagged).alignment);
}
```

CONSOLE / OUTPUT

```
raw    size=8 align=4
tagged size=16 align=8
```

The largest member is `[3]u16`, six bytes with alignment 2, but `c` raises the alignment to 4, so the record rounds to 8. Struct layout, including the field reordering that raw unions have no equivalent of, is covered in Struct Memory Layout.

Construction and reading

The construction forms are the tagged union's, minus the ones that only set a tag. A qualified call needs its parentheses; an assignment to a member stores through it.

Form	Meaning
`q := Word.i(258);`	Qualified call; stores 258 through the `i` member.
`w: Word; w.i = 5;`	Member assignment; stores through `i` in place.
`v := w.f;`	Member read; reinterprets the stored bytes as an `f64`.

The read is the important difference. On a tagged union, naming a payload member is a compile error that redirects the program to a pattern match, and the value comes out only through a capture or the `#enable(allow_union_unwrap)` gate; see Reading a Union Payload. A raw union is exempt from that rule entirely, because the rule exists to stop a program from trusting a tag, and a raw union has no tag to trust.

> **NOTE**
> Nothing checks a raw-union read, at any optimization level. The tagged-union gate emits a runtime tag check at `-O0` and `-O1` and drops it at `-O2`; a raw union has no check to drop, so `-O0` and `-O2` produce identical results. Keeping the discriminant correct is entirely the program's job.

What a raw union gives up

Every feature that reads the tag is unavailable, and each is rejected by name rather than silently misbehaving.

Construct	Diagnostic
`if w == { case .i(v); ... }`	`error: cannot switch on a raw union (it has no tag)`
`union #raw #derive(Show)`	`error: Show derive is not defined for #raw union`
`union #raw #derive(PartialEq)`	`error: PartialEq derive is not defined for #raw union`
`union #raw #derive(Error)`	`error: Error derive is not defined for #raw union`
`union #raw { #from a: E1; }`	`error: #from is not defined for #raw union`

- Exhaustiveness has no meaning without a tag, so the checks described in Exhaustiveness, Guards, and Reachability never apply.
- `#pattern` against a raw union is not refused at check time. It reaches lowering, where the missing tag surfaces as an `error: internal error:` naming the `GetTag` opcode. Read the member directly instead.
- `#raw`, `#derive`, and `#magic` are the only union directives. `union #packed` is `error: unexpected token 'packed'` and `union #c_call` is `error: unexpected token 'c_call'`; a union has no field order to pack or to fix.
- Reflection still works, because it reads the declaration rather than a value: `variant_count(Word)` is 3 and `variant_name(Word, 0)` is `i`.

Carrying the tag by hand

The C idiom a raw union usually appears in is a struct that pairs an explicit discriminant with the raw payload. The discriminant is an ordinary enum, so it can be switched on with the exhaustiveness the raw union cannot offer, and the payload member read in each arm is the one that arm's tag value promises.

GLOSSO

```
#import "Io";
#import "Meta";
#import "Format";

Kind :: enum u8 { As_Int; As_Float; }

Payload :: union #raw {
    i: s64;
    f: f64;
}

Cell :: struct {
    kind: Kind;
    value: Payload;
}

render :: (c: Cell) -> string {
    if c.kind == {
        case .As_Int;   return tprint("int %", c.value.i);
        case .As_Float; return tprint("float %", c.value.f);
    }
    return "?";
}

main :: () {
    a: Cell;
    a.kind = .As_Int;
    a.value.i = 7;

    b: Cell;
    b.kind = .As_Float;
    b.value.f = 2.5;

    print("% / %\n", render(a), render(b));
    print("size=%\n", size_of(Cell));
}
```

CONSOLE / OUTPUT

```
int 7 / float 2.5
size=16
```

`Cell` is sixteen bytes: a one-byte tag padded out beside the eight-byte payload. The tagged union `union { As_Int: s64; As_Float: f64; }` is also sixteen, and it carries its own discriminant, is exhaustiveness-checked, and refuses a mismatched read. The hand-rolled pair buys nothing here, which is the general case: reach for `#raw` when a C declaration or a deliberate reinterpretation requires it, not to save space.

5.11 The Option Type `?T`

`?T` is the option type: a value of type `T`, or nothing. It is a compiler-built-in, injected the way `#Context` is rather than declared in any `.glo` file, so it is available in every program with no import. Its definition is exactly this two-variant union:

CONSOLE / OUTPUT

```
Option :: union #magic "option" {
    None: void;
    Some: $T;
}
```

That block is shown for reference and is not writable source. The name is the compiler's; `?T` is the only spelling a program has, and writing the identifier produces a parse error that says so:

CONSOLE / OUTPUT

```
error: the built-in option type is written '?T'; its variants are '.Some(x)' and '.None'
```

The compiler owns the declaration for two reasons. The `#magic "option"` marker is what buys the niche layout described below, which is the guarantee that non-null `*T` rests on. And the parser rewrites `?T` into this name, so a program able to redeclare or shadow the union would leave the syntax pointing at nothing.

`?` is a prefix type modifier

`?` has no precedence tier of its own. It is annotation syntax that rewrites to the built-in union, it applies to whatever type follows it, and it nests. It binds tighter than `!`, so in `?E1!s64` the `?` takes the error type and the result is a fallible `s64` whose error is optional.

GLOSSO

```
#import "Io";
#import "Meta";

E1 :: struct { code: s64; }

main :: () {
    a: ?s64;
    b: ??s64;
    c: ?*mut s64;
    d: ?E1!s64;
    print("?s64   = %\n", type_name(type_of(a)));
    print("??s64  = %\n", type_name(type_of(b)));
    print("?*mut s64  = %\n", type_name(type_of(c)));
    print("?E1!s64 = %\n", type_name(type_of(d)));
    print("size_of(?s64) = %\n", size_of(?s64));
}
```

CONSOLE / OUTPUT

```
?s64   = Option_s64
??s64  = Option_Option_s64
?*s64  = Option__s64
?E1!s64 = Result_s64_Option_E1
size_of(?s64) = 16
```

The mangled names show the tree directly. `Option_Option_s64` is `?(?s64)`, and `Result_s64_Option_E1` is a result whose payload is `s64` and whose error is `?E1`. Operator precedence generally is covered in Operator Precedence and the 0-20 Scale. One further property of `?` is that it parses where a type-valued expression is expected, so both `size_of(?s64)` and `size_of(?[*]s64)` work. Bracket types such as `[]u8`, `[4]s64`, `[*]u8`, and `[..]u8` are accepted directly as type values too.

`.Some` and `.None`

The variants are dotted. `.Some(x)`, `.None`, and a pattern's `.Some(binding)` all resolve in the subject's namespace, so no qualifier is needed wherever an expected type is present. Where one is needed, the sugar supplies it in parentheses.

Form	Meaning
`a: ?s64 = .Some(3);`	Dotted call against the annotation.
`b: ?s64 = .None;`	Dotted, no parentheses, for the empty variant.
`c := (?s64).Some(7);`	Qualified call where no expected type exists.

Form	Meaning
`d := (?s64).None();`	Qualified empty variant; the parentheses are required.

The asymmetry in the last two rows is the general union rule, not an option quirk: a qualified variant is a call and always needs its parentheses, so `(?s64).None` is `error: member of non-struct`. In the other direction a bare `.None` with no expected type at all is `error: enum shorthand needs an enum context`.

Reading the payload

`?T` is an ordinary tagged union as far as the payload rule is concerned, so `o.Some` is `error: tagged-union payload '.Some' may only be bound by a pattern match`. A switch arm, an `if #pattern` test, or a `#pattern ... :=` binding is how the value comes out; Reading a Union Payload covers all three and the gate that suspends the rule.

GLOSSO

```
#import "Io";

lookup :: (key: s64) -> ?s64 {
    if key == 1 return .Some(10);
    return .None;
}

doubled :: (key: s64) -> ?s64 {
    v := lookup(key)?;
    return .Some(v * 2);
}

strict :: (key: s64) -> s64 {
    #pattern .Some(v) := lookup(key) else { return -1; };
    return v;
}

main :: () {
    a: ?s64 = .Some(3);
    c := (?s64).Some(7);

    if a == {
        case .Some(v); print("switch captured %\n", v);
        case .None;    print("switch saw none\n");
    }
    if #pattern .Some(v) = c print("pattern captured %\n", v);

    print("not: % %\n", !a, !(?s64).None());
    print("propagated: % %\n",
        (doubled(1)).unwrap_or(-1), (doubled(2)).unwrap_or(-1));
    print("bind or diverge: % %\n", strict(1), strict(2));
}
```

CONSOLE / OUTPUT

```
switch captured 3
pattern captured 7
not: 0 1
propagated: 20 -1
bind or diverge: 10 -1
```

`!o` is the `LogicalNot` instance and means "is none". `?` on an option is the `Try` instance, whose `Output` is `T` and whose `Residual` is the one-variant enum `Option_Residual`, so `lookup(key)?` either yields the `s64` or returns `.None` from the enclosing function.

The null niche

When a built-in option or result has exactly two variants, one of them zero-sized and the other carrying a pointer-like payload, the tag word is dropped and the empty variant is represented by the null bit pattern. A valid pointer is never zero, so zero is a spare pattern the payload can never occupy, and dropping the tag stays sound. `?*T` is therefore exactly a `*T` in memory and crosses a C ABI boundary as an ordinary pointer.

GLOSSO

```
#import "Io";
#import "Meta";

Bits :: union #raw {
    opt: ?*mut s64;
    raw: u64;
}

main :: () {
    n: s64 = 11;
    b: Bits;

    b.opt = .None;
    print("None bits = %\n", b.raw);

    b.opt = &n;
    print("Some bits = zero? %\n", b.raw == 0);
```

```
    print("sizes: ?*mut s64=% *mut s64=% ?s64=% s64=%\n",
        size_of(?*mut s64), size_of(*mut s64), size_of(?s64), size_of(s64));

    p: ?*mut s64 = &n;
    q: ?*mut s64 = .None;
    print("null test: % %\n", p == null, q == null);
}
```

CONSOLE / OUTPUT

```
None bits = 0
Some bits = zero? 0
sizes: ?*s64=8 *s64=8 ?s64=16 s64=8
null test: 0 1
```

Reading the same eight bytes through a raw union shows that `.None` is literally address zero. Which types get the niche and which pay for a tag follows entirely from the pointer-like rule:

Type	Size	Layout
`?*T`, `?[*]T`, `?#fn_ptr(...)`	8	Niche; `.None` is address 0.
`?bool`, `?u8`, `?enum`	16	Tag plus payload.
`?[]T`	24	Tag plus a 16-byte view.
`?string`	24	Tag plus a 16-byte counted string view.
`??*T`	16	The inner takes the niche; the outer's payload is a union.

A user-written union never gets a niche, however its variants are shaped: only the compiler's `#magic "option"` and `#magic "result"` unions qualify. `union { A: void; B: *u8; }` is sixteen bytes.

Widening, and `null`

`*T` widens to `?*T` implicitly, in an assignment and at a call site, because the niche makes the conversion a no-op. Nothing else widens: `o: ?s64 = 5;` is `error: got ssize, expected Option_s64`, and the payload must be written `.Some(5)`. In the other direction `null` no longer reaches a plain pointer at all, and the diagnostic names `?*T` as the replacement. Pointers themselves are covered in Pointers and in Many-Pointers and Null.

CONSOLE / OUTPUT

```
error: 'null' is not a value of *ssize
note: a plain pointer excludes zero - write ?*ssize for a pointer that may be absent
note: its absent value is spelled '.None'
```

- `p == null` and `p != null` compile on a `?*T` and test for absence. On an option with a non-pointer payload there is no such operator: `error: no operator '==' for Option_s64, null`.
- Testing does not refine the type. After `if p != null`, `p.*` is `error: operator '.*' expects a typed pointer, got Option__s64`. A capture is the only route to the pointer.
- A declaration cannot use `null` as an option initializer even where the niche applies: `op: ?[*]s64 = null;` is `error: null is not a valid Option____s64`. Write `.None`.

The library surface

Everything except the union itself is ordinary Glosso in `std/Prelude/Option.glo`, and needs no import.

Declaration	Purpose
`(?T).make_none()`, `option_none(T)`	The empty value for an explicit type.
`(?T).make_some(value)`, `value.option_some()`	Wraps a value.
`value.is_some()`, `value.is_none()`	Inspect the current variant through a borrowed receiver.
`value.unwrap_or(fallback)`	Payload or eagerly evaluated fallback.

The instances are `LogicalNot`, `Try`, a hand-written `PartialEq` (`?T` is `#magic`, so it cannot carry `#derive`), and `Show` for any `T` that is itself `Show`. Two options compare equal when both are `.None` or both hold equal payloads.

> **NOTE**
> The default `Formatter` instantiates `Show` for `?T` directly. `print("%\n", o)` therefore prints `Some(3)` for an option holding `3`, or `None` for an empty option. An earlier direct `show` call is unnecessary. See `Show`, `FromString`, and `print` Formatting.

Options also support `map`, `map_or`, `and_then`, `or_else`, `filter`, `ok_or`, and `ok_or_else`. `unwrap_or_else` and the other `_else` forms call their fallback only when needed. `unwrap_or`, `map_or`, and `ok_or` evaluate the supplied fallback argument eagerly.

5.12 The Result Type `E!T`

`E!T` is the result type: a `T`, or an error of type `E`. It is the binary counterpart to `?T` and, like it, a compiler-built-in that no `.glo` file declares and every program has without an import. Its definition is this two-variant union:

CONSOLE / OUTPUT
```
Result :: union #magic "result" {
    Ok:  $T;
    Err: $E;
}
```

The sugar is error-first: the LEFT operand of `!` is what `.Err` carries and the right is what `.Ok` carries, so `Disk_Error!s64` is a fallible `s64`. The union's own parameters stay payload-first, as written above; the sugar is what swaps, not the declaration, which is why `type_name(Disk_Error!s64)` is `Result_s64_Disk_Error`.

Error-first is the order that reads. A signature ends on the type its success path hands back, which is the type the caller is looking for, and the error type sits next to the `!` that introduces it. It is also the order Zig's `E!T` and Haskell's `Either e a` use, and it is what makes right associativity the natural rule for a repeated `!`.

As with `?T`, the compiler's name for the union is not writable. `E!T` is the only spelling and the identifier is a parse error:

CONSOLE / OUTPUT
```
error: the built-in result type is written 'E!T'; its variants are '.Ok(x)' and '.Err(e)'
```

Precedence and associativity

`!` binds looser than every prefix type modifier, so `?`, `*`, `[]`, `[..]`, and `[*]` all stay glued to whichever operand they were written next to. It is right-associative, which is what error-first order asks for: `E1!E2!T` is `E1!(E2!T)`, each `!` wrapping the one payload in another result.

GLOSSO
```
#import "Io";
#import "Meta";

E1 :: struct { a: s64; }
E2 :: struct { b: s64; }
User :: struct { id: s64; }

main :: () {
    p: E1!?*mut User;
    q: ?E1!User;
    r: E1!E2!User;
    s: E1![]mut u8;
    t: []mut u8!E1;

    print("E1!?*mut User  = %\n", type_name(type_of(p)));
    print("?E1!User   = %\n", type_name(type_of(q)));
    print("E1!E2!User = %\n", type_name(type_of(r)));
    print("E1![]mut u8    = %\n", type_name(type_of(s)));
    print("[]mut u8!E1    = %\n", type_name(type_of(t)));
}
```

CONSOLE / OUTPUT
```
E1!?*User  = Result_Option__User_E1
?E1!User   = Result_User_Option_E1
E1!E2!User = Result_Result_User_E2_E1
E1![]u8    = Result___u8_E1
[]u8!E1    = Result_E1___u8
```

The mangled name is `Result_<payload>_<error>`, so each line can be read straight off:

Written	Reading
`E1!?*User`	A fallible optional pointer: error `E1`, payload `?*User`.
`?E1!User`	A result whose ERROR is optional, not an optional result.
`E1!E2!User`	`E1!(E2!User)`, from right associativity.

Written	Reading
`E1![]u8`	Error `E1`, payload `[]u8`.
`[]u8!E1`	Error `[]u8`, payload `E1`; the slice modifier stayed on the left.

> **NOTE**
> `!*` lexes as a single operator token, so `E1!*User` is `error: unexpected token '!*'` and `E1!**User` is `error: unexpected token '!**'`. Separate them with a space, as `E1! *User`, or write the pointer through a named alias. `!?` and `![` do not have this problem, because `?` is emitted alone and `[` is not an operator character.

`.Ok` and `.Err`

The variants are dotted and resolve in the subject's namespace, and the qualified form parenthesises the type exactly as an option's does. `.Ok()` with no argument is how an `E!void` reports success.

GLOSSO

```
#import "Io";

Bad :: struct { text: string; }

main :: () {
    a: Bad!s64 = .Ok(3);
    b: Bad!s64 = .Err(.{ .text = "nope" });
    c := (Bad!s64).Ok(7);
    d := (Bad!s64).Err(Bad.{ .text = "x" });

    if a == {
        case .Ok(n);  print("switch ok %\n", n);
        case .Err(e); print("switch err %\n", e.text);
    }
    if #pattern .Err(e) = b print("pattern err %\n", e.text);

    print("is_ok: % % % %\n", a.is_ok(), b.is_ok(),
        c.is_ok(), d.is_ok());
    print("not: % %\n", !a, !b);
    print("unwrap_or: % %\n", a.unwrap_or(-1), b.unwrap_or(-1));
}
```

CONSOLE / OUTPUT

```
switch ok 3
pattern err nope
is_ok: 1 0 1 0
not: 0 1
unwrap_or: 3 -1
```

`!r` is the `LogicalNot` instance and means "is err", the mirror of `!o` on an option. Payload reads obey the same rule as any tagged union: `r.Ok` is `error: tagged-union payload '.Ok' may only be bound by a pattern match`, and the legal readers are the ones in Reading a Union Payload.

`Try` and `?`

`E!T` carries a `Try` instance whose `Output` is `T` and whose `Residual` is `Result_Residual(E)`, a one-field struct holding the error. `x?` on a result therefore yields the payload or returns early with the error already wrapped in the enclosing function's carrier. Returning a bare `Output` from a result-returning function is also legal: `from_output` wraps it, so `return x + y;` means `return .Ok(x + y);`.

GLOSSO

```
#import "Io";
#import "Format";

Bad_Number :: struct { text: string; }

number :: (text: string) -> Bad_Number!s64 {
    parsed := from_string(s64, text);
    if parsed == {
        case .Some(n); return .Ok(n);
        case .None;    return .Err(.{ .text = text });
    }
}

sum_two :: (a: string, b: string) -> Bad_Number!s64 {
    x := number(a)?;
    y := number(b)?;
    return x + y;
}

report :: (r: Bad_Number!s64) -> string {
    if r == {
        case .Ok(n);  return tprint("ok %", n);
```

```
        case .Err(e); return tprint("bad %", e.text);
    }
}

main :: () {
    print("%\n", report(sum_two("12", "30")));
    print("%\n", report(sum_two("12", "3x")));
    inner := #try { number("4")? + number("5")? };
    print("try block: %\n", report(inner));
}
```

CONSOLE / OUTPUT

```
ok 42
bad 3x
try block: ok 9
```

`?` needs an enclosing Try boundary. Outside one it is `error: postfix '?' requires a Try-returning function or an enclosing #try block`; `#try { ... }` supplies a boundary for a single expression, as the last two lines show. The operator itself is documented in `Try`, `?`, and `#try`.

When the propagated error type differs from the enclosing function's, the inherited `?` body calls `propagate` between the two residuals. The Prelude lifts any `Propagate(From, To)` between error types to the corresponding `Result_Residual` pair, and `#from` variants on a union generate those instances; see `#from` Variants and `Propagate` and Multi-Error Programs.

A result-returning `main`

`$E!void ::: instance Termination where Error(E)` is what lets `main` return a result: `.Ok` becomes status 0, and `.Err` reports the error through its `Error` instance and becomes status 1. The constraint is real, so an `E` with no `Error` instance gives `error: main return type Result_void_E does not implement Termination`. Only `E!void` qualifies; a general `E!T` and `?T` do not.

GLOSSO

```
#import "Io";
#import "Typeclass/Error";

Missing :: struct { path: string; }

Missing ::: instance Error {
    report_error :: (e: Missing) -> void {
        ("missing").error_report_line(e.path);
    }
}

load_setting :: (ok: bool) -> Missing!s64 {
    if !ok return .Err(.{ .path = "config" });
    return .Ok(17);
}

main :: () -> Missing!void {
    n := load_setting(true)?;
    print("n = %\n", n);
    m := load_setting(false)?;
    print("unreachable %\n", m);
    return .Ok();
}
```

CONSOLE / OUTPUT

```
n = 17
missing: config
glosso: program exited with status exit code: 1
```

Size and niche

A result is a tag word plus the larger of its two payloads. With `Small :: struct { c: s64; }`, `size_of(Small!s64)` is 16; with `Bad :: struct { text: string; }`, whose `string` view is 16 bytes, `size_of(Bad!s64)` is 24. A result takes the niche under the same rule as an option, and either side may be the zero-sized one.

GLOSSO

```
#import "Io";
#import "Meta";

Nil :: struct { }

main :: () {
    a: *mut s64!Nil;
    b: []mut u8!Nil;
    print("% %\n", size_of(type_of(a)), size_of(type_of(b)));
    print("% %\n", type_name(type_of(a)), type_name(type_of(b)));
}
```

CONSOLE / OUTPUT

```
8 24
Result_Nil__s64 Result_Nil___u8
```

`*s64!Nil` has a zero-sized `.Ok` payload and a pointer `.Err` payload, so the tag goes and the whole result is one pointer word. `[]u8!Nil` does not qualify because a view is not pointer-like, and pays the tag.

- `E!T` has no `Show` instance, so a default `%` has no matching `Formatter` for the result. Match on the result and format the arms, or implement a selector for that result type.
- `E!T` has no `PartialEq` either, and unlike `Show` the omission is not caught in the checker: `a == b` on two results passes checking and fails during code generation with `error: Invalid operand types for ICmp instruction`.

The library surface

The Prelude provides result constructors and inherent methods without an import. `map` transforms a success payload; `map_error`, `bind_error`, and `ap_error` operate on the error side. `and_then` chains a fallible success callback. `ok()` and `err()` select one side as an option.

Declaration	Purpose
`(E!T).make_ok(value)`, `result_ok(E, value)`	Success with an explicit result or error type.
`(E!T).make_err(error)`, `result_err(T, error)`	Failure with an explicit result or payload type.
`value.is_ok()`, `value.is_err()`	Inspect the current variant through a borrowed receiver.
`value.unwrap_or(fallback)`	Payload or eagerly evaluated fallback.
`value.map_error(f)`	Rewrites the error type while preserving a success.

`unwrap_or_else` and `map_or_else` defer fallback work until it is needed. `unwrap_or` and `map_or` evaluate their fallback eagerly. `Result_Residual :: struct { error: $E; }` remains an ordinary type, so a hand-written `Propagate` instance can convert between residuals.

6 Pointers and collections

6.1 Pointers

*T points to one non-null T and is read-only by default. *mut T permits writes through the pointer. Both are one machine word and carry no count. Use [*]T for an unbounded run of elements and []T for a counted view.

```
GLOSSO
main :: () {
    value: s64 = 10;
    writable: *mut s64 = &value;
    writable.* = 20;
    readonly: *s64 = writable;
    copy := readonly.*;
}
```

Address-of and dereference

&value and value.& take an address. *pointer and pointer.* dereference it. Address-of requires addressable storage, such as a local, field, or array element. Dereferencing a read-only pointer permits reads; only a writable pointer permits assignment. The old address-of use of prefix * has been removed.

Member access does not implicitly dereference. Use p.*.field or (*p).field. Pointer receiver methods can use an existing compatible pointer or take the address of an addressable value; mutable receivers require writable storage.

Qualifiers and nested pointers

`*T`	read-only pointee
`*mut T`	writable pointee
`*volatile T`	read-only volatile access
`*mut volatile T`	writable volatile access

A writable pointer converts implicitly to a read-only pointer, preserving the pointee type. The reverse conversion is rejected. The const qualifier has been removed: omit it for read-only access and write mut for writable access.

**T is a read-only pointer to a read-only pointer. Write *mut *mut T when both levels must be writable. In **mut T, mut applies to the innermost pointee; the outer pointer remains read-only.

Nullability

*T excludes address zero. Use ?*T for an optional pointer: .None occupies the zero niche, so the optional has the same size as a pointer. Match the optional before dereferencing. cast(*T) of a known zero is rejected, and a runtime zero is checked at -O0 and -O1.

An unassigned non-null pointer uses an invalid nonzero sentinel, not useful storage. Initialize it before dereferencing. Pointers do not own their pointee and do not keep its storage alive.

Single-element pointer operations

A single pointer has dereference and address comparison, but no arithmetic or indexing. Many-pointers provide those operations. Use an explicit cast when crossing pointer shapes; pointer_punning relaxes shape checks without granting writable access to read-only storage.

6.2 Many-Pointers and Null

[*]T is a read-only many-pointer: one address with no element count. [*]mut T permits writes. These shapes support element indexing and arithmetic scaled by size_of(T). The caller must know that every accessed element exists; the compiler cannot bounds-check an unbounded pointer.

```
GLOSSO
main :: () {
    values: [3]s64 = .[10, 20, 30];
    writable: [*]mut s64 = values.data;
    writable[1] = 42;
    readonly: [*]s64 = writable;
    next := readonly + 1;
```

```
    distance := next - readonly;
}
```

A fixed array, view, dynamic array, or string exposes its data pointer through .data. Read-only storage yields a read-only pointer. A many-pointer carries no .count and cannot itself be sliced; slice the original counted array or view.

Non-null pointers and absence

Single pointers, many-pointers, and function pointers all exclude zero. The literal null cannot initialize a bare [*]T or #fn_ptr. Use ?[*]T or an optional function-pointer type for absence; their .None variant occupies address zero without adding storage.

```
GLOSSO
main :: () {
    maybe: ?[*]s64 = .None;
    values: [2]s64 = .[1, 2];
    maybe = values.data;
    if #pattern .Some(pointer) = maybe {
        first := pointer[0];
    }
}
```

Match an optional pointer before using its payload. A cast from a runtime integer to a bare pointer checks zero at -O0 and -O1; a known zero is rejected at compile time. An uninitialized non-null pointer is an invalid sentinel, not valid storage.

Converting pointer shapes

Single pointers and many-pointers do not convert implicitly. Use cast([*]T) or cast(*T) when the backing storage justifies the new shape. Writable forms can be used through their matching read-only form; adding writable access implicitly is rejected.

The pointer_punning feature gate relaxes pointee and pointer-shape checks but does not permit read-only storage to acquire writable access. Keep casts and gates adjacent to the code that needs them. Neither a pointer nor its optional form owns the pointee or extends its lifetime.

6.3 Fixed Arrays, Views, and Dynamic Arrays

[N]T stores N elements inline. []T borrows a counted read-only run; []mut T permits writes. [..]T is a move-only owner of a growable allocation. [_]T infers the fixed length from the initializer, including b"..." byte-array literals.

`[N]T`	inline storage, `N * size_of(T)` bytes
`[]T`, `[]mut T`	count followed by data, two native words
`[..]T`	count, data, capacity, three native words

An unallocated dynamic array has zero count and capacity. Declare one without an initializer, then call push(), append(), or reserve(). reserve() requests total capacity and does not change count. push() appends one or more values and grows geometrically.

```
GLOSSO
main :: () {
    values: [..]s64;
    values.reserve(8);
    values.push(1, 2, 3);
    view: []s64 = values.view();
    writable: []mut s64 = values.view();
    writable[0] = 10;
    // Scope exit destroys the array and frees its buffer.
}
```

A dynamic array owns its live elements. Passing it by value moves it; pointers and views borrow it. Iterate through managed elements with for *item: values. take() transfers storage and leaves an empty reusable source; replace(new_array) installs a moved array and returns the previous allocation.

free() and deinit() destroy live elements and release storage, leaving an empty array. They require infallible element destruction. reset() retains allocation and is available only when elements do not need destruction. release_storage() frees only storage and is for code that has already managed every element's lifetime.

Dynamic arrays allocate and release through context.allocator. Keep that allocator installed through destruction. Standard container constructors instead capture their supplied allocator.

Views and invalidation

A view copies a pointer and count; it does not copy elements or update its count after mutation. Growth may relocate storage. Freeing, replacing, or destroying an array can leave existing views dangling. Memory analysis diagnoses tracked invalidation and dangling views; use --borrow-check=error to reject findings. Keep backing storage valid for every access.

Writable views convert to read-only views. Read-only views cannot be used as writable ones. Indexing fixed arrays, slices, dynamic arrays, and counted strings is checked at -O0 and -O1; [*]T carries no count and is never bounds-checked.

6.4 Ranges and Slicing

`..` builds a range that excludes its upper bound and `..=` builds one that includes it. Both are ordinary operators declared in the prelude at precedence 2, left-associative, so they bind looser than arithmetic: `0..n - 1` is `0..(n - 1)`. There is no `..<`; writing it is `error: unexpected token '<'`.

Either bound may be omitted, and the omitted bound changes the type rather than filling in a default. That is what lets `a[2..]` and `a[..3]` mean different things to the same container without any special syntax in the index: five ranges are five distinct struct types, and slicing is an `Indexable` instance for each.

Written	Type	Fields
`lo..hi`	`Range(T)`	`start`, `end`
`lo..=hi`	`RangeInclusive(T)`	`start`, `end`
`lo..`	`RangeFrom(T)`	`start`
`..hi`	`RangeTo(T)`	`end`
`..=hi`	`RangeToInclusive(T)`	`end`
`..` in an index	`RangeFull(T)`	none; the type is zero-sized

GLOSSO

```
#import "Io";
#import "Meta";

main :: () {
    r   := 2..5;
    ri  := 2..=5;
    rf  := 2..;
    rt  := ..5;
    rti := ..=5;
    print("2..5   % start=% end=% size=%\n",
        type_name(type_of(r)), r.start, r.end, size_of(type_of(r)));
    print("2..=5  % start=% end=% size=%\n",
        type_name(type_of(ri)), ri.start, ri.end, size_of(type_of(ri)));
    print("2..    % start=% size=%\n",
        type_name(type_of(rf)), rf.start, size_of(type_of(rf)));
    print("..5    % end=% size=%\n",
        type_name(type_of(rt)), rt.end, size_of(type_of(rt)));
    print("..=5   % end=% size=%\n",
        type_name(type_of(rti)), rti.end, size_of(type_of(rti)));
}
```

CONSOLE / OUTPUT

```
2..5   Range_ssize start=2 end=5 size=16
2..=5  RangeInclusive_ssize start=2 end=5 size=16
2..    RangeFrom_ssize start=2 size=8
..5    RangeTo_ssize end=5 size=8
..=5   RangeToInclusive_ssize end=5 size=8
```

The bound type is carried through: `lo..hi` over two `u8` values is a `Range(u8)`, not the `Range(ssize)` inferred from untyped integer bounds. The operators are provided by instances constrained on `Step`, which the integer types satisfy and the float types do not: `0.0..1.0` is `error: no operator '..' for f64, f64`.

GLOSSO

```
#import "Io";

main :: () {
    n := 5;
    r := 0..n - 1;
    print("0..n - 1 is %..%\n", r.start, r.end);
    s := 1 + 1..n * 2;
    print("1 + 1..n * 2 is %..%\n", s.start, s.end);
}
```

CONSOLE / OUTPUT
```
0..n - 1 is 0..4
1 + 1..n * 2 is 2..10
```

Ranges are values

A range is an ordinary struct value. It can be stored in a variable, passed to a procedure, returned, and used as an index later; `a[r]` for a stored `r` means exactly what `a[2..5]` would have meant. `RangeFull(T)` has no fields and `size_of` is 0, so an all-elements index costs nothing to carry.

GLOSSO
```
#import "Io";
#import "Meta";

main :: () {
    a: [6]s64 = .[10, 20, 30, 40, 50, 60];
    r := 1..4;
    print("stored Range: count=% first=%\n", a[r].count, a[r][0]);
    full: RangeFull(s64);
    print("stored RangeFull: count=% size=%\n", a[full].count, size_of(type_of(full)));
    lo: u8 = 1;
    hi: u8 = 4;
    typed := lo..hi;
    print("% count=%\n", type_name(type_of(typed)), a[typed].count);
}
```

CONSOLE / OUTPUT
```
stored Range: count=3 first=20
stored RangeFull: count=6 size=0
Range_u8 count=3
```

Ranges have no `Show` instance, so `print("%\n", r)` has no matching default `Formatter` for `Range_ssize`. The fields print individually.

The slice spellings

Indexing a container with a range slices it. Each spelling is a separate `Indexable` instance keyed on the container type and the range type, which is why all six work uniformly on fixed arrays, views, dynamic arrays, and strings. Indexing and Slicing Operators describes the typeclass; the table below is what the six mean.

Form	Range type	Elements
`a[lo..hi]`	`Range`	`hi - lo`, starting at `lo`
`a[lo..=hi]`	`RangeInclusive`	`hi - lo + 1`, starting at `lo`
`a[lo..]`	`RangeFrom`	from `lo` to the last element
`a[..hi]`	`RangeTo`	from 0, stopping before `hi`
`a[..=hi]`	`RangeToInclusive`	from 0, through `hi`
`a[..]`	`RangeFull`	every element

GLOSSO
```
#import "Io";

dump :: (label: string, v: []mut s64) {
    print("% count=%", label, v.count);
    for x: v print(" %", x);
    print("\n");
}

main :: () {
    a: [6]s64 = .[10, 20, 30, 40, 50, 60];
    dump("a[1..4] ", a[1..4]);
    dump("a[1..=4]", a[1..=4]);
    dump("a[..]   ", a[..]);
    dump("a[2..]  ", a[2..]);
    dump("a[..3]  ", a[..3]);
    dump("a[..=3] ", a[..=3]);
}
```

CONSOLE / OUTPUT
```
a[1..4]  count=3 20 30 40
a[1..=4] count=4 20 30 40 50
a[..]    count=6 10 20 30 40 50 60
a[2..]   count=4 30 40 50 60
```

```
a[..3]   count=3 10 20 30
a[..=3]  count=4 10 20 30 40
```

What a slice produces

Slicing a fixed array, a view, or a dynamic array all give `[]T`, and none of the three copies an element: the result is a count and a pointer into the original storage. Slicing `string` or `cstring` gives a borrowed `string`; slicing either UTF-16 form gives `string16`. A many-pointer cannot be sliced at all, because it has no length to bound the result: `mp[1..3]` is `error: no operator '[]' for [*]s64, Range_ssize`.

GLOSSO

```
#import "Io";
#import "Meta";

main :: () {
    a: [6]s64 = .[10, 20, 30, 40, 50, 60];
    v := a[..];
    d: [..]s64;
    defer d.free();
    d.push(1, 2, 3, 4);
    s := "hello world";
    print("view sliced:   count=% type=%\n", v[2..4].count, type_name(type_of(v[2..4])));
    print("dyn sliced:    count=% type=%\n", d[1..].count, type_name(type_of(d[1..])));
    print("string sliced: '%' type=%\n", s[0..5], type_name(type_of(s[0..5])));
}
```

CONSOLE / OUTPUT

```
view sliced:   count=2 type=[]s64
dyn sliced:    count=3 type=[]s64
string sliced: 'hello' type=string
```

`type_name` and diagnostics preserve `[]s64`; a slice whose element is the native integer instead prints `[]ssize`. A slice of a dynamic array is a borrow, so a later `array.push` invalidates it: see Aliasing, Invalidation, and Storage Generations.

Slicing is bounds-checked

A slice is checked like an index. The range's bounds are compared against the container's own count at `-O0` and `-O1`, and the slice prints `runtime check failed: slice.bounds` and aborts when the start is negative, when the end runs past the count, or when the end precedes the start. An empty slice whose bounds are equal and within the count is legal, and so is one that ends exactly at the count. At `-O2` and above the check is not emitted and the descriptor is built from the bounds as written. In compile-time evaluation the same failure is `error: slice 0..5 is out of bounds`.

GLOSSO

```
#import "Io";

main :: () {
    a: [3]s64 = .[1, 2, 3];
    print("a[2..2] count=%\n", a[2..2].count);
    print("a[0..3] count=%\n", a[0..3].count);
    print("a[1..]  count=%\n", a[1..].count);
    past := a[0..5];
    print("unreachable %\n", past.count);
}
```

CONSOLE / OUTPUT

```
a[2..2] count=0
a[0..3] count=3
a[1..]  count=2
runtime check failed: slice.bounds
glosso: program exited with status exit code: 134
```

> **NOTE**
> The guard is a runtime check like the others, so it disappears at `-O2` and above. A slice bound that comes from input, arithmetic, or a search result should still be clamped against `.count` before it reaches the brackets; an optimized build will not do it.

Ranges as iterables

A `for` loop accepts a range directly, and the open-ended shapes iterate too: `RangeTo` and `RangeToInclusive` start from the type's origin, which is 0 for the integers, and `RangeFrom` never terminates on its own. `for` Loops and Iteration and Range Typeclasses cover the loop forms and the `Step` and `RangeOrigin` contracts behind them.

GLOSSO

```
#import "Io";

main :: () {
    print("0..3 ");
    for i: 0..3 print("% ", i);
    print("\n0..=3 ");
    for i: 0..=3 print("% ", i);
    print("\n..4 ");
    for i: ..4 print("% ", i);
    print("\n..=2 ");
    for i: ..=2 print("% ", i);
    print("\n10.. ");
    for i: 10.. { print("% ", i); if i == 13 break; }
    print("\n");
}
```

CONSOLE / OUTPUT

```
0..3 0 1 2
0..=3 0 1 2 3
..4 0 1 2 3
..=2 0 1 2
10.. 10 11 12 13
```

6.5 Structure-of-Arrays and Array-of-Structs

`#soa` and `#aos` are prefixes written in front of an array type. They select how the elements are stored without changing the element type or anything written at a use site. `#aos` stores whole elements one after another; `#soa` stores each field of the element in its own contiguous run.

`#aos` is the layout every array already has, so the prefix exists only to state the choice in the source. `#soa` is the one that changes the representation: the descriptor stops carrying a single `data` pointer and carries one pointer per storage field of the element type instead.

GLOSSO

```
#import "Io";
#import "Meta";

Particle :: struct {
    id: u8;
    x:  s64;
    y:  s64;
}

main :: () {
    aos: #aos [..]Particle;
    soa: #soa [..]Particle;
    defer aos.free();
    defer soa.free();

    aos.push(Particle.{ 1, 10, 20 });
    soa.push(Particle.{ 1, 10, 20 });
    aos.push(Particle.{ 2, 30, 40 });
    soa.push(Particle.{ 2, 30, 40 });

    print("aos: count=% [1].y=%\n", aos.count, aos[1].y);
    print("soa: count=% x[1]=%\n", soa.count, soa.x[1]);
    print("descriptors: aos=% soa=%\n", size_of(type_of(aos)), size_of(type_of(soa)));
}
```

CONSOLE / OUTPUT

```
aos: count=2 [1].y=40
soa: count=2 x[1]=30
descriptors: aos=24 soa=40
```

The descriptor sizes follow from that. `Particle` has three storage fields, so its `#soa` descriptor holds three data pointers where the `#aos` descriptor holds one. The pointer count comes from the laid-out field list, so an `#empty` marker field contributes none; the plain array shapes are described in the chapter on fixed arrays, views, and dynamic arrays.

Type	Descriptor	Size
`[..]T`	`count`, `data`, `capacity`	24
`#soa [..]T` over `k` fields	`count`, `k` pointers, `capacity`	`8 + 8k + 8`
`#soa []T` over `k` fields	`count`, `k` pointers	`8 + 8k`

Where the prefixes may be written

Both prefixes apply to an array type and to nothing else. `#soa` is further restricted to the two descriptor-carrying forms, `[..]T` and `[]T`, since those are the only shapes with a header that can hold more than one pointer. Everything else is rejected by name.

Written	Result
`#soa [..]T`	Accepted; per-field runs, plus a count and a capacity.
`#soa []T`	Accepted; per-field pointers, plus a count.
`#soa [N]T`	`error: #soa fixed arrays are not supported`
`#soa [*]T`	`error: #soa applies to an array type`
`#soa T` for a non-array	`error: #soa applies to an array type`

The element type must be a struct, since a scalar has no fields to split: `#soa [..]s64` is `error: #soa arrays require a struct element type, got s64`. `#aos` carries no restriction at all; it is accepted on `[N]T`, `[]T`, and `[..]T` alike and changes nothing about any of them.

Element syntax is unchanged

Nothing at a use site distinguishes the two layouts. Elements are appended with `array.push` and a whole element value, read back with `array[i]`, and iterated with `for`, which yields reassembled elements and an optional index. A `#soa [..]T` is also an ordinary struct field type.

GLOSSO

```
#import "Io";
Particle :: struct { id: u8; x: s64; y: s64; }
World :: struct { particles: #soa [..]Particle; }

main :: () {
    w: World;
    defer w.particles.free();
    w.particles.push(Particle.{ 1, 10, 20 });
    w.particles.push(Particle.{ 2, 30, 40 });

    for e: w.particles print("elem % %\n", e.id, e.x);
    for e, i: w.particles print("[%] %\n", i, e.y);
}
```

CONSOLE / OUTPUT

```
elem 1 10
elem 2 30
[0] 20
[1] 40
```

Per-field views

On a `#soa` array a field name of the element type reads as a view of that field's run: `soa.x` has type `[]s64` and length `soa.count`. The view aliases the array rather than copying it, so a store through the view is a store into the array. Indexing the array itself gathers a whole element on a read and scatters on a write, whether the write names one field or replaces the element.

GLOSSO

```
#import "Io";
#import "Meta";

Particle :: struct { id: u8; x: s64; y: s64; }

main :: () {
    soa: #soa [..]Particle;
    defer soa.free();
    soa.push(Particle.{ 1, 10, 20 });
    soa.push(Particle.{ 2, 30, 40 });

    xs := soa.x;
    print("field view: type=% count=% size=%\n",
          type_name(type_of(xs)), xs.count, size_of(type_of(xs)));

    soa.y[1] = 99;             // write through the field view
    e := soa[1];               // gather one whole element
    print("gathered: id=% x=% y=%\n", e.id, e.x, e.y);

    soa[0] = Particle.{ 7, 70, 700 };   // scatter a whole element
    soa[0].y = 7000;                    // scatter one field
    print("scattered: id=% x=% y=%\n", soa.id[0], soa.x[0], soa.y[0]);

    print("raw run: %\n", soa.x.data[1]);
}
```

CONSOLE / OUTPUT

```
field view: type=[]s64 count=2 size=16
gathered: id=2 x=30 y=99
scattered: id=7 x=70 y=7000
raw run: 30
```

`soa.data` is `error: #soa array has per-field data`: there is no single run for it to name. A raw pointer comes from one field view instead, as `soa.x.data`, which is an ordinary `[*]s64` over that field's run.

The split is one level deep. A field whose own type is a struct becomes a view of that struct; it is not decomposed further.

GLOSSO

```
#import "Io";
#import "Meta";
V2   :: struct { x: f64; y: f64; }
Body :: struct { pos: V2; mass: f64; }

main :: () {
    bodies: #soa [..]Body;
    defer bodies.free();
    bodies.push(Body.{ V2.{ 1.0, 2.0 }, 10.0 });
    bodies.push(Body.{ V2.{ 3.0, 4.0 }, 20.0 });

    print("pos view: % / mass view: %\n",
          type_name(type_of(bodies.pos)), type_name(type_of(bodies.mass)));
    print("bodies.pos[1].x=%\n", bodies.pos[1].x);
}
```

CONSOLE / OUTPUT

```
pos view: []V2 / mass view: []f64
bodies.pos[1].x=3
```

NOTE

The descriptor's own members win over element fields of the same name. On a `#soa [..]P` whose `P` declares `count: s64`, `d.count` is the element count and not a view; that field is reachable only through a gathered element, as `d[0].count`. The same shadowing applies to `capacity` and to `data`.

`#soa []T` as a parameter

`array.view` on a `#soa [..]T` yields a `#soa []T`, and a parameter declared `#soa []T` receives it. The prefix is part of the parameter type, so a routine written against per-field views states that in its signature, and a plain `[]T` parameter will not accept the value.

GLOSSO

```
#import "Io";
#import "Meta";

Particle :: struct { id: u8; x: s64; y: s64; }

total_x :: (v: #soa []mut Particle) -> s64 {
    total: s64 = 0;
    for i: 0..v.count total += v.x[i];
    return total;
}

scale_y :: (v: #soa []mut Particle, k: s64) {
    for i: 0..v.count v.y[i] *= k;
}

main :: () {
    soa: #soa [..]Particle;
    defer soa.free();
    soa.push(Particle.{ 1, 10, 20 });
    soa.push(Particle.{ 2, 30, 40 });

    v := soa.view();
    print("view type=% size=%\n", type_name(type_of(v)), size_of(type_of(v)));
    print("total_x=%\n", total_x(v));
    scale_y(v, 10);
    print("y=% %\n", soa.y[0], soa.y[1]);
}
```

CONSOLE / OUTPUT

```
view type=[]Particle size=32
total_x=40
y=200 400
```

The element type may be inferred, so `f :: (v: #soa []$T)` is a generic routine over any struct-element SoA view. Diagnostics render both view kinds as `[]P`, which makes a mismatch read as `error: arg type []P != []P in call take_plain`; the prefix is the difference the message is not showing.

> **NOTE**
> A structure-of-arrays descriptor cannot be sliced as an ordinary array of structs. Use array.view() for its #soa view, or slice one field's column, such as array.x[lo..hi]. Ordinary array slicing is rejected because each element field lives in its own buffer.

6.6 Aggregate and Collection Literals

A literal that names no type is written with a leading dot and brackets chosen by what it builds: `.{ }` for an aggregate, `.[ ]` for an array, and `.( )` for a tuple. The tuple form belongs to the chapter on tuples. `.{ }` also has a qualified spelling, `T.{ }`, which carries its own type and therefore needs no context.

Inside `.{ }`, elements are either positional or named. Positional elements follow declaration order, which is not the same as storage order; named elements are written `.field = value`. A literal may stop early, may mix the two so long as no field is written twice, and may be empty, in which case every field takes its declared default. Separate elements with commas; a trailing comma is allowed. Array literals likewise require a comma between elements.

GLOSSO

```
#import "Io";

Vec2 :: struct #derive(Show) { x: s64 = 0; y: s64 = 0; }

main :: () {
    a := Vec2.{ 10, 20 };        // qualified: the type is written
    b: Vec2 = .{ 30, 40 };       // inferred: the annotation supplies it
    c: Vec2 = .{ .y = 7 };       // named, partial; x keeps its default
    d: Vec2 = .{ 1, .y = 2 };    // positional then named
    e: Vec2 = .{};               // every field takes its default
    print("% % % % %\n", a, b, c, d, e);
}
```

CONSOLE / OUTPUT

```
{10, 20} {30, 40} {0, 7} {1, 2} {0, 0}
```

- More positional elements than fields is `error: no field '' in struct literal for Vec2`; the empty name is the field that was not found.
- A name that is not a field is `error: no field 'z' in struct literal for Vec2`.
- Writing one field twice, whether by two names or by a positional element that a named one repeats, is `error: field 'x' is initialized more than once`.
- Fields a literal omits take their declared defaults, and fields with no default are zeroed, exactly as a bare `v: Vec2;` initializes them.

Where the expected type comes from

The inferred form is legal wherever checking already knows the type the expression must produce. That is a fixed list of positions, and each one serves `.{ }` and `.[ ]` alike.

Position	Example
Annotated declaration	`b: Vec2 = .{ 30, 40 };`
Assignment to a typed place	`v = .{ 50, 60 };`
Argument to a declared parameter	`span(.{ 5, 5 })`
A parameter's default value	`f :: (v: Vec2 = .{ 0, 0 })`
Return with a declared result type	`return .{ .a = .{ 1, 2 } };`
A struct field's default	`origin: [2]s64 = .[ 3, 4 ];`
An element of an enclosing literal	`.[ .{ 1, 1 }, .{ 2, 2 } ]`
A union variant payload	`o: ?Vec2 = .Some(.{ 5, 5 });`

> **NOTE**
> An `ifx` arm may need its own type: qualify its literal as `Vec2.{ ... }` rather than relying on the surrounding target annotation.

A variadic argument of known element type	`d.push(.{ 1, 2 })`

GLOSSO

```
#import "Io";
#import "Meta";

Vec2 :: struct { x: s64; y: s64; }
Line :: struct { a: Vec2; b: Vec2; }
Grid :: struct { cells: [3]s64; name: string; }

ORIGIN :: Vec2.{ 0, 0 };
CAMERA: Vec2 = .{ 8, 9 };

span :: (v: Vec2) -> s64 { return v.x + v.y; }
diagonal :: () -> Line { return .{ .a = .{ 1, 2 }, .b = .{ 3, 4 } }; }

main :: () {
    print("argument: %\n", span(.{ 5, 5 }));
    print("return endpoints: % %\n", diagonal().a.x, diagonal().b.y);
    print("constant x: % global x: %\n", ORIGIN.x, CAMERA.x);

    pts: [2]Vec2 = .[ .{ 1, 1 }, .{ 2, 2 } ];
    g: Grid = .{ .cells = .[ 7, 8, 9 ], .name = "g" };
    m: [2][3]s64 = .[ .[ 1, 2, 3 ], .[ 4, 5, 6 ] ];
    print("nested: % % % %\n", pts[1].x, g.cells[2], g.name, m[1][0]);

    o: ?Vec2 = .Some(.{ 5, 5 });
    print("union payload: %\n", type_name(type_of(o)));

    v: Vec2;
    v = .{ 50, 60 };
    print("assignment: % %\n", v.x, v.y);
}
```

CONSOLE / OUTPUT

```
argument: 10
return endpoints: 1 4
constant x: 0 global x: 8
nested: 2 9 g 4
union payload: Option_Vec2
assignment: 50 60
```

Nesting works because each inner literal is checked against the field or element type the outer one already fixed. `.{ .a = .{ 1, 2 } }` needs no inner type name, and `.[ .[ 1, 2, 3 ], .[ 4, 5, 6 ] ]` builds a `[2][3]s64` from the annotation alone.

GLOSSO

```
#import "Io";
Vec2 :: struct { x: s64; y: s64; }

with_default :: (v: Vec2 = .{ 0, 0 }) -> s64 { return v.x; }

main :: () {
    d: [..]Vec2;
    defer d.free();
    d.push(.{ 1, 2 }, .{ 3, 4 });

    s: #soa [..]Vec2;
    defer s.free();
    s.push(.{ 7, 8 });

    print("variadic x: % %\n", d[0].x, d[1].x);
    print("soa: %\n", s.x[0]);
    print("default parameter: % %\n", with_default(), with_default(.{ 42, 0 }));

    flag := true;
    w: Vec2 = ifx flag Vec2.{ 1, 0 } else Vec2.{ 0, 1 };
    print("ifx arm: % %\n", w.x, w.y);
}
```

CONSOLE / OUTPUT

```
variadic x: 1 3
soa: 7
default parameter: 0 42
ifx arm: 1 0
```

Where the qualified form is required

Outside that list there is no expected type, and a bare `.{ }` is `error: struct literal needs a type`. `a := .{ 1, 2 };` is the everyday case: `:=` takes its type from the initializer, so the initializer cannot take its type from `:=`. A literal handed to an inferred generic parameter has no expected type either, and the call reports `error: no overload 'generic' accepts these arguments`.

NOTE
A qualified literal accepts a struct type value, including a generic application such as `Box(f64).{ .value = 1.5 }` or an alias such as `Alias.{ 1, 2 }`. The qualifier fixes the type without an annotation on the variable. An annotation with the inferred `.{ ... }` form remains useful when the surrounding declaration already names the type.

GLOSSO
```
#import "Io";
Box   :: struct { value: $T; }
Pair  :: struct { a: s64; b: s64; }
Alias :: Pair;

main :: () {
    b := Box(f64).{ .value = 1.5 };
    p := Alias.{ 1, 2 };
    print("% % %\n", b.value, p.a, p.b);
}
```

CONSOLE / OUTPUT
```
1.5 1 2
```

Collection literals

`.[ ]` builds a fixed array. Against an annotation the element count must equal the declared count exactly, and that count may be spelled by a constant. With no annotation the literal synthesizes `[N]T` from what it holds, which for bare integer elements means an element type of `comptime_int`; compile-time numbers and their defaulting have their own chapter. The expected fixed-array type is also preserved through `#comptime`, so `a: [3]ssize = #comptime .[1, 2, 3];` has exactly the type named by the annotation.

GLOSSO
```
#import "Io";
#import "Meta";

N :: 3;
Cfg :: struct { origin: [2]s64 = .[ 3, 4 ]; }

main :: () {
    fixed: [3]s64 = .[ 1, 2, 3 ];
    named: [N]s64 = .[ 4, 5, 6 ];
    lanes: Simd(f32, 4) = #simd .[ 1.0, 2.0, 3.0, 4.0 ];
    inferred := .[ 10, 20, 30 ];
    trailing: [3]s64 = .[ 1, 2, 3, ];
    c: Cfg;

    print("fixed=% named=% lane0=%\n", fixed[2], named[0], lanes[0]);
    print("inferred: count=% type=%\n", inferred.count, type_name(type_of(inferred)));
    print("trailing=% field default=%\n", trailing[2], c.origin[1]);
}
```

CONSOLE / OUTPUT
```
fixed=3 named=4 lane0=1
inferred: count=3 type=[3]comptime_int
trailing=3 field default=4
```

- A wrong count is `error: fixed array literal has 2 element(s), expected 3`, with the same wording when there are too many.
- `.[]` with nothing to fix its type is `error: empty array literal needs an array context`.
- A view or a dynamic array cannot be initialized from a collection literal. `a: []s64 = .[ 1, 2, 3 ];` is `error: runtime array literals require a fixed array, Simd, or Matrix context`. Build a fixed array and take `a[..]`, or grow a `[..]T` with `array.push`. For an immediate borrowed view, `&.[1, 2, 3]` instead creates a pointer to temporary fixed-array storage that can coerce to an expected `[]s64`; that storage lasts only for the surrounding expression.
- A SIMD value takes the `#simd .[ ... ]` spelling and needs its own context: a bare `#simd .[ ... ]` is `error: #simd literal requires a Simd(T, N) context`, and a plain `.[ ... ]` against a SIMD annotation is `error: ordinary array literal .[...] cannot initialize Simd(f32, 4); use #simd .[...]`. SIMD vectors and matrices have their own chapter.
- Both bracket forms accept a trailing comma before the closing bracket.

NOTE
There is no `T.[ ... ]`. The qualified spelling exists for `.{ }` only, and `s64.[10, 20, 30]` does not fail to parse: it reads as the type `s64` followed by an unrelated statement, so the binding ends up with type `type` and the next use reports something remote such as `error: no operator '[]' for type, ssize`. Annotate the target instead.

The two bracket forms are not interchangeable. `.[ 1, 2 ]` against a struct annotation is `error: got [2]comptime_int, expected Vec2`, and `.{ ... }` against an array or a tuple annotation is `error: struct literal needs a type`, because neither an array type nor a tuple type names a struct the literal could be checked against. A tagged union takes neither form; it is built by variant, as the chapter on tagged unions describes.

6.7 Member Access, Calls, and Argument Expansion

A dot followed by a name selects a member. Fields resolve against the object's type. Calls can also resolve inherent methods and named typeclass operations, with an implicit receiver argument. An ordinary free function does not become a method merely because its first parameter has this type.

Object	Members
`struct`	Its declared fields, by name.
Tuple	`.0`, `.1`, and so on, plus `.count`.
`[N]T`, `[]T`, `[..]T`	`.count`, `.data`; only `[..]T` also has `.capacity`
SoA array or view	`.count`, one view per element field; dynamic arrays also have `.capacity`
`string`, `string16`, `cstring`, `cstring16`	read-only `.count` and `.data`; no `.capacity`
`any`	`.type` and `.value`
Tagged union	Variant names, bound by a pattern.

Member access composes with indexing, slicing, and calls, so a field of an element of a slice of a call result is one expression. The array and string members are described in the chapter on fixed arrays, views, and dynamic arrays; union variants in the chapter on reading a union payload; `any` in the chapter on reflection.

GLOSSO

```
#import "Io";

Vec2 :: struct { x: s64; y: s64; }
Line :: struct { a: Vec2; b: Vec2; }

diagonal :: () -> Line { return .{ .a = .{ 1, 2 }, .b = .{ 3, 4 } }; }

main :: () {
    l := diagonal();
    print("chained: %\n", l.a.y);
    print("call result: %\n", diagonal().b.x);

    pts: [2]Vec2 = .[ .{ 1, 2 }, .{ 3, 4 } ];
    print("index then field: %\n", pts[1].x);
    print("slice then field: %\n", pts[..][0].y);
    print("many-pointer: %\n", pts.data[1].y);

    t: (s64, string, bool) = .(1, "a", true);
    print("tuple: .count=% .1=%\n", t.count, t.1);

    v: any = 5;
    print("any: name=% size=%\n", v.type.*.name, v.type.*.size);
}
```

CONSOLE / OUTPUT

```
chained: 2
call result: 3
index then field: 3
slice then field: 2
many-pointer: 4
tuple: .count=3 .1=a
any: name=ssize size=8
```

An unknown field on a struct is `error: no field 'z' in 'Vec2'`, and the compiler offers the nearest declared name as `help: did you mean 'x'?`. A bad tuple member is `error: tuple member must be a numeric field like .0` for a name and `error: tuple index .5 is out of range for 2-value tuple` for a number. `any` reports its whole member set: `error: any has fields type and value, not 'size'`.

Field access follows pointers explicitly

A pointer is not one of the objects in that table, so `p.x` where `p` is a `*Vec2` is `error: member of non-struct`. The dereference is written: the suffix `.*` produces the pointee, and `p.*.x` reads its field. This follows from `*T` and `T` being different types with no coercion between them, and it holds on the left of an assignment as well.

GLOSSO
```
#import "Io";

Vec2 :: struct { x: s64; y: s64; }

bump :: (p: *mut Vec2) { p.*.x += 1; }

main :: () {
    v := Vec2.{ 3, 4 };
    p := &v;
    print("read: %\n", p.*.x);
    p.*.y = 40;
    bump(&v);
    print("after: % %\n", v.x, v.y);

    pp := &p;
    print("double: %\n", pp.*.*.x);
}
```

CONSOLE / OUTPUT
```
read: 3
after: 4 40
double: 4
```

The symmetric conversion does not happen either. The prefix `&` takes an address and is written at the call site, as in `bump(&v)`. Passing the value where a pointer is declared is `error: arg type Vec2 != *Vec2 in call bump`, and passing the pointer where the value is declared is `error: arg type *Vec2 != Vec2 in call byval`. Pointer syntax is covered in the chapter on pointers.

> **NOTE**
> `member of non-struct` is the message for every pointer-shaped object, not only struct pointers. `p.count` where `p` is a `*[..]s64` reports it, and so does `.count` on a `[*]T`, which carries no length at all. The first is written `p.*.count`; for the second there is nothing to write.

Inherent methods and associated functions

Attach methods to an owned type with `Vec2 ::: { ... }`. The first parameter `#self` receives a value, `*mut #self` a mutable pointer, and `*#self` a read-only pointer. `#Self_Type` names the complete owner type. A function without a receiver is an associated function, called through the type. The block below makes its members public explicitly.

`v.length2()` inserts the receiver; `Vec2.length2(v)` spells it explicitly. A pointer receiver can use an existing pointer or borrow an addressable value automatically. Mutable receivers cannot borrow a temporary or accept a read-only pointer. Field access inside a pointer receiver still writes `#self.*.x`. A parameter default cannot refer to `#self`: defaults are evaluated at the call site, where the receiver binding is not in scope.

GLOSSO
```
#import "Io";
Vec2 :: struct { x: s64; y: s64; }
Vec2 ::: #public {
    make :: (x: s64, y: s64) -> #Self_Type { return .{ x, y }; }
    length2 :: (#self) -> s64 { return #self.x * #self.x + #self.y * #self.y; }
    shift :: (*mut #self, amount: s64) { #self.*.x += amount; }
}
main :: () {
    v := Vec2.make(3, 4);
    print("length2=%\n", v.length2());
    v.shift(1);
    print("shifted=%\n", Vec2.length2(v));
}
```

CONSOLE / OUTPUT
```
length2=25
shifted=32
```

Methods live in the owner's namespace and do not create unqualified functions. `v.make(...)` is invalid for an associated function, and `v.length2` or `Vec2.length2` cannot be stored as a first-class method reference. A function-pointer field remains a normal field whose value can be called; function pointers have their own chapter.

Calls

A call is a callee expression followed by a parenthesized argument list. Arguments are separated by commas, and a trailing comma before the closing parenthesis is accepted. Named arguments and defaults are covered in the chapter on default and named parameters; matching against a generic signature in the chapter on generic functions.

Variadic packs

Inside the callee a variadic parameter is an ordinary value with `.count` and indexing. Its shape depends on how the parameter was declared and how the call supplied it: a typed pack such as `...s64` is a `[]s64`, an `...any` pack written out at the call site is a tuple of the argument types, and the same `...any` pack filled by expansion is a `[]any` with every element boxed.

GLOSSO

```
#import "Io";
#import "Meta";

pack :: (values: ...s64) { print("% count=%\n", type_name(type_of(values)), values.count); }
mixed :: (values: ...any) { print("% count=%\n", type_name(type_of(values)), values.count); }

main :: () {
    xs: [3]s64 = .[ 7, 8, 9 ];
    pack(xs...);
    mixed(1, "two");
    mixed(xs...);
}
```

CONSOLE / OUTPUT

```
[]ssize count=3
(ssize, string) count=2
[]any count=3
```

Argument expansion

A suffix `...` on a call argument spreads an existing array into the pack instead of passing it as a single element. The source may be a fixed array, a view, or a dynamic array. Fixed parameters ahead of the variadic one are unaffected and are still written normally.

GLOSSO

```
#import "Io";

label_sum :: (tag: string, values: ...s64) -> s64 {
    total: s64 = 0;
    for v: values total += v;
    return total;
}

main :: () {
    xs: [4]s64 = .[ 1, 2, 3, 4 ];
    d: [..]s64;
    defer d.free();
    d.push(5, 6);

    print("written out: %\n", label_sum("t", 1, 2, 3));
    print("fixed array: %\n", label_sum("t", xs...));
    print("view:        %\n", label_sum("t", xs[..]...));
    print("dynamic:     %\n", label_sum("t", d.view()...));
    print("empty pack:  %\n", label_sum("t"));
}
```

CONSOLE / OUTPUT

```
written out: 6
fixed array: 10
view:        10
dynamic:     11
empty pack:  0
```

An expanded argument replaces the whole pack rather than contributing to it, and the next two rules follow from that: there is no operation that concatenates a run-time array onto written-out arguments, so the compiler refuses the mixture instead of inventing one.

Rule	Diagnostic
At most one expanded argument per call.	call `sum_all` has more than one expanded argument
The expansion must be the only argument in the variadic slot.	expanded argument must be the final variadic argument in call `sum_all`
The source must be an array type.	expanded variadic argument to `sum_all` must be an array
Its element type must match the pack's.	expanded variadic arg type `[2]f64` != `[]ssize` in call `sum_all`
Its element type must satisfy the pack's constraints.	a `...Show` wrapper rejects elements without `Show`; `print` checks each selected `Formatter`

- `sum_all(100, xs...)` and `sum_all(xs..., 100)` both fail the second rule, even though the expansion is written last in the first of them: the test is that the variadic slot holds exactly one argument, not that the expansion comes last.
- A `string` is not an array for this purpose, so `take(s...)` reports the third rule's message rather than spreading the bytes.
- `...` is call-argument syntax and nothing else. `y := xs...;` is `error: unexpected token '...'`.

> **NOTE**
> `array.push` does not accept an expanded argument. Its variadic parameter is served by a `#magic` implementation that expansion does not reach, so `d.push(xs...)` is `error: got [4]s64, expected s64`; append in a loop, or route through an ordinary variadic. `print` does accept expansion but counts the pack as one argument when it checks placeholders, so `print("% %\n", ss...)` is `error: print: format has 2 '%' placeholder(s) but 1 argument(s) given`.

A generic owner head such as `Box($T) ::: { ... }` binds `T` from the receiver or concrete type. Methods may add their own generic parameters. An inherent block must be defined in the ownership unit that owns its type; importing a type or aliasing it does not transfer ownership. User code cannot add inherent blocks to primitives, arrays, pointers, options, or results. Wrap a dependency's type in a locally owned type to add behavior.

Named typeclass operations also support receiver syntax. `reader.read(buffer)` uses the first non-erased argument of `Reader.read(reader, buffer)` as its receiver. Fields and applicable inherent methods take precedence. If more than one class applies, qualify the class explicitly. Core standard-library behavior uses methods, including `array.push(...)` and `text.substring(...)`. Type queries use the free function `type_of(value)`. Domain operations with independent inputs use free functions, such as `alloc(size)`, `exists(path)`, and `simd_store(destination, value)`.

Factories belong to the type they construct. For example, `Path.path_from_string(text)` creates a path and `f64.from_bits(bits)` creates a float; neither operation is a method on its input string or integer. Once constructed, a resource's own operations remain methods, such as `file.read()` and `file.close()`. Import the component that declares each factory, function, or method.

7 Functions and generics

7.1 Function Declarations

A procedure is declared the way every other compile-time constant is declared: a name, `::`, and an initializer. What makes it a procedure is that the initializer begins with a parenthesized parameter list. Nothing about the declaration is a separate "function" statement, which is why a procedure may appear anywhere a declaration may appear, including inside another procedure's body.

GRAMMAR

```
procedure := name "::" "(" [ parameters ] ")"
             [ "->" result-type ]
             { function-directive }
             { "where" comptime-bool-expression }
             ( block | ";" )

result-type := type
             | "(" type { "," type } ")"
             | "(" name ":" type { "," name ":" type } ")"
```

The order of those five slots is fixed. The parser reads all function directives in one loop and stops at the first `where`, so a directive written after a constraint is no longer in the directive position and the declaration is rejected: `f :: (a: ssize) where true #inline { }` gives `error: function directives must appear before where clauses`. The corrected spelling puts every directive first and every constraint after them, as in `plain :: (a: ssize) -> ssize #inline where true { return a + 1; }`.

Slot	Spelling	Rule
parameters	`( ... )`	required, even when empty
result type	`-> R`	optional; omitting it means `void`
directives	`#inline`, `#must`, ...	any number, before the first `where`
constraints	`where <expr>`	any number, after the last directive
body	`{ ... }`	required, except on `#magic`, `#foreign`, and signature-only typeclass requirements

The directive vocabulary is given in the chapter Function and Aggregate Directives, and the constraint language in Constraints with `where`. This chapter covers only the shape the two of them sit in.

Result types

A procedure with no `->` returns `void`. Writing `-> void` explicitly is accepted and means the same thing. A result type that is not `void` constrains every `return` expression in the body: returning a value from a `void` procedure is `error: got ssize, expected void`, and returning the wrong type is the ordinary conversion diagnostic, `error: got ssize, expected string`.

GLOSSO

```
hello :: () { print("hello\n"); }
nothing :: () -> void { print("nothing\n"); }
add :: (a: ssize, b: ssize) -> ssize { return a + b; }
add3 :: (a, b, c: ssize) -> ssize { return a + b + c; }
```

`add3` uses a shared type annotation: one type applies to every name in the group. Shared annotations, defaults, and named arguments are the subject of Default and Named Parameters.

> **NOTE**
> A procedure whose result type is neither `void` nor a union, and whose body can reach its closing brace without executing a `return`, is rejected at the declaration: `error: function 'no_return_stmt' can reach the end of its body without returning a value`, framed on the name. The check is on the control flow, not on the presence of the keyword, so a body whose only exit is a forgotten branch is caught too. Three shapes are exempt: a `void` procedure needs no `return`; a union result, `E!T` and `?T` included, still yields its zero value, because reaching the end of an `E!void` body is that type's success path; and the named-result form below has its `return` appended by the parser.

GLOSSO

```
#import "Io";

no_return_stmt :: (a: ssize) -> ssize { }

main :: () {
    print("no_return_stmt=%\n", no_return_stmt(5));
}
```

CONSOLE / OUTPUT

```
error: function 'no_return_stmt' can reach the end of its body without returning a value
  --> ret.glo:3:1
  |
3 | no_return_stmt :: (a: ssize) -> ssize { }
  | ^^^^^^^^^^^^^^
```

Multiple and named results

A parenthesized list after the arrow declares more than one result. There are two spellings. A list of bare types produces a tuple result; a list in which every element is written `name: type` produces named results.

GLOSSO

```
#import "Io";

divmod :: (a: ssize, b: ssize) -> (ssize, ssize) { return a / b, a % b; }

minmax :: (a: ssize, b: ssize) -> (lo: ssize, hi: ssize) {
    if a < b { lo = a; hi = b; return; }
    lo = b;
    hi = a;
}

main :: () {
    q, r := divmod(7, 2);
    print("q=% r=%\n", q, r);
    t := divmod(7, 2);
    print("t.0=% t.1=%\n", t.0, t.1);
    lo, hi := minmax(9, 4);
    print("lo=% hi=%\n", lo, hi);
    _, only_r := divmod(9, 4);
    print("only_r=%\n", only_r);
}
```

CONSOLE / OUTPUT

```
q=3 r=1
t.0=3 t.1=1
lo=4 hi=9
only_r=1
```

A named result list is rewritten by the parser before checking: each name becomes a zero-initialized local at the top of the body, and an implicit `return <names>;` is appended at the end. A bare `return;` inside such a body is rewritten to return the names too. This is why `minmax` can assign `lo` and `hi` and then leave, and why a named-result procedure with an empty body returns zeroes rather than being rejected.

GLOSSO

```
#import "Io";

split :: (a: ssize) -> (lo: ssize, hi: ssize) {
    lo = a - 1;
    hi = a + 1;
    return;
}

overwrite :: (a: ssize) -> (lo: ssize, hi: ssize) {
    lo = 0;
    hi = 0;
    return a, a * 2;
}

zeroes :: () -> (x: ssize, y: ssize) { }

main :: () {
    a, b := split(10);
    print("a=% b=%\n", a, b);
    c, d := overwrite(3);
    print("c=% d=%\n", c, d);
    e, f := zeroes();
    print("e=% f=%\n", e, f);
}
```

CONSOLE / OUTPUT
```
a=9 b=11
c=3 d=6
e=0 f=0
```

- `q, r := divmod(7, 2);` destructures the result into two bindings.
- `t := divmod(7, 2);` binds the whole tuple; the elements are then `t.0` and `t.1`.
- `_` in a destructuring position discards that result.
- `return e1, e2;` is a statement form only. A comma in value position belongs to an argument list, so there is no comma expression that builds a multiple-result value.

NOTE
A tuple result has no `Show` instance, so the default `%` has no matching `Formatter` for it. Destructure the result and print its elements, or supply a custom rendering.

Bodyless declarations

A body is required except in three cases: the declaration carries `#magic`, it carries `#foreign`, or it is a requirement inside a typeclass. In those cases the declaration ends at a semicolon and the implementation comes from the compiler, the linker, or an instance.

GLOSSO
```
#import "Io";

libc :: #library,system "msvcrt";

c_abs :: (v: s32) -> s32 #foreign libc "abs";
raw_add :: (a: s64, b: s64) -> s64 #magic "int_add";

main :: () {
    print("c_abs=% raw_add=%\n", c_abs(-5), raw_add(2, 3));
}
```

CONSOLE / OUTPUT
```
c_abs=5 raw_add=5
```

The rule runs in both directions. An ordinary signature ended with a semicolon instead of a body is `error: unexpected token ';'`. At file scope, a `#foreign` or `#magic` declaration must itself end with `;`: putting a body directly after the directive first gives `error: declaration must end with ';'`, while putting the required semicolon before that body leaves `{` to be read as the next declaration. It gives `error: bad decl name '{'`. Conversely, a procedure that has a body already ended at its `}`. At file scope, following it with `;` is `error: expected a declaration, found ';'`; inside another body the stray token is the ordinary `error: unexpected token ';'`. The nested-declaration parser currently accepts an omitted terminator on a bodyless `#foreign` or `#magic` signature; that narrow exception does not apply at file scope.

Nested procedures

A procedure declared inside a body is visible within its containing lexical block, including to sibling local procedures. It can be called before its declaration. Structs, enums, unions, distinct types, typeclasses, and function-pointer aliases declared in a body also stay local. Their names do not escape into other top-level procedures.

GLOSSO
```
#import "Io";

first :: () -> ssize {
    nested :: (x: ssize) -> ssize { return x * 2; }
    return nested(21);
}

second :: () -> ssize {
    nested :: (x: ssize) -> ssize { return x + 1; }
    return nested(2);
}

main :: () { print("nested=% other=%\n", first(), second()); }
```

CONSOLE / OUTPUT
```
nested=42 other=3
```

- A top-level procedure cannot call either local nested function. Pass a function pointer explicitly when a helper must escape its scope.

- An ordinary local procedure cannot capture enclosing runtime locals. Pass them as parameters. A macro expands into its caller and follows the macro hygiene rules described in Macros with #expand.
- Local functions with the same name in different blocks are independent. Duplicate function declarations within one block are rejected.
- A name :: expression binding remains a local constant. Type aliases such as Byte :: u8 may be used in annotations within that same scope.

7.2 Default and Named Parameters

A parameter may carry a default expression, and a call may select parameters by name instead of by position. The two features are designed together: a default is what makes a parameter skippable, and a named argument is what makes a skipped parameter reachable again.

Form	Meaning
`b: ssize = 3`	a default for `b`
`a, b: ssize = 3`	one shared type and one shared default, applied to both names
`factor := 2`	a default whose type is inferred from it; exactly one name
`f(x, .name = v)`	a named argument at the call site

The default expression is written in the declaration and is evaluated at the call, not stored with the procedure. Its type must convert to the parameter's type under the ordinary rules.

GLOSSO

```
#import "Io";

greet :: (name: string, greeting: string = "hello", punct: string = "!") {
    print("% %%\n", greeting, name, punct);
}

main :: () {
    greet("world");
    greet("world", .punct = "?");
    greet("world", .greeting = "hi", .punct = "?");
}
```

CONSOLE / OUTPUT

```
hello world!
hello world?
hi world?
```

Shared types and shared defaults

A group of names separated by commas and ended with one `: type` gives every name in the group that type. If the group also carries `= value`, the default is shared as well; it is not attached only to the last name. This follows from the grammar: the annotation and the default belong to the parameter group, and the group is what the names were collected into.

GLOSSO

```
#import "Io";

shared :: (a, b: ssize = 3) -> ssize { return a * 10 + b; }

main :: () {
    print("shared()=% shared(1)=% shared(1,2)=%\n",
          shared(), shared(1), shared(1, 2));
}
```

CONSOLE / OUTPUT

```
shared()=33 shared(1)=13 shared(1,2)=12
```

Inferred-type defaults

`name := value` in a parameter list declares a defaulted parameter whose type is taken from the default expression, with no annotation written. The `:=` form takes exactly one name, because there is no type annotation for a group to share; `(a, b := 2)` is `error: unexpected token ','`.

GLOSSO

```
#import "Io";
```

```
scaled :: (x: ssize, factor := 2) -> ssize { return x * factor; }

main :: () {
    print("scaled=% scaled3=%\n", scaled(5), scaled(5, 3));
}
```

CONSOLE / OUTPUT

```
scaled=10 scaled3=15
```

Defaults need not come last

A defaulted parameter may precede a non-defaulted one. Nothing in the declaration is rejected; the consequence lands at the call site, where the later parameter can no longer be reached positionally and must be named. Named arguments are what make this coherent, so the language does not need the usual "defaults last" restriction.

GLOSSO

```
#import "Io";

early :: (a: ssize = 1, b: ssize) -> ssize { return a * 100 + b; }

main :: () {
    print("early=%\n", early(.b = 2));
    print("named-first=%\n", early(.b = 2, 9));
}
```

CONSOLE / OUTPUT

```
early=102
named-first=902
```

Named arguments

A named argument is written `.name = value`. The leading dot is part of the syntax and is not optional: `f(1, b = 3)` is `error: unexpected token '='`, because without the dot the parser is reading an assignment expression in an argument list. The dot spelling matches the one used for enum variants and aggregate literal fields.

GLOSSO

```
#import "Io";

box :: (w: ssize = 1, h: ssize = 1, d: ssize = 1) -> ssize { return w * h * d; }

main :: () {
    print("box=% box2=%\n", box(.d = 2, .w = 3, .h = 4), box(3, .d = 2, .h = 4));
}
```

CONSOLE / OUTPUT

```
box=24 box2=24
```

- Named and positional arguments may be mixed in one call, as `box(3, .d = 2, .h = 4)` does.
- A named argument may precede a positional one. In `early(.b = 2, 9)` the `9` fills the first parameter that no name claimed, which is `a`, giving `902`.
- A trailing comma is allowed after the last parameter in the declaration, so a long defaulted parameter list can be written one parameter per line without a ragged last entry.

A name that matches no parameter is not a separate diagnostic; it is reported as a failed call, together with the signatures that were considered. This is the same message a wrong argument count or a wrong argument type produces.

CONSOLE / OUTPUT

```
error: no overload 'f' accepts these arguments
note: possible candidates for 'f':
candidate: f :: (a: ssize, b: ssize = ...) -> ssize
```

When a default is evaluated

The default is an expression, evaluated at each call that omits the argument. It is not folded once at the declaration, so a default that calls a procedure calls it again on every such call, and a default that reads a mutable global reads its current value.

GLOSSO

```
#import "Io";
```

```
counter := 0;

bump :: () -> ssize { counter += 1; return counter; }

with_call :: (x: ssize = bump()) -> ssize { return x; }

main :: () {
    print("call1=% call2=% counter=%\n", with_call(), with_call(), counter);
}
```

CONSOLE / OUTPUT

```
call1=1 call2=2 counter=2
```

NOTE

Evaluation at the call does not relocate source syntax. `location: Source_Location = #source_location` captures the position where that default was written, not the caller. The parser already materialized the directive's filename, line, and column as constants. Pass `.location = #source_location` explicitly to capture a call site.

GLOSSO

```
#import "Io";
locate :: (location: Source_Location = #source_location) -> ssize {
    return location.line;
}
main :: () {
    print("default=% caller=%\n", locate(), locate(.location = #source_location));
}
```

CONSOLE / OUTPUT

```
default=2 caller=6
```

Defaults and overloading

Glosso has no ad-hoc overloading: two ordinary procedures may not share a name, and the second one is `error: function 'render' is already declared`. The only overloading is typeclass method dispatch, described in Typeclasses: Declaring a Contract and How an Instance Is Chosen. Defaults therefore never participate in choosing between candidates by arity; the instance is chosen by the argument types first, and the chosen method's defaults fill the rest.

GLOSSO

```
#import "Io";

Render :: typeclass (T: type) {
    render :: (value: T, width: ssize = 4) -> ssize;
}

s64 ::: instance Render {
    render :: (value: s64, width: ssize = 4) -> ssize { return value * width; }
}

bool ::: instance Render {
    render :: (value: bool, width: ssize = 9) -> ssize { if value return width; return 0; }
}

main :: () {
    print("ssize  = % / %\n", render(3), render(3, 10));
    print("bool = % / %\n", render(true), render(true, 2));
}
```

CONSOLE / OUTPUT

```
ssize  = 12 / 30
bool = 9 / 2
```

`render(true)` prints `9`, not `4`. The default that applies is the one written on the instance method that was selected, not the one written on the class requirement. A named argument works the same way at a typeclass method call as anywhere else: `render(3, .width = 5)` gives `15`.

NOTE

An instance method must repeat the requirement's defaults. Declaring `render :: (value: s64, width: ssize)` against a requirement that reads `width: ssize = 4` no longer matches, and the instance is rejected with `error: instance 's64 :: Render' does not satisfy the implicit minimal definition; missing render`. The value of the default may differ; its presence may not.

7.3 Variadic Parameters

A variadic parameter accepts any number of trailing arguments and presents them to the body as one collection. It is spelled with `...` as a prefix on the parameter's type, and what follows the `...` decides how much the body knows about the elements.

Spelling	Elements	The body sees
`xs: ...`	any types, mixed	`[]any`
`xs: ...any`	any types, mixed	a heterogeneous tuple; constant indices retain concrete types
`xs: ...ssize`	all `ssize`	`[]ssize`
`xs: ...$T`	all one type, captured as `T`	`[]T`
`xs: ...Show`	any type implementing `Show`	a pack whose elements keep their own types
`xs: ...(Show && PartialEq)`	any type implementing both	the same, with both constraints

The `...` is a type prefix, not a suffix on the name, and it is three dots. Two dots is the range operator, and the compiler says so: `xs: ..ssize` is `error: variadic parameter uses '...T'; '..' is the range operator`. A variadic parameter takes exactly one name (`variadic parameter must have one name`) and cannot carry a default (`variadic parameter cannot have a default value`), since the empty pack already is the default.

The pack interface

Every form supports the same three operations: `.count` gives the number of arguments as an `ssize`, `for` walks the elements with `it` and `it_index`, and a constant index selects one element. A call that passes no variadic arguments is legal and gives a pack of count zero.

GLOSSO

```
#import "Io";

describe :: (xs: ...) {
    print("count=%\n", xs.count);
    for xs print("  %: %\n", it_index, it);
    if xs.count > 0 print("  first=%\n", xs[0]);
}

main :: () {
    n: ssize = 7;
    describe(n, "hi", true);
    describe();
}
```

CONSOLE / OUTPUT

```
count=3
  0: 7
  1: hi
  2: 1
  first=7
count=0
```

For the untyped, homogeneous, and generic forms the parameter really is an array view and can be bound as one: `view: []any = xs;` inside `describe`, `view: []ssize = xs;` inside a `...ssize` procedure, and `view: []T = xs;` inside a `...$T` procedure all compile.

> **NOTE**
> An untyped pack boxes each argument into `any`, and a bare integer literal has no type to box. `count_all(1, 2)` against `count_all :: (xs: ...)` reaches the backend undecided and fails late with `error: comptime_int reached code generation without a decided type [opcode BoxAny] while lowering 'main'`. Give the argument a type, or use a typed pack.

Explicit `...any` is different from bare `...`: it preserves the argument types in a heterogeneous tuple and accepts ordinary numeric literals. A constant index or inline iteration retains each element's type, which lets a forwarding wrapper select a `Formatter` instance. The whole pack cannot be assigned to `[]any`; that assignment reports `got tuple, expected []any`. Expand it with `args...` when forwarding.

> **NOTE**
> Variadic packs do not destroy their elements. Passing a managed owner through a variadic parameter is rejected, including through `print`'s pack. Supply a borrowed pointer or non-owning view supported by the callee, or render an owner to a string without moving it into the pack.

GLOSSO
```
#import "Format";

render :: (#comptime fmt: string, args: ...any) -> string {
    return tprint(fmt, args...);
}

main :: () {
    text := render("%x %c", 255, #char "λ"); // "ff λ"
}
```

Homogeneous and generic packs

`...T` fixes the element type in the declaration; every argument must convert to `T`. `...$T` leaves the element type open and captures it from the call, so each call site is a separate instantiation but a single call is still homogeneous.

GLOSSO
```
#import "Io";

sum :: (xs: ...ssize) -> ssize {
    total := 0;
    for xs total += it;
    return total;
}

firstof :: (xs: ...$T) -> T { return xs[0]; }

main :: () {
    print("sum=% sum0=%\n", sum(1, 2, 3, 4), sum());
    print("firstof=% firstofs=%\n", firstof(10, 20), firstof("a", "b"));
}
```

CONSOLE / OUTPUT
```
sum=10 sum0=0
firstof=10 firstofs=a
```

A wrong element type is reported per argument: `sum(1, "two")` is `error: arg type string != ssize in call sum`. For a generic pack the first argument decides, and the message names the type it decided on: `firstof(1, "two")` is `error: arg type string != comptime_int in call firstof`.

Typeclass-constrained packs

`...C`, where `C` is a typeclass, accepts arguments of different types so long as each one has an instance of `C`. Several classes are combined with `&&` inside parentheses. `&&` is the only connective; `...(Show || PartialEq)` is `error: unexpected token '||'`, and an empty list is `error: constrained variadic parameter needs at least one typeclass`. `print` instead uses `args: ...any`: its compile-time format parser checks `Formatter` for each argument's selected rendering. A wrapper that specifically requires default `Show` values can use `...Show`.

GLOSSO
```
#import "Io";

both :: (v: ...(Show && PartialEq)) {
    for v print("[%]", it);
    print("\n");
}

main :: () {
    both(1, "two", true);
}
```

CONSOLE / OUTPUT
```
[1][two][1]
```

A constrained pack is not an array of `any`. Each element keeps its own static type, and the `for` over it is unrolled at compile time so that `it` has a different type on each iteration. That is what allows the per-element instance to be selected. It also means a body that is valid for one argument list can be invalid for another.

GLOSSO

```
#import "Io";

only_int :: (x: ssize) -> ssize { return x + 1; }

f :: (values: ...Show) { for values print("%\n", only_int(it)); }

main :: () { n: ssize = 1; f(n, "two"); }
```

CONSOLE / OUTPUT

```
error: arg type string != ssize in call only_int
```

The same body compiles and runs when both arguments are `ssize`. Indexing follows the same rule in reverse: a constant index selects an element with its own type, but an index the compiler cannot fold degrades the element to `any`, giving `error: arg type any != ssize in call only_int`.

Parameters after a variadic

A variadic parameter does not have to be last, but every parameter after it must have a default: otherwise there is no way to tell where the pack ends. Writing one without a default is `error: parameters after a variadic parameter must have default values`. The trailing parameters are then supplied by name.

GLOSSO

```
#import "Io";

tagged :: (xs: ...ssize, tag: string = "sum") -> ssize {
    total := 0;
    for xs total += it;
    print("% = %\n", tag, total);
    return total;
}

main :: () {
    _ := tagged(1, 2, 3);
    _ := tagged(1, 2, .tag = "z");
}
```

CONSOLE / OUTPUT

```
sum = 6
z = 3
```

Call-site expansion

An existing array can be passed as the whole pack by suffixing the argument with `...`. Use a fixed array or a view whose element type matches the pack. A dynamic array owns its storage and cannot move into a variadic pack; expand its borrowed view instead, as in sum(dyn[..]...). Managed elements cannot be passed by value through a pack, which does not destroy them.

GLOSSO

```
#import "Io";

sum :: (xs: ...ssize) -> ssize { t := 0; for xs t += it; return t; }
sum_plus :: (base: ssize, xs: ...ssize) -> ssize { return base + sum(xs...); }

main :: () {
    fixed: [3]ssize = .[1, 2, 3];
    view: []mut ssize = fixed[..];
    print("fixed=% view=%\n", sum(fixed...), sum(view...));
    dyn: [..]ssize;
    dyn.push(10);
    dyn.push(20);
    print("dyn=%\n", sum(dyn[..]...));
    print("forward=%\n", sum_plus(100, 1, 2, 3));
}
```

CONSOLE / OUTPUT

```
fixed=6 view=6
dyn=30
forward=106
```

`sum_plus` shows the other use of the suffix: a procedure forwards its own pack to another variadic procedure by expanding it. An expansion must be the only variadic argument in the call, so it can neither be preceded nor followed by loose elements. Fixed parameters before the variadic are unaffected, which is why `sum_plus(100, 1, 2, 3)` and `sum(xs...)` are both fine.

- `sum(5, a...)` and `sum(a..., 5)` both report error: expanded argument must be the final variadic argument in call `sum`.

- `sum(a..., b...)` reports error: call `sum` has more than one expanded argument.
- Expanding something that is not an array reports error: expanded variadic argument to `sum` must be an array. Expanding the wrong element type reports error: expanded variadic arg type `[2]string` != `[]ssize` in call `sum`.

> **NOTE**
> A bare `...` as the last parameter, with no name and no type, is a different feature: it is the C varargs marker, legal only on `#foreign` declarations and `#c_call` definitions. It is covered in Calling C from Glosso.

7.4 `noalias` Pointer Parameters

noalias is a pointer parameter modifier and a promise made by the caller. During the call, access through that pointer must obey the non-aliasing contract. It is independent of pointer mutability and ownership.

GLOSSO
```
copy_one :: (noalias destination: *mut s64, source: *s64) {
    destination.* = source.*;
}

main :: () {
    source: s64 = 42;
    destination: s64;
    copy_one(&destination, &source);
}
```

A receiver can also carry noalias, as in noalias *mut #self. A writable destination and a read-only source still need distinct storage when the noalias contract requires it. Memory analysis checks overlapping arguments and conflicting live borrows; findings are warnings by default and errors under --borrow-check=error.

> **NOTE**
> Keep noalias only where callers can satisfy it. Backend optimization may rely on the declaration; runtime bounds and null checks do not check aliasing.

7.5 Lazy Parameters and Short-Circuit Operators

A parameter marked `#lazy` is not evaluated at the call. The argument expression travels to the callee unevaluated, is evaluated the first time the body reads the parameter, and the value produced by that evaluation is reused for every later read. A `#lazy` parameter the body never reads is never evaluated at all, so a call can discard an expensive or failing computation without paying for it.

GLOSSO
```
#import "Io";

evals: ssize = 0;

probe :: (label: string) -> bool {
    evals += 1;
    print("  evaluated %\n", label);
    return true;
}

ignore_it :: (#lazy v: bool) -> ssize { return 0; }
use_once  :: (#lazy v: bool) -> ssize { if v return 1; return 2; }
use_twice :: (#lazy v: bool) -> ssize { if v { if v return 3; } return 4; }

main :: () {
    print("ignore_it:\n");
    r := ignore_it(probe("a"));
    print("  result=% evals=%\n", r, evals);

    evals = 0;
    print("use_once:\n");
    r = use_once(probe("b"));
    print("  result=% evals=%\n", r, evals);

    evals = 0;
    print("use_twice:\n");
    r = use_twice(probe("c"));
    print("  result=% evals=%\n", r, evals);
}
```

CONSOLE / OUTPUT
```
ignore_it:
  result=0 evals=0
use_once:
  evaluated b
```

```
  result=1 evals=1
use_twice:
  evaluated c
  result=3 evals=1
```

Three calls pass the same side-effecting expression. `ignore_it` never names `v`, and `probe` does not run. `use_once` and `use_twice` both run it exactly once, whether the body reads the parameter one time or two. The counter is the whole story: zero reads means zero evaluations, and any positive number of reads means one.

Evaluated at first use, not at the call

The delay is observable, not merely an optimization. Because the argument expression is evaluated inside the callee, it sees the state the callee has already produced, not the state that existed at the call.

GLOSSO

```
#import "Io";

counter: ssize = 1;

delayed :: (#lazy v: ssize) -> ssize {
    counter = 100;
    return v;
}

eager :: (v: ssize) -> ssize {
    counter = 100;
    return v;
}

main :: () {
    counter = 1;
    a := delayed(counter);
    counter = 1;
    b := eager(counter);
    print("delayed=% eager=%\n", a, b);
}
```

CONSOLE / OUTPUT

```
delayed=100 eager=1
```

`eager` receives the value `1` that `counter` held at the call. `delayed` receives the expression `counter`, reads it after the assignment, and gets `100`. A `#lazy` parameter is therefore a deferred expression rather than a deferred value, and an argument with a side effect changes when that side effect happens.

Forwarding, and reads inside a loop

Memoisation is per call, and it survives forwarding: passing a `#lazy` parameter on to another `#lazy` parameter does not force it, and the inner function's first read is the one evaluation the whole chain gets.

GLOSSO

```
#import "Io";

n: ssize = 0;
tick :: () -> bool { n += 1; return true; }

inner :: (#lazy v: bool) -> bool { if v return v; return false; }
outer :: (#lazy v: bool) -> bool { return inner(v); }

main :: () {
    print("outer=% n=%\n", outer(tick()), n);
}
```

CONSOLE / OUTPUT

```
outer=1 n=1
```

A read inside a loop is still one evaluation. A body that writes `while i < n` over a `#lazy` parameter `n` evaluates the bound once and compares against the memoised value on every iteration.

> **NOTE**
> `#lazy` is not textual substitution. A macro would re-run the expression at every occurrence; a `#lazy` parameter runs it at most once and caches the result. If the intent is to re-evaluate, pass a procedure or a lambda and call it, or use `#expand`, which is covered in Macros with `#expand`.

How `&&` and `||` are built on it

`&&` and `||` are not built-in control flow. They are ordinary operators owned by the Prelude typeclasses `LogicalAnd` and `LogicalOr`, and short-circuiting is spelled out in the class declaration as `#lazy` on the right operand.

GLOSSO

```
LogicalAnd :: typeclass (L: type, R: type) {
    Output :: type;
    '&&' :: (left: L, #lazy right: R) -> Output #operator(left, 6);
}

LogicalOr :: typeclass (L: type, R: type) {
    Output :: type;
    '||' :: (left: L, #lazy right: R) -> Output #operator(left, 4);
}

(bool, bool) ::: instance LogicalAnd {
    Output :: bool;
    '&&' :: (left: bool, #lazy right: bool) -> Output #magic "bool_and";
}
```

The checker marks a binary expression as short-circuiting exactly when the second parameter of the instance it selected carries `#lazy`. Short-circuiting is therefore a property of the chosen instance, not of the spelling `&&`: the same source text is a short-circuit for one operand type and a plain two-argument call for another.

GLOSSO

```
#import "Io";

side :: (label: string, value: bool) -> bool {
    print("  eval %\n", label);
    return value;
}

main :: () {
    print("false && side:\n");
    a := side("L", false) && side("R", true);
    print("  = %\n", a);

    print("true && side:\n");
    b := side("L", true) && side("R", false);
    print("  = %\n", b);

    print("true || side:\n");
    c := side("L", true) || side("R", false);
    print("  = %\n", c);

    print("false || side:\n");
    d := side("L", false) || side("R", true);
    print("  = %\n", d);
}
```

CONSOLE / OUTPUT

```
false && side:
  eval L
  = 0
true && side:
  eval L
  eval R
  = 0
true || side:
  eval L
  = 1
false || side:
  eval L
  eval R
  = 1
```

`%` prints a `bool` as `1` or `0`. The right operand runs only when the left one fails to decide the result, which is what makes the ordinary guard idiom safe: the bounds test below never indexes out of range, because `view[i]` is the unevaluated right operand of a `&&` whose left operand is already false.

GLOSSO

```
#import "Io";

main :: () {
    xs: [3]s64 = .[1, 2, 3];
    view := xs[..];
    i: s64 = 5;
    if i < view.count && view[i] > 0 print("in range and positive\n");
    else print("guard held\n");
}
```

CONSOLE / OUTPUT
```
guard held
```

A user type that short-circuits

Writing an instance of `LogicalAnd` or `LogicalOr` gives a user type the same behaviour, and unlike the Prelude's `bool` instance the body is ordinary Glosso rather than a `#magic` intrinsic.

GLOSSO
```
#import "Io";

Flag :: struct { on: bool; }

(Flag, Flag) ::: instance LogicalAnd {
    Output :: Flag;
    '&&' :: (a: Flag, #lazy b: Flag) -> Flag { if !a.on return a; return b; }
}

hits: ssize = 0;
mk :: (v: bool) -> Flag { hits += 1; return .{ .on = v }; }

main :: () {
    x: Flag = .{ .on = false };
    r := x && mk(true);
    print("r=% hits=%\n", r.on, hits);
}
```

CONSOLE / OUTPUT
```
r=0 hits=0
```

The laziness of an instance method has to agree with the class it implements, parameter for parameter. Dropping the `#lazy` from the instance above is `error: operator '&&' must match the #lazy parameter contract of typeclass 'LogicalAnd'`; the same check on a non-operator method reads `error: function 'pick' must match the #lazy parameter contract of typeclass 'Pick'`. Without it a caller could not tell from the class whether its argument will run.

The inlining requirement

Deferring an expression into a callee means splicing the expression into the callee's body, so a call that passes a `#lazy` argument is always inlined and the callee gets no independent definition: a program whose only lazy procedure is `pick` compiles to LLVM IR containing no `glo_pick` symbol at all. Everything that would defeat that splice is rejected.

Rejected combination	Diagnostic
`#magic` body	`#lazy parameters on #magic function 'lazyf' require compiler-defined lazy semantics`
`#foreign` body	`#lazy parameters on 'puts2' are not part of a foreign ABI`
Direct or mutual recursion	`recursive call to lazy function 'a_fn' is not supported`
Assigned to a function pointer	`lazy function 'lz' cannot be used as an ordinary function pointer`
A variadic parameter	`#lazy cannot qualify a variadic parameter`
A `#empty` parameter	`#lazy cannot qualify a #empty parameter`
A lambda parameter	`error: unexpected token 'lazy'` from the parser
An instance that drops it	`must match the #lazy parameter contract of typeclass`

The Prelude's own `&&` and `||` are the one exception to the body requirement: they are `#magic "bool_and"` and `#magic "bool_or"`, and the compiler knows their lazy semantics directly rather than by inlining a Glosso body.

Combining with `#comptime`

`#lazy` and `#comptime` may qualify the same parameter, in either order. The argument then has to be compile-time known, and it is still only evaluated if the body reads it, which is how a compile-time call can pass an expression that would fail to fold.

GLOSSO
```
#import "Io";

comptime_ignore :: (#lazy #comptime value: s64) -> s64 { return 7; }
comptime_use    :: (#comptime #lazy value: s64) -> s64 { return value * 2; }

UNUSED :: #comptime comptime_ignore(1 / 0);
USED   :: #comptime comptime_use(21);
```

```
main :: () { print("UNUSED=% USED=%\n", UNUSED, USED); }
```

CONSOLE / OUTPUT

```
UNUSED=7 USED=42
```

`1 / 0` is never evaluated because `comptime_ignore` never names `value`. `#comptime` parameters in general are covered in Compile-Time and Type-Only Parameters, and the operator machinery behind `&&` in The Operator System.

7.6 Compile-Time and Type-Only Parameters

Two parameter qualifiers move a parameter out of the runtime calling convention entirely. `#comptime` keeps the parameter's value but demands that the caller supply it as a compile-time constant; `#empty:` keeps only a type and no value at all. Both occupy an argument position at the call site, and neither appears in the generated function's parameter list.

`#comptime` parameters

`#comptime` prefixes the parameter name. Inside the body the parameter is a compile-time constant, so it may drive `#inline for`, `#if`, `#assert`, an array length, or any other construct that needs a value before the program runs.

GLOSSO

```
#import "Io";

pow :: (#comptime n: ssize, x: ssize) -> ssize {
    r := 1;
    #inline for i: 0..n r *= x;
    return r;
}

addc :: (#comptime a, b: ssize) -> ssize { return a + b; }

main :: () {
    print("pow(3,2)=%\n", pow(3, 2));
    print("addc(4,5)=%\n", addc(4, 5));
    K :: 5;
    print("pow(K,2)=%\n", pow(K, 2));
}
```

CONSOLE / OUTPUT

```
pow(3,2)=8
addc(4,5)=9
pow(K,2)=32
```

One `#comptime` covers every name in a shared-type group, so `(#comptime a, b: ssize)` makes both `a` and `b` compile-time. The argument must be a constant the compile-time evaluator can reduce: a literal, a `::` constant, or a compile-time call. A mutable local is not one, and `k := 3; pow(k, 2)` is `error: argument for #comptime parameter 'n' must be known at compile time` with `note: unknown const k`.

A `#comptime` parameter is otherwise an ordinary parameter. It may carry a default, and it may be supplied by name with the usual `.name = value` spelling.

GLOSSO

```
#import "Io";
#import "String_Builder";
#import "Strings/Memory";

banner :: (#comptime width: ssize = 4, ch: string) -> string {
    builder := String_Builder.make_len_cap();
    for 0..width builder.write_string(ch);
    return builder.into_string();
}

main :: () {
    a := banner(.ch = "-");
    defer a.free();
    b := banner(6, "=");
    defer b.free();
    c := banner(.width = 2, .ch = "*");
    defer c.free();
    print("%\n%\n%\n", a, b, c);
}
```

CONSOLE / OUTPUT

```
----
======
**
```

Erasure and specialization identity

A `#comptime` argument is part of the identity of the specialization, not part of its signature. Two calls with different constants compile to two functions; each of them takes only the runtime parameters. The program below has one `scale` in source and two in the object code.

GLOSSO

```
#import "Io";

scale :: (#comptime k: ssize, x: ssize) -> ssize { return k * x; }
zero_of :: (#empty: $T) -> T { d: T; return d; }

main :: () {
    n := 10;
    print("%\n", scale(2, n));
    print("%\n", scale(3, n));
    print("%\n", zero_of(s16));
}
```

Compiled with `--backend llvm-ir`, the three definitions the file produces are these.

CONSOLE / OUTPUT

```
define i64 @glo_scale__ssize_ssize__comptime_df69fa1c4e5525eb(i64 %0) {
define i64 @glo_scale__ssize_ssize__comptime_df6d1e1c4e5798ee(i64 %0) {
define i16 @glo_zero_of__s16() {
```

Each `scale` specialization takes one `i64`, the runtime `x`; the constant `k` survives only in the mangled name and in the folded body. `zero_of` takes nothing at all, because its only parameter is a `#empty:` slot. A `#comptime` parameter is consequently a code-size decision as much as a typing one: every distinct constant is another copy of the body.

Qualifier combination	Result
`#comptime` twice on one parameter	`duplicate #comptime parameter directive`
`noalias #comptime`	`noalias cannot qualify a #comptime parameter`
`using #comptime`	`using cannot qualify a #comptime parameter`
`#comptime` on a variadic	`error: unexpected token '...'` from the parser
`#comptime #lazy` or `#lazy #comptime`	Accepted, in either order

`#empty:` type-only parameters

`#empty:` replaces the parameter name with the marker itself and declares a slot that carries a type and no value. The caller writes a type where the argument goes. This is how a procedure whose result type cannot be inferred from any argument gets told which type to produce.

GLOSSO

```
#import "Io";

zero_of  :: (#empty: $T) -> T { d: T; return d; }
name_of  :: (#empty: $A, $B) -> string { return "two types"; }
parse_as :: (#empty: $T, text: string) -> T { d: T; return d; }

main :: () {
    print("zero_of(ssize)=%\n", zero_of(ssize));
    print("zero_of(f64)=%\n", zero_of(f64));
    print("zero_of(bool)=%\n", zero_of(bool));
    print("name_of=%\n", name_of(ssize, string));
    print("parse_as=%\n", parse_as(s16, "x"));
}
```

CONSOLE / OUTPUT

```
zero_of(ssize)=0
zero_of(f64)=0
zero_of(bool)=0
name_of=two types
parse_as=0
```

One `#empty:` marker may list several types separated by commas, as `name_of` does, and a signature may contain several markers. All anonymous witnesses must appear before runtime parameters: `pair :: (#empty: $A, #empty: $B, x: s64)` is called as `pair(s16, s32, 1)`. A witness after `x` is rejected with `#empty parameters must appear before runtime parameters`. Named `#comptime` parameters retain their own placement rules.

What the slot takes is the type of the expression written there, and the expression itself is discarded rather than evaluated. Passing a call in a `#empty:` position selects that call's return type and does not run it.

GLOSSO
```
#import "Io";

hits: ssize = 0;
side :: () -> f64 { hits += 1; return 1.0; }

zero_of :: (#empty: $T) -> T { d: T; return d; }

main :: () {
    z := zero_of(side());
    print("z=% hits=%\n", z, hits);
}
```

CONSOLE / OUTPUT
```
z=0 hits=0
```

A type name such as `f64` is the normal spelling and the one to prefer, because it says what is meant. A generic type application is not an expression, so `zero_of(Box(s64))` fails with `error: unknown function Box`; a `#empty:` slot cannot select a partially applied generic carrier.

> **NOTE**
> `#empty:` parameters are anonymous. The compiler names them `__empty0`, `__empty1`, and so on, which is what surfaces in an overload note: calling `pair :: (#empty: $A, $B)` with one type gives `error: no overload 'pair' accepts these arguments` and `candidate: pair :: (__empty0: A, __empty1: B) -> s64`. They cannot be passed by name, cannot take a default, and cannot be qualified with `using`, `noalias`, or `#lazy`.

Choosing an instance by its output type

The standard library uses `#empty:` wherever a typeclass has to dispatch on a type that appears only in the result. `FromString` declares `from_string :: (#empty: T, text: string) -> ?T`, so the destination type is an argument and instance selection has something to match on.

GLOSSO
```
#import "Io";

main :: () {
    a := from_string(s64, "-42");
    b := from_string(f64, "2.5");
    c := from_string(bool, "nope");
    print("a=% b=% c=%\n",
          a.unwrap_or(0),
          b.unwrap_or(0.0),
          c.is_none());
}
```

CONSOLE / OUTPUT
```
a=-42 b=2.5 c=1
```

Without the witness the class parameter `To` would appear nowhere in the argument list and no call could ever decide it. The same pattern appears in `Try` (`from_output`, `from_residual`, and `?`) and in `Propagate`; see `Try`, `?`, and `#try`.

The four ways to pass a type

Form	Call site	Runtime slot
`#comptime v: ssize`	A constant expression	None; folded into the body
`#comptime T: type`	A type name	None
`T: type`	A type name	None; promoted only if T is used in the signature
`#empty: $T`	A type name or any typed expression	None; the expression is discarded

`T: type` and the `$T` capture form belong to Generic Functions, which covers their ordering and defaulting rules. The `#empty:` marker also exists on struct fields, where it declares a phantom type parameter; that use is in `using`, `#empty`, and `#as`.

7.7 Generic Functions

A procedure is generic when its signature introduces a type variable. There are two spellings. A `$` prefix on a name inside a parameter annotation captures that name as a type parameter and infers it from the argument; a parameter declared

`T: type` names the type parameter explicitly and is supplied by the caller. Both produce the same thing: a template that is instantiated once per distinct set of type arguments.

GLOSSO

```
#import "Io";
#import "Meta";

Box :: struct { value: $A; }

idf   :: (x: $T) -> T { return x; }
wider :: (a: $T, b: T) -> T { if a > b return a; return b; }
first :: (xs: []mut $T) -> T { return xs[0]; }
deref :: (p: *mut $T) -> T { return p.*; }
unbox :: (b: Box($T)) -> T { return b.value; }
tag   :: (a: $A, b: $B) -> s64 { return size_of(A) * 100 + size_of(B); }

main :: () {
    print("idf=% idf=%\n", idf(3), idf("hi"));
    print("wider=%\n", wider(4, 9));
    arr: [3]s64 = .[10, 20, 30];
    print("first=%\n", first(arr[..]));
    n: s64 = 7;
    print("deref=%\n", deref(&n));
    bx: Box(s64) = .{ .value = 41 };
    print("unbox=%\n", unbox(bx));
    print("tag=%\n", tag(cast(s16)1, cast(s8)2));
}
```

CONSOLE / OUTPUT

```
idf=3 idf=hi
wider=9
first=10
deref=7
unbox=41
tag=201
```

The captured name is an ordinary type name inside the signature and the body: `size_of(A)`, a local `d: T`, a `cast(T)`, and a `where` clause all see it. Distinct captures in one signature are independent type parameters, so `tag` is generic in two.

Where a `$` capture may appear

The capture does not have to be the whole annotation. It may sit at any depth inside it, and the compiler matches the annotation's shape against the argument's type to solve for it. That is what makes `(xs: []$T)` mean "a view of anything" rather than "a view of a type literally named T".

Annotation	Binds T to
`x: $T`	The argument's own type
`xs: []$T`	The element type of a view
`xs: [..]$T`	The element type of a dynamic array
`p: *$T`	The pointee type
`p: [*]$T`	The pointee type of a many-pointer
`o: ?$T`	The payload of an option
`r: $E!$T`	Two parameters: the error type and the payload
`b: Box($T)`	The argument of a generic aggregate
`f: ($T) -> $R`	The parameter and result of a function type

GLOSSO

```
#import "Io";

takes_opt  :: (o: ?$T) -> T { if #pattern .Some(v) = o return v; d: T; return d; }
takes_res  :: (r: $E!$T) -> T { if #pattern .Ok(v) = r return v; d: T; return d; }
takes_many :: (p: [*]mut $T) -> T { return p[0]; }
takes_fn   :: (f: ($T) -> $R, x: T) -> R { return f(x); }

dbl :: (x: s64) -> s64 { return x * 2; }

main :: () {
    o: ?s64 = .Some(9);
    print("opt=%\n", takes_opt(o));
    r: string!s64 = .Ok(4);
    print("res=%\n", takes_res(r));
    arr: [3]s64 = .[1,2,3];
    print("many=%\n", takes_many(arr.data));
```

```
    print("fn=%\n", takes_fn(dbl, 21));
}
```

CONSOLE / OUTPUT

```
opt=9
res=4
many=1
fn=42
```

Only types are captured. There is no capture for the length of a fixed array: `xs: [$N]$T` is `error: bracket type expects [], [*], [..], [N], or [_]`. A procedure that has to work over fixed arrays of several lengths takes a view instead, and reads `xs.count`.

The rebinding rule

A `$` prefix declares the parameter. Every later mention of the same name, in the same signature, is written plain. Repeating the `$` is `error: polymorph $T is already bound; use plain T after the first $T`, and the rule applies at any depth: `(a: []$T, b: []$T)` is rejected for the same reason `(a: $T, b: $T)` is.

GLOSSO

```
zip_len :: (a: []mut $T, b: []mut T) -> ssize {
    if a.count < b.count return a.count;
    return b.count;
}
```

The reason is that a `$` name is a binder, not a wildcard. Two independent element types are two names, `$A` and `$B`; one shared element type is `$T` then `T`. Nothing about the annotation would otherwise distinguish the two intentions.

Passing a captured parameter explicitly

Captured type parameters may also be supplied at the call, positionally and before the runtime arguments. This matters when inference has nothing to work from, which is the case for a type variable that appears only in the return type: such a variable becomes a synthetic type parameter appended after the ones bound by the parameter list.

GLOSSO

```
#import "Io";

explicit :: (x: $T) -> T { return x; }
zero     :: () -> $R { d: R; return d; }
convert  :: (x: $T) -> $R { return cast(R) x; }

main :: () {
    print("explicit=%\n", explicit(s16, 3));
    print("zero=%\n", zero(f64));
    print("convert=%\n", convert(ssize, f64, 3));
}
```

CONSOLE / OUTPUT

```
explicit=3
zero=0
convert=3
```

The order is the declaration order: parameter captures first, then return-only captures. Supplying too few produces a diagnostic that spells the order out, so `convert(f64, 3)` is `error: call convert supplies 1 compact generic type(s), but unresolved order is (T, R)`.

Explicit `type` parameters

A parameter annotated `type` is a type-valued parameter. It is promoted to a real type parameter when its name is used elsewhere in the signature; if it is used only in the body it stays an ordinary compile-time value of type `type`, which behaves the same at the call site.

GLOSSO

```
#import "Io";
#import "Meta";

make      :: (T: type, n: ssize) -> ssize { d: T; return n + size_of(T); }
idt       :: (T: type, x: T) -> T { return x; }
conv      :: (x: ssize, T := f64) -> T { return cast(T) x; }
defaulted :: (T: type = ssize, n: ssize = 3) -> ssize { d: T; return n + size_of(T); }

main :: () {
    print("make=%\n", make(s16, 7));
    print("idt=%\n", idt(s64, 3));
    print("conv default=%\n", conv(3));
```

```
    print("conv f32=%\n", conv(3, f32));
    print("defaulted=%\n", defaulted());
    print("defaulted s8=%\n", defaulted(s8, 10));
}
```

CONSOLE / OUTPUT

```
make=9
idt=3
conv default=3
conv f32=3
defaulted=11
defaulted s8=11
```

Rule	Effect
`T: type` before every runtime parameter	Accepted
`T: type` after a runtime parameter	`type parameter 'T' must appear before runtime parameters unless it has a default`
`T := <Type>` after a runtime parameter	Accepted; the name must start with an uppercase letter
`T: type` twice	`type parameter 'T' is already declared`
`(T: type, x: $T)`	`type parameter 'T' is already declared; use plain T after the declaration`

NOTE

The trailing-position rule accepts only the inferred spelling. Writing `f :: (x: ssize, T: type = f64)` still fails with `error: type parameter 'T' must appear before runtime parameters unless it has a default`, even though the parameter has one; the accepted form is `T := f64`, with no `: type` annotation and an uppercase initial letter.

One specialization per distinct argument tuple

Generics are monomorphised. The compiler instantiates the body once for each distinct tuple of type arguments and reuses that instantiation for every call that resolves to it. Four calls to one generic procedure over three distinct types give three functions.

GLOSSO

```
#import "Io";

pick :: (a: $T, b: T) -> T { if a > b return a; return b; }

main :: () {
    print("%\n", pick(cast(s64)1, cast(s64)2));
    print("%\n", pick(cast(s64)3, cast(s64)4));
    print("%\n", pick(cast(s16)5, cast(s16)6));
    print("%\n", pick(1.5, 2.5));
}
```

CONSOLE / OUTPUT

```
define i64 @glo_pick__s64_s64(i64 %0, i64 %1) {
define i16 @glo_pick__s16_s16(i16 %0, i16 %1) {
define double @glo_pick__f64_f64(double %0, double %1) {
```

The mangled name is the procedure name followed by the argument types, which is exactly the specialization key; the two `s64` calls share one definition. Because the key is the whole tuple, a procedure generic in two parameters distinguishes the orders: `tag :: (a: $A, b: $B)` yields `glo_tag__s64_s16` and `glo_tag__s16_s64` as separate functions.

Instantiation is also where a generic body is really checked. Two procedures may not share a name, so a generic procedure cannot be split into per-type overloads: `f :: (x: $T) -> string where Is_Integer(T)` next to `f :: (x: $T) -> string where Is_Float(T)` is `error: function 'f' is already declared`. Selecting behaviour by type is done with `#if` inside one body, or with typeclass instances; see Static Branching with `#if` and How an Instance Is Chosen.

The guards that restrict which types may instantiate a generic procedure are `where` clauses, covered in Constraints with `where`.

7.8 Constraints with `where`

A `where` clause is a compile-time boolean guard attached to a declaration. It is evaluated at each call, after the call's type arguments have been solved and before the candidate is accepted. A clause that evaluates to false removes the declaration from consideration; if nothing is left, the call fails.

GLOSSO

```
#import "Io";
#import "Meta";
#import "String_Builder";
#import "Strings/Memory";

scaled :: (x: $T, k: T) -> T #inline where Is_Numeric(T) where !Is_Bool(T) {
    return x * k;
}

rep :: (#comptime n: ssize, s: string) -> string where n > 0 {
    builder := String_Builder.make_len_cap();
    for 0..n builder.write_string(s);
    return builder.into_string();
}

main :: () {
    print("scaled=%\n", scaled(6, 7));
    print("scaled=%\n", scaled(1.5, 2.0));
    repeated := rep(3, "ab");
    defer repeated.free();
    print("rep=%\n", repeated);
}
```

CONSOLE / OUTPUT

```
scaled=42
scaled=3
rep=ababab
```

Placement and repetition

The clauses come after the parameter list, after the return type, and after every function directive, immediately before the body. The order is fixed: `f :: (a: ssize) where true #inline { }` is `error: function directives must appear before where clauses`.

A declaration may carry any number of `where` clauses. They are conjoined, so repeating the keyword means "and". Writing `where A where B` and `where A && B` are the same constraint; the repeated form is usually easier to read when the predicates are long.

GLOSSO

```
both :: (x: $T) -> T where is_int_like(T) where T != s32 { return x; }
conj :: (x: $T) -> T where Show(T) && PartialEq(T, T) { return x; }
```

Compile-time predicates over type parameters

Any expression the compile-time evaluator can reduce to a `bool` is a legal clause, and a type parameter is an ordinary compile-time value inside it. Type equality, `size_of`, and the `Is_*` reflection helpers from `Meta` are all available; so is a user procedure taking a `type`.

GLOSSO

```
#import "Io";

is_int_like :: (T: type) -> bool { return T == ssize || T == s64 || T == s32; }

double  :: (x: $T) -> T where is_int_like(T) { return x + x; }
show_it :: (x: $T) -> string where Show(T) { return show(x); }
both    :: (x: $T) -> T where is_int_like(T) where T != s32 { return x; }
conj    :: (x: $T) -> T where Show(T) && PartialEq(T, T) { return x; }

main :: () {
    print("double=%\n", double(21));
    print("show_it=%\n", show_it(1.5));
    print("both=%\n", both(cast(s64)3));
    print("conj=%\n", conj(7));
}
```

CONSOLE / OUTPUT

```
double=42
show_it=1.5
both=3
conj=7
```

A clause may also constrain a `#comptime` value parameter rather than a type. `rep :: (#comptime n: ssize, s: string) -> string where n > 0` accepts `rep(3, "ab")` and rejects `rep(0, "ab")`, because the guard sees the constant the caller supplied.

Typeclass predicates

A class name applied to a type is an evidence query: `Show(T)` is a compile-time `bool` meaning "does `T` have a `Show` instance?". This is the form to reach for whenever the body calls a class method, because it states the requirement the body actually has. Multi-parameter classes are written with all their arguments, as in `PartialEq(T, T)`.

Evidence queries are not general expressions. The evaluator handles a bare query and a conjunction of queries; anything else is `error: non-constant function call` at the declaration.

Clause	Accepted
`where Show(T)`	Yes
`where Show(T) && PartialEq(T, T)`	Yes
`where Show(T) || PartialEq(T, T)`	No: `non-constant function call`
`where !Show(T)`	No: `non-constant function call`
`where Show(T) == false`	No: `non-constant function call`
`where !Is_Bool(T)`	Yes: an ordinary predicate, not an evidence query

The asymmetry is real and worth remembering: negation works on a `(T: type) -> bool` procedure and does not work on a class name. A body that has to branch on the absence of an instance uses `#if` instead, which does accept `!Class(T)`.

GLOSSO

```
#import "Io";

describe :: (x: $T) -> string {
    #if Show(T) return show(x);
    else return "opaque";
}

Plain :: struct { v: ssize; }

main :: () {
    p: Plain = .{ .v = 1 };
    print("% %\n", describe(7), describe(p));
}
```

CONSOLE / OUTPUT

```
7 opaque
```

What a failed constraint looks like

A call whose only candidate fails its guard reports the guard, not the body: `double("hi")` for the `double` above is `error: constraint failed for double`. The diagnostic names the declaration and points at the call, and it does not say which clause failed, so a declaration with several clauses is easier to diagnose when they are split across repeated `where` keywords.

> **NOTE**
> A `where` clause naming something that does not exist is not reported at the declaration. `f :: (x: $T) -> ssize where Show(U)`, with no `U` anywhere, compiles; every call to `f` then fails with `error: constraint failed for f`. A constraint that rejects every argument is usually a misspelled type-parameter name.

The definition-site pre-check

A generic body is normally checked once per instantiation, which means a mistake in it surfaces at a call site rather than at the declaration. To narrow that gap the checker also runs the body once at its definition with every type parameter bound to a skolem: an opaque type that supports exactly the classes the declaration's `where` clauses name, and nothing else. An operator or method used on a skolem is resolved against that evidence set instead of against the global instance table.

The pre-check is deliberately one-sided. It reports only what the declared evidence provably cannot support, and it stays silent everywhere a decision would need the concrete type. Literals do not default against a skolem, class methods resolve abstractly with no dictionary, and `#meaningful` probes are not run at all, because a structural probe asks a question about a concrete type by construction. What it finds it reports as a warning as a warning, never as an error, so a program the pre-check complains about still compiles and runs.

An operator the declared evidence cannot supply is exactly what the pre-check reports. A body that adds two values of a type parameter whose `where` clause names only `Show` draws `warning: no '+' for <T>: the where-clause only provides Show` at the declaration, with `note: reported at the definition; add the class to the 'where' clause`. The error itself is raised by the first instantiation that lacks the instance, and it is anchored inside the body rather than at the call; that error aborts the compilation, so the program below reports it alone and the warning is not printed beside it.

GLOSSO

```
#import "Io";

sum_two :: (a: $T, b: T) -> T where Show(T) { return a + b; }

Point :: struct #derive(Show) { x: ssize; }

main :: () {
    p: Point = .{ .x = 1 };
    print("%\n", sum_two(p, p));
}
```

CONSOLE / OUTPUT

```
error: no operator '+' for Point, Point
  --> sum.glo:3:56
  |
3 | sum_two :: (a: $T, b: T) -> T where Show(T) { return a + b; }
  |                                                        ^
```

The same file with the call removed compiles clean, which is the other half of the picture: a generic procedure that is never instantiated is not checked in any other way, and its body can name things that do not exist.

GLOSSO

```
#import "Io";

never_called :: (a: $T) -> T { print("%\n", totally_unbound_name); return a; }

main :: () { print("x\n"); }
```

CONSOLE / OUTPUT

```
x
```

That program compiles and runs. Adding one call to `never_called` is what turns `totally_unbound_name` into an error. The practical rule is that a generic procedure is only as checked as its call sites make it, so a library generic wants a test that instantiates it.

Where else `where` appears

The same clause syntax attaches to a typeclass declaration, where it states a superclass requirement, and to an instance, where its clauses are copied onto every method and associated type of that instance. Those are covered in Typeclasses: Declaring a Contract and in Instances: Heads, Bodies, and `where`.

A constraint that asks whether an operation typechecks at all, rather than whether a named class is implemented, is written with `#meaningful`; a predicate such as `Addable :: (T: type) -> bool { return #meaningful { p, q: T; p + q; }; }` is a legal `where` predicate and is the subject of Structural Constraints with `#meaningful`.

7.9 Structural Constraints with `#meaningful`

`#meaningful` asks the checker a question: would this code type-check? It takes a block, checks every statement in it against the current type environment, and folds to `true` if all of them passed and `false` if any of them failed. The verdict is an ordinary compile-time `bool`, so it can initialize a constant, guard a `#if`, stand in a `where` clause, or be the condition of an `#assert`.

This is a structural constraint: it is satisfied by the shape of the type rather than by a registered instance. A typeclass predicate asks whether evidence was declared; `#meaningful` asks whether the expression the generic body actually writes would resolve. Field access, indexing, calls to ordinary generic procedures and operator resolution all fall inside its reach, and none of them is a typeclass query.

GLOSSO

```
#import "Io";

Vec   :: struct { x: s64; y: s64; }
Plain :: struct { a: s64; }

(Vec, Vec) ::: instance Add {
    Output :: Vec;
    '+' :: (a: Vec, b: Vec) -> Vec { return .{ .x = a.x + b.x, .y = a.y + b.y }; }
}

ADDS_VEC   :: #meaningful { p, q: Vec;   p + q; };
ADDS_PLAIN :: #meaningful { p, q: Plain; p + q; };

main :: () {
    print("Vec=% Plain=%\n", ADDS_VEC, ADDS_PLAIN);
}
```

CONSOLE / OUTPUT
```
Vec=1 Plain=0
```

The block is checked, never executed

Statements inside the probe are type-checked and then discarded. They are not lowered, not run at compile time, and not run at runtime, so a call written inside a probe has no effect on the program. The declarations a probe introduces are local to it and are gone when it finishes.

GLOSSO
```
#import "Io";

Vec :: struct { x: s64; }

calls := 0;
count :: () -> s64 { calls += 1; return calls; }

main :: () {
    ok := #meaningful { v: Vec; v.x; count(); };
    no := #meaningful { v: Vec; v.nope; };
    print("ok=% no=% calls=%\n", ok, no, calls);
}
```

CONSOLE / OUTPUT
```
ok=1 no=0 calls=0
```

The two spellings

A brace after `#meaningful` starts a block, which runs to its closing brace and may hold any number of statements. Without a brace the directive takes exactly one unary expression and wraps it in a single-statement block. The block form is the one to reach for by default, because it can declare the values the probe needs.

Form	Body	Ends at
`#meaningful { s1; s2; }`	any number of statements	the closing brace
`#meaningful <expr>`	one unary expression	the end of that unary expression

> **NOTE**
> The braceless form binds only as far as a unary expression, so any binary operator after it belongs to the enclosing expression. `#meaningful v.left == 3` reads as `(#meaningful v.left) == 3` and reports `error: no operator '==' for bool, ssize`. Write `#meaningful { v.left == 3; }` when the probe should cover the comparison.

How it differs from a typeclass predicate

`Class(T)` and `#meaningful` answer different questions, and they disagree. The evidence query consults the instance table for the class named; the probe re-runs the checker over the code written. A type can satisfy one and fail the other in either direction.

Query	Asks	Answers for
`Class(T)`	is there an instance of this class for `T`?	one named typeclass
`#meaningful { … }`	does this code type-check for these types?	whatever the block writes

A `distinct` type shows the split plainly. `Meters :: distinct f64;` declares no derive list, so it borrows no operators, yet the generic `Show` instance for floats still covers it. The evidence query says yes and the structural probe says no, and both are correct about the question they were asked.

GLOSSO
```
#import "Io";

Meters :: distinct f64;

SHOWS :: Show(Meters);
ADDS  :: #meaningful { p, q: Meters; p + q; };

main :: () {
    m: Meters = cast(Meters) 3.5;
    print("Show=% adds=% show=%\n", SHOWS, ADDS, show(m));
}
```

CONSOLE / OUTPUT
```
Show=1 adds=0 show=3.5
```

- Only the probe can constrain an operation no typeclass owns: a field by name, a member call, a call to an ordinary generic procedure.
- Only the evidence query can name a class that no single expression exercises, and it is the form that participates in instance selection. How an Instance Is Chosen covers that side.

In a `where` clause

A `where` clause accepts a `#meaningful` block directly; the clause evaluator recognises the node before it tries constant folding. The clause's environment is the candidate's binding environment, so the parameter names of the signature are in scope inside the probe.

Wrapping the probe in a `(T: type) -> bool` procedure gives it a name and, more usefully, makes it an ordinary compile-time boolean. Such a predicate composes with `!`, `&&` and `||` in a `where` clause, because the clause is then just a constant-foldable call.

GLOSSO
```
#import "Io";

Left  :: struct { left: s64; }
Right :: struct { right: s64; }

Has_Left  :: (T: type) -> bool { return #meaningful { v: T; v.left; }; }
Has_Right :: (T: type) -> bool { return #meaningful { v: T; v.right; }; }

// A probe written straight into the clause.
read_left :: (v: $T) -> s64 where #meaningful { v.left; } { return v.left; }

// A predicate composes with '!', '&&' and '||' like any other comptime bool.
only_left :: (v: $T) -> s64 where Has_Left(T) where !Has_Right(T) { return v.left; }

either :: (v: $T) -> s64 where Has_Left(T) || Has_Right(T) {
    #if Has_Left(T) return v.left;
    else return v.right;
}

main :: () {
    l: Left  = .{ .left = 1 };
    r: Right = .{ .right = 2 };
    print("read=% only=%\n", read_left(l), only_left(l));
    print("either=% %\n", either(l), either(r));
}
```

CONSOLE / OUTPUT
```
read=1 only=1
either=1 2
```

Calling `read_left` with a `Right` fails at the call site, with the same message any unsatisfied `where` clause produces. The probe's own reason is not attached here; `#assert` is the form that reports it.

CONSOLE / OUTPUT
```
error: constraint failed for read_left
   --> probe.glo:12:18
    |
12 |     print("%\n", read_left(r));
    |                  ^^^^^^^^^
```

A raw evidence query does not compose the same way. `where Class(T)` and `where A(T) && B(T)` are the only shapes the clause evaluator understands; `where Show(T) || PartialOrd(T, T)` is `error: non-constant function call`. Routing the same question through a `#meaningful` predicate is the way to get negation and disjunction.

In `#assert`

When a probe fails, the checker keeps the diagnostic that made it fail. `#assert` prints that reason underneath the assertion message, which turns an opaque "does not work" into the specific resolution failure. This is the most useful place to spend a probe.

GLOSSO
```
Vec :: struct { x: s64; }

main :: () {
    #assert #meaningful { p, q: Vec; p + q; }, "Vec does not support +";
}
```

CONSOLE / OUTPUT

```
error: assertion failed: Vec does not support +
  --> probe.glo:4:5
   |
4 |     #assert #meaningful { p, q: Vec; p + q; }, "Vec does not support +";
   |     ^^^^^^^
note: constraint is not meaningful for these types
note: no operator '+' for Vec, Vec
```

Probe hygiene and rollback

A probe is a transaction. Nothing it caused survives it unless some non-probe context asks for the same work: constants it forced are restored to their previous state, source it generated is dropped, its `#insert` expansions are unwound, and every generic instance it triggered is marked speculative. A speculative instance is skipped during lowering, so asking whether `sort_items(x)` would compile does not put `sort_items` in the binary. A later real call clears the mark and the instance is emitted normally.

Errors do not escape either, and that includes deliberate ones. A `#compile_error` reached while checking a probe becomes the probe's `false` answer rather than a compile failure, so a generic guarded by `#compile_error` can still be asked about.

GLOSSO

```
#import "Io";

strict :: (v: $T) -> s64 {
    #if !#meaningful { v.left; } #compile_error "needs a left field";
    return v.left;
}

Left :: struct { left: s64; }

PROBE :: #meaningful { n: s64; strict(n); };

main :: () {
    l: Left = .{ .left = 4 };
    print("probe=% call=%\n", PROBE, strict(l));
}
```

CONSOLE / OUTPUT

```
probe=0 call=4
```

Probes nest, and each answers only for itself. The checker stashes the enclosing failure reason before running an inner probe and puts it back when that probe succeeds, so the reason cited against a constraint is the one that constraint produced rather than whichever probe happened to run last. An outer probe over a call whose own `where` clause failed reports `constraint failed for take_left`, not the inner probe's missing-field message.

The verdict is computed once per probe and cached on the expression together with its reason. Consumers that run later, such as an `#assert` diagnostic or overload resolution, read the cached answer instead of re-running the block, which is why a failing probe can still explain itself.

> **NOTE**
> A `where #meaningful` clause is part of a typeclass method's signature key. An instance method must repeat an equivalent probe or the instance fails its minimal check: probing `value.right` against a class that probed `value.left` gives `error: instance 'Carrier :: Witness' does not satisfy the implicit minimal definition; missing inspect`. `#minimal` and Required Methods covers the comparison.

7.10 Lambdas

A lambda is an anonymous procedure written inline. The parameter list is delimited by `|` rather than parentheses, which is what keeps it distinguishable from a parenthesised expression; `||` is the empty parameter list, and `||->` is the empty list followed by a return arrow. The lexer produces `||` and `||->` as single tokens and the parser splits them.

A lambda's value is a function pointer. It can initialize a local, be passed as an argument, sit in an array, and convert to any `#fn_ptr` type with a matching signature. It cannot capture; that restriction is what lets the value be a bare code pointer.

GLOSSO

```
#import "Io";

apply :: (f: (ssize, ssize) -> ssize, a: ssize, b: ssize) -> ssize { return f(a, b); }

main :: () {
    inc: (ssize) -> ssize = |x| x + 1;              // parameter type from the target type
    zero: () -> ssize = || 0;                      // empty parameter list
    dbl := |x: ssize| -> ssize { return x * 2; };   // annotated, block body
    z2  := ||->ssize { return 5; };                // no parameters, explicit return type
```

```
    add := |a, b| a + b;                        // parameters decided by the first call
    print("inc=% zero=% dbl=% z2=%\n", inc(4), zero(), dbl(21), z2());
    print("add=%\n", add(1, 2));
    print("apply=%\n", apply(|a, b| a * b, 6, 7));
}
```

CONSOLE / OUTPUT

```
inc=5 zero=0 dbl=42 z2=5
add=3
apply=42
```

The spellings

Spelling	Parameters	Body
`|x| x + 1`	one, type from context	one expression, implicitly returned
`|x: ssize| x + 1`	one, annotated	one expression
`|x: ssize| -> ssize { … }`	one, annotated	block, return type written
`|a, b| a * b`	two, types from context	one expression
`|| 0`	none	one expression
`||->ssize { … }`	none	block, return type written
`|noalias p: *ssize| -> ssize { … }`	one, `noalias`	block

An explicit return type requires a block. Without one the body is a single expression that is wrapped in an implicit `return`, and there is nowhere for the arrow to attach: `|x: ssize| -> ssize x * 2` is `error: a lambda with an explicit return type requires a block`. A block body without a return type is fine; the type comes from the body.

- Named results work in a lambda return type exactly as in a procedure: `|v: s64| -> (lo: s64, hi: s64) { lo = v % 10; hi = v / 10; }` declares both names and returns them.
- Repeating a parameter name is `error: duplicate lambda parameter 'x'`, and omitting the separator is `error: expected ',' or '|' after lambda parameter`.

Where the parameter types come from

An unannotated parameter has no type of its own. The checker gives the lambda a signature of inference variables and defers it. Two things can fill those variables in: an expected type at the point the lambda is written, or the arguments of the first call made through it.

Source	Example	When
the target type	`inc: (ssize) -> ssize = |x| x + 1;`	at the binding
the callee's parameter type	`apply(|a, b| a * b, 6, 7)`	at the argument position
the first call's arguments	`add := |a, b| a + b; add(1, 2);`	at that call

The first call decides the parameters permanently, and a compile-time number reaching a parameter takes its default type rather than staying a literal, because a parameter is a runtime binding and needs a representation. So `add := |a, b| a + b; add(1, 2); add(1.0, 2.0);` fixes the parameters at `ssize` and then rejects the second call with `error: callback argument comptime_float != ssize`.

A lambda with nothing at all to fix its parameters is rejected at the binding rather than at some later use: a lone `f := |a| a + 1;` is `error: cannot infer a type here; annotate it or give it a use that fixes one`.

The return type comes from the body

Only the parameters have to be decided from outside. Once they are concrete the body is checked and whatever it produces is the return type, which is why an unannotated lambda can return a `string` built from an `s64` parameter. When both an annotation and a target type are present they must agree: `f: Op = |x: ssize| -> string { … };` against `Op :: #fn_ptr(a: ssize) -> ssize;` is `error: lambda return type is string, but the expected type is ssize`.

GLOSSO

```
#import "Format";
#import "Io";
#import "String_Builder";
#import "Strings/Memory";
```

```
find_if :: (values: []mut $T, predicate: (T) -> bool) -> ssize {
    for values { if predicate(it) return it_index; }
    return -1;
}

main :: () {
    // The return type is whatever the body produces; nothing is annotated.
    render := |x: s64| tprint("%!", x);
    print("%\n", render(7));

    // A lambda as a callback: the parameter type comes from the callee's
    // signature after $T is bound.
    storage: [4]s64 = .[3, 9, 12, 5];
    print("index=%\n", find_if(storage[..], |v| v > 10));
}
```

CONSOLE / OUTPUT

```
7!
index=2
```

Captures

There are no closures. A lambda body may read anything with static storage, which means globals, top-level constants and other procedures, but reaching an enclosing local is rejected. A captured local would have to be copied into an environment record and carried alongside the code pointer, and a lambda is a bare code pointer.

GLOSSO

```
main :: () {
    outer_total := 1;
    f := |x: ssize| -> ssize { return x + outer_total; };
    _ := f(1);
}
```

CONSOLE / OUTPUT

```
error: lambda captures local 'outer_total', but captured closures are not supported
  --> capture.glo:3:39
  |
3 |     f := |x: ssize| -> ssize { return x + outer_total; };
  |                                         ^^^^^^^^^^^
help: pass the value as an explicit callback or context argument
```

The expression-body form reports the weaker `error: unbound 'outer_total'`, because the name never resolves far enough to be recognised as a capture. Either way the fix is the same: pass the value as an extra parameter, or put it in the implicit context.

GLOSSO

```
#import "Io";

SCALE :: 10;
offset := 3;

Bin_Op :: #fn_ptr(a: s64, b: s64) -> s64;
C_Op   :: #fn_ptr(v: s64) -> s64 #c_call;

main :: () {
    // Globals and constants are visible; enclosing locals are not.
    scaled := |x: s64| -> s64 { return x * SCALE + offset; };
    print("scaled=%\n", scaled(4));

    // Named results in a lambda return type.
    split := |v: s64| -> (lo: s64, hi: s64) { lo = v % 10; hi = v / 10; };
    a, b := split(47);
    print("lo=% hi=%\n", a, b);

    // noalias parameters.
    load := |noalias p: *mut s64| -> s64 { return p.*; };
    n: s64 = 8;
    print("load=%\n", load(&n));

    // A lambda converts to a named function-pointer type, including a #c_call one.
    op: Bin_Op = |a: s64, b: s64| -> s64 { return a - b; };
    c: C_Op = |v: s64| -> s64 { return v + 1; };
    print("op=% c=%\n", op(9, 4), c(41));
}
```

CONSOLE / OUTPUT

```
scaled=43
lo=7 hi=4
load=8
op=5 c=42
```

Lambdas as values

A lambda and a named procedure produce the same kind of value, so they mix freely in a table of function pointers, and a block-bodied lambda assigned to a `#fn_ptr` variable takes its parameter from the target type and its result from the body.

GLOSSO

```
#import "Io";

Op :: #fn_ptr(v: s64) -> s64;

named :: (v: s64) -> s64 { return v + 1; }

main :: () {
    f: Op = |v| { return v * 3; };
    print("f=%\n", f(4));

    table: [3]Op = .[named, f, |v: s64| -> s64 { return -v; }];
    for table print("table[%]=%\n", it_index, it(6));
}
```

CONSOLE / OUTPUT

```
f=12
table[0]=7
table[1]=18
table[2]=-6
```

NOTE

A lambda cannot appear in a top-level initializer. Both `bump: Op = |v: s64| -> s64 { return v + 1; };` and the constant form `BUMP : Op : |v: s64| -> s64 { … };` fail with `error: unknown const __glo_lambda_0`. Declare a named procedure at file scope and assign that instead; the lambda forms are for use inside a body.

7.11 Function Pointers

`#fn_ptr` declares a named function-pointer type. It is a top-level declaration form, written after `::` like any other type declaration, and it may not be spelled inline in a type position: writing `f: #fn_ptr(v: s64) -> s64` as a parameter type is `error: unexpected token 'fn_ptr'`. For an unnamed function type there is a separate syntax, described below.

GLOSSO

```
Bin_Op  :: #fn_ptr(a: s64, b: s64) -> s64;
Sink    :: #fn_ptr(v: s64);
Nullary :: #fn_ptr() -> s64;
Cb      :: #fn_ptr(v: s64) -> s64 #c_call;
Copy    :: #fn_ptr(noalias dst: *mut s64, noalias src: *s64);
Varargs :: #fn_ptr(fmt: *s8, ...) -> s32 #c_call;
```

Parameter names are mandatory here, even though they contribute nothing to the type. The parser reads each parameter as `name: type` and a bare type is a parse error: `#fn_ptr(s64, s64) -> s64` reports `error: unexpected token ','` at the comma that follows the name it thought it was reading. The arrow and return type are optional and default to `void`.

The suffixes

Suffix	On a `#fn_ptr` type	On a function declaration
`#c_call`	part of the type's identity	C calling convention, no implicit context
`#no_context`	accepted and ignored	suppresses the implicit context parameter

A convention mismatch in either direction is `error: got fnptr, expected fnptr`. The message repeats itself because both sides print as `fnptr`; the difference is in the convention, not in the parameters. A `#foreign` declaration does not carry `#c_call` implicitly, so a foreign function assigns to a plain `#fn_ptr` type and not to a `#c_call` one.

GLOSSO

```
#import "Io";

Bin_Op :: #fn_ptr(a: s64, b: s64) -> s64;
Sink   :: #fn_ptr(v: s64);
Cb     :: #fn_ptr(v: s64) -> s64 #c_call;
Bare   :: #fn_ptr(v: s64) -> s64 #no_context;

add   :: (a: s64, b: s64) -> s64 { return a + b; }
shout :: (v: s64) { print("sink %\n", v); }
twice :: (v: s64) -> s64 #c_call { return v * 2; }
plain :: (v: s64) -> s64 #no_context { return v + 100; }
```

```
main :: () {
    op: Bin_Op = add;
    s: Sink = shout;
    c: Cb = twice;
    b: Bare = plain;
    print("op=% c=% b=%\n", op(2, 3), c(21), b(1));
    s(7);
}
```

CONSOLE / OUTPUT

```
op=5 c=42 b=101
sink 7
```

- Parameter names are not part of the identity: a value of `#fn_ptr(a: s64) -> s64` assigns to `#fn_ptr(different: s64) -> s64`.
- `noalias` is. `Copy :: #fn_ptr(noalias dst: *mut s64, noalias src: *s64);` rejects a procedure without both markers with `error: cannot implicitly convert fnptr to fnptr because their noalias parameter contracts differ`. `noalias` Pointer Parameters covers the marker itself.
- Inside a `#no_context` or `#c_call` body, `context` is unreachable: `error: cannot access 'context' in a #c_call / #no_context procedure`.

C varargs in a function-pointer type

A bare `...` may be the last parameter, with no name and no type. It requires the C convention: without `#c_call` the declaration is `error: C vararg function pointer 'Varargs' requires #c_call`. The type then accepts a `#c_call` vararg function, including a `#foreign` one.

> **NOTE**
> A call made through a vararg function pointer accepts only the fixed parameters. `s: Summer = sum_s64;` is fine, but `s(3, 10, 20, 12)` against `Summer :: #fn_ptr(count: ssize, ...) -> s64 #c_call;` reports `error: callback expects 1 argument(s), got 4`. Call the vararg function by name when the extra arguments matter.

The anonymous function type

Any parenthesised type list followed by `->` is a function type. This is the form to use in a parameter, a field, or anywhere else a type is expected. Names and `noalias` are optional; the names are documentation only.

GLOSSO

```
#import "Io";

call0 :: (f: () -> s64) -> s64 { return f(); }
call1 :: (f: (s64) -> s64, v: s64) -> s64 { return f(v); }
call2 :: (f: (x: s64, y: s64) -> s64, v: s64) -> s64 { return f(v, v); }
callN :: (f: (noalias p: *mut s64) -> s64, p: *mut s64) -> s64 { return f(p); }

five :: () -> s64 { return 5; }
add  :: (a: s64, b: s64) -> s64 { return a + b; }
load :: (noalias p: *mut s64) -> s64 { return p.*; }

main :: () {
    k: s64 = 12;
    print("% % % %\n", call0(five), call1(|v: s64| -> s64 { return v + 1; }, 8),
          call2(add, 6), callN(load, &k));
}
```

CONSOLE / OUTPUT

```
5 9 12 12
```

Without the arrow the same parentheses are a tuple type, and a single unnamed element in parentheses is just that type. `(s64, string)` is a two-element tuple; `(s64, string) -> bool` is a function type. `noalias` outside a function type is `error: noalias is only valid on function parameters`.

Null and function pointers

A `#fn_ptr` type excludes the zero address, in the same way `*T` and `[*]T` do, because the optional form takes the null representation as its niche. `?Op` is therefore the same size as `Op` on a 64-bit target and costs nothing to carry. An absent callback is `.None`, and reading it back goes through a pattern match like any other option payload.

GLOSSO

```
#import "Io";
#import "Meta";
```

```
Op :: #fn_ptr(v: s64) -> s64;

Hook :: struct { procedure: ?Op; }

twice :: (v: s64) -> s64 { return v * 2; }

call_or :: (h: Hook, v: s64) -> s64 {
    if #pattern .Some(f) = h.procedure return f(v);
    return -1;
}

main :: () {
    empty: Hook;
    print("none=%\n", call_or(empty, 5));
    filled: Hook = .{ .procedure = twice };
    print("some=%\n", call_or(filled, 5));
    print("Op=% ?Op=%\n", size_of(Op), size_of(?Op));
}
```

CONSOLE / OUTPUT

```
none=-1
some=10
Op=8 ?Op=8
```

This is the shape the standard library uses for every vtable slot. `Allocator :: struct { procedure: ?Allocator_Proc; data: ?*void; }` relies on `.None` being address zero, so a zero-initialized `Allocator` is the "none installed" value without any extra tag.

Producing a function pointer from an address is a checked cast. At `-O0` and `-O1` a zero address aborts with `cast to fnptr of an address that is zero`. Hosted Windows and Linux x86-64 use exit status 134; WASI uses 125. The check fires for a written literal too: `cast(Op) 0` builds and then aborts rather than being rejected during checking. At `-O2` and above the check is not emitted. Runtime Safety Checks and Panics covers the family.

> **NOTE**
> `f: Op = null;` currently compiles and produces a value the program must not call: at `-O0` the resulting binary traps, and at higher levels the call is undefined. `null` denotes `.None` and belongs to an optional type; write `?Op` when a slot may be empty.

7.12 Function and Aggregate Directives

There is no unknown-directive diagnostic. A name the parser does not accept in that position simply fails to parse, and the error names the token rather than the concept.

GLOSSO

```
f :: () #bogus { }
main :: () {}
```

CONSOLE / OUTPUT

```
error: unexpected token 'bogus'
  --> directive.glo:1:9
   |
1 | f :: () #bogus { }
   |         ^^^^^^
```

File and top-level directives

Directive	Form	Effect
`#enable` / `#disable`	`#enable(feature[, mode]);`	opens or closes a feature gate, sticky to the end of the file
`#import`	`#import[,only(…)] "M";`, or `,hide(…)`	loads a standard-library module, optionally under an alias
`#load`	`#load "path";`	loads an isolated relative module; `#load,embed` shares this unit; `#export` reexports a relative dependency
`#library`	`N :: #library "file";` with `,system`, `,dyn` or `,static`	declares a foreign library handle for `#foreign`
`#thread_local`	`#thread_local <global var decl>`	gives the variable thread-local storage

Directive	Form	Effect
`#comptime`	`#comptime { … }` / `X :: #comptime e;`	runs during compilation
`#insert`	`#insert <string expr>;`	splices generated declarations into the file

NOTE
Module-level `#if` selects declaration blocks at compile time, with optional `else #if` and `else` branches. Inactive branches contribute no imports, libraries, or declarations. Operator syntax must remain unconditional. `#thread_local` is narrower: it prefixes only a global variable declaration.

Function directives

Ordinary function directives are written after the return type and before the `where` clauses. The order is fixed, and reversing it is diagnosed rather than accepted, because the clause parser stops looking for directives once a `where` has been seen.

CONSOLE / OUTPUT

```
error: function directives must appear before where clauses
  --> order.glo:1:26
   |
1 | f :: (a: s64) where true #inline { }
   |                          ^^^^^^^
```

Directive	Argument	Meaning
`#operator`	`left`, `right` or `assign`, with an optional integer level	declares binary fixity and associativity; a level must be declared somewhere for the spelling
`#operator`	`prefix` or `suffix`, with no level	declares unary fixity; `suffix, try(Target)` also passes the enclosing Try boundary type
`#expand`	none, or `,expression`	macro; an explicit non-`void` return type also makes it value-producing
`#magic`	`"name"`	compiler intrinsic; the declaration has no body
`#foreign`	a library handle or `"file"`, then an optional `"symbol"`	foreign function; no body
`#c_call`	none	C calling convention; no implicit context
`#no_context`	none	do not pass the implicit context
`-> #Never`	return type	the call never returns; replaces the removed `#noreturn` directive
`#returns_twice`	none	setjmp-style; the call may resume more than once
`#dump`	none	print this function's typed IR during compilation
`#must`	none	discarding the result is an error
`#inline`	none, or `,always` / `,never`	inlining hint, or a forced decision
`#modify`	an expression or a block	rewrites the type parameters before instantiation
`#memory { ... }`	a leading effect block before the declaration	checked body contract or external contract; historical trailing placement is also accepted
`#fresh_result`	none; #foreign only	declares a result independent of argument storage; the foreign API must satisfy that promise

Directive	Argument	Meaning
`#unchecked_transfer`	none	accepted on raw storage helpers; it adds no lifetime protection and does not make moved values reusable

`#inline,sometimes` is `error: #inline expects always or never after ','`, and `#expand,stmt` is `error: #expand expects expression after ','`. `#foreign` naming a handle that was never declared is `error: #foreign refers to unknown library 'nolib' (declare it with #library)`, and a `#library` modifier outside the three link kinds is `error: #library modifier must be system, dyn, or static`.

`#expand,expression` is the concise inferred expression form. A macro with an explicit non-`void` return type is also an expression macro even without that modifier; it may use a statement body and multiple `return` sites, but every reachable path must return a value.

`#must` is the one whose diagnostic a reader will meet by accident. It fires on a call statement whose value is dropped, and the help names the way to say the drop was deliberate.

CONSOLE / OUTPUT

```
error: discarded return value of #must function 'make'
  --> must.glo:3:14
   |
3 | main :: () { make(1); }
   |              ^^^^
help: bind it explicitly, for example `_ := make(...);`, if discarding is intentional
```

`#dump` writes to the compiler's own output, not the program's. The dump is produced while the function is lowered, so a procedure nothing calls is never lowered and prints nothing at all.

GLOSSO

```
addd :: (a: s64, b: s64) -> s64 #dump { return a + b; }
main :: () { _ := addd(1, 2); }
```

CONSOLE / OUTPUT

```
; ---- #dump typed-ir addd(a: ssize, b: ssize) -> ssize  [4 ins] ----
   0  temp_mark v2
   1  magic v3 'int_add' [v0,v1]
   2  temp_restore v2
   3  ret v3
; ---- end addd ----
```

Parameter directives

Three directives may prefix a parameter, after the `using` and `noalias` keywords and before the name. `#comptime` and `#lazy` may appear together in either order; each may appear once.

Directive	Form	Meaning
`#comptime`	`#comptime n: s64`	the argument must be compile-time known; the parameter has no runtime existence
`#lazy`	`#lazy v: bool`	the argument is evaluated at first use inside the callee, and at most once
`#empty`	`#empty: $T` or `#empty: $A, $B`	a type-only slot; the caller passes the type positionally

GLOSSO

```
#import "Io";

pow     :: (#comptime n: s64, x: s64) -> s64 { r: s64 = 1; #inline for i: 0..n r *= x; return r; }
zero_of :: (#empty: $T) -> T { d: T; return d; }
pick    :: (take: bool, #lazy value: s64, fallback: s64) -> s64 { if take return value; return fallback; }

calls := 0;
costly :: () -> s64 { calls += 1; return 5; }

main :: () {
    print("pow=% zero=%\n", pow(3, 2), zero_of(s64));
    print("lazy=% calls=%\n", pick(false, costly(), 1), calls);
}
```

CONSOLE / OUTPUT
```
pow=8 zero=0
lazy=1 calls=0
```

Aggregate and field directives

Aggregate directives sit between the `struct`, `union`, `typeclass` or `distinct` keyword and the body. Each kind accepts a different set, and `enum` and `enum_flags` accept none at all.

Target	Accepted	Notes
`struct`	`#c_call` `#packed` `#align(N)` `#modify` `#derive(…)` `#magic "…"`	`#c_call` and `#packed` both lock declaration order
`union`	`#raw` `#derive(…)` `#magic "…"`	`#raw` makes it a reinterpreting union with no tag
`enum`, `enum_flags`	none	`Color :: enum #derive(Show)` is `error: unexpected token 'derive'`
`typeclass`	`#derive(…)` `#minimal(…)`	`#derive` here names the class's own derive macro
`distinct`	`#derive(…)` `#derive,all`	selects which classes the nominal type borrows
struct field	`using` `#as` `#empty: T[, T…];`	written before the field name
union field	`#from`	generates a `Propagate` instance from the payload type

`#magic` on an aggregate names a layout the compiler already knows, the way `Context_Storage :: struct #magic "jump_context_storage_struct" {}` does in `std/Jump.glo`. It is not an extension point: a name the compiler does not recognise is `error: unknown aggregate #magic "nope_struct"`.

`#c_call` and `#packed` both switch a struct off the default layout, which sorts fields by alignment and then size. `#c_call` keeps declaration order with normal padding, so a C header's layout is reproduced; `#packed` keeps declaration order and removes the padding. Struct Memory Layout covers the ordering rule itself.

GLOSSO
```
#import "Io";
#import "Meta";

Reordered :: struct { a: u8; b: u64; c: u8; }
C_Layout  :: struct #c_call { a: u8; b: u64; c: u8; }
Packed    :: struct #packed { a: u8; b: u64; c: u8; }

main :: () {
    print("reordered=% c_call=% packed=%\n",
          size_of(Reordered), size_of(C_Layout), size_of(Packed));
}
```

CONSOLE / OUTPUT
```
reordered=16 c_call=24 packed=10
```

Statement, expression, and type positions

The remaining directives are not attached to a declaration at all. Several spellings appear in more than one of these positions and mean something different in each; `#comptime` and `#inline` are the two that catch readers out, and What Runs When and `#inline for` respectively cover them.

Position	Directives
type	`#Context`, `#Never`, `#Self_Type`, `#aos [..]T`, `#soa [..]T`
switch and case	`#partial` on the switch, `#falling` on a case

`#inline` outside a loop is rejected by name: `error: #inline can only prefix while or for`. `#Context` is the only spelling of the built-in context type, and writing `Context` without the `#` reports `error: the built-in Context type is written #Context`.

> **NOTE**
> `#char` is not a parser directive. The lexer consumes the whole `#char "A"` form and emits a numeric token carrying scalar value `65` plus a character-literal marker. The checker preserves that marker: ASCII defaults to `u8` and may adopt `s8` or `rune` context, while

> a non-ASCII value defaults to and accepts only `rune`. Because validation happens while lexing, malformed forms report `#char expects a string literal` or `unterminated #char string literal`. `#char` Character Literals covers the complete contextual-typing and escape rules.

`#public`, `#public(unit)`, `#private`	declaration prefix or group; also an inherent block header after `:::`	declaration audience
`#self`, `*#self`, `*#self`	first parameter of an inherent method	receiver value or pointer
`#Self_Type`	type in an inherent block or instance	the complete matched owner type
`#on_drop_error(value, error) { ... }`	after an owning local declaration	handles failure from automatic cleanup

8 Typeclasses

8.1 Typeclasses: Declaring a Contract

A typeclass is a named contract over one or more type parameters. It is introduced by an ordinary constant declaration whose value is a `typeclass` body, so the form is `Name :: typeclass (P: kind, ...) { ... }`. The body lists requirements: methods that an instance must supply and associated types that an instance must choose. A class declares no representation and produces no runtime value of its own; it fixes a set of names and signatures that instances then fill in.

GLOSSO

```
#import "Format";
#import "Io";
#import "String_Builder";

Describable :: typeclass (T: type) #minimal(describe) {
    describe :: (value: T) -> string;
}

Port :: struct { number: s64; }

Port ::: instance Describable {
    describe :: (value: Port) -> string { return tprint("port %", value.number); }
}

main :: () {
    p: Port = .{ .number = 80 };
    print("%\n", describe(p));
}
```

CONSOLE / OUTPUT

```
port 80
```

Class names live in a single global namespace, and so do method names. Re-declaring a name the standard library already uses is `error: typeclass 'Show' is already declared`, and declaring a method whose name another class owns is `error: function 'show' is already owned by typeclass 'Show'`. The second rule applies between two user classes as well, so a class body may not reuse `show`, `fmap`, `bind`, or any other name already claimed. The reason is dispatch: a method call is resolved by name across every instance in the program, so one name can belong to only one contract.

The declaration header

The header between `typeclass` and the opening brace has a fixed clause order. Everything after the parameter list is optional, but a `where` clause may not follow a directive: `#minimal(f) where Base(T)` is `error: unexpected token 'where'`.

Clause	Count	Position
`(P: kind, ...)`	exactly one	immediately after `typeclass`
`where <expr>`	zero or more	after the parameter list
`#derive(macro, ...)`	zero or more	after the `where` clauses
`#minimal(formula)`	at most one	after the `where` clauses

A trailing comma is allowed in the parameter list. `#derive` may appear more than once; the macro it names is what generates an instance when a type asks for one, which is the subject of Deriving Instances with `#derive`. A second `#minimal` is `error: duplicate #minimal directive on typeclass 'K'`.

Parameter kinds

A parameter is written as an ordinary annotated binding, and the annotation is its kind. `type` binds a type and an arrow binds a type constructor. A `#comptime` string parameter binds a value, as in `Formatter`. The number of parameters decides the shape of every instance head for the class.

Kind	Binds	Standard-library example
`T: type`	one type	`Show(T)`, `Iterable(T)`, `Try(Carrier)`
`L: type, R: type`	a pair of types	`Add(L, R)`, `PartialEq(L, R)`
`F: (type) -> type`	a one-argument type constructor	`Functor(F)`, `Monad(M)`

Kind	Binds	Standard-library example
`#comptime spec: string`	a compile-time string value	`Formatter(T, spec, W)`

GLOSSO

```
#import "Format";
#import "Io";
#import "String_Builder";

// one type parameter
Tagged :: typeclass (T: type) #minimal(tag_of) {
    tag_of :: (value: T) -> string;
}

// two type parameters
Blend :: typeclass (L: type, R: type) #minimal(blend) {
    Output :: type;
    blend :: (left: L, right: R) -> Output;
}

// a type-constructor parameter
Holder :: typeclass (F: (type) -> type) #minimal(peek) {
    peek :: (container: F($A)) -> A;
}

Cell :: struct { value: $A; }

s64 ::: instance Tagged { tag_of :: (value: s64) -> string { return "s64"; } }

(s64, string) ::: instance Blend {
    Output :: string;
    blend :: (left: s64, right: string) -> string { return tprint("%%", left, right); }
}

Cell ::: instance Holder {
    peek :: (container: Cell($A)) -> A { return container.value; }
}

main :: () {
    n: s64 = 3;
    c: Cell(string) = .{ .value = "inside" };
    print("%\n", tag_of(n));
    print("%\n", blend(7, "x"));
    print("%\n", peek(c));
}
```

CONSOLE / OUTPUT

```
s64
7x
inside
```

The three heads differ because the classes do. A one-parameter class takes a single type; a two-parameter class takes a parenthesised argument tuple; a constructor parameter takes the bare constructor name, with evidence asked at the applied type. Arrows of higher arity are accepted, so `(type, type) -> type` binds a two-argument constructor. Value parameters must be marked `#comptime` and currently require `string`; another type reports `compile-time typeclass value parameters currently require string`. A mixed head such as `(Token, "token", $W)` therefore fixes both a type and a selector while leaving the writer generic. Instance heads are the subject of Instances: Heads, Bodies, and `where`, and constructor parameters of Higher-Kinded Typeclasses.

> **NOTE**
> A class declared with an empty parameter list parses, but it can never acquire evidence, because an evidence query is a question about a type argument and there is nowhere to put one. Its instance methods fail that check at every call, so `ping(n)` is `error: no overload 'ping' accepts these arguments` even when an instance declares exactly that signature. Give every class at least one parameter.

Methods and associated types

A class body holds two kinds of member. A member whose value starts with a parenthesis is a method; any other member is an associated type. The distinction is made at parse time, which is why there are no associated constants: a non-parenthesised member is read with the type parser.

Member	Meaning
`Name :: <type-expression>;`	an associated type an instance must choose
`name :: (params) -> Ret;`	a method requirement with no default
`name :: (params) -> Ret { ... }`	a method requirement with a default body

Member	Meaning
`'op' :: (params) -> Ret #operator(...);`	an operator requirement

GLOSSO

```
#import "Format";
#import "Io";
#import "String_Builder";

Describable :: typeclass (T: type) #minimal(describe) {
    describe :: (value: T) -> string;
    describe_twice :: (value: T) -> string {
        return tprint("% / %", describe(value), describe(value));
    }
}

Port :: struct { number: s64; }
Host :: struct { name: string; }

Port ::: instance Describable {
    describe :: (value: Port) -> string { return tprint("port %", value.number); }
}

Host ::: instance Describable {
    describe :: (value: Host) -> string { return tprint("host %", value.name); }
    describe_twice :: (value: Host) -> string { return "host pair"; }
}

label_for :: (value: $T) -> string where Describable(T) {
    return describe_twice(value);
}

main :: () {
    p: Port = .{ .number = 80 };
    h: Host = .{ .name = "local" };
    print("%\n", label_for(p));
    print("%\n", label_for(h));
}
```

CONSOLE / OUTPUT

```
port 80 / port 80
host pair
```

`Port` inherits the default `describe_twice`; `Host` replaces it. An instance method always beats a class default, because selection drops every default candidate as soon as one instance candidate has matched. A default body is therefore a fallback, never a preference. The class parameters are in scope through the whole body as plain names, and writing one of them again as a generic is `error: typeclass parameter 'T' is already declared; use plain T inside the typeclass method`.

Superclass constraints

A `where` clause on the class declaration is a superclass constraint: it is a condition on the class parameters that must hold before any instance of the class counts. The clauses are evaluated on every evidence query, ahead of the instance list.

GLOSSO

```
#import "Io";

Weighted :: typeclass (T: type) #minimal(weight_of) {
    weight_of :: (value: T) -> s64;
}

Ranked :: typeclass (T: type) where Weighted(T) #minimal(rank_of) {
    rank_of :: (value: T) -> s64;
}

Crate :: struct { kilos: s64; }
Crate ::: instance Weighted { weight_of :: (value: Crate) -> s64 { return value.kilos; } }
Crate ::: instance Ranked   { rank_of   :: (value: Crate) -> s64 { return 1; } }

Balloon :: struct { kilos: s64; }
Balloon ::: instance Ranked { rank_of :: (value: Balloon) -> s64 { return 2; } }

RANKED_CRATE   :: Ranked(Crate);
RANKED_BALLOON :: Ranked(Balloon);

main :: () {
    c: Crate = .{ .kilos = 5 };
    print("Ranked(Crate)=% Ranked(Balloon)=%\n", RANKED_CRATE, RANKED_BALLOON);
    print("rank %\n", rank_of(c));
}
```

CONSOLE / OUTPUT

```
Ranked(Crate)=1 Ranked(Balloon)=0
rank 1
```

The `Balloon` instance is accepted at its declaration: a superclass clause is not checked when an instance is written, only when the class is consulted. The declaration is simply inert. Every query answers `0` and every call is rejected, so calling `rank_of` on a `Balloon` in the program above is `error: no overload 'rank_of' accepts these arguments`. Supplying the missing `Weighted` instance revives the `Ranked` one with no other edit.

The standard library uses this to layer contracts: `Eq :: typeclass (T: type) where PartialEq(T, T) {}` is a marker over structural equality, `Applicative` requires `Functor`, `Monad` requires `Applicative`, and `Fractional` requires `Num(T) && Div(T, T)`. A single clause may join conditions with `&&`, and several clauses may be chained; all of them are conjoined.

What a class declaration emits

- A compile-time predicate under the class's own name. `Describable(Port)` is a constant boolean, described in Evidence, Constraints, and Negative Instances.
- One globally owned overload set per method name. Instance methods and class defaults are candidates in that set; nothing else may declare the name.
- One projectable name per associated type, written `Class(args).Name` in type position and covered in Associated Types.
- For an operator member, the spelling, arity, fixity and precedence, all registered globally. The class alone makes `a <*> b` parse; only the instance is missing, and the call is `error: no operator '<*>' for ssize, ssize` until one exists. The Operator System covers the rules.

8.2 `#minimal` and Required Methods

`#minimal` states the smallest set of members an instance must supply for the class to consider it complete. Members outside that set are expected to have default bodies that are written in terms of the members inside it, so the directive is what makes a class with several mutually defined methods usable: an instance picks whichever member it can implement directly and the rest follow.

GLOSSO

```
#import "Io";

Signed_Pair :: typeclass (T: type) #minimal(low | high) {
    low  :: (value: T) -> s64 { return -high(value); }
    high :: (value: T) -> s64 { return -low(value); }
}

Floor :: struct {}
Ceil  :: struct {}

Floor ::: instance Signed_Pair { low  :: (value: Floor) -> s64 { return 5; } }
Ceil  ::: instance Signed_Pair { high :: (value: Ceil)  -> s64 { return 9; } }

main :: () {
    f: Floor;
    c: Ceil;
    print("Floor low=% high=%\n", low(f), high(f));
    print("Ceil  low=% high=%\n", low(c), high(c));
}
```

CONSOLE / OUTPUT

```
Floor low=5 high=-5
Ceil  low=-9 high=9
```

Both members carry default bodies, so without the directive neither would be required, an empty instance would be accepted, and `low` and `high` would call each other forever. `#minimal(low | high)` states the real obligation: implement one of the two, and the other is derived.

The formula grammar

The formula is a small propositional language over method names. It is parsed by a dedicated parser, not by the expression parser, so the operator table plays no part and the spellings below are the whole vocabulary.

GRAMMAR

```
minimal     = disjunction ;
disjunction = conjunction { "|" conjunction } ;
conjunction = atom { "," atom } ;
atom        = method-name | quoted-operator | "(" disjunction ")" ;
```

Form	Meaning
`name`	the method `name` is implemented
`'op'`	the operator member `op` is implemented

Form	Meaning
`A, B`	both `A` and `B`
`A | B`	either `A` or `B`
`( ... )`	grouping; empty parentheses are an error

Conjunction binds tighter than disjunction, so `a, b | c` means `(a, b) | c`. Parentheses are needed only for the other reading. An empty group is `error: #minimal parentheses cannot be empty`, and a dangling separator is `error: expected a method name after ',' in #minimal` or the matching message for `|`. A name that is not a method of the class is `error: #minimal references unknown typeclass method nope`.

GLOSSO

```
#import "Io";

Split :: typeclass (T: type) #minimal((head_of, tail_of) | whole_of) {
    head_of  :: (value: T) -> s64 { return whole_of(value) / 10; }
    tail_of  :: (value: T) -> s64 { return whole_of(value) % 10; }
    whole_of :: (value: T) -> s64 { return head_of(value) * 10 + tail_of(value); }
}

Joined :: struct {}
Parted :: struct {}

Joined ::: instance Split { whole_of :: (value: Joined) -> s64 { return 47; } }
Parted ::: instance Split {
    head_of :: (value: Parted) -> s64 { return 1; }
    tail_of :: (value: Parted) -> s64 { return 2; }
}

main :: () {
    j: Joined;
    p: Parted;
    print("Joined head=% tail=% whole=%\n", head_of(j), tail_of(j), whole_of(j));
    print("Parted head=% tail=% whole=%\n", head_of(p), tail_of(p), whole_of(p));
}
```

CONSOLE / OUTPUT

```
Joined head=4 tail=7 whole=47
Parted head=1 tail=2 whole=12
```

Operator atoms

An operator member is named in a formula with its quoted spelling, the same form used to declare it. Quoting is required: the formula parser reads an unquoted operator token as punctuation, and `#minimal(+)` is `error: operators in #minimal must use quoted operator names`.

GLOSSO

```
#import "Io";

Nearby :: typeclass (L: type, R: type) #minimal('~=' | '!~') {
    '~=' :: (a: L, b: R) -> bool #operator(left, 8) { return !(a !~ b); }
    '!~' :: (a: L, b: R) -> bool #operator(left, 8) { return !(a ~= b); }
}

Reading :: struct { value: s64; }

(Reading, Reading) ::: instance Nearby {
    '~=' :: (a: Reading, b: Reading) -> bool {
        d := a.value - b.value;
        if d < 0 d = -d;
        return d <= 2;
    }
}

main :: () {
    x: Reading = .{ .value = 10 };
    y: Reading = .{ .value = 11 };
    z: Reading = .{ .value = 20 };
    print("x ~= y  %\n", x ~= y);
    print("x !~ z  %\n", x !~ z);
}
```

CONSOLE / OUTPUT

```
x ~= y  1
x !~ z  1
```

This is exactly the shape of the Prelude's `PartialEq`, which is declared `#minimal('==' | '!=')` with each operator defaulting to the negation of the other. An unsatisfied operator formula reports the quoted names back: `error: instance '(Z, Z) :: Nearby' does not satisfy #minimal('~=' | '!~')`.

The empty formula and the implicit minimal

`#minimal()` requires nothing. It marks a class whose purpose is the evidence itself rather than any method, and it is what lets an instance body be empty even when the class declares members.

GLOSSO

```
#import "Io";

Trusted :: typeclass (T: type) #minimal() {
    audit :: (value: T) -> s64 { return 0; }
}

Ledger :: struct {}
Ledger ::: instance Trusted {}

TRUSTED_LEDGER :: Trusted(Ledger);

main :: () {
    l: Ledger;
    print("audit=% evidence=%\n", audit(l), TRUSTED_LEDGER);
}
```

CONSOLE / OUTPUT

```
audit=0 evidence=1
```

Omitting the directive is not the same as writing `#minimal()`. With no directive the class gets an implicit minimal: every method requirement that has no default body must be implemented. That is the common case and the reason most classes need no directive at all. Its failure has its own wording, and it names the members that are missing.

GLOSSO

```
Pairish :: typeclass (T: type) {
    first  :: (value: T) -> s64;
    second :: (value: T) -> s64;
    both   :: (value: T) -> s64 { return first(value) + second(value); }
}

Empty :: struct {}
Empty ::: instance Pairish {}

main :: () {}
```

CONSOLE / OUTPUT

```
error: instance 'Empty :: Pairish' does not satisfy the implicit minimal definition; missing first, second
  --> minimal.glo:8:1
  |
8 | Empty ::: instance Pairish {}
  | ^^^^^
```

How satisfaction is decided

The check runs at declaration time, once per instance group, before any call is resolved. For each member the class requires, the compiler builds a signature key from the name, the arity, the fixity, and the parameter and return annotations with the head arguments and the instance's associated types substituted in; a supplied method satisfies a requirement when the two keys agree. The formula is then evaluated over the requirements that were matched.

GLOSSO

```
Bounded :: typeclass (T: type) #minimal(least, greatest) {
    least    :: (value: T) -> s64;
    greatest :: (value: T) -> s64;
}

Span :: struct {}
Span ::: instance Bounded {
    least :: (value: Span) -> s64 { return 1; }
}

main :: () {}
```

CONSOLE / OUTPUT

```
error: instance 'Span :: Bounded' does not satisfy #minimal(least, greatest)
  --> formula.glo:7:1
  |
7 | Span ::: instance Bounded {
  | ^^^^
note: minimal complete definition declared here
  --> formula.glo:1:32
  |
1 | Bounded :: typeclass (T: type) #minimal(least, greatest) {
  |                                 ^^^^^^^^
```

Because the comparison is on whole signature keys, a method that is present but wrong reports as absent. Declaring `size_of_it :: (value: Z) -> s32` against a requirement returning `s64`, or adding a parameter the requirement does not have, both produce `error: instance 'Z :: K' does not satisfy #minimal(size_of_it)` rather than a separate mismatch message. When a formula fails, comparing the supplied signature against the class declaration character by character is the way to read it. The message re-renders the formula from its parse, so grouping parentheses are dropped: `#minimal((head_of, tail_of) | whole_of)` reports as `#minimal(head_of, tail_of | whole_of)`.

A supplied method that matches no requirement at all is reported separately, once the minimal formula is satisfied: `error: instance method hint does not match any declaration in typeclass 'K'`. An instance may therefore not carry a helper of its own; helpers belong outside the instance body.

> **NOTE**
> A `#modify` block on an instance method changes the signature key the compiler computes, so the instance fails its `#minimal` check even when the body is correct. Express the condition as a `where` clause on the instance instead; that path is checked by evaluating the clause, not by comparing keys.

Diagnostic	Cause
`does not satisfy #minimal(...)`	the formula is unsatisfied, or a member's signature differs
`does not satisfy the implicit minimal definition; missing ...`	no directive, and a defaultless requirement was not supplied
`instance method X does not match any declaration in typeclass 'C'`	a supplied member matches no requirement
`#minimal references unknown typeclass method nope`	an atom names something the class does not declare
`duplicate #minimal directive on typeclass 'K'`	two directives on one class

8.3 Instances: Heads, Bodies, and `where`

An instance supplies a class's members for particular type arguments. It is written `<head> ::: instance Class { ... }`, where the head names the type or types the instance is for and the body holds the methods and associated type values. An instance has no name of its own and is never referred to directly; it is found by the head, through the ordinary overload machinery described in How an Instance Is Chosen.

GLOSSO

```
#import "Io";
#import "Meta";

Area_Of :: typeclass (T: type) #minimal(area_of) {
    area_of :: (value: T) -> f64;
}

Rect :: struct { w: f64; h: f64; }
Disc :: struct { r: $T; }

Rect ::: instance Area_Of {
    area_of :: (value: Rect) -> f64 { return value.w * value.h; }
}

Disc($T) ::: instance Area_Of where Is_Float(T) {
    area_of :: (value: Disc(T)) -> f64 { return 3.14159 * cast(f64)(value.r * value.r); }
}

main :: () {
    r: Rect = .{ .w = 2.0, .h = 3.0 };
    d: Disc(f32) = .{ .r = 1.0 };
    print("% %\n", area_of(r), area_of(d));
}
```

CONSOLE / OUTPUT

```
6 3.14159
```

The head is every token before the `:::`. The parser collects those tokens raw, then re-lexes and re-parses them as a type annotation, which is why a head may be any type expression and why a malformed one reports the text it collected: `Z + ::: instance K` is `error: invalid instance head 'Z+'`. The `:::` is what ends the scan, so omitting it does not give a bad head; it gives no instance at all, and the declaration fails as `error: unexpected token 'instance'`.

The head supplies exactly one argument per class parameter. For a one-parameter class the whole head is that argument; for a class with more than one, the head is a parenthesised tuple read left to right. A head with the wrong number of

arguments cannot be substituted into the requirement signatures, so it surfaces as a `#minimal` failure rather than as an arity message.

Head shapes

Head	Matches
`s64`	one nominal type
`Box($A)`	every instantiation of the generic struct `Box`
`Pair(s64, $B)`	a partially concrete instantiation
`Box`	the bare constructor, for a class with a `(type) -> type` parameter
`$T`	any type; narrowed by `where`
`[]$T`	a view over any element type
`[..]$T`	a dynamic array
`*$T`	a pointer
`[*]$T`	a read-only many-pointer
`?$T`	the built-in option type
`$E!$T`	the built-in result type, error first
`(A, B)`	one argument per parameter of a multi-parameter class

GLOSSO

```
#import "Io";
#import "Meta";

Label_Of :: typeclass (T: type) #minimal(label_of) {
    label_of :: (value: T) -> string;
}

Box  :: struct { value: $A; }
Pair :: struct { a: $A; b: $B; }

s64          ::: instance Label_Of { label_of :: (value: s64) -> string { return "s64"; } }
Box($A)      ::: instance Label_Of { label_of :: (value: Box(A)) -> string { return "Box"; } }
$T           ::: instance Label_Of where Is_Float(T) { label_of :: (value: T) -> string { return "float"; } }
[]mut $T         ::: instance Label_Of { label_of :: (value: []mut T) -> string { return "view"; } }
*mut $T          ::: instance Label_Of { label_of :: (value: *mut T) -> string { return "pointer"; } }
[*]$T  ::: instance Label_Of { label_of :: (value: [*]T) -> string { return "read-only many-pointer"; } }
?$T          ::: instance Label_Of { label_of :: (value: ?T) -> string { return "option"; } }
$E!$T        ::: instance Label_Of { label_of :: (value: E!T) -> string { return "result"; } }
Pair(s64,$B) ::: instance Label_Of { label_of :: (value: Pair(s64, B)) -> string { return "Pair(s64, B)"; } }

main :: () {
    n: s64 = 1;
    b: Box(string) = .{ .value = "x" };
    f: f64 = 1.5;
    storage: [3]s64;
    v: []mut s64 = storage[..];
    p: *mut s64 = &n;
    m: [*]s64 = v.data;
    o: ?s64 = .Some(2);
    r: string!s64 = .Ok(3);
    q: Pair(s64, string) = .{ .a = 1, .b = "y" };
    print("% % % % % % % % %\n",
        label_of(n), label_of(b), label_of(f), label_of(v), label_of(p),
        label_of(m), label_of(o), label_of(r), label_of(q));
}
```

CONSOLE / OUTPUT

```
s64 Box float view pointer read-only many-pointer option result Pair(s64, B)
```

A `$X` anywhere in the head declares a section generic that is bound when the head is matched. Inside the body it is spelled without the `$`, as `Box(A)` and `[]T` above, because the binder is already introduced. The body may use it in parameter annotations, in return types, and in associated type values.

Structural heads match on the exact form, not on what converts to it. The `[]mut $T` instance does not accept a `[..]s64` or a `[*]s64`, and the `[*]$T` instance does not accept a writable `[*]mut s64`; each is `error: no overload 'label_of' accepts these arguments`. Where two forms should behave alike, declare both heads, as the program above declares `[]mut $T` and `[*]$T` separately.

Tuple heads

A class with several parameters is keyed on the whole argument tuple, so `(s64, string)` and `(string, s64)` are different instances. This is what makes asymmetric operations expressible: a program may give `Mul` a `(Vector, f64)` instance and an `(f64, Vector)` instance with different bodies. A tuple head may mix concrete types, section generics, and a repeated binder.

GLOSSO

```
#import "Format";
#import "Io";
#import "Meta";
#import "String_Builder";

Convert :: typeclass (From: type, To: type) #minimal(convert) {
    convert :: (#empty: To, value: From) -> To;
}

Route :: typeclass (A: type, B: type, C: type) #minimal(route) {
    route :: (a: A, b: B, c: C) -> string;
}

Flag :: struct { on: bool; }

(s64, string) ::: instance Convert {
    convert :: (#empty: string, value: s64) -> string { return tprint("n%", value); }
}

($T, T) ::: instance Convert where Is_Float(T) {
    convert :: (#empty: T, value: T) -> T { return value + value; }
}

(Flag, $T) ::: instance Convert where Is_Integer(T) {
    convert :: (#empty: T, value: Flag) -> T { if value.on return cast(T)1; return cast(T)0; }
}

(s64, string, Flag) ::: instance Route {
    route :: (a: s64, b: string, c: Flag) -> string { return tprint("%%%", a, b, c.on); }
}

main :: () {
    print("%\n", convert(string, 7));
    print("%\n", convert(f64, 1.5));
    f: Flag = .{ .on = true };
    print("%\n", convert(s64, f));
    print("%\n", route(2, "-", f));
}
```

CONSOLE / OUTPUT

```
n7
3
1
2-1
```

`($T, T)` is a single binder used twice: it matches only when both arguments are the same type. The `#empty:` parameter carries the second class parameter into the signature so that a call can decide it; witness parameters are covered in Compile-Time and Type-Only Parameters. Giving a two-parameter class a bare head such as `s64 ::: instance Convert` is `error: instance 's64 :: Convert' does not satisfy #minimal(convert)`, because nothing binds `To`.

Instance-level `where`

A `where` clause after the class name guards the whole instance. It is what turns a blanket head into a targeted one, and it is evaluated at compile time against the bound head arguments.

GLOSSO

```
#import "Io";
#import "Meta";

Sized_Note :: typeclass (T: type) #minimal(note_of) {
    note_of :: (value: T) -> string;
}

$T ::: instance Sized_Note where Is_Integer(T) && size_of(T) <= 4 {
    note_of :: (value: T) -> string { return "small integer"; }
}

$T ::: instance Sized_Note where Is_Integer(T) where size_of(T) > 4 {
    note_of :: (value: T) -> string { return "wide integer"; }
}

$T ::: instance Sized_Note where Is_Float(T) {
    note_of :: (value: T) -> string { return "float"; }
}

main :: () {
    a: s16 = 1;
    b: s64 = 1;
    c: f32 = 1.0;
```

```
    print("% / % / %\n", note_of(a), note_of(b), note_of(c));
}
```

CONSOLE / OUTPUT
```
small integer / wide integer / float
```

Several `where` clauses may be chained and are conjoined, and a single clause may join conditions with `&&`; the two spellings above are equivalent. A clause is an ordinary compile-time expression, so a type predicate, `size_of`, an equality on types, and an evidence query all work. The clauses are copied onto every member of the instance, so an associated type value is guarded exactly as the methods are: three blanket instances can supply three different associated types for three disjoint families of type.

> **NOTE**
> A `#modify` block is not accepted on an instance head: `$T ::: instance C #modify { ... }` is `error: unexpected token 'modify'`. On an instance method it parses but breaks the `#minimal` check. Instance conditions belong in `where`.

Instance bodies

A body holds two kinds of member, matching the class: a method, written like any procedure but with the head's type arguments substituted in, and an associated type value, written `Name :: <type>;`. A body may be empty, which is what a marker class and a negative instance use. A method may end at a `#magic` intrinsic instead of a body; the Prelude's arithmetic instances are written that way, as `'+' :: (left: T, right: T) -> Output #magic "int_add";`.

Every member must correspond to a declaration in the class. A member the class does not declare is `error: function 'g' has no owning typeclass`, which is also what naming an unknown class reports when the body holds a method; with an empty body the same mistake is `error: unknown typeclass 'Nonexistent'`. Associated type names are checked too: `error: typeclass 'Registry' has no associated type 'Wrong'`.

- There is no orphan rule. An instance may pair anyone's type with anyone's class, including a standard-library type with a standard-library class. Which instance wins is decided by specificity, not by locality; see Evidence, Constraints, and Negative Instances.
- `<head> ::: instance !Class { }` declares that the head explicitly does not implement the class. Negative instances are covered in Evidence, Constraints, and Negative Instances.

8.4 Associated Types

An associated type is a type that the instance chooses rather than the caller. The class declares the name and the instance supplies a value for it, so a single class parameter can carry a whole family of dependent types: `Indexable` names the element an index produces, `Try` names the output and the residual of a fallible carrier, `Add` names the type a sum has.

GLOSSO
```
#import "Io";

Collection :: typeclass (C: type) #minimal(first_of, count_of) {
    Elem :: type;
    first_of :: (container: C) -> Elem;
    count_of :: (container: C) -> ssize;
}

Words  :: struct { items: []string; }
Scores :: struct { items: []mut s64; }

Words ::: instance Collection {
    Elem :: string;
    first_of :: (container: Words) -> string { return container.items[0]; }
    count_of :: (container: Words) -> ssize { return container.items.count; }
}

Scores ::: instance Collection {
    Elem :: s64;
    first_of :: (container: Scores) -> s64 { return container.items[0]; }
    count_of :: (container: Scores) -> ssize { return container.items.count; }
}

main :: () {
    w_storage: [2]string;
    w_storage[0] = "alpha";
    w_storage[1] = "beta";
    s_storage: [2]s64;
    s_storage[0] = 4;
    s_storage[1] = 5;

    w: Words  = .{ .items = w_storage[..] };
    s: Scores = .{ .items = s_storage[..] };

    a: Collection(Words).Elem  = first_of(w);
```

```
    b: Collection(Scores).Elem = first_of(s);
    print("% %\n", a, b);
    print("% %\n", count_of(w), count_of(s));
}
```

CONSOLE / OUTPUT

```
alpha 4
2 2
```

On the class side the member is `Elem :: type;`: a name followed by a type expression and a semicolon, with no parentheses, which is what tells the parser it is not a method. On the instance side the same name is bound to a concrete type, `Elem :: string;`. Inside the class, the name is used as an ordinary type in the requirement signatures, and the minimal check substitutes the instance's value before comparing signatures, which is why `first_of :: (container: Words) -> string` satisfies `first_of :: (container: C) -> Elem`.

There is no default value. The type expression the class writes is a requirement, not a fallback: writing a concrete type such as `Key :: s64;` on the class side and omitting `Key` from the instance still fails the minimal check, because the requirement signatures keep an unresolved `Key` in them and nothing the instance supplies can match. The class side is not a constraint on the choice either: an instance that answers `Key :: string;` to a class that wrote `Key :: s64;` is accepted. The conventional spelling is therefore `Name :: type;`.

Projection

An associated type is read back with `Class(args).Name`. This is a type annotation form: it names a type and may appear anywhere a type may. The arguments are the class's parameters, so the projection is resolved by finding the instance for those arguments and reading its value.

GLOSSO

```
#import "Io";
#import "Meta";

Widen :: typeclass (T: type) #minimal(widen) {
    Wide :: type;
    widen :: (value: T) -> Wide;
}

$T ::: instance Widen where Is_Integer(T) { Wide :: s64; widen :: (value: T) -> s64 { return cast(s64)value; } }
$T ::: instance Widen where Is_Float(T)   { Wide :: f64; widen :: (value: T) -> f64 { return cast(f64)value; } }

main :: () {
    a: s16 = 7;
    b: f32 = 1.5;
    x: Widen(s16).Wide = widen(a);
    y: Widen(f32).Wide = widen(b);
    print("% %\n", x, y);
}
```

CONSOLE / OUTPUT

```
7 1.5
```

`Widen(s16).Wide` is `s64` and `Widen(f32).Wide` is `f64`, decided by the same `where` guards that decide which `widen` runs. The instance-level clauses are copied onto the associated type value as well as onto the methods, so a projection can never resolve through an instance whose guard is false.

Projection in generic code

The arguments of a projection may be generic binders, which is the form that matters: a generic procedure states its return type in terms of the caller's type without knowing it.

GLOSSO

```
#import "Io";

Collection :: typeclass (C: type) #minimal(first_of) {
    Elem :: type;
    first_of :: (container: C) -> Elem;
}

Words  :: struct { items: []string; }
Scores :: struct { items: []mut s64; }
Words  ::: instance Collection { Elem :: string; first_of :: (c: Words)  -> string { return c.items[0]; } }
Scores ::: instance Collection { Elem :: s64;    first_of :: (c: Scores) -> s64    { return c.items[0]; } }

head_of :: (container: $C) -> Collection(C).Elem where Collection(C) {
    return first_of(container);
}

main :: () {
```

```
    ws: [1]string; ws[0] = "alpha";
    ss: [1]s64;    ss[0] = 9;
    w: Words  = .{ .items = ws[..] };
    s: Scores = .{ .items = ss[..] };
    print("% %\n", head_of(w), head_of(s));
}
```

CONSOLE / OUTPUT

```
alpha 9
```

> **NOTE**
> A projection works in a return annotation but not in a parameter annotation. Adding `fallback: Collection(C).Elem` as a second parameter to `head_of` above and calling it makes the call `error: generic argument does not match call head_of`, even when the argument's type is exactly what the projection resolves to. Take such a value through a second generic binder and constrain it separately.

The argument-free projection

`Class.Name` with no arguments is accepted when exactly one instance in the program supplies that associated type. It is a convenience for a class with a single implementation, and it stops being legal the moment a second instance appears.

GLOSSO

```
#import "Io";

Registry :: typeclass (T: type) #minimal(key_of) {
    Key :: type;
    key_of :: (value: T) -> Key;
}

Session :: struct { id: s64; }
Session ::: instance Registry {
    Key :: s64;
    key_of :: (value: Session) -> s64 { return value.id; }
}

main :: () {
    s: Session = .{ .id = 12 };
    k: Registry.Key = key_of(s);
    print("%\n", k);
}
```

CONSOLE / OUTPUT

```
12
```

Adding a second instance turns the same annotation into an error, and the fix is to write the arguments.

GLOSSO

```
Registry :: typeclass (T: type) #minimal(key_of) {
    Key :: type;
    key_of :: (value: T) -> Key;
}

Session :: struct { id: s64; }
Ticket  :: struct { code: string; }

Session ::: instance Registry { Key :: s64;    key_of :: (value: Session) -> s64    { return value.id; } }
Ticket  ::: instance Registry { Key :: string; key_of :: (value: Ticket)  -> string { return value.code; } }

main :: () {
    k: Registry.Key = 1;
    _ := k;
}
```

CONSOLE / OUTPUT

```
error: associated type 'Registry.Key' is ambiguous; provide type arguments
   --> assoc.glo:13:8
    |
13 |     k: Registry.Key = 1;
    |        ^^^^^^^^^^^
```

Why the projection is not an expression

`Class(args).name` is also the spelling for a scoped typeclass method reference, and that is the reading the expression parser takes. The two forms are told apart by position, not by the name: in a type annotation the member is looked up among the class's associated types, and in an expression it is looked up among the class's methods.

GLOSSO

```
#import "Io";

Registry :: typeclass (T: type) #minimal(key_of) {
    Key :: type;
    key_of :: (value: T) -> Key;
}

Session :: struct { id: s64; }
Session ::: instance Registry {
    Key :: s64;
    key_of :: (value: Session) -> s64 { return value.id; }
}

main :: () {
    s: Session = .{ .id = 12 };
    k: Registry(Session).Key = Registry(Session).key_of(s);
    print("%\n", k);
}
```

CONSOLE / OUTPUT

```
12
```

The annotation projects the associated type and the initializer calls the method, from the same spelling of the same class at the same argument. Using an associated type in expression position therefore reports a missing method: `t := Registry(Session).Key;` is `error: typeclass 'Registry' has no method 'Key'`. Where the projected type is wanted as a value, declare a binding with the projection as its annotation and take `type_of(value)` of the binding.

Projecting another class

A projection may appear inside an instance, including in the value of that instance's own associated type. This is how a derived type is threaded through: the instance defers to whatever `Add` decided rather than committing to a concrete type.

GLOSSO

```
#import "Io";

Doubler :: typeclass (T: type) #minimal(twice) {
    Result_Of :: type;
    twice :: (value: T) -> Result_Of;
}

$T ::: instance Doubler where Add(T, T) {
    Result_Of :: Add(T, T).Output;
    twice :: (value: T) -> Add(T, T).Output { return value + value; }
}

main :: () {
    n: s64 = 21;
    x: f64 = 1.25;
    print("% %\n", twice(n), twice(x));
    d: Doubler(s64).Result_Of = twice(n);
    print("%\n", d);
}
```

CONSOLE / OUTPUT

```
42 2.5
42
```

Diagnostic	Cause
`no associated type value for Registry.Key`	the matching instance does not supply it
`associated type 'Registry.Key' is ambiguous; provide type arguments`	the argument-free form with more than one instance
`associated type arguments must follow the typeclass name`	written `Registry.Key(A)` instead of `Registry(A).Key`
`typeclass 'Registry' has no associated type 'Wrong'`	an instance supplies a name the class does not declare
`cyclic associated type definition involving 'Cy(A).Elem'`	a value that projects itself

8.5 Evidence, Constraints, and Negative Instances

Applying a class to a type produces a compile-time boolean: `Marker(s64)` asks whether `s64` has a `Marker` instance and folds to `1` or `0` during checking. This is the same query the compiler runs when it selects an instance, exposed to the program so that generic code can branch on it or demand it.

GLOSSO

```
#import "Io";
#import "Meta";

Marker :: typeclass (T: type) #minimal() {}
Other  :: typeclass (T: type) #minimal() {}

$T ::: instance Marker where Is_Integer(T) {}
$T ::: instance Other  where Is_Integer(T) {}

MARKED_S64 :: Marker(s64);
MARKED_F64 :: Marker(f64);

report :: (value: $T) -> string {
    #if Marker(T) return "marked";
    else return "unmarked";
}

both :: (value: $T) -> string where Marker(T) && Other(T) { return "both"; }

main :: () {
    #assert Marker(s16);
    n: s64 = 1;
    f: f64 = 1.0;
    print("constants = % %\n", MARKED_S64, MARKED_F64);
    print("static if = % %\n", report(n), report(f));
    print("where     = %\n", both(n));
    print("comptime  = %\n", #comptime Marker(u8));
}
```

CONSOLE / OUTPUT

```
constants = 1 0
static if = marked unmarked
where     = both
comptime  = 1
```

The query folds only in the contexts the constraint evaluator understands. A `where` clause accepts a bare query or several joined with `&&`; every other shape of `where` expression containing a query is rejected, because the evaluator recognises the call form itself rather than evaluating an arbitrary expression that contains it.

Context	Folds
`where Class(T)`	yes
`where A(T) && B(T)`	yes
`#if Class(T)`, `#if !Class(T)`	yes
`EV :: Class(T);` at file scope	yes
`#comptime Class(T)`, `#assert Class(T)`	yes
`where !Class(T)`	no: `error: non-constant function call`
`where Class(T) == false` or `where A(T) || B(T)`	no: `error: non-constant function call`
a runtime expression	no: `error: internal error: 'Marker' is called but was never lowered`

A negated or disjoint constraint is therefore written with `#if` inside the body rather than with `where` on the signature, or split into two procedures. Wrapping a query in a helper predicate does not work either: the evaluator needs to see the class name at the call site, so `where P(T)` where `P` returns `Class(T)` is `error: constraint failed for use_it`.

> **NOTE**
> An evidence query supplies the class's full compile-time argument list. `PartialEq(s64, s64)` asks about both operand types, just as it does in a `where` clause. Omitting an argument is diagnosed as `PartialEq expects 2 compile-time argument(s)`; it does not silently produce false evidence.

GLOSSO

```
#import "Io";

EQ_S64        :: Eq(s64);
EQ_F64        :: Eq(f64);
```

```
PARTIAL_EQ_S64 :: PartialEq(s64, s64);

main :: () {
    print("Eq(s64)=% Eq(f64)=% PartialEq(s64,s64)=%\n", EQ_S64, EQ_F64, PARTIAL_EQ_S64);
}
```

CONSOLE / OUTPUT

```
Eq(s64)=1 Eq(f64)=0 PartialEq(s64,s64)=1
```

Negative instances

`<head> ::: instance !Class { }` declares that the head does not implement the class. It is not the absence of an instance but a statement of absence, and it overrides any positive instance that would otherwise match: evidence is positive and not negative.

GLOSSO

```
#import "Io";
#import "Meta";

Tagged :: typeclass (T: type) #minimal(tag_of) {
    tag_of :: (value: T) -> string;
}

$T ::: instance Tagged where Is_Integer(T) { tag_of :: (value: T) -> string { return "integer"; } }
u8 ::: instance !Tagged {}

TAGGED_S64 :: Tagged(s64);
TAGGED_U8  :: Tagged(u8);

main :: () {
    n: s64 = 1;
    print("Tagged(s64)=% Tagged(u8)=%\n", TAGGED_S64, TAGGED_U8);
    print("%\n", tag_of(n));
}
```

CONSOLE / OUTPUT

```
Tagged(s64)=1 Tagged(u8)=0
integer
```

The exclusion reaches dispatch as well as queries. With the declaration above, calling `tag_of` on a `u8` value is `error: no overload 'tag_of' accepts these arguments`: the blanket instance matches the argument, then loses its evidence and is dropped. A generic procedure guarded by `where Tagged(T)` reports the constraint instead, as `error: constraint failed for guarded`.

A negative instance's body is not checked and is never used, so it may be empty even for a class with required methods, and a body written into one is dead. Its purpose is to carve an exception out of a blanket instance that is otherwise right. The Prelude uses it for floating point: `$T ::: instance !Eq where Is_Float(T) {}` keeps floats comparable with `==` while denying them the total-equality marker, because NaN is not equal to itself.

Coherence and overlap

Glosso has no orphan rule and no coherence check at declaration time. Two instances may cover the same type, and the program compiles until a call needs one of them. Overlap is reported at the use site, with a note on each of the two instances that tied.

GLOSSO

```
#import "Io";
#import "Meta";

Label_Of :: typeclass (T: type) #minimal(label_of) {
    label_of :: (value: T) -> string;
}

$T ::: instance Label_Of where Is_Integer(T) { label_of :: (value: T) -> string { return "one"; } }
$T ::: instance Label_Of where Is_Integer(T) { label_of :: (value: T) -> string { return "two"; } }

main :: () {
    n: s64 = 1;
    print("%\n", label_of(n));
}
```

CONSOLE / OUTPUT

```
error: ambiguous typeclass operation 'label_of' for these argument types
   --> overlap.glo:12:18
   |
12 |     print("%\n", label_of(n));
```

```
   |                 ^^^^^^^^
note: first matching instance is declared here
  --> overlap.glo:8:47
   |
8 | $T ::: instance Label_Of where Is_Integer(T) { label_of :: (value: T) -> string { return "two"; } }
   |                                                ^^^^^^^^
note: another matching instance is declared here
  --> overlap.glo:7:47
   |
7 | $T ::: instance Label_Of where Is_Integer(T) { label_of :: (value: T) -> string { return "one"; } }
   |                                                ^^^^^^^^
```

Most overlap is not ambiguous, because selection scores candidates and a concrete parameter outranks a generic one. A specific instance placed beside a blanket instance wins for the type it names and leaves the rest to the blanket, with no annotation and no ordering rule.

GLOSSO

```
#import "Io";
#import "Meta";

Label_Of :: typeclass (T: type) #minimal(label_of) {
    label_of :: (value: T) -> string;
}

$T  ::: instance Label_Of where Is_Integer(T) { label_of :: (value: T)   -> string { return "an integer"; } }
s64 ::: instance Label_Of                     { label_of :: (value: s64) -> string { return "specifically s64"; } }

main :: () {
    a: s64 = 1;
    b: s16 = 2;
    print("% / %\n", label_of(a), label_of(b));
}
```

CONSOLE / OUTPUT

```
specifically s64 / an integer
```

The same freedom applies across module boundaries. A program may add an instance for a standard-library type and a standard-library class, and it outranks the Prelude's blanket instance for the same type by being concrete.

GLOSSO

```
#import "Format";
#import "Io";
#import "String_Builder";

u8 ::: instance Show { show :: (value: u8) -> string { return tprint("byte(%)", cast(s64)value); } }

main :: () {
    b: u8 = 7;
    n: s64 = 7;
    print("% %\n", show(b), show(n));
}
```

CONSOLE / OUTPUT

```
byte(7) 7
```

> **NOTE**
> Evidence and dispatch are different questions. A method call is ordinary overload resolution, so it goes through implicit coercions and a concrete instance can capture an argument of a different but convertible type, even though the evidence query for that type is false. Where an instance must key on one type exactly, use a blanket head with `where T == s64` rather than a concrete head.

GLOSSO

```
#import "Io";

Label_Of :: typeclass (T: type) #minimal(label_of) {
    label_of :: (value: T) -> string;
}

s64 ::: instance Label_Of { label_of :: (value: s64) -> string { return "the s64 instance"; } }

LABEL_U32 :: Label_Of(u32);

main :: () {
    c: u32 = 3;
    print("call    = %\n", label_of(c));
    print("evidence = %\n", LABEL_U32);
}
```

CONSOLE / OUTPUT

```
call    = the s64 instance
evidence = 0
```

Associated-type definitions are checked eagerly

The lazy overlap rule applies to methods, not to two definitions of the same associated type. Those definitions are registered as a compile-time mapping, so a duplicate semantic instance head is rejected immediately with `associated type 'Class.Name' for <head> is already defined`. The checker normalises generic names and the order of `where` terms while it compares definitions. Two instances with genuinely different predicates, such as `where Is_Integer(T)` and `where Is_Float(T)`, remain separate.

> **NOTE**
> Concrete native and fixed-width heads use their target representation for this duplicate check. On a 64-bit target, associated-type definitions for `ssize` and `s64` are the same head when their `where` clauses are the same; on a 32-bit target the corresponding pair is `ssize` and `s32`. This is protocol-head compatibility, not ordinary type equality.

8.6 How an Instance Is Chosen

Glosso has no separate constraint-solving pass for typeclasses. A call to a class method is resolved by the ordinary overload machinery, over the set of every instance method and every class default that carries that name. Instance selection is overload resolution, and every rule that governs overloads governs it.

That single design decision produces most of the behaviour worth memorising. A more specific instance outranks a more general one. An implicit conversion applied to an argument can change which instance is selected. Two equally good instances are a diagnostic at the call site, not at the declaration.

The resolution steps

The checker performs these steps in order for every call whose name is owned by a typeclass. Steps 5 and 6 exist only to break ties, and a program that never writes overlapping instances never reaches them.

Step	What the checker does
1	Collect every declaration with that name. A call written `Class(T).m(...)` drops candidates that do not belong to `Class`.
2	Unify and coerce every argument against each candidate's parameter annotations, accumulating a specificity score.
3	Run the candidate's `#modify` block, then its `where` clauses, then check that the instance head itself has evidence.
4	If any surviving match is a real instance method, discard every class-default match.
5	Sort by implicit `#empty` witness first, then by score, highest first.
6	If the top two still tie, narrow by the expected return type, then by compile-time-number defaulting.
7	A remaining tie is `error: ambiguous typeclass operation '<name>' for these argument types`.

The specificity score

The score is a small integer accumulated while the arguments are matched. It measures how much of the candidate's signature was pinned down by the call rather than left generic, so a signature that names a concrete type outscores one that names a type variable.

Contribution	When it applies
`+2`	A parameter with a concrete annotation that the argument coerced to.
`+1`	A generic parameter, when the candidate is an instance method or a class default.
`+2`	An argument that is a typeclass method reference resolved against a concrete function parameter.
`+10_000`	Every argument's own type equalled its parameter type, with no coercion anywhere.

The exactness bonus dominates everything else, which is why an instance that matches the argument type on the nose beats one that would need a widening conversion, however concrete the second one is. The primary sort key is not the score at all: a candidate that consumed an implicit `#empty:` type witness sorts behind one that did not, and only then does the score decide.

Specific instances beat general ones

A blanket instance guarded by a `where` predicate and a concrete instance for one type may both apply. When both match exactly, the concrete one scores `+2` against the blanket's `+1` and wins.

GLOSSO

```
#import "Format";
#import "Io";
#import "Meta";
#import "String_Builder";

Label_Of :: typeclass (T: type) #minimal(label_of) {
    label_of :: (value: T) -> string;
    describe :: (value: T) -> string { return tprint("a %", label_of(value)); }
}

$T  ::: instance Label_Of where Is_Integer(T) {
    label_of :: (value: T) -> string { return "integer"; }
}
s64 ::: instance Label_Of {
    label_of :: (value: s64) -> string { return "s64"; }
    describe :: (value: s64) -> string { return "exactly s64"; }
}

main :: () {
    a: s64 = 1;
    b: u16 = 2;
    print("%\n", label_of(a));
    print("%\n", label_of(b));
    print("%\n", describe(a));
    print("%\n", describe(b));
}
```

CONSOLE / OUTPUT

```
s64
integer
exactly s64
exactly s64
```

`label_of(a)` scores `10002` for the `s64` instance against `10001` for the blanket, so the concrete instance is chosen. `label_of(b)` reverses it: `u16` matches the blanket's `T` exactly and earns the exactness bonus, while the `s64` instance needs a widening conversion and scores only `2`.

The last line is the one that surprises. `describe(b)` has two candidates: the class default, which matches `u16` exactly for `10001`, and the `s64` instance method, which scores `2`. Step 4 runs before the sort, so the default is discarded the moment any instance matched, and the low-scoring instance method is what remains. An instance method always beats a class default, regardless of score, because the default exists to be overridden.

> **NOTE**
> Because selection runs through the ordinary implicit conversions, an instance written for one concrete type can capture arguments of every type that converts to it. With no blanket instance in the program, there is nothing to score higher and the conversion goes unremarked.

GLOSSO

```
#import "Io";

Tag_Of :: typeclass (T: type) #minimal(tag_of) {
    tag_of :: (value: T) -> string;
}

s64 ::: instance Tag_Of { tag_of :: (value: s64) -> string { return "s64 instance"; } }

main :: () {
    u: u16 = 1;
    print("u16 -> %\n", tag_of(u));
}
```

CONSOLE / OUTPUT

```
u16 -> s64 instance
```

An instance keyed on exactly one type is written as a blanket instance whose predicate is a type equality, because a `where` clause is checked after the argument has been bound and admits no conversion. `$T ::: instance Tag_Of where T == s64` matches `s64` and nothing else, and `where Is_Integer(T) && T != s64` is the complementary instance.

Dispatch by the expected return type

A method whose class parameter appears only in return position gives the arguments nothing to decide. Every instance then matches equally well, the scores tie, and step 6 consults the type the call site expects.

GLOSSO

```
#import "Io";

Unit_Of :: typeclass (T: type) #minimal(unit_of) {
    unit_of :: () -> T;
}

s64 ::: instance Unit_Of { unit_of :: () -> s64 { return 7; } }
f64 ::: instance Unit_Of { unit_of :: () -> f64 { return 1.5; } }

pick :: () -> f64 { return unit_of(); }

main :: () {
    a: s64 = unit_of();
    print("annotated = %\n", a);
    print("returned  = %\n", pick());
    b: f64 = 0.0;
    b = unit_of();
    print("assigned  = %\n", b);
}
```

CONSOLE / OUTPUT

```
annotated = 7
returned  = 1.5
assigned  = 1.5
```

An annotated binding, a `return` in a function with a declared return type, and an assignment to an already-typed variable all supply an expected type. Nothing else does. `x := unit_of();` and `cast(f64)unit_of()` both leave the call undirected and are `error: ambiguous typeclass operation 'unit_of' for these argument types`, with a note pointing at each of the two instances.

The narrowing prefers a candidate whose return type equals the expected type exactly; if exactly one candidate's return type merely converts to it, that one is taken instead, and two exact matches leave the tie standing. An undecided numeric literal can match several numeric instances and leave a call ambiguous. Give the argument an explicit type when the overload set accepts both integers and floats; this fixes the instance before the call is checked.

GLOSSO

```
#import "Io";

Twice :: typeclass (T: type) #minimal(twice) { twice :: (v: T) -> T; }

s64 ::: instance Twice { twice :: (v: s64) -> s64 { return v * 2; } }
f64 ::: instance Twice { twice :: (v: f64) -> f64 { return v * 2.0; } }

main :: () {
    print("twice(3)   = %\n", twice(cast(s64)3));
    print("twice(2.5) = %\n", twice(2.5));
}
```

CONSOLE / OUTPUT

```
twice(3)   = 6
twice(2.5) = 5
```

Coherence, orphans, and evidence

Two identical instances are legal to declare. Coherence is checked lazily, at the call that cannot choose between them, and the diagnostic names both declarations: `error: ambiguous typeclass operation 'tag_of' for these argument types`, then `note: first matching instance is declared here` and `note: another matching instance is declared here`. The instance declared last is reported first, because candidates are matched in reverse declaration order.

There is no orphan rule. A program may declare an instance of a standard class for a standard type, and it wins wherever it is more specific: `u8 ::: instance Show` outranks the Prelude's `$T ::: instance Show where Is_Integer(T)` for `u8` values and leaves every other integer alone. Specificity, not locality, decides.

Step 3 is where a class-level `where` takes effect. The clause is a superclass constraint evaluated when evidence is asked for, not when the instance is declared, so an instance of `Refined :: typeclass (T: type) where Base(T)` for a type with no `Base` instance compiles and is then dropped from every candidate list. The call reports `error: no overload 'sub_of' accepts these arguments` and lists the candidates it rejected.

- Selection never consults a dictionary, so an instance method that is selected is called directly and instance dispatch costs nothing at run time. A class default and an instance method are distinguished only by which body was written where; both are ordinary declarations in the same candidate set.

> **NOTE**
> A class default whose signature omits one of the class parameters can never be dispatched. For `Convert :: typeclass (From: type, To: type)`, a default `describe :: (value: From) -> string` leaves `To` undetermined and calling it is `error: no overload 'describe' accepts these arguments`. Give every class parameter a position in the signature; an anonymous `#empty:` witness parameter is the usual way to place one that carries no value.

Native and fixed-width protocol heads

Protocol dispatch has one narrow target-dependent compatibility rule. A concrete fixed-width instance can service the native integer type with the same signedness and target representation: `s64` covers `ssize` on a 64-bit target, and `s32` covers it on a 32-bit target. The unsigned pairs are `u64`/`usize` and `u32`/`usize`. Matching recurses through pointers, arrays, tuples, and function-shaped protocol heads.

GLOSSO
```
#import "Io";

Native_Unary_Fallback :: typeclass (T: type) {
    '%%' :: (value: T) -> s64 #operator(prefix) { return 0; }
}
s64 ::: instance Native_Unary_Fallback {
    '%%' :: (value: s64) -> s64 { return 42; }
}

main :: () {
    value: ssize = 1;
    print("%\n", %% value);
}
```

CONSOLE / OUTPUT
```
42
```

The selected declaration still has its written formal type: in the example, the operator body receives `s64`. The same fallback works for multi-parameter protocols such as `IndexSet`, including assignment lowering. It does not merge the language types: `ssize == s64` remains false, ordinary generic unification keeps them separate, and a matrix or other value whose element type is `s64` is not a matrix of `ssize`.

> **NOTE**
> This relation never crosses width or signedness, never removes a nominal `distinct` boundary, and never treats `rune` as its `u32` storage type. It exists only for protocol instance heads.

Receiver calls and explicit class selection

A named typeclass operation may be called through its first non-erased parameter: `value.show()` uses the same evidence as `Show.show(value)`, and `reader.read(buffer)` dispatches through `Reader`. A storage field or applicable inherent method takes precedence over this lookup. When the intended class is ambiguous, write `Reader.read(reader, buffer)` or `Reader(T).read(reader, buffer)` explicitly.

Receiver syntax does not change defaults, associated types, or instance selection. In instance signatures, associated type definitions, and bodies, `#Self_Type` denotes the complete matched type. For `?$T ::: instance Try` it denotes `?T`, including the option wrapper. Typeclass requirements and instance slots follow the class contract's visibility.

8.7 Higher-Kinded Typeclasses

A typeclass parameter carries a kind, written as an ordinary type annotation. `T: type` declares a parameter that stands for a type; `F: (type) -> type` declares one that stands for a type constructor, a generic type that has not yet been applied to its argument. A class over such a parameter is higher-kinded.

Inside the class body the parameter is applied like a generic type: `F($A)` is the constructor at some inferred element type, and `F(B)` is the same constructor at another. This is what lets one method signature describe a transformation that changes the element type but keeps the container.

GLOSSO
```
#import "Io";

Container :: typeclass (F: (type) -> type) #minimal(mapc, unwrap) {
    mapc   :: (fa: F($A), f: (A) -> $B) -> F(B);
```

```
    unwrap :: (fa: F($A)) -> A;
}

Box  :: struct { value: $A; }
Pair :: struct { first: $A; second: A; }

Box ::: instance Container {
    mapc   :: (fa: Box($A), f: (A) -> $B) -> Box(B) { return .{ .value = f(fa.value) }; }
    unwrap :: (fa: Box($A)) -> A { return fa.value; }
}

Pair ::: instance Container {
    mapc   :: (fa: Pair($A), f: (A) -> $B) -> Pair(B) {
        return .{ .first = f(fa.first), .second = f(fa.second) };
    }
    unwrap :: (fa: Pair($A)) -> A { return fa.first; }
}

double :: (x: s64) -> s64 { return x * 2; }
tag    :: (x: s64) -> string { return "n"; }

main :: () {
    b: Box(s64)  = .{ .value = 21 };
    p: Pair(s64) = .{ .first = 3, .second = 4 };
    print("box  = %\n", unwrap(mapc(b, double)));
    print("pair = %\n", unwrap(mapc(p, double)));
    print("retype = %\n", unwrap(mapc(b, tag)));
}
```

CONSOLE / OUTPUT

```
box  = 42
pair = 6
retype = n
```

Bare-constructor instance heads

The instance head for a higher-kinded class is the bare constructor name. `Box ::: instance Container` supplies `Box` itself, not `Box` at some element type, so a single instance covers every application of the constructor at once. Writing `Box($A) ::: instance Container` would supply a type where a constructor is wanted.

Class parameter	Instance head	Method parameter
`T: type`	`Box($A) ::: instance C`	`(value: Box(A))`
`F: (type) -> type`	`Box ::: instance C`	`(fa: Box($A))`

In both forms the method's own generics are introduced with `$` at their first occurrence and referred to without it afterwards. In the higher-kinded form the `$A` belongs to the method, not to the head, which is exactly why the head has no hole to fill.

Evidence is asked at the applied type

Although the instance is registered against the bare constructor, an evidence query is answered for the applied type. `Box(s64)` and `Box(string)` both implement `Functor` on the strength of one `Box ::: instance Functor`, and a type with no such instance is rejected with the element type in the message.

GLOSSO

```
#import "Io";
#import "Typeclass";

Box   :: struct #derive(Functor) { value: $A; }
Plain :: struct { n: s64; }

count_functors :: (values: ...Functor) -> ssize { return values.count; }

main :: () {
    b: Box(s64)    = .{ .value = 1 };
    c: Box(string) = .{ .value = "x" };
    print("functors = %\n", count_functors(b, c));
}
```

CONSOLE / OUTPUT

```
functors = 2
```

Passing a `Plain` to the same procedure is `error: variadic arg type `Plain` does not implement `Functor` in call `count_functors`` . Typeclass-constrained variadics are covered in Typeclass-Constrained Variadics.

> **NOTE**
> There is no source-level spelling for a higher-kinded evidence query. The constructor form `Container(Box)` is `error: generic struct Box needs type arguments`, and the applied form `Container(Box(s64))` is `error: non-constant function call`. Ask for the evidence through a constrained variadic or a `where` clause on a method that takes the applied type as an argument.

Functor, Applicative, and Monad

The standard library declares three higher-kinded classes in `std/Typeclass`. They form a chain: each names the previous one as a superclass in its class-level `where`, so evidence for `Monad` requires evidence for `Applicative`, which requires `Functor`.

GLOSSO

```
Functor :: typeclass (F: (type) -> type) #derive(derive_functor) {
    fmap :: (fa: F($A), f: (A) -> $B) -> F(B);
}

Applicative :: typeclass (F: (type) -> type) where Functor(F) #derive(derive_applicative) {
    pure :: (x: $A) -> F(A);
    ap   :: (ff: F(($A) -> $B), fa: F(A)) -> F(B);
}

Monad :: typeclass (M: (type) -> type) where Applicative(M) #derive(derive_monad) {
    bind :: (ma: M($A), f: (A) -> M($B)) -> M(B);
}
```

All three carry a derive hook, so a one-field struct or a tagged union can request the instances instead of writing them. The derived `Applicative` and `Monad` for a struct handle one-field structs only; for a union they pick the single variant whose payload is the focused type parameter.

GLOSSO

```
#import "Io";
#import "Typeclass";

Ident :: struct #derive(Functor, Applicative, Monad) { value: $A; }

inc  :: (x: s64) -> s64 { return x + 1; }
step :: (x: s64) -> Ident(s64) { return .{ .value = x * 10 }; }

main :: () {
    i: Ident(s64) = .{ .value = 4 };
    m := fmap(i, inc);
    f: Ident((s64) -> s64) = .{ .value = inc };
    a := ap(f, i);
    b := bind(m, step);
    print("fmap = %\n", m.value);
    print("ap   = %\n", a.value);
    print("bind = %\n", b.value);
}
```

CONSOLE / OUTPUT

```
fmap = 5
ap   = 5
bind = 50
```

`fmap`, `ap` and `bind` all dispatch on an argument whose type mentions the class parameter, so ordinary selection resolves them. Deriving Instances with `#derive` covers what the three macros generate, and Writing a Derive Macro covers how they generate it.

`pure` cannot be called

`pure :: (x: $A) -> F(A)` mentions the class parameter only in return position. The expected-type narrowing described in How an Instance Is Chosen does not rescue it, because the candidate's return type is itself generic and never equals the expected type.

GLOSSO

```
#import "Io";
#import "Typeclass";

Ident :: struct #derive(Functor, Applicative) { value: $A; }

main :: () {
    p: Ident(s64) = pure(3);
    print("pure = %\n", p.value);
}
```

CONSOLE / OUTPUT

```
error: no overload 'pure' accepts these arguments
note: call pure cannot produce expected type Ident_s64
```

```
note: possible candidates for 'pure':
candidate: pure :: (x: A) -> Ident(A)
candidate: pure :: (x: A) -> F(A)
```

A hand-written instance does not help; the limitation is in the dispatcher, not in the derive macro. Construct the carrier directly instead: `p: Ident(s64) = .{ .value = 3 };` says the same thing and resolves without a class method.

A worked example

`examples/typeclasses.glo` in the compiler repository derives `Functor` for a generic tagged union and writes `Applicative` and `Monad` by hand on top of it. Both hand-written instance heads are the bare `Foo`.

GLOSSO

```
#import "Io";
#import "Typeclass";

Foo :: union #derive(Functor) {
    Some: $T;
    None: void;
}

Foo ::: instance Applicative {
    pure :: (x: $A) -> Foo(A) {
        return Foo.Some(x);
    }

    ap :: (ff: Foo(($A) -> $B), fa: Foo(A)) -> Foo(B) {
        if ff == {
            case .Some(function);
                return fmap(fa, function);
            case .None;
                return .None;
        }
        return .None;
    }
}

Foo ::: instance Monad {
    bind :: (ma: Foo($A), f: (A) -> Foo($B)) -> Foo(B) {
        if ma == {
            case .Some(input);
                return f(input);
            case .None;
                return .None;
        }
        return .None;
    }
}

inc :: (x: s64) -> s64 { return x + 1; }
to_option :: (x: s64) -> Foo(s64) { return Foo.Some(x + 10); }

main :: () {
    o: Foo(s64) = .Some(42);
    mapped := fmap(o, inc);
    chained := bind(mapped, to_option);
    if #pattern Foo.Some(total) = mapped && #pattern Foo.Some(chained_total) = chained
        print("typeclass totals % %\n", total, chained_total);
}
```

CONSOLE / OUTPUT

```
typeclass totals 43 53
```

- A union payload is read through a pattern capture, never through `ma.Some`. Every derived instance emits `case Variant(payload);` arms for the same reason.
- A class-level `where` on a higher-kinded class behaves exactly as it does on a first-order one: it is evaluated when evidence is asked for, and an instance that fails it is silently unusable rather than rejected at its declaration.
- A parameter of kind `(type) -> type` accepts a generic struct or a generic union. The built-in `?T` and `E!T` have no bare constructor spelling: the head `?$T ::: instance C` supplies a type, and a lone `?` is `error: invalid instance head '?'`.

8.8 Deriving Instances with `#derive`

> **NOTE**
> The standard `Show` derive macro generates formatting code for structs and tagged unions. Its compile-time formatting path supports managed builders in the current compiler; the examples below run as written.

A `struct` or `union` declaration may carry `#derive(Class, ...)` between the keyword and the body. Each named class must itself declare a derive macro; the compiler runs that macro at declaration time and splices the instance it produces into the program.

GLOSSO

```
#import "Io";

Point :: struct #derive(Show, PartialEq) { x: s64; y: string; }

Shape :: union #derive(Show, PartialEq) {
    Dot:   void;
    Line:  s64;
    Label: string;
}

main :: () {
    a: Point = .{ .x = 1, .y = "left" };
    b: Point = .{ .x = 1, .y = "left" };
    c: Point = .{ .x = 2, .y = "left" };
    print("show   = %\n", show(a));
    print("a == b = %\n", a == b);
    print("a == c = %\n", a == c);

    s: Shape = Shape.Line(7);
    t: Shape = Shape.Line(7);
    u: Shape = Shape.Dot;
    print("union  = %\n", show(s));
    print("dot    = %\n", show(u));
    print("s == t = %\n", s == t);
    print("s == u = %\n", s == u);
}
```

CONSOLE / OUTPUT

```
show   = {1, left}
a == b = 1
a == c = 0
union  = Line(7)
dot    = Dot
s == t = 1
s == u = 0
```

The generated instance is an ordinary instance in every respect. It takes part in selection on the same terms as a hand-written one, under the rules in How an Instance Is Chosen, and a second instance declared for the same type is the ordinary ambiguity rather than an override.

A class with no derive macro cannot be derived. Naming one is `error: typeclass 'Plain' does not declare a derive macro`, reported at the type declaration, and every later call to the class method fails as well because no instance was ever created.

What the standard classes generate

Class	Macro	What it writes
`Show`	`derive_show`	`{f1, f2}` for a struct, `Variant` or `Variant(payload)` for a union.
`PartialEq`	`derive_partial_eq`	Field-by-field `&&` for a struct; same-variant payload comparison for a union.
`Functor`	`derive_functor`	`fmap` over the final type parameter, mapping every field or variant that uses it.
`Applicative`	`derive_applicative`	`pure` and `ap`; one-field structs only, or the single focused union variant.
`Monad`	`derive_monad`	`bind`, under the same shape restrictions as `Applicative`.
`Error`	`derive_error`	`report_error` for a non-generic tagged union whose every variant payload is itself `Error`.
`Serialize`, `Deserialize`	`derive_serialize`, `derive_deserialize`	Field-wise and variant-wise encoders and decoders, in `Serialization`.

`FromString` deliberately ships no derive macro. A parser for a user type has to decide what its accepted spellings are, and no field-wise rule produces a useful answer. The `Error` derive and the `#from` union variants that pair with it are covered in the chapter on `Error`, `Termination`, and `main`.

- The `#derive(...)` list on a struct or union is a comma-separated list of plain identifiers. There is no syntax for passing options to a macro, so the `Derive_Options` a macro receives is always empty and every macro uses its default focus.

- `#derive` is accepted on `struct`, `union`, and `distinct` declarations. On an `enum` it does not parse: `Color :: enum #derive(Show) { ... }` is `error: unexpected token 'derive'`.
- A derive macro that rejects the shape it was handed reports its own `#compile_error` text, such as `Show derive is not defined for #raw union` or `Functor derive needs exactly one focused type parameter`.

> **NOTE**
> The default `Formatter` instance calls the selected `Show` method directly. `print("%", value)` therefore instantiates a generic derived `Show` on demand, just as `show(value)` does. No preliminary direct `show` call is needed for a generic struct, union, or option.

GLOSSO
```
#import "Io";

Tagged :: union #derive(Show) { Some: $A; None: void; }

main :: () {
    t: Tagged(s64) = Tagged.Some(9);
    print("through print = %\n", t);
}
```

CONSOLE / OUTPUT
```
through print = Some(9)
```

Deriving on a `distinct` type

`#derive` on a `distinct` declaration is a different feature that shares the spelling. It generates no code at all. A `distinct` type inherits none of its representation's operators by default, and the derive list is the list of operator classes it is permitted to borrow.

GLOSSO
```
#import "Io";

Meters  :: distinct f64 #derive(Add, Sub, Neg, PartialEq, PartialOrd, Mul(Meters, f64), Show);
Seconds :: distinct f64 #derive,all;

main :: () {
    a: Meters = 3.0;
    b: Meters = 4.0;
    print("a+b   = %\n", show(a + b));
    print("a-b   = %\n", show(a - b));
    print("-a    = %\n", show(-a));
    print("a*2   = %\n", show(a * 2.0));
    print("a<b   = %\n", a < b);
    print("a==b  = %\n", a == b);

    s: Seconds = 3.0;
    t: Seconds = 4.0;
    print("s+t   = %\n", show(s + t));
    print("s/t   = %\n", show(s / t));
}
```

CONSOLE / OUTPUT
```
a+b   = 7
a-b   = -1
-a    = -3
a*2   = 6
a<b   = 1
a==b  = 0
s+t   = 7
s/t   = 0.75
```

The operator is resolved on the representation and the nominal tag is put back only when the representation's result is the representation itself. `Meters + Meters` is `Meters`, and so is `Meters / Meters`, but `Meters == Meters` stays `bool`.

Naming operand types

A bare class name in the list derives the same-type instance: every operand and the result are the nominal type. An entry with type arguments names the operand types where they differ, which is what makes scaling a dimensioned quantity expressible.

Derive entry	Instance it permits
`Add`	`Meters + Meters`, yielding `Meters`.
`Mul(Meters, f64)`	`Meters * f64`, yielding `Meters`. The mirrored form needs its own `Mul(f64, Meters)`.
`#derive,all`	Every operator the representation supports, written instead of a list.

GLOSSO

```
#import "Io";
#import "Meta";

Meters :: distinct f64
    #derive(Add, Show, PartialEq, Mul(Meters, f64), Mul(f64, Meters), Div(Meters, Meters));

main :: () {
    a: Meters = 3.0;
    b: Meters = 4.0;
    scale: f64 = 2.0;
    print("a * scale = %\n", show(a * scale));
    print("scale * a = %\n", show(scale * a));
    ratio := a / b;
    print("a / b     = %\n", show(ratio));
    print("ratio type= %\n", type_name(type_of(ratio)));
    print("cmp  type = %\n", type_name(type_of(a == b)));
}
```

CONSOLE / OUTPUT

```
a * scale = 6
scale * a = 6
a / b     = 0.75
ratio type= Meters
cmp  type = bool
```

An operator that was not derived names the fix in its diagnostic: `no operator '*' for Meters and Meters: a distinct type gets only the classes it derives; add '#derive(...)' or '#derive,all' on the declaration of 'Meters'`. `#derive` on a `distinct` with neither a list nor `,all` is `error: '#derive' on a distinct type needs a class list or ',all'`.

> **NOTE**
> The derive list on a `distinct` type gates operators only. A named typeclass method still resolves through the ordinary generic instances, because the predicates those instances test are true of the representation. `Meters :: distinct f64;` with no derive at all still answers `show(a)` with `3.5`.

8.9 Writing a Derive Macro

A derive macro is an ordinary `#expand` macro, named by the `#derive(...)` directive on a typeclass declaration. The compiler calls it once per type that requests the class, and the macro's job is to `#insert` an `instance` declaration.

The call is always made with the same three parameters: the type itself, a reflection summary of it, and the options the request carried. The macro returns nothing; its effect is whatever it inserts.

GLOSSO

```
#import "Io";
#import "Meta";
#import "Typeclass/Derive";

Arity :: typeclass (T: type) #derive(derive_arity) #minimal(arity_of) {
    arity_of :: (value: T) -> ssize;
}

derive_arity :: (T: type, info: Derive_Type_Info, opts: Derive_Options) #expand {
    if !Is_Struct(T) #compile_error "Arity can only be derived for structs";
    empty_focuses: []mut ssize;
    empty_replacements: []mut Code;
    section    := info.type_section(empty_focuses);
    input_type := info.type_apply(empty_focuses, empty_replacements);
    count      := info.fields.count;
    #insert #code {
        ``section ::: instance Arity {
            arity_of :: (value: ``input_type) -> ssize { return ``count; }
        }
    };
}

Point  :: struct #derive(Arity, Show) { x: s64; y: s64; }
Triple :: struct #derive(Arity) { a: s64; b: s64; c: s64; }
Pair   :: struct #derive(Arity) { k: $K; v: $V; }

main :: () {
    p: Point  = .{ .x = 1, .y = 2 };
    t: Triple;
    q: Pair(s64, string) = .{ .k = 1, .v = "x" };
    print("Point  % arity=%\n", show(p), arity_of(p));
    print("Triple arity=%\n", arity_of(t));
    print("Pair   arity=%\n", arity_of(q));
}
```

CONSOLE / OUTPUT

```
Point  {1, 2} arity=2
Triple arity=3
Pair   arity=2
```

Everything the macro computes is spliced into the `#code` block with the double-backtick escape. `section` and `input_type` are `Code` values built by the reflection intrinsics, and `count` is an ordinary compile-time integer; splicing all three is what makes one macro serve `Point`, `Triple`, and the generic `Pair`. A body whose shape depends on a loop over the members is usually built as a source string and `#insert`ed instead, which is what the standard derives do.

The reflection types

`std/Typeclass/Derive.glo` declares the structures the compiler fills in. The Prelude imports `Typeclass`, so `Derive_Type_Info`, `Derive_Options`, `Code`, and the intrinsics below resolve in a program with no import line at all; the `#import "Typeclass";` commonly written beside a derive macro is not what makes them visible.

Type	Member	Meaning
`Derive_Type_Info`	`base`	The ordinary `Type_Info`: tag, canonical name, size, alignment.
`Derive_Type_Info`	`params`	Generic parameters in declaration order, as `Type_Param_Info`.
`Derive_Type_Info`	`fields`	Struct fields, as `Derive_Field_Info`. Empty for a union.
`Derive_Type_Info`	`variants`	Tagged-union variants, as `Derive_Variant_Info`. Empty for a struct.
`Derive_Type_Info`	`raw_union`	True for a `#raw union`, where tagged-union logic does not apply.
`Type_Param_Info`	`name`, `index`	Source name and zero-based position; `index` is what the query helpers take.
`Derive_Field_Info`	`name`, `ty`	Field name and its type resolved in the type's generic scope.
`Derive_Variant_Info`	`name`, `payload`	Variant name and payload type; a payload-less variant reports `void`.
`Derive_Options`	`focus_params`	Requested focus parameter names. Always empty in current source.

Because there is no syntax for passing options in `#derive(...)`, a macro that calls `info.focus_params(opts, n)` always gets the default: the final `n` type parameters. A macro with no use for `opts` may leave the parameter untouched; `derive_error` writes `_ := opts;` to record the intent, and the other standard derives that ignore it write nothing.

The code-building intrinsics

Call	Result
`derive_type_info(T)`	The `Derive_Type_Info` for `T`. The compiler supplies it to a derive macro already.
`info.type_section(focuses)`	The instance head, with focused parameters left as holes: `Pair($K, _)`.
`info.type_apply(focuses, reps)`	A concrete type expression, substituting `reps` for the focused parameters.
`derive_type_fn(params, ret)`	A function-type expression, for building `F(($A) -> $B)`.
`type_uses_param(T, i)`	True when `T` mentions parameter `i` anywhere.
`type_is_param(T, i)`	True when `T` is exactly parameter `i`: true for `A`, false for `[]A`.
`type_occurs_covariantly(T, i)`	True when every occurrence of parameter `i` in `T` is covariant.
`info.focus_params(opts, n)`	The `n` focused parameter indexes, defaulting to the final `n`.

The head shape a macro must emit depends on the arity of the class. A one-parameter class takes the section alone; a two-parameter class such as `PartialEq(L, R)` takes a tuple head, which for a same-type instance pairs the section with the applied type.

GLOSSO

```
#import "Io";
#import "Meta";
#import "Typeclass";
#import "Typeclass/Derive";

Same_Shape :: typeclass (L: type, R: type) #derive(derive_same_shape) #minimal(same_shape) {
    same_shape :: (a: L, b: R) -> bool;
}

derive_same_shape :: (T: type, info: Derive_Type_Info, opts: Derive_Options) #expand {
    _ := opts;
    empty_focuses: []mut ssize;
    empty_replacements: []mut Code;
    section := info.type_section(empty_focuses);
    input_type := info.type_apply(empty_focuses, empty_replacements);
    #insert #code {
        (``section, ``input_type) ::: instance Same_Shape {
            same_shape :: (a: ``input_type, b: ``input_type) -> bool {
                return true;
            }
        }
    };
}

Point :: struct #derive(Same_Shape) { x: s64; y: s64; }

main :: () {
    p: Point;
    q: Point;
    print("same = %\n", same_shape(p, q));
}
```

CONSOLE / OUTPUT

```
same = 1
```

Reading fields and variants

A macro that handles both struct and union shapes tests `Is_Struct` and `Is_Union`, refuses a `#raw union` explicitly, and walks `info.fields` or `info.variants`. Both loops run at compile time, so the string they build is a constant by the time the instance is inserted.

GLOSSO

```
#import "Io";
#import "Meta";
#import "Typeclass";
#import "String_Builder";
#import "Format";
#import "Typeclass/Derive";

Members :: typeclass (T: type) #derive(derive_members) #minimal(member_names) {
    member_names :: (value: T) -> string;
}

members_body :: (info: Derive_Type_Info) -> string
{
    source := String_Builder.new();
    source.append("return \"");
    first := true;
    for field: info.fields {
        if !first source.append(", ");
        source.append(field.name);
        first = false;
    }
    for variant: info.variants {
        if !first source.append(", ");
        source.append(variant.name);
        if variant.payload != void source.append("(_)");
        first = false;
    }
    source.append("\";");
    return source.finish_source();
}

derive_members :: (T: type, info: Derive_Type_Info, opts: Derive_Options) #expand {
    if !Is_Struct(T) && !Is_Union(T) {
        #compile_error "Members can only be derived for struct or union types";
    }
    if Is_Union(T) && info.raw_union {
        #compile_error "Members derive is not defined for #raw union";
    }
    _ := opts;
    empty_focuses: []mut ssize;
    empty_replacements: []mut Code;
```

```
    section := info.type_section(empty_focuses);
    input_type := info.type_apply(empty_focuses, empty_replacements);
    #insert #code {
        ``section ::: instance Members {
            member_names :: (value: ``input_type) -> string {
                #insert members_body(info);
            }
        }
    };
}

Point :: struct #derive(Members) { x: s64; y: s64; }
Shape :: union  #derive(Members) { Dot: void; Line: s64; }
Cell  :: struct #derive(Members) { key: $K; value: $V; }

main :: () {
    p: Point;
    s: Shape = Shape.Dot;
    c: Cell(s64, string);
    print("Point = %\n", member_names(p));
    print("Shape = %\n", member_names(s));
    print("Cell  = %\n", member_names(c));
}
```

CONSOLE / OUTPUT

```
Point = x, y
Shape = Dot, Line(_)
Cell  = key, value
```

Deriving `Members` for `Cell` produces one instance covering every application, because `Derive_Type_Info.type_section` with an empty focus list is the type name alone and `Derive_Type_Info.type_apply` reproduces the declared parameters.

Focused type parameters

A class that transforms one type parameter needs the head to leave that parameter open. `Derive_Type_Info.focus_params` chooses which parameter that is, `Derive_Type_Info.type_section` turns it into a hole, and `Derive_Type_Info.type_apply` substitutes a replacement for it in each method signature. For a two-parameter `Cell(K, V)` focused on the last parameter, the head becomes `Cell($K, _)`, the input type `Cell(K, $A)`, and the output `Cell(K, B)`.

GLOSSO

```
#import "Io";
#import "Meta";
#import "Typeclass";
#import "Typeclass/Derive";

Map_Last :: typeclass (F: (type) -> type) #derive(derive_map_last) #minimal(map_last) {
    map_last :: (fa: F($A), f: (A) -> $B) -> F(B);
}

map_last_field :: (field: Derive_Field_Info, focus: ssize) -> Code {
    name := field.name;
    if !type_uses_param(field.ty, focus) return #code { .``name = fa.``name };
    if !type_is_param(field.ty, focus) {
        #compile_error "Map_Last handles a direct focused field only";
    }
    return #code { .``name = f(fa.``name) };
}

derive_map_last :: (F: type, info: Derive_Type_Info, opts: Derive_Options) #expand {
    if !Is_Struct(F) #compile_error "Map_Last can only be derived for structs";
    focuses := info.focus_params(opts, 1);
    if focuses.count != 1 #compile_error "Map_Last needs exactly one focused type parameter";
    focus := focuses[0];
    section     := info.type_section(focuses);
    input_type  := info.type_apply(focuses, .[#code { $A }]);
    output_type := info.type_apply(focuses, .[#code { B }]);
    #insert #code {
        ``section ::: instance Map_Last {
            map_last :: (fa: ``input_type, f: (A) -> $B) -> ``output_type {
                return .{
                    #inline for field: info.fields {
                        ``map_last_field(field, focus),
                    }
                };
            }
        }
    };
}

Cell :: struct #derive(Map_Last) { key: $K; value: $V; }

length :: (s: string) -> ssize { return s.count; }

main :: () {
    c: Cell(string, string) = .{ .key = "id", .value = "hello" };
    d := map_last(c, length);
    print("key = % value = %\n", d.key, d.value);
```

```
    print("type = %\n", type_name(type_of(d)));
}
```

CONSOLE / OUTPUT

```
key = id value = 5
type = Cell_string_ssize
```

`map_last_field` is a compile-time helper returning `Code`, and the `#inline for` inside the aggregate literal splices one element per field. `type_uses_param` distinguishes the field that must be transformed from the ones that are copied; `type_is_param` refuses a field such as `[]A` that mentions the parameter but is not it, which is the same restriction the standard `Functor` derive enforces.

> **NOTE**
> A generated union arm must bind its payload with a pattern capture. Every standard derive macro emits `case Variant(__glo_payload);` rather than reading `value.Variant`, because a direct payload read is rejected unless the tag has been proved. Writing `out.Variant = ...` on the way out is a variant selection and needs no capture.

- A macro reports a bad shape with `#compile_error`, and the text appears verbatim at the failing directive: deriving `Arity` for a union gives `error: Arity can only be derived for structs`. The method the macro writes still has to satisfy the class's `#minimal`, and because signature keys are compared after the head arguments are substituted, a wrong return type in generated code surfaces as a `#minimal` failure rather than as a signature mismatch.

8.10 Equality and Ordering

Equality is a two-parameter class over the operand types, so the left and right sides of `==` need not be the same type. Both operators have default bodies written in terms of each other, and `#minimal` records that implementing either one is enough.

GLOSSO

```
PartialEq :: typeclass (L: type, R: type) #derive(derive_partial_eq)
    #minimal('==' | '!=')
{
    '==' :: (left: L, right: R) -> bool #operator(left, 8) #inline {
        return !(left != right);
    }
    '!=' :: (left: L, right: R) -> bool #operator(left, 8) #inline {
        return !(left == right);
    }
}
```

Writing both is permitted and is what the Prelude does for strings, where each has its own intrinsic. Writing neither is `error: instance '(Handle, Handle) :: PartialEq' does not satisfy #minimal('==' | '!=')`, with a note pointing at the `#minimal` directive in the standard library.

GLOSSO

```
#import "Io";

Handle :: struct { id: s64; }
Name   :: struct { text: string; }

(Handle, Handle) ::: instance PartialEq {
    '!=' :: (a: Handle, b: Handle) -> bool { return a.id != b.id; }
}

(Handle, Name) ::: instance PartialEq {
    '==' :: (a: Handle, b: Name) -> bool { return b.text == "h"; }
}

main :: () {
    x: Handle = .{ .id = 1 };
    y: Handle = .{ .id = 1 };
    n: Name   = .{ .text = "h" };
    print("x == y = %\n", x == y);
    print("x != y = %\n", x != y);
    print("x == n = %\n", x == n);
    print("x != n = %\n", x != n);
    print("orderings = % % % %\n",
        cast(s64)Ordering.Less, cast(s64)Ordering.Equal,
        cast(s64)Ordering.Greater, cast(s64)Ordering.Unordered);
}
```

CONSOLE / OUTPUT

```
x == y = 1
x != y = 0
```

```
x == n = 1
x != n = 0
orderings = 0 1 2 3
```

The first instance supplies only `!=` and gets `==` from the default; the second supplies only `==` for a mixed operand pair and gets `!=`. Because the instance is keyed on the operand tuple, `(Handle, Name)` and `(Name, Handle)` are separate instances and only the one that was written exists.

`#derive(PartialEq)` writes a field-by-field conjunction for a struct and a same-variant payload comparison for a union, which is described in Deriving Instances with `#derive`. A hand-written instance for the same operand pair does not replace the derived one: both are declared, and the first call that has to choose between them is the ambiguity.

`Eq` is a marker with a superclass

`Eq` has no members at all. Its content is entirely in its class-level `where`, which demands same-type `PartialEq`, and in the meaning attached to it: an `Eq` type's equality is a real mathematical equality, reflexive for every value.

GLOSSO

```
Eq :: typeclass (T: type) where PartialEq(T, T) {}

$T ::: instance Eq where Is_Integer(T) {}
$T ::: instance !Eq where Is_Float(T) {}
bool ::: instance Eq {}
$T ::: instance Eq where Is_String(T) {}
type ::: instance Eq {}
label ::: instance Eq {}
void ::: instance Eq {}
```

`$T ::: instance !Eq where Is_Float(T)` is a negative instance. Floats are `PartialEq`, because `==` on them is well defined, but they are explicitly not `Eq`, because a NaN is not equal to itself. A negative instance wins over any positive one, so no later blanket instance can quietly restore `Eq` for a float.

Marking a user type `Eq` is an empty instance body, and the evidence query `Eq(T)` is a compile-time constant usable in a `where` clause, an `#if`, or a top-level constant.

GLOSSO

```
#import "Io";

Version :: struct #derive(Show) { major: s64; minor: s64; }

(Version, Version) ::: instance PartialEq {
    '==' :: (a: Version, b: Version) -> bool {
        return a.major == b.major && a.minor == b.minor;
    }
}

Version ::: instance Eq {}

(Version, Version) ::: instance PartialOrd {
    '<=>' :: (a: Version, b: Version) -> Ordering {
        if a.major < b.major return Ordering.Less;
        if a.major > b.major return Ordering.Greater;
        if a.minor < b.minor return Ordering.Less;
        if a.minor > b.minor return Ordering.Greater;
        return Ordering.Equal;
    }
}

EQ_VERSION :: Eq(Version);
EQ_F64     :: Eq(f64);
EQ_S64     :: Eq(s64);

main :: () {
    a: Version = .{ .major = 1, .minor = 2 };
    b: Version = .{ .major = 1, .minor = 7 };
    print("a == b  = %  a != b = %\n", a == b, a != b);
    print("a <  b  = %  a <= b = %\n", a < b, a <= b);
    print("a >  b  = %  a >= b = %\n", a > b, a >= b);
    print("a <=> b = %\n", a <=> b);
    print("Eq(Version)=% Eq(f64)=% Eq(s64)=%\n", EQ_VERSION, EQ_F64, EQ_S64);
}
```

CONSOLE / OUTPUT

```
a == b  = 0  a != b = 1
a <  b  = 1  a <= b = 1
a >  b  = 0  a >= b = 0
a <=> b = 0
Eq(Version)=1 Eq(f64)=0 Eq(s64)=1
```

A procedure that requires reflexive equality names it in its own `where`. `find_at :: (values: []$T, wanted: T) -> ssize where Eq(T)` accepts an `[]s64` or an `[]string` and rejects an `[]f64` with `error: constraint failed for find_at`. The constraint expresses an intent that `PartialEq` alone cannot: a lookup that must find back the value it was given.

> **NOTE**
> `where Eq(T)` alone does not license `==` inside the body. `Eq` has no members, and the body needs the class that owns the operator. The compiler warns rather than failing: `warning: no '==' for <T>: the where-clause only provides Eq`, with `note: reported at the definition; add the class to the 'where' clause`. Write `where PartialEq(T, T)`, adding `Eq(T)` beside it when reflexivity is the real requirement.

`PartialOrd` and `<=>`

Ordering is also a two-parameter class. One member has no default; the four comparison operators are all defined in terms of it, so an instance that supplies `<=>` acquires `<`, `<=`, `>`, and `>=` unchanged.

```
GLOSSO
PartialOrd :: typeclass (L: type, R: type) {
    '<=>' :: (a: L, b: R) -> Ordering #operator(left, 10);

    '<' :: (a: L, b: R) -> bool #operator(left, 10) #inline {
        c := a <=> b;
        return c == Ordering.Less;
    }
    '<=' :: (a: L, b: R) -> bool #operator(left, 10) #inline {
        c := a <=> b;
        return c == Ordering.Less || c == Ordering.Equal;
    }
    '>' :: (a: L, b: R) -> bool #operator(left, 10) #inline {
        c := a <=> b;
        return c == Ordering.Greater;
    }
    '>=' :: (a: L, b: R) -> bool #operator(left, 10) #inline {
        c := a <=> b;
        return c == Ordering.Greater || c == Ordering.Equal;
    }
}
```

`PartialOrd` has no `#minimal` directive, so the implicit minimal applies: every member without a default body must be implemented, which here is `<=>` alone. Overriding one of the four defaults is allowed and is occasionally worth doing when the type can answer `<` more cheaply than it can answer a full three-way comparison.

All five operators sit at precedence tier 10, above the equality operators at tier 8. The whole scale is set out in Operator Precedence and the 0-20 Scale.

`Ordering` and what `Unordered` means

`Ordering` is a plain enum with four variants, in this order: `Less`, `Equal`, `Greater`, `Unordered`. `show` on an enum yields the ordinal rather than the variant name, which is why `a <=> b` printed `0` above rather than `Less`.

Variant	Meaning
`Less`	The left operand precedes the right.
`Equal`	Neither precedes the other and they compare the same.
`Greater`	The left operand follows the right.
`Unordered`	The two operands are not comparable at all.

`Unordered` is the reason the class is `PartialOrd` rather than a total `Ord`. Floating point produces it for any comparison involving a NaN. The four defaulted operators all answer `false` for an `Unordered` result, so `a < b`, `a > b`, and `a == b` can all be false at once.

Every comparison-driven container and algorithm in the standard library treats `Unordered` as a failure rather than as an ordering. Sorting, `BinaryHeap`, `BTreeMap`, and `BTreeSet` all detect it and report an error instead of proceeding, because a sort or a search tree built on an inconsistent comparison has no correct answer to give.

```
GLOSSO
#import "Io";
#import "Sort";
#import "Math";

main :: () {
    good: [4]f64 = .[3.0, 1.0, 2.0, 0.5];
    r1 := (good[..]).sort();
    if r1 == {
        case .Ok;         print("sorted = % % % %\n", good[0], good[1], good[2], good[3]);
        case .Err(error); print("sort err = %\n", error.reason());
    }

    bad: [3]f64 = .[1.0, quiet_nan(), 2.0];
    r2 := (bad[..]).sort();
    if r2 == {
        case .Ok;         print("nan sort ok\n");
```

```
        case .Err(error); print("nan sort err = %\n", error.reason());
    }

    cmp := 1.0 <=> quiet_nan();
    print("1.0 <=> nan is Unordered = %\n", cmp == Ordering.Unordered);
}
```

CONSOLE / OUTPUT

```
sorted = 0.5 1 2 3
nan sort err = comparison produced an unordered value
1.0 <=> nan is Unordered = 1
```

The two error enums carry a variant for it: `Sort_Error.Unordered` and `Container_Error.Unordered`, both reporting `comparison produced an unordered value`. The container operations preflight the comparisons they will need and leave the structure logically unchanged; a sort may already have rearranged part of the slice by the time it detects the problem.

- `void` has hand-written `PartialEq` and `PartialOrd` instances that answer `true` and `Ordering.Equal` unconditionally. One inhabitant means one answer, so neither is a comparison of stored bits.
- `?T` carries a hand-written `PartialEq` guarded by `where PartialEq(T, T)` rather than a derived one, because the built-in option type cannot carry a `#derive` directive. Two absences are equal, an absence and a presence never are, and two presences defer to the payload.
- A `distinct` type inherits neither class automatically. `PartialEq` and `PartialOrd` have to appear in its `#derive` list, and the result of a comparison is `bool` rather than the nominal type.

8.11 `Show`, `Formatter`, and Text Conversion

`Formatter` selects a rendering by value type, compile-time selector, and writer type. The default selector in `%` and `%{}` delegates to `Show`; selectors such as `%x` and `%{token}` use their own instances. A type can therefore support a named rendering without implementing `Show`. `FromString` is the parsing counterpart to `Show`: `Show` ships a derive macro, while `FromString` does not.

GLOSSO

```
Show :: typeclass (T: type) #derive(derive_show) {
    show :: (value: T) -> string
}

FromString :: typeclass (T: type) {
    from_string :: (#empty: T, text: string) -> ?T
}

Formatter :: typeclass (T: type, #comptime spec: string, W: type)
    where Writer(W) {
    format_to :: (writer: *mut W, value: T) -> Io_Error!ssize
}
```

The Prelude instances

The Prelude keys most of its `Show` instances on a predicate rather than on a type, so a whole family is covered by one instance. A program may still add a concrete instance for any of these types; a concrete head outranks a blanket one, and there is no orphan rule.

Instance head	Renders as	Implementation
`string`, `string16`	the text itself	borrow; `string16` is decoded to UTF-8
`$T where Is_Integer(T)`	decimal digits	`#magic "show_int"`
`$T where Is_Float(T)`	decimal, `1.5`	`#magic "show_float"`
`$T where Is_Bool(T)`	`0` or `1`	`#magic "show_bool"`
`$T where Is_Enum(T)`	the ordinal, not the name	`#magic "show_enum"`
`$T where Is_Pointer(T) || Is_Procedure(T)`	the address	`#magic "show_pointer"`
`type`	the canonical type name	`#magic "type_name"`
`any`	the boxed value	`#magic "show_any"`
`label`	the label	`#magic "show_label"`
`void`	`.()`	hand written

Instance head	Renders as	Implementation
`?$T where Show(T)`	`Some(payload)` or `None`	hand written, `?T` cannot carry `#derive`

`FromString` has four Prelude instances: `string`, and blanket instances guarded by `Is_Bool(T)`, `Is_Integer(T)` and `Is_Float(T)`. The integer parser is overflow checked at every width including 128 bits, and the float parser accepts `inf`, `-inf`, `nan`, exponents and subnormals. All of them consume the whole string and answer `.None` on any leftover byte.

There is no `Show` instance for `E!T`, so its default `%` formatter is unavailable. Match on a result and show its payload, or define a `Formatter` instance for the result type and a chosen selector. `?T` does have `Show` when its payload does, so an option prints directly.

What `#derive(Show)` produces

`derive_show` accepts a struct or a tagged union. A struct renders as its fields in declaration order inside braces; a union renders as the variant name, with the payload in parentheses when the variant has one. The generated body builds the text with a `String_Builder` in the temporary allocator, and it reads a union payload through a pattern capture rather than a field access, because a payload may only be bound by a match. A `#raw` union is `error: Show derive is not defined for #raw union`, since a raw union has no tag to dispatch on.

GLOSSO

```
#import "Io";
#import "Format";

Money :: struct { cents: s64; }

Money ::: instance Show {
    show :: (value: Money) -> string {
        return tprint("$%.%", value.cents / 100, value.cents % 100);
    }
}

Point :: struct #derive(Show) { x: s64; name: string; }

Tag :: union #derive(Show) { Plain: void; Count: s64; }

main :: () {
    m: Money = .{ .cents = 1234 };
    p: Point = .{ .x = 1, .name = "origin" };
    t: Tag = .Count(9);
    u: Tag = .Plain;
    print("money = %\n", m);
    print("point = %\n", p);
    print("tag   = % and %\n", t, u);
}
```

CONSOLE / OUTPUT

```
money = $12.34
point = {1, origin}
tag   = Count(9) and Plain
```

Selectors and compile-time formatting

Formatting entry points take a `#comptime fmt: string` followed by `args: ...any`. The pack preserves each argument's concrete type. `value.format_to_writer` parses the format string at compile time and checks `Formatter(Value_Type, spec, Writer_Type)` for each substitution. Every argument is evaluated once; only the selected writes run at runtime.

A bare `%` or `%{}` selects the empty key. `%c` selects the one-letter key `"c"`, and `%{name}` selects `"name"`. A short selector consumes exactly one ASCII letter whether an instance exists or not, so `%xray` means hexadecimal followed by `ray`. Use `%{}x` when the letter `x` should follow a default rendering. `%%` means two substitutions, not a literal percent; no C-style width or precision grammar is built in.

Missing arguments produce `format has more '%' placeholders than arguments`; surplus arguments produce `format has fewer '%' placeholders than arguments`. Unknown keys or unsupported argument types report `no Formatter instance for selector '<key>' and value type <T>`. Unterminated or nested braces report `unterminated or nested formatter selector`. These are compile-time errors, including when the format string is computed by a compile-time expression.

Writing a literal percent

A percent sign that is not a placeholder is written `\%`. The formatter recognises the two-byte sequence and emits a single `%`, and the compile-time placeholder count skips it, so an escaped percent consumes no argument.

GLOSSO

```
#import "Io";

main :: () {
```

```
    print("% of % is %\n", 50, 200, 100);
    print("50\% of 200 is 100\n");
    print("progress: %\% done\n", 40);
}
```

CONSOLE / OUTPUT

```
50 of 200 is 100
50% of 200 is 100
progress: 40% done
```

The `Format` helpers

Call	Result	Allocator
`print(fmt, args)`	written to standard output	temporary; `print` formats through `tprint`
`format(fmt, args)`	an owned `string`; release with `string.free`	`context.allocator`
`tprint(fmt, args)`	a temporary `string`	`context.temp_allocator`
`format(fmt, args, .allocator = a)`	an owned `string`; free with the same allocator	the named allocator; replaces removed `format_args`
`value.format_int(base)`	digits in `base`; do not free individually	temporary arena; also usable at compile time
`value.format_int_with_allocator(base, a)`	digits in `base`	the explicit allocator

NOTE

`format_int` writes into the compiler-provided temporary arena directly; replacing `context.temp_allocator` or using `#push_allocator` does not redirect it. `format_int_with_allocator` copies those digits into the requested allocator so the result can outlive a temporary-arena reset. Release that allocated result with the same allocator.

GLOSSO

```
#import "Io";
#import "Format";
#import "Context";
#import "String_Builder";

main :: () {
    heap := format("% and %", 1, "two");
    defer heap.free();
    temp := tprint("% and %", 3, "four");
    explicit := format("%-%", 5, 6, .allocator = context.allocator);
    defer explicit.free(.allocator = context.allocator);
    print("format     = %\n", heap);
    print("tprint     = %\n", temp);
    print("explicit   = %\n", explicit);
    print("hex        = %\n", (255).format_int(16));
    print("binary     = %\n", (10).format_int(2));
}
```

CONSOLE / OUTPUT

```
format     = 1 and two
tprint     = 3 and four
explicit   = 5-6
hex        = ff
binary     = 1010
```

Writing a `FromString` instance

`FromString` carries no derive macro on purpose: a parser has to decide what it accepts, and no field-by-field expansion can guess that. Asking for one is `error: typeclass 'FromString' does not declare a derive macro`. An instance returns `?T`, answers `.None` for anything it does not fully consume, and is free to call `from_string` for its components.

GLOSSO

```
#import "Io";
#import "Format";

Celsius :: struct { degrees: s64; }

Celsius ::: instance Show {
    show :: (value: Celsius) -> string {
```

```
        return tprint("%{}C", value.degrees);
    }
}

Celsius ::: instance FromString {
    from_string :: (#empty: Celsius, text: string) -> ?Celsius {
        if text.count == 0 return .None;
        if text[text.count - 1] != cast(u8)(#char "C") return .None;
        digits := text[0..text.count - 1];
        n := from_string(s64, digits);
        if n == {
            case .Some(v); return .Some(.{ .degrees = v });
            case .None;    return .None;
        }
    }
}

main :: () {
    a := from_string(Celsius, "21C");
    b := from_string(Celsius, "21F");
    if a == { case .Some(v); print("parsed  = %\n", v); case .None; print("parsed  = none\n"); }
    if b == { case .Some(v); print("refused = %\n", v); case .None; print("refused = none\n"); }
}
```

CONSOLE / OUTPUT

```
parsed  = 21C
refused = none
```

NOTE

A default placeholder selects `Formatter(T, "", W)` and calls `Show(T).show` directly. Generic `Show` instances are instantiated by the formatting call, so derived generic unions and `?T` render their values without requiring an earlier direct `show` call.

GLOSSO

```
#import "Io";

Tagged :: union #derive(Show) { Some_Value: $A; Nothing: void; }

Color :: enum { Red; Green; Blue; }

main :: () {
    t: Tagged(s64) = .Some_Value(9);
    o: ?s64 = .Some(3);
    print("print only  = %\n", t);
    print("option only = %\n", o);
    print("enum        = %\n", Color.Green);
    print("bool        = %\n", true);
}
```

CONSOLE / OUTPUT

```
print only  = Some_Value(9)
option only = Some(3)
enum        = 1
bool        = 1
```

NOTE

Enums and booleans still use the default `Show` rendering as ordinals in every case: `show_enum` is the integer shower, so `Color.Green` is `1` and `show(true)` is `1`. A variant name in output requires a hand-written instance.

Standard and user-defined selectors

Selector	Argument	Rendering
`%`, `%{}`	any `Show` type	its default `show` rendering
`%d`, `%x`, `%X`, `%b`, `%o`	integer	decimal, lower/upper hexadecimal, binary, octal; no radix prefix
`%c`	`u8` or `rune`	a UTF-8 character; a byte denotes U+0000 through U+00FF

Numeric selectors preserve signedness and all integer widths, including 128-bit values. `%c` encodes `cast(u8)16#e9` as the two UTF-8 bytes for `é`, not one raw byte. Invalid rune values created by casts render as U+FFFD. A `rune` does not satisfy the integer selectors; use `%` for its decimal scalar value or cast explicitly.

GLOSSO

```
#import "Io";
#import "Format";

Token :: struct { value: u8; }

(Token, "token", $W) ::: instance Formatter where Writer(W) {
    format_to :: (writer: *mut W, value: Token) -> Io_Error!ssize {
        return writer.write_format("<%X>", value.value);
    }
}

main :: () {
    token: Token = .{ .value = 16#ab };
    print("%{token} %x %c %{}C\n", token, 255, #char "λ", 21);
}
```

CONSOLE / OUTPUT

```
<AB> ff λ 21C
```

`Token` has no `Show` instance and needs none for `%{token}`. Selectors are ordinary typeclass instance keys: no registration or parser modification is required, and the same selector may render different types differently. A one-letter key supports both `%Z` and `%{Z}`. Wrappers should retain a `#comptime` format parameter and forward `args: ...any` as `args...` to preserve this dispatch.

For fallible output, `value.write_format` formats to any `Writer` and returns `Io_Error!ssize`. Its success count measures bytes, not characters. An error includes the bytes completed before and inside the failing formatter, so custom instances must preserve partial-write counts and must not retain writer or value borrows. A `String_Builder` destination stages the rendering before appending it, so a later argument can safely borrow the builder's existing text even when an earlier write would grow its allocation.

8.12 `Try`, `?`, and `#try`

Postfix `?` is an operator owned by the `Try` typeclass. Any type that implements `Try` is a carrier that `?` can unwrap. The class splits a carrier into an `Output` for success and a `Residual` for failure, and supplies methods that convert between them. The suffix annotation `#operator(suffix, try(Target))` makes the compiler pass the enclosing boundary's type to the operator and interpret its returned control token.

GLOSSO

```
Propagate :: typeclass (From: type, To: type) {
    propagate :: (#empty: To, value: From) -> To;
}

Try_Branch :: union {
    Continue: $T;
    Break: $R;
}

Try :: typeclass (Carrier: type)
    #minimal(from_output, from_residual, branch) {
    Output :: type;
    Residual :: type;

    from_output :: (#empty: Carrier, output: Output) -> Carrier;
    from_residual :: (#empty: Carrier, residual: Residual) -> Carrier;
    branch :: (value: Carrier) -> Try_Branch(Output, Residual);

    '?' :: (#empty: $Target, value: Carrier)
        -> Try_Branch(Output, Target)
        #operator(suffix, try(Target))
        where Try(Target) &&
            (Try(Carrier).Residual == Try(Target).Residual ||
             Propagate(Try(Carrier).Residual, Try(Target).Residual)) {
        if branch(value) == {
            case .Continue(output);
                #if Try(Carrier).Output == void return .Continue;
                else return .Continue(output);
            case .Break(residual);
                #if Try(Carrier).Residual == Try(Target).Residual {
                    return .Break(from_residual(Target, residual));
                } else {
                    converted := propagate(Try(Target).Residual, residual);
                    return .Break(from_residual(Target, converted));
                }
        }
    }
}
```

The declaration lives in `std/Prelude/Try.glo` and is available without an import. The default `?` body is inherited by an instance, which may override it with the same boundary witness, return type, and `#operator(suffix, try(Target))` contract. `#minimal` names the three required methods; `Output` and `Residual` must also be supplied to make their signatures resolvable.

Member	Direction	Role
`Output`	associated type	what a successful carrier holds
`Residual`	associated type	what a failed carrier holds, stripped of the carrier
`from_output`	value into carrier	wraps a bare `Output` returned from a Try-returning function
`from_residual`	residual into carrier	builds the carrier that `?` returns early
`branch`	carrier into token	splits success from residual failure

`Try_Branch(T, R)` is an ordinary two-variant union. `branch` returns `Try_Branch(Output, Residual)`, while `?` returns `Try_Branch(Output, Target)`: its `.Break` holds the completed boundary carrier. Returning this union from an ordinary procedure has no control-flow effect on its caller. A carrier can be a struct or a union; its `branch` implementation decides which side it holds.

How `x?` is checked

A potentially failing `?` needs a boundary that can absorb a `Break`. That is either an enclosing function whose return type implements `Try`, or an enclosing `#try` block. Without one the operator is `error: postfix '?' requires a Try-returning function or an enclosing #try block`. With one, the checker performs these steps. An infallible `#Never!T` operand cannot take the failure branch, so its `?` also works in an ordinary non-fallible function and yields `T` directly.

- The operand and boundary types must implement `Try`. The operand is evaluated once, and the boundary type is supplied as the leading erased witness in the equivalent call `'?'(Target, operand)`. A `#try` expression takes its carrier from its first propagated operand.
- The selected operator body runs, including an instance override. It must return exactly `Try_Branch(Try(Source).Output, Target)`, where `Source` is the operand's type. A wrong token type is rejected with a diagnostic requiring the boundary result in `Break`.
- On `.Continue(v)` the expression's value is `v`, typed as the operand's `Output`.
- The inherited operator calls `branch`. On failure it calls `propagate` only when the two residual types differ, then calls `from_residual(Target, residual)`. These are ordinary calls in the library operator body, not conversions inserted by the compiler. Matching residuals need no `Propagate` instance, even when the source and destination success types differ.
- On `.Break(carrier)`, the compiler exits the nearest boundary with that already completed carrier. The operator's conversion and construction finish before the exiting scopes run their `defer` and `#try defer` entries in reverse registration order.
- Separately, returning a bare `Output` from a Try-returning function wraps it with `from_output`, which is why `return x + y;` is legal in a function declared `E!s64`.

A quoted call returns the token without leaving the caller, so the control-flow expansion can be written out. These functions produce the same results; an explicit `return` does not run `#try defer`, so this hand-written expansion would need additional cleanup if the function registered error-only defers.

GLOSSO

```
#import "Io";

fetch :: (ok: bool) -> string!s64 {
    if !ok return .Err("bad input");
    return .Ok(20);
}

sugared :: (ok: bool) -> string!s64 {
    v := fetch(ok)?;
    return v + 1;
}

expanded :: (ok: bool) -> string!s64 {
    v: s64;
    token := '?'(string!s64, fetch(ok));
    if token == {
        case .Continue(output); v = output;
        case .Break(carrier);   return carrier;
    }
    return from_output(string!s64, v + 1);
}

report :: (r: string!s64) -> string {
    if r == {
        case .Ok(v);  return show(v);
```

```
        case .Err(e); return e;
    }
}

main :: () {
    print("sugared  % %\n", report(sugared(true)), report(sugared(false)));
    print("expanded % %\n", report(expanded(true)), report(expanded(false)));
}
```

CONSOLE / OUTPUT

```
sugared  21 bad input
expanded 21 bad input
```

The quoted call requires both the boundary type and the operand; `'?'(value)` is rejected for a missing boundary witness. It can be used outside a Try boundary because it returns an ordinary token. `Try.'?'(Target, value)` and `Try(Source).'?'(Target, value)` also select the class explicitly. `Try(C).Output` and `Try(C).Residual` are available as ordinary type annotations.

The built-in carriers

`?T` and `E!T` are compiler-built-in types, but their `Try` instances are ordinary Glosso in `std/Prelude/Option.glo` and `std/Prelude/Result.glo`. Each residual is a separate type that carries the failure without the carrier around it, which is what lets one function's failure be converted into another's.

Carrier	Output	Residual
`?T`	`T`	`Option_Residual`, an enum with the single variant `None`
`E!T`	`T`	`Result_Residual(E)`, a struct holding one `error: E`

GLOSSO

```
#import "Io";

Parse_Error :: struct #derive(Show) { text: string; }

parse :: (text: string) -> Parse_Error!s64 {
    v := from_string(s64, text);
    if v == {
        case .Some(n); return .Ok(n);
        case .None;    return .Err(.{ .text = text });
    }
}

sum_two :: (a: string, b: string) -> Parse_Error!s64 {
    x := parse(a)?;
    y := parse(b)?;
    return x + y;
}

head :: (xs: []mut s64) -> ?s64 {
    if xs.count == 0 return .None;
    return .Some(xs[0]);
}

first_plus_one :: (xs: []mut s64) -> ?s64 {
    return head(xs)? + 1;
}

main :: () {
    r := sum_two("3", "4");
    e := sum_two("3", "oops");
    if r == { case .Ok(v); print("ok  %\n", v); case .Err(x); print("err %\n", show(x)); }
    if e == { case .Ok(v); print("ok  %\n", v); case .Err(x); print("err %\n", show(x)); }

    storage: [1]s64;
    storage[0] = 41;
    empty: []mut s64;
    print("some %\n", show(first_plus_one(storage[..])));
    print("none %\n", show(first_plus_one(empty)));
}
```

CONSOLE / OUTPUT

```
ok  7
err {oops}
some Some(42)
none None
```

`#try` blocks

`#try { expr }` introduces a boundary without introducing a function. It is an expression whose value is the carrier, so a `?` inside it stops at the closing brace instead of leaving the enclosing function. The carrier type is inferred from the first `?` in the block; a block with none is `error: cannot infer the carrier type of #try expression; propagate at least one value with '?'`.

GLOSSO

```
#import "Io";

Parse_Error :: struct #derive(Show) { text: string; }

parse :: (text: string) -> Parse_Error!s64 {
    v := from_string(s64, text);
    if v == {
        case .Some(n); return .Ok(n);
        case .None;    return .Err(.{ .text = text });
    }
}

main :: () {
    good := #try { parse("5")? + parse("6")? };
    bad  := #try { parse("5")? + parse("x")? };
    if good == { case .Ok(v); print("ok  %\n", v); case .Err(e); print("err %\n", show(e)); }
    if bad  == { case .Ok(v); print("ok  %\n", v); case .Err(e); print("err %\n", show(e)); }
}
```

CONSOLE / OUTPUT

```
ok  11
err {x}
```

The grammar is narrow on purpose, because the block has to have a value. It holds exactly one expression and no statements: a trailing semicolon is `error: #try contains one expression; remove the semicolon`, and a `return` inside it is `error: return is not allowed inside a #try expression`. The expression's type must be the carrier's `Output`. Error-only cleanup at a boundary is written with `#try defer`, described in `defer` and `#try defer`.

Writing a `Try` instance

An instance supplies the two associated types and the three methods. `branch` carries the real logic; `from_output` and `from_residual` are usually one constructor each. Because the head may be generic, a carrier parameterised by its payload is written with a section hole and referred to without the `$` inside the body.

GLOSSO

```
#import "Io";

Validated :: union { Valid: $T; Invalid: string; }
Invalid_Reason :: struct { message: string; }

Validated($T) ::: instance Try {
    Output   :: T;
    Residual :: Invalid_Reason;

    from_output :: (#empty: Validated(T), output: T) -> Validated(T) {
        return .Valid(output);
    }
    from_residual :: (#empty: Validated(T), residual: Invalid_Reason) -> Validated(T) {
        return .Invalid(residual.message);
    }
    branch :: (value: Validated(T)) -> Try_Branch(T, Invalid_Reason) {
        if value == {
            case .Valid(v);   return .Continue(v);
            case .Invalid(m); return .Break(.{ .message = m });
        }
    }
}

positive :: (n: s64) -> Validated(s64) {
    if n <= 0 return .Invalid("not positive");
    return .Valid(n);
}

doubled :: (n: s64) -> Validated(s64) {
    v := positive(n)?;
    return v * 2;
}

main :: () {
    a := doubled(7);
    b := doubled(-1);
    if a == { case .Valid(v); print("valid   %\n", v); case .Invalid(m); print("invalid %\n", m); }
    if b == { case .Valid(v); print("valid   %\n", v); case .Invalid(m); print("invalid %\n", m); }
}
```

```
CONSOLE / OUTPUT
valid   14
invalid not positive
```

`return v * 2;` in `doubled` is a bare `Output`, wrapped by `from_output`. The `Residual` is a type of the instance's own choosing, and keeping it distinct from the carrier is what allows a `Propagate` instance to convert it later.

> **NOTE**
> A missing associated type does not report itself as a missing associated type. Dropping `Output` makes the declared signature of `from_output` unresolvable, and the instance is rejected as `error: instance 'X :: Try' does not satisfy #minimal(from_output, from_residual, branch)` with a note pointing at the class. Check the associated types first when that message appears.

8.13 `Propagate` and Multi-Error Programs

`Propagate` is the conversion `?` performs when the value being propagated fails with one residual and the enclosing boundary requires another. It is deliberately narrower than a general implicit conversion protocol: nothing else in the language consults it, and it is never applied to a successful value.

```
GLOSSO
Propagate :: typeclass (From: type, To: type) {
    propagate :: (#empty: To, value: From) -> To;
}
```

The class has two parameters and one method, and the head is written as a tuple: `(From_Type, To_Type) ::: instance Propagate`. The leading `#empty:` witness names the target so instance selection can match its destination. The inherited `?` body calls `propagate` only when the two residual types differ; matching residuals need no conversion instance, even for carriers with different success types.

The Prelude's residual lifting

A residual is not the error; for `E!T` it is `Result_Residual(E)`, a struct wrapping one `E`. Writing conversions at the residual level would mean writing wrapper structs by hand for every pair of error types. The Prelude removes that step with one blanket instance in `std/Prelude/Result.glo`.

```
GLOSSO
(Result_Residual($From), Result_Residual($To)) ::: instance Propagate
    where Propagate(From, To) {
    propagate :: (
        #empty: Result_Residual(To),
        value: Result_Residual(From),
    ) -> Result_Residual(To) {
        return .{ .error = propagate(To, value.error) };
    }
}
```

The consequence is the rule to remember: a `Propagate` instance between two error types is enough to make `?` widen between the corresponding results. The instance below converts `Low_Error` into `High_Error`, and `?` in a `High_Error!s64` function then accepts a `Low_Error!s64` with no other change.

```
GLOSSO
#import "Io";

Low_Error  :: struct #derive(Show) { code: s64; }
High_Error :: struct #derive(Show) { code: s64; note: string; }

(Low_Error, High_Error) ::: instance Propagate {
    propagate :: (#empty: High_Error, value: Low_Error) -> High_Error {
        return .{ .code = value.code, .note = "lifted" };
    }
}

low :: (ok: bool) -> Low_Error!s64 {
    if !ok return .Err(.{ .code = 7 });
    return .Ok(2);
}

high :: (ok: bool) -> High_Error!s64 {
    v := low(ok)?;
    return v * 10;
}

main :: () {
    a := high(true);
    b := high(false);
    if a == { case .Ok(v); print("ok  %\n", v); case .Err(e); print("err %\n", show(e)); }
```

```
    if b == { case .Ok(v); print("ok  %\n", v); case .Err(e); print("err %\n", show(e)); }
}
```

CONSOLE / OUTPUT

```
ok  20
err {7, lifted}
```

Instances between residuals

The lifting instance only covers result-to-result. Converting between different carriers means writing the instance at the residual level, because there is no error type on the option side to lift from. Absorbing an option into a result is the common case: the residual `Option_Residual` carries no information, so the instance supplies the error it stands for.

GLOSSO

```
#import "Io";

Missing :: struct #derive(Show) { key: string; }

(Option_Residual, Result_Residual(Missing)) ::: instance Propagate {
    propagate :: (
        #empty: Result_Residual(Missing),
        value: Option_Residual,
    ) -> Result_Residual(Missing) {
        return .{ .error = .{ .key = "unknown" } };
    }
}

lookup :: (present: bool) -> ?s64 {
    if !present return .None;
    return .Some(9);
}

required :: (present: bool) -> Missing!s64 {
    return lookup(present)? + 1;
}

main :: () {
    a := required(true);
    b := required(false);
    if a == { case .Ok(v); print("ok  %\n", v); case .Err(e); print("err %\n", show(e)); }
    if b == { case .Ok(v); print("ok  %\n", v); case .Err(e); print("err %\n", show(e)); }
}
```

CONSOLE / OUTPUT

```
ok  10
err {unknown}
```

The opposite direction, discarding an error to reach an option, is not in the Prelude either, and for a stronger reason: throwing information away is a decision a program makes, not a default. One generic instance over every error type makes `?` usable from a `?T` function.

GLOSSO

```
(Result_Residual($E), Option_Residual) ::: instance Propagate {
    propagate :: (#empty: Option_Residual, value: Result_Residual(E)) -> Option_Residual {
        return .None;
    }
}
```

Without the needed residual conversion, the inherited operator's `where` condition fails. A postfix use reports `error: no overload '?' accepts these arguments`. Supply a `Propagate` instance for the residual pair, or for the error pair when the Prelude's result lifting instance applies.

Widening across a library boundary

Because instance selection is global and there is no orphan rule, the widening instance belongs to whichever code owns the wider error, not to the library that raised the narrow one. A library can therefore define a small, precise error type and stay unaware of every application that folds it into a larger one.

GLOSSO

```
// config.glo
Config_Error :: struct #derive(Show) { key: string; }

lookup :: (key: string) -> Config_Error!string {
    if key == "port" return .Ok("8080");
    return .Err(.{ .key = key });
}
```

`#from` variants

Writing one `Propagate` instance per contributing error is mechanical, so a union variant may be marked `#from`. The compiler then generates the instance from the source type to the union. The application below folds a library error and a local one into a single `Boot_Error` and propagates both with plain `?`.

GLOSSO

```
#import "Io";
#load "config.glo";

Bad_Number :: struct #derive(Show) { text: string; }

Boot_Error :: union #derive(Show) {
    #from config: Config_Error;
    #from number: Bad_Number;
}

to_number :: (text: string) -> Bad_Number!s64 {
    v := from_string(s64, text);
    if v == {
        case .Some(n); return .Ok(n);
        case .None;    return .Err(.{ .text = text });
    }
}

port :: (key: string) -> Boot_Error!s64 {
    text := lookup(key)?;
    return to_number(text)?;
}

main :: () {
    a := port("port");
    b := port("host");
    if a == { case .Ok(v); print("ok  %\n", v); case .Err(e); print("err %\n", show(e)); }
    if b == { case .Ok(v); print("ok  %\n", v); case .Err(e); print("err %\n", show(e)); }
}
```

CONSOLE / OUTPUT

```
ok  8080
err config({host})
```

`#from` is independent of `#derive(Error)`; it is a property of the variant, and the generated instance is exactly what a hand-written one would be. Writing the two instances below in place of the two `#from` markers produces the same program.

GLOSSO

```
(Config_Error, Boot_Error) ::: instance Propagate {
    propagate :: (#empty: Boot_Error, value: Config_Error) -> Boot_Error {
        return .config(value);
    }
}
(Bad_Number, Boot_Error) ::: instance Propagate {
    propagate :: (#empty: Boot_Error, value: Bad_Number) -> Boot_Error {
        return .number(value);
    }
}
```

The generator is narrow, because each restriction would otherwise produce an instance that cannot be keyed on a single source type.

Rejected form	Diagnostic
a `#raw` union	`#from is not defined for #raw union`
a generic union	`#from does not yet support generic unions`
a payload-less variant	`#from variant 'a' must have a payload`
two variants of one type	`#from variant 'b' duplicates the source type A_Error`

> **NOTE**
> `Propagate` is consulted only by `?`. Returning `.Err(e)` explicitly from a function whose error type differs is an ordinary type mismatch, and no conversion is attempted. This keeps the widening visible: it happens exactly where a `?` is written.

8.14 `Error`, `Termination`, and `main`

`Error` and `Termination` are the two classes at the edge of a program. `Error` says how a value explains itself to the operator; `Termination` says how a value becomes a process exit status. `main` may return any type that implements `Termination`, and the standard pairing of the two is what makes `main :: () -> App_Error!void` a complete error-reporting program.

```
GLOSSO
Error :: typeclass (E: type) #derive(derive_error) #minimal(report_error) {
    report_error :: (error: E) -> void;
}

Termination :: typeclass (T: type) #minimal(termination_status) {
    termination_status :: (value: T) -> s32;
}
```

`report_error` returns nothing and is expected to write a diagnostic, conventionally to standard error. The class does not fix a format, a destination, or a severity; it fixes only that an error type knows how to describe itself without being handed a formatter.

Allocation-free reporting

Reporting is the last thing a failing program does, and it must work when the allocator is exhausted and when the higher-level output modules are themselves the thing that failed, since their own error types implement `Error`. `std/Typeclass/Error.glo` therefore provides two helpers that write straight to file descriptor 2 through the `file_write_result` intrinsic, touching neither the allocator nor `Io`.

```
GLOSSO
string ::: #public {
    error_report_write :: (#self) {
        written, code := __error_file_write_result(2, #self);
        _ := written;
        _ := code;
    }

    error_report_line :: (#self, message: string) {
        #self.error_report_write();
        (": ").error_report_write();
        message.error_report_write();
        ("\n").error_report_write();
    }
}
```

Helper	Writes
`text.error_report_write()`	`text`, exactly, with no newline
`category.error_report_line(message)`	`category`, `": "`, `message`, newline

Both ignore the write result. A report that cannot be written has nowhere left to report that fact, so the failure is dropped rather than escalated. An instance that wants formatted output can still call `tprint`, at the cost of depending on the temporary allocator on the failure path.

`derive_error`

`#derive(Error)` on a tagged union writes the obvious instance: match the union, and call `report_error` on the payload. It composes with `#from`, so one union declaration can both aggregate several error types and report all of them.

Requirement	Diagnostic when violated
must be a tagged union	`Error can currently be derived only for tagged unions`
not `#raw`	`Error derive is not defined for #raw union`
not generic	`Error derive does not yet support generic unions`
every variant has a payload	`Error derive requires every union variant to have an Error payload`

A payload that exists but has no `Error` instance is not caught by that last check. The generated body fails to compile instead, with `error: no overload 'report_error' accepts these arguments` pointing into `std/Typeclass/Error.glo`, which is the line the macro emitted.

`Termination` and the return type of `main`

`prepare_main_termination_dispatch` checks `main`'s return type at compile time and inserts the `termination_status` call. Two instances ship with the standard library: one for every integer type, and one for `E!void` where `E` implements `Error`.

```
GLOSSO
$T ::: instance Termination where Is_Integer(T) {
    termination_status :: (value: T) -> s32 {
        return cast(s32)value;
    }
}

$E!void ::: instance Termination where Error(E) {
```

```
    termination_status :: (value: E!void) -> s32 {
        if value == {
            case .Ok; return 0;
            case .Err(error);
                report_error(error);
                return 1;
        }
    }
}
```

`main` returns	Status	Notes
nothing	0	the ordinary form
any integer type	the value, narrowed to `s32`	the host applies its own exit-code rules
`E!void` where `E: Error`	0 on `.Ok`, 1 on `.Err`	`report_error(error)` runs first
a type with a `Termination` instance	whatever `termination_status` returns	a struct or enum works; see below
`?T`, `bool`, `E!T` for non-`void T`	rejected	`main return type Option_s64 does not implement Termination`

The `E!void` instance is the only one keyed on a result, and it is keyed on `void` specifically: `main :: () -> My_Err!s64` is `error: main return type Result_s64_My_Err does not implement Termination`, because a successful `s64` has no meaning at the process boundary that the integer instance does not already give. `bool` is rejected for the same reason `Is_Integer(bool)` is false.

An application-specific instance is a small declaration and needs nothing else. Its `termination_status` must have a runtime body: a `#magic` implementation is `main's Termination implementation must have a runtime function body`.

GLOSSO

```
#import "Io";
// Termination is supplied by the Prelude.

Exit_Report :: struct { failures: s64; }

Exit_Report ::: instance Termination {
    termination_status :: (value: Exit_Report) -> s32 {
        if value.failures == 0 return 0;
        return 3;
    }
}

main :: () -> Exit_Report {
    print("two tests failed\n");
    return .{ .failures = 2 };
}
```

CONSOLE / OUTPUT

```
two tests failed
glosso: program exited with status exit code: 3
```

A complete failing program

Putting the pieces together: each leaf error implements `Error`, a union aggregates them with `#from` and derives `Error`, and `main` returns that union in a result. Every `?` widens through the generated `Propagate` instances, the union's derived `report_error` dispatches to the payload's, and the `E!void Termination` instance turns the whole thing into status 1.

GLOSSO

```
#import "Io";
#import "Typeclass/Error";

Disk_Full :: struct { path: string; bytes: s64; }
Disk_Full ::: instance Error {
    report_error :: (error: Disk_Full) -> void {
        ("disk full writing ").error_report_write();
        error.path.error_report_write();
        ("\n").error_report_write();
    }
}

Timed_Out :: struct { seconds: s64; }
Timed_Out ::: instance Error {
    report_error :: (error: Timed_Out) -> void {
        ("timed out").error_report_line("the request took too long");
    }
}

Job_Error :: union #derive(Error) {
```

```
    #from disk: Disk_Full;
    #from timeout: Timed_Out;
}

run :: (which: s64) -> Job_Error!void {
    if which == 0 return .Err(.disk(.{ .path = "out.bin", .bytes = 4096 }));
    return .Err(.timeout(.{ .seconds = 30 }));
}

main :: () -> Job_Error!void {
    print("starting\n");
    run(1)?;
    return .Ok();
}
```

CONSOLE / OUTPUT

```
starting
timed out: the request took too long
glosso: program exited with status exit code: 1
```

Returning `.Ok()` produces status 0 and prints nothing extra. The `glosso:` line is the driver reporting a nonzero status under `--run`; the exit status itself is the subject of Environment, Toolchain, and Exit Codes.

> **NOTE**
> `Error` and `Termination` are separate on purpose. An error type reports; a termination type decides a number. A program that wants several distinct nonzero statuses writes its own `Termination` instance over its error union rather than trying to encode the status in `report_error`, whose return type is `void`.

8.15 Typeclass-Constrained Variadics

A variadic parameter may name a typeclass instead of a type. `args: ...Show` accepts any number of arguments provided every one of them implements `Show`. The constraint is checked element by element at the call site, so a rejected argument is reported at the argument that caused it rather than inside the callee. The other variadic forms are covered in Variadic Parameters.

GLOSSO

```
show_all         :: (values: ...Show)                { }
count_both       :: (values: ...(Show && PartialEq)) { }
anything         :: (values: ...any)                 { }
```

A parenthesised `&&` list requires every named class of every element. There is no `||` form; `...(Show || PartialEq)` is `error: unexpected token '||'`, because a disjunction would leave the callee with no method it could call on all elements. `...any` is the unconstrained pack, useful when the body only reflects over the element types.

GLOSSO

```
#import "Io";
#import "Format";
#import "Meta";

Named :: typeclass (T: type) #minimal(name_of) {
    name_of :: (value: T) -> string;
}

$T ::: instance Named where Is_Integer(T) { name_of :: (v: T) -> string { return "ssize"; } }
$T ::: instance Named where Is_String(T)  { name_of :: (v: T) -> string { return "text"; } }

join_names :: (args: ...Named) -> string {
    out := "";
    for args {
        if it_index > 0 out = tprint("%,%", out, name_of(it));
        else out = name_of(it);
    }
    return out;
}

count_both :: (values: ...(Show && PartialEq)) -> s64 {
    return values.count;
}

main :: () {
    print("names = %\n", join_names(1, "x", 2));
    print("both  = %\n", count_both(10, "y"));
}
```

CONSOLE / OUTPUT

```
names = ssize,text,ssize
both  = 2
```

Element-wise checking

A failing element is named with its type, the class it lacks, and the call it appeared in: passing a `string` to `join_names` above with no string instance in scope is `error: variadic arg type` string `does not implement` Named `in call` join_names`. A conjunction reports the first class the element misses, not the whole list, so an argument that is `Show` but not `PartialEq` reports only `does not implement `PartialEq`.

Expansion is checked the same way. `xs...` splices a typed array into the pack, and because every element has one type the check runs once for that type: `error: expanded variadic element type` Plain `does not implement` Show `in call` show_all``. Formatting itself now takes `...any` and checks `Formatter` after selecting each placeholder; `show_all` retains an explicit `...Show` constraint.

GLOSSO

```
#import "Io";

show_all :: (values: ...Show) {
    for values print("  [%] %\n", it_index, it);
}

main :: () {
    fixed: [3]s32;
    fixed[0] = 10; fixed[1] = 20; fixed[2] = 30;
    show_all(fixed...);
    show_all(1, "two", 3.5);
}
```

CONSOLE / OUTPUT

```
  [0] 10
  [1] 20
  [2] 30
  [0] 1
  [1] two
  [2] 3.5
```

The pack inside the callee

The pack is a heterogeneous, read-only view. `type_of(value)` on the pack itself answers `tuple`; each element is boxed so it can be indexed with a runtime index, which is what makes the `[]any` view real and what `__format` relies on when it walks `args[ai]`. Indexing with a constant, or iterating, recovers the element's precise static type.

GLOSSO

```
#import "Io";
#import "Meta";

inspect :: (values: ...Show) {
    pack_type :: type_of(values);
    elem_type :: type_of(values[1]);
    print("pack type = %\n", pack_type);
    print("elem type = %\n", elem_type);
    print("count     = %\n", values.count);
    print("values[1] = %\n", values[1]);
}

main :: () {
    inspect(1, "two", 3.5);
}
```

CONSOLE / OUTPUT

```
pack type = tuple
elem type = string
count     = 3
values[1] = two
```

- `values.count` is the number of arguments, but it is an ordinary runtime value inside the callee. Binding it with `::` or testing it in a `#if` is `error: unknown const values`; a body that must branch on arity does so with `#inline for` instead.
- A plain `for` over the pack is unrolled, so `it` has the element's own static type in each iteration and `name_of(it)` dispatches to the right instance. `it_index` is the position.
- A runtime index is allowed and reads through the boxed view, so a `while i < values.count` loop over `values[i]` renders each element correctly even though the index is not a constant.
- The view is read-only. Assigning to a pack element is not supported and currently fails during lowering with `error: internal error: 'op_lbrrbreq_tuple_ssize_ssize' is called but was never lowered`; copy the element into a local instead.

`#inline for` and per-element evidence

`#inline for` over a pack is the tool for code that must differ per element. The body is expanded once per argument with the element's type known, so `type_of(it)` can be bound to a compile-time constant and that constant used in an evidence query. `#if Class(T)` answers whether `T` has an instance, and only the taken branch is checked.

GLOSSO

```
#import "Io";
#import "Meta";

Unit :: typeclass (T: type) #minimal(unit_of) {
    unit_of :: (value: T) -> string;
}

Meters  :: struct { n: f64; }
Seconds :: struct { n: f64; }

Meters  ::: instance Unit { unit_of :: (value: Meters)  -> string { return "m"; } }
Seconds ::: instance Unit { unit_of :: (value: Seconds) -> string { return "s"; } }
Meters  ::: instance Show { show :: (value: Meters)  -> string { return "meters"; } }

label_all :: (values: ...Unit) {
    #inline for values {
        elem :: type_of(it);
        #if Show(elem) {
            print("  % (%) shows as %\n", elem, unit_of(it), Show(elem).show(it));
        } else {
            print("  % (%) has no Show\n", elem, unit_of(it));
        }
    }
}

main :: () {
    m: Meters  = .{ .n = 3.0 };
    s: Seconds = .{ .n = 4.0 };
    label_all(m, s);
}
```

CONSOLE / OUTPUT

```
  Meters (m) shows as meters
  Seconds (s) has no Show
```

`Show(elem).show` in that body is a scoped typeclass method reference: it names the `show` of the instance for exactly `elem`, which is what lets the call be written in a branch that only some elements reach. The unqualified `unit_of(it)` needs no scope because the constraint on the pack already guarantees an instance.

Structural conditions rather than class membership are written with `#if !#meaningful { it.field; } continue;`, which skips an element whose shape the body cannot use. That form belongs to Structural Constraints with `#meaningful`.

> **NOTE**
> An evidence query is compile-time only. `Class(T)` in a runtime expression is `error: internal error: 'Marker' is called but was never lowered`; bind it to a constant or use it directly in `#if`. Supply every declared compile-time argument: a two-parameter class is queried as `Convert(s64, string)`. The same full argument list works in constant expressions and `where` clauses; a missing argument produces an arity diagnostic.

9 Operators

9.1 The Operator System

Every operator in Glosso is a typeclass method. There is no separate operator-overloading feature and no free operator function: a spelling is declared by exactly one typeclass, and the bodies that give it meaning live in that class's instances. Writing an operator at top level is rejected outright with `error: operator '<+>' must be declared by a typeclass and implemented by its instances`.

The class declares the operator's signature, its fixity and, for a binary operator, its precedence; each instance supplies a body for one operand-type combination. Because instance heads are keyed on the complete operand tuple, `(Vec2, f64)` and `(f64, Vec2)` are separate instances and an asymmetric operator is expressible.

GLOSSO

```
#import "Io";

Vec2 :: struct { x: f64; y: f64; }

Dot :: typeclass (L: type, R: type) {
    Output :: type;
    '<*>' :: (left: L, right: R) -> Output #operator(left, 19);
}

(Vec2, Vec2) ::: instance Dot {
    Output :: f64;
    '<*>' :: (a: Vec2, b: Vec2) -> f64 { return a.x * b.x + a.y * b.y; }
}

main :: () {
    a: Vec2 = .{ .x = 1.0, .y = 2.0 };
    b: Vec2 = .{ .x = 3.0, .y = 4.0 };
    print("a <*> b     = %\n", a <*> b);
    print("a <*> b + 1 = %\n", a <*> b + 1.0);
}
```

CONSOLE / OUTPUT

```
a <*> b     = 11
a <*> b + 1 = 12
```

The `Output` associated type is the house pattern for an operator class: it lets `<*>` return `f64` from two `Vec2` operands without the class fixing a result type for everybody. Associated types are covered in Associated Types; instance heads in Instances: Heads, Bodies, and `where`.

One owner per name

Ownership is recorded when the class is declared, and a second claim on the same operator is an error wherever it comes from. The check also runs on instances, so an instance may only implement an operator its own class declares.

CONSOLE / OUTPUT

```
error: operator '+' is already owned by typeclass 'Add'
error: operator '[]' is already owned by typeclass 'Indexable'
error: operator '<+>' has no owning typeclass
error: operator '<+>' declares conflicting precedence
```

The first two come from a class trying to redeclare a Prelude spelling. The third comes from an instance whose class never declared that operator. The fourth comes from an instance repeating `#operator` with a level or associativity that disagrees with the class.

GLOSSO

```
'<+>' :: (a: s64, b: s64) -> s64 #operator(left, 17) { return a + b; }
main :: () {}
```

CONSOLE / OUTPUT

```
error: operator '<+>' must be declared by a typeclass and implemented by its instances
```

Arity and fixity are part of the name

The ownership key is the spelling together with its arity and its fixity: `prefix` or `suffix` for a unary operator, `infix` for everything else. Two classes may therefore share a spelling as long as they differ in fixity, which is exactly how the Prelude gives `-` and `..` more than one meaning.

Spelling	Fixity	Owning class
`-`	prefix	`Neg`
`-`	infix	`Sub`
`..`	prefix	`RangeToOp`
`..`	infix	`RangeExclusiveOp`
`..`	suffix	`RangeFromOp`

A program can do the same with its own spellings. Two classes below claim `~~` as a prefix operator and `~~~` as a suffix operator, and one type implements both.

GLOSSO

```
Norm :: typeclass (T: type) {
    Output :: type;
    '~~' :: (value: T) -> Output #operator(prefix);
}

Total :: typeclass (T: type) {
    Output :: type;
    '~~~' :: (value: T) -> Output #operator(suffix);
}

Vec2 ::: instance Norm {
    Output :: f64;
    '~~' :: (v: Vec2) -> f64 { return v.x * v.x + v.y * v.y; }
}

Vec2 ::: instance Total {
    Output :: f64;
    '~~~' :: (v: Vec2) -> f64 #operator(suffix) { return v.x + v.y; }
}

main :: () {
    a: Vec2 = .{ .x = 1.0, .y = 2.0 };
    print("~~a  = %\n", ~~a);
    print("a~~~ = %\n", a~~~);
}
```

CONSOLE / OUTPUT

```
~~a  = 5
a~~~ = 3
```

> **NOTE**
> A suffix instance method must repeat `#operator(suffix)`. Fixity is part of the ownership key and defaults to `prefix`, so an instance that omits the directive is looking for a prefix owner and reports `error: operator '~~~' has no owning typeclass`. Prefix and infix instances may omit the directive, because the class already settled the fixity that its key matches.

The `#operator` directive

CONSOLE / OUTPUT

```
#operator( left | right | assign , level )
#operator( prefix | suffix )
#operator( suffix, try(Target) )
```

Mode	Means	Level
`left`	infix, groups to the left	required
`right`	infix, groups to the right	required
`prefix`	unary, before its operand	rejected
`suffix`	unary, after its operand	rejected
`assign`	update assignment, arity 2	required

`#operator` registers all five modes, on both typeclass method signatures and instance implementations. Levels are integer literals from 0 to 1000; the standard operators use the 0-20 scale described in Operator Precedence and the 0-20 Scale. An instance method may repeat the mode alone, as `#operator(left)`, and inherit the level from its class.

- A binary or assignment operator with no level anywhere is `error: binary operator '<+>' needs #operator(left|right|assign, n)`. Bare `#operator(left)` on the class declaration counts as no level.
- A level on a unary mode does not parse. Write `#operator(prefix)` or `#operator(suffix)` with no numeric argument. A suffix can also carry `try(Target)`, which requests a Try boundary rather than a precedence level.
- An unknown mode is `error: #operator expects left, right, prefix, suffix, or assign`, and a level outside the range is `error: operator precedence level must be between 0 and 1000`.
- An operator function takes one, two or three parameters; anything else is `error: operator functions need one, two, or three parameters`. `assign` additionally requires arity 2, and `[]` and `[]=` are fixed at 2 and 3 and may not declare precedence at all.

Claiming a spelling

An operator name is written between single quotes, which is the only way to get characters like `[` past the lexer, and its characters must come from the eighteen-character operator set `+ - * / < > ! @ % ^ & | ~ ? . , : =`. Built-in spellings are reserved and rejected at the token level; the full rules, including why `?` can never appear inside a longer operator, are in Reserved Operators and Operator Names.

- A name made only of letters can be declared but never written: at a use site the letters lex as an identifier, so `2 plus 3` is `error: unexpected token 'plus'`.
- `[` and `]` are refused except in the two fixed forms `[]` and `[]=`: `error: operator '<[]>' cannot contain '[' or ']'`. All-stars and all-ampersands names are reserved as prefix operators: `*p` dereferences and `&x` takes an address. For example, an attempted prefix `**` overload reports `operator '**' cannot be overloaded as a prefix operator`. The postfix spellings `.*` and `.&` are also built in. Mixed names such as `&&*&&` remain available for custom operators.

> **NOTE**
> The parser learns fixities and levels from an early prescan. It scans the Prelude and relative `#load`/`#export` dependencies before parsing the caller, so a loaded module can supply new operator syntax there. The owning class and instance must also be visible to resolve the expression. The prescan is described in The Operator Prescan.

Suffix operators with a Try boundary

`#operator(suffix, try(Target))` gives a suffix operator one erased boundary witness followed by its operand. Its signature is `(#empty: $Target, value: Source)` and its result must be `Try_Branch(Try(Source).Output, Target)`. Both carriers implement `Try`. The witness is excluded from the unary ownership key, but is included when specializing the operator for a destination type.

GLOSSO

```
#import "Io";
Forward_Try :: typeclass (Source: type) {
    '!!' :: (#empty: $Target, value: Source)
        -> Try_Branch(Try(Source).Output, Target)
        #operator(suffix, try(Target))
        where Try(Source) && Try(Target) &&
            (Try(Source).Residual == Try(Target).Residual ||
             Propagate(Try(Source).Residual, Try(Target).Residual)) {
        return '?'(Target, value);
    }
}
string!s64 ::: instance Forward_Try {}
describe :: (input: string!s64) -> string!string {
    value := input!!;
    return show(value);
}
main :: () {
    print("%\n", (describe(.Ok(41))).unwrap_or("failed"));
    print("%\n", (describe(.Err("stop"))).unwrap_or("failed"));
}
```

CONSOLE / OUTPUT

```
41
failed
```

Postfix `input!!` passes the nearest boundary's carrier type and interprets `.Continue` as a value and `.Break` as an early exit. The quoted call `'!!'(Target, input)` returns the ordinary token; it works outside a boundary. Scoped calls such as `Forward_Try.'!!'(Target, input)` do the same. `?` uses this same annotation; its body can be overridden by a `Try` instance.

> **NOTE**
> `try(Target)` is accepted only on a suffix operator. Write the plain identifier `Target` inside the annotation, matching `$Target` in the leading witness. The operand cannot be lazy, variadic, or defaulted. The declaration must have a Glosso body or be a typeclass signature; `#expand`, `#magic`, and foreign bodies are rejected. An instance override must retain the class's Try boundary contract.

9.2 Assignment Operators

An update assignment such as `x += y` is resolved through a typeclass of its own, one per operator, named after the operation it updates. Plain `=` is not one of them: it is a reserved token, it cannot be overloaded, and the statement form is covered in Assignment and Update Assignment.

Each class follows the same shape as any other binary operator class, with `#operator(assign, 0)` in place of a normal precedence, and each is paired in the Prelude with a blanket instance that defines the compound form in terms of the plain one.

GLOSSO
```
AddAssign :: typeclass (L: type, R: type) {
    Output :: type;
    '+=' :: (left: L, right: R) -> Output #operator(assign, 0);
}

($L, $R) ::: instance AddAssign where Add(L, R) {
    Output :: Add(L, R).Output;
    '+=' :: (left: L, right: R) -> Output { return left + right; }
}
```

The consequence is that `+=` starts working the moment `+` does. A type that implements `Add` needs no further declaration, and a type that wants a different compound behaviour writes its own instance, which outranks the blanket one by ordinary overload scoring.

The assignment classes

Classes	Operators	Blanket guard
`AddAssign` `SubAssign` `MulAssign` `DivAssign` `RemAssign`	`+=` `-=` `*=` `/=` `%=`	`Add` `Sub` `Mul` `Div` `Rem`
`WrapAddAssign` `WrapSubAssign` `WrapMulAssign`	`+%=` `-%=` `*%=`	`WrapAdd` `WrapSub` `WrapMul`
`SaturatingAddAssign` `SaturatingSubAssign` `SaturatingMulAssign`	`+|=` `-|=` `*|=`	`SaturatingAdd` `SaturatingSub` `SaturatingMul`
`ShiftLeftAssign` `ShiftRightAssign` `SaturatingShiftLeftAssign`	`<<=` `>>=` `<<|=`	`ShiftLeft` `ShiftRight` `SaturatingShiftLeft`
`BitAndAssign` `BitOrAssign` `BitXorAssign`	`&=` `|=` `^=`	`BitAnd` `BitOr` `BitXor`
`LogicalAndAssign` `LogicalOrAssign`	`&&=` `||=`	`LogicalAnd` `LogicalOr`

Nineteen classes, nineteen blanket instances, and no others. There is no compound form of a unary operator, and no class for `=`.

Declaring a new assignment operator

A new one is an ordinary operator class whose member carries `#operator(assign, level)`. The level is mandatory; a non-zero level compiles, but `0` is the tier the Prelude uses for the whole family and is what an assignment operator should take. `#operator(assign)` without a level is `error: binary operator '<<<=' needs #operator(left|right|assign, n)`, and arity 1 or 3 is `error: assignment operator '<<<=' must have arity 2`.

GLOSSO
```
#import "Io";

Counter :: struct #derive(Show) { n: s64; }

(Counter, Counter) ::: instance Add {
    Output :: Counter;
    '+' :: (a: Counter, b: Counter) -> Counter { return .{ .n = a.n + b.n }; }
}

Merge :: typeclass (L: type, R: type) {
    Output :: type;
    '<<<=' :: (left: L, right: R) -> Output #operator(assign, 0);
}

(Counter, Counter) ::: instance Merge {
```

```
    Output :: Counter;
    '<<<=' :: (a: Counter, b: Counter) -> Counter { return .{ .n = a.n * 10 + b.n }; }
}

main :: () {
    c: Counter = .{ .n = 6 };
    d: Counter = .{ .n = 5 };
    c += d;
    print("c += d   -> %\n", show(c));
    c = .{ .n = 6 };
    c <<<= d;
    print("c <<<= d -> %\n", show(c));
}
```

CONSOLE / OUTPUT

```
c += d   -> {11}
c <<<= d -> {65}
```

`+=` here came from the blanket `AddAssign` instance over the program's own `Add`; nothing in the program mentions `AddAssign`. `<<<=` came from `Merge`, and it would work just as well written above the `Merge` declaration, because the prescan registers the spelling for the whole file before parsing begins.

The right-hand side is coerced first

A compound assignment is checked by coercing the right operand to the type of the left one and only then resolving `op=`. Every compound assignment is therefore homogeneous, whatever instances exist. A heterogeneous `Add` gives a working `+` and an unreachable `+=`.

GLOSSO

```
#import "Io";

Counter :: struct #derive(Show) { n: s64; }

(Counter, s64) ::: instance Add {
    Output :: Counter;
    '+' :: (a: Counter, b: s64) -> Counter { return .{ .n = a.n + b }; }
}

(Counter, s64) ::: instance AddAssign {
    Output :: Counter;
    '+=' :: (a: Counter, b: s64) -> Counter { return .{ .n = a.n + b }; }
}

main :: () {
    c: Counter = .{ .n = 6 };
    print("c + 5 -> %\n", show(c + 5));
    c += 5;
}
```

CONSOLE / OUTPUT

```
error: no operator '+=' for Counter, Counter
```

The diagnostic is the proof: the program wrote `c += 5`, and the compiler reports `Counter, Counter`, because the literal `5` was already coerced to the left type before any instance was consulted. Adding the matching `(Counter, s64)` instance of `AddAssign` changes nothing, since the resolution never asks for that pair.

- The fix is to write the expression form: `c = c + 5;` resolves `(Counter, s64) ::: instance Add` and compiles.
- The standard library hits this too. Pointer offsetting is `([*]$T, ssize) ::: instance Add`, so `p += 1` on a `[*]s64` is `error: no operator '+=' for [*]ssize, [*]ssize`. Write `p = p + 1;`.

> **NOTE**
> This is the limitation to remember about the operator system: `+` and `+=` are not interchangeable for mixed operand types. An API that wants both scaled multiplication and `*=` must offer a same-type `Mul` as well as the mixed one.

The result type

A compound assignment stores the operator's result back into the left operand, so `Output` must be assignable to the left type. When it is not, the failure is reported as a missing operator with a note naming the real problem.

CONSOLE / OUTPUT

```
error: no operator '<<<=' for Counter, Counter
note: assign ssize to Counter
```

When the left operand is a subscript, both halves of the indexing protocol run: `a[i] += 1` reads through `'[]'` and stores through `'[]='`. Indexing and Slicing Operators shows the expansion.

9.3 Indexing and Slicing Operators

A subscript expression is an operator call like any other. Reading `c[k]` dispatches on `Indexable`, storing `c[k] = v` dispatches on `IndexSet`, and both classes are declared in the Prelude and available without an import.

GLOSSO

```
Indexable :: typeclass (Container: type, Key: type) {
    Output :: type;
    '[]' :: (container: Container, key: Key) -> Output;
}

IndexSet :: typeclass (Container: type, Key: type, Value: type) {
    '[]=' :: (container: Container, key: Key, value: Value);
}
```

`[]` and `[]=` are the only operator names allowed to contain brackets, and they are special in two further ways: their arities are fixed at 2 and 3, and they may not declare precedence, because the subscript is parsed as syntax rather than as an infix operator. A `#operator(left, level)` directive on either is `error: operator '[]' is special syntax and cannot declare operator precedence`.

Declaring them for a container

`Key` is an arbitrary type. A grid indexed by a coordinate struct needs one `Indexable` instance for reading and one `IndexSet` instance for storing; a third `Indexable` instance keyed on a range type adds slicing.

GLOSSO

```
#import "Io";

Coord :: struct #derive(Show) { row: ssize; col: ssize; }
Grid  :: struct { cells: []mut s64; }

(Grid, Coord) ::: instance Indexable {
    Output :: s64;
    '[]' :: (g: Grid, k: Coord) -> s64 { return g.cells[k.row * 3 + k.col]; }
}

(Grid, Coord, s64) ::: instance IndexSet {
    '[]=' :: (g: Grid, k: Coord, value: s64) { g.cells[k.row * 3 + k.col] = value; }
}

(Grid, Range(ssize)) ::: instance Indexable {
    Output :: []mut s64;
    '[]' :: (g: Grid, k: Range(ssize)) -> []mut s64 { return g.cells[k]; }
}

main :: () {
    storage: [9]s64;
    g: Grid = .{ .cells = storage[..] };
    c: Coord = .{ .row = 1, .col = 2 };
    g[c] = 42;
    print("g[%] = %\n", c, g[c]);
    g[c] += 8;
    print("after += : %\n", g[c]);
    view := g[3..6];
    print("view.count = %, view[2] = %\n", view.count, view[2]);
}
```

CONSOLE / OUTPUT

```
g[{1, 2}] = 42
after += : 50
view.count = 3, view[2] = 50
```

`IndexSet` has no `Output`: a store is a statement, and the requirement returns nothing. A read with no matching instance names the exact tuple that failed to resolve, as in `error: no operator '[]' for Bag, ssize`; a store with no matching instance is the case the compiler still handles badly, described at the end of this chapter.

What a subscript lowers to

`c[k]` becomes the call `'[]'(c, k)`, `c[k] = v` becomes `'[]='(c, k, v)`, and a compound assignment to a subscript becomes both: `c[k] += n` is `'[]='(c, k, '[]'(c, k) + n)`. Printing from the two operator bodies makes the order visible.

GLOSSO

```
#import "Io";

Bag :: struct { cells: []mut s64; }

(Bag, ssize) ::: instance Indexable {
    Output :: s64;
    '[]' :: (b: Bag, k: ssize) -> s64 { print("  get %\n", k); return b.cells[k]; }
}
(Bag, ssize, s64) ::: instance IndexSet {
    '[]=' :: (b: Bag, k: ssize, v: s64) { print("  set % = %\n", k, v); b.cells[k] = v; }
```

```
}

main :: () {
    storage: [4]s64;
    b: Bag = .{ .cells = storage[..] };
    print("b[1] = 7:\n");
    b[1] = 7;
    print("read b[1]:\n");
    x := b[1];
    print("b[1] += 2:\n");
    b[1] += 2;
    print("final %\n", x);
}
```

CONSOLE / OUTPUT

```
b[1] = 7:
  set 1 = 7
read b[1]:
  get 1
b[1] += 2:
  get 1
  set 1 = 9
final 7
```

The quoted names are also callable directly. `'[]'(b, 1)` and `'[]='(b, 1, 9)` are legal expressions that are rewritten into the same subscript nodes, which is occasionally useful inside a macro that is building an index expression by hand.

> **NOTE**
> `'[]='` receives the container by value. A container that owns its elements inline, such as a struct holding a `[4]s64`, will appear to accept a store and then read back the old element, because the instance mutated a copy. Mutation has to travel through an interior pointer or a view: hold a `[]s64` rather than a `[4]s64`.

Index types

The built-in containers coerce their index to `ssize` before resolving. That covers strings, arrays, views and many-pointers, so `a[i]` on a `[]s64` accepts a `u8` or an `s32` index. This conversion must not narrow: a fixed-width `s64` index needs a checked conversion on a 32-bit target. The examples use `ssize` coordinates and range keys to work at either pointer width, while the stored values stay `s64`. A user container does not get that conversion: its key is synthesised as written, given the default type if it is a compile-time number, and matched against the instance heads. With only `(Bag, ssize) ::: instance Indexable` in scope, `b[3]` resolves and `b[i]` with `i: u8` is `error: no operator '[]' for Bag, u8`.

The remedy is either a blanket instance, `(Bag, $K) ::: instance Indexable where Is_Integer(K)`, or a second concrete instance for the key type in question. Instance selection itself is described in How an Instance Is Chosen.

Slicing

Slicing is not a separate feature. A slice expression is a subscript whose key is one of the six range values, so a container supports the shapes it declares `Indexable` instances for and no others. The Prelude supplies all six for `string`, `cstring`, `string16`, `cstring16`, views in every mutability and volatile combination, and dynamic arrays; a fixed array reaches them by coercing to a view.

Written	Key type	Produced by
`c[a..b]`	`Range(ssize)`	`RangeExclusiveOp`
`c[a..=b]`	`RangeInclusive(ssize)`	`RangeInclusiveOp`
`c[a..]`	`RangeFrom(ssize)`	`RangeFromOp`
`c[..b]`	`RangeTo(ssize)`	`RangeToOp`
`c[..=b]`	`RangeToInclusive(ssize)`	`RangeToInclusiveOp`
`c[..]`	`RangeFull(ssize)`	the parser, not an operator

The last row is the odd one. A bare `..` inside brackets is its own syntax node, typed as `RangeFull(ssize)` by the checker; there is no class that produces a `RangeFull` value, and it cannot be written anywhere else. Declaring `Indexable` on `RangeFull(ssize)` is what makes `c[..]` legal for a user container.

GLOSSO

```
#import "Io";

Bag :: struct { cells: []mut s64; }

(Bag, RangeFull(ssize)) ::: instance Indexable {
    Output :: []mut s64;
```

```
    '[]' :: (b: Bag, k: RangeFull(ssize)) -> []mut s64 { return b.cells; }
}

(Bag, RangeFrom(ssize)) ::: instance Indexable {
    Output :: []mut s64;
    '[]' :: (b: Bag, k: RangeFrom(ssize)) -> []mut s64 { return b.cells[k]; }
}

main :: () {
    xs: [4]s64;
    for i: 0 .. 4 xs[i] = i * 10;
    b: Bag = .{ .cells = xs[..] };
    all := b[..];
    tail := b[2..];
    print("b[..].count  = %\n", all.count);
    print("b[2..].count = % first = %\n", tail.count, tail[0]);
}
```

CONSOLE / OUTPUT

```
b[..].count  = 4
b[2..].count = 2 first = 20
```

> **NOTE**
> Reading and writing are resolved independently. A container that is `Indexable` but has no matching `IndexSet` instance is rejected during checking: assigning an `s64` through `(Bag, ssize) ::: Indexable` reports `error: no operator '[]=' for Bag, ssize, s64`. Likewise `s[0] = 65` on a read-only `string` reports no `[]=` for `string, ssize, u8` rather than reaching lowering. Declare `IndexSet`, or do not write the store.

Range values, their fields, and the classes that build them are covered in Iteration and Range Typeclasses; the slicing semantics of the built-in collections are in Ranges and Slicing.

9.4 Iteration and Range Typeclasses

An ordinary `for` over an array, string, `Simd`, matrix, or custom container goes through `Iterable`. Direct and stored ranges keep their native lowering, and tuples are expanded statically. The class has one requirement, a macro: the loop body arrives as a `Code` value and the instance decides what to do with it.

GLOSSO

```
Iterable :: typeclass (T: type) {
    for_expansion :: (val: T, body: Code, reverse: bool) #expand;
}

PointerIterable :: typeclass (T: type) where Iterable(T) {
    for_pointer_expansion :: (val: *mut T, body: Code, reverse: bool) #expand
}
```

`for x: container { ... }` becomes a call to `for_expansion` with the container, the body wrapped as `Code`, and the direction as a literal `true` or `false`. The instance declares the loop variables with the backtick escape so that they land in the caller's scope rather than being renamed by macro hygiene, and inserts the body wherever an iteration belongs.

GLOSSO

```
#import "Io";

Countdown :: struct { from: ssize; }

Countdown ::: instance Iterable {
    for_expansion :: (value: Countdown, body: Code, reverse: bool) #expand {
        c := value;
        cursor: ssize = 0;
        while cursor < c.from {
            step := cursor;
            cursor += 1;
            `it_index := step;
            #if reverse {
                `it := step + 1;
            } else {
                `it := c.from - step;
            }
            #insert body;
        }
    }
}

main :: () {
    c: Countdown = .{ .from = 4 };
    for c print("% (index %)\n", it, it_index);
    for x: c print("named %\n", x);
    for < c print("reversed %\n", it);
}
```

CONSOLE / OUTPUT

```
4 (index 0)
3 (index 1)
2 (index 2)
1 (index 3)
named 4
named 3
named 2
named 1
reversed 1
reversed 2
reversed 3
reversed 4
```

The names the body sees are `it` and `it_index`; the named form `for x: c` is an alias the compiler inserts ahead of the body, so an instance never has to know which spelling the caller used. Keep the expansion's own cursor hygienic, snapshot the public names immediately before insertion, and advance private state first. That prevents a `continue` in the inserted body from repeating an element and prevents a nested expansion's public loop names from corrupting the outer walk.

Loop spelling	Class	Arguments
`for x: c` and `for < c`	`Iterable`	the container by value; `reverse` is `false`, then `true`
`for *x: c`	`PointerIterable`	the container by pointer, plus the same `reverse`

An instance is free to ignore `reverse`, in which case `for <` and `for` iterate identically. The argument arrives as a literal, so `#if reverse` folds at expansion time and an expansion pays nothing for the direction it was not called with. Binding by pointer is a separate class rather than a flag: `for *x: c` on a type with only an `Iterable` instance is `error: 'for *x:' needs a PointerIterable instance for Countdown`. The loop statement itself, including `break`, `continue` and the explicit `for :macro_name` form, is described in `for` Loops and Custom Iteration with `for_expansion`.

The range operators

`..` and `..=` are ordinary user-visible operators with ordinary classes. Five classes cover the five writable shapes, split by fixity: two infix, two prefix, one suffix. The two infix ones sit at precedence level 2, which binds looser than everything except assignment, so `1 .. 1 + 3` is the range `1 .. 4`.

Written	Class	Fixity
`a .. b`	`RangeExclusiveOp`	infix, `#operator(left, 2)`
`a ..= b`	`RangeInclusiveOp`	infix, `#operator(left, 2)`
`a ..`	`RangeFromOp`	`#operator(suffix)`
`.. b`	`RangeToOp`	`#operator(prefix)`
`..= b`	`RangeToInclusiveOp`	`#operator(prefix)`

Each produces a plain struct. All six range types are ordinary generic structs in the Prelude and their fields are readable like any other; `RangeFull($T)` is the one with no operator behind it, produced only by a bare `..` inside a subscript.

Expression	Value type	Fields
`2 .. 5`	`Range(ssize)`	`start`, `end`
`2 ..= 5`	`RangeInclusive(ssize)`	`start`, `end`
`2 ..`	`RangeFrom(ssize)`	`start`
`.. 5`	`RangeTo(ssize)`	`end`
`..= 5`	`RangeToInclusive(ssize)`	`end`
`c[..]`	`RangeFull(ssize)`	none

GLOSSO

```
#import "Io";

name_of :: (value: $T) -> string { return show(T); }

main :: () {
    a := 2 .. 5;
    b := 2 ..= 5;
    c := 2 ..;
    d := ..5;
```

```
    e := ..=5;
    print("2 .. 5  -> %  start=% end=%\n", name_of(a), a.start, a.end);
    print("2 ..= 5 -> %\n", name_of(b));
    print("2 ..    -> %  start=%\n", name_of(c), c.start);
    print("..5     -> %  end=%\n", name_of(d), d.end);
    print("..=5    -> %\n", name_of(e));
}
```

CONSOLE / OUTPUT

```
2 .. 5  -> Range_ssize  start=2 end=5
2 ..= 5 -> RangeInclusive_ssize
2 ..    -> RangeFrom_ssize  start=2
..5     -> RangeTo_ssize  end=5
..=5    -> RangeToInclusive_ssize
```

`Step` and `RangeOrigin`

Nothing above is specific to integers. The Prelude's range-operator instances are blanket instances guarded by two small classes, so a type becomes rangeable by implementing them rather than by implementing `...`

GLOSSO

```
Step :: typeclass (T: type) {
    next :: (value: T) -> T;
    prev :: (value: T) -> T;
}

RangeOrigin :: typeclass (T: type) {
    range_origin :: (#empty: T) -> T;
}

($T, T) ::: instance RangeExclusiveOp where Step(T) {
    Output :: Range(T);
    '..' :: (start: T, end: T) -> Output #magic "range";
}

$T ::: instance RangeToOp where Step(T) where RangeOrigin(T) {
    Output :: RangeTo(T);
    '..' :: (end: T) -> Output #operator(prefix) #magic "range_to";
}
```

`Step` alone unlocks `a .. b`, `a ..= b` and `a ...`. The two open-at-the-start forms `.. b` and `..= b` additionally need `RangeOrigin`, which supplies the value the iteration begins at. To iterate the resulting range the type also needs `PartialOrd`, because the Prelude's `Iterable` instances for the range structs compare `it` against the endpoint.

GLOSSO

```
#import "Io";

Tick :: struct #derive(Show) { n: s64; }

Tick ::: instance Step {
    next :: (value: Tick) -> Tick { return .{ .n = value.n + 1 }; }
    prev :: (value: Tick) -> Tick { return .{ .n = value.n - 1 }; }
}

(Tick, Tick) ::: instance PartialOrd {
    '<=>' :: (a: Tick, b: Tick) -> Ordering {
        if a.n < b.n return Ordering.Less;
        if a.n > b.n return Ordering.Greater;
        return Ordering.Equal;
    }
}

Tick ::: instance RangeOrigin {
    range_origin :: (#empty: Tick) -> Tick { return .{ .n = 0 }; }
}

main :: () {
    a: Tick = .{ .n = 2 };
    b: Tick = .{ .n = 5 };
    for t: a .. b print("tick %\n", show(t));
    for t: ..b print("to %\n", show(t));
}
```

CONSOLE / OUTPUT

```
tick {2}
tick {3}
tick {4}
to {0}
to {1}
to {2}
to {3}
to {4}
```

Without `Step` the operator does not resolve at all: `error: no operator '..' for Tick, Tick`. Without `RangeOrigin` the prefix form fails on its own with `error: no prefix operator '..' for Tick`, while `a .. b` keeps working. Trying to iterate a bare value with no `Iterable` instance is `error: 'Tick' is not iterable; it needs an Iterable instance`.

`ToPattern`

`ToPattern` is what lets a range appear in a `case` arm. It is compile-time only by construction: `to_pattern` takes a `#comptime` value and returns a `Pattern`, and `Pattern` values cannot reach lowering, so an instance is resolved during checking and erased afterwards. The Prelude implements it for all six range shapes over integers, each with an intrinsic body that normalises the shape to a closed interval before the coverage analysis sees it.

GLOSSO

```
ToPattern :: typeclass (T: type) {
    to_pattern :: (#comptime value: T) -> Pattern;
}

Range($T) ::: instance ToPattern where Is_Integer(T) {
    to_pattern :: (#comptime value: Range(T)) -> Pattern #magic "range_to_pattern";
}
```

GLOSSO

```
#import "Io";

classify :: (n: s64) -> string {
    if n == {
        case ..=0;    return "non-positive";
        case 1..=9;   return "small";
        case 10..100; return "medium";
        case 100..;   return "large";
    }
    return "?";
}

main :: () {
    for v: .[-5, 0, 3, 42, 1000] print("% -> %\n", v, classify(v));
}
```

CONSOLE / OUTPUT

```
-5 -> non-positive
0 -> non-positive
3 -> small
42 -> medium
1000 -> large
```

The ranges are exclusive where written exclusive: `10..100` does not cover `100`, which is why the last arm starts there. Because the class is keyed on `Is_Integer`, a user `Step` type does not get range patterns for free. Ranges in Patterns and `ToPattern` covers the matching side, and Exhaustiveness, Guards, and Reachability covers what the normalised intervals are used for.

10 Control flow

10.1 Statements and Semicolons

A statement is the unit a procedure body is built from. The ordinary simple forms are terminated statements: local declarations, assignments and update assignments, expression statements such as a call, `return`, and legacy inline assembly all require `;`. A newline never substitutes for it. Control-flow and other body-ended statements instead finish when their own body finishes.

GLOSSO

```
#import "Io";

main :: () {
    a := 1;
    b := 2;
    c := a + b;
    print("a=% b=% c=%\n", a, b, c);
}
```

CONSOLE / OUTPUT

```
a=1 b=2 c=3
```

Newlines are not terminators

A line break is ordinary whitespace. Glosso is C-like on this point and not Go-like: nothing is inserted at the end of a line, so a statement continues across as many lines as its grammar needs. The consequence is that a token which could continue the previous expression will continue it, however the source is laid out.

GLOSSO

```
#import "Io";

double :: (n: ssize) -> ssize { return n * 2; }

main :: () {
    z := double
    (3);
    print("z=%\n", z);
}
```

CONSOLE / OUTPUT

```
z=6
```

> **NOTE**
> `double` on one line and `(3)` on the next is one call expression, `double(3)`, not a reference to `double` followed by a parenthesised `3`; the required terminator comes after the closing parenthesis. A leading `(`, `[`, `.`, or binary operator on a continuation line joins the line above it, however the source is laid out.

Where a semicolon is mandatory

Required semicolons fall into the simple-statement forms and the delimiter positions below. File scope adds its own declaration rule.

Place	Form
After a switch case label	`case 1;`, `case .Red;`, `case P if guard;`
After a switch `else`	`else;`
After a label statement	`'name;`; the older `'name:` is also accepted
After a local binding	`x := 1;`, `LIMIT :: 64;`, `buffer: [8]u8;`
After assignment or update assignment	`x = 2;`, `total += x;`
After `return`	`return;`, `return value;`, `return a, b;`
After an expression statement or legacy `#asm`	`work();`, `value;`, `#asm "nop";`

Place	Form
After a non-definition top-level declaration	`#import "Io";`, `LIMIT :: 64;`, `Alias :: ssize;`

A `#code` quote provides the main contextual exception. A local declaration, assignment, expression statement, value-bearing `return`, or legacy `#asm` immediately before any `}` while inside the quote may end there. The quote mode includes nested blocks, not just the outer quote; at the outer brace this is what makes `#code { value }` an expression. In an expression-position `#inline while` or `#inline for`, an ordinary expression statement in the generated body may instead end with `,`. Outside a `#code` quotation, ordinary runtime and compile-time blocks receive neither shortcut.

A case label is an ordinary expression, so without the semicolon the parser has no way to know that the label has ended and the arm body has begun. Omitting it reports the first token of the body.

GLOSSO

```
#import "Io";

main :: () {
    n := 1;
    if n == {
    case 1 print("one\n");
    }
}
```

CONSOLE / OUTPUT

```
error: unexpected token 'print'
  --> switch.glo:6:12
   |
6 |     case 1 print("one\n");
   |            ^^^^^
```

`else` without its semicolon reports the same way: `else print("other");` is `error: unexpected token 'print'`. The switch shape is set out in Switching on a Value.

A `'name` token is read as a label statement only when the next token is `;` or `:`. Without either punctuation it begins an ordinary expression statement, whose required terminator is missing; the parser immediately blames the first token of the following statement.

GLOSSO

```
#import "Io";
#import "Jump";

main :: () {
    i := 0;
'here
    i += 1;
    if i < 3 goto('here);
    print("i=%\n", i);
}
```

CONSOLE / OUTPUT

```
error: unexpected token 'i'
  --> label.glo:7:5
   |
7 |     i += 1;
   |     ^
```

NOTE

There is no empty statement. A semicolon is consumed only by the statement in front of it, so a stray one has nothing to attach to. Inside a body, both `;` on its own and the second `;` of `a := 1;;` are `error: unexpected token ';'`. At file scope a stray semicolon is `error: expected a declaration, found ';'`; after a braced definition its help says that the declaration already ended at `}`.

The statement inventory

The parser tests the statement forms in a fixed order and the first match wins. The order matters only where two forms can begin with the same token: a nested declaration is recognised before an expression statement because `name ::` would otherwise start an expression, and a label is recognised before an expression for the same reason.

Form	What it is
`name :: (...)`, `name :: struct`, `name :: enum`, `name :: union`, `name :: distinct`	A declaration local to its containing lexical block.
`{ ... }`	A bare block, which introduces a scope.

Form	What it is
`'name;`	A label.
`return;`, `return e;`, `return a, b;`	Leaves the procedure.
`break` / `break;`, `continue` / `continue;`	Leaves or restarts the innermost loop.
`while cond body`, `for ... body`	The two loops.
`#inline while ...`, `#inline for ...`	The unrolled loops.
`defer stmt`, `#try defer stmt`	Runs a statement at block exit.
`using name` / `using name;`	Opens an aggregate's fields into the scope.
`if ...`	The conditional, the switch, and the pattern switch.
`#if ...`	The static conditional and the static switch.
`#push_context [e] body`, `#push_allocator (e)` [`;`]	Overrides the implicit context or its allocator.
`#comptime e` [`;`], `#comptime { ... }` [`;`], `#comptime -> T { ... }` [`;`]	Compile-time evaluation.
`#assert cond, msg` [`;`], `#compile_error msg` [`;`]	Compile-time checks.
`#insert e` [`;`], `#bytes e` [`;`], `#asm { ... }` [`;`] or legacy `#asm e;`	Splices a generated statement, raw bytes, or assembly.
`case label;`	A case fragment. Legal only inside a `#code` quote.
`x := e;`, `x : T = e;`, `x :: e;`, `a, b := f();`	A variable or constant declaration.
`target = value;`, `target op= value;`	Assignment and update assignment.
`expression;`	An expression statement.

- A nested `X :: (...)`, `struct`, `enum`, `union` or `distinct` inside a procedure body is not a local: it is lifted onto the file's declaration list at parse time and is visible from every procedure in the file. Blocks, Scope, and Shadowing covers what this does and does not scope.
- `return`, `break` and `continue` are also expressions, of the bottom type `#Never`, so they may appear where a value is expected. `ifx` Expressions shows the arms this makes possible.
- `#pattern P := subject else { ... };` is parsed as an expression and reaches statement position inside an expression statement. It is described in Bind or Diverge.

One program that exercises the less common entries follows. Every required terminator is shown, and body-ended forms close with `}`:

GLOSSO

```
#import "Io";
#import "Memory";
#import "Context";

Point :: struct { x: ssize; y: ssize; }

main :: () {
    p: Point = .{ .x = 1, .y = 2 };
    using p;
    print("using % %\n", x, y);

    { inner := 3; print("block %\n", inner); }

    #assert 1 + 1 == 2, "arithmetic still works";
    N :: #comptime -> ssize { total := 0; for i: 1..=4 total += i; return total; };
    print("N %\n", N);

    #insert "generated := 5;";
    print("generated %\n", generated);

    defer print("deferred\n");

    #push_allocator(temp_allocator()) { print("push_allocator\n"); }
    ctx := context;
    #push_context ctx { print("push_context\n"); }

    #if OUT.os == Target_OS.Windows print("static if\n");
}
```

CONSOLE / OUTPUT
```
using 1 2
block 3
N 10
generated 5
push_allocator
push_context
static if
deferred
```

10.2 `if`, `else`, and Truthiness

`if` takes a condition and a body. The condition is not parenthesised, and the body is either a braced block or one statement. Parentheses are accepted around a condition, but only as grouping; they are not part of the form. `else` follows the body, and `else if` chains without a separate keyword.

GLOSSO
```
#import "Io";

main :: () {
    n := 5;

    if n > 3 print("gt3\n");

    if n > 100 print("gt100\n"); else print("le100\n");

    if n == 5 {
        print("eq5\n");
    } else if n == 6 {
        print("eq6\n");
    } else {
        print("other\n");
    }

    if (n > 3) && (n < 10) print("parens are grouping\n");
}
```

CONSOLE / OUTPUT
```
gt3
le100
eq5
parens are grouping
```

An unbraced body is a single statement, which includes a nested `if`. Either spelling gets its own scope, so a declaration made in an unbraced body is no more visible afterwards than one made in a braced one: `if n > 0 x := 5;` followed by a use of `x` is `error: unbound 'x'`.

The dangling else

When an `else` could attach to either of two nested `if` statements, it attaches to the nearest one. The parser reads the inner `if` as the outer one's body, and the `else` it then meets belongs to the statement it is still parsing.

GLOSSO
```
#import "Io";

main :: () {
    n := 5;
    if n > 0
        if n > 100 print("inner\n");
        else print("dangling else went to the inner if\n");
}
```

CONSOLE / OUTPUT
```
dangling else went to the inner if
```

> **NOTE**
> Indentation has no effect on this. An `else` that is meant for the outer `if` has to be forced there by bracing the inner one.

Truthiness

A condition does not have to be `bool`. Branching on a value applies a truthiness rule chosen by the value's representation, and the same rule serves `if` and `while`. The rule is per representation rather than per type, which is why it is total over the scalar kinds and undefined for anything with no distinguished zero.

Kind	Truthy when
`bool`	The value is `true`.
integer, enum	The value is not zero. An enum is tested by discriminant, so its first constant is falsy.
float	The value compares ordered-unequal to zero, so `0.0`, `-0.0` and NaN are all falsy.
pointer, many-pointer, function pointer, `label`	The value is not null.
`?T` whose payload is pointer-shaped	The value is present.
`string`, `cstring`, `string16`, `cstring16`	`.count` is not zero.
`[]T`, `[..]T`	`.count` is not zero.

GLOSSO

```
#import "Io";

Level :: enum { Info; Warn; }

main :: () {
    if 0     print("0 truthy\n");     else print("0 falsy\n");
    if -2    print("-2 truthy\n");    else print("-2 falsy\n");
    if 0.0   print("0.0 truthy\n");   else print("0.0 falsy\n");
    if ""    print("\"\" truthy\n");  else print("\"\" falsy\n");
    if "x"   print("\"x\" truthy\n"); else print("\"x\" falsy\n");

    lv := Level.Info;
    if lv    print("Level.Info truthy\n"); else print("Level.Info falsy\n");

    p: ?*mut ssize = .None;
    if p     print("null ?*mut ssize truthy\n"); else print("null ?*mut ssize falsy\n");

    xs: [3]ssize = .[1, 2, 3];
    empty := xs[0..0];
    if empty print("empty view truthy\n"); else print("empty view falsy\n");
}
```

CONSOLE / OUTPUT

```
0 falsy
-2 truthy
0.0 falsy
"" falsy
"x" truthy
Level.Info falsy
null ?*ssize falsy
empty view falsy
```

`?*T` is truthy exactly when it is present because the optional pointer is stored in the pointer's null niche: `.None` is the null bit pattern, so the presence test and the null test are the same instruction. An optional over a payload with no niche has no such coincidence, and it has no rule at all.

A value whose kind is not in the table is rejected when the branch is lowered. A struct condition is `error: values of type P have no native truthiness rule`, and `o: ?s64` is `error: values of type Option_s64 have no native truthiness rule`. Test a struct against something explicit, and match an optional with `if #pattern .Some(v) = o` or `if o == { case .Some(v); ... }`.

The three conditionals compared

`if` is the only one of the three that accepts a non-`bool` condition. `ifx` coerces its condition to `bool` because it is an expression whose arms have to agree on a type, and `#if` demands a `bool` that is also known at compile time because it selects which branch is checked at all.

Construct	Condition	Evaluated
`if`, `while`	Any kind with a truthiness rule	Run time
`ifx`	Must be `bool`	Run time
`#if`	Must be `bool` and compile-time known	Compile time

`ifx 3 "a" else "b"` is `error: got ssize, expected bool`, and `#if 1` is `error: #if condition must be bool, got comptime_int`. The expression form is covered in `ifx` Expressions and the static form in Static Branching with `#if`.

An `if` whose condition is a subject followed by `== {` is not a conditional at all but a switch, and an `if` whose condition begins with `#pattern` is a pattern test. Both are separate forms of the same keyword; see Switching on a Value and Pattern Matching with `#pattern`.

10.3 Switching on a Value

Glosso has no `switch` keyword. A switch is an `if` whose condition is a subject expression followed immediately by `==` and `{`. The braces hold case arms rather than statements, each arm opening with a `case` label that ends in a semicolon, and the default arm is spelled `else;`.

GLOSSO

```
#import "Io";

classify :: (n: ssize) -> string {
    if n == {
    case ..=0;     return "non-positive";
    case 1, 2, 3;  return "small";
    case 4..=9;    return "single digit";
    case 10..;     return "large";
    }
    return "unreachable";
}

main :: () {
    print("% % % % %\n", classify(-5), classify(0), classify(2), classify(7), classify(99));
}
```

CONSOLE / OUTPUT

```
non-positive non-positive small single digit large
```

Why `== {` is a switch

The rule lives in the binary-operator loop of the expression parser: `==` stops binding as an operator when the token after it is `{`. It is a lexical rule rather than an `if`-specific one, and it costs nothing, because `{` can never begin an expression in Glosso. An aggregate literal is written `.{ ... }`, so a brace where a right operand was expected is always a mistake.

GLOSSO

```
#import "Io";

P :: struct { a: ssize; }

main :: () {
    p: P = { .a = 1 };
    print("%\n", p.a);
}
```

CONSOLE / OUTPUT

```
error: unexpected token '{'
  --> brace.glo:6:12
  |
6 |     p: P = { .a = 1 };
  |            ^
```

Only the `==` directly in front of the brace is affected, so an ordinary comparison followed by a block is unambiguous: `if a == b { ... }` is a conditional, because the token after `==` is `b`. To switch on the result of a comparison, parenthesise it and let the outer `==` be the switch marker: `if (a == b) == { case true; ... }`. Any expression may be the subject, including a call or an arithmetic expression.

The shape

CONSOLE / OUTPUT

```
if [#partial] <subject> == {
case <label> [, <label>]* ;
    <statements>
...
else;
    <statements>
}
```

The semicolon after a label and after `else` is mandatory, and it is mandatory because a label is an ordinary expression: nothing else marks where the label ends and the arm body begins. Omitting it reports the first token of the body, `error: unexpected token 'print'` for `case 1 print("one\n");`. Arm bodies are statement lists, not blocks, and they are not braced.

An arm runs to the next `case`, the `else`, or the closing brace. There is no implicit fall-through and no `break` is needed to prevent it; entering the next arm's body has to be asked for with `#falling`, which is described together with multi-label arms in `case A, B;` Groups and `#falling`.

Case labels by subject type

A label is a literal, a range, an enum constant, or a union variant, depending on what the subject is. Several labels may share one arm by separating them with commas.

Subject	Label spellings
integer	`case 7;`, `case 1, 2, 3;`, `case ..=0;`, `case 4..=9;`, `case 10..;`
float	`case 2.5;`
`string`	`case "red";`, `case "red", "green";`
character	`case #char "q";`
`bool`	`case true;`, `case false;`
enum	`case Level.Warn;`, `case .Warn;`, `case .Warn, .Error;`
tagged union	`case Name;`, `case .Name;`, `case Name(text);`, `case .Name(text);`
`?T`, `E!T`	`case .Some(v);`, `case .None;`, `case .Ok(v);`, `case .Err(e);`
any	`else;`

Range labels go through the `ToPattern` conversion rather than through dedicated grammar, so `4..=9` in a label is the same range expression it would be anywhere else. Its bounds must be known at compile time. A bare `..` is not accepted as a label: `case ..;` is `error: bare '..' needs an index context`. Ranges in Patterns and `ToPattern` covers the conversion.

GLOSSO

```
#import "Io";

Level :: enum { Info; Warn; Error; }
Token :: union { Name: string; Code: s64; Eof: void; }

main :: () {
    lv := Level.Warn;
    if #partial lv == { case Level.Info; print("qualified Info\n"); case Level.Warn; print("qualified Warn\n"); }
    if lv == { case .Info; print("dotted Info\n"); case .Warn, .Error; print("dotted Warn or Error\n"); }

    t: Token = .Code(42);
    if t == {
    case Name(text); print("bare Name %\n", text);
    case Code(v);    print("bare Code %\n", v);
    case Eof;        print("bare Eof\n");
    }

    e: Token = .Eof;
    if e == {
    case .Name(text);   print("dotted Name %\n", text);
    case .Code(v);      print("dotted Code %\n", v);
    case .Eof(nothing); print("dotted Eof, payload is the empty tuple: %\n", nothing == .());
    }
}
```

CONSOLE / OUTPUT

```
qualified Warn
dotted Warn or Error
bare Code 42
dotted Eof, payload is the empty tuple: 1
```

NOTE

An enum constant may be written qualified or with a leading dot, but a union variant may not be qualified here: `case Token.Name(s);` is `error: '' is not a variant of union Token`. Use `Name(s)` or `.Name(s)`. The qualified spelling is accepted in a `#pattern` switch, which is a different form.

Nothing matched

A #partial statement switch can leave values unhandled. If no arm matches and there is no else, execution continues after that switch. Without #partial, the compiler requires exhaustive coverage or an else arm. An ifx expression must always produce a value.

GLOSSO

```
#import "Io";

main :: () {
    n := 7;
    if #partial n == { case 1; print("one\n"); case 2; print("two\n"); }
```

```
    print("fell out of the ssize switch\n");
}
```

CONSOLE / OUTPUT

```
fell out of the ssize switch
```

Scalar and enum statement switches are coverage-checked by default. A gap reports non-exhaustive switch with a missing-case witness. Cover every value, add else, or use #partial for intentional fall-through. Tagged-union switches likewise require all variants, else, or #partial. Covered arms can receive an unreachable-pattern warning.

`#partial` waives that requirement and is written immediately after `if`, before the subject and before a `#pattern` marker. Elsewhere it is rejected: `if #partial true { }` is `error: #partial can only modify an if-switch`, and `if #pattern #partial s == { ... }` is `error: unexpected directive 'partial'`.

GLOSSO

```
#import "Io";

Token :: union { Name: string; Code: s64; Eof: void; }

main :: () {
    t: Token = .Code(7);
    if #partial t == { case .Code(v); print("partial code %\n", v); }
}
```

CONSOLE / OUTPUT

```
partial code 7
```

> **NOTE**
> A union payload is reachable only through a capture in the arm that proved the tag. Reading `t.Code` directly is `error: tagged-union payload '.Code' may only be bound by a pattern match`. Reading a Union Payload gives the rule and the escape hatch, and a raw union has no tag to switch on at all: `error: cannot switch on a raw union (it has no tag)`.

One arm may not capture from several tags whose payloads differ: `case .Name(x), .Code(x);` is `error: a case with several tags cannot capture a payload - the tags have different payload types`. Grouping tags that share a body is covered in `case A, B;` Groups and `#falling`, and the coverage analysis itself in Exhaustiveness, Guards, and Reachability.

10.4 `ifx` Expressions

`ifx` is the expression conditional. It exists because `if` is a statement and produces nothing, and because a value that depends on a test should not have to be built by declaring a variable and assigning to it from two branches. Every `ifx` must produce a value on every path, which is the single rule the rest of its behaviour follows from.

Form	Shape
conditional	`ifx cond then-expr else else-expr`
value switch	`ifx subject == { case L; expr; ... else; expr; }`
pattern switch	`ifx #pattern [subject] == { case P; expr; ... }`

GLOSSO

```
#import "Io";

Color :: enum { Red; Green; Blue; }
Shape :: union { Circle: f64; Rect: f64; Empty: void; }

main :: () {
    enabled := true;
    print("%\n", ifx enabled "on" else "off");

    n := 5;
    print("%\n", ifx n > 10 "big" else ifx n > 3 "mid" else "small");

    score := 85;
    print("%\n", ifx score == { case 90; "A"; case 85; "B"; else; "C"; });

    c := Color.Blue;
    print("%\n", ifx c == { case .Red; "red"; case .Green; "green"; case .Blue; "blue"; });

    s: Shape = .Rect(3.0);
    print("%\n", ifx #pattern s == {
    case Shape.Circle(r); 3.0 * r * r;
    case Shape.Rect(w);   w * w;
```

```
    case Shape.Empty;     0.0;
    });

    print("%\n", ifx #pattern == { case n > 3; "gt"; else; "le"; });
}
```

CONSOLE / OUTPUT

```
on
mid
B
blue
9
gt
```

The switch forms share the subject and label grammar of the statement switch, including the mandatory semicolon after each label and after `else`. The subject-less pattern form takes arbitrary boolean predicates as its labels. `ifx` does not accept `#partial`: `ifx #partial c == { ... }` is `error: unexpected directive 'partial'`, since waiving coverage would leave the expression with no value to produce.

The `else` is mandatory on the conditional form

A two-armed test with one arm missing has nothing to yield when the condition is false, so the omission is an error rather than a `void` result.

CONSOLE / OUTPUT

```
error: cannot decide a value: an 'ifx' needs an 'else' arm
  --> decide.glo:5:21
  |
5 |     x := ifx b "yes";
  |                     ^
note: add 'else <expr>', or use the switch form 'ifx <subject> == { ... }'
```

Arms end in an expression

An arm of a switch form is a statement list, so it may compute before it answers, but its last statement has to be an expression. That expression is the arm's value.

GLOSSO

```
#import "Io";

main :: () {
    score := 85;
    w := ifx score == { case 85; print("computing\n"); score * 2; else; 0; };
    print("w %\n", w);
}
```

CONSOLE / OUTPUT

```
computing
w 170
```

An arm ending in something that is not an expression is `error: an ifx arm must end in an expression`, reported at the offending statement. An arm that ends in a `void` call is subtler: the arm's type becomes `void`, and the mismatch surfaces on the next arm instead, as `error: got ssize, expected void`.

The arms are expressions, not blocks. `ifx b { 1 } else { 2 }` does not parse: a brace never begins an expression, so this is `error: unexpected token '{'`.

The join across arms

The arms meet at a numeric join rather than at the first arm's type. An integer literal beside a float arm becomes the float type, and a sized integer or a narrower float on one side pulls a literal on the other side to it. This is the same defaulting the rest of the language uses, applied to the set of arms.

GLOSSO

```
#import "Io";
#import "Meta";

main :: () {
    b := true;
    n: s32 = 4;
    a := ifx b 1 else 2.5;
    c := ifx b n else 7;
    d := ifx b cast(f32)1.0 else 2.0;
    print("% % %\n", type_name(type_of(a)), type_name(type_of(c)), type_name(type_of(d)));
    print("% % %\n", a, c, d);
}
```

CONSOLE / OUTPUT
```
f64 s32 f32
1 4 1
```

Arms with no join are an error reported against the arm that does not fit: `ifx b 1 else "two"` is `error: got ssize, expected string`.

Totality

A switch form must be provably total or carry `else;`. Totality is computed, not demanded: an enum or union subject whose arms name every constructor needs no `else`, which is why the `Color` example above ends at `case .Blue;`. When the proof fails the compiler names the witnesses it found.

GLOSSO
```
#import "Io";

Color :: enum { Red; Green; Blue; }

main :: () {
    c := Color.Red;
    name := ifx c == { case .Red; "red"; case .Green; "green"; };
    print("%\n", name);
}
```

CONSOLE / OUTPUT
```
error: cannot decide a value for this ifx
  --> total.glo:7:13
  |
7 |     name := ifx c == { case .Red; "red"; case .Green; "green"; };
  |             ^^^
note: missing case: Color.Blue
note: cover every case, or add an 'else' arm
```

Subjects whose domain cannot be enumerated are never total on literals alone. An `f64` or `string` subject with only literal labels reports `note: missing case: _`, and the subject-less predicate form reports the same witness under `error: non-exhaustive #pattern switch`, because an arbitrary boolean predicate proves nothing about coverage. The analysis behind the witnesses is described in Exhaustiveness, Guards, and Reachability.

When the subject is known at compile time and no arm matches it, the failure is escalated rather than left to run time, because the panic would be unconditional.

CONSOLE / OUTPUT
```
error: no arm matches the comptime value Int(5), and there is no else
  --> known.glo:5:10
  |
5 |     x := ifx N == { case 1; "one"; case 2; "two"; };
  |          ^^^
note: the subject is known at compile time, so this would panic on every evaluation
```

The lowerer still emits a backstop on the no-arm path, reporting `ifx matched no arm and has no else` through the assertion machinery. A correct totality proof makes it unreachable; it exists so that a gap in the analysis fails loudly rather than yielding an undefined value.

Diverging arms

`return`, `break` and `continue` are expressions of type `#Never`, the bottom type, which coerces to every type because the coercion can never run. An `ifx` arm may therefore leave instead of producing a value, which is how a loop body filters and binds in one step.

GLOSSO
```
#import "Io";

main :: () {
    xs: [5]s64 = .[1, 3, 4, 6, 7];
    total := 0;
    for v: xs {
        w := ifx v < 6 v else break;
        total += w;
    }
    print("total=%\n", total);

    evens := 0;
    for v: xs {
        w := ifx v % 2 == 0 v else continue;
        evens += w;
    }
    print("evens=%\n", evens);
}
```

CONSOLE / OUTPUT

```
total=8
evens=10
```

> **NOTE**
> The join takes the other arm's type first, so a bare numeric literal beside a diverging arm is checked against `#Never` and fails: `x := ifx b 1 else return -1;` is `error: got ssize, expected #Never`. Naming the value first (`ifx b y else return -1`) or writing the `ifx` directly in a `return` both work, because the expected type is then known from context.

10.5 Pattern Matching with `#pattern`

`#pattern` introduces the pattern language. A pattern is one construct that does two things at once: it tests a value's shape and it binds names to the parts it found. That pairing is why patterns are the only way to read a tagged union's payload, since the binding cannot happen without the test having succeeded.

This chapter covers where a pattern may appear and the two rules that govern all of the positions. The patterns themselves are catalogued in The Pattern Vocabulary, ranges in Ranges in Patterns and `ToPattern`, and coverage in Exhaustiveness, Guards, and Reachability.

Form	Position	Meaning
`if #pattern P = subject { ... }`	condition	One-shot test; captures reach the then-branch.
`if #pattern { P1 = s1; P2 = s2; }`	condition	Multi-arm test; all arms must match; captures join.
`while #pattern P = subject { ... }`	condition	Re-tested and re-bound each iteration.
`if [#partial] #pattern [subject] == { case P; ... }`	statement	Pattern switch.
`ifx #pattern [subject] == { case P; expr; ... }`	expression	Pattern switch that yields a value.
`#pattern P := subject else { ... };`	statement	Bind into the enclosing scope or diverge.
`#pattern P`	expression	A quote: a compile-time value of type `Pattern`.

The parser tells these apart by scanning ahead from the pattern for a depth-zero `=` or `:=`. A depth-zero `=` makes it a test, a `:=` makes it the bind-or-diverge form, and neither makes it a quote. `==` is a different token, so a `== CONST` element inside a pattern is never mistaken for the test marker, and a `{` found at depth zero ends the scan because that brace is the enclosing `if`'s body.

GLOSSO

```
#import "Io";

Shape :: union { Circle: f64; Rect: f64; Empty: void; }

port_of :: (o: ?s64) -> s64 {
    #pattern .Some(v) := o else { return -1; }
    return v;
}

main :: () {
    s: Shape = .Circle(2.0);

    if #pattern Shape.Circle(r) = s  print("test: r = %\n", r);

    if #pattern Shape.Rect(w) = s { print("rect %\n", w); }
    else                          { print("test with else: not a rect\n"); }

    o: ?s64 = .Some(3);
    while #pattern .Some(v) = o {
        print("loop: v = %\n", v);
        if v <= 1 { o = .None; } else { o = .Some(v - 1); }
    }

    if #pattern s == {
    case Shape.Circle(r); print("switch: circle %\n", r);
    case Shape.Rect(w);   print("switch: rect %\n", w);
    case Shape.Empty;     print("switch: empty\n");
    }
```

```
    print("bind or diverge: % %\n", port_of(.Some(8)), port_of(.None));

    CIRCLE :: #pattern Shape.Circle(r);
    if #pattern #insert CIRCLE = s  print("quote: r = %\n", r);
}
```

CONSOLE / OUTPUT

```
test: r = 2
test with else: not a rect
loop: v = 3
loop: v = 2
loop: v = 1
switch: circle 2
bind or diverge: 8 -1
quote: r = 2
```

The multi-arm block form tests several subjects at once and succeeds only when every arm matches; the captures of all arms are then in scope together, and a later arm may read an earlier arm's capture.

GLOSSO

```
#import "Io";

Pair :: struct { first: s64; second: s64; }

main :: () {
    p: Pair = .{ .first = 4, .second = 9 };
    q: Pair = .{ .first = 4, .second = 1 };
    if #pattern { Pair.{ .first = a, ... } = p; Pair.{ .first = == a, ... } = q; }
        print("both start at %\n", a);
    else
        print("they differ\n");
}
```

CONSOLE / OUTPUT

```
both start at 4
```

A pattern test is an expression of type `bool`, and the subject side is parsed at a precedence above `&&` and `||`, so `if #pattern P = subject && g` reads as `(#pattern P = subject) && g` rather than testing `P` against `subject && g`. The extra condition then sees the captures.

GLOSSO

```
#import "Io";

Shape :: union { Circle: f64; Rect: f64; Empty: void; }

main :: () {
    s: Shape = .Rect(3.0);
    if #pattern .Rect(w) = s && w > 1.0   print("wide rect %\n", w);
    if #pattern .Rect(w) = s && w > 100.0 print("huge\n"); else print("not huge\n");
}
```

CONSOLE / OUTPUT

```
wide rect 3
not huge
```

> **NOTE**
> Do not put `||` between two pattern tests. It compiles, but `||` short circuits, so the untaken alternative's captures are in scope and never written; the arm reads uninitialised storage. `|` inside a single pattern is the supported spelling for alternatives, and it forces every alternative to bind the same names at the same types.

A test cannot be negated. `!(#pattern P = subject)` is `error: a '#pattern' test cannot be negated`, with `note: its captures would have no scope to bind into - '!' inverts the value, not the branch they belong to` and a pointer to the `:=` form. Bind or Diverge covers that form.

The quote

`#pattern P` with no subject is not a test but a quote: a value of the built-in type `Pattern`. The type is compile-time only and has no storage, so a quote is bound with `::` and spliced back into pattern position with `#insert`. Binding one with `:=` is `error: a 'Pattern' value is compile-time only; bind it with '::' rather than ':='`. Patterns as Compile-Time Values covers the introspection intrinsics built on this.

Tag-first lowering

A variant pattern lowers to a tag comparison and a branch to the failure label. The payload is only extracted after that branch, so a pattern that does not match never forms the payload it was going to bind. The reason is that a union's storage read as the wrong variant is not a value of that variant's type at all; forming it first and testing afterwards would mean building an ill-typed value on every miss.

The consequence a program can observe is that a subpattern under a variant is only ever applied to that variant's payload. An interval test nested inside `Value.Number(...)` never runs against a `Value.Text` payload, even though the two share storage.

GLOSSO

```
#import "Io";

Value :: union { Number: s64; Text: string; Nothing: void; }

describe :: (v: Value) -> string {
    if #pattern Value.Number(0..=9) = v  return "small number";
    if #pattern Value.Number(n) = v      return "big number";
    if #pattern Value.Text(t) = v        return t;
    return "nothing";
}

main :: () {
    print("%\n", describe(Value.Number(3)));
    print("%\n", describe(Value.Number(400)));
    print("%\n", describe(Value.Text("hello")));
    print("%\n", describe(Value.Nothing()));
}
```

CONSOLE / OUTPUT

```
small number
big number
hello
nothing
```

The same ordering discipline runs through the other composite patterns. A slice pattern emits its length test before any index is formed, so an element is never read out of a slice too short to hold it, and a struct pattern's fields are tested left to right with each failure jumping straight to the pattern's failure label.

This is also the mechanism behind the rule that a tagged-union payload may only be reached through a capture. A direct `value.Variant` read has no tag test in front of it, so there is nothing to make it well typed: `error: tagged-union payload '.Code' may only be bound by a pattern match`. Reading a Union Payload has the full rule.

Captures bind in the block that proved the match

A capture is written where the whole pattern succeeded, not at the textual point of the binder. A binder that appears early in a pattern therefore has no effect if a later part of the same pattern rejects, and an outer name of the same spelling survives untouched.

GLOSSO

```
#import "Io";

Pair :: struct { first: s64; second: s64; }

main :: () {
    d := 7;
    p: Pair = .{ .first = 5, .second = 9 };
    if #pattern Pair.{ .first = d, .second = == 0 } = p { print("matched, d = %\n", d); }
    else                                                { print("rejected, d = %\n", d); }
    print("after, d = %\n", d);

    q: Pair = .{ .first = 4, .second = 4 };
    if #pattern Pair.{ .first = a, .second = == a } = q  print("both fields are %\n", a);
}
```

CONSOLE / OUTPUT

```
rejected, d = 7
after, d = 7
both fields are 4
```

The second half of that program shows the compensating rule: within one pattern, a later element may refer to an earlier capture, because the register holding the matched part is computed on the straight-line path before any sibling test runs. What is deferred is the binding the surrounding block sees, not the value the pattern is working with.

- Captures do not escape the construct that proved them. Using a capture after the `if` that bound it is `error: unbound 'r'`.
- Alternatives inside one pattern funnel their captures into a shared set of registers, so an arm body reads the alternative that actually matched rather than the last one written.

- A `void` payload still binds a name; the name has a type and no storage, and it compares equal to the empty tuple `.()`.
- `#pattern P := subject else { ... };` is the exception that binds into the enclosing scope. It is sound only because the `else` is required to diverge, so the binding is unreachable on the failing path.

10.6 The Pattern Vocabulary

A pattern is a shape a value is tested against, together with the names the test introduces when it succeeds. Every shape below composes with every other to any depth: a variant capture may hold a struct pattern whose field pattern is a range, and an or-pattern may join two of those. The positions in which a pattern may be written are described in the chapter on pattern matching with `#pattern`; this chapter is the vocabulary those positions accept.

Two rules govern how a pattern is read. A bare identifier always binds, so `LIMIT` in pattern position names the matched value rather than comparing against the constant `LIMIT`; the `==` form exists for the comparison. And an element that is not pattern syntax is an ordinary expression, evaluated at compile time and converted to a pattern, which is how ranges and spliced quotes enter without new grammar.

Shape	Spelling	Matches
literal	`3`, `2.5`, `"hi"`, `true`	a value equal to it
binding	`name`	anything; names the matched value
wildcard	`_`	anything; binds nothing
pointer binding	`*name`	anything; names a pointer to the place
enum constant	`Color.Green`, `.Green`	that discriminant
variant, tag only	`Shape.Circle`, `.Circle`	that tag, payload discarded
variant capture	`.Circle(r)`	that tag; `r` names the payload
struct, named	`Pair.{ .first = a, .second = b }`	every field pattern matching
struct, shorthand	`Pair.{ .second, ... }`	binds `second`, ignores the rest
struct, positional	`.{ a, b }`	fields in declaration order
slice	`.[head, ..., tail]`	an array, view or dynamic array of a matching length
or-pattern	`P | Q`	either alternative
`@` binding	`whole @ P`	`P`, and names the whole value too
constant	`== LIMIT`	a value equal to the constant
range	`0..=9`, `100..`	an integer or enum in the interval

Literals, bindings, and the wildcard

An integer, float, string, `#char` or boolean literal matches a value equal to it. A bare identifier matches anything and binds it. `_` matches anything and binds nothing, which is what it is for: a position that must be present but is not read. `*name` also matches anything, but binds a pointer to the place the value occupies rather than a copy of it, so an assignment through it writes back into the subject.

```
GLOSSO
#import "Io";

Pair :: struct { first: s64; second: s64; }

main :: () {
    n := 3; f := 2.5; s := "hi"; b := true;
    if #pattern 3 = n     print("ssize literal\n");
    if #pattern 2.5 = f   print("float literal\n");
    if #pattern "hi" = s  print("string literal\n");
    if #pattern true = b  print("bool literal\n");

    if #pattern v = n  print("binding v = %\n", v);
    if #pattern _ = n  print("wildcard\n");

    p: Pair = .{ .first = 1, .second = 2 };
    if #pattern Pair.{ .first = *fp, ... } = p  fp.* = 42;
    print("through the pointer, first = %\n", p.first);
}
```

CONSOLE / OUTPUT

```
ssize literal
float literal
string literal
bool literal
binding v = 3
wildcard
through the pointer, first = 42
```

Variants and enum constants

A tagged-union variant is written qualified (`Token.Name`) or with a leading dot (`.Name`); both spellings are accepted everywhere a pattern is. Parentheses after the tag hold the payload pattern, which is usually a binding. An enum constant takes the same two spellings and never carries a payload. A variant written without parentheses proves the tag and discards the payload.

GLOSSO

```
#import "Io";

Token :: union { Number: f64; Name: string; Eof: void; }
Color :: enum { Red; Green; Blue; }

main :: () {
    t: Token = .Name("total");
    if #pattern Token.Name(text) = t  print("qualified: %\n", text);
    if #pattern .Name(text) = t       print("leading dot: %\n", text);

    e: Token = .Eof;
    if #pattern Token.Eof = e            print("tag only\n");
    if #pattern Token.Eof(nothing) = e   print("void capture: %\n", nothing == .());

    c := Color.Green;
    if #pattern Color.Green = c  print("enum qualified\n");
    if #pattern .Green = c       print("enum leading dot\n");
    if #pattern .Blue = c        print("unreached\n"); else print("enum mismatch\n");
}
```

CONSOLE / OUTPUT

```
qualified: total
leading dot: total
tag only
void capture: 1
enum qualified
enum leading dot
enum mismatch
```

A `void` payload still admits a capture: `Token.Eof(nothing)` binds the empty tuple, so `nothing == .()` is true. `?T` and `E!T` are ordinary tagged unions to the checker, so `.Some`, `.None`, `.Ok` and `.Err` are variant patterns like any other; the two types have their own chapters.

Struct and slice patterns

A struct pattern is written with the aggregate-literal punctuation: `Pair.{ ... }` names the type, `.{ ... }` infers it. Fields may be given by name (`.first = a`), by shorthand (`.second`, which binds a name equal to the field name), or positionally in declaration order. `...` as the last element states that the fields not mentioned are ignored; it is documentation rather than a requirement, because an unmentioned field is treated as a wildcard in any case, both when matching and in the coverage analysis. A positional pattern may likewise stop short and bind a prefix of the fields. A slice pattern uses `.[ ... ]` and matches a fixed array, a view or a dynamic array; `...` inside it stands for any number of skipped elements and may appear at most once, so the compiler can fix each named element's offset from one end or the other.

GLOSSO

```
#import "Io";

Pair :: struct { first: s64; second: s64; }

main :: () {
    p: Pair = .{ .first = 4, .second = 9 };
    if #pattern Pair.{ .first = a, .second = b } = p  print("named % %\n", a, b);
    if #pattern Pair.{ .second, ... } = p             print("shorthand %\n", second);
    if #pattern .{ x, y } = p                         print("positional % %\n", x, y);

    items: [5]s64 = .[10, 20, 30, 40, 50];
    if #pattern .[head, ..., tail] = items[..]  print("head % tail %\n", head, tail);
    if #pattern .[a2, b2, ...] = items[..]      print("prefix % %\n", a2, b2);
    two: [2]s64 = .[7, 8];
    if #pattern .[u, v] = two[..]  print("exact % %\n", u, v);
    if #pattern .[u2] = two[..]    print("unreached\n"); else print("length mismatch\n");
}
```

CONSOLE / OUTPUT

```
named 4 9
shorthand 9
positional 4 9
head 10 tail 50
prefix 10 20
exact 7 8
length mismatch
```

A second `...` is rejected: `.[a, ..., b, ..., c]` is `error: a slice pattern may use '...' at most once`, because two gaps of unknown width leave the elements between them at no fixed offset. A slice pattern applied to a non-array is `error: slice pattern needs an array or slice value, got ssize`, and a struct pattern applied to a non-struct is `error: struct pattern needs a struct value, got ssize`.

Or-patterns, `@`, and constant patterns

`P | Q` matches when either alternative matches, and the alternatives must bind the same names at the same types, because the block that runs afterwards cannot know which alternative proved the match. `whole @ P` matches `P` and additionally binds `whole` to the value as a whole, which is how a pattern can destructure and still keep the original. `== expr` compares the matched value against `expr` instead of binding; `expr` is an ordinary expression, so it may be a named constant or an earlier capture from the same pattern.

GLOSSO

```
#import "Io";

Shape :: union { Circle: f64; Rect: f64; Empty: void; }
Pair  :: struct { first: s64; second: s64; }
LIMIT :: 255;

main :: () {
    s: Shape = .Rect(3.0);
    if #pattern Shape.Circle(r) | Shape.Rect(r) = s  print("or-pattern r = %\n", r);

    q: Pair = .{ .first = 255, .second = 1 };
    if #pattern Pair.{ .first = == LIMIT, ... } = q  print("first is at LIMIT\n");
    if #pattern Pair.{ .first = a, .second = == a } = q  print("fields agree\n");
    else print("fields differ\n");

    if #pattern whole @ Pair.{ .first, ... } = q
        print("first = %  whole.second = %\n", first, whole.second);
}
```

CONSOLE / OUTPUT

```
or-pattern r = 3
first is at LIMIT
fields differ
first = 255  whole.second = 1
```

The three ways an or-pattern's alternatives can disagree each have their own diagnostic: `error: or-pattern binds 'r' as f64 here but as ssize in another alternative`, `error: or-pattern alternative binds 'q', which the other alternatives do not`, and `error: or-pattern alternative does not bind 'r', which another alternative does`.

Guards and compound conditions

A case in a `#pattern` switch may carry a guard: `case P if <cond>;` matches when `P` matches and `cond` is then true, and a failing guard tries the next case rather than leaving the switch. That is the behaviour an `if` nested inside the arm body cannot express. The guard is checked with the full condition grammar, so it sees the arm's captures and may itself contain a `#pattern` test that binds more names. Outside a switch, `&&` composes a test with an ordinary condition in the same way.

GLOSSO

```
#import "Io";

Shape :: union { Circle: f64; Rect: f64; Empty: void; }
Size  :: union { Big: f64; Small: void; }

classify :: (r: f64) -> Size {
    if r > 10.0 return Size.Big(r);
    return Size.Small();
}

describe :: (s: Shape) -> string {
    if #pattern s == {
    case Shape.Circle(r) if r > 100.0;                              return "huge circle";
    case Shape.Circle(r) if #pattern Size.Big(scale) = classify(r);
        print("  scale %\n", scale);
        return "big circle";
    case Shape.Circle(r);  return "small circle";
    case Shape.Rect(w);    return "rect";
    case Shape.Empty;      return "empty";
    }
```

```
}

main :: () {
    print("%\n", describe(Shape.Circle(500.0)));
    print("%\n", describe(Shape.Circle(50.0)));
    print("%\n", describe(Shape.Circle(1.0)));
    r: Shape = .Rect(3.0);
    print("%\n", describe(r));
    if #pattern .Rect(w) = r && w > 1.0  print("wide rect %\n", w);
}
```

CONSOLE / OUTPUT

```
huge circle
  scale 50
big circle
small circle
rect
wide rect 3
```

A non-bool guard is `error: case guard must be bool, got f64`. A guarded arm contributes nothing to the coverage analysis, since the guard can fail at run time; the chapter on exhaustiveness, guards, and reachability has the consequences.

NOTE

`||` between two pattern tests compiles but is unsound. `||` short-circuits, so the captures of the alternative that was not evaluated are in scope and were never written: `if #pattern .X(p) = u || #pattern .Y(q) = w print("% %\n", p, q);` prints `1 0`, where the `0` is an uninitialised capture. Use a switch or nested tests instead.

NOTE

A pattern test cannot be negated. `if !(#pattern .Some(v) = o)` is `error: a '#pattern' test cannot be negated`, with `note: its captures would have no scope to bind into - '!' inverts the value, not the branch they belong to` and a second note pointing at the bind-or-diverge form, which is the construct for leaving on failure.

10.7 Ranges in Patterns and `ToPattern`

`..` and `..=` are operators, not pattern syntax. A case or pattern element that uses one is therefore parsed as an ordinary expression and converted into a pattern afterwards, through the `ToPattern` typeclass. That is why `0..=9`, `100..` and `..` work in pattern position without a single new production, and why the same conversion admits user types.

The consequence to keep in mind is that a bound is an expression, so a bare name in a bound is a constant to look up, not a capture to introduce. This is the one place inside a pattern where an identifier does not bind. The range types themselves are described in the chapter on ranges and slicing.

Spelling	Interval matched	Prelude type
`a..b`	`a` through `b - 1`	`Range(T)`
`a..=b`	`a` through `b`	`RangeInclusive(T)`
`a..`	`a` through the domain maximum	`RangeFrom(T)`
`..b`	the domain minimum through `b - 1`	`RangeTo(T)`
`..=b`	the domain minimum through `b`	`RangeToInclusive(T)`
`..`	the whole domain	`RangeFull(T)`

Every shape normalises to a closed interval, and the interval is then clamped to the subject's own domain. Both steps exist for the coverage analysis: it never has to reason about a half-open interval, and an open end needs no separate intrinsic for the type's extreme value, because the widest interval plus a clamp says exactly what `100..=max` would say. Clamping is also why a witness over a `u8` subject reads `10..=99` rather than a signed 64-bit span.

GLOSSO

```
#import "Io";

report :: (n: s64) {
    print("n = %:", n);
    if #pattern 0..3 = n   print(" 0..3");
    if #pattern 0..=3 = n  print(" 0..=3");
    if #pattern 5.. = n    print(" 5..");
    if #pattern ..3 = n    print(" ..3");
    if #pattern ..=3 = n   print(" ..=3");
    if #pattern .. = n     print(" ..");
    print("\n");
```

```
}

main :: () {
    report(-1);
    report(0);
    report(3);
    report(7);
}
```

CONSOLE / OUTPUT

```
n = -1: ..3 ..=3 ..
n = 0: 0..3 0..=3 ..3 ..=3 ..
n = 3: 0..=3 ..=3 ..
n = 7: 5.. ..
```

In a switch the same spellings are case labels. Because each one is a closed interval over an enumerable domain, a set of ranges that tiles the subject's type is proved total and needs no `else`.

GLOSSO

```
#import "Io";

grade :: (score: s64) -> string {
    if #pattern score == {
    case 90..;     return "A";
    case 80..=89;  return "B";
    case 70..=79;  return "C";
    case ..=69;    return "F";
    }
}

main :: () {
    print("% % % %\n", grade(95), grade(83), grade(70), grade(12));
}
```

CONSOLE / OUTPUT

```
A B C F
```

Subjects and bounds

A bound must be known at compile time, because the interval is baked into the pattern the analysis reasons over. A runtime local as a bound is `error: a range bound must be known at compile time: unknown const hi`, naming the offending identifier. A non-integer bound is `error: a range bound must be an integer`, which is what a float range such as `0.0..=9.0` produces: the Prelude instances are all constrained `where Is_Integer(T)`.

The subject must be an integer or an enum. Anything else is `error: a range pattern needs an integer subject, got string`. Enum subjects work because a discriminant is an integer underneath. ASCII `#char` literals default to `u8`, so byte ranges work directly. `rune` is not an integer: a rune subject reports `error: a range pattern needs an integer subject, got rune`. Cast to an integer explicitly when the intended domain is scalar numbers.

GLOSSO

```
#import "Io";

Color :: enum { Red; Green; Blue; Violet; }

main :: () {
    c := Color.Green;
    if #pattern Color.Red..=Color.Blue = c  print("green is in the first three\n");
    v := Color.Violet;
    if #pattern Color.Red..=Color.Blue = v  print("unreached\n");
    else print("violet is outside\n");

    b: u8 = cast(u8)(#char "q");
    if #pattern cast(u8)(#char "a") ..= cast(u8)(#char "z") = b
        print("lower-case byte\n");
}
```

CONSOLE / OUTPUT

```
green is in the first three
violet is outside
lower-case byte
```

The `ToPattern` typeclass

`ToPattern` is a one-method class declared in the Prelude. Its method takes its argument `#comptime`, since the whole point is to produce a pattern during checking, and returns the compile-time-only type `Pattern`.

```
GLOSSO
ToPattern :: typeclass (T: type) {
    to_pattern :: (#comptime value: T) -> Pattern;
}

Range($T) ::: instance ToPattern where Is_Integer(T) {
    to_pattern :: (#comptime value: Range(T)) -> Pattern #magic "range_to_pattern";
}

RangeInclusive($T) ::: instance ToPattern where Is_Integer(T) {
    to_pattern :: (#comptime value: RangeInclusive(T)) -> Pattern #magic "range_inclusive_to_pattern";
}
```

`RangeFrom`, `RangeTo`, `RangeToInclusive` and `RangeFull` follow the same shape, each with its own intrinsic and each constrained `where Is_Integer(T)`. Those six instances are the specification of what a range means as a pattern. In practice the checker recognises the six built-in shapes from their syntax and builds the interval directly, because only the two bounds matter and they are plain integers; a user type is not special-cased away, and reaches its instance through an actual synthesized call to `to_pattern`.

Writing an instance

A user type joins the pattern language by declaring a `ToPattern` instance whose method returns a pattern quote. `#insert` in pattern position is the splice, both for a value that is already a `Pattern` and for a value whose type has an instance, which is what makes `case #insert Ascii_Class.Digit;` legal against a `u8` subject.

```
GLOSSO
#import "Io";

Ascii_Class :: enum { Digit; Alpha; }

Ascii_Class ::: instance ToPattern {
    to_pattern :: (#comptime c: Ascii_Class) -> Pattern {
        if c == {
        case .Digit; return to_pattern(cast(u8)(#char "0") ..= cast(u8)(#char "9"));
        case .Alpha; return #pattern #insert to_pattern(cast(u8)(#char "a") ..= cast(u8)(#char "z"))
                                  | #insert to_pattern(cast(u8)(#char "A") ..= cast(u8)(#char "Z"));
        }
    }
}

describe :: (byte: u8) {
    if #pattern byte == {
    case #insert Ascii_Class.Digit;  print("digit\n");
    case #insert Ascii_Class.Alpha;  print("letter\n");
    else;                            print("other\n");
    }
}

main :: () {
    describe(cast(u8)(#char "x"));
    describe(cast(u8)(#char "7"));
    describe(cast(u8)(#char "!"));
}
```

```
CONSOLE / OUTPUT
letter
digit
other
```

`#insert` is required. Written bare, `Ascii_Class.Digit` is an ordinary enum-constant pattern and is checked against the subject's type, giving `error: got Ascii_Class, expected u8`. Splicing a value whose type has no instance is `error: this case is not a pattern, and its type has no 'ToPattern' instance`, followed by the candidate list for `to_pattern`.

Because the conversion happens before the coverage analysis runs, an instance joins that analysis on the same footing as the built-in ranges. Dropping the `else` from `describe` above produces witnesses drawn from the intervals the instance returned, clamped to `u8`.

```
CONSOLE / OUTPUT
error: non-exhaustive #pattern switch
note: missing case: 0..=47
note: missing case: 58..=64
note: missing case: 91..=96
note: missing case: 123..=255
note: cover them, or add 'else' or '#partial'
```

> **NOTE**
> A `Pattern` value is compile-time only. `p := #pattern .Some(v);` is `error: a 'Pattern' value is compile-time only; bind it with '::' rather than ':='`. The type and its introspection intrinsics belong to the chapter on patterns as compile-time values.

10.8 Exhaustiveness, Guards, and Reachability

Exhaustiveness in Glosso is computed, not demanded. The compiler runs a usefulness analysis over a matrix of deconstructed patterns: one row per arm, one column per position being matched. A row is useful if some value matches it that no earlier row already matches. The same routine answers both questions the language needs: a switch is exhaustive when the all-wildcard row is no longer useful against its arms, and arm `N` is reachable exactly when row `N` is useful against rows `0` through `N - 1`.

When a switch is not exhaustive the analysis reconstructs a concrete value that no arm matches, by unspecializing on the way back out of the recursion, and prints it as a witness. A witness is a pattern, not a value, so it may contain `_` where any value would do.

The coverage analysis governs #pattern switches, scalar and enum statement switches, and the switch forms of ifx. An incomplete statement switch reports non-exhaustive switch and lists missing cases; add else or #partial when coverage is intentionally incomplete. An ifx reports cannot decide a value for this ifx and requires an else arm or complete coverage. Tagged-union statement switches also check that every variant is handled.

Enumerable and opaque domains

Whether coverage can be proved depends on whether the subject's constructors can be enumerated. Three answers are possible: a finite set of constructors that can be listed, an infinite domain that only a wildcard covers, and a pattern the analysis cannot interpret as a constructor at all.

Subject	Treated as	Covered by
tagged union	one constructor per variant	every variant, or a wildcard
enum	one constructor per discriminant	every constant, or a wildcard
`bool`	two constructors	`true` and `false`, or a wildcard
struct	one constructor, decided by its fields	its fields being covered
integer	intervals, split rather than enumerated	intervals that tile the type
float, string, pointer	infinite	only a wildcard
literal, `==` constant, slice	opaque	nothing; they never cover

The last row is the one that surprises. A float or string literal arm matches at run time but proves nothing at compile time, so a switch built entirely from them is never total and its witness is the bare wildcard `_`. The same holds for `==` constant patterns and for slice patterns: they are matched at run time, and absent from the coverage matrix.

The diagnostic

GLOSSO

```
#import "Io";

Shape :: union { Circle: f64; Rect: f64; Empty: void; }

area :: (s: Shape) -> f64 {
    if #pattern s == {
    case Shape.Circle(r);  return 3.0 * r * r;
    case Shape.Rect(w);    return w * w;
    }
}

main :: () { print("%\n", area(Shape.Rect(2.0))); }
```

CONSOLE / OUTPUT

```
error: non-exhaustive #pattern switch
note: missing case: Shape.Empty
note: cover them, or add 'else' or '#partial'
```

The error is reported on the `if` and one `note: missing case:` line is emitted per witness, up to six, after which the count is summarised as `note: ...and N more`. The final note names the three ways out: cover the missing cases, add `else;`, or mark the switch `#partial`.

Subject and arms	Witness
`Shape` covered but for `Empty`	`Shape.Empty`
`Shape` with `Circle(1.0)` and `Empty`	`Shape.Circle(_)`
`Color { Red; Green; Blue }` with two arms	`Color.Blue`

Subject and arms	Witness
`bool` with `true`	`false`
`s64` with `1` and `2`	`-9223372036854775808..=0`, `3..=9223372036854775807`
`u8` with `0..=9` and `100..`	`10..=99`
a two-`bool` struct with the two matching arms	`Pair.{ .first = false, .second = true }` and `Pair.{ .first = true, .second = false }`
one `string` or `f64` literal	`_`
subject-less predicate arms	`_`

Two of those rows are worth reading twice. An integer subject yields intervals rather than a list of values, because the domain is split at the boundaries the arms mention instead of being enumerated. And a subject-less `#pattern` switch, whose arms are arbitrary boolean predicates, is never provably total: the witness is always `_`.

GLOSSO

```
#import "Io";

Big :: enum { A; B; C; D; E; F; G; H; I; J; }

main :: () {
    v := Big.A;
    if #pattern v == { case Big.A; print("a\n"); }
}
```

CONSOLE / OUTPUT

```
error: non-exhaustive #pattern switch
note: missing case: Big.B
note: missing case: Big.C
note: missing case: Big.D
note: missing case: Big.E
note: missing case: Big.F
note: missing case: Big.G
note: ...and 3 more
note: cover them, or add 'else' or '#partial'
```

How a guard affects coverage

A guarded arm contributes nothing. The pattern may match and the guard may still decline at run time, so the arm proves no coverage and is absent from the matrix altogether. Covering a variant only under a guard therefore leaves it missing.

GLOSSO

```
#import "Io";

Shape :: union { Circle: f64; Rect: f64; Empty: void; }

main :: () {
    s: Shape = .Empty;
    if #pattern s == {
    case Shape.Circle(r) if r > 1.0;  print("big circle\n");
    case Shape.Rect(w);               print("rect\n");
    case Shape.Empty;                 print("empty\n");
    }
}
```

CONSOLE / OUTPUT

```
error: non-exhaustive #pattern switch
note: missing case: Shape.Circle(_)
note: cover them, or add 'else' or '#partial'
```

`#falling` is the opposite case and is easy to confuse with a guard: a `#falling` arm does not decline its match, it adds the next arm's body to its own, so it covers its case exactly like an ordinary arm. The chapter on `case A, B;` groups and `#falling` has the rest.

Reachability

The same usefulness call, run against the arms that precede each arm, reports arms that can never be entered. This is a warning, not an error, and it runs whether or not the switch has an `else`. It applies only to `#pattern` switches with a subject; duplicate labels in a plain switch are not diagnosed, and the first matching arm wins.

GLOSSO

```
#import "Io";
```

```
main :: () {
    n: u8 = 5;
    if #pattern n == {
    case 0..=99;   print("low\n");
    case 50..=60;  print("mid\n");
    else;          print("high\n");
    }
}
```

CONSOLE / OUTPUT

```
warning: unreachable pattern - already covered by an earlier case
low
```

A guarded arm is neither reported by this lint nor counted against later arms, for the same reason it contributes no coverage: it can decline. So `case any if false;` written first leaves every following arm reachable, while an unguarded `case any;` makes them all dead.

> **NOTE**
> `#partial`, written immediately after `if` and before `#pattern`, waives the coverage requirement; control then falls out of the switch when nothing matches. It is the right answer when the uncovered cases are genuinely uninteresting, and the wrong answer when a later edit adds a variant, because `#partial` is exactly the annotation that stops the compiler from pointing at the new one.

10.9 `case A, B;` Groups and `#falling`

A case label may name several values at once: `case A, B;` runs its body when the subject matches any of the listed labels. The labels are values of the subject's type, so the form works for integers, strings, `#char` literals, floats, booleans, enum constants and union tags. It is one arm with several labels, not several arms.

GLOSSO

```
#import "Io";

Level :: enum { Trace; Debug; Info; Warn; Error; }

classify :: (n: s64) -> string {
    if n == {
    case 2, 3, 5, 7;  return "small prime";
    case 0, 1;        return "unit";
    else;             return "other";
    }
    return "unreached";
}

main :: () {
    print("% % %\n", classify(5), classify(1), classify(9));

    lvl := Level.Warn;
    if lvl == {
    case .Trace, .Debug;  print("verbose\n");
    case .Info;           print("normal\n");
    case .Warn, .Error;   print("loud\n");
    }

    word := "yes";
    if #partial word == {
    case "y", "yes", "true";  print("affirmative\n");
    case "n", "no", "false";  print("negative\n");
    }
}
```

CONSOLE / OUTPUT

```
small prime unit other
loud
affirmative
```

The no-binding restriction

A multi-label case may not capture. The arm body would have to be checked against every listed tag at once, and the captured name would have a different type in each, so the compiler rejects the combination outright rather than picking one.

GLOSSO

```
#import "Io";

Token :: union { Name: string; Text: string; Eof: void; }
```

```
main :: () {
    t: Token = .Text("hi");
    if t == {
    case .Name(x), .Text(x);  print("stringy %\n", x);
    case .Eof;                print("eof\n");
    }
}
```

CONSOLE / OUTPUT

```
error: a case with several tags cannot capture a payload - the tags have different payload types
```

The diagnostic fires on the presence of a capture in a multi-tag case, not on a comparison of the payload types, so it appears even when the tags agree, as `Name` and `Text` do above. Grouping tags whose payloads are irrelevant is fine: `case .Eof, .Nil;` on two `void` variants compiles and covers both.

The construct for one body over several shapes with a capture is the or-pattern in a `#pattern` switch, which requires the alternatives to bind the same names at the same types and so has no such problem.

GLOSSO

```
#import "Io";

Shape :: union { Circle: f64; Rect: f64; Empty: void; }

main :: () {
    s: Shape = .Rect(2.0);
    if #pattern s == {
    case Shape.Circle(r) | Shape.Rect(r);  print("sized %\n", r);
    case Shape.Empty;                      print("empty\n");
    }
}
```

CONSOLE / OUTPUT

```
sized 2
```

The comma spelling is not available in a `#pattern` switch: `case Shape.Circle(r), Shape.Rect(r);` there is `error: unexpected token ','`. A pattern case takes exactly one pattern, and `|` is how alternatives are written inside it.

`#falling`

`#falling;` gives C-style fall-through, with one meaning: it appends the next case's body to this one, unconditionally and without running that case's test. The arm does its own work and then the following arm's. It must be the last statement of the case body.

GLOSSO

```
#import "Io";

main :: () {
    lvl := 1;
    if lvl == {
    case 1;  print("T"); #falling;
    case 2;  print("D"); #falling;
    case 3;  print("I");
    else;    print("-");
    }
    print("\n");
    lvl2 := 3;
    if lvl2 == {
    case 1;  print("T"); #falling;
    case 2;  print("D"); #falling;
    case 3;  print("I");
    else;    print("-");
    }
    print("\n");
}
```

CONSOLE / OUTPUT

```
TDI
I
```

Four placements are rejected, each with its own message. Anything written after `#falling` in the same body is unreachable by construction, so the parser refuses it; an `else` arm and the final case have no following body to enter. The fourth is the restriction on binding switches, developed below.

Placement	Diagnostic
not the last statement of the case	`error: '#falling' must be the last statement of a case`

Placement	Diagnostic
on the `else` arm	`error: '#falling' on an 'else' arm has nothing to fall into`
on the last case	`error: '#falling' on the last case has no body to fall into`
in a switch whose cases bind	`error: '#falling' enters the next case's body without running its test, so it is not allowed in a switch whose cases can bind`

The last-case error carries `note: it would leave the switch on every subject that reaches it; delete it`. A `#falling` with nowhere to fall is not a subtle mistake: it turns the arm into a no-op exit on every subject that reaches it, and the compiler can see that at check time.

Where `#falling` is allowed

Entering a body whose test never ran is harmless when that body binds nothing, and unsound the moment it binds something: the body would read a payload from a variant it was not dispatched on. The rule follows the hazard rather than trying to re-test. Two switch forms can bind and are excluded outright: the tagged-union switch and every `#pattern` switch. What remains is the plain value switch over integers, strings, characters, floats and booleans, and the enum switch, which is what the feature is for.

GLOSSO

```
#import "Io";

Token :: union { Name: string; Code: s64; Eof: void; }

main :: () {
    t: Token = .Code(3);
    if t == {
    case .Name(s);  print("name %\n", s); #falling;
    case .Code(v);  print("code %\n", v);
    case .Eof;      print("eof\n");
    }
}
```

CONSOLE / OUTPUT

```
error: '#falling' enters the next case's body without running its test, so it is not allowed in a switch whose cases can bind
note: S8 requires a tagged union's payload to be reached through a capture in the arm that proved the tag, so every case here binds
note: group the tags with 'case A, B;' if they share a body, or lift the shared work into a function
```

In a `#pattern` switch the first note reads `note: a #pattern switch exists to destructure, so its cases bind` instead. The second note is the same in both, and is the whole remedy: group the labels, or move the shared work into a procedure both arms call.

A `#falling` arm still covers its own case. It always runs its own body, so it is an ordinary row in the coverage matrix, unlike a guarded arm; the chapter on exhaustiveness, guards, and reachability draws that distinction. `#falling` also composes with a grouped label.

GLOSSO

```
#import "Io";

Level :: enum { Trace; Debug; Info; Warn; Error; }

main :: () {
    lvl := Level.Info;
    if lvl == {
    case .Trace, .Debug;  print("verbose"); #falling;
    case .Info;           print(" info\n");
    case .Warn, .Error;   print("loud\n");
    }
}
```

CONSOLE / OUTPUT

```
 info
```

NOTE

`#falling` means something different in a `#if` switch: a static `#falling` arm re-tests from the next case rather than entering its body. The chapter on static branching with `#if` covers that form.

10.10 Bind or Diverge: `#pattern P := subject else`

`#pattern P := subject else { ... }` binds `P`'s captures into the enclosing scope and requires the `else` block to leave. It exists because the ordinary test nests: every optional unwrapped with `if #pattern .Some(v) = e { ... }` pushes the rest of the procedure one block deeper, and a procedure that unwraps four things ends up written four levels in, with the failure handling scattered down the right margin.

The bind form inverts that. The failure path is stated once, immediately, and the success path continues at the same indentation with the captures live from that statement to the end of the enclosing block.

GLOSSO

```
#import "Io";

Config :: struct { name: string; port: s64; }

lookup :: (found: bool) -> ?Config {
    if found return Config.{ .name = "svc", .port = 8080 };
    return .None;
}

serve :: (found: bool) -> s64 {
    #pattern .Some(cfg) := lookup(found) else { return -1; }
    print("serving % on %\n", cfg.name, cfg.port);
    return cfg.port;
}

main :: () {
    print("ok = %\n", serve(true));
    print("missing = %\n", serve(false));

    xs: [3]?s64 = .[.Some(1), .None, .Some(3)];
    total := 0;
    for x: xs {
        #pattern .Some(v) := x else { continue; }
        total += v;
    }
    print("total = %\n", total);
}
```

CONSOLE / OUTPUT

```
serving svc on 8080
ok = 8080
missing = -1
total = 4
```

`cfg` is an ordinary local from the statement onwards, not a name that lives only inside a nested block. Inside a loop the `else` may `continue`, which turns the form into "skip the elements that are not there", as the `total = 4` line shows: the `.None` element is skipped and `1 + 3` remains.

Why the `else` must diverge

Binding into the enclosing scope is only sound if the code after the statement is unreachable when the match failed. That is exactly what requiring the `else` to leave buys: there is no path on which `cfg` is read without having been written. The requirement is not a style rule, it is the premise the scoping decision rests on.

GLOSSO

```
#import "Io";

main :: () {
    o: ?s64 = .Some(3);
    #pattern .Some(v) := o else { print("absent\n"); }
    print("v = %\n", v);
}
```

CONSOLE / OUTPUT

```
error: the 'else' of a '#pattern ... :=' binding must not return normally
note: it has to be #Never - end it with 'return', 'break', 'continue', or a call to a #Never function
```

For the same reason the `else` block is checked without the bindings. They do not exist on the failure path, so naming one there is `error: unbound 'v'`, with the usual `help: did you mean 'o'?` suggestion pointing at the subject.

What counts as diverging

The check is a syntactic walk over the block's statements, asking whether control can reach the end. Any of the following forms as the last thing the block does satisfies it.

Form	Diverges	Why
`return`, `return e`	yes	leaves the procedure

Form	Diverges	Why
`break`, `continue`	yes	leaves the enclosing loop iteration
`goto('label)`	yes	`goto` is declared `#Never`
a call to a `#Never` procedure	yes	the declaration says it does not return
`if` with both branches diverging	yes	neither path reaches the end
an exhaustive switch, all arms diverging	yes	every arm leaves and one always runs
`while true { }`	no	the walk does not evaluate the condition

The last row is the trap. An infinite loop plainly never returns, but the check does not reason about loop conditions, so `#pattern .Some(v) := o else { while true { } }` is rejected with the same "must not return normally" error. Write an explicit `return` or a call to a `#Never` procedure instead.

GLOSSO

```
#import "Io";
#import "Jump";

bail :: (msg: string) -> #Never { print("fatal: %\n", msg); exit(2); }

both_ways :: (o: ?s64, flag: bool) -> s64 {
    #pattern .Some(v) := o else { if flag { return -1; } else { return -2; } }
    return v;
}

by_bail :: (o: ?s64) -> s64 {
    #pattern .Some(v) := o else { bail("no value"); }
    return v;
}

by_goto :: (o: ?s64) -> s64 {
    #pattern .Some(v) := o else { goto('fail); }
    return v;
'fail;
    return -3;
}

by_switch :: (o: ?s64, k: s64) -> s64 {
    #pattern .Some(v) := o else {
        if k == {
        case 1;  return -1;
        else;    return -9;
        }
    }
    return v;
}

main :: () {
    print("% % % %\n", both_ways(.None, true), by_goto(.None),
                       by_switch(.None, 1), by_switch(.None, 7));
    print("%\n", by_bail(.Some(5)));
}
```

CONSOLE / OUTPUT

```
-1 -3 -1 -9
5
```

Shape and restrictions

The `else` is mandatory and must be a block. Omitting it is `error: unexpected token ';'` at the end of the statement, and writing a single statement instead of a block, as in `else return -1;`, is `error: unexpected token 'return'`. This form explicitly accepts an optional trailing `;` after the block.

The parser decides between the bind form, the test form and the quote by scanning ahead for a `:=` at nesting depth zero. That scan counts `(`, `[` and `{` as openers but does not count `.{` or `.[`, so a top-level struct or slice pattern makes the scan stop early and the statement is read as a test. Both of the following are `error: unexpected token ':='`.

GLOSSO

```
#pattern Pair.{ .first = a, .second = b } := p else { return -1; }
#pattern .[a, b] := xs else { return -1; }
```

A variant pattern is unaffected, because its payload sits inside `(` and `)`, so the practical form is to bind the payload and destructure it afterwards: `#pattern .Some(p) := o else { return -1; }` accepts a `?Pair` and leaves `p` a plain `Pair`. `whole @ .Some(v) := o` also parses, for the same reason.

Every pattern shape is otherwise available, and the subject may be any expression. `?T` and `E!T` are the common subjects: `#pattern .Ok(v) := parse(text) else { return -1; }` is the error-handling spelling for a procedure that does not want to propagate, which the chapter on `Try`, `?`, and `#try` contrasts with propagation.

> **NOTE**
> The bind form is the answer the compiler points at when a pattern test is negated. `if !(#pattern .Some(v) = o)` is rejected with `note: to bind and leave on failure, write '#pattern P := subject else { ... }'`, because inverting a test would leave its captures with no branch to bind into.

10.11 Patterns as Compile-Time Values

A `#pattern P` written without `= subject` is a quote rather than a test: it produces a value of the built-in type `Pattern` that carries the pattern's syntax. The parser decides between the two by scanning ahead for a depth-zero `=`. `==` is a different token, so a constant pattern such as `#pattern == LIMIT` is never mistaken for a test.

A pattern was the one piece of Glosso syntax that could not be named. A predicate can be a procedure, a type can be a value, a block can be a `Code` value, but a pattern had to be written out in full at every site that matched it. The quote names it; `#insert` in pattern position puts it back.

GLOSSO

```
#import "Io";

Token :: union { Number: f64; Name: string; Eof: void; }

NUMBER :: #pattern Token.Number(n);
NAME   :: #pattern Token.Name(text);

describe :: (t: Token) {
    if #pattern t == {
    case #insert NUMBER;   print("number %\n", n);
    case #insert NAME;     print("name %\n", text);
    case Token.Eof;        print("eof\n");
    }
}

main :: () {
    describe(Token.Number(2.5));
    describe(Token.Name("hi"));
    describe(Token.Eof());
}
```

CONSOLE / OUTPUT

```
number 2.5
name hi
eof
```

A quote carries its binders with it. The arm that spliced `NUMBER` reads `n` and the arm that spliced `NAME` reads `text`, under the names the quote wrote, in the scope the splice landed in. Everything the pattern vocabulary accepts may be quoted; the vocabulary itself is in The Pattern Vocabulary.

A pattern has no storage

`Pattern` is a compile-time-only type. `size_of(Pattern)` is `0`, its values are dropped from every runtime argument list, and reaching lowering with one is an error. The checker rejects them where they are written rather than where they are first used, which is why the diagnostics below point at the declaration.

A pattern must therefore be bound with `::`, the compile-time constant form, and may not be stored in an aggregate or in a runtime variable.

- `p := #pattern .Some(v);` is `error: a 'Pattern' value is compile-time only; bind it with '::' rather than ':='`.
- `Holder :: struct { p: Pattern; }` is `error: field 'p' has the compile-time-only type 'Pattern', which has no storage`.
- A quote written where a test was meant, as in `if #pattern n { ... }`, reaches the lowerer and is `error: cannot lower PatternQuote expression`.

`Code` is the other compile-time-only type and obeys the same rules; the two share their machinery. `Pattern` differs in two ways, both covered below: identity is structural rather than by node, and a splice deep-clones instead of adopting the quoted nodes.

Splicing with `#insert`

`#insert e` where a pattern is expected splices the pattern that `e` evaluates to. `#insert` keeps its single meaning, "splice this compile-time syntax value here", and the position selects which kind of syntax is expected. Every pattern position accepts it: the test form, a case label, the bind-or-diverge form, and the inside of another quote.

A splice binds tighter than `|` and at the same level as the `==` constant marker, and looser than the postfix forms. `#insert A | #insert B` is therefore an or-pattern over two splices rather than one bitwise-or expression.

GLOSSO

```
#import "Io";

Shape :: union { Circle: f64; Rect: f64; Empty: void; }

CIRCLE :: #pattern Shape.Circle(r);
RECT   :: #pattern Shape.Rect(r);

main :: () {
    s: Shape = .Rect(3.0);
    if #pattern #insert CIRCLE | #insert RECT = s print("either r=%\n", r);
}
```

CONSOLE / OUTPUT

```
either r=3
```

The splice deep-clones the quoted syntax at every site. A `Code` value spliced at two differently typed sites reuses the same nodes, and the types stamped by the first check survive into the second; a pattern is matched against a type by definition, so it cannot afford that. A spliced value that is not already a `Pattern` is converted through its type's `ToPattern` instance, which is how a range enters a pattern; see Ranges in Patterns and `ToPattern`.

Patterns as arguments

A procedure takes a pattern through a `#comptime` parameter. The argument is const-evaluated at the call site and bound into the callee's compile-time environment, which is what lets the callee splice it.

GLOSSO

```
#import "Io";

Token :: union { Number: f64; Name: string; Eof: void; }

EOF_PATTERN :: #pattern Token.Eof;

matches :: (#comptime p: Pattern, t: Token) -> bool { return #pattern #insert p = t; }

main :: () {
    t := Token.Eof();
    print("% % %\n", matches(EOF_PATTERN, t), matches(#pattern Token.Eof, t),
                     matches(#pattern Token.Name(s), t));
}
```

CONSOLE / OUTPUT

```
1 1 0
```

Pattern identity is structural. The monomorphization key is a rendering of node kinds and parser-set attributes only, never node ids and never spans, so two quotes that read the same are one instantiation. The three calls above are two: `--backend llvm-ir` emits two definitions of `matches`, because `EOF_PATTERN` and the quote written at the call site render identically.

CONSOLE / OUTPUT

```
define i1 @glo_matches__pattern_Token__comptime_8a4b84710d3e5507
define i1 @glo_matches__pattern_Token__comptime_fc76ffc729883eba
```

The two binding tiers

A bare identifier in a pattern binds, so a quote needs a way to say "this name is a name". Two markers do it, the pair `Code` already has. A single-backtick name is a name rather than a binder: it is read as an ordinary constant when the pattern is checked. A double-backtick name is evaluated in the environment the quote was written in, and the value it had is baked into the quote.

Spelling	In a pattern	Reads from
bare name	binds a fresh capture	nothing; it is a binder
one backtick	a name, not a binder	the file's constants
two backticks	the value that name held	the quoting environment

GLOSSO

```
#import "Io";

MAX :: 255;

main :: () {
    n := 255;
    m := 7;
    print("backtick name vs MAX: % %\n", #pattern `MAX = n, #pattern `MAX = m);
    print("bare ident binds:     % %\n", #pattern MAX = n, #pattern MAX = m);
}
```

CONSOLE / OUTPUT

```
backtick name vs MAX: 1 0
bare ident binds:     1 1
```

The eager tier is what lets a quote built inside a generic carry a value that does not exist at the splice site. `exactly` below is evaluated at compile time with `v` bound to `2.5`; the pattern it returns holds `2.5`, not a reference to a parameter named `v`.

GLOSSO

```
#import "Io";

Token :: union { Number: f64; Name: string; Eof: void; }

exactly :: (#comptime v: f64) -> Pattern {
    return #pattern Token.Number(== ``v);
}

EXACT :: #comptime exactly(2.5);

main :: () {
    print("2.5 -> %\n", #pattern #insert EXACT = Token.Number(2.5));
    print("1.0 -> %\n", #pattern #insert EXACT = Token.Number(1.0));
}
```

CONSOLE / OUTPUT

```
2.5 -> 1
1.0 -> 0
```

NOTE

A single-backtick name is looked up among the file's compile-time constants, not among the locals of the procedure that splices the pattern. A quote that names a constant declared inside that procedure is `error: unbound 'LIMIT'`, reported at the quote. Where the value is known at quote time, the double-backtick tier avoids the question entirely.

The pattern intrinsics

Six intrinsics in `std/Meta/Utilities.glo` read a pattern. Each is a compile-time `#magic` declaration, shown here without its directives. The subject type comes first in every one of them because a binder's type depends on both the pattern and the type it is matched against: `Token.Number(n)` binds `n` as `f64` only against `Token`.

GLOSSO

```
pattern_binder_count  :: (T: type, #comptime P: Pattern) -> ssize
pattern_binder_name   :: (T: type, #comptime P: Pattern, i: ssize) -> string
pattern_binder_type   :: (T: type, #comptime P: Pattern, i: ssize) -> type
pattern_missing_count :: (T: type, Ps: []mut Pattern) -> ssize
pattern_missing       :: (T: type, Ps: []mut Pattern, i: ssize) -> string
pattern_is_useful     :: (T: type, Ps: []mut Pattern, #comptime Q: Pattern) -> bool
```

GLOSSO

```
#import "Io";
#import "Meta";

Token :: union { Number: f64; Name: string; Eof: void; }

NUMBER :: #pattern Token.Number(n);
NAME   :: #pattern Token.Name(text);

main :: () {
    print("binder count = %\n", pattern_binder_count(Token, NUMBER));
    print("binder 0 = % : %\n", pattern_binder_name(Token, NUMBER, 0),
                               type_name(pattern_binder_type(Token, NUMBER, 0)));
    print("missing count = %\n", pattern_missing_count(Token, .[NUMBER, NAME]));
    print("missing 0 = %\n", pattern_missing(Token, .[NUMBER, NAME], 0));
    print("useful = %\n",  pattern_is_useful(Token, .[NUMBER], #pattern Token.Number(2.0)));
    print("useful2 = %\n", pattern_is_useful(Token, .[NUMBER], #pattern Token.Eof));
}
```

CONSOLE / OUTPUT
```
binder count = 1
binder 0 = n : f64
missing count = 1
missing 0 = Token.Eof
useful = 0
useful2 = 1
```

`pattern_missing_count` and `pattern_is_useful` are the coverage analysis, asked directly. The contract is one-sided: the analysis errs only towards reporting more missing cases than a reader would, so a count of zero is a genuine totality proof and a non-zero count means "not proved" rather than "not total". Scalars, guarded arms and raw unions are the usual reasons for the difference; Exhaustiveness, Guards, and Reachability describes them, and Pattern Reflection covers the wider reflection surface.

10.12 Static Branching with `#if`

`#if` chooses between branches while the program is being checked. The condition is const-evaluated, the selected branch is spliced in, and the branches that were not selected are neither type-checked nor lowered. That last property is the whole point: a branch written for another target may name procedures that do not exist on this one.

There are two spellings, matching the two runtime forms. The conditional form takes a `bool` condition and an optional `else`; the switch form takes a subject and `==` followed immediately by a brace, with `case` labels and an optional `else;` arm. Neither takes parentheses, and both accept a single statement in place of a block.

GLOSSO
```
#import "Io";

main :: () {
    #if OUT.os == Target_OS.Windows print("single statement, no braces\n");

    #if OUT.os == Target_OS.Windows {
        print("windows\n");
    } else #if OUT.os == Target_OS.Linux {
        print("linux\n");
    } else {
        print("something else\n");
    }

    #if OUT.cpu == {
    case .X86_64;             print("64-bit x86\n");
    case .Aarch64, .Armv7;    print("arm\n");
    else;                     print("other cpu\n");
    }
}
```

CONSOLE / OUTPUT
```
single statement, no braces
windows
64-bit x86
```

> **NOTE**
> A static chain continues with `else #if`. Written `else if`, without the directive, the else branch is one ordinary runtime `if` statement: if the chain selects it, its condition is evaluated at run time and may read runtime locals, and both of its arms are checked and lowered.

The condition

The condition must be `bool`. `#if` does not take the truthiness rule that `if` takes, so `#if 1 { ... }` is `error: #if condition must be bool, got comptime_int`. The reason is that a static branch is a decision about the program text, and a silent coercion there would delete code on the strength of a guess.

The condition must also be compile-time evaluable. A runtime local reports `error: unknown const i` against the name it could not read. What qualifies is anything the constant evaluator can reach: `::` constants, `#comptime` parameters, compile-time intrinsics such as `Is_Integer(T)` and `size_of(T)`, and the target constants below.

- A file-scope `::=` binding is not entered into the compile-time environment, only local ones are, so `G ::= 7` at top level followed by `#if G == 7` is `error: unknown const G`. Use `::` for a file-scope constant.

Only the selected branch is checked

The branch that was not selected is stored and then ignored. It is parsed, so it must be syntactically well-formed, but no name in it is resolved and no type in it is computed.

GLOSSO

```
#import "Io";

WIDE :: true;

main :: () {
    #if WIDE {
        print("selected\n");
    } else {
        this_name_does_not_exist(1, 2, 3);
    }

    #if OUT.os == {
    case .Windows;  print("windows\n");
    else;           nonsense := "x" + 1;
    }
}
```

CONSOLE / OUTPUT

```
selected
windows
```

The same property makes `#if` the instantiation guard for generic code. A `#compile_error` in the branch that a bad instantiation selects turns a deep type error into one sentence at the call site.

GLOSSO

```
#import "Io";
#import "Meta";

only_ints :: (x: $T) -> T {
    #if !Is_Integer(T) { #compile_error "only_ints requires an integer"; }
    return x + x;
}

main :: () { print("%\n", only_ints(21)); }
```

CONSOLE / OUTPUT

```
42
```

`only_ints(1.5)` selects the other branch and reports `error: only_ints requires an integer` at the `#compile_error`. Constraints that belong to the signature rather than the body are better written with `where`; see Constraints with `where`.

`OUT` and `HOST`

Two constants are installed by the compiler and are the usual subjects of a `#if`. They are ordinary struct values of the types declared in `std/Prelude/Utilities.glo`, so their fields are read with `.`.

Constant	Fields	Describes
`OUT`	`.os`, `.cpu`, `.abi`	the target being built
`HOST`	`.os`, `.cpu`	the machine running the compiler

The field types are the enums `Target_OS`, `Target_CPU` and `Target_ABI`; their members and the flags that set them are in Targets and Cross-Compilation. The standard library uses exactly this shape, for instance to answer the byte order of the target:

GLOSSO

```
target_endian :: () -> Target_Endian {
    #if OUT.cpu == {
    case .Powerpc, .Powerpc64, .S390x, .Mips, .Mips64;    return .Big;
    else;            return .Little;
    }
}
```

`OUT` and `HOST` differ whenever the build is a cross-compilation. A `#comptime` block runs on the host and a compiled procedure runs on the target, so a gate around a system call in ordinary code reads `OUT`, and a gate around something the compiler itself will do reads `HOST`.

The limits

At module scope, #if selects declarations, imports, and libraries. Only the active branch contributes them to the module. Operator syntax and loads that supply operators must remain unconditional because the operator prescan runs before semantic branch selection.

Inside a procedure, local functions and types belong to their lexical block. A function declared inside a braced #if branch does not become a top-level function. Use module-level conditional declarations when a selected function should be callable elsewhere in the module.

GLOSSO

```
#import "Io";

#if true {
    selected :: () -> ssize { return 1; }
} else {
    selected :: () -> ssize { return 2; }
}

main :: () { print("selected=%\n", selected()); }
```

CONSOLE / OUTPUT

```
selected=1
```

> **NOTE**
> Module-level #if can select between complete implementations with the same name. Inactive branches introduce no declarations or dependencies. Source Files, Imports, and Loads covers conditional module loading.

A braced `#if` branch does not open a scope of its own. The selected block's statements are spliced into the enclosing block, so a variable declared in the taken branch is visible after it. This is a consequence of splicing rather than a decision, and it is what lets a `#if` chain define one name three different ways.

GLOSSO

```
#import "Io";

main :: () {
    #if true { v := 5; }
    print("%\n", v);
}
```

CONSOLE / OUTPUT

```
5
```

`#falling` means something different here. In a runtime switch it enters the next case's body unconditionally; in a `#if` switch it re-tests from the next case, so an arm that falls out of a static switch whose remaining labels do not match lands in `else`.

GLOSSO

```
#import "Io";

main :: () {
    #if OUT.cpu == {
    case .X86_64;   print("x86_64\n"); #falling;
    case .Aarch64;  print("aarch64\n");
    else;           print("re-tested into else\n");
    }
}
```

CONSOLE / OUTPUT

```
x86_64
re-tested into else
```

Question	`if`	`#if`
condition	any truthy value	must be `bool`
evaluated	at run time	at compile time
other branch	checked, lowered	neither
runtime locals	yes	no
`#falling`	enters the next body	re-tests the next case
file scope	n/a	not allowed

10.13 `while` and `#inline while`

`while` takes a condition and a body, and re-tests the condition before every iteration. The body may be a block or a single statement, and the condition is not parenthesised; parentheses around it are ordinary grouping.

The condition is mandatory. `while { ... }` is `error: unexpected token '{'`, because the parser reads an expression immediately after the keyword and a block is not one. The infinite loop is spelled `while true`.

GLOSSO

```
#import "Io";

main :: () {
    i := 0;
    while i < 3 { print("i=%\n", i); i += 1; }

    j := 0;
    while j < 2 j += 1;
    print("j=%\n", j);

    k := 0;
    while true { k += 1; if k == 4 break; }
    print("k=%\n", k);

    seen := 0;
    n := 0;
    while n < 5 { n += 1; if n % 2 == 0 continue; seen += n; }
    print("seen=%\n", seen);

    countdown := 3;
    while countdown { countdown -= 1; }
    print("countdown=%\n", countdown);
}
```

CONSOLE / OUTPUT

```
i=0
i=1
i=2
j=2
k=4
seen=9
countdown=0
```

The condition follows the same truthiness rule as `if`, so the integer `countdown` above is a legal condition and stops the loop at zero. The full set of falsy values is in `if`, `else`, and Truthiness.

- `break` leaves the innermost loop and `continue` starts its next iteration. Neither takes an argument and neither takes a label; to leave a nested loop, jump with `goto`, described in Labels and `goto`.
- A `defer` written in the body runs at the end of every iteration, including the iteration that `break`s or `continue`s out of it.
- A loop that never terminates is not diagnosed, and neither is a procedure that ends in one without returning; a function that runs off its end returns the zero value.

Pattern conditions

The condition may be a `#pattern` test. The captures reach the loop body and are re-bound from scratch on every iteration, which makes the form the natural spelling for walking an optional-terminated structure.

GLOSSO

```
#import "Io";

Node :: struct { value: s64; next: ?*mut Node; }

main :: () {
    opt: ?s64 = .Some(3);
    while #pattern .Some(v) = opt {
        print("v=%\n", v);
        if v <= 1 opt = .None; else opt = .Some(v - 1);
    }

    o: ?s64 = .Some(4);
    steps := 0;
    while #pattern .Some(v) = o && v > 2 { steps += 1; o = .Some(v - 1); }
    print("steps=%\n", steps);

    c: Node = .{ .value = 3, .next = .None };
    b: Node = .{ .value = 2, .next = .Some(&c) };
    a: Node = .{ .value = 1, .next = .Some(&b) };
    total := 0;
    cursor: ?*mut Node = .Some(&a);
    while #pattern .Some(p) = cursor { total += p.*.value; cursor = p.*.next; }
    print("total=%\n", total);
}
```

CONSOLE / OUTPUT

```
v=3
v=2
v=1
steps=2
total=6
```

The subject side of a test is parsed at a precedence above `&&` and `||`, so `while #pattern .Some(v) = o && v > 2` reads as `(#pattern .Some(v) = o) && v > 2`: the test runs first and the guard sees its capture.

The multi-arm block form takes several tests against several subjects. Every arm must match for the loop to run, and every arm is re-tested on every iteration, so the loop stops as soon as any one of them fails.

GLOSSO

```
#import "Io";

main :: () {
    left:  ?s64 = .Some(3);
    right: ?s64 = .Some(10);
    while #pattern {
        .Some(l) = left;
        .Some(r) = right;
    } {
        print("l=% r=%\n", l, r);
        if l == 1 left = .None; else left = .Some(l - 1);
        right = .Some(r + 10);
    }
    print("stopped\n");
}
```

CONSOLE / OUTPUT

```
l=3 r=10
l=2 r=20
l=1 r=30
stopped
```

NOTE

Two pattern tests must not be joined with `||`. The combination compiles, but `||` short-circuits, so the alternative that did not run leaves its captures in scope and never writes them, and a body that reads them sees zeroes. `&&` between tests is sound, and so is the multi-arm block above.

`#inline while`

`#inline while` is a compile-time loop. The condition is re-evaluated in the compile-time environment and the body is unrolled into straight-line statements until the condition is false. Nothing about the loop survives into the program.

The counter must therefore be a compile-time binding. `i ::= 0` declares one; a plain `i := 0` is an ordinary runtime variable and the condition that reads it is `error: #inline while condition must be compile-time evaluable`.

GLOSSO

```
#import "Io";
#import "Meta";

Vec3 :: struct { x: f32; y: f32; z: f32; }

main :: () {
    sum := 0;
    i ::= 0;
    #inline while i < 4 { sum += i; i += 1; }
    print("sum=%\n", sum);

    k ::= 0;
    #inline while k < field_count(Vec3) {
        print("field %\n", field_name(Vec3, k));
        k += 1;
    }
}
```

CONSOLE / OUTPUT

```
sum=6
field x
field y
field z
```

The second loop is what `#inline while` is for. `field_name` demands a compile-time index, and `#inline while` records its counter as one; `#inline for` deliberately does not, which is the difference between the two constructs and is covered in `#inline for`.

Each iteration must change some compile-time variable that the condition reads. Without that the unroller cannot terminate, so it refuses rather than looping, and there is a hard cap on the count in case the change never reaches the bound.

Written	Diagnostic
a runtime counter	`#inline while condition must be compile-time evaluable`
a body that never updates it	`#inline while made no compile-time progress; update a compile-time variable used by the condition`
more than 100000 iterations	`#inline while exceeded 100000 iterations`

NOTE

`break` inside `#inline while` is `error: break outside loop`. The construct unrolls into a run of ordinary statements, so at the point the checker reaches the `break` there is no loop to leave. The same applies to a genuinely unrolled `#inline for`. End the unrolling with the condition instead.

10.14 `for` Loops

`for` iterates. Its header names an iterable and, optionally, the variables the body reads it through; everything else in the header is a one-token modifier.

GRAMMAR

```
for [ "<" ] [ ":" expansion ] [ [ "*" ] value [ "," index ] ":" ] iterable body
```

Piece	Written	Effect
reverse	`<` right after `for`	iterate from the high end
expansion	`:name`	use that iteration macro
value	`v:`	name the element variable
index	`v, i:`	name the index variable too
pointer binding	`*v:` or `*v, i:`	bind the element's address through `PointerIterable`

The value variable defaults to `it` and the index variable to `it_index`. Renaming them adds names rather than replacing them: the compiler prepends `v := it;` and `i := it_index;` to the body, so both the chosen name and the default are live in the same body. Compiler-native and bundled collection and range paths bind the index as `ssize`; a user expansion may deliberately bind another type. Runtime and macro-expanded loops count steps from zero, including in reverse; a statically expanded tuple or fixed array instead uses the element's actual index, so those indices descend in reverse.

GLOSSO

```
#import "Io";

main :: () {
    for 0..3 print("it=% it_index=%\n", it, it_index);
    for i: 0..3 print("i=%\n", i);
    for value, index: 10..13 print("value=% index=%\n", value, index);
    for n: 0..2 print("both live: n=% it=% it_index=%\n", n, it, it_index);
}
```

CONSOLE / OUTPUT

```
it=0 it_index=0
it=1 it_index=1
it=2 it_index=2
i=0
i=1
i=2
value=10 index=0
value=11 index=1
value=12 index=2
both live: n=0 it=0 it_index=0
both live: n=1 it=1 it_index=1
```

Ranges

A range in the header is iterated by the compiler directly rather than through a library. Written inline or stored in a variable, all five usable shapes work; `..` alone is not one of them, because a bare `..` is `error: bare '..' needs an index context` and cannot be stored at all.

Shape	Written	Iterates
`Range`	`a..b`	`a` up to but not including `b`
`RangeInclusive`	`a..=b`	`a` through `b`
`RangeTo`	`..b`	the origin up to `b`
`RangeToInclusive`	`..=b`	the origin through `b`
`RangeFrom`	`a..`	`a` upwards, without end

GLOSSO

```
#import "Io";

main :: () {
    for i: 0..=3 print("inclusive %\n", i);
    for < i: 0..4 print("reverse %\n", i);

    r := 0..3;
    for i: r print("stored Range %\n", i);

    rt := ..4;
    for i: rt print("RangeTo %\n", i);

    rti := ..=2;
    for i: rti print("RangeToInclusive %\n", i);

    rf := 10..;
    for i: rf { print("RangeFrom %\n", i); if i == 12 break; }

    for < i: rt print("reverse RangeTo %\n", i);
}
```

CONSOLE / OUTPUT

```
inclusive 0
inclusive 1
inclusive 2
inclusive 3
reverse 3
reverse 2
reverse 1
reverse 0
stored Range 0
stored Range 1
stored Range 2
RangeTo 0
RangeTo 1
RangeTo 2
RangeTo 3
RangeToInclusive 0
RangeToInclusive 1
RangeToInclusive 2
RangeFrom 10
RangeFrom 11
RangeFrom 12
reverse RangeTo 3
reverse RangeTo 2
reverse RangeTo 1
reverse RangeTo 0
```

The induction variable takes the low bound's type, and a literal low bound is a compile-time number that takes the default `ssize`. So `for i: 0..count` counts in `ssize`, so `count` must be convertible to that width without narrowing. An `s64` upper bound is rejected on a 32-bit target even when its runtime value is small; write `for i: cast(s64)0..count` to keep a fixed-width induction variable, or check the bound before converting it to `ssize`. `for i: lo..hi` with `lo` declared `u8` counts in `u8`. A stored range does not go through that rule at all: `Range(T)` unified both operands when it was built, so `r := 0..count; for i: r` counts in `count`'s type.

> **NOTE**
> A finite range decides whether another value exists before stepping outside its bounds. Narrow and unsigned endpoints are therefore safe: `for < i: lo..hi` over `u8` bounds `0` and `3` yields `2, 1, 0` and stops, while `0..=255` yields all 256 `u8` values and stops at `255`. An open-ended `RangeFrom` has no such end condition; integer stepping wraps after the maximum, so its body must eventually `break` or return.

Reverse iteration of an open-ended range is `error: reverse for cannot use an open-ended RangeFrom`, since there is no last element to start from. A float range is `error: for range needs prev/next for f64`: the element type must be an integer or supply a `Step` instance. An enum without one receives the same diagnostic; defining `Step` makes a bounded enum range iterable. Prefix ranges such as `..last` additionally require a `RangeOrigin` instance.

Arrays, views, and dynamic arrays

A fixed array, a view and a dynamic array all use the prelude's blanket `Iterable` instance. They iterate by element, with `it_index` counting steps. A view produced by slicing iterates the slice, and its indices start at zero rather than at the slice's offset.

GLOSSO

```
#import "Io";
#import "Memory";

main :: () {
    xs: [4]ssize = .[10, 20, 30, 40];
    for x: xs print("fixed x=% at %\n", x, it_index);
    for x, i: xs[1..3] print("view x=% at %\n", x, i);

    d: [..]ssize;
    defer d.free();
    d.push(7);
    d.push(8);
    for x, i: d print("dyn x=% at %\n", x, i);
}
```

CONSOLE / OUTPUT

```
fixed x=10 at 0
fixed x=20 at 1
fixed x=30 at 2
fixed x=40 at 3
view x=20 at 0
view x=30 at 1
dyn x=7 at 0
dyn x=8 at 1
```

A tuple is also iterable, and is unrolled rather than looped, because its elements need not share a type. Writing `#inline for` over one is the explicit spelling of the same expansion; see `#inline for`. Strings instead decode Unicode and yield `rune` for every encoding. UTF-8 and UTF-16 iteration combines a complete scalar, malformed input produces U+FFFD while consuming at least one code unit, and reverse iteration follows scalar boundaries. `it_index` is an `ssize` scalar ordinal starting at zero in either direction, not a byte or code-unit offset. Direct indexing remains code-unit-oriented. `Simd` vectors and matrices are iterable too; a matrix walks row-major. These non-tuple paths are Prelude instances.

GLOSSO

```
#import "Io";
#import "Meta";

main :: () {
    text := "A가";
    for r, i: text
        print("%:%:%\n", i, type_name(type_of(r)), cast(u32)r);
    for < r, i: text print("reverse %:%\n", i, cast(u32)r);
}
```

CONSOLE / OUTPUT

```
0:rune:65
1:rune:44032
reverse 0:44032
reverse 1:65
```

A by-value array walk snapshots the container before it starts: its count and element reads come from that snapshot even if the original is changed in the body. The element variable is a copy, so assigning to it does not write back. `for <` is honoured and reads the snapshot from the high end; its `it_index` is still the step count `0, 1, 2, ...`, not the original element position.

GLOSSO

```
#import "Io";

main :: () {
    xs: [3]ssize = .[1, 2, 3];
    for < v, k: xs print("reverse v=% at step %\n", v, k);

    for v: xs v += 100;
    print("array unchanged: % % %\n", xs[0], xs[1], xs[2]);

    for i: 0..5 { print("range i=%\n", i); i += 2; }
    for v, k: xs { print("index k=%\n", k); k += 100; }
}
```

CONSOLE / OUTPUT
```
reverse v=3 at step 0
reverse v=2 at step 1
reverse v=1 at step 2
array unchanged: 1 2 3
range i=0
range i=3
index k=0
index k=1
index k=2
```

NOTE
Assigning to the induction variable of a native range loop changes its counter, so the range above visits `0` and `3`. The array expansion's named value and index are aliases local to the inserted body: changing `v` leaves the array untouched, and changing `k` does not change the expansion's private cursor, so every index is still visited.

Iterating by pointer

`for *v: xs` selects `PointerIterable.for_pointer_expansion` and binds `v` (and the default `it`) to `*T`, the address of the current element. Write through `v.*` to change the container. The prelude supplies this instance for every array shape, and `for < *v: xs` walks it backwards.

The container expression must be addressable. Ranges compute values and have no elements to point into; tuples have no addressable homogeneous element; strings are read-only; and `Simd` lanes and matrix elements have no pointer iteration instance. Mutable local array storage is addressable inside `#comptime` too, so pointer iteration can update it. A pointer array walk is deliberately not snapshotted: a dynamic array's count is reread each step, and growing it far enough to reallocate can leave the current element pointer dangling. Avoid structural mutation while such a loop is active.

GLOSSO
```
#import "Io";

main :: () {
    xs: [4]ssize = .[10, 20, 30, 40];
    for *x, i: xs x.* += cast(ssize)(i + 1);
    print("after write-through: % % % %\n", xs[0], xs[1], xs[2], xs[3]);
}
```

CONSOLE / OUTPUT
```
after write-through: 11 22 33 44
```

`break`, `continue` and `defer` behave in a `for` exactly as they do in a `while`: `break` leaves the loop, `continue` starts the next iteration, and a `defer` in the body runs at the end of every iteration. A type without the protocol is `'P' is not iterable; it needs an Iterable instance`; supplying that instance is Custom Iteration with `for_expansion`.

10.15 Custom Iteration with `for_expansion`

A non-inline `for` over an array, string, `Simd`, matrix, or custom container dispatches to a macro named `for_expansion`, chosen by the target's type. The contract is a one-method typeclass, so a container joins the loop syntax by writing an instance. Direct and stored ranges retain native range lowering, while tuples are expanded statically.

GLOSSO
```
// std/Prelude/Primitive_Typeclasses.glo
Iterable :: typeclass (T: type) {
    for_expansion :: (val: T, body: Code, reverse: bool) #expand;
}

PointerIterable :: typeclass (T: type) where Iterable(T) {
    for_pointer_expansion :: (val: *mut T, body: Code, reverse: bool) #expand
}
```

`#expand` makes it a macro: the body is cloned into the loop's place rather than called. The loop body arrives as the `Code` parameter, and the expansion decides how many times and in what order to splice it. When the loop renamed its variables, the compiler has already prepended `v := it;` and `i := it_index;` to that `Code`, which is why an expansion never sees the chosen names. A pointer expansion instead receives `*T`, the address of the container, and must bind `it` to an element address rather than an element copy.

What an expansion owes the loop

- It declares `it` with a leading backtick, holding the current element.
- It declares `it_index` with a leading backtick, counting iterations from zero.
- It writes `#insert body` where one iteration's work belongs.

The backticks are load-bearing. Macro expansion renames every local a macro declares, to keep it from colliding with the caller's names; a leading backtick opts out of that renaming, which is exactly what these two names need, since the spliced body refers to them. An expansion that declares a plain `it` compiles until a loop uses it, and then the body reports `error: unbound 'it'`.

GLOSSO

```
#import "Io";

Node :: struct { value: s64; next: ?*mut Node; }
List :: struct { head: ?*mut Node; }

List ::: instance Iterable {
    for_expansion :: (list: List, body: Code, reverse: bool) #expand {
        cursor := list.head;
        step := 0;
        while #pattern .Some(node) = cursor {
            `it := node.*.value;
            `it_index := step;
            cursor = node.*.next;
            step += 1;
            #insert body;
        }
    }
}

main :: () {
    c: Node = .{ .value = 30, .next = .None };
    b: Node = .{ .value = 20, .next = .Some(&c) };
    a: Node = .{ .value = 10, .next = .Some(&b) };
    list: List = .{ .head = .Some(&a) };

    for list print("default names: % at %\n", it, it_index);

    total := 0;
    for value, index: list { total += value; print("named: % at %\n", value, index); }
    print("total = %\n", total);
}
```

CONSOLE / OUTPUT

```
default names: 10 at 0
default names: 20 at 1
default names: 30 at 2
named: 10 at 0
named: 20 at 1
named: 30 at 2
total = 60
```

A linked list is the case that makes the feature necessary: it has no contiguous storage and no length, so no array or range spelling describes it, and yet the loop that walks it reads like every other loop.

The direction parameter

The third parameter carries what the loop header asked for. The compiler builds it from the header alone, and it is a macro parameter, so `#if reverse` selects a branch at expansion time. Walking by pointer is not a flag on this class: `for *x: c` selects `PointerIterable.for_pointer_expansion`, which receives the container by pointer and binds `it` to each element's address.

Header	reverse
`for x: c`	`false`
`for < x: c`	`true`

An expansion that ignores `reverse` silently iterates forward. Honouring it is a matter of reading the parameter and choosing a direction; the prelude's own instances all do, arrays and ranges included, which is why `for <` over an array walks backwards without the compiler lowering the loop itself.

GLOSSO

```
#import "Io";

Ring :: struct { items: [8]s64; start: ssize; count: ssize; }

ring_push :: (r: *mut Ring, value: s64) {
    r.*.items[(r.*.start + r.*.count) % 8] = value;
    r.*.count += 1;
}

Ring ::: instance Iterable {
    for_expansion :: (r: Ring, body: Code, reverse: bool) #expand {
        step := 0;
        while step < r.count {
            offset := ifx reverse r.count - 1 - step else step;
            `it := r.items[(r.start + offset) % 8];
```

```
            `it_index := step;
            step += 1;
            #insert body;
        }
    }
}

main :: () {
    r: Ring;
    r.start = 6;
    ring_push(&r, 10);
    ring_push(&r, 20);
    ring_push(&r, 30);

    for r print("% at %\n", it, it_index);
    for < v, i: r print("reverse % at %\n", v, i);
    for v: r { if v == 20 continue; print("kept %\n", v); }
}
```

CONSOLE / OUTPUT

```
10 at 0
20 at 1
30 at 2
reverse 30 at 0
reverse 20 at 1
reverse 10 at 2
kept 10
kept 30
```

The ring's elements wrap around its storage, so the three values start at slot six and continue at slot zero. Nothing about that reaches the loop: the expansion converts a step count into a slot, and the body sees only `it` and `it_index`.

`break` and `continue`

The body is spliced literally into whatever loop the expansion wrote, so `break` leaves that loop and `continue` jumps to its condition test. `break` needs no thought. `continue` does, because it skips everything the expansion placed after `#insert body`.

> **NOTE**
> An expansion must advance its cursor before `#insert body`, not after, or `continue` in a loop body repeats the same element forever. The `Ring` above increments `step` first for this reason. Every prelude instance follows the same rule, including the `Range` instance reached explicitly by `for :for_expansion`; `continue` is therefore safe on both the native and macro-expanded paths.

Naming an expansion

`for :name` selects an expansion by name instead of by type. The named procedure need not be a typeclass method: any `#expand` macro with the value, `Code` and `bool` parameters will do, which is the way to give one type several iteration orders.

GLOSSO

```
#import "Io";

Span :: struct { start: s64; count: ssize; }

stepping :: (s: Span, body: Code, reverse: bool) #expand {
    step := 0;
    while step < s.count {
        `it := s.start + step * 2;
        `it_index := step;
        step += 1;
        #insert body;
    }
}

main :: () {
    s: Span = .{ .start = 100, .count = 3 };
    for :stepping v, i: s print("stepping % at %\n", v, i);
    for :for_expansion v, i: 1..4 print("explicit builtin % at %\n", v, i);
}
```

CONSOLE / OUTPUT

```
stepping 100 at 0
stepping 102 at 1
stepping 104 at 2
explicit builtin 1 at 0
explicit builtin 2 at 1
explicit builtin 3 at 2
```

A name that does not resolve is `error: unknown for expansion 'nosuch'`. `:for_expansion` names the ordinary path explicitly, and on a range that is not a no-op: it routes the loop through the prelude's `Iterable` instance rather than the compiler's native range lowering.

An explicit expansion selector is consulted only on the non-inline path, so `#inline` cannot select it. A selector also cannot be combined with the pointer binder: because a named expansion decides what `it` means, write `for :for_pointer_expansion x: value` without `*` when that is deliberately the macro you want. A type with no instance and no named expansion is `'P' is not iterable; it needs an Iterable instance`. The related range protocols, `Step` and `RangeOrigin`, are described in Iteration and Range Typeclasses.

10.16 `#inline for`

`#inline for` is an ordinary `for` prefixed with the `#inline` directive. It asks the checker to replace the loop with one copy of the body per element rather than emit a loop. The request is honoured for exactly two iterables: a tuple and a fixed-size array. Every other iterable either compiles to an ordinary runtime loop or is rejected outright.

Static expansion is a dedicated compiler path. A tuple and a fixed array have a shape the checker can enumerate, while a view or dynamic array has only a run-time count. A user `Iterable` is rejected even if its author knows a fixed length: `#inline` does not invoke iteration macros, so custom compile-time unrolling must be written explicitly.

GLOSSO

```
#import "Io";

main :: () {
    xs :: .[2, 3, 5];
    p := 0;
    #inline for x: xs p += x;
    print("p=%\n", p);

    t :: .(1, "two", 3.0);
    #inline for t print("tuple elem: %\n", it);

    q := 0;
    #inline for n: 0..3 q += n;
    print("q=%\n", q);

    ys: [3]s64 = .[10, 20, 30];
    #inline for v, i: ys print("v=% i=%\n", v, i);
}
```

CONSOLE / OUTPUT

```
p=10
tuple elem: 1
tuple elem: two
tuple elem: 3
q=3
v=10 i=0
v=20 i=1
v=30 i=2
```

Three of those four loops behave differently. The tuple and the two arrays are expanded into straight-line statements; the `0..3` range loop is a plain runtime loop that happens to carry a directive the range path never reads. The array case does not require the array's elements to be constants, only its length: `ys` is an ordinary local and still unrolls.

Iterable	Effect of `#inline for`
a tuple literal, `.(a, b, c)`	unrolled; `#inline` is optional, a tuple is always expanded statically
a fixed array, `[N]T`	unrolled into N copies; `#inline for *x:` may bind each copy to an element address
a literal range, `0..n` or `0..=n`	an ordinary runtime loop; the directive is ignored
a stored `Range` value	rejected
a view `[]T` or a dynamic array `[..]T`	rejected
a type with an `Iterable` instance	rejected

The two rejections have distinct wording. A view or a dynamic array gives `error: #inline for needs a compile-time iterable; a runtime-length array expansion cannot be unrolled`, because the type is an array and only the length is missing. Everything else gives `error: #inline for expects a compile-time array`, which is what a stored range, an `Iterable` struct, and a bare integer all produce. Putting `#inline` in front of any other statement is `error: #inline can only prefix while or for`.

Heterogeneous elements

Unrolling is what makes a tuple iterable at all. Each copy of the body is checked against the type of the element it was made for, so one loop can call a generic procedure at four different instantiations and can branch on the element type with `#if`.

GLOSSO
```
#import "Io";
#import "Meta";

describe :: (v: $T) {
    print("% : %\n", v, type_name(T));
}

main :: () {
    row :: .(7, "seven", 7.5, true);
    #inline for row describe(it);

    #inline for row {
        #if type_of(it) == string print("  a string\n");
        else                      print("  not a string\n");
    }
}
```

CONSOLE / OUTPUT
```
7 : ssize
seven : string
7.5 : f64
1 : bool
  not a string
  a string
  not a string
  not a string
```

Writing `for row` without the directive produces the same expansion. A tuple has no runtime element type to loop over, so the checker takes the static path whether or not `#inline` is present. The directive matters only for a fixed array, where both lowerings exist and the plain `for` expands through the prelude to a real loop.

The loop variable is not a compile-time constant

Each unrolled copy begins with a synthesised `it := <element>` and an `it_index := <literal>`, declared as ordinary non-constant locals. `it` therefore has a known type in every copy but no known value, and any construct that demands a compile-time value fails.

GLOSSO
```
#import "Io";
#import "Format";

main :: () {
    xs :: .[1, 2, 3];
    #inline for xs {
        #insert tprint("k_% :: %;", it, it);
    }
    print("%\n", k_1);
}
```

CONSOLE / OUTPUT
```
error: unknown const it
```

`#inline while` does record its counter. The two constructs unroll through different code paths, and only the `while` path raises the checker's inline-expansion depth, which is the flag that turns an assignment into a recorded compile-time value.

GLOSSO
```
#import "Format";
#import "Io";
#import "String_Builder";

main :: () {
    i ::= 0;
    #inline while i < 3 {
        #insert tprint("k_% :: % * 10;", i, i);
        i += 1;
    }
    print("% % %\n", k_0, k_1, k_2);
}
```

CONSOLE / OUTPUT
```
0 10 20
```

The difference is deliberate and the checker records why at that code path: raising the depth in the `for` path does make `it` a known constant, but it also changes constant folding and const-argument resolution inside the body, which moves conformance cases. It is a change to make on its own rather than a side effect of unrolling.

The most common way to hit the rule is an array of types. `type_name` and `size_of` take their argument as a compile-time `type`, so the loop variable cannot be handed to them.

GLOSSO

```
#import "Io";
#import "Meta";

main :: () {
    Ts :: .[s64, f64, bool, string];
    #inline for T: Ts {
        print("% is % bytes\n", type_name(T), size_of(T));
    }
}
```

CONSOLE / OUTPUT

```
error: unknown const T
```

The replacement is an `#inline while` over a compile-time index, which indexes the same constant array. The index is a recorded constant, so `Ts[i]` folds and the intrinsic accepts it.

GLOSSO

```
#import "Io";
#import "Meta";

main :: () {
    Ts :: .[s64, f64, bool];
    i ::= 0;
    #inline while i < 3 {
        print("%\n", type_name(Ts[i]));
        i += 1;
    }
}
```

CONSOLE / OUTPUT

```
ssize
f64
bool
```

NOTE

The restriction is on the loop variable's value, not its type. A generic call binds `$T` from the argument's type and works per element, and `#if type_of(it) == string` selects a branch per copy. Only intrinsics and declarations that need the value itself are affected.

break, scope, and defer

A genuinely unrolled `#inline for` produces a list of blocks, not a loop, so there is nothing for `break` or `continue` to leave.

GLOSSO

```
xs :: .[1, 2, 3, 4];
total := 0;
#inline for x: xs { if x == 3 break; total += x; }
```

CONSOLE / OUTPUT

```
error: break outside loop
```

The same body is accepted under `#inline for n: 0..6`, because that form was never unrolled and is a real loop with a real exit edge. This is the clearest test of whether a given `#inline for` unrolled: if `break` compiles, it did not.

Each copy is wrapped in its own block, so a local declared in the body does not collide across copies and a `defer` in the body runs at the end of the copy that declared it rather than at the end of the whole expansion.

GLOSSO

```
#import "Io";

main :: () {
    xs :: .[1, 2, 3];
    #inline for xs {
        doubled := it * 2;
        defer print("leaving %\n", it);
        print("doubled %\n", doubled);
    }
}
```

CONSOLE / OUTPUT
```
doubled 2
leaving 1
doubled 4
leaving 2
doubled 6
leaving 3
```

- Assigning to the by-value element variable writes to that copy's local only. `#inline for v: xs { v += 100; }` leaves `xs` unchanged, exactly as the runtime array loop does. A fixed array may instead use `#inline for *v: xs { v.* += 100; }`; each unrolled copy binds the element's address and changes the original array. Tuples do not support pointer binding.

Reverse, and explicit expansions

Both `#inline for <` and runtime `for <` reverse an array walk. Their indices differ: the unroller emits copies in reverse original-index order, so `it_index` descends; the prelude expansion defines `it_index` as a step count, so it still rises from zero.

GLOSSO
```
#import "Io";

main :: () {
    xs: [3]s64 = .[1, 2, 3];
    print("inline reverse:\n");
    #inline for < v, i: xs print("  v=% i=%\n", v, i);
    print("runtime reverse:\n");
    for < v, i: xs print("  v=% i=%\n", v, i);
}
```

CONSOLE / OUTPUT
```
inline reverse:
  v=3 i=2
  v=2 i=1
  v=1 i=0
runtime reverse:
  v=3 i=0
  v=2 i=1
  v=1 i=2
```

An explicit expansion selector cannot be combined with `#inline`. The checker consults the `for_expansion` name only on the non-inline path, so `#inline for :looping v: d` skips the macro, falls through to the static path, and reports `error: #inline for expects a compile-time array`. Custom iteration is covered in Custom Iteration with `for_expansion`, and the unrolled `while` in `while` and `#inline while`.

One further position accepts `#inline for`: the case list of a `#code` quote, where it expands into real switch arms instead of statements. That is the mechanism the derive macros use, and it is described in Writing a Derive Macro.

10.17 Labels and `goto`

A label marks a position in a procedure body. It is written as a single leading quote followed by an identifier, `'name`, and it is the destination of a `goto`. Neither the label nor `goto` is a keyword: the label is a token kind the lexer produces, and `goto` is a procedure in `std/Jump.glo`.

The lexer decides between a label and an operator name by looking for the closing quote. After `'` it takes an identifier run, and if the character following that run is not another `'` the token is a label. So `'add` is a label and `'add'` is an operator name; the two never collide. Operator names are covered in Reserved Operators and Operator Names.

A label statement is written `'name;`. The terminator is load-bearing: the parser reads a label statement only when the token after the label is a terminator, so the semicolon is what separates defining a label from mentioning one.

GLOSSO
```
'first;      // label statement: defines the label
('third);    // expression statement: evaluates a `label` value
```

Parentheses force `'third` into expression position; the final `;` then terminates that expression statement, so no label is defined and a later `goto('third)` reports that the procedure has no such label. A bare `'third` followed by another statement is instead missing the required expression terminator and fails at that next statement. A label statement at file scope is not a statement at all and is rejected as a declaration: `error: bad decl name 'here'`.

> **NOTE**
> The parser also accepts a colon, `'name:`, and defines the same label. Nothing in the standard library or the examples uses it, and this manual does not, so treat `'name;` as the spelling. The colon form survives in one visible place: the compiler's own

> suggestion when a jump names a label the procedure does not declare reads `error: no label 'nowhere' in this procedure (define it with 'nowhere:)`. Write the semicolon anyway.

Jumping

`goto` requires `#import "Jump";`. Without it the call is `error: unknown function goto`. The declaration takes the target and three optional parameters that only the context-restoring modes use; the one-argument form is the plain jump and keeps the compiler's native local and computed-label lowering.

GLOSSO

```
goto :: (
    target: label,
    ctx: ?*mut Execution_Context = .None,
    value: any = false,
    return_code: s32 = 1,
) -> #Never #magic "jump_goto"

label ::: #public {
    is_valid :: (#self) -> bool #magic "label_is_valid";
    is_local :: (#self) -> bool #magic "label_is_local";
}
```

GLOSSO

```
#import "Io";
#import "Jump";

main :: () {
    i := 0;
'top;
    i += 1;
    if i < 3 goto('top);

    goto('done);
    print("skipped\n");
'done;
    print("i=%\n", i);

    target := 'first;
    target = 'second;
    goto(target);
'first;
    print("wrong\n");
'second;
    print("right\n");
}
```

CONSOLE / OUTPUT

```
i=3
right
```

The label pre-pass

Before a procedure body is checked, a pre-pass walks the whole body and collects every label it finds, recursing into loop bodies, `then` and `else` branches, switch case bodies and bare blocks. Labels are therefore function-scoped, not block-scoped, and a `goto` may name a label that has not been written yet or that sits deeper than the jump. A name the pre-pass never saw is rejected.

CONSOLE / OUTPUT

```
error: no label 'nowhere' in this procedure (define it with `'nowhere:`)
```

The outward jump is the everyday use. `break` and `continue` take no label, so leaving a nested loop from the inside is what `goto` is for.

GLOSSO

```
#import "Io";
#import "Jump";

main :: () {
    for i: 0..4 {
        for j: 0..4 {
            if i * j > 4 goto('escaped);
            print("i=% j=%\n", i, j);
        }
    }
'escaped;
    print("escaped\n");
}
```

CONSOLE / OUTPUT
```
i=0 j=0
i=0 j=1
i=0 j=2
i=0 j=3
i=1 j=0
i=1 j=1
i=1 j=2
i=1 j=3
i=2 j=0
i=2 j=1
i=2 j=2
escaped
```

A forward jump may cross a declaration or enter a nested construct. Execution resumes at the label. A crossed variable still has runtime storage and remains assignable, but its initializer and every other statement skipped on the way to the label do not run.

GLOSSO
```
#import "Io";
#import "Jump";

main :: () {
    goto('over);
    y := 42;
'over;
    y = 7;
    print("y=%\n", y);

    i := 0;
    goto('inside);
    while i < 3 {
        print("top %\n", i);
'inside;
        print("inside %\n", i);
        i += 1;
    }
}
```

CONSOLE / OUTPUT
```
y=7
inside 0
top 1
inside 1
top 2
inside 2
```

> **NOTE**
> Labels are collected for the whole procedure before unreachable code is discarded. Lowering reserves storage for a declaration that a jump crosses; only its initializer is dead. A label inside a block or loop also keeps that construct as an entry point. After the first mid-body pass, normal loop control resumes, as the `top 1` line demonstrates.

Duplicate labels

A label may be defined once per procedure. Two procedures in the same file may each define `'entry`; two `'entry` statements in one procedure may not.

CONSOLE / OUTPUT
```
error: function 'main' defines label 'U.again' more than once
```

This is also how a labelled `break` fails. Glosso has no labelled `break` or `continue`; `break 'outer;` parses as a `break` statement followed by a separate label statement `'outer`, which then collides with the real `'outer` the program declared and produces the duplicate-label error rather than a syntax error.

`label` is a type

A label in value position is an expression of the built-in type `label`. It can be stored in a variable, passed to a procedure, held in a struct field, and compared. `#undefined` is the absent label, and `label.is_valid` distinguishes it from a real one. A `goto` through a variable is a computed jump and is not checked against the procedure's label set.

Form	Meaning
`'name;`	statement; defines the label
`'name`	expression of type `label`
`#undefined`	the absent label; `label.is_valid` is false

Form	Meaning
`proc,,'name`	a stable entry point at `'name` inside `proc`
`t.is_local()`	true only for a label in the procedure making the call

GLOSSO
```
#import "Io";
#import "Jump";

Handler :: struct { landing: label; name: string; }

first :: () {
    n := 0;
'entry;
    n += 1;
    if n < 2 goto('entry);
    print("first n=%\n", n);
}

second :: () {
    m := 0;
'entry;
    m += 2;
    if m < 4 goto('entry);
    print("second m=%\n", m);
}

main :: () {
    first();
    second();
    h := Handler.{ .landing = 'stop, .name = "stop" };
    print("field valid: % name %\n", h.landing.is_valid(), h.name);
    absent: label = #undefined;
    print("absent is label: %\n", absent.is_valid());
    goto(h.landing);
    print("skipped\n");
'stop;
    print("stopped\n");
}
```

CONSOLE / OUTPUT
```
first n=2
second m=4
field valid: 1 name stop
absent is label: 0
stopped
```

The `,,` operator builds a label value naming a position in another procedure. Such a target is valid but not local, and entering it bypasses the ordinary call sequence, so arguments have to travel another way. That is what the payload mechanism in Explicit Control Transfer with the Jump Library is for.

`goto` and `defer`

A `goto` does not run pending `defer` statements. `defer` is tied to block exit, and a transfer is not a block exit: it abandons the block without unwinding it. This is the one behaviour that separates `goto` from `break`, `continue` and `return`, all of which do run their defers.

GLOSSO
```
#import "Io";
#import "Jump";

main :: () {
    {
        defer print("inner defer\n");
        print("before goto\n");
        goto('out);
    }
'out;
    print("after goto\n");
}
```

CONSOLE / OUTPUT
```
before goto
after goto
```

Targets

A `goto` whose target is a label literal in the same procedure lowers to a plain branch and works on every target the compiler emits, WebAssembly included. A computed target, whether a stored `label` or a `,,` entry point, needs an indirect

jump instruction and is rejected where the backend has none: `error: raw label goto is unsupported on portable WASM targets`. Other CPUs report `raw label goto is unsupported for target CPU '<cpu>'`.

10.18 Explicit Control Transfer with the Jump Library

`std/Jump.glo` builds setjmp and longjmp, exceptions, stackful coroutines, awaitables and a scheduler on a single transfer operation. That operation is `goto`. Everything else in the module is ordinary Glosso written on top of it, which is why the library has one primitive and no privileged internal entry point.

A `label` is only a code address; it carries no machine state. The state lives in `Execution_Context`, which owns a saved register image, a resume address, an inbox for the value a transfer carries, and a pointer to the logical execution-local state that holds the exception stack, the current coroutine and the jump payload.

GLOSSO

```
Context_Storage :: struct #magic "jump_context_storage_struct" {}

Execution_Context :: struct {
    machine: Context_Storage;
    resume_at: label;
    inbox: any;
    local: ?*mut Execution_Local;
}

Execution_Local :: struct {
    handler_top: ?*mut Exception_Frame;
    caught: ?*mut Exception_Frame;
    coroutine: ?*mut Coroutine;
    scheduler: ?*mut Scheduler;
    jump_payload: [..]any;
    pending_jump_payload: [..]any;
}
```

`Context_Storage` is opaque: the compiler supplies its size and alignment for the target. On `x86_64-pc-windows-msvc`, `size_of(Context_Storage)` is 640. A program never reads its fields; it only ever hands the address of the enclosing `Execution_Context` to `Execution_Context.save`, `Execution_Context.prepare` and `goto`.

`Execution_Context.save`

`Execution_Context.save` captures the current continuation into a context and returns `0`. When that context is later restored, the same call returns again, this time with the restoring `goto`'s `return_code`, and the `goto`'s `value` has been written into `ctx.inbox`. The declaration carries `#returns_twice` so the backend keeps the frame in a shape that survives the second return.

GLOSSO

```
Execution_Context ::: #public {
    save :: (*mut #self) -> s32
        #returns_twice #magic "jump_context_save"

    prepare :: (*mut #self, stack: []mut u8,
        local: ?*mut Execution_Local = .None)
        #magic "jump_context_prepare"
}
```

GLOSSO

```
#import "Io";
#import "Jump";

resume_saved :: (saved: *mut Execution_Context) -> #Never {
    goto(#undefined, .ctx = saved, .value = "carried", .return_code = 7);
}

main :: () {
    saved: Execution_Context;
    saved.local = execution_local();

    code := saved.save();
    if code == 0 {
        print("captured, code=%\n", code);
        resume_saved(&saved);
    }
    print("restored, code=% inbox=%\n", code, saved.inbox);
}
```

CONSOLE / OUTPUT

```
captured, code=0
restored, code=7 inbox=carried
```

`save()` records the active execution-local block automatically; the explicit assignment above is redundant. `prepare()` instead uses the block supplied by its `local` argument. Restoring a context publishes its recorded block, including its exception stack and payload storage.

The three modes of `goto`

`goto`'s optional `ctx` parameter is what separates a jump from a context switch, and its `target` selects where the switched-to context resumes. The three combinations are the whole interface.

Target	`ctx`	Effect
a valid label	absent	raw transfer to that label in the current context
`#undefined`	present	restore the context and resume where it was saved
a valid label	present	restore the context, overriding its resume address with the label

`value` and `return_code` are only meaningful in the two restoring modes. `value` is stored in the destination context's `inbox` before control arrives, and `return_code` becomes the value the destination's `Execution_Context.save` returns; it defaults to `1` so a restored capture can never be mistaken for the original `0`.

`Execution_Context.prepare`

`Execution_Context.prepare` builds a complete machine image over a stack the caller owns, without transferring control. Entering it with the third mode above runs the named label on that stack. This is the primitive behind coroutines.

GLOSSO

```
#import "Io";
#import "Jump";

State :: struct {
    fresh: Execution_Context;
    fresh_local: Execution_Local;
    caller: Execution_Context;
    stack: [65536]u8;
}

#thread_local storage: State = .{};

on_fresh_stack :: () -> #Never {
'entry;
    state := &storage;
    print("running on the fresh stack, inbox=%\n", state.*.fresh.inbox);
    goto(#undefined, .ctx = &state.*.caller, .value = "and back", .return_code = 9);
}

main :: () {
    state := &storage;

    state.*.fresh.prepare(state.*.stack[..], .local = &state.*.fresh_local);
    state.*.caller.local = execution_local();

    code := state.*.caller.save();
    if code == 0 {
        goto(on_fresh_stack,,'entry, .ctx = &state.*.fresh, .value = "hello");
    }
    print("caller resumed: code=% inbox=%\n", code, state.*.caller.inbox);
}
```

CONSOLE / OUTPUT

```
running on the fresh stack, inbox=hello
caller resumed: code=9 inbox=and back
```

> **NOTE**
> The context, its `Execution_Local` and the backing stack must all outlive every transfer that can reach them. A label entered nonlocally has no call arguments and no live caller frame to borrow from, which is why the example keeps all four in one thread-local object rather than in `main`'s locals.

setjmp and longjmp

`Jump_Buffer` aliases `Execution_Context`. Capture with `buffer.save()`; the old free `setjmp` spelling is no longer supplied. `buffer.longjmp(value)` calls `goto(#undefined, .ctx = &buffer, .return_code = value)` with a zero value promoted to `1`.

GLOSSO

```
#import "Io";
#import "Jump";
```

```
#import "Meta";

buffer: Jump_Buffer;

deep :: (depth: s32) {
    if depth == 0 buffer.longjmp(42);
    deep(depth - 1);
}

main :: () {
    print("Context_Storage is % bytes\n", size_of(Context_Storage));
    buffer.local = execution_local();
    code := buffer.save();
    if code == 0 {
        print("captured\n");
        deep(3);
    }
    print("unwound with code %\n", code);
}
```

CONSOLE / OUTPUT

```
Context_Storage is 640 bytes
captured
unwound with code 42
```

Exceptions

`try` is an `#expand` macro taking a `label` and two `#code` blocks. It expands into the caller, pushes an `Exception_Frame`, captures a landing context, and assigns the frame's unique label to the `label` variable passed in. `throw(target, payload)` walks the active frames for the one carrying that label, so an inner body may select an outer handler by name; `throw_current` takes the innermost frame instead. `catch()` reads the payload.

GLOSSO

```
#import "Io";
#import "Jump";

divide :: (handler: label, a: ssize, b: ssize) -> ssize {
    if b == 0 throw(handler, "division by zero");
    return a / b;
}

main :: () {
    outer: label = #undefined;
    inner: label = #undefined;

    try(outer, #code {
        print("first result %\n", divide(outer, 40, 5));
        try(inner, #code {
            print("before failing call\n");
            divide(outer, 10, 0);
            print("unreachable\n");
        }, #code {
            print("inner caught %\n", catch());
        });
        print("outer body done\n");
    }, #code {
        print("outer caught %\n", catch());
    });

    print("after try\n");
}
```

CONSOLE / OUTPUT

```
first result 8
before failing call
outer caught division by zero
after try
```

The inner `try` is skipped entirely: the failing call names `outer`, so the frame walk passes over `inner` and lands in the outer handler, and neither `outer body done` nor the inner handler runs. For code that cannot be expanded inline, `body.try_call(handler, data)` takes two function pointers instead of two quotes and always uses the innermost frame, and `Unhandled_Exception_Handler.set_unhandled_exception_handler` installs a last resort that a `throw_current` with no active frame reaches instead of terminating the process.

Coroutines

A coroutine is a `Execution_Context.prepare`d context with an owned stack, its own `Execution_Local`, and a pointer back to the caller's context. `Coroutine.resume` saves the caller and switches in; `co_yield` saves the coroutine and switches back, carrying a value each way through the inbox. Because the coroutine has a real stack, it may yield from any depth.

Call	Result
`Coroutine.coroutine_create(body, data, stack_size, scheduler)`	`?*Coroutine`; absent when allocation fails
`co.resume(input)`	runs until the next suspension, returns a `Coroutine_Event`
`co_yield(value)`	suspends; returns the next `input`
`co_return(value)`	`#Never`; ends the coroutine with `Returned`
`co.destroy()`	frees the stack, the payload arrays and the record

`Coroutine_Event` is a tagged union of `Yielded`, `Awaiting`, `Returned` and `Failed`, and `Coroutine_Status` runs `Fresh`, `Running`, `Suspended`, `Waiting`, `Done`, `Failed`. Resuming a coroutine that is already running, done or failed does not return an error: it exits the process with status 1.

GLOSSO

```
#import "Io";
#import "Jump";

counter :: (data: ?*mut void) -> any {
    input := co_yield(10);
    print("coroutine received %\n", input);
    input = co_yield(20);
    print("coroutine received %\n", input);
    return 30;
}

report :: (event: Coroutine_Event) {
    if event == {
    case .Yielded(value);  print("yielded %\n", value);
    case .Awaiting;        print("awaiting\n");
    case .Returned(value); print("returned %\n", value);
    case .Failed(error);   print("failed %\n", error);
    }
}

main :: () {
    if #pattern .Some(co) = Coroutine.coroutine_create(counter) {
        report(co.resume(0));
        report(co.resume(100));
        report(co.resume(200));
        print("status done: %\n", co.*.status == .Done);
        co.destroy();
    }
}
```

CONSOLE / OUTPUT

```
yielded 10
coroutine received 100
yielded 20
coroutine received 200
returned 30
status done: 1
```

The coroutine body never sees `goto`. It returns a value like any procedure, and an internal trampoline wraps it in a `Try_Body.try_call` so a throw that escapes the body arrives at the caller as a `Failed` event rather than unwinding past a stack that no longer has a caller.

Awaitables and the scheduler

`Awaitable.co_await` suspends into the `Waiting` status and links the coroutine onto the awaitable's waiter list. `Awaitable.complete` publishes the result and enqueues every waiter on the scheduler each one was created with. `Scheduler.run` drains the ready queue, re-enqueuing anything that merely yielded and leaving anything that is awaiting.

GLOSSO

```
#import "Io";
#import "Jump";

waiter :: (data: ?*mut void) -> any {
    if #pattern .Some(p) = data {
        value := (cast(*mut Awaitable)p).co_await();
        print("ready=%\n", value);
        return 9;
    }
    return 0;
}

main :: () {
    scheduler: Scheduler;
    awaitable: Awaitable;
    awaitable.init();
```

```
    created := Coroutine.coroutine_create(waiter, cast(*mut void)&awaitable, .scheduler = &scheduler);
    if #pattern .Some(co) = created {
        (&scheduler).scheduler_enqueue(co);
        scheduler.run();
        print("waiting=%\n", co.*.status == .Waiting);
        awaitable.complete(7);
        scheduler.run();
        print("done=%\n", co.*.status == .Done);
        co.destroy();
    }
}
```

CONSOLE / OUTPUT

```
waiting=1
ready=7
done=1
```

Nonlocal entry and the payload

Entering a `,,` label bypasses the calling sequence, so arguments cannot ride in registers. `long_goto` is an `#expand` macro that handles both cases: when `label.is_local` says the target is in the same procedure it jumps without evaluating its arguments at all, otherwise it packs the variadic pack into the execution-local payload and then jumps. The destination reads it with `payload()` or the typed `payload_get(T, i)` and calls `clear_payload()`.

GLOSSO

```
#import "Io";
#import "Jump";

worker :: () -> #Never {
'entry;
    arguments := payload();
    print("arrived with % arguments\n", arguments.count);
    n := payload_get(ssize, 0);
    flag := payload_get(bool, 1);
    print("n=% flag=%\n", n.unwrap_or(0), flag.unwrap_or(false));
    clear_payload();
    long_goto(main,,'back, "returning");
}

main :: () {
    entry := worker,,'entry;
    print("local: % valid: %\n", entry.is_local(), entry.is_valid());
    long_goto(entry, 40, true);

'back;
    message := payload_get(string, 0);
    print("back with %\n", message.unwrap_or(""));
    clear_payload();
    exit(0);
}
```

CONSOLE / OUTPUT

```
local: 0 valid: 1
arrived with 2 arguments
n=40 flag=1
back with returning
```

`payload_get` checks the recorded type name before it reads, so a mismatched index or type yields `.None` rather than a reinterpretation. A procedure entered this way has no caller to return to, which is why `worker` is `#Never` and why `main` ends with `exit(0)` instead of running off the end of a frame that was never entered normally.

> **NOTE**
> Restoring a context is the one part of the library that is not available everywhere. A plain local `goto` lowers on every target; a computed `goto` needs an indirect jump; a `Execution_Context.save` restore needs a hand-written register switch, and on a CPU without one the build stops with `error: context restore is not yet supported for this target CPU`. Portable programs guard the whole region with `#if OUT.cpu != Target_CPU.X86_64`, which is what the examples in `examples/goto.glo`, `examples/exceptions.glo` and `examples/coroutine.glo` do.

11 Compile-time programming

11.1 What Runs When

Glosso executes code during compilation with a constant interpreter and a register-based machine. Position decides when an answer is needed, while the body's operations also decide how it executes. Code that needs memory or managed cleanup can use the machine even when the checker needs its result as a constant.

The constant evaluator lives inside the type checker. It walks the syntax tree of the expression it has been asked to fold, at the moment the checker reaches that expression, and interprets the program directly from that tree. Before running a body, it can instead select VM-backed evaluation for pointer writes, view mutation, procedure-value calls, allocator access, or managed owners. Everything it produces is a value the checker then uses immediately: an array length, a branch decision, a spliced constant.

Top-level compile-time work runs later. Each top-level `#comptime` declaration is lowered into an ordinary IR function alongside the rest of the program, and once checking and lowering are finished the machine executes that function as bytecode, before the backend emits anything. The Compilation Pipeline places both against the full sequence of passes.

Which engine runs which construct

A top-level `#comptime` declaration goes to the machine after checking. Constructs that need an answer during checking go through constant evaluation, which can run a checked body in the VM when required. The engine is selected before execution, rather than retrying after a partially executed body and repeating its effects.

Construct	Evaluation phase
`X :: e;` and `X ::= e;` initialisers, and every global initialiser	Constant evaluator.
`#comptime e` and `#comptime { … }` in a value position	During checking; the interpreter or VM is selected for the body.
The condition of `#if`, `#assert`, and `#inline while`, and every `where` clause	Constant evaluator.
The operand of `#insert`, and the argument for a `#comptime` parameter	Constant evaluator.
`#modify` blocks, `#code` materialisation, and derive macro bodies	Constant evaluator.
`#comptime { … }` and `#comptime f();` written as a top-level declaration	Compile-time machine.

How often each one runs

The machine runs each top-level `#comptime` exactly once, in declaration order. Evaluation during checking makes no such execution count or output promise. Bodies can be checked repeatedly, and a VM-backed value evaluation can discard its captured output. Use these forms to compute values, and put observable build steps at top level.

GLOSSO

```
#import "Io";

never_called :: () { #comptime { print("[ct] never_called\n"); } }

generic_fn :: (x: $T) -> T { #comptime { print("[ct] generic_fn\n"); } return x; }

#comptime { print("[ct] top-level block\n"); }

main :: () {
    #comptime { print("[ct] main body\n"); }
    y := generic_fn(1);
    z := generic_fn("s");
    print("runtime main\n");
}
```

CONSOLE / OUTPUT

```
glosso: compile program.glo (target=x86_64-pc-windows-msvc)
[ct] top-level block
glosso: link program.exe (target=x86_64-pc-windows-msvc, -O1)
```

```
glosso: done program.exe  (backend=llvm, target=x86_64-pc-windows-msvc, -O1)
runtime main
```

The installed compiler prints the top-level block's output; the in-body blocks produce no visible lines in this example. That is not an execution-count API. A side effect belongs in a top-level `#comptime`, and an in-body block should be used for the value it produces.

What the constant evaluator refuses

The evaluator handles statements and expressions one kind at a time, and a kind it has no case for is refused rather than guessed at. Labels and `goto` are the visible example: an unstructured jump has no meaning in a tree walk, so the evaluator stops at the label. The same body runs without complaint in the machine, because by then the label and the jump have become ordinary IR instructions.

GLOSSO

```
#import "Io";
#import "Jump";

counted :: () -> s64 {
    n := 0;
'top;
    n += 1;
    if n < 3 goto('top);
    return n;
}

#comptime { print("[vm] counted = %\n", counted()); }

main :: () {}
```

CONSOLE / OUTPUT

```
glosso: compile program.glo (target=x86_64-pc-windows-msvc)
[vm] counted = 3
glosso: link program.exe (target=x86_64-pc-windows-msvc, -O1)
glosso: done program.exe  (backend=llvm, target=x86_64-pc-windows-msvc, -O1)
```

Replacing the `#comptime` block with `X :: #comptime counted();` sends the same body to the constant evaluator instead, and it is `error: statement is not available at compile time`, reported at the label rather than at the constant. An `#asm` block in a folded body gives the same message for the same reason.

Both compile-time engines bound execution so an infinite loop cannot hang a build. The default budget is 2^28 interpreter steps or VM instructions for one evaluation entry. `GLOSSO_COMPTIME_STEP_LIMIT` overrides it; `0` removes the limit. Exhausting it stops compilation with a diagnostic. The budget counts execution steps, not loop iterations.

Managed builders at compile time

The constant evaluator supports managed `String_Builder` values and compile-time formatting. Use the ordinary builder operations to collect text. `to_string()` only borrows the builder's buffer, so returning that view from a local builder would outlive its owner. `into_string()` consumes the builder and transfers the allocation to the result.

GLOSSO

```
#import "Io";
#import "String_Builder";

build_text :: () -> string {
    text := String_Builder.make_len_cap();
    text.write_string("hello ");
    text.write_string("builder");
    return text.into_string();
}

TEXT :: #comptime build_text();

main :: () { print("build_text = <%>\n", TEXT); }
```

CONSOLE / OUTPUT

```
glosso: compile program.glo (target=x86_64-pc-windows-msvc)
glosso: link program.exe (target=x86_64-pc-windows-msvc, -O1)
glosso: done program.exe  (backend=llvm, target=x86_64-pc-windows-msvc, -O1)
build_text = <hello builder>
```

> **NOTE**
> Compile-time source generation also supports String_Builder.new(), append(), and finish_source(). The last call consumes the builder and returns its text. Ordinary make_len_cap(), write_string(), and into_string() also work. Use `tprint` or `format` for formatted source and diagnostics; import `Format` to name them directly.

Globals, memory, and foreign calls

Neither engine runs arbitrary target foreign code: a `#foreign` function has no body to interpret. Compile-time code does have its own globals initialized from the program's declarations. Reading or writing them affects that compile-time execution, not the eventual program's runtime storage. Its `context.allocator` can serve VM heap blocks, including typed loads and stores through pointers. Each raw heap block is limited to 4 MiB in the compile-time VM.

GLOSSO

```
libc :: #library,system "c";
now_c :: () -> s32 #foreign libc "rand";

counter: s64 = 5;
```

Situation	Diagnostic
`now_c()` under the constant evaluator	`error: 'now_c' is foreign, so its result is not compile-time known`
`now_c()` under the machine	`error: #comptime stopped: VM cannot call foreign/undeclared function 'now_c'; use the LLVM backend`
Allocation and pointer writes	Supported through VM-backed compile-time execution; the compiler chooses the required engine.
A raw VM heap block larger than 4 MiB	Compilation stops at the VM's allocation limit.
Reading `counter` during compile-time execution	Observes its compile-time initial value, `5`, independently of later runtime writes.

Reflection is the case that looks as though it should differ and does not. `size_of`, `type_name`, `field_name` and the rest of `std/Meta` are compile-time-only intrinsics with no runtime implementation at all, and the machine has no entry for any of them. They still work in a top-level `#comptime` block, because the checker folds every such call to a constant before lowering; by the time the machine runs, the call is gone and only its answer is left in the instruction stream.

GLOSSO

```
#import "Format";
#import "Io";
#import "Meta";
#import "String_Builder";

Vec3 :: struct { x: f32; y: f32; z: f32; }

describe :: (T: type) -> string {
    return tprint("%.%", type_name(T), field_name(T, 0));
}

DESC :: #comptime describe(Vec3);

#comptime {
    print("[vm] describe = %\n", describe(Vec3));
    print("[vm] size_of  = %\n", size_of(Vec3));
}

main :: () { print("[ceval] describe = %\n", DESC); }
```

CONSOLE / OUTPUT

```
glosso: compile program.glo (target=x86_64-pc-windows-msvc)
[vm] describe = Vec3.x
[vm] size_of  = 12
glosso: link program.exe (target=x86_64-pc-windows-msvc, -O1)
glosso: done program.exe  (backend=llvm, target=x86_64-pc-windows-msvc, -O1)
[ceval] describe = Vec3.x
```

Compile-time output

Both engines send `print` output to the compiler's standard output, while the compiler's own `glosso:` progress lines go to standard error; redirecting one does not capture the other. The machine's output is buffered and flushed in a single block once every top-level `#comptime` has finished, so where it lands relative to the progress lines is not something to depend on.

GLOSSO

```
#import "Context";
#import "Io";

counter: s64 = 3;
read_counter :: () -> s64 { return counter; }
SNAP :: #comptime read_counter();

allocated_value :: () -> s64 {
    p := alloc(8);
    if #pattern .Some(raw) = p {
        cell := cast(*mut s64)raw;
```

```
        cell.* = 42;
        value := cell.*;
        p.free();
        return value;
    }
    return 0;
}
VALUE :: #comptime allocated_value();
main :: () { print("% %\n", SNAP, VALUE); }
```

CONSOLE / OUTPUT

```
3 42
```

11.2 `#comptime`

`#comptime` asks for a piece of the program to run during compilation. There is one spelling and several positions, and the position decides both what the directive leaves behind and which engine runs it.

In a value position `#comptime` produces a constant that is spliced into the program in place of the expression. As a top-level declaration it produces nothing: the block runs for its effects and emits no code. `#run`, the earlier spelling, no longer exists; `X :: #run 1 + 1;` is `error: unexpected directive 'run'`.

The forms

Form	Position	Meaning
`#comptime e`	Expression	Evaluate `e` during checking and splice the constant in.
`#comptime { … return e; }`	Expression	Run a block; the returned value is the constant, typed by inference.
`#comptime -> T { … return e; }`	Expression	The same, with the result typed exactly `T`.
`#comptime { … }`	Top-level declaration	Run the block in the compile-time machine for its effects.
`#comptime f();`	Top-level declaration	The same, written as one call.
`#comptime x: T`	Parameter	The argument must be compile-time known; see Compile-Time and Type-Only Parameters.

GLOSSO

```
#import "Io";
#import "Meta";

Inline_Expr :: #comptime 16 * 16;
Table_Size  :: #comptime { base := 16; return base * base; };
Byte_Limit  :: #comptime -> u8 { return 200; };

Squares :: #comptime -> [4]s64 {
    out: [4]s64;
    i := 0;
    while i < 4 { out[i] = i * i; i += 1; }
    return out;
};

main :: () {
    local := #comptime 6 * 7;
    print("Inline_Expr=% Table_Size=% local=%\n", Inline_Expr, Table_Size, local);
    print("Table_Size: % (% bytes)\n",
          type_name(type_of(Table_Size)), size_of(type_of(Table_Size)));
    print("Byte_Limit: % (% bytes)\n",
          type_name(type_of(Byte_Limit)), size_of(type_of(Byte_Limit)));
    print("Squares = % % % %\n", Squares[0], Squares[1], Squares[2], Squares[3]);
}
```

CONSOLE / OUTPUT

```
Inline_Expr=256 Table_Size=256 local=42
Table_Size: ssize (8 bytes)
Byte_Limit: u8 (1 bytes)
Squares = 0 1 4 9
```

`-> T` and the type of the result

Without the annotation the constant is typed by inference, and an integer literal returned from a block defaults to `ssize`, which is eight bytes. `#comptime -> u8 { return 200; }` is one byte instead. The annotation is checked rather than coerced, so it also refuses a value that does not belong: `#comptime -> u8 { return 500; }` is `error: 500 does not fit u8, whose range is 0..=255`. Compile-Time Numbers and Literal Defaulting explains the defaulting rule the annotation is overriding.

Any type the evaluator can build is accepted, including aggregates. That is what `Squares` above relies on: a lookup table is computed by a loop during checking and ends up in the program as ordinary constant data, with no initialisation code and no way for the two to drift apart.

Nothing of the computation survives into the output. A `#comptime` expression is replaced by its result before the backend sees it, so the emitted function contains a literal where the call used to be.

GLOSSO

```
fib :: (n: s64) -> s64 { if n < 2 return n; return fib(n - 1) + fib(n - 2); }

seed :: () -> s64 #dump {
    return #comptime fib(20);
}

main :: () { d := seed(); }
```

CONSOLE / OUTPUT

```
; ---- #dump typed-ir seed() -> ssize  [4 ins] ----
   0  temp_mark v0
   1  const v1 =6765
   2  temp_restore v0
   3  ret v1
; ---- end seed ----
```

A top-level `#comptime` is a build step

At file scope `#comptime` is a declaration. It runs once, in declaration order, after checking and before the backend emits anything, and it contributes no code to the program. This is where a build does work that is not part of the program: inspect the target, run an external tool, generate a source file, refuse to proceed.

GLOSSO

```
#import "Io";

check_target :: () {
    if OUT.os == Target_OS.Windows print("[build] targeting Windows\n");
    else print("[build] targeting something else\n");
    if OUT.cpu != Target_CPU.X86_64 {
        print("[build] this program has only been tested on x86-64\n");
        exit(1);
    }
}

#comptime check_target();

main :: () { print("hello\n"); }
```

CONSOLE / OUTPUT

```
glosso: compile program.glo (target=x86_64-pc-windows-msvc)
[build] targeting Windows
glosso: link program.exe (target=x86_64-pc-windows-msvc, -O1)
glosso: done program.exe  (backend=llvm, target=x86_64-pc-windows-msvc, -O1)
```

`exit(status)` called from compile time stops the build where it stands. No binary is written, the remaining top-level `#comptime` declarations do not run, and the compiler itself exits with the requested status, which is what makes the guard above a real gate rather than a warning. Output already produced by the block is still flushed.

The chosen backend makes no difference. `--backend llvm-ir` and `--backend object` run every top-level `#comptime` exactly as the default backend does; the compile-time machine sits before the point where the backends diverge.

> **NOTE**
> A program with a top-level `#comptime` is never restored from the native artifact cache. The cache cannot know what the block did, so a rebuild that skipped it would skip a build step, and the compiler declines to cache the artifact at all. A `#comptime` used only for a constant does not have this effect, which also means its side effects are skipped on a cached rebuild. The Build Cache and Generated Sources covers the cache.

The standard library turns this mechanism into a full build system: a top-level `#comptime` that calls into `std/Compiler` can set build options, add source files, and drive nested compilations. Build Scripts covers that API, and The Compile-Time Command Line and Artifact Cache covers the arguments a build step can read.

Inside a function body

In a body, a `#comptime` block is evaluated while checking. The compiler can use its VM for that value calculation, and output produced there is not a reliable observable effect. What Runs When compares this with top-level compile-time declarations.

The consequence is a rule rather than a number. An in-body `#comptime` is for its value; anything that writes a file, prints, or calls the compiler belongs in a top-level `#comptime` where the count is exactly one.

> **NOTE**
> A bare expression after `#comptime` in statement position is a trap. The parser consumes the directive and then wraps the expression in an ordinary expression statement, dropping the compile-time marking entirely, so `#comptime f();` inside a body runs at run time. Only the block form runs during compilation there.

GLOSSO

```
#import "Io";

main :: () {
    print("runtime start\n");
    #comptime print("A: statement-level #comptime expr\n");
    #comptime { print("B: statement-level #comptime block\n"); }
    print("runtime end\n");
}
```

CONSOLE / OUTPUT

```
glosso: compile program.glo (target=x86_64-pc-windows-msvc)
glosso: link program.exe (target=x86_64-pc-windows-msvc, -O1)
glosso: done program.exe  (backend=llvm, target=x86_64-pc-windows-msvc, -O1)
runtime start
A: statement-level #comptime expr
runtime end
```

What `#comptime` cannot reach

A `#comptime` fails when the code it names needs something that does not exist in that engine. The eventual program is not running, and arbitrary target foreign calls or runtime locals cannot be evaluated. Compile-time context reads and compiler-provided build, file, and string services are a different matter: they have explicit host implementations. What Runs When compares the two engines' answers; the table below contrasts supported state with remaining limits.

Case	Current behavior
A `#foreign` function	`error: 'now_c' is foreign, so its result is not compile-time known`
Allocation through `Context`	VM-backed evaluation supports heap blocks and typed pointer access, subject to its 4 MiB block limit.
Reading a mutable global	Supported in the compile-time program; it does not observe later runtime writes.
Reading a runtime local or parameter	`error: unknown const x`
`goto`, a label, `#asm`, and other unsupported statements	`error: statement is not available at compile time`
An evaluation that exceeds its execution budget	`GLOSSO_COMPTIME_STEP_LIMIT` stops it; the default is 2^28 steps, and 0 disables the bound

`#comptime` is one half of the metaprogramming surface: it computes values. The other half turns values back into program text, and is covered by Code Values: `#code` and Backticks, `#insert`, and Macros with `#expand`.

11.3 Compile-Time Constants

A name is a compile-time value when the checker has recorded its value in the compile-time environment. Only such a name can drive `#if`, an array length, an `#inline while` counter, an `#assert #comptime`, or a `#comptime` parameter. Which spellings put a name there is the subject here; Values, Constants, and Variables covers the declaration forms themselves.

`::` asks for immutability, `::=` asks for a compile-time value

`::` declares a constant. The checker tries to evaluate the initialiser: if that succeeds the name becomes a compile-time constant, and if it fails the name becomes a runtime-frozen constant instead, which is still immutable but no longer usable during checking. `::=` inside a body declares a compile-time binding, and there the evaluation is required rather than attempted.

Spelling	Compile-time evaluation	Usable in `#if`
`X :: e;`	Attempted.	Only if it succeeded.
`X : T : e;` and `X : T :: e;`	Attempted.	Only if it succeeded.
`X ::= e;` in a body	Required.	Yes.
`X : T := e;` in a body	Required.	Yes.
`X ::= e;` at file scope	Not attempted; this is a mutable global.	No.
`X := e;` and `X : T = e;`	Not attempted.	No.

The typed forms differ from the untyped ones only in the type they give the name, and that difference is visible through `type_of(value)`. An untyped integer constant keeps the literal type `comptime_int`, which has not yet been given a width; a typed one has the width it was written with. Compile-Time Numbers and Literal Defaulting explains what `comptime_int` is for.

GLOSSO

```
#import "Io";
#import "Meta";

main :: () {
    A ::  10;
    B ::= 20;
    C : s64 : 30;
    D : s64 := 40;
    print("A=% B=% C=% D=%\n", A, B, C, D);
    print("types: % % % %\n",
          type_name(type_of(A)), type_name(type_of(B)),
          type_name(type_of(C)), type_name(type_of(D)));
    #if A + B + C + D == 100 print("all four are compile-time values\n");
}
```

CONSOLE / OUTPUT

```
A=10 B=20 C=30 D=40
types: comptime_int comptime_int ssize ssize
all four are compile-time values
```

Runtime-frozen constants

A `::` whose initialiser cannot be folded is not an error. Immutability and compile-time knowledge are separate properties, and `::` asks for only the first of them; refusing the declaration would force a writer to give up immutability for a reason that has nothing to do with it.

GLOSSO

```
#import "Io";

report :: (samples: s64) {
    HALF :: samples / 2;
    i := 0;
    while i < 3 {
        print("bucket % of %\n", i, HALF);
        i += 1;
    }
}

main :: () { report(8); }
```

CONSOLE / OUTPUT

```
bucket 0 of 4
bucket 1 of 4
bucket 2 of 4
```

`HALF` is derived from a parameter, so the checker cannot know it. The declaration is accepted anyway: the expression is evaluated once, when control reaches the declaration, and the name is bound to that result for the rest of the scope. That is what a reader of `HALF ::` expects, and it is what the loop below it needs.

The price is that a frozen constant is not a compile-time value, and every construct that wanted one says so at the use rather than at the declaration. For a local `FROZEN :: n * 2` over a runtime `n`, `#if FROZEN == 14` is `error: unknown const FROZEN`, `grid: [FROZEN]s64;` is `error: array count 'FROZEN' is not compile-time known`, and `#assert #comptime FROZEN == 14` is `error: #assert #comptime condition is not compile-time evaluable`.

`::=` asks for the other property, so it reports the failure at the declaration: `K ::= n * 2;` over a runtime `n` is `error: comptime variable initializer must be compile-time evaluable`. A later write to a compile-time binding is held to the same standard; `i: s64 := 0; i = n;` is `error: comptime assignment must be compile-time evaluable`.

A compile-time binding is a variable

`::=` binds a name that may be reassigned while the checker runs. That is what makes an unrolled loop possible: `#inline while` re-evaluates its condition in the compile-time environment after each unrolled body, so the body has to move something the condition reads.

GLOSSO

```
#import "Io";
#import "Meta";

Vec3 :: struct { x: f32; y: f32; z: f32; }

main :: () {
    i ::= 0;
    #inline while i < 3 {
        print("%\n", field_name(Vec3, i));
        i += 1;
    }
}
```

CONSOLE / OUTPUT

```
x
y
z
```

`i := 0` would not serve here: the condition would not be compile-time evaluable and `field_name(Vec3, i)` would not have an index. `while` and `#inline while` covers the loop itself. A compile-time binding has no runtime storage and therefore no address, so `p := &i;` is `error: cannot take pointer to i`.

> **NOTE**
> `::=` means something else at file scope. Only a declaration inside a body is entered into the compile-time environment, so a top-level `G ::= 7;` is an ordinary mutable global and `#if G == 7` after it is `error: unknown const G`. Use `::` for a file-scope constant.

Compile-time meta-types

`Code`, `Pattern`, `Namespace`, and `Library` describe values that exist only while the compiler is building the program. Their layouts are zero-sized and the checker rejects a runtime local immediately. The capitalisation is part of the language: the former lower-case `pattern` spelling is now an ordinary identifier, not a type.

Type or annotation form	Role
`Code`	Type of a `#code` quotation.
`Pattern`	Type of `#pattern P` with no subject.
`Namespace`	Type produced by `#import` and `#load`.
`Library`	Type produced by `#library`.
`Typeclass(T, …)`	Explicit annotation for a typeclass declaration with those type parameters.
`Macro(signature)`	Explicit annotation for a `#expand` declaration with that function signature.

Bind one of these values with `::`, or state its meta-type in the new typed-definition form. `:=` asks for runtime storage and reports, for example, `a 'Pattern' value is compile-time only; bind it with '::' rather than ':='`. The older `type` value is also zero-sized, but it does not receive that immediate storage diagnostic; use `::` when the name must be available to compile-time operations.

> **NOTE**
> `Typeclass(...)` and `Macro(...)` are accepted as explicit declaration annotations. They are not general type-value expressions or callable constructors.

GLOSSO

```
IO : Namespace : #import "Io";
C_Runtime : Library : #library,system "c";
Predicate : Typeclass(type) : typeclass(T: type) {}
twice : Macro((ssize) -> ssize) :
    (value: ssize) -> ssize #expand { return value + value; }
```

```
main :: () { IO.print("%\n", twice(21)); }
```

GLOSSO

```
#import "Io";
#import "Meta";

Token :: union { Number: f64; Name: string; Eof: void; }

Elem   :: s32;
NUMBER :: #pattern Token.Number(n);
BANNER :: #code { print("generated line\n"); };

main :: () {
    Local  :: f64;
    print("Elem=% Local=%\n", type_name(Elem), type_name(Local));
    #insert BANNER;
    t: Token = .Number(2.0);
    if #pattern t == {
    case #insert NUMBER; print("number\n");
    else;                print("something else\n");
    }
}
```

CONSOLE / OUTPUT

```
Elem=s32 Local=f64
generated line
number
```

A procedure that needs a type for reflection normally takes it as a compile-time parameter. Compile-Time and Type-Only Parameters covers those parameters, Code Values: `#code` and Backticks covers `Code`, and Patterns as Compile-Time Values covers `Pattern`.

11.4 `#assert` and `#compile_error`

`#assert cond` is one statement doing two jobs. When the condition can be evaluated during checking it is a compile-time check that emits no code; when it cannot, the same statement compiles into a runtime check. There is no separate spelling for the runtime one and no flag that turns one into the other.

The decision runs in order. The condition is type-checked first and must be `bool`. Then the constant evaluator is asked for its value. If the answer is `false` the compilation fails; if it is `true` the statement lowers to nothing at all; if there is no answer the statement becomes a runtime assertion.

Condition	Result
Evaluates to `true`	Nothing is emitted.
Evaluates to `false`	`error: assertion failed`, or `error: assertion failed: <message>`.
Cannot be evaluated	A runtime check is compiled in.
Is not a `bool`	`#assert 5;` is `error: got ssize, expected bool`.

The compile-time assertion

Anything the constant evaluator can answer is fair game for a compile-time assertion, which in practice means layout facts, reflection results, and target queries. A passing assertion costs nothing, so it can sit permanently next to the declaration whose invariant it guards.

GLOSSO

```
#import "Io";
#import "Meta";

Vec3 :: struct { x: f32; y: f32; z: f32; }

main :: () {
    #assert size_of(Vec3) == 12;
    #assert #comptime field_count(Vec3) == 3, "Vec3 must stay three-wide";
    print("checks passed at compile time\n");
}
```

CONSOLE / OUTPUT

```
checks passed at compile time
```

A failing one stops the build and points at the `#assert` keyword. `#assert 1 + 1 == 3;` is `error: assertion failed`, and `#assert #comptime 1 + 1 == 3, "math is broken";` is `error: assertion failed: math is broken`.

A typeclass predicate is an ordinary compile-time `bool`, so an instance requirement can be written as an assertion. For a struct with no instance, `#assert Show(Opaque), "Opaque must be printable";` is `error: assertion failed: Opaque must be printable`. How an Instance Is Chosen explains what the predicate is asking.

The runtime assertion

When the condition depends on a value the compiler cannot know, the same statement becomes a check in the emitted program. It fires on the way past, prints the source position and the message, and aborts.

GLOSSO

```
#import "Io";

main :: () {
    x := 5;
    #assert x > 100, "x must be big";
    print("unreachable\n");
}
```

CONSOLE / OUTPUT

```
…\program.glo:5:5: assertion failed: x must be big
glosso: program exited with status exit code: 134
```

The runtime form is not removed at any optimisation level: `-O0` through `-O3` all abort here. It exits through the same path as the language's other runtime checks, which Runtime Safety Checks and Panics describes.

What the two lower to

Nothing about the two spellings differs. Only whether the condition folded decides which of them a given `#assert` becomes, and `#dump` shows the difference directly.

GLOSSO

```
static_ok :: (x: s64) -> s64 #dump {
    #assert 1 + 1 == 2, "arithmetic still works";
    return x;
}

dynamic_check :: (x: s64) -> s64 #dump {
    #assert x > 0, "x must be positive";
    return x;
}

main :: () { d := static_ok(1) + dynamic_check(2); }
```

CONSOLE / OUTPUT

```
; ---- #dump typed-ir dynamic_check(x: s64) -> s64  [9 ins] ----
   0  temp_mark v1
   1  const v2 =0
   2  call v3 'op_gt_s64_s64' [v0,v2]
   3  jnz v3 'L1'
   4  const v4 ="…\program.glo:7:5: assertion failed: x must be positive
"
   5  magic _ 'assert_failed' [v4]
   6  label 'L1'
   7  temp_restore v1
   8  ret v0
; ---- end dynamic_check ----
; ---- #dump typed-ir static_ok(x: s64) -> s64  [3 ins] ----
   0  temp_mark v1
   1  temp_restore v1
   2  ret v0
; ---- end static_ok ----
```

`static_ok` has no instruction for its assertion; the statement disappeared during checking, message and all. `dynamic_check` gained a comparison, a branch over the failure, and a call to `assert_failed` with a string that already carries the source position, because the position is known then and not later.

`#assert #comptime`

Writing `#comptime` after `#assert` removes the third outcome. The condition must now be evaluable, and failing to evaluate becomes an error rather than a silent downgrade. That matters for a check that is meant to hold at build time: without the modifier, a change that makes the condition depend on a local quietly turns a compile-time guarantee into a runtime one and nothing reports it.

For `x := 5;` followed by `#assert #comptime x > 0;` the diagnostic is `error: #assert #comptime condition is not compile-time evaluable`, with `note: unknown const x` pointing at the part that could not be folded. The note names the first thing the evaluator gave up on, which is usually enough to find the mistake.

Inside a generic body the assertion is checked once per instantiation, and only for the types that actually reach it.

GLOSSO

```
#import "Io";
#import "Meta";

label_of :: (x: $T) -> string {
    #assert #comptime Show(T), "label_of needs a Show instance";
    return show(x);
}

main :: () {
    print("% %\n", label_of(41), label_of("hi"));
}
```

CONSOLE / OUTPUT

```
41 hi
```

Adding `label_of(o)` for a struct with no `Show` instance is `error: assertion failed: label_of needs a Show instance`, reported at the assertion inside the generic body rather than at the call.

The message

The message is optional, follows a comma, and must be a compile-time `string`. It need not be a literal: any expression the evaluator folds to a string is accepted, so `tprint` can compose one from the values that made the check fail. A value of the wrong type is `error: #assert message must be a compile-time string`, and a runtime local used as the message is `error: unknown const reason`, pointing at the message rather than the condition.

`#compile_error`

`#compile_error <expr>` evaluates its operand and reports the result as the error message. It has no condition of its own and fires whenever checking reaches it, which makes it the payload for a `#if` guard rather than a check in its own right. The message being a value rather than a fixed string is the point: the failing case can name itself.

GLOSSO

```
#import "Format";
#import "String_Builder";

NAME :: "widget";

main :: () {
    #compile_error tprint("unsupported backend: %", NAME);
}
```

That is `error: unsupported backend: widget`. The operand is held to the same standard as an `#assert` message: a runtime local is `error: unknown const reason`.

Guarding an instantiation

`#if` prunes the branch it does not select, so a `#compile_error` inside an unselected branch never fires. That pairing is how a generic body rejects an instantiation with a message of the author's choosing instead of whatever the body happens to fail on first.

GLOSSO

```
#import "Io";
#import "Meta";

byte_width :: (x: $T) -> ssize {
    #if !Is_Integer(T) {
        #compile_error "byte_width needs an integer, got " + type_name(T);
    }
    return size_of(T);
}

main :: () {
    print("byte_width(u8)  = %\n", byte_width(cast(u8) 1));
    print("byte_width(ssize) = %\n", byte_width(1));
}
```

CONSOLE / OUTPUT

```
byte_width(u8)  = 1
byte_width(ssize) = 8
```

Adding `byte_width(1.5)` is `error: byte_width needs an integer, got f64`. A `where` clause is the other way to say this, and the two differ in what they do to overload selection: `where Is_Integer(T)` removes the candidate and reports `error: constraint failed for byte_width`, while the `#if` guard keeps the candidate and reports the chosen message. Constraints with `where` covers the first.

> **NOTE**
> Both directives are statements, not declarations. At file scope `#assert 1 == 1;` is `error: bad decl name 'assert'` and `#compile_error "no";` is `error: bad decl name 'compile_error'`. A whole-file invariant goes inside a top-level `#comptime` block, which is a declaration and takes statements.

GLOSSO

```
#import "Io";
#import "Meta";

Header :: struct { magic: u32; count: s32; }

#comptime {
    #assert size_of(Header) == 8, "Header must stay 8 bytes wide";
    print("[ct] Header layout checked\n");
}

main :: () {}
```

CONSOLE / OUTPUT

```
glosso: compile program.glo (target=x86_64-pc-windows-msvc)
[ct] Header layout checked
glosso: link program.exe (target=x86_64-pc-windows-msvc, -O1)
glosso: done program.exe  (backend=llvm, target=x86_64-pc-windows-msvc, -O1)
```

Adding a `pad: u8;` field to `Header` then gives `error: assertion failed: Header must stay 8 bytes wide`, reported at the `#assert` inside the block. Static Branching with `#if` covers the other half of the guard, and Struct Memory Layout covers what `size_of` is measuring.

`#source_location`: application diagnostics

`#source_location` is a primary expression with no arguments. It constructs the Prelude's `Source_Location`, whose fields are `filename: string`, `line: ssize`, and `column: ssize`. The position is the directive's own `#` token: lines and Unicode-scalar columns are one-based. The filename is the compiler's source path, or `<unknown>` when no source filename is available.

GLOSSO

```
#import "Io";

emit_diagnostic :: (location: Source_Location, message: string) {
    print("%: error: %\n", location, message);
}

main :: () {
    emit_diagnostic(#source_location, "invalid value");
}
```

CONSOLE / OUTPUT

```
…\program.glo:8:21: error: invalid value
```

`Source_Location` already implements `Show`, so `%` renders exactly `filename:line:column`; no custom formatter or import of its Prelude component is necessary. The example deliberately prints an editor-recognized diagnostic, but it only prints text: it does not abort, set an exit status, or fail compilation. Use `exit`, `#assert`, or `#compile_error` separately when that is the intended control flow.

Lexical positions survive calls and expansion

A directive in a helper's body names the helper's source, even when called from another file. A default argument containing the directive names the default declaration, not the omitted argument's call site. A directive written in a `#code` quote or `#expand` body keeps that written location after insertion or macro expansion. To report the caller, pass `#source_location` explicitly from the caller, as above.

The parser materializes the three fields before those transformations, so the result is also usable in constants, `#comptime`, and direct member access such as `#source_location.line`. It has the fixed nominal type `Source_Location`: an unrelated struct with identical field names is not an acceptable destination, including in generated or expanded code.

GLOSSO

```
#import "Io";
#import "Format";
HERE :: #source_location;
MESSAGE :: Source_Location.{
    .filename = "generated.glo", .line = 12, .column = 34 };
main :: () {
    #assert HERE.line == 3 && HERE.column == 9;
    print("%: error: generated\n", MESSAGE);
}
```

CONSOLE / OUTPUT
```
generated.glo:12:34: error: generated
```

NOTE
The filename in a directive-produced value is static text. Copying or returning that `Source_Location` does not create a release obligation, and the filename must not be freed. A manually constructed value, such as the one above, borrows whatever string you supply. At runtime `show(location)` produces temporary-arena text, valid only until that arena is reset or restored; the static filename's lifetime does not make the formatted result static.

11.5 Code Values: `#code` and Backticks

`#code` quotes Glosso syntax and produces a value of the built-in compile-time type `Code`. A quotation is parsed where it is written, but names and types inside it are resolved when `#insert` places it into a program. The same value can therefore be inserted more than once and can acquire a different meaning in each insertion scope.

Block, statement, and expression quotations

Form	Contents
`#code { statements }`	A statement list, including nested declarations.
`#code statement`	One statement; the surrounding semicolon terminates both the quotation and its use.
`#code (expression)`	One expression, ready for expression `#insert`.

GLOSSO
```
#import "Io";

GREET :: #code { print("hello from Code\n"); };
STEP  :: #code count += 1;

main :: () {
    count := 0;
    #insert GREET;
    #insert STEP;
    #insert STEP;

    comparison :: #code (count == 2);
    print("count=% equal=%\n", count, #insert comparison);
}
```

CONSOLE / OUTPUT
```
hello from Code
count=2 equal=1
```

The single form is new shorthand for the common one-statement or one-expression case. `#code print("x")` and `#code (a < b)` need no braces. A block remains necessary when order or several declarations are part of the quotation.

A `Code` value is compiler-only

`Code` needs no import, has size zero, and never reaches generated machine code. Bind a quotation with `::` or pass it directly to a `#expand` macro. `body := #code print("x");` is rejected with `a 'Code' value is compile-time only; bind it with '::' rather than ':='`, and an ordinary runtime procedure cannot consume one.

The contents are syntax, not a closure. `STEP` above names `count` before any such local exists; only the insertion sites are checked, and both have a `count` in scope. Inserting a quote into a scope where one of its names is absent reports the error at that insertion.

Double-backtick substitution

Inside a quotation, two backticks ask the compiler to substitute a compile-time binding while it materialises the syntax. A `string` can supply an identifier, a `type` can supply a type spelling, and another `Code` value can supply syntax. This is distinct from the single backtick used by macros to name the caller's scope.

GLOSSO
```
#import "Io";

Vec2 :: struct { x: s32 = 0; y: s32 = 0; }

show_field :: (v: Vec2, #comptime name: string) {
    #insert #code print("field % = %\n", name, v.``name);
```

```
}

main :: () {
    value: Vec2 = .{ .x = 7, .y = 9 };
    show_field(value, "x");
    show_field(value, "y");
}
```

CONSOLE / OUTPUT

```
field x = 7
field y = 9
```

The plain `name` in the format call remains the string parameter. The spliced `v.``name` becomes `v.x` or `v.y` before member lookup. The argument of a `#comptime` parameter must be known at compile time, so a runtime string cannot be used to synthesize an identifier.

What quotations can carry

A block quotation uses the statement grammar. Besides executable statements it can carry a nested procedure, type, macro, or typeclass instance declaration, and it can carry quoted `case` fragments for insertion into a switch. A quotation is still not source text: it is already parsed, retains hygiene marks, and is never written to the generated-source cache.

- Use `Code` when the generated shape can be expressed as parsed Glosso syntax and only names need to vary.
- Use a `string` insertion when the generator must construct tokens or whole declarations that cannot be represented by one fixed quotation.

11.6 `#insert`

`#insert` evaluates its operand at compile time and replaces itself with the resulting syntax. Its position selects both the accepted source and the grammar applied to that source.

Position and form	Requirement	Result
Statement `#insert source;`	`Code` or `string`	Statements in the surrounding body.
Top-level `#insert source;`	`Code` or `string`	Program declarations.
Expression `#insert code`	`Code` containing exactly one expression	That expression and its type.
Scoped `#insert,scope() code`	One-expression `Code`	An expression resolved after macro-local hygiene.
Pattern `#insert value`	`Pattern`	A pattern element.

Statement and expression insertion

GLOSSO

```
#import "Io";

repeat_three_times :: (body: Code) #expand {
    #insert body;
    #insert body;
    #insert body;
}

scoped_less :: (comparison: Code) -> bool #expand {
    a := 1;
    b := 2;
    return (#insert,scope() comparison);
}

insert_in_caller :: (body: Code) #expand {
    #insert body;
}

main :: () {
    total := 0;
    repeat_three_times(#code total += 14);
    print("repeated=%\n", total);

    message := "quotation scope";
    insert_in_caller(#code print("%\n", message));
    print("scoped=%\n", scoped_less(#code (a < b)));
}
```

CONSOLE / OUTPUT

```
repeated=42
quotation scope
scoped=1
```

A statement insertion accepts either a quotation or generated text. An expression insertion is intentionally narrower: it accepts only `Code` with one expression statement. A string, a block with two statements, or a quotation whose only item is a declaration is rejected rather than guessed into an expression.

`scope()` matters inside a macro. It makes the inserted quotation see the hygienically renamed locals established by that expansion. In `scoped_less`, the caller writes names `a` and `b` that do not exist at the call site; the scoped insertion resolves them as the macro's two locals and produces a `bool`.

Generated text and a typed insertion block

A `string` operand is lexed and parsed at the insertion position. Instead of calling a separate generator, `#insert -> string { ... }` runs the block in the constant evaluator and checks its returned value against the stated type before parsing it.

GLOSSO

```
#import "Io";

ANSWER_SOURCE :: "ANSWER :: 40 + 2;";
#insert ANSWER_SOURCE;

main :: () {
    #insert "print(\"plain source %\\n\", ANSWER);";

    #insert -> string {
        prefix := "typed source";
        return "print(\"typed source %\\n\", ANSWER);";
    }
}
```

CONSOLE / OUTPUT

```
plain source 42
typed source 42
```

The annotation after `->` is the compile-time block's result type, not the type of the generated expression. A statement or top-level insertion still ultimately requires `Code` or `string`. The typed form is especially useful when a generator has local control flow and a mistyped return should be diagnosed before parsing generated source.

Top-level declarations

At top level, text is parsed as declarations. That is the general way to synthesize a constant, an aggregate with a computed field list, or several related overloads. A `Code` block can also carry already-parsed procedure, aggregate, macro, and instance declarations.

GLOSSO

```
#import "Format";
#import "Io";
#import "String_Builder";

struct_source :: (name: string, fields: ssize) -> string {
    source := String_Builder.new();
    source.append(name, " :: struct {\n");
    i := 0;
    while i < fields {
        source.append("    f", tprint("%", i), ": s32 = 0;\n");
        i += 1;
    }
    source.append("}\n");
    return source.finish_source();
}

#insert struct_source("Generated", 2);

main :: () {
    value: Generated = .{ .f0 = 20, .f1 = 22 };
    print("%\n", value.f0 + value.f1);
}
```

CONSOLE / OUTPUT

```
42
```

Pattern insertion

A subjectless `#pattern P` quotation produces a compiler-only `Pattern` value. In pattern position, `#insert` accepts that value and clones its pattern at every use. `#insert A | #insert B` is an or-pattern because insertion binds tighter than `|`.

GLOSSO

```
#import "Io";

Token :: union { Number: f64; Name: string; Eof: void; }
NUMBER :: #pattern Token.Number(n);
NAME   :: #pattern Token.Name(text);

classify :: (token: Token) -> string {
    if #pattern token == {
    case #insert NUMBER; return "number";
    case #insert NAME;   return "name";
    else;                return "end";
    }
}

main :: () { print("%\n", classify(.Number(2.5))); }
```

CONSOLE / OUTPUT

```
number
```

Generated-source files and diagnostics

Every string insertion is materialised under `<cache>/generated/insert/v1/<sha256>.glo` before parsing. Equal text shares a cache entry. `--show-generated` prints those paths, while `--cache-dir` selects the cache root. `Code` and `Pattern` insertion uses syntax trees directly and creates no file.

A generated parse or type error points both to the written `#insert` and to the generated file. Nested insertion adds one `generated by this #insert` note per level, preserving the expansion chain instead of flattening it into one location.

> **NOTE**
> `#source_location` inside inserted `Code` retains the location where the quote was written. In contrast, a directive contained in a string is parsed as new source and names that generated cache file and its own line and column there. Pass an original `Source_Location` into your helper explicitly when a generated diagnostic should refer back to application source instead of generated text.

Refusals and bounds

Written	Diagnostic
`#insert 42;` in a body	`#insert expects Code or string source`
`#insert 42;` at top level	`top-level #insert expects Code or string source`
Expression insertion of a string	`expression #insert expects Code`
Expression insertion of several statements	`expression #insert expects Code containing exactly one expression`
`#insert` inside a struct body	`unexpected token 'insert'`

Recursive insertion is bounded: nesting stops at 16 and the total number of expansions at 16384. Crossing either boundary reports `#insert expansion limit exceeded`.

11.7 Macros with `#expand`

`#expand` declares a macro. The checker selects it using the argument types, substitutes argument syntax for parameters, gives declarations fresh hygienic names, and checks the expanded result in the caller. A macro with no value return is invoked as a statement; a macro with an explicit non-`void` return type is a value macro and can contain a full statement body.

Declaration	Use
`f :: (...) #expand { ... }`	Statement macro.
`f :: (...) -> T #expand { ... }`	Typed value macro; every reachable path must return `T`.
`f :: (...) #expand,expression { return e; }`	Legacy inferred one-expression macro.

Typed value macros

The return annotation now selects value context by itself; writing `#expand,expression` is unnecessary when `-> T` is present. Unlike the legacy inferred form, a typed value macro may use ordinary control flow, local declarations, several returns, and `defer`. Its expansion is checked as a value-producing block and may not fall through.

GLOSSO

```
#import "Io";

maximum :: (a: ssize, b: ssize) -> ssize #expand {
    if a > b return a;
    return b;
}

decision :: () -> ssize #expand {
    if `lower < `upper return 0;
    defer print("defer inside decision\n");
    return 1;
}

main :: () {
    lower := 7;
    upper := 3;
    print("maximum=%\n", maximum(lower, upper));
    print("decision=%\n", decision());
}
```

CONSOLE / OUTPUT

```
maximum=7
defer inside decision
decision=1
```

The unquoted parameters of `maximum` are substituted with the caller's expressions. `decision` instead uses single-backtick identifiers to look up `lower` and `upper` directly in the caller. Its ordinary `defer` belongs to the value-macro block and runs before that block hands its result back.

Hygiene and caller escapes

An unquoted local declared by a macro is renamed for each expansion, so it cannot collide with a same-named caller local. A single backtick opts an identifier or control-flow keyword into the caller's scope. That supports caller-variable access, deliberately published declarations, caller returns, and caller-scoped defers.

GLOSSO

```
#import "Io";

bump_caller :: () #expand {
    number := 1000;
    _ := number;
    `number += 10;
}

leave_caller :: () -> ssize #expand {
    defer print("macro defer\n");
    if `stop `return 42;
    return 0;
}

caller_defer :: () #expand {
    `defer print("caller defer\n");
    print("registered\n");
}

escaped :: () -> ssize {
    stop := true;
    _ := leave_caller();
    return -1;
}

main :: () {
    number := 0;
    bump_caller();
    print("number=% escaped=%\n", number, escaped());
    caller_defer();
    print("after registration\n");
}
```

CONSOLE / OUTPUT

```
macro defer
number=10 escaped=42
registered
after registration
caller defer
```

Putting a backtick before `return 42` leaves `escaped`, the procedure surrounding the expansion; the value is checked against that caller's return type. The ordinary defer in `leave_caller` still runs while the expansion unwinds. A backtick before `defer` is different: it registers with the caller, so its action runs only when `main` leaves.

> **NOTE**
> A backticked declaration can intentionally publish a name to inserted caller code, as when the macro writes a backtick before `it := 0;`, then `#insert body;`. Use that narrowly: ordinary macro locals are hygienic for a reason.

Nested and recursive macros

Macros may be declared inside procedures and inside other macros. A nested macro participates in the surrounding expansion and can use the names established there. Recursive value macros are also supported when static branching makes the expansion terminate.

GLOSSO

```
#import "Io";

outer_message :: () #expand {
    inner_message :: () #expand { print("nested macro\n"); }
    inner_message();
}

factorial :: (n: ssize) -> ssize #expand {
    #if n <= 1 return 1;
    else { return n * factorial(n - 1); }
}

main :: () {
    outer_message();
    print("factorial=%\n", factorial(5));
}
```

CONSOLE / OUTPUT

```
nested macro
factorial=120
```

`#if` is essential in `factorial`: it is selected during expansion, so the recursive call disappears at the base case. Ordinary `if` would remain runtime control flow and would not by itself stop compile-time macro expansion.

`Code` parameters

A macro may take `Code` from its caller and insert it zero, one, or many times. This is the basis of loop, conditional, and resource-wrapper macros. Use `#insert,scope()` when a one-expression quote must resolve against the macro's hygienic locals; use ordinary `#insert` when caller code should remain in caller scope.

GLOSSO

```
#import "Io";

with_value :: (body: Code) #expand {
    `value_from_macro := 42;
    #insert body;
}

main :: () {
    with_value(#code print("value=%\n", value_from_macro));
}
```

CONSOLE / OUTPUT

```
value=42
```

Selection, defaults, and `#modify`

Macro parameters may be concrete or generic and may have defaults. Candidate selection uses their inferred argument types and any `where` clauses. A `#modify` block runs after generic variables are inferred and may rewrite those variables before the expanded body is checked.

GLOSSO

```
#import "Io";
#import "Meta";

widen_small :: (value: $T) -> $R
#modify {
    R = T;
    if T == u8 { R = u32; }
    if T == s8 { R = s32; }
    return true;
}
{ return value; }

main :: () {
    small: u8 = 200;
```

```
    widened := widen_small(small);
    print("% %\n", widened, type_name(type_of(widened)));
}
```

CONSOLE / OUTPUT

```
200 u32
```

- A macro name still cannot be redeclared at file scope, even for a different parameter type or a different statement/value form.
- `#derive` procedures and typeclass expansion methods use the same macro machinery with signatures fixed by their protocols.

11.8 Reflection

Reflection is the set of compile-time intrinsics that answer questions about a type: its tag, its size, the names and types of its fields, the members of an enum, the variants of a union. They live in `std/Meta` and arrive with `#import "Meta";`. Every one of them is folded to a constant by the checker, so a reflection query costs nothing at run time and is available wherever a constant is: `#comptime`, `#if`, `#assert`, a `where` clause, a constant initialiser, and the argument of `#insert`.

GLOSSO

```
#import "Io";
#import "Meta";

Vec3  :: struct { x: f32 = 0.0; y: f32 = 0.0; z: f32 = 0.0; }
Color :: enum { Red; Green :: 5; Blue; }
Shape :: union { circle: s32; box: s64; empty: void; }

main :: () {
    info :: type_info(Vec3);
    print("tag % name % size % alignment %\n",
        cast(s64) info.tag, info.name, info.size, info.alignment);
    print("kinds: Vec3 % Color % Shape %\n",
        cast(s64) type_kind(Vec3), cast(s64) type_kind(Color),
        cast(s64) type_kind(Shape));
    print("fields %: % % %\n", field_count(Vec3),
        field_name(Vec3, 0), field_name(Vec3, 1), field_name(Vec3, 2));
    print("field 0 type %\n", type_name(field_type(Vec3, 0)));
    print("members %: %=% %=% %=%\n", enum_count(Color),
        enum_name(Color, 0), enum_value(Color, 0),
        enum_name(Color, 1), enum_value(Color, 1),
        enum_name(Color, 2), enum_value(Color, 2));
    print("variants %: %:% %:% %:%\n", variant_count(Shape),
        variant_name(Shape, 0), type_name(variant_type(Shape, 0)),
        variant_name(Shape, 1), type_name(variant_type(Shape, 1)),
        variant_name(Shape, 2), type_name(variant_type(Shape, 2)));
}
```

CONSOLE / OUTPUT

```
tag 7 name Vec3 size 12 alignment 4
kinds: Vec3 7 Color 8 Shape 9
fields 3: x y z
field 0 type f32
members 3: Red=0 Green=5 Blue=6
variants 3: circle:s32 box:s64 empty:void
```

`Type_Info`

`type_info(T)` answers every shape question about `T` at once. It is an ordinary struct declared in `std/Meta/Types.glo`, so its result is bound to a constant and read with `.` like any other value.

Field	Type	Holds
`tag`	`Type_Info_Tag`	which family `T` belongs to
`name`	`string`	the printed spelling, as `type_name` gives it
`size`	`ssize`	size in bytes under the target layout, including `#packed` when present
`alignment`	`ssize`	required alignment in bytes
`shower`	`Any_Show_Proc`	the `Show` thunk the `any` boxing carries

`alignment` is the only route to a type's alignment; there is no `align_of`, and writing one is `unknown function align_of`. The compiler populates `shower` with the non-null rendering thunk used when a value of the described type is boxed into `any`; user code neither implements nor registers that thunk. `Show`, `FromString`, and `print` are covered in Formatting, which is the public formatting surface.

`Type_Info_Tag`

The tag is a closed enum of nineteen members numbered `0` through `17` and `19`. Tag `18` is reserved for the removed character-byte type; `Rune` remains `19`, and `type_kind(T)` is the same value without the surrounding record. Types as Values lists the numbering. What matters when reflecting is which written spellings collapse onto one tag, because a single tag often has to be taken apart further.

Tag	Covers
`Array`	`[N]T`, `[]T` and `[..]T` alike
`Pointer`	`*T` and `[*]T`
`Union`	tagged unions, raw unions, `?T` and `E!T`
`Procedure`	procedure types and `#fn_ptr` types
`Enum`	`enum` and `enum_flags`
`String`	`string`, `cstring`, `string16` and `cstring16`

The named predicates `Is_Integer`, `Is_Struct`, `Is_Union` and the rest wrap the tag comparison and are the working vocabulary of `where` clauses and instance heads; Types as Values gives the full list.

The intrinsic table

Intrinsic	Answers
`type_info(T)`	the record above
`type_kind(T)`	the tag alone
`field_count(T)` / `field_name(T, i)` / `field_type(T, i)`	struct fields, in declaration order
`enum_count(T)` / `enum_name(T, i)` / `enum_value(T, i)`	enum members and their numeric values
`variant_count(T)` / `variant_name(T, i)` / `variant_type(T, i)`	union variants; `void` where a variant has no payload
`simd_lane_count(V)` / `simd_element_type(V)`	the shape of a `Simd(T, N)`
`matrix_row_count(M)` / `matrix_column_count(M)` / `matrix_element_count(M)`	the shape of a `Matrix(T, R, C)`
`matrix_element_type(M)` / `matrix_transpose_type(M)` / `matrix_product_type(A, B)`	matrix types computed from matrix types

`type_of(value)`, `type_name` and `size_of` belong to the same family and are described in Types as Values; the `pattern_binder_*` and `pattern_missing*` intrinsics belong to Pattern Reflection; the `derive_type_*` intrinsics belong to Writing a Derive Macro. `simd_*` comes with `#import "Simd";` and `matrix_*` with `#import "Math/Matrix";`, the rest with `#import "Meta";`.

> **NOTE**
> Every one of these queries is registered compile-time-only. A query that cannot be folded is an error at the query, not a lookup at run time. The compiler also emits static `Type_Info` records for runtime `any` boxes, which is distinct from evaluating these queries. That is why the index arguments below have to be constants.

Indices must be compile-time constants

`field_name(T, i)` folds only if `i` folds. An ordinary loop variable does not: an `i := 0` counter driving a `while` gives `unknown const i` at the intrinsic, and so does the loop variable of a plain `for` and of `#inline for`, which unrolls the body but does not record a compile-time value for its variable. Two spellings do work: a literal index, and a counter that is itself a compile-time binding.

```
GLOSSO
#import "Format";
#import "Io";
```

```
#import "Meta";
#import "String_Builder";

Vec3 :: struct { x: f32 = 0.0; y: f32 = 0.0; z: f32 = 0.0; }

fields_desc :: (#comptime T: type, #comptime i: ssize) -> string {
    #if i >= field_count(T) { return ""; } else {
        one := tprint(" %: %", field_name(T, i), type_name(field_type(T, i)));
        return tprint("%%", one, fields_desc(T, i + 1));
    }
}

VEC3_FIELDS :: #comptime fields_desc(Vec3, 0);

main :: () {
    print("recursive helper:%\n", VEC3_FIELDS);
    print("literal index:    %\n", field_name(Vec3, 1));
    i ::= 0;
    #inline while i < field_count(Vec3) {
        print("#inline while:    %\n", field_name(Vec3, i));
        i += 1;
    }
}
```

CONSOLE / OUTPUT

```
recursive helper: x: f32 y: f32 z: f32
literal index:    y
#inline while:    x
#inline while:    y
#inline while:    z
```

The recursive helper is the idiom the standard library uses. It carries the type and index as #comptime parameters. A #if branch stops recursion before checking an out-of-range field index, so each live intrinsic call folds at its specialization. It also composes: the helper returns text, which is what `#insert` wants. `#inline while` works because the counter is declared `i ::= 0`, a compile-time binding; declaring it `i := 0` is `#inline while condition must be compile-time evaluable`.

Optionals, results, vectors and matrices

`?T` and `E!T` are compiler-baked unions, so reflection sees them as unions with two variants apiece. `Simd` and `Matrix` are flagged array types with tags of their own and shape intrinsics of their own.

GLOSSO

```
#import "Io";
#import "Meta";
#import "Simd";
#import "Math/Matrix";

Maybe    :: ?s32;
Fallible :: string!s32;
V4       :: Simd(f32, 4);
M23      :: Matrix(f32, 2, 3);

main :: () {
    print("?s32        kind % variants % : %:% %:%\n",
        cast(s64) type_kind(Maybe), variant_count(Maybe),
        variant_name(Maybe, 0), type_name(variant_type(Maybe, 0)),
        variant_name(Maybe, 1), type_name(variant_type(Maybe, 1)));
    print("string!s32  kind % variants % : %:% %:%\n",
        cast(s64) type_kind(Fallible), variant_count(Fallible),
        variant_name(Fallible, 0), type_name(variant_type(Fallible, 0)),
        variant_name(Fallible, 1), type_name(variant_type(Fallible, 1)));
    print("Simd(f32,4) kind % lanes % element %\n",
        cast(s64) type_kind(V4), simd_lane_count(V4),
        type_name(simd_element_type(V4)));
    print("Matrix      kind % rows % cols % elements % transpose %\n",
        cast(s64) type_kind(M23), matrix_row_count(M23),
        matrix_column_count(M23), matrix_element_count(M23),
        type_name(matrix_transpose_type(M23)));
    print("names: % % %\n",
        type_name(Maybe), type_name(Fallible), type_name(V4));
}
```

CONSOLE / OUTPUT

```
?s32        kind 9 variants 2 : None:void Some:s32
string!s32  kind 9 variants 2 : Ok:s32 Err:string
Simd(f32,4) kind 16 lanes 4 element f32
Matrix      kind 17 rows 2 cols 3 elements 6 transpose Matrix(f32, 3, 2)
names: ?s32 string!s32 Simd(f32, 4)
```

- `?T` has `None: void` at index `0` and `Some: T` at index `1`. `E!T` has `Ok: T` at index `0` and `Err: E` at index `1`, so the variant order is the reverse of the spelling.
- `type_name` of an optional or a result gives its source spelling, such as `?s32` and `string!s32`. Nested types include parentheses where needed, as in `?(string!s64)`.

- Structural types also use source spellings: `[]s32`, `[*]u8`, `(s64, string)`, and `(s32) -> s32`. Instantiated aggregates include their arguments, such as `Box(s64)`. `Type_Info.name` and the static record carried by `any` agree with `type_name` in native and compile-time execution.
- Native and fixed integers retain their identities, so `variant_type` on a `box: s64` variant reports `s64`, while one declared `box: ssize` reports `ssize`. A tuple has tag `Tuple` but `field_count` of a tuple type is `0`; its components are not reachable through the field intrinsics.

Generating code from a type

Reflection pays off when its answers become source. The helper below uses a compile-time index to emit one print per field and splices the result, so a single definition prints any struct without a `Show` instance and without a runtime type table.

GLOSSO

```
#import "Format";
#import "Io";
#import "Meta";
#import "String_Builder";

Vec3 :: struct { x: f32 = 0.0; y: f32 = 0.0; z: f32 = 0.0; }
Span :: struct { lo: s64 = 0; hi: s64 = 0; }

describe :: (value: $T) {
    print("% {\n", type_name(T));
    i ::= 0;
    #inline while i < field_count(T) {
        name :: field_name(T, i);
        #insert tprint("print(\"  % = \\%\\n\", `value.%);", name, name);
        i += 1;
    }
    print("}\n");
}

main :: () {
    describe(Vec3.{ .x = 1.0, .y = 2.0, .z = 3.0 });
    describe(Span.{ .lo = 4, .hi = 9 });
}
```

CONSOLE / OUTPUT

```
Vec3 {
  x = 1
  y = 2
  z = 3
}
Span {
  lo = 4
  hi = 9
}
```

`describe` is instantiated once per argument type and each instantiation splices its own generated statements, which --show-generated reports. The inline while index is a compile-time binding. T is a type parameter, so the reflection calls fold inside the generic body; `value` is an ordinary parameter. The generated source spells `value with a backtick to resolve that name in the insertion site rather than the generator.

> **NOTE**
> Generating source is the general answer to a per-field or per-variant job, and it is often the only one. There is no way to index a struct's fields at run time, so a loop over `field_count(T)` has to be unrolled into text or into `#inline while` before it means anything.

> **NOTE**
> Reflection follows the visibility permissions of its defining source file. Inaccessible fields and variants are omitted from reflected member lists, without changing physical layout or discriminants. Generic bodies and default expressions keep the permissions of their defining file after specialization; they do not gain access to caller-private fields.

11.9 Pattern Reflection

A `#pattern` written with no `= subject` and no `==` switch after it is a quote: it produces a value instead of performing a match. The value has the built-in type `Pattern`, and it can be stored in a constant, passed to a procedure, returned from one, and spliced back into pattern position with `#insert`. Splicing is covered in Patterns as Compile-Time Values; this chapter covers the intrinsics that ask a quote questions.

`Pattern` is a compiler built-in name and needs no import, as `type` and `Code` do. The six intrinsics that read a quote are declared in `std/Meta/Utilities.glo` and need `#import "Meta";`. Every one of them is compile-time only: the checker folds the call to a constant while it still has the types in hand, so the call may be written in an ordinary runtime expression but nothing of it survives to run time.

GLOSSO

```
#import "Io";
#import "Meta";

Token :: union { Number: f64; Name: string; Eof: void; }

NUMBER :: #pattern Token.Number(n);
NAME   :: #pattern Token.Name(text);

main :: () {
    print("binders of NUMBER: %\n", pattern_binder_count(Token, NUMBER));
    print("  % : %\n", pattern_binder_name(Token, NUMBER, 0),
                      type_name(pattern_binder_type(Token, NUMBER, 0)));
    print("uncovered by NUMBER and NAME: %\n",
          pattern_missing_count(Token, .[NUMBER, NAME]));
    print("  %\n", pattern_missing(Token, .[NUMBER, NAME], 0));
}
```

CONSOLE / OUTPUT

```
binders of NUMBER: 1
  n : f64
uncovered by NUMBER and NAME: 1
  Token.Eof
```

A quote has no runtime representation, so it must be bound with `::` rather than `:=`. `P := #pattern 1;` is `error: a 'Pattern' value is compile-time only; bind it with '::' rather than ':='`. The same rule governs `Code` and the other compiler-only meta-values, and it is why the pattern parameters of the intrinsics are declared `#comptime`.

The intrinsics

Call	Result
`pattern_binder_count(T, P)`	How many names `P` brings into scope when it is matched against `T`.
`pattern_binder_name(T, P, i)`	The name of binder `i`, in the order the pattern was written.
`pattern_binder_type(T, P, i)`	The type binder `i` receives.
`pattern_missing_count(T, Ps)`	How many witnesses of `T` no pattern in the view `Ps` matches.
`pattern_missing(T, Ps, i)`	Witness `i`, rendered as source-shaped pattern text.
`pattern_is_useful(T, Ps, Q)`	Whether adding `Q` to `Ps` would match anything `Ps` does not.

The subject type comes first in all six because a pattern's meaning is a property of the pattern and the type it is matched against together, not of the pattern alone: `.Some(v)` binds an `s32` against `?s32` and a `string` against `?string`, and a member name is a union variant for one subject and an enum constant for another. `Ps` is an ordinary `[]Pattern` view, so a set of arms can be declared once as `ARMS : []Pattern : .[A, B, C];` and reused; an untyped `.[A, B]` constant has type `[2]Pattern` and does not convert on its own.

A quote carries no type until it is asked about one

The compiler checks a copy of the quote against the requested subject each time it is asked, and discards whatever that check instantiated. The quote itself is never stamped, so one quote serves several subject types and the binder types follow the subject rather than the quote.

GLOSSO

```
#import "Io";
#import "Meta";

SOME :: #pattern .Some(v);
NONE :: #pattern .None;

main :: () {
    print("?s32    v: %\n", type_name(pattern_binder_type(?s32, SOME, 0)));
    print("?string v: %\n", type_name(pattern_binder_type(?string, SOME, 0)));
    print("?f64    v: %\n", type_name(pattern_binder_type(?f64, SOME, 0)));
    print("uncovered by .Some alone: %\n", pattern_missing_count(?s32, .[SOME]));
    print("  %\n", pattern_missing(?s32, .[SOME], 0));
    print("uncovered by .Some and .None: %\n",
          pattern_missing_count(?s32, .[SOME, NONE]));
}
```

CONSOLE / OUTPUT

```
?s32    v: s32
?string v: string
?f64    v: f64
uncovered by .Some alone: 1
```

```
  Option_s32.None
uncovered by .Some and .None: 0
```

`Option_s32.None` is the internal name of the compiler-baked option union; `?T` is the whole surface syntax for it, and the witness text is the one place the internal name shows through. Because checking happens at the question and not at the declaration, a quote that cannot be matched against the requested subject reports at the quote's own span: `#pattern 0..=9` asked about `f64` is `error: a range pattern needs an integer subject, got f64`, and an alternative that binds different names from its sibling is `error: or-pattern alternative does not bind 'r', which another alternative does`.

Binders

Binders are reported in source order. `_` is not a binder, a name that appears twice is reported once, and `whole @ P` contributes the outer name followed by everything `P` binds. An or-pattern's alternatives must agree on the names they bind, so a usable or-pattern either binds the same set in every alternative or binds nothing at all.

GLOSSO

```
#import "Io";
#import "Meta";

Shape :: union { Circle: f64; Rect: s32; Empty: void; }

BOUND :: #pattern whole @ Shape.Circle(r);
EITHER :: #pattern Shape.Circle(_) | Shape.Empty;

main :: () {
    print("binders of BOUND: %\n", pattern_binder_count(Shape, BOUND));
    print("  % : %\n", pattern_binder_name(Shape, BOUND, 0),
                      type_name(pattern_binder_type(Shape, BOUND, 0)));
    print("  % : %\n", pattern_binder_name(Shape, BOUND, 1),
                      type_name(pattern_binder_type(Shape, BOUND, 1)));
    print("binders of EITHER: %\n", pattern_binder_count(Shape, EITHER));
}
```

CONSOLE / OUTPUT

```
binders of BOUND: 2
  whole : Shape
  r : f64
binders of EITHER: 0
```

An index past the end is `error: pattern binder index out of range`, and the witness accessor reports `error: pattern witness index out of range`. Both are compile errors, since both calls are folded.

Coverage

`pattern_missing_count` and `pattern_missing` run the same Maranget usefulness analysis the compiler uses to decide whether a `#pattern` switch is exhaustive, detached from any switch node. The result is a list of witnesses: values of `T` that no pattern in `Ps` matches, rendered as source-shaped pattern text. No more than 32 witnesses are collected, so a count of 32 means "at least 32".

GLOSSO

```
#import "Io";
#import "Meta";

Color :: enum { Red; Green; Blue; }
Point :: struct { x: s32 = 0; y: s32 = 0; }

RED    :: #pattern Color.Red;
GREEN  :: #pattern Color.Green;
TRUE   :: #pattern true;
ORIGIN :: #pattern Point.{ .x = 0, .y = 0 };
LOW    :: #pattern 0..=9;
HIGH   :: #pattern 10..=255;
TEXT   :: #pattern "hello";

main :: () {
    print("Color after Red, Green: % (%)\n",
          pattern_missing_count(Color, .[RED, GREEN]),
          pattern_missing(Color, .[RED, GREEN], 0));
    print("bool after true: % (%)\n",
          pattern_missing_count(bool, .[TRUE]),
          pattern_missing(bool, .[TRUE], 0));
    print("Point after origin: % (%)\n",
          pattern_missing_count(Point, .[ORIGIN]),
          pattern_missing(Point, .[ORIGIN], 0));
    print("u8 after 0..=9: % (%)\n",
          pattern_missing_count(u8, .[LOW]),
          pattern_missing(u8, .[LOW], 0));
    print("u8 after 0..=9, 10..=255: %\n", pattern_missing_count(u8, .[LOW, HIGH]));
    print("string after \"hello\": % (%)\n",
          pattern_missing_count(string, .[TEXT]),
          pattern_missing(string, .[TEXT], 0));
}
```

CONSOLE / OUTPUT
```
Color after Red, Green: 1 (Color.Blue)
bool after true: 1 (false)
Point after origin: 4 (Point.{ .x = 0, .y = -2147483648..=-1 })
u8 after 0..=9: 1 (10..=255)
u8 after 0..=9, 10..=255: 0
string after "hello": 1 (_)
```

The witnesses show how each type is taken apart. Union variants, enum constants and booleans are enumerated, so the uncovered ones are named directly. A struct has exactly one constructor, so coverage is decided field by field and the report is a cross product over the fields that were pinned down. An integer domain is not enumerated but split into closed intervals, which is why `0..=9` over `u8` leaves the single witness `10..=255` and why adding `10..=255` leaves none. Floats, strings and pointers have no enumerable structure at all, so only a wildcard covers them and the witness is the bare `_`.

> **NOTE**
> The analysis errs in one direction only. It may report a case missing that a reader can see is covered, and it never reports zero for a set of patterns that is not total. So `pattern_missing_count(T, Ps) == 0` is a genuine proof of totality, and a non-zero answer means "not proved", not "definitely reachable". That is the same contract the compiler's own switches are held to.

Three constructs sit outside the analysis by design, and each of them is a source of witnesses a reader would not have expected.

- Where the same analysis runs over a switch, guarded, predicate and `#falling` arms are excluded from the matrix, because every one of them can decline a value after matching its shape.
- A raw union carries no tag, so its values cannot be discriminated at all and nothing about it can be proved.
- An or-pattern nested inside a constructor is opaque unless one alternative is a wildcard. Over `Shape`, adding `#pattern Shape.Rect(-2147483648..=2147483647)` to the other two variants leaves nothing uncovered, while the equivalent `#pattern Shape.Rect(-2147483648..=-1 | 0..=2147483647)` still reports one witness.

Usefulness, subsumption, and equivalence

`pattern_is_useful(T, Ps, Q)` answers the other half of the same algorithm: does `Q` match anything `Ps` does not? A `false` answer is exactly "`Q` would be a dead arm after `Ps`", which is also "`Ps` subsumes `Q`". Two patterns are equivalent when neither is useful against the other. That relation is what a reader usually means by pattern equality, and it is why `Pattern` has no `==`: whether `A | B` equals `B | A`, or whether `_` equals a binder, has no structural answer, while mutual non-usefulness has one.

GLOSSO
```
#import "Io";
#import "Meta";

Shape :: union { Circle: f64; Rect: s32; Empty: void; }

A :: #pattern Shape.Circle(_) | Shape.Empty;
B :: #pattern Shape.Empty | Shape.Circle(_);
C :: #pattern Shape.Circle(r);
W :: #pattern _;

equivalent :: (T: type, #comptime P: Pattern, #comptime Q: Pattern) -> bool {
    return !pattern_is_useful(T, .[P], Q) && !pattern_is_useful(T, .[Q], P);
}
subsumes :: (T: type, #comptime P: Pattern, #comptime Q: Pattern) -> bool {
    return !pattern_is_useful(T, .[P], Q);
}

main :: () {
    print("A equivalent B = %\n", #comptime equivalent(Shape, A, B));
    print("A equivalent C = %\n", #comptime equivalent(Shape, A, C));
    print("A subsumes C   = %\n", #comptime subsumes(Shape, A, C));
    print("C subsumes A   = %\n", #comptime subsumes(Shape, C, A));
    print("W subsumes A   = %\n", #comptime subsumes(Shape, W, A));
    print("A subsumes W   = %\n", #comptime subsumes(Shape, A, W));
}
```

CONSOLE / OUTPUT
```
A equivalent B = 1
A equivalent C = 0
A subsumes C   = 1
C subsumes A   = 0
W subsumes A   = 1
A subsumes W   = 0
```

Computing with the answers

The index arguments are ordinary `ssize` parameters, but they must still be compile-time constants, because the whole call is folded during checking. A `while` loop that walks the witnesses with a runtime counter is `error: unknown const i`. The

working shape is a recursive compile-time helper that carries the index as an argument, the same idiom type reflection uses and for the same reason; it is described in Reflection.

GLOSSO

```
#import "Format";
#import "Io";
#import "Meta";
#import "String_Builder";

Token :: union { Number: f64; Name: string; Eof: void; }

NUMBER :: #pattern Token.Number(n);
NAME   :: #pattern Token.Name(text);
ARMS : []mut Pattern : .[NUMBER, NAME];

witness_list :: (T: type, Ps: []mut Pattern, i: ssize) -> string {
    if i >= pattern_missing_count(T, Ps) return "";
    tail := witness_list(T, Ps, i + 1);
    if tail == "" return pattern_missing(T, Ps, i);
    return tprint("%, %", pattern_missing(T, Ps, i), tail);
}

REPORT :: #comptime witness_list(Token, ARMS, 0);

main :: () {
    #if REPORT != "" { #compile_error tprint("switch is not total; uncovered: %", REPORT); }
    print("ok\n");
}
```

CONSOLE / OUTPUT

```
error: switch is not total; uncovered: Token.Eof
   --> …\total.glo:21:24
    |
21 |     #if REPORT != "" { #compile_error tprint("switch is not total; uncovered: %", REPORT); }
    |                        ^^^^^^^^^^^^^^
```

A shorter obligation is `#assert #comptime pattern_missing_count(Token, ARMS) == 0, "the arm list is not total";`, which fails as `error: assertion failed: the arm list is not total`. Once the proof holds, the same constants can drive the switch through `#insert`, so the arms that were proved total are literally the arms that run.

GLOSSO

```
#import "Io";
#import "Meta";

Token :: union { Number: f64; Name: string; Eof: void; }

NUMBER :: #pattern Token.Number(n);
NAME   :: #pattern Token.Name(text);
EOF    :: #pattern Token.Eof;
ARMS : []mut Pattern : .[NUMBER, NAME, EOF];

classify :: (t: Token) -> string {
    #assert #comptime pattern_missing_count(Token, ARMS) == 0, "classify is not total";
    if #pattern t == {
    case #insert NUMBER; return "number";
    case #insert NAME;   return "name";
    case #insert EOF;    return "eof";
    }
}

main :: () {
    print("%\n", classify(Token.Number(1.5)));
    print("%\n", classify(Token.Name("x")));
    print("%\n", classify(Token.Eof));
}
```

CONSOLE / OUTPUT

```
number
name
eof
```

11.10 Build Scripts

A build script is an ordinary Glosso file whose work happens in a top-level `#comptime`. `#import "Compiler";` brings in the build API, the `#comptime` declaration runs it once during compilation, and `main :: () {}` is still present because the file is a program like any other. Nothing in the API is available at run time; every operation is a compiler service reached from compile-time execution.

GLOSSO

```
#import "Io";
#import "Compiler";
#import "Typeclass";

run_build :: () {
    set_build_options_dc(.do_output = false);

    workspace := Workspace.create("hello");
    options := workspace.get_build_options();
    options.output_type = .Executable;
    options.output_executable_name = "hello.exe";
    options.set_optimization(.Optimized);
    options.set_build_options(workspace);

    if add_build_file("hello.glo", workspace) == {
    case .Ok(log);
        print("[build] nested compilation said:\n%", log);
    case .Err(failure);
        print("[build] failed with status %\n", failure.status);
        report_error(failure);
        exit(failure.status);
    }
}

#comptime run_build();
main :: () {}
```

CONSOLE / OUTPUT

```
$ glosso build.glo
glosso: compile build.glo (target=x86_64-pc-windows-msvc)
[build] nested compilation said:
glosso: compile hello.glo (target=x86_64-pc-windows-msvc)
glosso: link hello.exe (target=x86_64-pc-windows-msvc, -O2)
glosso: done hello.exe  (backend=llvm, target=x86_64-pc-windows-msvc, -O2)
$ ./hello.exe
hello from the nested build
```

`set_build_options_dc(.do_output = false)` suppresses the emit step of the compilation that is running the script, so no `build.exe` appears. Without it the driver would also link the build script itself, which is almost never wanted: the script's `main` is empty and its whole purpose has already been served. The name means "during compile", and it is the only setter that changes the current compilation rather than a nested one.

A nested compilation is a new `glosso` process, taken from `GLOSSO_COMPILER_EXE` if that is set and from the running executable otherwise. `add_build_file` and `add_build_string` wait for that child and return its standard output followed by its standard error instead of passing either stream through. That is why the child's transcript appears under `[build]` above and why discarding the `.Ok` payload discards the child's log. A failed child returns `Compiler_Error`; that type implements `Error`, so `report_error(failure)` prints `compiler error:` followed by the captured diagnostics.

`Workspace` and `Build_Options`

A `Workspace` is a `distinct s64` handle to one reusable `Build_Options` configuration. Create one with `Workspace.create`, obtain a detached copy of its options with `Workspace.get_build_options`, edit that copy, and install it with `Build_Options.set_build_options`. Nothing is queued: `add_build_file(path, workspace)` and `add_build_string(source, workspace)` each start and await one child compilation immediately. Omitting the workspace argument uses the shared implicit handle `-1`.

Field	Default	Child argument
`output_type`	`.Executable`	Selects the backend when `backend` is `.Default`; `.No_Output` omits `-o` but does not suppress the child's artifact.
`backend`	`.Default`	`--backend` with the matching name, or derive it from `output_type`.
`output_executable_name`	empty	The name passed through `-o`; empty falls back to the source stem.
`output_path`	empty	Directory joined to the output name; empty uses the working directory.
`target`	zeroed	An all-zero target inherits the caller; raw `compiler_args` may

Field	Default	Child argument
		override it, while a nonzero field is appended last and wins.
`optimization_level`	`.Default`	One optimization flag; see the table below.
`compiler_args`	empty	Raw child arguments before the flags derived from this record.
`import_path`	empty	One `--module-path` per entry, after the standard library in search order.
`linker_args`	empty	One `--linker-arg` per entry.
`compile_time_command_line`	empty	Entries after `--`, preserving spaces and empty strings as individual arguments.
`memory_sanitizer`, `thread_sanitizer`	`false`	`--memory-sanitizer` and `--thread-sanitizer`.
`borrow_check`	`Compiler_Borrow_Check.Warning`	`--borrow-check=off`, `warning`, or `error` for the child compilation.
`color`	`.Auto`	`--color always` or `--color never`, resolved against the parent's stderr.
`temp_allocator_size`	4 MiB	`--temp-allocator-size <bytes>` when positive.
`no_main`	`false`	`--no-main`.
`intermediate_path`, `do_output`, `write_added_strings`, `text_output_flags`	reserved defaults	Currently ignored by workspace builds.

`.Auto` never reaches the child as `auto`. The child's standard error is a capture pipe and can never be a terminal, so the parent resolves `auto` against its own stderr and passes the answer. A script that wants plain text in the captured log sets `.Never` explicitly.

The fields are written through mutators that take a pointer to the record: `Build_Options.set_backend`, `Build_Options.set_target`, `Build_Options.set_optimization`, `Build_Options.set_memory_sanitizer`, `Build_Options.set_thread_sanitizer`, `Build_Options.set_color`, `Build_Options.set_temp_allocator_size`, `Build_Options.set_no_main`, `Build_Options.add_compiler_arg`, `Build_Options.add_linker_arg`, `Build_Options.add_linker_args` and `Build_Options.add_module_path`. They mutate only the detached local record; call `Build_Options.set_build_options` afterwards. Direct field assignment and `array.push` on its dynamic-array fields are equivalent.

`Compiler_Backend` has seven members. `LLVM`, `Object`, `LLVM_IR`, `Static_Lib` and `Shared_Lib` map to `--backend llvm`, `object`, `llvm-ir`, `static-lib` and `shared-lib`. `Default` passes no `--backend` and lets the output type decide. `X64` maps to nothing at all and behaves exactly like `Default`; it is a dead value.

> **NOTE**
> `Build_Output_Type` supplies the default backend independently of the file name: `.Executable`, `.Object_File`, `.LLVM_IR`, `.Static_Library` and `.Shared_Library` select the corresponding artifact. `.No_Output` means "do not pass `-o`"; the child still emits to its own source-derived default path. To suppress the build script's own artifact, use `set_build_options_dc(.do_output = false)` instead.

Optimization levels

`Optimization` is an explicitly numbered enum because its values are part of the compile-time service ABI. `Default` is not a synonym for `Very_Debug`: it passes no optimization flag at all, and the child then applies its own default, which is `-O1`.

Member	Value	Child flag and current result
`Debug`	0	`-O0`; runtime checks stay enabled
`Very_Debug`	1	`-O1`; runtime checks stay enabled
`Optimized`	2	`-O2`

Member	Value	Child flag and current result
`Very_Optimized`	3	`-O3`
`Optimized_Small`	4	`-Os`; LLVM size optimization and dead-section removal
`Optimized_Very_Small`	5	`-Oz`; aggressive size optimization
`Optimized_Tiny`	6	`-Ozz`; reaches the same rejected `default<Oz>` pipeline before stripping
`Default`	7	none; child defaults to working `-O1`

NOTE
The size modes -Os, -Oz, and -Ozz produce runnable artifacts. They combine LLVM optimization with optsize/minsize attributes and target-specific linker size policies. Runtime checks are disabled in these modes; use -O0 or -O1 for checked builds. -Ozz also strips symbols and metadata.

Named and implicit workspaces

`Workspace.create(name)` returns fresh handles numbered from zero for the life of the compiler process. The name is retained but is otherwise informational; it need not be unique, and there is no destroy operation. Each new handle starts with independent default options. Handle `-1` is reserved for the process-wide implicit workspace used by every call that omits its workspace argument. Reading any other unknown handle materializes it with defaults rather than reporting an error.

GLOSSO
```
#import "Io";
#import "Compiler";

drive :: () {
    set_build_options_dc(.do_output = false);

    implicit: Workspace = -1;
    defaults := implicit.get_build_options();
    print("output_type=% backend=% optimization=% do_output=%\n",
          defaults.output_type, defaults.backend,
          defaults.optimization_level, defaults.do_output);

    space := Workspace.create("tool");
    options := space.get_build_options();
    options.output_executable_name = "tool.exe";
    options.set_optimization(.Optimized);
    options.set_build_options(space);

    if add_build_file("hello.glo", space) == {
    case .Ok(log);      print("[ws] built:\n%", log);
    case .Err(failure); print("[ws] failed: %\n", failure.message);
    }
}

#comptime drive();
main :: () {}
```

CONSOLE / OUTPUT
```
output_type=1 backend=0 optimization=7 do_output=1
[ws] built:
glosso: compile hello.glo (target=x86_64-pc-windows-msvc)
glosso: link tool.exe (target=x86_64-pc-windows-msvc, -O2)
glosso: done tool.exe  (backend=llvm, target=x86_64-pc-windows-msvc, -O2)
```

Enum values print as their numbers, so the defaults read as `output_type = Executable`, `backend = Default` and `optimization_level = Default`. The output name comes from `output_executable_name`, joined onto `output_path` when that is set, and falls back to the source's file stem. The whole name is passed through as `-o`, so a fallback name carries no platform extension: a workspace with only `output_path = "outdir"` set produces `glosso: done outdir\hello`. `add_build_string(source, w)` is the same operation over text: the source is written to a temporary file named `glosso_build_string_<pid>_<n>.glo` and that file is compiled and left in place, which is why its diagnostics name a path in the system temporary directory. Give it an explicit output name when the temporary source stem would not be a useful artifact name.

NOTE
`compile_time_command_line` is active: a nonempty list is appended after `--`, and the child reads exactly those entries through `get_cmdline_args()`. Four other fields are currently reserved: `intermediate_path`, `do_output`, `write_added_strings`, and

> `text_output_flags`. In particular, changing `Build_Options.do_output` does not suppress either compilation, and `write_added_strings = false` does not stop `add_build_string` from writing its temporary source.

Building C sources

`Native_Compile_Output.compile_c_archive(name, sources)` compiles a list of C files with the compiler's bundled native toolchain and returns `Native_Compile_Output`, whose `artifact` field is the path of the static library it produced and whose `output` field is the tool's log. The archive is written into the native cache, keyed by the archive name, target, parameters, and verified source/header dependencies. If dependency inputs cannot be verified, the archive uses a fresh `uncached-...` directory and is rebuilt. Pass `include_paths`, `preprocessor_definitions`, and `compiler_args` as named `[]mut string` parameters, which default to `.[]`. Their slices and strings are borrowed for the call. `sources` is also `[]mut string`; pass a fixed or dynamic array's `[..]` view. `position_independent` defaults to `false`, and `optimization_level` defaults to `.Optimized`; only `.Debug`, `.Very_Debug`, `.Optimized`, and `.Very_Optimized` are accepted. The input option record and its specialized mutators are no longer public APIs.

GLOSSO

```
#import "Format";
#import "Io";
#import "Compiler";
#import "Strings/Utf8";
#import "String_Builder";

adder_bindings :: () -> string {
    sources: [..]string;
    sources.push("adder.c");

    arguments: [1]string = .["-fno-builtin"];

    if Native_Compile_Output.compile_c_archive("adder", sources[..], .compiler_args = arguments[..]) == {
    case .Ok(archive);
        print("[c] archive at %\n", archive.artifact);
        slashes := archive.artifact.replace("\\", "\\\\");
        defer slashes.free();
        escaped := slashes.replace("\"", "\\\"");
        defer escaped.free();
        return tprint("adder :: #library,static \"%\";\nc_add :: (a: s32, b: s32) -> s32 #foreign adder \"c_add\";", escaped);
    case .Err(failure);
        return "#comptime { #compile_error \"the adder archive did not build\"; }";
    }
}

#insert #comptime adder_bindings();

main :: () { print("c_add(3, 4) = %\n", c_add(3, 4)); }
```

CONSOLE / OUTPUT

```
glosso: compile archive.glo (target=x86_64-pc-windows-msvc)
[c] archive at C:\Users\…\glosso\cache\native\v9\adder\x86_64-windows-msvc\…\adder.lib
glosso: link archive.exe (target=x86_64-pc-windows-msvc, -O1)
glosso: done archive.exe  (backend=llvm, target=x86_64-pc-windows-msvc, -O1)
c_add(3, 4) = 7
```

The `#library` declaration and the `#foreign` declaration have to be inserted together. A top-level string `#insert` that declares only the library, with the `#foreign` procedure written in the file, is `error: #foreign refers to unknown library 'adder' (declare it with #library)`. Handing `archive.artifact` to Bindgen's `library_spec` and inserting the generated source does the same job for a real header; that route is covered in Generating Bindings with Bindgen. A path inserted into a quoted source literal must escape backslashes and quotes, as above; printing a native Windows path verbatim does not produce a valid Glosso string literal.

Other compiler services

> **NOTE**
>
> `macro_unit(path, argument)` is the native-code route for a reusable source generator. It builds `path` as a program through a nested compiler, benefits from the native artifact cache, runs the result with `argument` after `--`, and returns the program's standard output as source text. Write `#insert macro_unit("macros/derive_json.glo", "record");` at the use site. A build or run failure is returned as diagnostic text so the following insertion fails with the generator's error visible.

The build-driver idiom

The shape every real build script converges on is a single top-level `#comptime` that turns off its own output, reads a task name from the compile-time command line, and dispatches. This manual is built that way: `glosso first.glo -- build` compiles the manual's `main.glo` for `wasm32-freestanding`, and other task names regenerate its reference data and its syntax highlighting.

GLOSSO

```
#import "Io";
#import "Compiler";
#import "Typeclass";

build_release :: () {
    workspace := Workspace.create("release");
    options := workspace.get_build_options();
    options.output_executable_name = "hello.exe";
    options.set_optimization(.Optimized);
    options.set_color(.Always);
    options.set_build_options(workspace);
    if #pattern .Err(failure) = add_build_file("hello.glo", workspace) {
        print("release build failed:\n");
        report_error(failure);
        exit(failure.status);
    }
    print("built hello.exe\n");
}

usage :: () {
    print("usage: glosso first.glo -- <command>\n");
    print("commands:\n");
    print("  release   build hello.glo optimized\n");
}

run_task :: () {
    set_build_options_dc(.do_output = false);
    args := get_cmdline_args();
    if args.count == 0 { usage(); exit(1); }
    if args[0] == {
    case "release"; build_release(); return;
    else;
        print("unknown task: %\n", args[0]);
        usage();
        exit(1);
    }
}

#comptime run_task();
main :: () {}
```

CONSOLE / OUTPUT

```
$ glosso first.glo
glosso: compile first.glo (target=x86_64-pc-windows-msvc)
usage: glosso first.glo -- <command>
commands:
  release   build hello.glo optimized
$ echo $?
1
$ glosso first.glo -- release
glosso: compile first.glo (target=x86_64-pc-windows-msvc)
built hello.exe
$ glosso first.glo -- deploy
glosso: compile first.glo (target=x86_64-pc-windows-msvc)
unknown task: deploy
usage: glosso first.glo -- <command>
commands:
  release   build hello.glo optimized
```

`exit(status)` from compile-time execution stops the build with that status and writes no binary, which is what makes a build script usable from a shell script or a continuous-integration job. Because the driver emits nothing, the run above produces `hello.exe` and no `first.exe`. Cross-compiling a nested build is a matter of one more setter: `options.set_target(.{ .os = Target_OS.Wasm, .cpu = Target_CPU.Wasm32, .abi = Target_ABI.Freestanding })` makes the child report `target=wasm32-unknown-unknown`. That changes only children using this workspace. `Native_Compile_Output.compile_c_archive` still uses the build script's active target; compile the script itself for the matching target when its C archive must be link-compatible with the nested Glosso artifact.

11.11 The Compile-Time Command Line and Artifact Cache

`glosso [options] <file.glo> -- <args...>` hands everything after the `--` separator to compile-time execution rather than to the driver. The separator itself, and the fact that `get_cmdline_args()` returns the process argument vector in the built program, are covered in Using the Compiler. What matters here is the compile-time half: the arguments are how a build script written in Glosso is parameterized, and they are part of the identity of the artifact the build produces.

GLOSSO

```
#import "Io";
#import "Compiler";

read_options :: () {
    set_build_options_dc(.do_output = false);
    args := get_cmdline_args();
    output := "app.exe";
```

```
    level := Optimization.Default;
    i := 0;
    while i < args.count {
        if args[i] == {
        case "--release"; level = .Optimized;
        case "--out";
            if i + 1 >= args.count {
                print("--out needs a path\n");
                exit(2);
            }
            output = args[i + 1];
            i += 1;
        else;
            print("unknown option: %\n", args[i]);
            exit(2);
        }
        i += 1;
    }
    print("[ct] output = %  optimization = %\n", output, level);
}

#comptime read_options();
main :: () {}
```

CONSOLE / OUTPUT

```
$ glosso opts.glo -- --release --out dist/tool.exe
glosso: compile opts.glo (target=x86_64-pc-windows-msvc)
[ct] output = dist/tool.exe  optimization = 2
$ glosso opts.glo -- --fast
glosso: compile opts.glo (target=x86_64-pc-windows-msvc)
unknown option: --fast
$ echo $?
2
```

A rejected option can end the build immediately because `exit` from compile-time execution aborts the compilation with the requested status and writes no output file. Compile-time `print` output is buffered and flushed to standard output in one block after every top-level `#comptime` has run, while the compiler's own progress lines go to standard error, so the interleaving of the two in a terminal is not something to depend on.

Forwarding arguments to a workspace build

A child started by `add_build_file` or `add_build_string` does not inherit the build script's own `--` arguments automatically. Put the arguments for that child in `Build_Options.compile_time_command_line`, then install the options before starting the build. The service writes one `--` separator followed by the list; each element remains one argument, even when it contains spaces or is empty.

GLOSSO

```
#import "Io";
#import "Compiler";
#import "Typeclass";

drive :: () {
    set_build_options_dc(.do_output = false);
    print("[parent] % argument(s)\n", get_cmdline_args().count);

    workspace := Workspace.create("child");
    options := workspace.get_build_options();
    options.output_executable_name = "child.exe";
    options.compile_time_command_line.push("release");
    options.set_build_options(workspace);

    if add_build_file("child.glo", workspace) == {
    case .Ok(log);      print("%", log);
    case .Err(failure); report_error(failure);
    }
}

#comptime drive();
main :: () {}
```

CONSOLE / OUTPUT

```
$ glosso parent.glo -- top
glosso: compile parent.glo (target=x86_64-pc-windows-msvc)
[parent] 1 argument(s)
[child] 1 argument(s)
[child]   release
glosso: compile child.glo (target=x86_64-pc-windows-msvc)
glosso: link child.exe (target=x86_64-pc-windows-msvc, -O1)
glosso: done child.exe  (backend=llvm, target=x86_64-pc-windows-msvc, -O1)
```

The native artifact cache

When a compilation would produce a native artifact and nothing about it has changed, the driver copies the previous result out of a per-user cache instead of running the pipeline. The restore is announced, and it replaces every phase after argument parsing, so no `compile` or `link` line appears at all.

GLOSSO

```
#import "Io";
main :: () { print("%\n", 6 * 7); }
```

CONSOLE / OUTPUT

```
$ glosso --run sum.glo
glosso: compile sum.glo (target=x86_64-pc-windows-msvc)
glosso: link sum.exe (target=x86_64-pc-windows-msvc, -O1)
glosso: done sum.exe  (backend=llvm, target=x86_64-pc-windows-msvc, -O1)
42
$ glosso --run sum.glo
glosso: restored sum.exe from the native artifact cache
42
$ glosso --run sum.glo -- release
glosso: compile sum.glo (target=x86_64-pc-windows-msvc)
glosso: link sum.exe (target=x86_64-pc-windows-msvc, -O1)
glosso: done sum.exe  (backend=llvm, target=x86_64-pc-windows-msvc, -O1)
42
$ glosso -O2 --run sum.glo
glosso: compile sum.glo (target=x86_64-pc-windows-msvc)
glosso: link sum.exe (target=x86_64-pc-windows-msvc, -O2)
glosso: done sum.exe  (backend=llvm, target=x86_64-pc-windows-msvc, -O2)
42
```

The key

The key is a hash of an identity string assembled from everything that could change the bytes of the artifact. Adding `--release` to an otherwise identical command line produced a third cache entry above, because a build script can read those words and emit something different for them.

Group	What goes into the identity
Sources	The canonical input and compiler paths, the working directory, and the resolved standard-library directory.
Target	The LLVM triple, and separately every key and value of `--target`, because the triple collapses distinct maps such as `uefi` and `msvc`.
Code generation	Backend name, optimization level, size mode, sanitizer selection, and diagnostic color.
Runtime	The temporary allocator size, `GLOSSO_DEBUG_ALLOCATOR` selection, `--no-main`, and `--use-std`.
Linking	Every `--linker-arg`, in order.
Compile time	Every argument after `--`, in order.
Modules	Every `--module-path` directory, in order, and explicit `--unit-source` mappings.

The output path is deliberately absent. Building the same source with `-o other.exe` restores the cached artifact and copies it to the new name: `glosso: restored other.exe from the native artifact cache`. The key is 32 hexadecimal characters. An entry holds `<key>.artifact`, `<key>.manifest`, `<key>.stdout`, and `<key>.stderr` under `<cache>/native/artifacts/v1`. A hit replays the stored checker output and diagnostics. Missing output sidecars turn the entry into a miss; new entries use temporary files and publish their manifest last.

The manifest

The key decides which entry to look at; the manifest decides whether it is still valid. It is a small binary file beginning with the magic `GLOART01`, followed by a count and then one record per dependency: the path, the file's length, and its modification time in seconds and nanoseconds. The artifact is restored only if every stamp still matches. The dependency list is the source file, every file reached through `#load` and `#import`, the compiler executable itself, and any `--linker-arg` that names an existing file.

GLOSSO

```
// uses.glo
#import "Io";
#load "helper.glo";
main :: () { print("%\n", greeting()); }
```

```
// helper.glo
greeting :: () -> string { return "one"; }
```

CONSOLE / OUTPUT

```
$ glosso --run uses.glo
glosso: compile uses.glo (target=x86_64-pc-windows-msvc)
glosso: link uses.exe (target=x86_64-pc-windows-msvc, -O1)
glosso: done uses.exe  (backend=llvm, target=x86_64-pc-windows-msvc, -O1)
one
$ glosso --run uses.glo
glosso: restored uses.exe from the native artifact cache
one
$ # helper.glo edited to return "two"
$ glosso --run uses.glo
glosso: compile uses.glo (target=x86_64-pc-windows-msvc)
glosso: link uses.exe (target=x86_64-pc-windows-msvc, -O1)
glosso: done uses.exe  (backend=llvm, target=x86_64-pc-windows-msvc, -O1)
two
```

NOTE

Validation is by length and modification time, not by content hash. Restoring a checkout, copying a tree, or any operation that rewrites a file's timestamp without changing its bytes therefore forces a rebuild rather than a stale restore, which is the safe direction. A tool that preserves timestamps while changing content is the unsafe direction and has no defence here.

When caching is skipped

A cache miss costs a compile; a wrong cache hit costs correctness. The driver therefore declines the cache entirely in every case where it cannot account for what the build did.

- Only the executable, object and static-library backends are cached. `--backend llvm-ir` and `--backend shared-lib` recompile every time.
- A program with any top-level `#comptime` is never stored, because that code can read files, run commands and write outputs that the manifest does not describe. This is why a build script recompiles on every invocation, and why a build script must not be made expensive to check.
- `--show-generated` disables the cache for that run, so the generated `#insert` sources it reports are actually produced rather than skipped.
- Setting the environment variable `GLOSSO_DISABLE_ARTIFACT_CACHE` to anything turns the cache off for the process.
- A build that emitted nothing is not stored: a compile-time `exit`, or `set_build_options_dc(.do_output = false)`, both end the run before the store step.

A failure to write an entry is a warning and not an error: `glosso: warning: could not update artifact cache: <reason>`, after which the build finishes normally. Entries are never removed by the compiler; the cache directory grows until it is deleted by hand.

`--cache-dir` and `--print-cache-dir`

--print-cache-dir prints the resolved root and exits without reading source. --cache-dir overrides that root; a relative path is resolved against the working directory. The override moves generated sources, native C archives, and final-artifact entries together. For example, glosso --cache-dir build/cache --print-cache-dir selects a project cache.

With no override the root is chosen from the host convention, and the environment variable `GLOSSO_CACHE_DIR` overrides that choice without a flag. It too must hold an absolute path.

Host	Default cache root
Windows	`%LOCALAPPDATA%\glosso\cache`
macOS	`~/Library/Caches/glosso`
Other Unix	`$XDG_CACHE_HOME/glosso`, or `~/.cache/glosso` when that is unset

12 Memory and context

12.1 The Implicit Context

Every procedure body sees one implicit value named `context`. It takes no import, no parameter and no declaration: the compiler creates the type, creates one instance of it, and makes the name visible everywhere. Allocation, logging and random-number generation all read their policy out of that one value, which is how a caller redirects a library's behaviour without changing a single signature the library declares.

The type is built into the compiler rather than written in a `.glo` file, and its spelling carries the directive sigil to say so: it is `#Context`. The definition is fixed and has six fields.

```
GLOSSO
Context :: struct {
    allocator:        Allocator;
    temp_allocator:   Allocator;
    logger:           Logger;
    random_generator: Random_Generator;
    user_ptr:         ?*mut void;
    user_index:       ssize;
}
```

Field	What reads it
`allocator`	`alloc`, `free`, `array.push`, `format`, and every standard-library call that defaults its allocator parameter
`temp_allocator`	`tprint`, and everything else that returns arena-backed storage
`logger`	structured logging; the vtable is zeroed in the default context
`random_generator`	the random-number vtable; also zeroed in the default context
`user_ptr`	nothing; reserved for the program
`user_index`	nothing; reserved for the program

`user_ptr` and `user_index` exist so that a program can thread its own state through code it does not own, in the same way the allocator is threaded. Nothing in the compiler or the standard library reads them.

Reading and writing `context`

`context` is an ordinary lvalue of type `#Context`. Its fields can be read, assigned, copied into a local and passed to a procedure that declares a `#Context` parameter. A bare assignment to a field is permanent: the context is one process-wide global, not a per-call record, so nothing restores it later.

```
GLOSSO
with_index :: (base: #Context, index: ssize) -> #Context {
    out := base;
    out.user_index = index;
    return out;
}

main :: () {
    context.user_index = 7;
    later := with_index(context, 9);
}
```

`#push_context`

`#push_context expr body` installs `expr` as the whole context for the duration of `body` and restores the previous value on the way out, including through an early `return`. The restore is scheduled as a `defer`, so it is not skipped by any exit the block has.

```
GLOSSO
#import "Io";

main :: () {
    context.user_index = 7;
    scoped := context;
    scoped.user_index = 42;
    #push_context scoped {
```

```
        print("scoped %\n", context.user_index);
    }
    print("restored %\n", context.user_index);
    print("hexadecimal %x\n", 255);
}
```

CONSOLE / OUTPUT

```
scoped 42
restored 7
hexadecimal ff
```

Written without an expression, `#push_context body` installs the default context, the same value the process started with. That is the way to run a piece of code under known settings regardless of what an enclosing frame installed. Numeric formatting is selected explicitly with `Formatter` selectors such as `%x`; the former `context.print_style` field no longer exists.

GLOSSO

```
#import "Io";

main :: () {
    context.user_index = 7;
    print("outer %\n", context.user_index);
    #push_context {
        print("bare %\n", context.user_index);
    }
    print("outer again %\n", context.user_index);
}
```

CONSOLE / OUTPUT

```
outer 7
bare 0
outer again 7
```

The body may be a brace block or a single statement, so `#push_context saved print("%\n", context.user_index);` is legal and scopes to that one statement.

`#push_allocator`

`#push_allocator(expr);` replaces only `context.allocator`. It is a statement rather than a block form: it takes effect where it is written and is undone at the end of the enclosing block, again through a deferred restore that also runs on `return`.

GLOSSO

```
#import "Io";
#import "Context";

served :: () -> bool {
    p := alloc(64);
    if #pattern .Some(raw) = p {
        p.free();
        return true;
    }
    return false;
}

main :: () {
    {
        #push_allocator(nil_allocator());
        print("nil served %\n", served());
    }
    print("default served %\n", served());
}
```

CONSOLE / OUTPUT

```
nil served 0
default served 1
```

Both directives are statements. At file scope `#push_allocator(...)` is `error: bad decl name 'push_allocator'`; a whole-program default is set by assigning `context.allocator` at the top of `main` instead.

`#no_context`

A procedure marked `#no_context` may not name `context` in its body. `#c_call` implies the same restriction, because a procedure entered from foreign code cannot assume the context global was ever initialised by Glosso startup.

GLOSSO

```
hash :: (n: s64) -> s64 #no_context {
    return n * 31 + context.user_index;
}
```

CONSOLE / OUTPUT

```
error: cannot access 'context' in a #c_call / #no_context procedure
  --> ...\bare_fn.glo:2:21
  |
2 |     return n * 31 + context.user_index;
  |                     ^^^^^^^
```

The restriction is on the name, not on the machinery. A `#no_context` procedure may still call `print` or `alloc`, and those callees read the context as usual, because the context is a global rather than a hidden parameter. `#no_context` is also part of a procedure's overload key, and a `#fn_ptr` type accepts the directive after its return type.

Naming a context field type

`#Context` and its fields are always reachable. The types of those fields are not: they are declared in the `Context` module and stay hidden until a file imports it, so that a program which never spells one out does not have the supporting type names dropped into its global namespace. Naming one without the import reports both the cause and the fix.

CONSOLE / OUTPUT

```
error: 'Allocator' is a #Context field type; add `#import "Context";` to name it
  --> ...\field_type.glo:1:13
  |
1 | pool :: (a: Allocator) -> s64 {
  |             ^^^^^^^^^
```

Context field	Types it introduces
`allocator`, `temp_allocator`	`Allocator`, `Allocator_Proc`, `Allocator_Mode`, `Allocator_Error`
`logger`	`Logger`, `Logger_Proc`, `Logger_Level`
`random_generator`	`Random_Generator`, `Random_Generator_Proc`, `Random_Generator_Mode`

The gate opens transitively: `#import "Io"` reaches `Context` through `Format`, so a file that prints can also name `Allocator`. Reading and writing `context.allocator` never needs the import; only spelling one of these names as a type does. `Context` written without the sigil is `error: the built-in Context type is written #Context`. This special type-name gate does not re-export the module's procedures: import `Context` directly to call `alloc`, `free`, or allocator helpers.

> **NOTE**
> The compile-time VM has its own readable and writable `context`. Changing it does not initialize the native program's runtime context. VM-backed evaluation supports allocation and typed pointer access; raw heap blocks are limited to 4 MiB each. Arbitrary target foreign calls are still unavailable during compile-time execution. Compile-time evaluation is described in `#comptime`.

GLOSSO

```
#import "Io";

#comptime {
    context.user_index = 7;
    print("compile-time context %\n", context.user_index);
}

main :: () {
    print("runtime context %\n", context.user_index);
}
```

CONSOLE / OUTPUT

```
compile-time context 7
runtime context 0
```

12.2 Allocators

Glosso ships no allocation policy in the language. An allocator is a value: a procedure pointer and an opaque data pointer, declared in the `Context` module and stored in `context.allocator`. Every allocation the language performs on a program's behalf goes through that one record, which is what makes an arena, a pool or a tracking wrapper a drop-in replacement rather than a rewrite.

GLOSSO

```
Allocator_Proc :: #fn_ptr(
    data: ?*mut void,
    mode: Allocator_Mode,
```

```
    size: ssize,
    alignment: ssize,
    old_memory: ?*mut void,
    old_size: ssize
) -> ?*mut void

Allocator :: struct { procedure: ?Allocator_Proc; data: ?*mut void; }
```

Four of the slots are optional, and each for a reason. A stateless allocator carries no `data`; `Alloc` and `Free_All` pass no `old_memory`; an unsatisfiable request yields no memory; and `procedure` itself is optional so that a zero-initialised `Allocator` is the "none installed" sentinel. Because `.None` is address zero, that sentinel is still the all-zero bit pattern.

Modes

One procedure serves every operation; `mode` selects which. The enum is closed.

Mode	Request	Value
`Alloc`	return `size` bytes at `alignment`	0
`Free`	release `old_memory`	1
`Free_All`	release everything this allocator owns	2
`Resize`	move `old_memory` from `old_size` to `size`	3
`Query_Features`	report which modes are implemented	4
`Query_Info`	report allocator metadata	5
`Alloc_Non_Zeroed`	`Alloc` without the obligation to zero	6
`Resize_Non_Zeroed`	`Resize` without zeroing the new tail	7

An allocator may implement a subset and return `.None` for the rest; that is what the supplied ones do. `Resize` is not optional in practice, because a dynamic array grown by `array.push` and a `String_Builder` grown by `String_Builder.write_string` both issue it, and an allocator that answers `.None` there hands the caller no memory to write into.

Errors

`Allocator_Error` names the ways a request can fail. Its variants carry explicit values so that the enum is stable across versions, and `Allocator_Error.reason` turns one into a message.

Variant	Reason string	Value
`None`	`no error`	0
`Out_Of_Memory`	`out of memory`	1
`Invalid_Pointer`	`invalid pointer`	2
`Invalid_Argument`	`invalid argument`	3
`Mode_Not_Implemented`	`allocator mode is not implemented`	4

`Allocator_Error` carries an `Error` instance, so a program that propagates one gets the reporting described in `Error`, `Termination`, and `main`. No entry point in the allocation API returns one today: an `Allocator_Proc` signals failure by returning `.None`, and so do `alloc` and `mem_alloc`.

The supplied allocators

The `Context` module supplies four allocators, each behind a constructor that builds the two-field record. All four are stateless and leave `data` empty.

Constructor	Modes it answers	Backs onto
`heap_allocator()`	`Alloc`, `Alloc_Non_Zeroed`, `Resize`, `Resize_Non_Zeroed`, `Free`	the process heap
`debug_allocator()`	the same five, plus `Free_All`, which prints the leak report	the heap, with bookkeeping
`default_temp_allocator()`, `temp_allocator()`	`Alloc`, `Alloc_Non_Zeroed`, `Free_All`	the temporary arena
`nil_allocator()`	none; every request answers `.None`	nothing

The default context installs the debug allocator in `context.allocator` when `GLOSSO_DEBUG_ALLOCATOR` is set to any value other than `0`; otherwise it installs the heap allocator at every optimization level. It always installs the default temporary allocator in `context.temp_allocator`. The environment-controlled swap is the subject of The Debug Allocator.

Allocating

Two layers sit on top of the vtable. `alloc` and `free` route through `context.allocator` and are the everyday pair; the `mem_*` family takes an explicit allocator, defaulting to `context.allocator` when the argument is omitted.

Call	What it does
`alloc`	`alloc(n) -> ?*void`; `Alloc` through `context.allocator` at alignment 16
`free`	`p.free()`; `Free` through `context.allocator`
`mem_alloc`	`mem_alloc(size, allocator)`; allocator defaults to `context.allocator`; `Alloc` at alignment 16
`pointer.mem_resize`	`old.mem_resize(old_size, size, allocator)`; allocator defaults to `context.allocator`; the old pointer is invalidated
`optional.mem_free`	`p.mem_free(allocator)`; allocator defaults to `context.allocator`
`Allocator.free_all`	`context.allocator.free_all()`; sends `Free_All`
`Allocator.mem_alloc`, `Allocator.mem_free`	the same allocate and release operations with the allocator written first

`optional.mem_free` accepts a `?*void`, so an allocation result is handed straight back to it; `pointer.mem_resize` requires a plain `*void`, so the result has to be matched first. An `Allocator` whose `procedure` slot is empty serves nothing and releases nothing rather than calling through it.

GLOSSO

```
#import "Io";
#import "Context";

main :: () {
    p := mem_alloc(8, heap_allocator());
    if #pattern .Some(raw) = p {
        cell := cast(*mut s64) raw;
        cell.* = 42;
        print("cell %\n", cell.*);
    }
    p.mem_free(heap_allocator());

    q := mem_alloc(8, nil_allocator());
    served := false;
    if #pattern .Some(raw) = q served = true;
    print("nil served %\n", served);
    q.mem_free(nil_allocator());
}
```

CONSOLE / OUTPUT

```
cell 42
nil served 0
```

Writing an allocator

An allocator is one procedure with the `Allocator_Proc` shape plus a constructor that packages it. The procedure switches on `mode` and returns `.None` for anything it does not handle. Installing it with `#push_allocator` redirects every allocation in the block, including the ones made inside library code such as `format`.

GLOSSO

```
#import "Io";
#import "Context";
#import "Format";
#import "String_Builder";

requests: ssize = 0;

tracking_proc :: (
    data: ?*mut void,
    mode: Allocator_Mode,
    size: ssize,
    alignment: ssize,
    old_memory: ?*mut void,
    old_size: ssize
) -> ?*mut void  {
    heap := heap_allocator();
    if mode == {
        case .Alloc, .Alloc_Non_Zeroed;
            requests += 1;
```

```
            return mem_alloc(size, heap);
        case .Resize, .Resize_Non_Zeroed;
            requests += 1;
            if #pattern .Some(old) = old_memory {
                return old.mem_resize(old_size, size, heap);
            }
            return mem_alloc(size, heap);
        case .Free;
            old_memory.mem_free(heap);
            return .None;
        else;
            return .None;
    }
}

tracking_allocator :: () -> Allocator  {
    a: Allocator;
    a.procedure = tracking_proc;
    a.data = .None;
    return a;
}

main :: () {
    #push_allocator(tracking_allocator());
    text := format("%-%", 1, 2);
    print("% after % request(s)\n", text, requests);
    text.free();
}
```

CONSOLE / OUTPUT

```
1-2 after 1 request(s)
```

NOTE

The standard library's convention for a routine that allocates is a trailing `allocator: Allocator = context.allocator` parameter, as in `clone`, `String_Builder.make_len_cap` and the `Container` constructors. A caller that wants a different allocator for one call passes it; a caller that wants it for a region uses `#push_allocator`.

12.3 Temporary Allocation

`context.temp_allocator` is a growing bump arena. It hands out successive slices of one block, ignores `Free` entirely, and reclaims everything at once when the arena is rewound. It exists for values that are built, used and abandoned inside a bounded region: a formatted line, a path joined for one system call, a line of input.

The initial block is not heap memory. It is a static buffer compiled into the binary, sized by `--temp-allocator-size`, which defaults to 4 MiB, or to 64 KiB on a freestanding target. When a request does not fit, the arena mallocs an overflow block whose capacity is the larger of the request and the initial size, and links it onto the front; rewinding pops and releases those blocks back down to the static one.

CONSOLE / OUTPUT

```
$ glosso --backend llvm-ir -o report.ll report.glo
$ grep "^@glo_temp_buf" report.ll
@glo_temp_buf = internal global [4194304 x i8] zeroinitializer, align 16

$ glosso --backend llvm-ir --temp-allocator-size 65536 -o report.ll report.glo
$ grep "^@glo_temp_buf" report.ll
@glo_temp_buf = internal global [65536 x i8] zeroinitializer, align 16
```

`--temp-allocator-size <bytes>` and `--temp-allocator-size=<bytes>` both work. The value must be a positive byte count; anything else, including `0`, is rejected before compilation starts and the driver exits with status 2.

CONSOLE / OUTPUT

```
$ glosso --temp-allocator-size 0 report.glo
glosso: --temp-allocator-size must be a positive byte count
```

Whether each thread gets its own arena is a target decision. The buffer and the three globals that track it are emitted `thread_local` on some targets and shared by every thread on others, Windows among them. The context itself is never thread-local anywhere, so a thread that wants its own settings installs them with `#push_context`.

Frame boundaries

The arena is rewound by the compiler, not by the program. Lowering records the arena's high-water mark on entry to a body and restores it on the way out, so an ordinary block is already a temporary frame. Nothing has to be freed and nothing accumulates.

The marks are emitted only where temporary storage provably cannot leave with the result. A signature is treated as able to carry it out, and therefore gets no marks anywhere in its body, when either of the following holds.

- The return type is a `string`, an `any`, a struct, a union, a pointer, a many-pointer, an array or a tuple.
- Any parameter is a typed pointer, `*T` or `[*]T`.

A loop body in a procedure that satisfies neither reuses the same arena bytes on every iteration.

GLOSSO

```
#import "Io";
#import "Format";

main :: () {
    first: s64 = 0;
    for i: 0..3 {
        row := tprint("row-%", i);
        here := cast(s64) row.data;
        if i == 0 first = here;
        print("% reuses the first block: %\n", row, here == first);
    }
}
```

CONSOLE / OUTPUT

```
row-0 reuses the first block: 1
row-1 reuses the first block: 1
row-2 reuses the first block: 1
```

The contrast is visible in one program. `scalar` returns an `s64`, so its mark is restored and two consecutive calls hand back the same address. `carrier` takes a `*s64`, so no mark is taken and its two temporaries sit sixteen bytes apart.

GLOSSO

```
#import "Io";
#import "Format";

scalar :: () -> s64 {
    return cast(s64) tprint("aaaa").data;
}

carrier :: (sink: *mut s64) {
    first := cast(s64) tprint("aaaa").data;
    second := cast(s64) tprint("bbbb").data;
    sink.* = second - first;
}

main :: () {
    print("scalar reuse %\n", scalar() == scalar());
    delta: s64 = 0;
    carrier(&delta);
    print("carrier delta %\n", delta);
}
```

CONSOLE / OUTPUT

```
scalar reuse 1
carrier delta 16
```

This is exactly why `tprint` can return arena memory at all. The return type is `string`, so no restore is emitted in `tprint` or in any caller whose own signature also carries storage out, and the result survives until the nearest enclosing frame that does take a mark closes.

> **NOTE**
> A temporary value assigned out of an inner block is stale the moment the block ends. The compiler does not diagnose it; the bytes are handed to the next request that asks for them.

GLOSSO

```
#import "Io";
#import "Format";

main :: () {
    held: string;
    {
        held = tprint("inner-%", 1);
    }
    _ := tprint("ABCDEFG");
    print("held=%\n", held);
}
```

CONSOLE / OUTPUT

```
held=ABCDEFG
```

reset_temporary_storage

`reset_temporary_storage()` calls `context.temp_allocator.free_all()`, which sends `Free_All` to the arena: overflow blocks are released and the mark returns to the start of the static block. It is the explicit spelling of what the compiler already does at a frame boundary.

What comes from temporary storage

A value backed by the arena does not need individual release and must not be passed to `free` or `string.free`. These are the standard-library entry points that produce one.

Call	Result
`tprint`	the formatted string, in the arena
`File.read_line`	the `text` field of its `Read_Line`, in the arena
`string.c_string`	a NUL-terminated copy of the string, in the arena
`format(..., .allocator = context.temp_allocator).into_temporary()`	the formatted arena string, with its per-value release obligation transferred to the arena
`String_Builder.into_temporary_string`	the builder's arena-backed bytes as a string
`String_Builder.make_len_cap(.allocator = context.temp_allocator)`	an arena-backed builder; `into_temporary_string()` consumes it and returns its text

`print` and `writef` build their output with `tprint`, and `File.write_int` formats through the arena as well, so a procedure that prints is already using temporary storage even if it never names it. `format` is the owning counterpart: it allocates from `context.allocator` and its result is released with `string.free`.

> **NOTE**
> A temporary value that has to outlive its frame is copied, not retained. `text.clone()` and `format("%", value)` both produce an owned string from `context.allocator`; writing `kept = (tprint(...)).clone()` inside the inner block above makes the earlier example print `kept=inner-1`. Built-in Strings covers the string representation itself.

12.4 The Debug Allocator

Every allocation it serves gets a 48-byte header holding the size, the source trace of the request and links into a live list, plus an eight-byte guard written past the end of the user bytes. That bookkeeping is what lets the allocator name the line a leaked block came from, and it is why diagnostic mode must be requested explicitly.

> **NOTE**
> A program can still install `debug_allocator()` explicitly. That routes allocations and releases through the wrapper, but the compiler-wide `GLOSSO_DEBUG_ALLOCATOR` mode additionally instruments pointer uses and requests the automatic report on normal exit, `exit`, failed `#assert`, and `process_abort`.

GLOSSO

```
#import "Io";
#import "Context";

main :: () {
    kept := alloc(64) ;
    scratch := alloc(32);
    scratch.free();
    print("done\n");
}
```

CONSOLE / OUTPUT

```
$ GLOSSO_DEBUG_ALLOCATOR=1 glosso -O1 --run trace.glo
done
glosso debug allocator: leaked 1 allocation(s), 64 byte(s)
  leak: 64 byte(s)
    allocated at:
      ...\trace.glo:5:13

$ glosso -O0 --run trace.glo
done
```

The report gives one `leak:` entry per block still live at exit, with the byte count and the trace of the call that produced it. A dynamic array is reported at its `array.push`, because that is the call that asked the allocator for storage.

Double free

A second `free` of a block the allocator already reclaimed is reported with all three positions: where the block came from, where it was first released, and where the second release happened.

```
GLOSSO
#import "Io";
#import "Context";

main :: () {
    p := alloc(64);
    p.free();
    p.free();
    print("still running\n");
}
```

Use after free

When debug-allocator mode is enabled, the backend inserts a check before every load and store through a pointer at every optimization level. If the address falls inside a block on the freed list, the read or write is reported and then allowed to proceed.

```
GLOSSO
#import "Io";
#import "Context";

main :: () {
    p := alloc(64);
    if #pattern .Some(raw) = p {
        bytes := cast([*]mut u8) raw;
        bytes[0] = 7;
        p.free();
        print("value=%\n", bytes[0]);
    }
}
```

Invalid free and invalid realloc

A pointer the debug allocator never handed out, and that is not on its freed list either, is an invalid free. Only the release site can be named, because there is no header to read a history from.

```
GLOSSO
#import "Io";
#import "Context";

main :: () {
    slot: s64 = 1;
    (cast(*mut void) &slot).free();
    print("continued\n");
}
```

The same test on the resize path is `invalid realloc`. Its output also shows what a trace is: not one position but the chain of call sites that reached the allocator, innermost first. The single-line traces above are single-line only because `alloc` and `free` lower directly at the point of use.

```
GLOSSO
#import "Io";
#import "Context";

main :: () {
    slot: s64 = 1;
    fresh := (cast(*mut void) &slot).mem_resize(8, 32);
    print("continued\n");
}
```

The tail guard

Eight bytes of `0xa5` are written past the end of every allocation and checked on `Free` and again when the block is walked for the leak report. A write that runs off the end of the requested size damages the guard and is counted.

```
GLOSSO
#import "Io";
#import "Context";

main :: () {
    p := alloc(16);
    if #pattern .Some(raw) = p {
        bytes := cast([*]mut u8) raw;
        bytes[16] = 1;
    }
    p.free();
```

```
    print("continued\n");
}
```

CONSOLE / OUTPUT

```
$ GLOSSO_DEBUG_ALLOCATOR=1 glosso -O1 --run overrun.glo
continued
glosso debug allocator: 1 allocator error(s)
```

A damaged guard has no message of its own; it raises the error count and nothing else. An error total with no fault report above it is therefore an overrun, and the block that carried it is found by narrowing the writes rather than by reading the output.

When the report runs

The report is emitted at most once per process, guarded by a flag the report itself sets. It runs on a normal return from `main`, and also before `exit`, before a failed `#assert` aborts and before an explicit process abort, so a program that stops deliberately still says what it was holding. It does not run on the safety-check path: a bounds or overflow failure at `-O0` prints its runtime-check category and then terminates without running the allocator report, as Runtime Safety Checks and Panics describes.

`debug_allocator_report_leaks()` forces it early. Because of the once-only flag, forcing it means nothing allocated afterwards is ever reported. The magic behind it, `__debug_report_leaks`, sits in a private section of the `Context` module and naming it directly is `error: unknown function __debug_report_leaks`. The same report is what `context.allocator.free_all()` triggers while the debug allocator is installed, because `Free_All` is the mode it maps to.

GLOSSO

```
#import "Io";
#import "Context";

main :: () {
    first := alloc(64) ;
    debug_allocator_report_leaks();
    second := alloc(32) ;
    print("done\n");
}
```

CONSOLE / OUTPUT

```
$ GLOSSO_DEBUG_ALLOCATOR=1 glosso -O1 --run early.glo
glosso debug allocator: leaked 1 allocation(s), 64 byte(s)
  leak: 64 byte(s)
    allocated at:
      ...\early.glo:5:14
done
```

The message set

Fault	Message after `glosso debug allocator:`	Traces
leak	`leaked N allocation(s), M byte(s)`, then a `leak:` entry per block	`allocated at:`
double free	`double free`	`allocated at:`, `first free at:`, `second free at:`
use after free	`use after free`	`allocated at:`, `freed at:`, `used at:`
invalid free	`invalid free`	`free at:`
invalid realloc	`invalid realloc`	`realloc at:`
total	`N allocator error(s)`, printed last	none

None of them terminates the process. Each fault is recorded, execution continues, and the total is printed with the report; the exit status is unaffected, so a build step that wants a failure has to read the output.

> **NOTE**
> The debug allocator sees only what passes through it. Storage from `context.temp_allocator` is never tracked, because the arena serves itself out of a static buffer, and neither is memory taken from a foreign `malloc` or from an allocator installed with `#push_allocator`.

12.5 Automatic Destruction with `Drop`

Glosso destroys managed values automatically when their lifetime ends. A type is managed when it implements Drop or contains managed fields. Dynamic arrays and standard containers are owners; primitive strings, pointers, and slices are views.

The Drop contract

```
GLOSSO
Resource :: struct { id: ssize; }
Resource ::: instance Drop {
    Error :: #Never;
    drop :: (value: *mut #Self_Type) -> #Never!void {
        // Release the resource represented by value.*.id here.
        return .Ok;
    }
}

main :: () {
    resource := Resource.{ .id = 1 };
    // The compiler calls the hook at scope exit.
}
```

The hook releases the value's own resource. The compiler separately destroys its remaining owned fields, including when the hook returns an error. Ordinary code cannot call Drop.drop or take its address. Use a nested scope to end ownership earlier.

Automatic destruction and explicit defer actions share reverse scope order. Cleanup runs on normal exits, return, and error propagation. Process termination and runtime traps do not unwind scopes.

> **NOTE**
> Use Error :: #Never for infallible cleanup. A fallible hook uses its own error type and requires a compatible result boundary or a local #on_drop_error handler.

12.6 Moves and Managed Values

Binding a new variable, passing a parameter, or returning a managed value by value transfers ownership. The source moves into its destination; the compiler rejects later use of the source, including uses on control-flow paths where it may already have moved.

```
GLOSSO
#import "String_Builder";

consume :: (builder: String_Builder) {
    text := builder.to_string();
}

main :: () {
    builder := String_Builder.builder_from_string("hello");
    consume(builder);
    // builder has moved and cannot be used here.
}
```

Use a read-only *T or writable *mut T parameter to operate on an existing owner while leaving ownership with the caller. Receiver methods may automatically take the address of an addressable value. Read-only pointers cannot be passed to writable receiver methods.

A struct, tuple, option, result, or fixed array containing a managed value also needs destruction. Moving the whole aggregate transfers its managed contents. Moving an individual managed field or indexed element directly is rejected; use a typed transfer operation or move the whole owner.

Iterate with for *item: values to visit managed elements through pointers. An ordinary for item: values copies each element and cannot copy a managed owner. Ending a pointer's scope does not destroy its pointee.

> **NOTE**
> Move checking tracks managed values separately from temporal memory analysis. Borrow diagnostics track views through calls, invalidation, escapes, and automatic destruction. They are warnings by default; use --borrow-check=error to reject findings. Keep backing storage valid even when analysis is disabled or cannot prove an origin.

Transfers through existing storage

Import Memory/Ownership or Memory for value_take(slot), value_replace(slot, replacement), and array_pop_last(array_pointer). value_take returns the old owner and leaves its ordinary default in the slot. value_replace

moves in the replacement and returns the previous owner without destroying it. array_pop_last moves the last live element out and decreases count; an empty array traps. End retained views before transferring their storage.

Direct assignment into existing managed storage is rejected. These typed operations keep a valid value in that storage and make ownership transfer explicit. The __value_move and __value_init intrinsics are internal tools for manually tracked raw slots; callers must maintain their lifecycle.

12.7 Dynamic Array and Container Ownership

A [..]T dynamic array owns both its backing allocation and its live elements. push() and append() move managed arguments into the array. Scope exit destroys the live elements and releases the buffer; a defer array.free() is unnecessary for ordinary ownership.

```
GLOSSO
#import "Memory";

main :: () {
    values: [..]ssize;
    values.push(1, 2);
    previous := values.take(); // values becomes empty and reusable.
    values.push(3);
    replaced := values.replace(previous);
    // values owns [1, 2]; replaced owns [3]. Both clean up at scope exit.
}
```

reserve(capacity) requests total capacity without changing count. reset() retains storage and sets count to zero; it is available only for elements without automatic destruction. free() and deinit() destroy live elements and release storage, leaving a valid empty array. They require infallible element cleanup; use take() with an error handler or result boundary for fallible elements.

> **NOTE**
> Dynamic arrays allocate and release through context.allocator. Keep the allocator used during growth installed until destruction. Their three-word descriptor does not capture an allocator. Standard container structs and String_Builder retain the allocator supplied to their constructor.

Standard containers

Deque, LinkedList, HashMap, HashSet, BTreeMap, BTreeSet, and BinaryHeap are move-only owners. Their Drop instances release elements and storage automatically. clear() destroys elements and may retain capacity; deinit() also releases storage and leaves an empty owner. Removal operations that return an element transfer it to the caller.

These containers require infallible element destruction: Drop_Error(T) == #Never, for both key and value in maps. Use clone_with for managed elements that need an explicit cloning callback. Managed keys need the borrowed _by hashing and comparison operations so comparisons do not consume them.

A pointer or view returned by a container borrows its storage. Structural mutation may invalidate it. A LinkedList element pointer remains stable until its node is removed. Import each Container component explicitly; Container is not an umbrella module.

12.8 Memory Contracts and Storage Lifetimes

A *T pointer, []T slice, or primitive string borrows storage without keeping its owner alive. Temporal memory analysis follows the storage origin and diagnoses uses after release, invalidation, or scope exit. This analysis is enabled as warnings by default and can be made an error with --borrow-check=error. It does not insert lifetime tokens or extend storage lifetime.

Read-only and writable access

`*T`, `[*]T`, `[]T`	read-only access
`*mut T`, `[*]mut T`, `[]mut T`	writable access

Writable pointers and slices can be used through read-only forms. The reverse implicit conversion is rejected. volatile marks accesses that must be emitted; *volatile T remains read-only and *mut volatile T permits stores.

Invalidation

Growing a dynamic array or builder may move its allocation. Clearing, freeing, replacing, transferring, or destroying storage can also invalidate retained views. A slice stores its original count; it does not follow later changes to an owner's descriptor.

String_Builder.to_string(), to_bytes(), and to_cstring() borrow the builder. clone(allocator) copies text into an independent allocation. into_string() and into_bytes() transfer the allocation; retain the allocator needed to free the transferred raw result. Import Strings/Memory for string.free(). For arena-backed text, construct the builder with context.temp_allocator and consume it with into_temporary_string(). Alternatively, an owned string allocated from that arena can use string.into_temporary() to hand its release obligation to the arena. Never use this on heap storage. The view expires when the arena is reset or restored.

Returning a builder's to_string() view after that local builder is destroyed creates a dangling view. The current analyzer does not diagnose every builder-backed escape, even in error mode. Return a cloned string or transferred allocation instead. Literal storage lasts for the program's lifetime and must not be freed.

> **NOTE**
> noalias marks unique access during a call. Memory analysis checks conflicting aliases and inferred borrow extents; it is independent of pointer mutability. Runtime bounds checks do not verify aliasing.

Leading memory contracts

Write #memory { ... } immediately before a function declaration. The block belongs to that declaration, and its operands resolve in the following signature: parameters, named results, and #self are available; body locals are not. It can precede foreign and #magic declarations, typeclass and instance methods, and named #fn_ptr types. Visibility prefixes may appear before or after the contract.

```
GLOSSO
#import "Io";

#memory {
    reads(source),
    noescape(source),
    returns_borrow(source),
}
view :: (source: []u8) -> []u8 {
    return source;
}

main :: () {
    bytes: [3]u8 = b"abc";
    text := view(bytes[..]);
    print("%\n", text.count);
}
```

A contract on a function with a body is checked against its implementation. A bodyless declaration supplies an external contract. Private functions can omit a contract and use inference. An explicitly written #memory {} asserts an empty contract; unknown(...) records uncertainty instead of silently claiming no effects. Ordinary function directives, such as #inline and #foreign, keep their position after the signature.

Effect	Meaning
reads(p), writes(p)	Access through the named place; several places can share one effect: reads(a, b).
noescape(p), escapes(p)	Whether a reference may be retained; escapes(p, into: q) names its destination, including global.
invalidates(p), kills(p)	Invalidate derived views, or release the resource itself.
owns(p), owns(p, into: owner)	Transfer an ownership obligation to the callee or into a named resource.
gives(p, released_by: release)	Transfer a fresh resource out through a parameter slot.
returns_borrow(p), returns_unique_borrow(p)	A result derived from p, with shared or unique access.
returns_fresh, returns_static	A newly acquired resource, or storage that needs no release.
released_by(by: release, instance: allocator)	The release operation and optional allocator identity for an acquired result.
unknown(arg: p, reason: "external API")	An explicit hole for one argument; result: returned selects a result place.

Contracts can select fields and tuple components, including returned.Ok, returned.0, and named results. For independent outputs, write separate returns_fresh(returned.0) and returns_fresh(returned.1) effects and give each its own releaser. Inferred maybe_kills and other maybe_ effects describe effects that are possible rather than guaranteed on every path.

Analysis modes and reports

Option	Behavior
--borrow-check=warning	Default: report temporal memory findings and continue compilation.

Option	Behavior
--borrow-check=error	Treat temporal memory findings as compilation errors.
--borrow-check=off	Disable body memory analysis; move and type checking still apply.
--borrow-check-std	Include standard-library bodies in memory diagnostics.
--memory-signatures	Print inferred contracts for functions with bodies.
--memory-axioms	Print normalized external contracts for bodyless declarations.
--memory-report	Print the per-module contract inventory, including deferred generic bodies.

#enable(memory_strict); requires written contracts at exported and external memory boundaries. Generic bodies whose ownership transfers depend on their payload type are checked at concrete calls. The inventory identifies deferred templates. Warning mode, unknown contracts, raw pointer arithmetic, and trusted assertions limit what a successful compilation establishes.

Other contract sites

A postfix call assertion, call(value) #memory { reads(value), noescape(value) };, supplies trusted effects at that call. A named overlay, #memory target { ... };, describes a named declaration; overload ambiguity is diagnosed. #memory trusted("reason") { ... } suspends selected alias and lifetime checks while still computing effects. These forms have different scopes from a leading declaration contract.

> **NOTE**
> Historical contracts after the signature remain accepted, including #memory(...). Prefer leading braced contracts in new code. Contracts describe memory behavior; Drop still determines automatic destruction and move semantics.

12.9 Owning Foreign Resources

Foreign function signatures describe ABI and pointer mutability. Add leading #memory contracts to describe allocation, retention, and release; the compiler treats bodyless contracts as external facts. Match every raw acquisition with the specified releaser and allocator identity. Bindgen emits contracts from the C signature and explicit unknown holes where that signature cannot establish behavior.

For an owned foreign handle, define a local wrapper with a Drop instance. Release that handle in the hook and move the wrapper when transferring ownership. Pass pointers to helpers that should leave ownership with the caller.

```
GLOSSO
#import "Context";

Buffer :: struct { memory: ?*mut void; }
Buffer ::: instance Drop {
    Error :: #Never;
    drop :: (buffer: *mut #Self_Type) -> #Never!void {
        buffer.*.memory.free();
        return .Ok;
    }
}

main :: () {
    buffer := Buffer.{ .memory = alloc(64) };
}
```

> **NOTE**
> This wrapper uses the current context allocator for both acquisition and release. A production wrapper can store its allocator and call memory.mem_free(allocator), or call a foreign library's own release function.

Use defer for a raw resource without a Drop instance. Use #try defer only when cleanup returns a Try carrier and its failure should propagate through the surrounding boundary. Do not schedule a second destructive release for a resource already managed by Drop.

```
GLOSSO
#memory { kills(pointer), noescape(pointer) }
release :: (pointer: *mut u8) #foreign "c" "release";

#memory { returns_fresh, released_by(by: release) }
acquire :: () -> *mut u8 #foreign "c" "acquire";
```

This schematic foreign API gives acquire() a release obligation that release(pointer) discharges. The actual foreign implementation must satisfy those contracts. Wrapping such a handle in a Drop owner makes cleanup automatic; a contract alone does not generate a release call.

12.10 Manual Destruction with `ManuallyDrop`

Import Memory/ManuallyDrop or Memory to use ManuallyDrop(T). The wrapper has T's size, alignment, and valid values; it adds no allocation or hidden live flag. Wrapping a value transfers ownership and suppresses automatic destruction of the contained value and its fields.

```
GLOSSO
#import "Memory";
#import "String_Builder";

main :: () {
    held := ManuallyDrop(String_Builder).new(
        String_Builder.builder_from_string("hello"));
    view := held.get().to_string();
    restored := held.into_inner();
    // restored owns the builder and is destroyed automatically.
}
```

get() returns *T and get_mut() returns *mut T without transferring ownership. into_inner() consumes the whole wrapper and restores automatic destruction. take() extracts through a mutable pointer, including from an enclosing owner's field. drop() runs the full destructor in place and returns Drop_Error(T)!void.

After take() or drop(), the slot must not be used again, including when destruction reports an error. An enclosing Drop hook must account for consumed slots with its own lifecycle state. Use into_inner() when the whole wrapper can be moved.

> **NOTE**
> ManuallyDrop does not make uninitialized storage valid and cannot copy an owner. Leaving an occupied wrapper at scope exit intentionally suppresses cleanup; release or transfer it when that is the intended lifecycle.

12.11 Cleanup Errors and `#on_drop_error`

A Drop hook returns Error!void. With Error :: #Never, cleanup cannot fail and needs no fallible enclosing function. With a real error type, implicit cleanup propagates as if the generated call used ?. The surrounding result type must accept that error or provide the required Propagate conversion.

Handling a cleanup error locally

```
GLOSSO
#import "Io";
#import "Typeclass/Error";

Cleanup_Error :: enum { Failed; }
Resource :: struct { id: ssize; }
Resource ::: instance Drop {
    Error :: Cleanup_Error;
    drop :: (resource: *mut #Self_Type) -> Cleanup_Error!void {
        return .Err(.Failed);
    }
}

main :: () {
    resource := Resource.{ .id = 1 };
    #on_drop_error(resource, error) {
        print("cleanup failed\n");
    }
}
```

Register #on_drop_error(owner, error) immediately after the owning declaration. Its body runs only if implicit cleanup fails. It handles that local owner's cleanup failure without requiring main to return a compatible error carrier.

Without a handler, a cleanup error can replace a successful return with an error. The compiler still destroys remaining owned fields when a hook fails. Dynamic arrays can contain fallible elements; container structs currently require infallible element destruction.

> **NOTE**
> Explicit ManuallyDrop.drop() returns its error to the caller directly. Handle that result or propagate it with ?. The wrapper's slot remains consumed even when cleanup fails.

12.12 Feature Gates: `#enable` and `#disable`

Gate	What `#enable` permits
`lossy_numerics`	Implicit integer-to-integer conversions that lose information: narrowing, and sign reinterpretation.
`pointer_punning`	Pointee-unchecked pointer conversions, and `*T` interchangeable with `[*]T`.
`memory_strict`	Require written memory contracts at exported and external boundaries.
`allow_union_unwrap`	Direct tagged-union payload access outside a pattern match.
`#public`, `#public(unit)`, `#private`	Declaration visibility prefixes; these are not feature gates.

The old `private_section`, `private_sections`, and `siblings` forms have been removed and report a migration diagnostic. Declarations are private to their source file by default. Use `#public` for an exported API or `#public(unit)` for sharing within an ownership unit; a visibility prefix may apply to one declaration or a braced group. The four feature gates above control static semantics and memory analysis independently of visibility.

Form and placement

A feature directive is a top-level section directive, not a statement. It takes effect at the point it appears and stays in effect until the matching `#disable` or the end of the file. It cannot appear inside a body.

GLOSSO

```
#import "Io";

Token :: union { Name: s64; Code: s64; }

main :: () {
    #enable(allow_union_unwrap);
    t := Token.Code(3);
    print("%\n", t.Code);
}
```

CONSOLE / OUTPUT

```
error: unexpected directive 'enable'
  --> ...\f1.glo:6:5
  |
6 |     #enable(allow_union_unwrap);
  |     ^^^^^^^
```

The reason is that a gate is a property of a declaration, not of a region of statements. The parser stamps the currently-open set onto every declaration it parses, and the checker reads the stamp back when it checks that declaration. A gate opened inside a body would have nothing to attach to, and would have to mean something different from a gate opened outside one.

GLOSSO

```
#import "Io";

Token :: union { Name: s64; Code: s64; }

#enable(allow_union_unwrap);

inside :: (t: Token) -> s64 { return t.Code; }

#disable(allow_union_unwrap);

after :: (t: Token) -> s64 { return t.Code; }
```

CONSOLE / OUTPUT

```
error: tagged-union payload '.Code' may only be bound by a pattern match
   --> ...\f2.glo:11:39
   |
11 | after :: (t: Token) -> s64 { return t.Code; }
   |                                      ^^^^
note: match on the union and use that arm's payload capture, or enable #enable(allow_union_unwrap) for unsafe access
```

`inside` compiles and `after` does not, and the split is decided at the declaration, not at the call. Deleting `after` leaves a program in which an ungated `main` calls the gated `inside` and prints `3`. A gate is not contagious in either direction.

The reach of a gate

Every position that carries an expression the checker evaluates is a declaration in this sense, so a gate reaches further than function bodies. It is installed for function bodies, global initialisers, constant initialisers, struct field defaults, and top-level `#comptime` blocks.

Position	Gated
Function body	yes
Global variable initialiser	yes
`::` constant initialiser	yes
Struct field default	yes
Top-level `#comptime` block	yes

This matters because the payload rule is now a default error rather than a relaxation of a permissive default: a gate that could not reach an initialiser would leave that position with no escape hatch at all.

GLOSSO

```
#import "Io";

Res :: union { Ok: s64; Err: s64; }

SOURCE :: Res.Ok(7);

#enable(allow_union_unwrap);

WIDTH :: SOURCE.Ok;

gated_global: s64 = SOURCE.Ok;

#disable(allow_union_unwrap);

ungated_global: s64 = SOURCE.Ok;

main :: () {
    print("% %\n", WIDTH, gated_global);
}
```

The constant and the gated global are accepted; only `ungated_global` is rejected, with the payload-access error pointing at line 15. Removing that line leaves a program that prints `7 7`. The same holds for a top-level `#comptime` block: gated, it runs and prints; after `#disable`, an identical block is an error.

What each gate actually relaxes

`lossy_numerics` is narrower than its name suggests. The gate is consulted at exactly one place in the checker, inside the branch that compares an integer type against an integer type. Narrowing and sign reinterpretation become implicit; float conversions do not, in either direction and at any width.

GLOSSO

```
#import "Io";

#enable(lossy_numerics);

main :: () {
    big: s64 = 300;
    small: u8 = big;
    negative: s64 = -1;
    wrapped: u8 = negative;
    print("% %\n", small, wrapped);
}
```

CONSOLE / OUTPUT

```
44 255
```

Under the same gate, `narrow: f32 = wide;` with `wide: f64` is `error: got f64, expected f32`, and the widening direction is rejected the same way. There are no implicit float conversions in Glosso, gate or no gate; the chapter on conversions gives the full rule and `cast` gives the explicit spelling.

`pointer_punning` relaxes the pointee check and the distinction between a single pointer and a many-pointer. The two assignments below are rejected without it: `error: got *s64, expected [*]s64` and `error: got *s64, expected *u8`. With it, both are accepted and the program prints `65 65`.

GLOSSO
```
#import "Io";

#enable(pointer_punning);

main :: () {
    value: s64 = 65;
    single: *mut s64 = &value;
    many: [*]mut s64 = single;
    bytes: *mut u8 = single;
    print("% %\n", many[0], bytes.*);
}
```

What it does not relax is read-only access. The gate is tested inside the branch that has already established the qualifier direction is legal, so adding writable access remains an error under the gate: `writable: *s64 = readonly;` with `readonly: *s64` reports `error: got *s64, expected *s64`.

The gate that tightens

Rule	Diagnostic

The checks a gate keeps

`allow_union_unwrap` removes a compile-time rule and replaces it with a runtime one; it does not remove the check. A gated payload read compiles to a tag comparison at `-O0` and `-O1`, and reading the wrong variant aborts with the site named.

GLOSSO
```
#import "Io";

#enable(allow_union_unwrap);

Res :: union { Ok: s64; Err: s64; }

make :: (flag: bool) -> Res {
    if flag return Res.Ok(5);
    return Res.Err(9);
}

main :: () {
    print("%\n", make(true).Ok);
    print("%\n", make(false).Ok);
}
```

CONSOLE / OUTPUT
```
$ glosso -O1 -o f6.exe f6.glo && ./f6.exe
5
...\f6.glo:14:30: union access: expected .Ok, but the value holds one of .Ok | .Err
$ echo $?
134

$ glosso -O2 -o f6.exe f6.glo && ./f6.exe
5
9
```

The size modes -Os, -Oz, and -Ozz produce runnable artifacts. They combine LLVM optimization with optsize/minsize attributes and target-specific linker size policies. Runtime checks are disabled in these modes; use -O0 or -O1 for checked builds. -Ozz also strips symbols and metadata.

13 Runtime behaviour

13.1 Runtime Safety Checks and Panics

Glosso compiles a small family of safety checks into the generated code. Each one guards an operation whose failure would otherwise corrupt memory quietly, and each one, when it fails, stops the process on the spot. They are not exceptions: nothing catches them, no `defer` runs. Each check writes a short failure category, and the way the target terminates determines its exit status.

The size modes -Os, -Oz, and -Ozz produce runnable artifacts. They combine LLVM optimization with optsize/minsize attributes and target-specific linker size policies. Runtime checks are disabled in these modes; use -O0 or -O1 for checked builds. -Ozz also strips symbols and metadata.

Flag	Underlying level	Runtime checks
`-O0`	0	on
`-O1` (default)	1	on
`-O2`	2	off
`-O3`	3	off
`-Os`	2 plus `Small`	off
`-Oz`	3 plus `VerySmall`	off
`-Ozz`	3 plus `Tiny`	off

The flags themselves are covered in the chapter on backends, artifacts, and optimization levels. The debug allocator is selected independently with `GLOSSO_DEBUG_ALLOCATOR` and has its own chapter. This chapter is about what the checks test and what happens when one of them fires.

The check family

There are exactly six checks. They are emitted by the backend, not by a library, so no import turns them on and no call disables an individual one.

Check	Guards	Message on failure
integer arithmetic	overflow in `+`, `-`, `*`, prefix `-`, and division; a zero divisor for `/`, `//`, `%`	`runtime check failed: arithmetic.overflow`, `divide.by.zero`, or `divide.overflow`
array bounds	indexing a value that carries a count	`<path>:<line>:<column>: runtime check failed: array.bounds` when the site is known; otherwise the category alone
slice bounds	`a[start..end]` on a value that carries a count	`runtime check failed: slice.bounds`
null address	`cast(*T)`, `cast([*]T)` and `cast(#fn_ptr)` of an integer or enum	for example, `cast to *s64 of an address that is zero`
union tag	a payload read under `#enable(allow_union_unwrap)`	names the wanted variant and the ones the type has
formatting base	`value.format_int` given a base outside 2..=16	`integer formatting base is outside 2..=16`

All six write a line to stderr before terminating. Array bounds and union-tag failures include the source site when available; the arithmetic and slice labels identify the failed operation but carry no source position. Null-address and formatting-base failures use explanatory prose.

Integer overflow

Addition, subtraction and multiplication on an integer or enum operand lower to the LLVM checked-arithmetic intrinsics, and prefix negation lowers to a checked subtraction from zero. Update assignment goes through the same operators and is checked with them. Integer `/`, `//` and `%` also check a zero divisor, and signed division checks `T.MIN / -1`; `T.MIN % -1` is the defined value zero. The shifts and every floating-point operation are outside the check family. A float-to-integer

cast truncates toward zero, saturates out-of-range values, and maps NaN to zero at every optimization level; it does not become unchecked merely because the build uses `-O2`.

GLOSSO

```
#import "Io";

main :: () {
    a: s32 = 2147483647;
    b: s32 = 1;
    print("before\n");
    sum := a + b;
    print("sum = %\n", sum);
    print("survived\n");
}
```

CONSOLE / OUTPUT

```
before
runtime check failed: arithmetic.overflow
```

That is the whole output at `-O1`: the second `print` never runs and on Windows the process exits with status 134. WASI uses 125; targets that call `abort()` expose that platform's abort status. At `-O2` the check is not emitted, the addition wraps, and the same program prints `sum = -2147483648` followed by `survived` and exits 0.

The wrapping and saturating operators are different operators, not a mode, and they are never checked at any level. `+%`, `-%`, `*%` and prefix `-%` wrap; `+|`, `-|` and `*|` clamp.

GLOSSO

```
#import "Io";

main :: () {
    a: s32 = 2147483647;
    b: s32 = 1;
    print("wrapping    = %\n", a +% b);
    print("saturating  = %\n", a +| b);
    n: s32 = -2147483648;
    print("negate wrap = %\n", -% n);
    print("survived\n");
}
```

CONSOLE / OUTPUT

```
wrapping    = -2147483648
saturating  = 2147483647
negate wrap = -2147483648
survived
```

Array bounds

An index is checked whenever a count travels with the value being indexed: a fixed array `[N]T`, a view `[]T`, a dynamic array `[..]T`, a pointer to a fixed array, and a `string` or `string16`. The comparison is unsigned, so a negative index fails the same test as one past the end. A compile-time known invalid index is rejected during checking, including an index computed by a folded helper call.

GLOSSO

```
#import "Io";

which :: (n: ssize) -> ssize { return n; }

main :: () {
    storage: [3]s64;
    storage[0] = 10; storage[1] = 20; storage[2] = 30;
    print("in range: %\n", storage[which(1)]);
    outside: ssize = 7;
    print("out of range: %\n", storage[which(outside)]);
    print("survived\n");
}
```

CONSOLE / OUTPUT

```
in range: 20
<path>:<line>:<column>: runtime check failed: array.bounds
```

The size modes -Os, -Oz, and -Ozz produce runnable artifacts. They combine LLVM optimization with optsize/minsize attributes and target-specific linker size policies. Runtime checks are disabled in these modes; use -O0 or -O1 for checked builds. -Ozz also strips symbols and metadata.

- A many-pointer `[*]T` carries no count, so indexing one is never checked at any level. Reading far past the end of the storage it points at is a plain hardware fault: on Windows the process ends with status `0xc0000005`.

- `cstring` and `cstring16` are outside the family. They are terminated rather than counted, so they carry no bound in their representation. Indexing does not scan for a terminator to check the index; `text[5]` on a three-byte `cstring` reads past the end and the program survives.

The checks that print

Casting an integer or enum to `*T`, `[*]T` or `#fn_ptr` asserts that the address is not zero, because those three types exclude zero so that `?*T`, `?[*]T` and `?#fn_ptr` can use it as the `.None` niche. A zero the checker can fold never reaches run time: `cast(*s64) 0` is `error: cast to *s64 of an address that is zero at compile time`. A zero that only appears at run time reaches the check, which writes `cast to *s64 of an address that is zero` and aborts. The type in both messages is the destination type as the compiler prints it.

The union check exists only because of the escape hatch. A tagged union normally yields its payload through a pattern match; under `#enable(allow_union_unwrap)` a direct payload read is allowed and the backend inserts a tag comparison in its place.

GLOSSO

```
#import "Io";
#enable(allow_union_unwrap);

lookup :: (flag: bool) -> string!s32 {
    if flag return .Ok(7);
    return .Err("no file");
}

main :: () {
    good := lookup(true);
    print("good = %\n", good.Ok);
    bad := lookup(false);
    print("bad = %\n", bad.Ok);
}
```

CONSOLE / OUTPUT

```
good = 7
...\unwrap.glo:13:28: union access: expected .Ok, but the value holds one of .Ok | .Err
```

The report is a fixed string built when the access is lowered, which is why it carries a source position. The array-bounds check now does the same; the other checks report a category without a site. At `-O2` the check disappears and the program reads the wrong payload silently: the same build prints `bad = 7` and exits 0. The gate itself is described in the chapter on feature gates.

`#assert` is not one of them

`#assert` is a separate mechanism and survives every working optimization level. A condition the checker can decide is decided at compile time and emits no code at all; a condition it cannot decide becomes a branch that stays in the program.

GLOSSO

```
#import "Io";

scale :: (n: s64) -> s64 {
    #assert n != 0, "n must not be zero";
    return 100 / n;
}

main :: () {
    print("100/4 = %\n", scale(4));
    print("100/0 = %\n", scale(0));
}
```

CONSOLE / OUTPUT

```
100/4 = 25
...\guard.glo:4:5: assertion failed: n must not be zero
```

That output is identical at `-O1` and `-O3`; on Windows the status is 134 at both levels. Without a message the line ends after `assertion failed`. When the condition folds to false the assertion becomes a compile error instead: `#assert 1 > 2, "arithmetic is broken";` is `error: assertion failed: arithmetic is broken`, framed at the `#assert` token. The directive's other uses are covered in the chapter on `#assert` and `#compile_error`.

What nothing checks

- Optimized integer division. At `-O2` and `-O3` the zero-divisor and division-overflow guards are omitted with the other runtime checks, so a bad operation reaches target/LLVM behavior. In a checked build both fail through Glosso's checked-failure path. `T.MIN % -1` remains zero at every level because that remainder is representable.

- Non-literal shift counts at or beyond the operand width have a defined result on every backend. Left and logical-right shifts produce zero; an arithmetic-right shift produces the sign (`0` or `-1`). A literal overshift is rejected earlier because the checker can see it directly.
- Allocator faults. Leaks, double frees and use after free are found by the debug allocator when `GLOSSO_DEBUG_ALLOCATOR` is nonzero. It reports and keeps running rather than aborting. The compile-time memory analysis covers a related static set; both have their own chapters.

How a failing check ends the process

Every check funnels into one termination sequence: write its report line, then take the target's abort path. The debug allocator's report is not part of it, which is why a build that fails a check says nothing about what it was holding even when the debug allocator is enabled; `#assert` and `process_abort` ask an enabled debug allocator to report first. Windows and Linux x86-64 spell out 134, WASI deliberately uses 125, and the remaining targets call the platform `abort()`.

Target family	How the process stops
hosted Windows	`ExitProcess(134)`
`wasm32-wasi`, `wasm64-wasi`	`__wasi_proc_exit(125)`
Linux on `x86_64`	the `exit_group` syscall with 134
everything else	`abort()`

> **NOTE**
> 134 is `128 + 6`, the status a POSIX shell reports for a process killed by `SIGABRT`. Picking that number keeps a Glosso panic indistinguishable from a C `abort()` in a build script. Windows spells that value out explicitly. WASI reserves statuses above 125, so its explicit path uses 125 rather than turning the failure into an indistinguishable guest trap.

`process_abort()` from `Process` uses the same target-dependent sequence. `exit(status)` does not: it takes the ordinary exit path and the status reaches the shell unchanged. Under `--run` the compiler prints the target's status — for example, `glosso: program exited with status exit code: 134` on Windows — and returns 1 of its own; the program's status is not propagated, which is covered in the chapter on environment, toolchain, and exit codes.

13.2 Sanitizers

Two driver flags turn on LLVM's sanitizers: `--memory-sanitizer` and `--thread-sanitizer`. Each one adds a function attribute to every function with a body, runs the matching instrumentation pass over the module, and routes the final link through the Clang driver instead of the raw LLD plan so that the sanitizer runtime is pulled in.

They are mutually exclusive, and the second one is an error rather than a replacement for the first. The message names the mode already in effect, so the order on the command line is visible in it.

CONSOLE / OUTPUT

```
$ glosso --memory-sanitizer --thread-sanitizer race.glo
glosso: choose only one sanitizer mode (already using --memory-sanitizer, got --thread-sanitizer)
$ glosso --thread-sanitizer --memory-sanitizer race.glo
glosso: choose only one sanitizer mode (already using --thread-sanitizer, got --memory-sanitizer)
```

Both exit 2, the driver's status for a rejected command line.

What each flag selects

The flag names describe the class of bug, not the LLVM sanitizer, and the mapping is target-dependent. On x86-64 Windows there is no MemorySanitizer implementation, so `--memory-sanitizer` selects AddressSanitizer there rather than being refused.

Flag	Target	Instrumentation
`--memory-sanitizer`	x86-64 Windows	`sanitize_address`, pass `asan<use-after-scope>`
`--memory-sanitizer`	Linux	`sanitize_memory`, pass `msan`
`--thread-sanitizer`	every accepted target	`sanitize_thread`, passes `function(tsan),tsan-module`

The Windows case also disables ASan's ODR indicator globally, because the textual pass syntax cannot pass that constructor argument, and it filters the explicit C-runtime libraries out of the link so the sanitizer's own choice wins.

The supported-target matrix

Support is decided by substring tests on the LLVM triple, before any linking is attempted. An unsupported target is a hard backend error, not a warning and not a silent no-op.

Sanitizer	Accepted	Rejected
`--memory-sanitizer`	any triple containing `-linux-`, plus x86-64 Windows	macOS, FreeBSD, wasm, freestanding
`--thread-sanitizer`	`-linux-`, `-darwin`, `-freebsd`, plus x86-64 Windows	wasm, freestanding

CONSOLE / OUTPUT

```
$ glosso --memory-sanitizer --target wasm32-wasi -o m.wasm race.glo
glosso: compile race.glo (target=wasm32-unknown-wasi)
glosso: link m.wasm (target=wasm32-unknown-wasi, -O1)
error: LLVM memory sanitizer is not supported for target 'wasm32-unknown-wasi'
```

`--thread-sanitizer` on the same target reports `error: LLVM thread sanitizer is not supported for target 'wasm32-unknown-wasi'`, and either flag aimed at `x86_64-freestanding` reports the same sentence with `'x86_64-unknown-none-elf'`. Because the check lives in the backend it fires for every backend, including `--backend object` and `--backend llvm-ir`, where nothing is linked at all. The linker also refuses sanitizers for wasm and for freestanding ELF, but the backend rejects those targets first, so those linker messages are unreachable from the command line.

What each one finds

- AddressSanitizer, which is what `--memory-sanitizer` means on Windows, finds out-of-bounds accesses to heap, stack and global storage, use after free, double free, and, because the pass is configured with `use-after-scope`, reads of a stack slot after its scope has ended.
- MemorySanitizer, which is what `--memory-sanitizer` means on Linux, finds reads of uninitialized memory. It tracks a shadow bit per byte and reports at the point where an uninitialized value changes control flow or leaves the program.
- ThreadSanitizer finds data races between threads, along with a small set of related faults such as unlocking a mutex owned by another thread. It needs the program to actually execute the interleaving, so it reports what a run did, not what a run could do.

Practical caveats

A sanitized build does not use the bundled LLD plan. It shells out to `clang`, found through the usual override chain, and hands it `-fsanitize=address`, `-fsanitize=memory` or `-fsanitize=thread`. Anything that Clang installation refuses, the build refuses.

- On Windows the driver tries to copy `clang_rt.asan_dynamic-x86_64.dll` next to the output, looking under `%USERPROFILE%/.local` for an LLVM build tree and then along `PATH`. If it finds none, the link still succeeds and the failure moves to run time: the loader cannot resolve the runtime and the process dies before `main` with `0xC0000135`, `STATUS_DLL_NOT_FOUND`.

CONSOLE / OUTPUT

```
> glosso --memory-sanitizer -o race.exe race.glo
glosso: compile race.glo (target=x86_64-pc-windows-msvc)
glosso: link race.exe (target=x86_64-pc-windows-msvc, -O1)
glosso: done race.exe  (backend=llvm, target=x86_64-pc-windows-msvc, -O1)
> .\race.exe
> $LASTEXITCODE
-1073741515
```

- `--thread-sanitizer` is accepted by the target matrix on x86-64 Windows but rejected by the Clang driver, so it fails at the link step with the C compiler's own wording.

CONSOLE / OUTPUT

```
> glosso --thread-sanitizer -o race.exe race.glo
glosso: compile race.glo (target=x86_64-pc-windows-msvc)
glosso: link race.exe (target=x86_64-pc-windows-msvc, -O1)
error: clang: error: unsupported option '-fsanitize=thread' for target 'x86_64-pc-windows-msvc'
```

> **NOTE**
> The target matrix and what the platform toolchain actually supports are two separate gates. Passing the first proves only that the compiler will instrument the module; the link is where a missing runtime shows up.

The instrumentation is applied during code generation, so it is present in every artifact and not only in linked executables. A sanitized `--backend llvm-ir` run emits a module carrying `sanitize_address` on each function and the `__asan_` globals the

pass introduces, and a sanitized `--backend object` or `--backend static-lib` build produces an instrumented object that some other link step has to supply the runtime for.

The selected sanitizer is part of the native artifact cache's identity string, so a sanitized build and a plain build of the same source never restore each other. Sanitizers are also orthogonal to `-O`: they neither enable nor suppress the `-O0` and `-O1` runtime checks, and combining `-O1` with a sanitizer keeps both.

13.3 Environment, Toolchain, and Exit Codes

The compiler reads a fixed set of environment variables. None of them changes the language; they change where the compiler looks for the standard library, its resources and its tools, how much it reports about itself, and which caches it is allowed to use. Wherever a command-line option covers the same ground, the option wins.

Variables that change behaviour

These are read by the driver itself and affect every build.

Variable	Effect
`GLOSSO_STD`	first candidate for the standard-library directory
`GLOSSO_CACHE_DIR`	absolute override for the per-user cache root; a relative value is an error
`GLOSSO_NATIVE_CACHE_DIR`	overrides only the cache of C archives built at compile time
`GLOSSO_DISABLE_ARTIFACT_CACHE`	any value disables the final-artifact cache
`GLOSSO_NO_PRUNE`	lowers every declaration instead of only the reachable ones
`GLOSSO_TIMINGS`	per-phase compiler timings on stderr
`GLOSSO_LLVM_TIMING`	LLVM pass and emit timings, and the active pipeline
`GLOSSO_LLVM_OBJECT_PIPELINE`, `GLOSSO_LLVM_VERIFY`	the first replaces the default LLVM pass pipeline (`none` disables it); the second enables LLVM module verification when present with a value other than `0`
`GLOSSO_COMPILER_EXE`	the path treated as the compiler when deriving discovery roots
`GLOSSO_JIT`	compiles top-level `#comptime` with the LLVM JIT, falling back to the interpreter per entry
`GLOSSO_JIT_REPORT`, `GLOSSO_JIT_DUMP`	with `GLOSSO_JIT`, the first prints the per-entry compilability report and the second writes each compiled entry as `<directory>/<entry>.ll`
`GLOSSO_EMBEDDED_LLD`	`0`, `off`, `no` or `false` spawns the LLD drivers instead of linking in process
`NO_COLOR`	forces color off, beating `--color always`
`GLOSSO_COMPTIME_STEP_LIMIT`	maximum interpreter steps or VM instructions per compile-time entry; defaults to 2^28, and 0 removes the limit
`GLOSSO_DEBUG_ALLOCATOR`	a value other than `0` installs the instrumented allocator as `context.allocator`; absent or `0` keeps the heap allocator, independently of `-O`

The two timing switches are independent and can be set together. The phase names are the compiler's own pipeline stages, and the LLVM line reports the pipeline string in force, which is what makes `GLOSSO_LLVM_OBJECT_PIPELINE` observable. A `GLOSSO_JIT_DUMP` directory must already exist; a failed write is reported as `glosso: could not dump <path>: <error>` and does not stop the build.

CONSOLE / OUTPUT

```
$ GLOSSO_TIMINGS=1 GLOSSO_LLVM_TIMING=1 glosso --backend object -o status.obj status.glo
glosso: compile status.glo (target=x86_64-pc-windows-msvc)
glosso: timing prepare 227.548 ms
glosso: timing check 616.655 ms
glosso: timing lower 54.163 ms
glosso: llvm object passes=0.025s emit=0.062s pipeline=function(mem2reg,instcombine<no-verify-fixpoint>,early-
cse,simplifycfg,adce)
glosso: wrote status.obj  (backend=object, target=x86_64-pc-windows-msvc, -O1)
glosso: timing consume 102.922 ms
```

> **NOTE**
> A warm artifact cache hides both switches, because a restored build never reaches the phases they measure. Set `GLOSSO_DISABLE_ARTIFACT_CACHE=1` alongside them when the numbers matter.

- `GLOSSO_NO_PRUNE` is a debugging escape hatch, not a switch with a visible effect: the pruned and unpruned modules are byte-identical for the same program.

- `NO_COLOR` is honoured for any value, including the empty string, and it is checked before `--color`, so `--color always` cannot override it. The reverse is not true: `--color never` is honoured regardless of `NO_COLOR`.

Finding the standard library

`--std <dir>` settles the question outright. The directory must contain either `Prelude.glo` or a `Prelude/` subdirectory, and the path is canonicalized. An empty value is `glosso: --std needs a non-empty directory` and a directory without a prelude is `glosso: --std directory must contain Prelude.glo or Prelude/`; both exit 2, because they are rejected while the command line is parsed.

Without `--std` the compiler walks a candidate list and takes the first entry that is a directory holding a prelude. Duplicates are skipped, so the same directory reached two ways is tried once.

Order	Candidate
1	`$GLOSSO_STD`
2	`<cwd>/std`
3	`<compiler dir>/std`, then `../std`, then `../../std`
4	Windows: `%APPDATA%/glosso/std`, then `%USERPROFILE%/.local/glosso/std`. Elsewhere: `/usr/local/glosso/std`, then `$HOME/.local/glosso/std`

Step 3 is why a compiler at `<repo>/build/bin/glosso.exe` finds `<repo>/std` from any working directory: `build/bin` walks up to `build` and then to `<repo>`. `GLOSSO_COMPILER_EXE` replaces the executable path used for that walk, but two further fallbacks derived from the real executable run afterwards, so pointing it somewhere empty does not make the standard library disappear. When nothing matches the compiler stops with `glosso: could not find the project-local or installed Glosso standard library` and exits 1.

Resource, library and toolchain roots

Resources are the non-Glosso material a build may need: the bundled musl sources, the mingw-w64 import libraries, the Clang resource directory, and the third-party C sources behind `Lua`. They are looked up under a list of roots rather than a single prefix.

Variable	What it supplies
`GLOSSO_RESOURCE_DIR`	the project or installed resource root; validated, and a value that is neither is an error
`GLOSSO_LIB_DIR`	the library root searched as `lib/` and `lib/glosso/`
`GLOSSO_MUSL_SOURCE_DIR`, `GLOSSO_MUSL_LIB_DIR`	the musl source tree and a prebuilt musl runtime bundle
`GLOSSO_MUSL_LIBRARY_PATH`	extra static system-library directories, path-separated
`GLOSSO_WINDOWS_GNU_LIB_DIR`, `GLOSSO_WINDOWS_GNU_INCLUDE_DIR`	the mingw-w64 bundle for `*-windows-gnu` targets
`GLOSSO_CLANG_RESOURCE_DIR`	the Clang resource directory holding the compiler builtins
`GLOSSO_INHERITED_NIX_LDFLAGS`	`-L` flags harvested from a Nix environment
`GLOSSO_LLVM_ROOT`, `LLVM_ROOT`	the LLVM install root searched for tools

The roots are tried in this order: `GLOSSO_RESOURCE_DIR`, `GLOSSO_LIB_DIR`, the parent of `GLOSSO_MUSL_SOURCE_DIR`, the current directory, the compiler's directory, and that directory's parent and grandparent. Under each root a named resource is looked for directly, then under `thirdparty/`, then `lib/`, then `share/glosso/`. A root counts as a project root when it holds either a development tree (`src/lib.rs` and `data/UnicodeData.txt`) or an installed tree (a `lib/` directory next to a `std/` with a prelude).

Library lookup has one development-only candidate ahead of those general roots. When the effective compiler executable (including a `GLOSSO_COMPILER_EXE` override) belongs to a Glosso checkout, that checkout's `build/lib` is searched for staged Clang headers, compiler-rt archives, and other target libraries. The candidate is derived from the compiler path, not the current directory: an installed compiler launched inside an unrelated checkout will not borrow that checkout's possibly stale `build/lib`.

Choosing the LLVM tools

Every external tool is resolved the same way. The tool's own environment names are consulted in order and the first non-empty one wins outright. Failing that, an LLVM root is derived from `GLOSSO_LLVM_ROOT`, then `LLVM_ROOT`, then `$HOME/.local/llvm-project` or its `%USERPROFILE%` equivalent, and the tool is looked for at `<root>/bin/`, at `<root>/<host-bundle>/bin/`, and finally under the `bin/` of each child directory in name order. If none of that finds it, the bare name is used and `PATH` decides.

Tool	Environment chain, first match wins
`clang`	`GLOSSO_CLANG`, `CLANG`, `CC`
`ld.lld`	`GLOSSO_LD_LLD`, `LD_LLD`
`lld-link`	`GLOSSO_LLD_LINK`, `LLD_LINK`
`wasm-ld`	`GLOSSO_WASM_LD`, `WASM_LD`
`ld64.lld`	`GLOSSO_LD64_LLD`, `LD64_LLD`
`llvm-ar`	`GLOSSO_LLVM_AR`, `LLVM_AR`, `AR`
`llvm-lib`	`GLOSSO_LLVM_LIB`, `LLVM_LIB`
`rust-lld` (Windows)	`GLOSSO_RUST_LLD`, `RUST_LLD`, then a rustup toolchain
the `#header` verifier's C compiler	`GLOSSO_CLANG`, `CLANG`, `CC`, then `cl`, `clang-cl` and `cc`; `#bindgen` itself has no override and runs the in-process Clang

`rust-lld` is the one entry with its own search. When neither variable names an existing file, the compiler looks under `$RUSTUP_HOME` (or `%USERPROFILE%/.rustup`), preferring `$RUSTUP_TOOLCHAIN` if it is set. It is reached only by a compiler built without the embedded LLD; an ordinary build links COFF in process, which is why linker diagnostics on Windows are prefixed `lld-link:`.

The compiler's exit codes

The driver uses three statuses of its own, and one borrowed. 2 means the command line was rejected and nothing was compiled; 1 means the compilation or the link failed; 0 means the requested work was done. The borrowed one is compile-time `exit`.

Situation	Exit code
a successful build	`0`
`--help`, `--print-targets`/`--targets`, `--print-cache-dir`	`0`
a compile, type, memory-contract or link error	`1`
an unreadable source file	`1`
`--run` on a non-host target, or where the built program exits nonzero	`1`
`--print-cache-dir` with an unusable `GLOSSO_CACHE_DIR`	`1`
an unknown option, or a bad value for one	`2`
no source file operand	`2`
top-level `#comptime` that called `exit(n)`	`n`

The program's exit code

A built program's status is its own and is not related to the compiler's. It comes from what `main` returns, through the `Termination` protocol described in the chapter on `Error`, `Termination`, and `main`: an integer result is narrowed to `s32` and used directly, a `void` result is 0, and `exit(status)` leaves immediately with the value given.

`--run` does not pass that status on. It prints what the program exited with and then returns 1 for any nonzero value, so `--run` cannot be used to observe an exact exit code. Build the executable and run it to see one.

GLOSSO

```
#import "Io";

main :: () -> ssize {
    print("finishing\n");
    return 5;
}
```

CONSOLE / OUTPUT

```
$ glosso --run status.glo
glosso: compile status.glo (target=x86_64-pc-windows-msvc)
glosso: link status.exe (target=x86_64-pc-windows-msvc, -O1)
glosso: done status.exe  (backend=llvm, target=x86_64-pc-windows-msvc, -O1)
```

```
finishing
glosso: program exited with status exit code: 5
```

Compile-time `exit` is the one path by which an arbitrary status reaches the shell from the compiler. A top-level `#comptime` that calls it stops the build at that point, and no artifact is written.

GLOSSO

```
#import "Io";

stop :: () {
    print("[comptime] stopping the build\n");
    exit(7);
}

#comptime stop();

main :: () { print("never built\n"); }
```

CONSOLE / OUTPUT

```
$ glosso --run ctexit.glo
glosso: compile ctexit.glo (target=x86_64-pc-windows-msvc)
[comptime] stopping the build
```

There is no `link` line and no `ctexit.exe`, and the compiler's own exit status is 7.

> **NOTE**
> Driver messages go to stderr, prefixed `glosso:` . Program output and the output of compile-time execution go to stdout. Redirecting one stream therefore separates the build log from what the program produced, which is what makes a `#comptime` build script usable in a pipeline.

Memory analysis options

--borrow-check=warning is the default. Use --borrow-check=error to reject temporal memory findings, --borrow-check=off to disable body analysis, and --borrow-check-std to include standard-library bodies. --memory-signatures prints inferred body contracts, --memory-axioms prints external contracts, and --memory-report prints the module inventory. #enable(memory_strict); requires written contracts at exported and external memory boundaries. See Memory Contracts and Storage Lifetimes for the effect vocabulary.

14 Interop and low-level code

14.1 Calling C from Glosso

Glosso calls C directly. There is no wrapper layer, no generated shim, and no marshalling step: a `#foreign` declaration gives a Glosso signature to a symbol that already exists in a native library, and the call site lowers to an ordinary call in the target's C ABI.

Two declarations are involved. A `#library` declaration names the native artifact the symbols live in, and each `#foreign` function declaration binds one symbol. The library declaration is a compile-time constant like any other, so it is named, scoped, and importable in the usual way. Match the target's C types exactly: `strlen` returns `size_t`, so its Glosso result is `usize`, not a fixed `u64` on 32-bit targets.

GLOSSO

```
#import "Io";

libc     :: #library "c";
c_strlen :: (s: *s8) -> usize #foreign libc "strlen";
c_puts   :: (s: *s8) -> s32 #foreign libc "puts";

main :: () {
    print("strlen = %\n", c_strlen("glosso"));
    c_puts("printed by C puts");
}
```

CONSOLE / OUTPUT

```
strlen = 6
printed by C puts
```

Declaring a library

A `#library` declaration is `<Name> :: #library[,<kind>] "<file>";`. The string is the library file as the linker should see it, not a Glosso module path. Exactly one link-kind modifier is allowed and the default is `static`.

GLOSSO

```
libc     :: #library "c";                 // static, the default
user32   :: #library,system "user32.lib";  // provided by the platform
kernel32 :: #library,dyn "kernel32.dll";   // resolved at run time
```

Modifier	Resolution	Contribution to the link
`static` (default)	at link time	the mapped library argument
`system`	at link time	the mapped library argument
`dyn`	at first call	nothing; `-ldl` where one is needed

`static` and `system` currently reach the linker through the same path: `#library "yyyprobe"` and `#library,system "yyyprobe"` both fail with `error: lld-link: error: could not open 'yyyprobe.lib'`. The distinction records intent, that a `system` library is expected to be found in the platform's own search path rather than shipped with the program.

`dyn` is different in kind. Each bound symbol gets a lazily initialized pointer slot named `__glo_dynsym_<symbol>`; the first call runs `LoadLibraryA` plus `GetProcAddress` on Windows, or `dlopen` with `RTLD_NOW` plus `dlsym` elsewhere, caches the address, and calls through it. Nothing is passed to the linker, so the library need not exist at build time. It is rejected where there is no dynamic loader: `error: #library,dyn is not supported for target 'wasm32-unknown-unknown'`.

Both malformed spellings are caught at parse time. An unrecognized modifier is `error: #library modifier must be system, dyn, or static`, and `#library,static,system "x"` is `error: #library accepts only one link-kind modifier`.

Binding a function with `#foreign`

`#foreign` is followed by either an identifier, which names a `#library` declaration, or a string, which is the library file itself and needs no `#library` declaration at all. An optional second string gives the C symbol name; without it the Glosso name is used unchanged.

GLOSSO

```
#import "Io";

libc :: #library "c";

// (a) reference a #library declaration, then name the C symbol
a_strlen :: (s: *s8) -> usize #foreign libc "strlen";

// (b) name the library file inline; no #library declaration needed
b_strlen :: (s: *s8) -> usize #foreign "c" "strlen";

// (c) omit the symbol string when the Glosso name is already the C name
strlen :: (s: *s8) -> usize #foreign libc;

main :: () {
    print("% % %\n", a_strlen("aa"), b_strlen("bbb"), strlen("cccc"));
}
```

CONSOLE / OUTPUT

```
2 3 4
```

`#foreign` makes the declaration bodyless. At file scope its `;` is required; omitting it, including by attaching a block directly, first reports `error: declaration must end with ';'`. If the terminator is written before that block, the declaration ends and the now-stray brace is `error: bad decl name '{'`. Naming a library that was never declared is `error: #foreign refers to unknown library 'nosuchlib' (declare it with #library)`.

From library name to linker argument

The compiler does not pass `#library` strings through verbatim. Each name is translated into the spelling the target's linker expects, so one source file can name `c` and link against the C runtime on every hosted platform.

Target	`c` or `libc`	Any other bare name
`*-windows-msvc`	`legacy_stdio_definitions.lib ucrt.lib`	`<base>.lib`
`*-windows-gnu`	`libmsvcrt.a`	`lib<base>.a`
ELF and Mach-O	nothing; libc is implicit	`-l<base>`

A name that already contains `/` or `\` or ends in a known library extension is passed through untouched, which is how a vendored archive is named. A leading `lib` is stripped before the target spelling is applied, so `libpng` and `png` produce the same argument.

CONSOLE / OUTPUT

```
# the program declares  zzz :: #library "zzzprobe";
$ glosso -o p08.exe p08_libnames.glo
error: lld-link: error: could not open 'zzzprobe.lib': no such file or directory

$ glosso --target x86_64-linux-musl -o p08l p08_libnames.glo
error: ld.lld: error: unable to find library -lzzzprobe
```

- Where a self-contained MinGW bundle is not available, an `x86_64-windows-gnu` link is re-planned onto the installed Windows SDK and the `lib<base>.a` spelling is rewritten back to `<base>.lib`. The observed failure on such a host names `zzzprobe.lib`, not `libzzzprobe.a`.
- If any reachable `#foreign` call uses a `dyn` library, `-ldl` is added on non-Windows, non-wasm targets so that `dlopen` resolves.

The reachability rule

Library arguments are collected by walking the runtime-reachable functions and recording the library of every non-dynamic `#foreign` call found there. A declaration that is never called contributes nothing. The rule exists so that a large bindings file, most of which the program does not use, does not drag its library onto the link line.

GLOSSO

```
#import "Io";

ghost :: #library "no_such_library_at_all";
never_called :: (x: s32) -> s32 #foreign ghost "never_called";

main :: () {
    print("linked without the ghost library\n");
}
```

CONSOLE / OUTPUT

```
linked without the ghost library
```

Adding one call to `never_called` changes the link, and the library that was invisible a moment ago becomes a hard requirement: `error: lld-link: error: could not open 'no_such_library_at_all.lib': no such file or directory`.

> **NOTE**
> The reachability rule is measured after pruning, not after type checking. A `#foreign` declaration reached only from dead code, or only from a branch a `#if` removed, does not put its library on the link line, and the build succeeds on a machine where that library is not installed.

Passing arguments to the linker directly

`--linker-arg <arg>` appends one raw argument and may be repeated. The final order is the plan's own flags and objects, then the library arguments derived from reachable `#foreign` calls, then every `--linker-arg` in command-line order, then anything a build script added with `add_current_linker_arg`. It is the mechanism for search paths, subsystem selection, and any flag Glosso has no directive for.

CONSOLE / OUTPUT

```
$ glosso -o usegreet.exe usegreet.glo
glosso: compile usegreet.glo (target=x86_64-pc-windows-msvc)
glosso: link usegreet.exe (target=x86_64-pc-windows-msvc, -O1)
error: lld-link: error: could not open 'greet.lib': no such file or directory

$ glosso --linker-arg /libpath:vendor -o usegreet.exe usegreet.glo
glosso: compile usegreet.glo (target=x86_64-pc-windows-msvc)
glosso: link usegreet.exe (target=x86_64-pc-windows-msvc, -O1)
glosso: done usegreet.exe  (backend=llvm, target=x86_64-pc-windows-msvc, -O1)

$ ./usegreet.exe
greet_add(19, 23) = 42
```

The value is not translated between linker flavours. On `x86_64-pc-windows-msvc` the linker is `lld-link`, so a POSIX spelling is dropped with `lld-link: warning: ignoring unknown argument '-L.'`. That warning is only visible when the link fails; a successful link does not surface the linker's own output.

A complete link against a system library

The program below binds one symbol from a Windows import library through `#library,system` and one from a DLL through `#library,dyn`, so only the first contributes a linker argument. Constants for the C header's `#define` names are written as ordinary Glosso constants.

GLOSSO

```
#import "Io";

kernel32     :: #library,dyn "kernel32.dll";
GetTickCount :: () -> u32 #foreign kernel32 "GetTickCount";

user32           :: #library,system "user32.lib";
GetSystemMetrics :: (index: s32) -> s32 #foreign user32 "GetSystemMetrics";

SM_CXSCREEN :: 0;
SM_CYSCREEN :: 1;

main :: () {
    print("tick count nonzero: %\n", GetTickCount() > 0);
    print("screen: % x %\n",
        GetSystemMetrics(SM_CXSCREEN),
        GetSystemMetrics(SM_CYSCREEN));
}
```

CONSOLE / OUTPUT

```
glosso: compile f03.glo (target=x86_64-pc-windows-msvc)
glosso: link f03.exe (target=x86_64-pc-windows-msvc, -O1)
glosso: done f03.exe  (backend=llvm, target=x86_64-pc-windows-msvc, -O1)
tick count nonzero: 1
screen: 1600 x 1000
```

14.2 Being Called from C

`#c_call` selects the target's C calling convention and layout rules for the declaration it is attached to. On a Glosso procedure with a body it turns that procedure into an export a C program can call; on a function-pointer type it describes a callback C will invoke; on a struct it selects C field order.

A `#c_call` procedure is an ordinary Glosso procedure in every other respect. It can be called from Glosso, it participates in overload resolution, and it is type checked normally. The one thing it gives up is the implicit context, which has no place in a C signature.

GLOSSO
```
// glomath.glo: build with --backend static-lib --no-main

add_s32 :: (left: s32, right: s32) -> s32 #c_call {
    return left + right;
}

scale_f64 :: (value: f64, factor: f64) -> f64 #c_call {
    return value * factor;
}
```

Site	Spelling	Effect
procedure	`f :: (…) -> T #c_call { … }`	C ABI, C-callable export, library root
function pointer	`#fn_ptr(…) -> T #c_call`	C ABI; part of the pointer's type
struct	`S :: struct #c_call { … }`	fields keep declaration order

`#c_call` does not declare an import. A declaration with no body is only legal with `#foreign` or `#magic`; writing `f :: (a: s32) -> s32 #c_call;` is `error: unexpected token ';'` followed by `error: unknown function f`. Calling into C is `#foreign`, which is handled in Calling C from Glosso; `#c_call` is the outbound direction.

Function pointers carry their convention

The calling convention is part of a `#fn_ptr` type, not a property of the value stored in it. A Glosso procedure without `#c_call` cannot be passed where a `#c_call` pointer is expected; the diagnostic names both sides by kind: `error: arg type fnptr != fnptr in call c_qsort`. This is what makes a callback handed to C safe to invoke.

GLOSSO
```
#import "Io";

Compare :: #fn_ptr(a: *void, b: *void) -> s32 #c_call;

libc    :: #library "c";
c_qsort :: (base: *mut void, count: u64, size: u64, compare: Compare)
    #foreign libc "qsort" ;

compare_s32 :: (a: *void, b: *void) -> s32 #c_call {
    left  := (cast(*s32)a).*;
    right := (cast(*s32)b).*;
    if left < right return -1;
    if left > right return 1;
    return 0;
}

main :: () {
    values: [5]s32;
    values[0] = 5; values[1] = 1; values[2] = 4; values[3] = 2; values[4] = 3;
    c_qsort(cast(*mut void)values.data, 5, 4, compare_s32);
    for i: 0..5 print("% ", values[i]);
    print("\n");
}
```

CONSOLE / OUTPUT
```
1 2 3 4 5
```

`#c_call` on a struct

An ordinary Glosso struct is free to reorder its fields, and it does: storage is sorted by descending alignment, then descending size, then declaration order. `#c_call` suppresses that and lays the fields out in the order written, which is what a C declaration of the same type produces. The general rules are in Struct Memory Layout.

GLOSSO
```
#import "Io";
#import "Meta";

Plain :: struct {
    a: u8;
    b: u32;
    c: u8;
}

C_Layout :: struct #c_call {
    a: u8;
    b: u32;
    c: u8;
}

main :: () {
```

```
    print("Plain    size = %\n", size_of(Plain));
    print("C_Layout size = %\n", size_of(C_Layout));
}
```

CONSOLE / OUTPUT

```
Plain    size = 8
C_Layout size = 12
```

> **NOTE**
> Stable compiler-service records need the same discipline. The `Compiler` and `Bindgen` modules now declare their private service result records with `#c_call`, preserving the service field order on 32-bit output targets as well as the host. Application code should call `Native_Compile_Output.compile_c_archive` and `generate_source` and match their public `Result` values, not copy the private records or redeclare their raw `#magic` entry points.

The implicit context

A C caller has no context to pass, so a `#c_call` procedure may not read one: `error: cannot access 'context' in a #c_call / #no_context procedure`. An export that needs an allocator must take it as a parameter or install one itself. `#no_context` imposes the same restriction without changing the calling convention.

The `glo_` symbol prefix

Glosso does not own the C namespace, so every symbol it defines is prefixed with `glo_`. A `#c_call` export named `add_s32` is the native symbol `glo_add_s32`, and the C header that declares it must say so. The prefix is what lets a Glosso program define `open` or `write` without colliding with the C runtime.

Declaration	Native symbol
`f :: (…) #c_call { … }`	`glo_f`
any other Glosso procedure `f`	`glo_f`
`main`, through the hosted entry wrapper	`glo_main`, called from `wmain` or `main`
`f :: (…) #foreign lib "sym"`	`sym`, exactly as declared

CONSOLE / OUTPUT

```
$ cat symbols.glo
exported :: (a: s32) -> s32 #c_call { return a + 1; }
plain    :: (a: s32) -> s32 { return a + 2; }
libc     :: #library "c";
c_strlen :: (s: *s8) -> usize #foreign libc "strlen";
main     :: () { x := exported(1) + plain(2); y := c_strlen("ab"); }

$ glosso --target x86_64-windows-msvc --backend llvm-ir -o symbols.ll symbols.glo
$ grep -E "glo_exported|glo_plain|glo_main|@strlen|@wmain" symbols.ll
define i32 @glo_exported(i32 %0) {
define i32 @glo_plain(i32 %0) {
define void @glo_main() {
declare i64 @strlen(ptr)
define i32 @wmain(i32 %0, ptr %1) {
```

One narrow exception exists: on a non-Windows, non-wasm target that is also the host, a `#c_call` procedure named `main`, or one that lowered to no code at all, keeps its bare name. Everything else, on every target, takes the prefix. WebAssembly exports follow the same rule, which is why the wasm link line reads `--export=glo_main`.

`--no-main` and library roots

Reachability normally starts at `main`. A library has no `main`, so the roots must change: with `--no-main`, or with `--backend static-lib` or `--backend shared-lib`, every defined `#c_call` procedure becomes a root and the hosted entry wrapper and the `argc`/`argv` globals are not emitted. Procedures reachable only from those roots are lowered with internal linkage; procedures reachable from nothing are dropped.

GLOSSO

```
exported_root :: (a: s32) -> s32 #c_call { return helper(a); }

helper :: (a: s32) -> s32 { return a * 3; }

not_reached :: (a: s32) -> s32 { return a - 1; }

also_exported :: (a: f64) -> f64 #c_call { return a / 2.0; }
```

CONSOLE / OUTPUT
```
$ glosso --backend static-lib --no-main -o roots.lib roots.glo
glosso: compile roots.glo (target=x86_64-pc-windows-msvc)
glosso: link roots.lib (target=x86_64-pc-windows-msvc, -O1)
glosso: done roots.lib  (backend=static-lib, target=x86_64-pc-windows-msvc, -O1)

$ llvm-nm --defined-only --extern-only --just-symbol-name roots.lib
__real@3fe0000000000000
_fltused
glo_also_exported
glo_exported_root
```

`not_reached` is gone and `helper` survives only as an internal symbol. The rule cuts the other way too: in a normal executable build, a defined but uncalled `#c_call` procedure is pruned exactly like any other dead code. The same source compiled without `--no-main` contains no `glo_exported` at all.

A static library consumed from C

CONSOLE / OUTPUT
```
/* driver.c */
#include <stdio.h>

int    glo_add_s32(int left, int right);
double glo_scale_f64(double value, double factor);

int main(void) {
    printf("glo_add_s32(19, 23) = %d\n", glo_add_s32(19, 23));
    printf("glo_scale_f64(2.5, 4.0) = %g\n", glo_scale_f64(2.5, 4.0));
    return 0;
}
```

CONSOLE / OUTPUT
```
$ glosso --backend static-lib --no-main -o glomath.lib glomath.glo
glosso: compile glomath.glo (target=x86_64-pc-windows-msvc)
glosso: link glomath.lib (target=x86_64-pc-windows-msvc, -O1)
glosso: done glomath.lib  (backend=static-lib, target=x86_64-pc-windows-msvc, -O1)

$ clang driver.c glomath.lib -o driver.exe
$ ./driver.exe
glo_add_s32(19, 23) = 42
glo_scale_f64(2.5, 4.0) = 10
```

`--backend shared-lib` produces the same exports in a dynamic library and additionally writes an import library where the platform uses one (`glomath.dll` plus `glomath.dll.lib` on Windows). The artifact names per backend and target are tabulated in Backends, Artifacts, and Optimization Levels.

> **NOTE**
> Linking two separately compiled Glosso objects or archives into one image collides on the floating-point marker symbol: `error: lld-link: error: duplicate symbol: _fltused`, reported against both objects. `--linker-arg /FORCE:MULTIPLE` gets past it, and the shared library route avoids it entirely. Linking one Glosso archive into a C program, as above, does not hit this.

14.3 C Data Across the Boundary

For fixed C parameters and results, the declared type selects the ABI image; the compiler does not repair a mismatched declaration with an implicit numeric conversion. Apart from the string-to-byte-pointer adaptation below and C's default promotions in a variadic tail, the argument is handed to the callee as that image. The checker verifies the Glosso signature that was written down, so a wrong signature becomes a wrong call.

The fixed-width integers, `f32`/`f64`, pointers, an enum with an explicit backing type, and a compatible `#c_call` struct can describe C data directly. A `distinct` type keeps its underlying representation. This is not a promise that every Glosso primitive has a C ABI: complex, SIMD, and matrix values require an explicit boundary representation. Compiling `#c_call` signatures to textual IR shows the lowering directly.

GLOSSO
```
Color :: enum u8 { Red; Green; Blue; }
Point :: struct #c_call { x: s32; y: s32; }
Meters :: distinct f64;
Handler :: #fn_ptr(value: s32) -> s32 #c_call;

f_scalar :: (a: s8, b: s32, c: u64, d: f32, e: f64) -> f64 #c_call { return e; }
f_bool   :: (a: bool) -> bool #c_call { return a; }
f_enum   :: (a: Color) -> Color #c_call { return a; }
f_ptr    :: (a: *mut s32, b: [*]mut u8, c: ?*mut s32, d: *mut void) -> *mut s32 #c_call { return a; }
f_fnptr  :: (a: Handler) -> Handler #c_call { return a; }
f_struct :: (a: Point) -> Point #c_call { return a; }
f_dist   :: (a: Meters) -> Meters #c_call { return a; }
```

```
f_array  :: (a: [4]s32) -> s64 #c_call { return 0; }

f_string :: (a: string) -> ssize #c_call { return a.count; }
f_view   :: (a: []mut u8) -> ssize #c_call { return a.count; }
f_option :: (a: ?s32) -> s64 #c_call { return 0; }
```

CONSOLE / OUTPUT

```
$ glosso --backend llvm-ir --no-main -o f04.ll f04.glo
define double @glo_f_scalar(i8 %0, i32 %1, i64 %2, float %3, double %4) {
define zeroext i1 @glo_f_bool(i1 zeroext %0) {
define i8 @glo_f_enum(i8 %0) {
define ptr @glo_f_ptr(ptr %0, ptr %1, ptr %2, ptr %3) {
define ptr @glo_f_fnptr(ptr %0) {
define i64 @glo_f_struct(i64 %0) {
define double @glo_f_dist(double %0) {
define i64 @glo_f_array([4 x i32] %0) {
define i64 @glo_f_string({ i64, ptr, i64 } %0) {
define i64 @glo_f_view({ i64, ptr } %0) {
define i64 @glo_f_option(%Option_s32 %0) {
```

Glosso type	Machine image	C spelling
`s8` `s16` `s32` `s64`	`i8` `i16` `i32` `i64`	`int8_t` through `int64_t`
`u8` `u16` `u32` `u64`	`i8` `i16` `i32` `i64`	`uint8_t` through `uint64_t`
`ssize`, `usize`	the target pointer width (`i32` or `i64`)	`intptr_t`, `uintptr_t`
`rune`	`i32`	`uint32_t`; a Unicode scalar, not an integer alias
`f32` `f64`	`float` `double`	`float`, `double`
`bool`	`i1 zeroext`	`_Bool`
`enum u8`, `enum_flags u32`	the backing integer	the enum's underlying type
`*mut T`, `*T`, `[*]T`, `*void`	`ptr`	`T *`, `const T *`, `void *`
`?*T`, `?[*]T`	`ptr`	a pointer that may be null
`#fn_ptr(…) #c_call`	`ptr`	a function pointer
`struct #c_call`	declared field order	the matching `struct`
`distinct T`	the image of `T`	whatever `T` maps to
`[N]T`	`[N x T]`	`T[N]` in a struct field

The remaining aggregate types are Glosso descriptors with no automatic C counterpart. A descriptor accepted in a `#c_call` signature passes the record, not the pointed-to data, so the other side must deliberately match its layout. `#c_call` does not turn a `string` into `char *` or a view into a pointer-and-length parameter pair. `word` below is the target's `ssize` width: 32 bits on a 32-bit target and 64 on a 64-bit one.

Glosso type	Machine image	Reason
`string`, `string16`	`{ ptr, word }`	data followed by count; no capacity
`cstring`, `cstring16`	`ptr`	nullable pointer; length is found by scanning for the terminator
`[]T`	`{ word, ptr }`	count and data
`[..]T`	`{ word, ptr, word }`	count, data, capacity
`?T` for non-pointer `T`	tag plus payload	a tagged union, not a null value
`E!T`, tagged `union`	tag plus aligned payload (unless a built-in niche applies)	a tagged union
tuples	elements in tuple order, with padding	a record, not a tagged union

NOTE

The checker rejects by-value complex primitives at C boundaries, including fields in `struct #c_call`: pass the real and imaginary components separately. `Simd(T, N)` has no portable C ABI; pass scalar memory and use `simd_load`/`simd_store` inside Glosso.

> Matrices likewise cross through a pointer to scalar storage. `f80` and `f128` must match the exact target's floating format and convention, not merely a C type with the same storage size; `f80` is not a portable alias for C `long double`.

`string` at a pointer parameter

A runtime `string` is a counted view and does not implicitly convert to a C pointer parameter. String literals and `cstring` values can pass to a compatible character pointer. Convert a counted view with `text.c_string()` for a temporary terminated copy, or pass its `.data` explicitly only when the foreign API also accepts a separate length.

String literals are stored NUL-terminated in the image. A substring borrows a byte range and does not add a terminator at its end. A buffer assembled by hand may have no terminator at all: the program below prints its eight bytes and then keeps going into whatever follows them in memory.

```
GLOSSO
#import "Io";

c_puts :: (s: *s8) -> s32 #foreign "c" "puts" ;

main :: () {
    c_puts("a string literal is NUL-terminated in the image");
    buf: [8]u8;
    buf[0] = cast(u8)(#char "A"); buf[1] = cast(u8)(#char "B");
    buf[2] = cast(u8)(#char "C"); buf[3] = cast(u8)(#char "X");
    buf[4] = cast(u8)(#char "Y"); buf[5] = cast(u8)(#char "Z");
    buf[6] = cast(u8)(#char "!"); buf[7] = cast(u8)(#char "?");
    c_puts(cast(*s8)buf.data);
}
```

The first line prints as written. The second prints `ABCXYZ!?` followed by trailing garbage that changes between runs, which is the whole reason `cstring` exists.

`cstring`

`cstring` is a nullable pointer to zero-terminated bytes. Its `.count` scans to the first zero byte, and a null value has count zero. It has no capacity and does not record ownership. A literal or foreign pointer is borrowed; an explicit allocating conversion carries a release obligation. The conversions live in `std/Strings/Utf8`.

The explicit `c"hello"` literal infers `cstring` and borrows NUL-terminated static storage, so it requires no allocating conversion and must not be freed. An embedded zero ends its visible `.count`. Use `b"..."` for exact byte data: it infers a fixed `u8` array, retains embedded zeros, and adds no terminator. `\xNN` inserts one byte, while ordinary Unicode text contributes its UTF-8 bytes.

```
GLOSSO
#import "Io";

c_puts :: (s: *s8) -> s32 #foreign "c" "puts";

main :: () {
    text := c"hello";
    bytes: [_]u8 = b"A\0B";
    bytes[2] = 67;
    _ := c_puts(text);
    print("C count = %, byte count = %\n", text.count, bytes.count);
}
```

Call	Result	Storage
`text.cstring_from_string(allocator)`	`cstring`	`context.allocator` by default; release it
`text.c_string()`	`cstring`	the temporary allocator; no release
`text.to_string()` on a `cstring`	`string`	a counted view borrowing the same bytes
`text.free(allocator)`	nothing	releases a `string.cstring_from_string` result

```
GLOSSO
#import "Io";
#import "Strings/Utf8";

c_atoi :: (s: *s8) -> s32 #foreign "c" "atoi" ;

main :: () {
    temporary := ("1234").c_string();            // temp-allocator copy
```

```
    print("atoi = %\n", c_atoi(temporary));

    owned := ("hello").cstring_from_string();     // context.allocator copy
    defer owned.free();
    print("count = % round trip = %\n", owned.count, owned.to_string());
}
```

CONSOLE / OUTPUT

```
atoi = 1234
count = 5 round trip = hello
```

NOTE

A `cstring` has pointer representation, so it can also appear directly in a matching C-ABI signature. A counted `string` still passes a pointer-and-count aggregate; declaring it where C expects `char *` does not convert the argument. Match the actual foreign signature.

C varargs

A final `...` parameter marks a C-ABI variadic. It is accepted only on `#foreign` declarations and `#c_call` definitions; anywhere else it is `error: C vararg marker '...' on 'f' is only supported on #foreign declarations or #c_call definitions`. Operators and macros are rejected separately, and a signature with `$T` or compile-time parameters gives `error: C vararg marker '...' on 'f' requires concrete runtime parameters`. Glosso's own variadics are a different feature, described in Variadic Parameters.

GLOSSO

```
#import "Io";
#import "VarArg";

libc   :: #library "c";
printf :: (format: *s8, ...) -> s32 #foreign libc "printf";

sum_s64 :: (count: ssize, ...) -> s64 #c_call {
    ap: Va_List;
    ap.start();
    defer ap.end();
    total: s64 = 0;
    i: ssize = 0;
    while i < count {
        total += arg(s64, &ap);
        i += 1;
    }
    return total;
}

main :: () {
    #if OUT.os == Target_OS.Wasm {
        print("printf through C varargs skipped on WASI\n");
    } else {
        printf("printf through C varargs: n=%lld flag=%d\n",
            cast(s64)42, cast(s32)true);
    }
    print("sum_s64(10, 20, 12) = %\n",
        sum_s64(3, cast(s64)10, cast(s64)20, cast(s64)12));
}
```

CONSOLE / OUTPUT

```
sum_s64(10, 20, 12) = 42
printf through C varargs: n=42 flag=1
```

`std/VarArg` supplies `Va_List`, which is opaque target-ABI storage, plus `start`, `end`, `copy`, and `arg(T, *Va_List)`. It also wraps `vprintf`, `vfprintf`, and `vsnprintf` over an opaque `C_File`. This is a native run's output; WASI skips `printf` but still exercises `sum_s64`. Glosso's `print` and C's `printf` buffer independently, so the output order can vary. Each value consumed by `arg(s64, &ap)` is explicitly `s64`: an unannotated integer otherwise defaults to target-width `ssize`, which is only 32 bits on a 32-bit target. `%lld` matches the 64-bit C `long long`; `%ld` asks for `long`, which is 32 bits even on 64-bit Windows.

NOTE

C's default argument promotions do happen in the variadic tail: `bool`, `s8`, `u8`, `s16`, and `u16` become `s32`, while `f16` and `f32` become `f64`. A string literal decays to a character pointer, so `c_printf("%s\n", "hello")` is accepted. A runtime counted `string` is rejected in the variadic tail. Pass `text.c_string()` for `%s`, or pass an `s32` precision and `.data` separately for `%.*s`.

14.4 Generating Bindings with Bindgen

`std/Bindgen` turns a C header into Glosso declarations. It runs during compile-time execution, which means bindings can be produced and consumed in the same build, and it reads the header for the target being compiled for rather than the machine running the compiler: `long tgt_long(long a)` generates `(a: s32) -> s32` on `x86_64-windows-msvc` and `(a: s64) -> s64` on `x86_64-linux-musl`.

There are two entry points. Both take the header path, a library descriptor, and optional settings as named parameters, and both return `Bindgen_Error!string`. The library argument must satisfy `Bindgen_Library`: use a `Library_Spec`, a bare library string, or `Libraries` when the header contains functions from several artifacts.

Call	Produces	Used with
`generate(header, output, library)`	writes a `.glo` file	a checked-in bindings file
`generate_source(header, library)`	returns the source text	`#insert`, no file on disk

GLOSSO

```
#import "Io";
#import "Bindgen";

emit :: () {
    if generate("counter.h", "counter_bindings.glo", Library_Spec.library_spec("counter")) == {
    case .Ok(message); print("[bindgen] %\n", message);
    case .Err(error);  print("[bindgen] failed: %\n", error.message);
    }
}

#comptime emit();

main :: () {}
```

CONSOLE / OUTPUT

```
/* counter.h */
#define COUNTER_VERSION 3

typedef struct Counter_Stats {
    int hits;
    int misses;
} Counter_Stats;

typedef struct Counter Counter;

Counter *counter_open(int start);
void counter_close(Counter *counter);
int counter_bump(Counter *counter, int by);
Counter_Stats counter_stats(Counter *counter);
```

GLOSSO

```
// Generated by Glosso's Clang C header bindgen from counter.h.
counter :: #library,static "counter";

#public {
Counter_Stats :: struct #c_call {
    hits: s32;
    misses: s32;
}

Counter :: struct #c_call {}

COUNTER_VERSION :: 3;

#memory { unknown(result: returned) }
counter_open :: (start: s32) -> ?[*]mut Counter #foreign counter "counter_open";

#memory { writes(counter) }
counter_close :: (counter: ?[*]mut Counter) #foreign counter "counter_close";

#memory { writes(counter) }
counter_bump :: (counter: ?[*]mut Counter, by: s32) -> s32 #foreign counter "counter_bump";

#memory { writes(counter) }
counter_stats :: (counter: ?[*]mut Counter) -> Counter_Stats #foreign counter "counter_stats";

}
// bindgen: 2 type declarations, 0 aliases, 0 callbacks, 1 constants, 0 macros, 4 functions, 0 skipped declaration parts/macros
(libclang).
```

Generated foreign declarations carry leading #memory blocks. C pointer constness determines reads or writes; pointer results use unknown(result: returned) because a C signature does not establish storage origin or ownership. A name such as counter_close does not prove release behavior. Supply the API's actual returns_fresh, released_by, kills, noescape, and escape rules before relying on it as a memory boundary.

The `.Ok` payload of `generate` is a status line, not a path: the run above printed `[bindgen] bindgen: wrote counter_bindings.glo`. The `.Ok` payload of `generate_source` is the source text itself.

Library descriptors

The library argument decides the `#library` declaration at the head of the generated file. The descriptor's name becomes the Glosso identifier every `#foreign` in the file refers to; its file or path becomes the string. `Library_Spec.library_spec(name, .file = "", .kind = .Static)` defaults an empty file to the name for `.Static` and `.System`. `.Dynamic` requires a path.

Descriptor	Emitted declaration
`Library_Spec.library_spec("small")`	`small :: #library,static "small";`
`Library_Spec.library_spec("small", "build/libsmall.a")`	`small :: #library,static "build/libsmall.a";`
`Library_Spec.library_spec("m", .kind = .System)`	`m :: #library,system "m";`
`Library_Spec.library_spec("small", "libsmall.so", .Dynamic)`	`small :: #library,dyn "libsmall.so";`

Direct generation parameters

GLOSSO

```
emit_small :: () -> Bindgen_Error!string {
    includes: [1]string = .["include"];
    definitions: [1]string = .["SMALL_EXTRA=1"];
    names: [2]string = .["small_add", "SMALL_VERSION"];
    return generate_source("small.h", Library_Spec.library_spec("small"), .include_paths =
includes[..], .preprocessor_definitions = definitions[..], .allowlist = names[..]);
}
```

Parameter	Becomes	Default
`include_paths`	`-I` arguments	empty
`preprocessor_definitions`	`-D` arguments, `NAME` or `NAME=VALUE`	empty
`compiler_args`	raw arguments to clang	empty
`allow_textual_fallback`	a limited non-clang parser	`false`
`allowlist`	exact declaration names and their transitive dependencies	empty, generating all supported local declarations

Include paths, definitions, compiler arguments and allowlists are ordinary `[]mut string` parameters, each defaulting to `.[]`. The call borrows these slices and their text only while generating. Pass a fixed array's `[..]` view or a slice; no option record or specialized array mutator is needed. Definitions use `NAME` or `NAME=VALUE` without `-D`, so `SMALL_EXTRA=1` selects the corresponding preprocessor branch. A nonempty allowlist also reaches declarations in external SDK includes, requires Clang even when textual fallback is enabled, and rejects unknown names.

`allow_textual_fallback` is `false` because a clang error is more useful than a plausible-looking guess. With it off, a header clang cannot preprocess fails the call with clang's own message, prefixed `bindgen_generate_source: Clang could not preprocess <path> (status 1):`. With it on, the same header is scanned by a textual parser that understands simple declarations and object-like macros and ignores what it cannot follow.

GLOSSO

```
// broken.h begins with: #include <no_such_header_anywhere.h>

emit :: (allow: bool) {
    if generate_source("broken.h", Library_Spec.library_spec("broken"), .allow_textual_fallback = allow) == {
    case .Ok(source);  print("[ok, allow=%]\n%\n", allow, source);
    case .Err(error);  print("[err, allow=%] %\n", allow, error.message);
    }
}
```

The banner distinguishes the two parsers: libclang output says `Clang C header bindgen` and ends its summary with `(libclang)`, the textual parser says `C header bindgen` and omits the marker.

What the generated declarations look like

CONSOLE / OUTPUT

```
/* rich2.h */
#include <stddef.h>
```

```
typedef enum Rich_Mode {
    RICH_MODE_OFF = 0,
    RICH_MODE_ON  = 1
} Rich_Mode;

typedef unsigned long long Rich_Id;

typedef int (*Rich_Callback)(int value, void *user);

typedef struct Rich_Config {
    Rich_Mode     mode;
    Rich_Id       id;
    const char   *name;
    Rich_Callback on_event;
} Rich_Config;

void   rich_configure(const Rich_Config *config);
char  *rich_describe(Rich_Mode mode);
size_t rich_count(void);
```

GLOSSO

```
// Generated by Glosso's Clang C header bindgen from rich2.h.
rich :: #library,static "rich";

Rich_Config :: struct #c_call {
    mode: s32;
    id: u64;
    name: ?[*]s8;
    on_event: ?Rich_Callback;
}

Rich_Mode :: s32;

Rich_Id :: u64;

Rich_Callback :: #fn_ptr(arg0: s32, arg1: ?[*]mut void) -> s32 #c_call ;

RICH_MODE_OFF :: 0;

RICH_MODE_ON :: 1;

rich_configure :: (config: ?[*]Rich_Config) #foreign rich "rich_configure" ;

rich_describe :: (mode: s32) -> ?[*]mut s8 #foreign rich "rich_describe" ;

rich_count :: () -> u64 #foreign rich "rich_count" ;

// bindgen: 1 type declarations, 2 aliases, 1 callbacks, 2 constants, 0 macros, 3 functions, 0 skipped declaration parts/macros
(libclang).
```

- Every record becomes a `struct #c_call`, so field order matches the C declaration. An opaque `typedef struct Counter Counter;` becomes an empty `struct #c_call {}` that exists only to be pointed at.
- A C `enum` becomes an integer alias plus one constant per enumerator, not a Glosso `enum`. Enumerators are top-level constants, so `RICH_MODE_ON` is written bare rather than as `.On`.
- Function-pointer typedefs become `#fn_ptr … #c_call` with positional parameter names `arg0`, `arg1`, and so on; C parameter names are not carried across. Uses of that typedef in parameters, fields, and return values are optional by default, such as `?Rich_Callback`. A `_Nonnull` annotation makes a C pointer or callback nonoptional.
- Bindgen maps C `T *` to `?[*]mut T` and `const T *` to `?[*]T`. These types preserve pointee mutability. Match optional pointers before use; read-only storage cannot be passed to an API that writes through its pointer.
- Object-like macros with constant values become constants. Static C functions have no exported symbol and are skipped, as are macros that have no Glosso translation. A `bindgen skipped` note identifies the omission, and the summary counts emitted and skipped declarations.

Ownership at the foreign boundary

Generated declarations describe C layouts and calling conventions. They do not track allocation, retention, or release. Wrap owning foreign handles in a type with Drop, or pair raw acquisitions with their documented release functions using defer.

14.5 Inline Assembly and Inline Bytes

`#asm` places target instructions directly in a procedure body. The instructions are written as text, but the values they read and write are named Glosso locals and expressions rather than register numbers, so register allocation stays with the compiler. A block declares what it reads, what it writes, and what it destroys; everything it does not declare is assumed untouched.

There are two spellings. The structured form opens a brace block after the directive and is the one to write. An older positional form takes a single string and colon-separated operand lists carrying raw backend constraint strings; it still parses, and it is the escape hatch for a constraint the portable vocabulary cannot spell.

GLOSSO

```
#import "Io";

add_via_asm :: (a: s64, b: s64) -> s64 {
    result: s64;
    #asm {
        \\movq {left}, {result}
        \\addq {right}, {result}
        result: out(.Register);
        left:  in(.Register) = a;
        right: in(.Register) = b;
        clobber: .Condition_Codes;
    }
    return result;
}

main :: () {
    print("asm add = %\n", add_via_asm(20, 22));
}
```

CONSOLE / OUTPUT

```
asm add = 42
```

That block lowers to one LLVM inline-assembly call. `--backend llvm-ir` shows what the compiler built out of the operand list: the placeholders became positional references, and the portable constraints became a single constraint string.

CONSOLE / OUTPUT

```
%inline.asm = call i64 asm sideeffect "movq $1, $0\0Aaddq $2, $0", "=r,r,r,~{cc}"(i64 %r0.load, i64 %r1.load)
```

A structured block has three parts and they must appear in this order: one or more template lines, then the operand bindings, then at most one `clobber:` clause. A block that starts with anything but a template line is `error: structured #asm expects one or more '\\' assembly lines`. An operand written after the clobber clause is `error: structured #asm operands must appear before the clobber clause`, and a second clobber clause is `error: structured #asm may contain only one clobber clause`.

Template lines

Each template line begins with `\\` and contributes exactly one line to the assembly text; the lines are joined with newlines and nothing else is done to them. Escape sequences are not decoded, so the line carries the characters written. Assembler directives and local numeric labels are ordinary text and pass through unchanged, which is how a multi-line sequence branches within itself.

`{name}` refers to the operand called `name`. `{name:modifier}` appends a backend operand modifier, which is how a 64-bit operand is spelled as its byte or word subregister. A literal brace is written `{{` or `}}`; a bare closing brace is `error: unmatched '}' in structured #asm template; write '}}' for a literal brace`. A placeholder naming nothing is `error: unknown structured #asm template operand '{source}'`.

GLOSSO

```
#import "Io";

rotate_left :: (value: u64, amount: u64) -> u64 {
    result := value;
    #asm {
        \\rolq {shift:b}, {result}
        result: inout(.Register);
        shift:  in(fixed(rcx)) = amount;
        clobber: .Condition_Codes;
    }
    return result;
}

popcount :: (value: u64) -> u64 {
    result: u64;
    #asm {
        \\popcntq {source}, {result}
        result: out(.Register);
        source: in(.Register) = value;
        clobber: .Condition_Codes;
    }
    return result;
}

main :: () {
    print("rotate_left(1, 4) = %\n", rotate_left(1, 4));
    print("popcount(255)     = %\n", popcount(255));
}
```

```
CONSOLE / OUTPUT
rotate_left(1, 4) = 16
popcount(255)     = 8
```

`u64.rotate_left` emits `"rolq ${1:b}, $0"` with the constraint string `"=r,{rcx},0,~{cc}"`: the modifier survived, the fixed register became a braced register name, and the `inout` operand became an output plus a tied input. `rolq` needs its shift count in `cl`, and `{shift:b}` is how a `u64` operand is named as that byte register without giving up the type.

Operand directions

Direction	Meaning	What follows the constraint
`in`	read before the block runs	`= expression`, or nothing to read the local of the same name
`out`	written by the block, previous value dead	`= local` to retarget, or nothing to write the local of the same name
`inout`	read and then written	`= local` to retarget, or nothing

An `out` or `inout` operand names storage, so its target must be a local that already exists; a name nothing declared is `error: #asm out operand needs an existing local, got 'result'`. A parameter counts as a local, so `inout` on a parameter is legal. An `in` operand names a value, so it takes any expression. The placeholder name and the target are independent: `sum: out(.Register) = total;` writes the local `total` through the placeholder `{sum}`.

An operand must have a type a machine register can hold: an integer, a float, a `bool`, a pointer, a many-pointer, or a SIMD vector. A `string` operand is `error: #asm input operand has unsupported type string` on the way in and `error: #asm out operand 'text' has unsupported type string` on the way out. A block with two outputs returns them as one aggregate value, which is why the compiler and not the programmer decides where they land.

The portable constraint vocabulary

Every operand carries either an `Asm_Constraint` member or `fixed(<register>)`. `Asm_Constraint` is declared in `std/Prelude/Assembly.glo` and is therefore always in scope. The members describe allocation semantics, not a backend's constraint letters; the compiler translates each one for the target being compiled, so user code never embeds an LLVM or GCC constraint string.

Constraint	Emitted class	Where it is accepted
`.Register`	`r`	any direction, every CPU
`.Byte_Register`	`q`	any direction, the x86 family only
`.Floating_Register`	`f`, `w` on aarch64, `t` on the arm family	any direction, every CPU
`.Vector_Register`	`x` on the x86 family, `w` on arm and aarch64, `v` on powerpc and s390x, `vr` on riscv64	any direction, those CPUs only
`.Predicate_Register`	`Yk` on the x86 family, `Upa` on aarch64	any direction, those CPUs only
`.Memory`	`m`	input operands only
`.Immediate`	`i`	input operands only
`.Constant`	`n`	input operands only
`.Address`	`p`	input operands only
`.Register_Or_Memory`, `.Register_Or_Immediate`, `.Register_Or_Memory_Or_Immediate`	`g`, `ri`, `rm`	input operands only
`.Any`	`X`	input operands only
`fixed(<register>)`	the register name in braces	any direction, register must exist on the CPU

The direction column is enforced, because an operand the block writes has to be allocated somewhere the block can write. `result: out(.Memory);` is `error: structured #asm constraint '.Memory' is valid only for input operands`. The availability column is enforced too, and it is checked against `--target`, not the host: `out(.Byte_Register)` compiled for `aarch64-linux-musl` is `error: .Byte_Register is unavailable for the aarch64 target`.

A misspelled member is `error: unknown structured #asm constraint '.Gpr'`, and a bare identifier where a constraint belongs is `error: unknown structured #asm operand placement 'rax'; expected an Asm_Constraint variant or fixed(register)`: a raw

register name must go through `fixed`. A misspelled direction is `error: unknown structured #asm direction 'outp'; expected 'in', 'out', or 'inout'`.

Fixed registers

`fixed(<register>)` pins an operand to one machine register, which is what a syscall ABI or a fixed-operand instruction requires. The name is validated against a per-CPU register table, so a wrong-architecture name is caught at compile time rather than by the assembler: `fixed(x0)` on x86_64 is `error: unknown x86_64 register 'x0'`. On MIPS the symbolic names are rewritten to their numbered form, so `fixed(a0)` and `fixed(r4)` denote the same register.

Clobbers and early clobber

The `clobber:` clause lists what the block destroys beyond its declared outputs. Entries are comma-separated and may mix the two `Asm_Clobber` members with bare register names.

Entry	Emitted as	Meaning
`.Memory`	`~{memory}`	the block reads or writes memory the compiler cannot see
`.Condition_Codes`	`~{cc}`	the block leaves the flags register in an unspecified state
a bare register name	`~{<register>}`	that register is destroyed; validated exactly like `fixed`

A member outside that pair is `error: unknown structured #asm clobber '.Flags'; expected '.Memory' or '.Condition_Codes'`. Anything comparing an instruction, and every syscall sequence, needs `.Condition_Codes`; a sequence that touches memory the compiler routed through a pointer needs `.Memory`.

`.Early_Clobber` is an optional second argument on an output constraint: `out(.Register, .Early_Clobber)`. It says the operand is written before all inputs have been consumed, which stops the allocator from putting an input and that output in the same register. Two sequential `slt` results computed from the same pair of inputs need it. It is meaningless on an input, and `in(.Register, .Early_Clobber)` is `error: structured #asm '.Early_Clobber' is valid only for out and inout operands`.

> **NOTE**
> The compiler checks the frame, never the instructions. It does not parse the template, so a sequence that writes a register absent from both the operand list and the clobber list is accepted and silently wrong. The declared lists are the whole contract.

One block per CPU

A template is target-specific by construction, so a portable procedure selects its block with `#if OUT.cpu == { … }` and ends the switch with `#compile_error`, which fires only for a target that reaches it. Static branching is covered in Static Branching with `#if`, and `OUT` in Targets and Cross-Compilation.

GLOSSO
```
#import "Io";

add_via_asm :: (a: s64, b: s64) -> s64 {
    result: s64;
    #if OUT.cpu == {
    case .X86_64;
        #asm {
            \\movq {left}, {result}
            \\addq {right}, {result}
            result: out(.Register);
            left:  in(.Register) = a;
            right: in(.Register) = b;
            clobber: .Condition_Codes;
        }
    case .Aarch64, .Riscv64;
        #asm {
            \\add {result}, {left}, {right}
            result: out(.Register, .Early_Clobber);
            left:  in(.Register) = a;
            right: in(.Register) = b;
        }
    else;
        #compile_error "add_via_asm has no instruction sequence for this CPU";
    }
    return result;
}

main :: () {
    print("asm add = %\n", add_via_asm(20, 22));
}
```

CONSOLE / OUTPUT

```
asm add = 42
```

That program builds for `x86_64-pc-windows-msvc`, `aarch64-linux-musl` and `riscv64-linux-musl`; for `mips-linux-musl` it stops at the guarded `#compile_error`. `examples/inline_asm.glo` is the full version of this pattern: raw Linux syscalls and comparison sequences for every CPU the Linux musl targets cover, using `inout(fixed(rax))`, `out(fixed(r7))`, `.Early_Clobber`, and register clobber lists mixed with `.Memory` and `.Condition_Codes`.

`#bytes`

`#bytes <expression>;` splices pre-assembled machine code into the function body. The expression is const-evaluated and must be a list of compile-time integers in `0..255`, which in practice means an array literal of `16#xx` values bound with `::`. It is the tool for an instruction the assembler in use will not accept, or for a sequence that has already been assembled elsewhere.

GLOSSO

```
#import "Io";

// 0xf3 0x90 is `pause`, the x86 spin-wait hint. It reads and writes nothing.
PAUSE_X86 :: .[16#f3, 16#90];

spin :: (rounds: ssize) {
    i := 0;
    while i < rounds {
        #if OUT.cpu == {
        case .X86_64, .X86, .I386; #bytes PAUSE_X86;
        else;                      #compile_error "no spin hint for this CPU";
        }
        i += 1;
    }
}

main :: () {
    spin(4);
    print("spun\n");
}
```

CONSOLE / OUTPUT

```
spun
```

Each byte becomes one `.byte` directive in an inline-assembly block with an empty constraint string: the emitted IR for that loop body is `call void asm sideeffect ".byte 0xf3\0A.byte 0x90", ""()`. Nothing else is attached to it.

> **NOTE**
> Because `#bytes` declares no operands and no clobbers, the byte sequence must be self-contained: it cannot name a Glosso value, and it must restore every register the surrounding code and the target ABI rely on. `examples/inline_bytes.glo` does exactly that: each of its thirteen per-CPU stubs saves the non-volatile registers it uses, carries its own message bytes at the end, and branches over them to continue.

The operand must fold. A non-list is `error: #bytes expects a compile-time list of bytes, got 5`; a value outside the range is `error: #bytes value must be a compile-time integer 0..255, got 300`; a runtime array is rejected while evaluating the name, as `error: unknown const code`.

Neither form runs at compile time

Both directives emit machine instructions, and neither compile-time engine executes machine instructions. A procedure containing `#asm` called from a constant is `error: statement is not available at compile time`. The same call from a top-level `#comptime` reaches the compile-time VM instead and is `error: #comptime stopped: #asm (inline assembly) cannot run on the VM; use the LLVM backend`, with the matching `#bytes (inline machine code) cannot run on the VM; use the LLVM backend`. The two engines are described in What Runs When.

14.6 `#magic` and the Intrinsic Table

`#magic "<name>"` on a declaration binds it to a compiler intrinsic. The declaration has no body: its parameters and result type are the signature the checker sees, and the string picks the implementation out of a registry built into the compiler. This is how the standard library reaches the type system, the host operating system, and the machine. The standard library uses registered intrinsics alongside ordinary Glosso implementations.

`#magic` exists so that these operations stay ordinary declarations. A keyword per intrinsic would sit outside the type system; a `#magic` declaration is a normal procedure, so it participates in overloading, in `where` constraints, in typeclass instances, and in the module system. A program may write its own, and the name may be any registered one.

GLOSSO

```
#import "Io";
#import "Meta";

// A user declaration may bind any registered intrinsic name.
byte_width :: (T: type) -> ssize #magic "size_of";
same_text  :: (a: string, b: string) -> bool #magic "str_eq";

main :: () {
    print("byte_width(s32) = %\n", byte_width(s32));
    print("same_text = %\n", same_text("glosso", "glosso"));
}
```

CONSOLE / OUTPUT

```
byte_width(s32) = 4
same_text = 1
```

The name is not checked when the declaration is read. An unregistered name type-checks, and the program fails only when code generation reaches the call: `weird :: (a: s64) -> s64 #magic "no_such_intrinsic";` produces `error: native LLVM lowering has no classification for #magic 'no_such_intrinsic' [opcode Magic] while lowering 'main' at IR instruction 2`. The check happens where the call is lowered, so a declaration that is never called never produces it.

The four availability classes

Each registry entry carries two independent flags: whether the compile-time VM has an implementation, and whether the checker may evaluate the call while type checking. Their four combinations describe evaluator availability. Native LLVM lowering is classified separately, so an intrinsic available to both compile-time engines can still be deliberately forbidden in a program.

Class	Runs where	Representative names
compile-time only	checker evaluation, not the VM; native lowering rejects these queries	`size_of`, `type_info`, `field_name`, `pattern_missing`
VM and compile time	both compile-time engines; native availability is decided independently	`str_eq`, `substring`, file services, `compiler_add_build_file`
VM, not foldable	the VM but not checker evaluation; these representatives also lower natively	`exit`, `assert_failed`, `int_add`, `num_lt`
native only	neither compile-time engine; compiled code only	`ptr_add`, `va_arg`, `ctx_alloc`, `jump_goto`

Membership follows the nature of the operation. A query about a type has no runtime existence, so it is compile-time only. An operation on a raw address or on the implicit context has no meaning inside an evaluator with neither, so it is native only. Pure value operations generally work everywhere. Host compiler and bindgen services work in the two compile-time engines but are rejected by native lowering: they are services of a build, not of the program that build emits.

Family	Class	Note
type and pattern reflection	compile-time only	`type_of(value)`, `type_name`, `type_kind`, the `field_*`, `enum_*`, `variant_*` and `pattern_*` families, `simd_*`, `matrix_*`, the `range*_to_pattern` conversions
strings and string slicing	VM and compile time	`str_eq`, `str_cmp`, `substring`, `byte_string`, the `show_*` formatters
files, filesystem, processes	VM and compile time	so a build script can read the host; the handle-relative sandbox primitives stay native only
compiler and bindgen services	VM and compile time, native forbidden	`compiler_add_build_file`, `compiler_add_build_string`, `compiler_compile_c_archive`, `compiler_run_command`, `bindgen_generate_source`
complex numbers, wrapping and saturating integers, indexing, ranges	VM and compile time	`complex_*`, `int_wrap_*`, `int_sat_*`, `index_array`, the `slice*` family, `type_eq`
arithmetic and comparison templates	VM, not foldable	`int_add`, `flt_mul`, `num_lt`, `bool_and`, `ptr_eq`

Family	Class	Note
pointer arithmetic, varargs, jumps, raw memory	native only	`ptr_add`, `ptr_diff`, `ptr_index`, `memory_copy_magic`, `va_start`, `va_arg`, `jump_context_save`, `jump_goto`, `string_from_raw_parts`
allocation	native only	`ctx_alloc`, `ctx_free`, `temp_alloc`, `temp_free_all`, and the debug-allocator entries
terminal query, mode, event, and console control	native only	the `os_*_terminal_query` and `os_*_terminal_set_mode` families, plus Windows native event, clear, cursor, wrapping, and title operations

One entry is special-cased against its class. `get_cmdline_args` is registered native only, but the compile-time runtime answers it anyway, so that a build script can read the arguments after `--`. The two vectors are different: at compile time it returns the trailing arguments, at run time the process argv including `argv[0]`.

GLOSSO

```
#import "Io";
#import "Compiler";

#comptime {
    print("[ct] compiler = %\n", executable_path());
    args := get_cmdline_args();
    print("[ct] %  compile-time argument(s)\n", args.count);
    for args print("[ct] arg: %\n", it);
}

main :: () {
    args := get_cmdline_args();
    print("[rt] % argument(s)\n", args.count);
}
```

CONSOLE / OUTPUT

```
$ glosso --run services.glo -- build --release
[ct] compiler = C:\Users\almag\Github\glosso\build\bin\glosso.exe
[ct] 2  compile-time argument(s)
[ct] arg: build
[ct] arg: --release
[rt] 1 argument(s)
```

What the compile-time engines refuse

Glosso runs compile-time code in two evaluators, described in What Runs When: the constant evaluator, which walks the tree during checking, and the compile-time VM, which executes top-level `#comptime` declarations as bytecode. They reject an unavailable intrinsic differently, and the wording is the fastest way to tell which engine was running.

Situation	Diagnostic
an intrinsic with no VM implementation, reached while checking	`#magic intrinsic 'ctx_alloc' is not available during compile-time evaluation`
the same intrinsic reached from a top-level `#comptime`	`#comptime stopped: VM: unknown #magic intrinsic 'ctx_alloc'`
no compile-time evaluator installed at all	`compile-time #magic '<name>' has no evaluator`
a `#foreign` call in a compile-time expression	`'c_strlen' is foreign, so its result is not compile-time known`
a statement neither engine implements, such as `#asm`	`statement is not available at compile time`
any other VM failure inside a top-level `#comptime`	`#comptime stopped: <error>`

GLOSSO

```
#import "Io";
#import "Context";

make :: () -> s64 {
    p := alloc(8);
    p.free();
    return 1;
}
```

```
X ::= make();

main :: () { print("%\n", X); }
```

CONSOLE / OUTPUT

```
error: #magic intrinsic 'ctx_alloc' is not available during compile-time evaluation
  --> …\alloc.glo:5:10
   |
5 |     p := alloc(8);
   |          ^^^^^
```

Neither engine has a context allocator, so `alloc` and `free` are unavailable in both, and so is `temp_alloc`. Dynamic arrays and `tprint` are handled separately and do work. Allocation is covered in Allocators.

What native lowering refuses

The LLVM backend classifies every intrinsic name independently of the registry, and its four answers produce four distinct messages. Each one names the function and the IR instruction being lowered.

Classification	Message
compile-time-only query or service	`#magic 'size_of' must not reach native LLVM lowering: type introspection query must be folded before native lowering`; compiler services use `compiler service or type query must be folded before native lowering`
deliberately unlowered representation	`native LLVM lowering rejects #magic 'sb_new': string-builder runtime representation is not lowered`
classified as implemented, but no lowering matched	`native LLVM magic coverage drift: #magic '<name>' is classified as implemented but no lowering matched`
name not in the backend's table	`native LLVM lowering has no classification for #magic '<name>'`

Type queries normally disappear before this classification: a call to a compile-time intrinsic is folded during lowering when the declaration has a `type`-valued parameter. That lets `size_of(Vec3)` and `field_name(Vec3, 1)` stand in a runtime body and become constants. Bind `"size_of"` to a signature with no `type` parameter and the call instead arrives at the backend. Compiler and bindgen services reach the same class when called from ordinary runtime code; use them from `#comptime`, where the VM supplies the service.

GLOSSO

```
#import "Io";
#import "Meta";

sneaky :: (a: s64) -> s64 #magic "size_of";

main :: () {
    print("%\n", sneaky(1));
}
```

CONSOLE / OUTPUT

```
error: #magic 'size_of' must not reach native LLVM lowering: type introspection query must be folded before native lowering
[opcode Magic] while lowering 'main' at IR instruction 2
```

> **NOTE**
> String_Builder.new(), append(), and finish_source() support compile-time source generation; finish_source() consumes the builder. The ordinary make_len_cap(), write_string(), and into_string() operations also work in compile-time code. Use a transferred allocation or clone when the resulting text must outlive the builder.

`#magic` on an aggregate

`#magic` also appears on a `union` declaration, where it is a layout marker rather than an intrinsic binding and the string is not looked up in the intrinsic registry. Exactly two markers are recognised, `"option"` and `"result"`, and they select niche layout instead of a tag word. They are carried by the compiler-embedded `Option` and `Result` unions that `?T` and `E!T` resolve to, which is why redeclaring `Option` in a program is `error: type 'Option' is already declared`. Those types are described in The Option Type `?T` and The Result Type `E!T`.

14.7 Embedding Lua

`#import "Lua"` is not a binding file. `std/Lua/module.glo` contains a single statement, `#insert static_bindings_source();`, and importing it therefore runs a build: the bundled Lua 5.5 C sources are compiled for the output target, cached as a static

archive, and bindgen is run over the Lua headers so that the resulting declarations are spliced into the program. The mechanism is `#insert` plus the compile-time build API, both described in `#insert` and Build Scripts; this chapter is what that machinery adds up to.

Nothing about the module is Lua-specific. `std/Lua/Build.glo` holds the logic and has no top-level side effects of its own, so it can be imported directly by a program that wants a different linkage.

The work `static_bindings_source` does on a supported target is: check the target against the support matrix, locate the Lua source tree, hand the 32 C files to `Native_Compile_Output.compile_c_archive` with `LUA_USE_POSIX` defined on the Unix family, add `-lm` to the link on Linux, macOS and FreeBSD, and return the bindgen output for `glosso_lua.h` bound to the archive it produced.

CONSOLE / OUTPUT

```
<cache>/native/v7/lua55/x86_64-windows-msvc/<key>/lua55.lib
<cache>/native/v7/lua55/x86_64-linux-musl/<key>/liblua55.a
<cache>/generated/insert/v1/<sha256>.glo
```

The archive path is keyed by the target and by the C compile options, so a host build and a cross build coexist. The bindings themselves are a string-valued `#insert`, so they are also materialised in the generated-source cache and `--show-generated` prints the path. Both caches are described in The Build Cache and Generated Sources.

The cost is paid once. Building `examples/lua/hello.glo` with both caches empty takes about eleven seconds, almost all of it the C archive; with the archive cached but the final-artifact cache disabled, about four seconds; and because the program has no top-level `#comptime`, the finished executable is itself cacheable, so an unchanged rebuild is restored whole.

CONSOLE / OUTPUT

```
$ glosso --run lua_hello.glo
glosso: restored lua_hello.exe from the native artifact cache
Lua computed 42
```

The support matrix

The bundled runtime is compiled from C by the same native compiler the rest of the toolchain uses, so it is available exactly where that compiler has a usable target bundle. Every unmet condition is turned into a generated stub that fails with `#compile_error`, so the diagnostic arrives during checking and no artifact is written.

Target	Result
the host Windows MSVC target with a matching CPU	supported
any Linux musl target	supported, including cross-compilation from Windows
a Linux non-musl target on a Linux host with a matching CPU	supported
a Linux non-musl target anywhere else	`cross-target static Lua on Linux requires a musl target bundle`
any other Windows target, including `windows-gnu` and a different CPU	`static Lua for Windows currently requires the host Windows MSVC target`
macOS, FreeBSD, wasm, freestanding	`bundled static Lua currently supports Windows and Linux targets`
sources missing from the resource tree	`Lua sources were not found (expected thirdparty/lua or an installed lib/lua)`

A rejection is reported at the `#insert` in `std/Lua/module.glo`, with a note pointing into the generated stub, so the message itself is visible alongside the location that produced it.

CONSOLE / OUTPUT

```
$ glosso --backend object --target wasm32-wasi -o lua.o lua_hello.glo
glosso: compile lua_hello.glo (target=wasm32-unknown-wasi)
error: bundled static Lua currently supports Windows and Linux targets
   --> …\std\Lua\module.glo:44:1
    |
44 | #insert static_bindings_source();
    | ^^^^^^^
note: generated source failed here
  --> …\cache\generated\insert\v1\929f9018….glo:1:49
   |
1 | __lua_unsupported_target :: #comptime -> bool { #compile_error "bundled static Lua currently supports Windows and Linux
targets"; return false; }
   |                                                 ^^^^^^^^^^^^^^
```

Two failures are reported through the same stub but print a `[Lua]` line first: `could not compile the bundled Lua 5.5 sources` and `could not generate Lua 5.5 bindings`, each followed by the underlying compiler or bindgen message. Those are toolchain failures rather than target rejections.

What the bindings look like

The spliced source is ordinary bindgen output, about a thousand lines of it: 5 type declarations, 4 aliases, 7 callbacks, 104 constants, 47 macros and 156 functions for this build. The shapes it produces are the general ones, covered in Generating Bindings with Bindgen; three of them matter for everyday Lua code.

- `lua_State` is an empty `struct #c_call`, so it is an opaque handle. Every C pointer becomes `?[*]T`, which is why `luaL_newstate` returns an option and the result of `lua_tostring` is matched rather than null-tested.
- A C macro that expands to an expression becomes an `#expand,expression` macro with inferred `Bindgen_Macro_T<N>` parameters. `lua_pop`, `lua_pcall`, `lua_tointeger`, `lua_pushcfunction`, `lua_register`, `luaL_openlibs` and `luaL_dostring` are all macros, not foreign functions.

Calling Lua from Glosso

With the bindings in scope the C API is used as it is in C: create a state, open the libraries, run a chunk, read a global off the stack. `defer` is the natural place for `lua_close`.

GLOSSO

```
#import "Io";
#import "Lua";

libc :: #library "c";
c_puts :: (message: *u8) -> s32 #foreign libc "puts";

main :: () {
    state := luaL_newstate();
    if state == null {
        print("could not create a Lua state\n");
        return;
    }
    defer lua_close(state);
    luaL_openlibs(state);

    if luaL_dostring(state, "answer = 6 * 7") != LUA_OK {
        // bindgen spells every C pointer `?[*]mut T`, so the error text is
        // matched out of the option rather than compared against null.
        if #pattern .Some(message) = lua_tostring(state, -1)
            c_puts(cast(*u8)message);
        return;
    }

    lua_getglobal(state, "answer");
    print("Lua computed %\n", lua_tointeger(state, -1));
    lua_pop(state, 1);
}
```

CONSOLE / OUTPUT

```
Lua computed 42
```

A string literal passed at a `?[*]s8` parameter is lowered to its data pointer, and literals are NUL-terminated in the image, so the chunk text and the global name above reach C correctly. A buffer built at run time is not terminated by anything and needs `cstring`; that boundary is covered in C Data Across the Boundary. Replacing the chunk with a syntax error takes the other branch and prints Lua's own message, `[string "answer = 6 *= 7"]:1: unexpected symbol near '='`.

Calling Glosso from Lua

The reverse direction needs a procedure of the `lua_CFunction` shape: `#c_call`, one `?[*]lua_State` parameter, and an `s32` result giving the number of values it left on the Lua stack. `lua_register` pushes it and binds it to a global name. Because it is `#c_call` it cannot touch the implicit context, which is covered in Being Called from C.

GLOSSO

```
#import "Io";
#import "Lua";

glosso_triple :: (state: ?[*]mut lua_State) -> s32 #c_call {
    n := lua_tointeger(state, 1);
    lua_pushinteger(state, n * 3);
    return 1;
}

main :: () {
    state := luaL_newstate();
    if state == null {
        print("could not create a Lua state\n");
        return;
    }
    luaL_openlibs(state);

    callback: lua_CFunction = glosso_triple;
    optional_callback: ?lua_CFunction = .Some(callback);
    lua_register(state, "triple", optional_callback);

    if luaL_dostring(state, "answer = triple(14)") != LUA_OK {
```

```
        print("lua error\n");
        lua_close(state);
        return;
    }

    lua_getglobal(state, "answer");
    print("Lua called back into Glosso: %\n", lua_tointeger(state, -1));
    lua_pop(state, 1);
    lua_close(state);
}
```

CONSOLE / OUTPUT

```
Lua called back into Glosso: 42
```

Linking an application-owned Lua

A program that ships its own Lua shared library imports `Lua/Build` instead of `Lua` and inserts `dynamic_bindings_source(path)` from its own bindings module. No C is compiled; bindgen runs against the same header and binds a `#library,dyn` descriptor instead of the cached archive. The path is required, so the empty call below is rejected by the same generated-stub mechanism as an unsupported target.

GLOSSO

```
#import "Io";
#import "Lua/Build";

#insert dynamic_bindings_source("");

main :: () {}
```

CONSOLE / OUTPUT

```
error: dynamic Lua linkage needs a library path
  --> …\lua_dyn.glo:4:1
   |
4 | #insert dynamic_bindings_source("");
   | ^^^^^^^
```

Its target rule is narrower than the static one, because the running program must be able to load the library: the output target has to equal the host in OS and CPU, must not be musl, and must be Windows MSVC or a dynamically linked Linux. Anything else is `dynamic Lua linkage requires a host Windows MSVC or dynamically linked Linux target`.

> **NOTE**
> The bindings are generated from the bundled Lua 5.5 headers whichever route is taken, so an application-supplied library must match that ABI. A different Lua version will link and then misbehave, because nothing in the build checks the library against the header it was described by.

15 Targets and build

15.1 Targets and Cross-Compilation

One compiler binary emits code for every target it supports, and `--target` selects the target for a build. With no `--target` the build is for the host, the machine the compiler process is running on. The choice is settled before checking begins, because the constant `OUT` describes the target and gated code reads it while types are being resolved.

A target is three named fields: `os`, `cpu` and `abi`. The LLVM triple handed to the backend is derived from those fields and is never written directly. The derivation loses information: `x86_64-freestanding-uefi` and `x86_64-windows-msvc` both produce the triple `x86_64-pc-windows-msvc`, and only the `abi` field separates the two builds.

CONSOLE / OUTPUT

```
$ glosso --target x86_64-linux-musl -o hello hello.glo
glosso: compile hello.glo (target=x86_64-pc-linux-musl)
glosso: link hello (target=x86_64-pc-linux-musl, -O1)
glosso: done hello  (backend=llvm, target=x86_64-pc-linux-musl, -O1)
```

Every progress line names the resolved triple, so the first line of a build is the confirmation that the spelling was understood. That Linux executable was produced on a Windows host with nothing else installed: the musl runtime for the target is bundled with the compiler, and the result is a static ELF carrying no dynamic interpreter.

Spelling a target

`--target` accepts both `os-cpu-abi` and `cpu-os-abi`. The parser decides which order was written by testing whether the first field is a known CPU name and the second a known operating-system name; there is no table of complete triples to match against. Any field left out is inherited from the host target.

- A spec whose first field names a wasm CPU is read wasm first: `os` becomes `wasm`, `cpu` is the wasm word, and `abi` is the following field, defaulting to `freestanding`.
- `<cpu>-freestanding` sets `os` and `abi` both to `freestanding`, so the CPU name in the first field is not mistaken for an operating system.
- `wasi-wasm32` is accepted for compatibility and normalizes to `os wasm` with `abi wasi`; WASI is never exposed as an operating system. `host` and `native` name the compiler's own target. An empty spec is tolerated, and so is a trailing hyphen, so `wasm32-` resolves to `wasm32-unknown-unknown`.

Field names are rewritten to a canonical spelling before the triple is formed. For the CPU field, `amd64` becomes `x86_64`, `arm64` becomes `aarch64`, `i686` becomes `i386`, and `ppc` and `ppc32` become `powerpc`. For the operating-system field, `darwin` and `osx` become `macos`, `win32` becomes `windows`, and `webassembly` and `wasi` become `wasm`. Three further rewrites happen only in the emitted triple: `x86` is spelled `i686`, `armv5` is spelled `armv5te`, and `ppc64le` is spelled `powerpc64le`.

> **NOTE**
> A spec with a single field fills the operating-system slot, not the CPU slot. `--target aarch64` therefore resolves to `x86_64-unknown-aarch64-msvc` and dies at the link step with `error: unsupported executable target` naming that triple. For the same reason `--target x86_64-windows` leaves `abi` empty, which triples as `x86_64-w64-windows-gnu` rather than the MSVC ABI of a Windows host. Write the full three-field spelling.

The supported set

`--print-targets` (also accepted as `--targets`) prints each supported target with the triple it expands to and the artifact it produces, then a second list headed `Available LLVM codegen targets:`. That second list is the set of architectures the linked LLVM was built with; it is not the set of Glosso targets, and naming one of its entries does not make a target.

Target	LLVM triple	Artifact
`host`, `native`	the compiler's own triple	whatever that target produces
`x86_64-windows-msvc`	`x86_64-pc-windows-msvc`	PE/COFF executable
`x86_64-windows-gnu`	`x86_64-w64-windows-gnu`	PE/COFF executable
`linux-x86_64-gnu`	`x86_64-pc-linux-gnu`	ELF executable
`aarch64-linux-gnu`	`aarch64-unknown-linux-gnu`	ELF executable
`x86_64-macos`, `aarch64-macos`	`x86_64-apple-darwin`, `aarch64-apple-darwin`	Mach-O executable

Target	LLVM triple	Artifact
`x86_64-freebsd`, `aarch64-freebsd`	`x86_64-unknown-freebsd`, `aarch64-unknown-freebsd`	ELF executable
`wasm32-wasi`, `wasm64-wasi`	`wasm32-unknown-wasi`, `wasm64-unknown-wasi`	WebAssembly module
`wasm32-freestanding`	`wasm32-unknown-unknown`	WebAssembly module
`x86_64-freestanding`	`x86_64-unknown-none-elf`	bare-metal ELF executable
`x86_64-freestanding-uefi`	`x86_64-pc-windows-msvc`	UEFI PE/COFF object

Fourteen further targets share the shape `<cpu>-linux-musl` and link against the bundled static musl runtime: `x86`, `x86_64`, `aarch64`, `armv5`, `armv7`, `loongarch64`, `mips`, `mipsel`, `mips64`, `mips64el`, `powerpc`, `ppc64le`, `riscv64` and `s390x`. Each expands to `<cpu>-unknown-linux-musl` with five exceptions: `linux-x86_64-musl` is `x86_64-pc-linux-musl`, `x86-linux-musl` is `i686-unknown-linux-musl`, `ppc64le-linux-musl` is `powerpc64le-unknown-linux-musl`, `armv5-linux-musl` is `armv5te-unknown-linux-musleabi`, and `armv7-linux-musl` is `armv7-unknown-linux-musleabihf`.

Entry points

A Glosso `main` is lowered under the symbol `glo_main`. What calls it depends on the target family: a hosted target gets a synthesized C entry point that forwards to `glo_main`, and a target with no host runtime has `glo_main` named as the entry or exported directly.

Target family	Emitted entry	Relation to `glo_main`
hosted Windows	`wmain(i32, ptr)`	wrapper that calls it
hosted ELF and Mach-O	`main(i32, ptr)`	wrapper that calls it
`wasm32-wasi`, `wasm64-wasi`	`_start()`	wrapper that calls it, linked with `--export=_start`
`wasm32-freestanding`	none	linked with `--export=glo_main`
`<cpu>-freestanding`	none	linked with `-e glo_main`, no argv

`--no-main` and the `static-lib` and `shared-lib` backends suppress the wrapper entirely, along with the `argc` and `argv` globals, and root reachability at every defined `#c_call` function instead. Being Called from C describes that rooting, and Backends, Artifacts, and Optimization Levels describes the backends.

What links from a given host

Emitting an object succeeds for every supported target, because that step needs only LLVM. Linking needs the target's runtime inputs. The compiler bundles the musl runtime, the mingw-w64 import libraries and the wasm builtins, so those targets link from any host; Apple and FreeBSD system libraries are not bundled and must come from an SDK.

- Measured on an `x86_64-pc-windows-msvc` host, these link with no further setup: both Windows targets, `linux-x86_64-gnu`, all fourteen musl targets, both WASI targets, `wasm32-freestanding` and `x86_64-freestanding`.
- These reach the linker and fail there: `x86_64-macos` and `aarch64-macos` on `ld64.lld: error: undefined symbol: _abort`, the two FreeBSD targets on `ld.lld: error: undefined symbol: abort`, and `aarch64-linux-gnu` on `ld.lld: error: undefined symbol: write`.

CONSOLE / OUTPUT

```
$ glosso --target aarch64-macos -o hello hello.glo
glosso: compile hello.glo (target=aarch64-apple-darwin)
glosso: link hello (target=aarch64-apple-darwin, -O1)
error: ld64.lld: error: undefined symbol: _abort
>>> referenced by C:\Users\…\Temp\glosso-35100-hello.o:(symbol _glo_write_string+0x248)

$ glosso --backend object --target aarch64-macos -o hello.o hello.glo
glosso: compile hello.glo (target=aarch64-apple-darwin)
glosso: wrote hello.o  (backend=object, target=aarch64-apple-darwin, -O1)
```

`--backend object` is therefore the portable smoke test: it proves that a program checks and lowers for a target without requiring that target's linker inputs. `x86_64-freestanding-uefi` produces an object by design, and Freestanding, Bare Metal, UEFI, and WebAssembly covers the link step it needs.

The OUT and HOST descriptors

The checker installs two constants into every compilation. `OUT` is an `Output_Target` with the fields `os`, `cpu` and `abi`, describing the target being compiled for. `HOST` is a `Host_Target` with `os` and `cpu`, describing the machine running the compiler. Their field types come from the Prelude and are always in scope, so reading them needs no import.

Type	Members
`Target_OS`	`Unknown`, `Linux`, `Macos`, `Windows`, `Freebsd`, `Wasm`, `Freestanding`
`Target_CPU`	`Unknown`, `X86_64`, `Aarch64`, `Arm`, `Thumb`, `I386`, `Powerpc`, `Powerpc64`, `Powerpc64le`, `Riscv64`, `Wasm32`, `Wasm64`, `Armv7`, `Loongarch64`, `S390x`, `X86`, `Armv5`, `Mips`, `Mipsel`, `Mips64`, `Mips64el`
`Target_ABI`	`None`, `Unknown`, `Gnu`, `Musl`, `Msvc`, `Mingw`, `Wasi`, `Freestanding`, `Emscripten`, `Uefi`
`Target_Endian`	`Little`, `Big`, returned by `target_endian()`

GLOSSO

```
#import "Io";

os_name :: () -> string {
    #if OUT.os == {
    case .Windows;      return "windows";
    case .Linux;        return "linux";
    case .Macos;        return "macos";
    case .Freebsd;      return "freebsd";
    case .Wasm;         return "wasm";
    case .Freestanding; return "freestanding";
    else;               return "unknown";
    }
}

main :: () {
    print("building for %\n", os_name());
    print("OUT.abi is Msvc: %\n", OUT.abi == Target_ABI.Msvc);
    print("host is the target: %\n", OUT.os == HOST.os && OUT.cpu == HOST.cpu);
}
```

CONSOLE / OUTPUT

```
building for windows
OUT.abi is Msvc: 1
host is the target: 1
```

`%` prints an enum member as its integer value, which is why the example maps the tag to a string rather than printing `OUT.os` directly. Both the bare `OUT.os == .Windows` and the qualified `OUT.os == Target_OS.Windows` are accepted.

`#if` over `OUT` selects target-specific code, and `#compile_error` rejects a target the program does not support. Static Branching with `#if` covers the construct itself. Both directives are statements, so a gate lives inside a declaration rather than at file scope.

GLOSSO

```
path_separator :: () -> string {
    #if OUT.cpu != Target_CPU.X86_64 && OUT.cpu != Target_CPU.Aarch64
        #compile_error "this program supports x86_64 and aarch64 only";
    #if OUT.os == {
    case .Windows;  return "\\";
    else;           return "/";
    }
}
```

CONSOLE / OUTPUT

```
$ glosso --target riscv64-linux-musl gating.glo
glosso: compile gating.glo (target=riscv64-unknown-linux-musl)
error: this program supports x86_64 and aarch64 only
  --> …\gating.glo:5:9
   |
5 |         #compile_error "this program supports x86_64 and aarch64 only";
   |         ^^^^^^^^^^^^^^
```

NOTE

The LLVM triple is not a unique target key. `x86_64-freestanding-uefi` and `x86_64-windows-msvc` share `x86_64-pc-windows-msvc`, so code that must tell them apart tests `OUT.abi`. The artifact cache keys on the `OUT` fields for exactly this reason; The Build Cache and Generated Sources shows the two builds keeping separate entries.

`--run` builds and then executes the result, and is refused for anything but the host: `glosso: --run can only execute the host target (got x86_64-pc-linux-musl)`, exit status 1. Using the Compiler covers the remaining driver options.

Target-width integers and descriptors

`ssize` and `usize` follow the target's pointer width, not the host's: 32 bits on targets such as `x86`, Arm, and `wasm32`, and 64 bits on targets such as `x86_64`, `aarch64`, and `wasm64`. Counts, indices, and the counts inside `string`, `string16`, slices and dynamic arrays use that same width. Only dynamic arrays carry a capacity word; counted strings store a pointer followed by a

count, while `cstring` and `cstring16` store one pointer. Consequently a cross-compiled descriptor has the target layout even when the compiler itself is a 64-bit process. Tagged unions use a target-width tag as well, with padding before a more strongly aligned payload. The compile-time VM uses those target offsets when copying or matching unions; the host's word size does not determine a cross-compiled result layout.

Wide floats and target runtimes

A target's scalar alignment and calling convention matter separately from its pointer width. On 32-bit x86 musl, `f64` and `f80` have alignment 4; their complex counterparts inherit that alignment. `f80` keeps sixteen-byte storage but uses x87 on x86 and a binary128 carrier elsewhere. Query `type_info(T).alignment` for the actual output target before designing an ABI or serialized layout.

The driver builds target-keyed compiler-runtime archives for wide floating operations. On PowerPC64 musl it builds the helper archive with IEEE binary128 support and the `kf` helper names expected by LLVM; that private archive setting does not change musl's `long double` ABI. On 32-bit Arm, MIPS, and PowerPC musl, conversion to `f64` uses an integer-word helper instead of assuming a C binary128 type exists. When linking an emitted Glosso object yourself, you must also supply the runtime helpers used by that object's target.

15.2 Freestanding, Bare Metal, UEFI, and WebAssembly

Three targets run with no hosted operating system underneath them: `x86_64-freestanding`, `x86_64-freestanding-uefi` and `wasm32-freestanding`. What they share is that the backend emits no host entry wrapper and no command-line globals, and that nothing links a C runtime in behind the program's back. The WASI targets are the hosted wasm option and are covered here for contrast.

The predicate the backend uses for bare metal is `os == "freestanding"`. That is true for `x86_64-freestanding` and for the UEFI target, and false for the wasm targets, whose `os` is `wasm`. The families therefore differ in the details below even though they share the same absence of a host.

Bare-metal ELF

`x86_64-freestanding` expands to `x86_64-unknown-none-elf` and sets both `OUT.os` and `OUT.abi` to `Freestanding`. The image is linked with `ld.lld -e glo_main`, so the Glosso `main` is the entry point and there is no `argc`/`argv` pair to receive. When the program has no `main` at all, reachability is rooted at every defined `#c_call` function, the same rule `--no-main` installs.

GLOSSO

```
kernel_add :: (a: s32, b: s32) -> s32 #c_call { return a + b; }

_start :: () #c_call {
    x := kernel_add(2, 3);
    while true {}
}
```

CONSOLE / OUTPUT

```
$ glosso -O2 --target x86_64-freestanding -o bare.elf bare.glo
glosso: compile bare.glo (target=x86_64-unknown-none-elf)
glosso: link bare.elf (target=x86_64-unknown-none-elf, -O2)
glosso: done bare.elf  (backend=llvm, target=x86_64-unknown-none-elf, -O2)
```

The `-O2` is not decoration. At `-O0` and `-O1` the backend emits runtime safety checks, and a failing check terminates through `abort`, which no freestanding image provides. The default level is `-O1`, so the same command without an explicit level fails at the link step.

CONSOLE / OUTPUT

```
$ glosso --target x86_64-freestanding -o bare.elf bare.glo
glosso: compile bare.glo (target=x86_64-unknown-none-elf)
glosso: link bare.elf (target=x86_64-unknown-none-elf, -O1)
error: ld.lld: error: undefined symbol: abort
>>> referenced by bare
>>>               C:\Users\…\Temp\glosso-33852-bare.o:(glo_kernel_add)
```

NOTE

The size modes -Os, -Oz, and -Ozz produce runnable artifacts. They combine LLVM optimization with optsize/minsize attributes and target-specific linker size policies. Runtime checks are disabled in these modes; use -O0 or -O1 for checked builds. -Ozz also strips symbols and metadata.

- No host entry is emitted: the module defines `glo_main` and no `main` or `wmain` wrapper, and the `__glo_argc` and `__glo_argv` globals are absent.
- The default temporary-arena size drops from 4 MiB to 64 KiB on a freestanding target, because the arena is a zero-initialized global in the image. `--temp-allocator-size` overrides it in either direction.

- Sanitizers are rejected by the backend before the link is attempted: `error: LLVM memory sanitizer is not supported for target 'x86_64-unknown-none-elf'`.

UEFI

`x86_64-freestanding-uefi` keeps `OUT.os` at `Freestanding`, so no hosted runtime is injected, but maps to the LLVM triple `x86_64-pc-windows-msvc`, which selects PE/COFF and the Microsoft x64 C ABI. `OUT.abi` is `Uefi`, and that field is the only way a program can tell a UEFI build from an ordinary Windows one. `x86_64-freestanding-msvc` is the legacy spelling; it produces the same triple and the same calling convention but leaves `OUT.abi` at `Msvc`, so a program that gates on `Target_ABI.Uefi` needs the `-uefi` spelling.

The executable backend cannot finish this target, because the COFF executable plan asks for the Microsoft C startup object and its `wmain`, which a firmware image does not have. The workflow is to emit an object and link it as an EFI application separately.

CONSOLE / OUTPUT

```
$ glosso -O2 --no-main --target x86_64-freestanding-uefi -o efi.exe efi.glo
glosso: compile efi.glo (target=x86_64-pc-windows-msvc)
glosso: link efi.exe (target=x86_64-pc-windows-msvc, -O2)
error: lld-link: error: undefined symbol: wmain
```

GLOSSO

```
efi_main :: (image: *mut void, system_table: *mut void) -> u64 #c_call {
    #if OUT.abi != Target_ABI.Uefi
        #compile_error "efi.glo builds only for the UEFI ABI";
    return 0;
}
```

CONSOLE / OUTPUT

```
$ glosso -O2 --no-main --backend object \
      --target x86_64-freestanding-uefi -o efi.obj efi.glo
glosso: compile efi.glo (target=x86_64-pc-windows-msvc)
glosso: wrote efi.obj  (backend=object, target=x86_64-pc-windows-msvc, -O2)

$ lld-link /nologo /machine:x64 /subsystem:efi_application \
      /entry:glo_efi_main /nodefaultlib /opt:ref /out:efi.efi efi.obj
```

The entry is spelled `glo_efi_main` because a `#c_call` definition is exported under `glo_` followed by its Glosso name; Being Called from C gives the full symbol rule. `/nodefaultlib` keeps the Microsoft CRT out, and `--no-main` on the Glosso side keeps the `main` wrapper out and roots reachability at the `#c_call` definitions.

Driving both steps from a build script is the usual arrangement, since the second step is an external command. The compile-time options mirror the flags: backend `Object`, target `os = .Freestanding`, `cpu = .X86_64`, `abi = .Uefi`, and `no_main = true`. Build Scripts covers the API.

WebAssembly

Target	Triple	Link
`wasm32-wasi`	`wasm32-unknown-wasi`	`wasm-ld --no-entry --export=_start`
`wasm64-wasi`	`wasm64-unknown-wasi`	as above, plus `-mwasm64`
`wasm32-freestanding`	`wasm32-unknown-unknown`	`wasm-ld --no-entry --export=glo_main`

A WASI build synthesizes a `_start` wrapper around `main` and reaches the outside world through the `wasi_snapshot_preview1` import namespace. The module built from a program that prints one line exports `memory` and `_start` and imports `fd_write`.

`wasm32-freestanding` is the other shape entirely. There is no WASI, no C library and no import namespace: the module built from the program below has no import section at all, and its export section holds exactly `memory` and `glo_main`. Everything the program needs is in the module, which is what makes the result loadable by a browser with a bare `WebAssembly.instantiate`.

GLOSSO

```
add :: (a: s32, b: s32) -> s32 #c_call { return a + b; }

main :: (seed: s32) -> s32 #c_call {
    if seed < 0 return add(1, 2);
    return seed;
}
```

CONSOLE / OUTPUT

```
$ glosso -O2 --target wasm32-freestanding -o wexp.wasm wexp.glo
glosso: compile wexp.glo (target=wasm32-unknown-unknown)
glosso: link wexp.wasm (target=wasm32-unknown-unknown, -O2)
glosso: done wexp.wasm  (backend=llvm, target=wasm32-unknown-unknown, -O2)
# module exports: memory, glo_main
```

`main` takes a parameter and returns a value there, which no other target allows. On `wasm32-freestanding` a `#c_call main` is exempt from the rule that `main` takes no parameters, because nothing wraps it: the host calls the export directly and passes whatever the signature says. Elsewhere the same declaration is `error: main must not take parameters; use get_cmdline_args() for command-line arguments`.

Anything beyond `glo_main` has to be exported explicitly, and has to be reachable. Reachability is computed before lowering, so an unreferenced `#c_call` function is pruned and the export then names a symbol that does not exist.

CONSOLE / OUTPUT

```
$ glosso -O2 --no-main --target wasm32-freestanding \
      --linker-arg "--export=glo_add" -o wexp3.wasm wexp.glo
glosso: compile wexp.glo (target=wasm32-unknown-unknown)
glosso: link wexp3.wasm (target=wasm32-unknown-unknown, -O2)
glosso: done wexp3.wasm  (backend=llvm, target=wasm32-unknown-unknown, -O2)
# module exports: memory, glo_add, glo_main
```

NOTE

`--export=glo_main` is passed unconditionally for this target, so `--no-main` on a program that has no `main` breaks the link with `wasm-ld: error: symbol exported via --export not found: glo_main`. A `wasm32-freestanding` program keeps a `main`, `#c_call` or otherwise.

- Sanitizers are rejected for wasm: `LLVM memory sanitizer is not supported for target 'wasm32-unknown-wasi'`.
- `#library,dyn` is rejected during lowering, with the message `#library,dyn is not supported for target 'wasm32-unknown-unknown'` followed by the instruction it was lowering. There is no dynamic loader to call.
- A `#foreign` call names a library file, and the wasm linker resolves that to `-l<name>`, so a host-provided function cannot be declared with `#foreign`. Imports come from undefined symbols admitted by `--linker-arg "--allow-undefined"`.

How this manual is built

The manual being read is a `wasm32-freestanding` module. A build script compiles the manual's entry file for that target, builds the Clay layout library from C sources for the same target, generates bindings for it, and links the result to a single `.wasm` file that the browser host loads.

GLOSSO

```
wasm_target := Output_Target.{
    .os = Target_OS.Wasm,
    .cpu = Target_CPU.Wasm32,
    .abi = Target_ABI.Freestanding,
};

workspace := Workspace.create("manual");
options := workspace.get_build_options();
options.output_type = Build_Output_Type.Executable;
options.output_path = "dist";
options.output_executable_name = "manual.wasm";
options.set_target(wasm_target);
options.set_optimization(.Optimized);
options.set_temp_allocator_size(1024 * 1024);
options.add_linker_arg("--export=glo_manual_set_view");
options.add_linker_arg("--export=glo_manual_select_section");
options.add_linker_arg("--allow-undefined");
options.add_linker_arg("--initial-memory=134217728");
options.set_build_options(workspace);
built := add_build_file("main.glo", workspace);
```

The size modes -Os, -Oz, and -Ozz produce runnable artifacts. They combine LLVM optimization with optsize/minsize attributes and target-specific linker size policies. Runtime checks are disabled in these modes; use -O0 or -O1 for checked builds. -Ozz also strips symbols and metadata.

The C dependency is built through the same target descriptor. The binding generator checks `OUT.os`, `OUT.cpu` and `OUT.abi` and returns a `#compile_error` string when they are not the wasm32 freestanding triple, so an accidental host build fails during checking instead of producing a module that cannot run. Generating Bindings with Bindgen and Build Scripts cover the two halves.

15.3 Diagnostics: Reading and Configuring Them

Every source-level diagnostic is written to stderr in one layout, whatever phase produced it. The layout is a headline naming the level and the message, a location line, a framed excerpt of the offending source with a caret run under the span, and then zero or more notes.

GLOSSO

```
main :: () {
    print("hello from glosso\n");
}
```

CONSOLE / OUTPUT

```
error: unknown function print
  --> …\diag1.glo:2:5
  |
2 |     print("hello from glosso\n");
  |     ^^^^^
help: did you mean 'tprint'?
```

Line	Content
location	`-->` then the file path, line and column
blank gutter	a bare `|` aligned with the line-number column
source	the line number, `|` , and the source line verbatim
caret	a run of `^` under the span, one per character
notes	a label, the note text, and optionally a frame of its own

The gutter width is the width of the line number, so a diagnostic on line 4 indents the frame by one column less than one on line 16. The caret run is the span's length clamped to what remains on the line, and a span with no length still produces one caret. If the source line is empty or the line number is out of range the frame is omitted and only the headline is printed.

Two note labels are produced: `help` for a suggested replacement, and `note` for everything else. A note may carry its own location frame, which is how a diagnostic points at a second place in the program.

A note with no span prints as a bare label and message, with no frame under it. That form carries advice rather than a second location.

CONSOLE / OUTPUT

```
error: tagged-union payload '.Ok' may only be bound by a pattern match
  --> …\diag3.glo:7:20
  |
7 |     print("%\n", r.Ok);
  |                    ^^
note: match on the union and use that arm's payload capture, or enable #enable(allow_union_unwrap) for unsafe access
```

Warnings and errors

Warnings and errors use the identical layout and differ in the headline word and its colour. An error ends the build and the compiler exits 1; a warning is printed and the build continues to completion with exit 0. Multiple diagnostics are separated by a blank line.

When more than one error is rendered, a final summary line closes the group. Warnings never get a summary, however many there are.

CONSOLE / OUTPUT

```
error: got f32, expected s32
  --> …\diag8.glo:3:12
  |
3 |     return x;
  |            ^

error: got f64, expected s32
  --> …\diag8.glo:8:12
  |
8 |     return y;
  |            ^

error: aborting due to 2 previous errors
```

Driver messages are a separate category and are not diagnostics. They are prefixed `glosso:` , carry no frame and no colour, and report command-line and toolchain problems rather than problems in the source. A bad option value exits 2; a build failure exits 1.

CONSOLE / OUTPUT

```
$ glosso --color rainbow diag1.glo
glosso: --color must be auto, always, or never
$ echo $?
2
```

Colour

`--color` takes `auto`, `always` or `never` and defaults to `auto`. Under `auto`, colour is enabled only when stderr is a terminal and either the host is Windows or `TERM` is set to something other than `dumb`. Redirecting stderr to a file or a pipe therefore turns colour off without any flag.

Setting	Effect
`--color auto`	on when stderr is a terminal and the terminal is known to be capable
`--color always`	on unconditionally
`--color never`	off unconditionally
`NO_COLOR` set to any value	off, and it beats `--color always`

CONSOLE / OUTPUT

```
$ glosso --color always diag1.glo
^[[1;31merror^[[0m^[[1m: ^[[0m^[[1munknown function print^[[0m
 ^[[1;34m --> ^[[0m…\diag1.glo:2:5

$ NO_COLOR=1 glosso --color always diag1.glo
error: unknown function print
  --> …\diag1.glo:2:5
```

The escape byte is written `^[` in that transcript. Five escapes are used and nothing else: bold red for `error`, bold yellow for `warning`, bold blue for the arrow and the gutter, bold cyan for a note label and its frame, and plain bold for the message text. Because `NO_COLOR` is checked before the mode, a script can strip colour from a build it does not control.

Diagnostics in generated source

A string-valued `#insert` splices text produced at compile time into the program, and that text has no file of its own in the project. The renderer solves this by retargeting the diagnostic: the primary span is moved to the outermost `#insert` expression, which is a real place in a real file, and the position inside the generated text becomes a note.

GLOSSO

```
#import "Io";

#insert emit_helper();

emit_helper :: () -> string {
    return "helper :: () -> s64 { return no_such_name; }\n";
}

main :: () {
    print("%\n", helper());
}
```

CONSOLE / OUTPUT

```
error: unbound 'no_such_name'
  --> …\gen_bad.glo:3:1
  |
3 | #insert emit_helper();
  | ^^^^^^^
note: generated source failed here
  --> …\cache\generated\insert\v1\82949d39….glo:1:30
  |
1 | helper :: () -> s64 { return no_such_name; }
  |                              ^^^^^^^^^^^^
```

Read that from the bottom up. The note names the line that is actually wrong and quotes it, and the path it names is a real file: the cache entry the generated text was written to, which can be opened in an editor. The headline names the `#insert` that produced it, which is the line to change.

When one generated source inserts another, every link in the chain gets its own note labelled `generated by this #insert`, ordered from the failure outwards, and the headline lands on the `#insert` written by hand.

CONSOLE / OUTPUT

```
error: unbound 'no_such_name'
  --> …\gen_nest.glo:3:1
   |
3 | #insert outer();
   | ^^^^^^^
note: generated source failed here
  --> …\insert\v1\82949d39….glo:1:30
   |
1 | helper :: () -> s64 { return no_such_name; }
   |                              ^^^^^^^^^^^^
note: generated by this #insert
  --> …\insert\v1\e387a996….glo:1:1
   |
1 | #insert inner();
   | ^^^^^^^
```

If the cache directory is unavailable the generated text is still rendered, but the note names the virtual path `<generated:#insert/<sha256>.glo>` instead, which cannot be opened. The Build Cache and Generated Sources covers where those files live and how to list them.

15.4 The Build Cache and Generated Sources

The compiler keeps one per-user cache root holding everything it can reuse between builds: materialized generated sources, finished artifacts, and static archives built from C sources for a target. `--print-cache-dir` resolves the root, prints it, and exits 0 without compiling anything.

CONSOLE / OUTPUT

```
$ glosso --print-cache-dir
C:\Users\almag\AppData\Local\glosso\cache
```

The root is resolved once per process, from the first source in this list that yields an absolute path.

Source	Value
`--cache-dir <dir>`	the given directory; relative paths resolve against the working directory
`GLOSSO_CACHE_DIR`	the given directory, which must be absolute
Windows default	the known LocalAppData folder, then `glosso/cache`
macOS default	`$HOME/Library/Caches/glosso`
other POSIX default	`$XDG_CACHE_HOME/glosso`, else `$HOME/.cache/glosso`

--cache-dir accepts relative paths and resolves them against the working directory. GLOSSO_CACHE_DIR must contain an absolute path. The command-line override takes precedence over the environment. Use --print-cache-dir to inspect the resolved root.

CONSOLE / OUTPUT

```
$ glosso --cache-dir "C:/temp/glosso-cache-probe" --print-cache-dir
C:\temp\glosso-cache-probe
$ GLOSSO_CACHE_DIR=C:/temp/env-cache glosso --print-cache-dir
C:\temp\env-cache
$ GLOSSO_CACHE_DIR=C:/temp/env-cache glosso --cache-dir C:/temp/flag-cache --print-cache-dir
C:\temp\flag-cache
```

The subtrees

Path under the root	Contents
`generated/insert/v1/<sha256>.glo`	materialized string-valued `#insert` sources
`native/artifacts/v1/` `<key>.artifact`, `.manifest`, `.stdout`, and `.stderr`	the final-artifact cache
`native/v9/<name>/<target>/<key>/`	static archives built from C sources for a target

The `v1` and `v9` components are format versions: a compiler that changes an entry's meaning changes the version and stops reading the old tree rather than misinterpreting it. `GLOSSO_NATIVE_CACHE_DIR` relocates the `native/v9` subtree alone, leaving the other two under the resolved root, so the C archives can be kept apart from the rest.

The artifact cache

The artifact cache stores the finished output of a build and restores it byte for byte when the same build is asked for again. A hit skips every phase, including checking, so it is reported by a single line and the build takes milliseconds.

CONSOLE / OUTPUT

```
$ glosso --run hello.glo
glosso: compile hello.glo (target=x86_64-pc-windows-msvc)
glosso: link hello.exe (target=x86_64-pc-windows-msvc, -O1)
glosso: done hello.exe  (backend=llvm, target=x86_64-pc-windows-msvc, -O1)
hello from 3

$ glosso --run hello.glo
glosso: restored hello.exe from the native artifact cache
hello from 3
```

Only three backends participate: `llvm`, `object` and `static-lib`. `shared-lib` and `llvm-ir` builds are never stored and never restored, and `--show-generated` disables the cache for the run it appears on. `GLOSSO_DISABLE_ARTIFACT_CACHE` set to any value disables it entirely.

A cache entry is keyed by an identity string covering everything that could change the output.

- The canonical paths of the source file and compiler executable, the standard-library directory, and the working directory.
- The LLVM triple, and separately every field of `OUT`, because the triple collapses distinct targets.
- Every `--linker-arg` in command-line order, every compile-time argument written after `--`, every `--module-path` directory, and all explicit `--unit-source` ownership mappings. Backend, optimization, sanitizer, color, temporary-arena size, debug-allocator selection, `--no-main`, and `--use-std` also participate in the identity.

The `OUT` fields in that list are load-bearing. Compiling the same source for `x86_64-windows-msvc` and for `x86_64-freestanding-uefi` produces the same triple in the progress line, and the two builds still keep separate entries.

CONSOLE / OUTPUT

```
$ glosso -O2 --no-main --backend object --target x86_64-windows-msvc -o b.obj lib2.glo
glosso: compile lib2.glo (target=x86_64-pc-windows-msvc)
glosso: wrote b.obj  (backend=object, target=x86_64-pc-windows-msvc, -O2)
$ glosso -O2 --no-main --backend object --target x86_64-windows-msvc -o b.obj lib2.glo
glosso: restored b.obj from the native artifact cache
$ glosso -O2 --no-main --backend object --target x86_64-freestanding-uefi -o b.obj lib2.glo
glosso: compile lib2.glo (target=x86_64-pc-windows-msvc)
glosso: wrote b.obj  (backend=object, target=x86_64-pc-windows-msvc, -O2)
$ glosso -O2 --no-main --backend object --target x86_64-freestanding-uefi -o b.obj lib2.glo
glosso: restored b.obj from the native artifact cache
```

Identity decides which entry is consulted; a manifest decides whether that entry is still valid. The manifest records the length and modification time of every source file the build read, of the compiler executable itself, and of any `--linker-arg` value that names an existing file. Touching any one of them turns the next build back into a full compile. A hit also replays saved checker output and diagnostics; entries missing either output sidecar are treated as misses.

> **NOTE**
> A program with a top-level `#comptime` is never stored. Compile-time execution can write files, run commands or print, and restoring an artifact would skip all of it. Build scripts therefore always run, which is the behaviour a build script needs.

Generated sources

At top level, `#insert <expression>` evaluates the expression during checking and splices the resulting text into the program. That text is hashed with SHA-256, written to `generated/insert/v1/<hash>.glo` under the cache root, and the path of that file becomes the source name for every span inside the generated code. Publication is atomic, and an entry that already exists has its contents verified rather than rewritten.

The name is the hash of the text alone, so the same generated source lands on the same file name under every cache root and in every project. That is what makes the entry shareable and makes a diagnostic pointing into it reproducible.

GLOSSO

```
#import "Io";

#insert emit_helper();

emit_helper :: () -> string {
    return "helper :: () -> s64 { return 42; }\n";
}

main :: () {
    print("%\n", helper());
}
```

CONSOLE / OUTPUT

```
$ glosso --show-generated --run gen_ok.glo
glosso: compile gen_ok.glo (target=x86_64-pc-windows-msvc)
glosso: generated source C:\Users\almag\AppData\Local\glosso\cache\generated\insert\v1\74f6963b….glo
```

```
glosso: link gen_ok.exe (target=x86_64-pc-windows-msvc, -O1)
glosso: done gen_ok.exe  (backend=llvm, target=x86_64-pc-windows-msvc, -O1)
42

$ cat C:\Users\almag\AppData\Local\glosso\cache\generated\insert\v1\74f6963b….glo
helper :: () -> s64 { return 42; }
```

`--show-generated` prints one `glosso: generated source <path>` line per materialized `#insert`, plus any cache warning as `glosso: warning: <text>`. It also disables the artifact cache for that run, since a restored artifact would report no generated sources at all.

The lines are printed after checking succeeds and before lowering begins, so a build that fails inside generated code prints no such line. In that case the path arrives through the diagnostic instead: the note under the error names the same cache entry. Diagnostics: Reading and Configuring Them shows the rendered form.

Redirecting the root moves the generated files with it, and the file name does not change, because it is derived from the text and not from the location.

CONSOLE / OUTPUT

```
$ glosso --cache-dir C:/probe/mycache --show-generated --backend llvm-ir -o go.ll gen_ok.glo
glosso: generated source C:\probe\mycache\generated\insert\v1\74f6963b….glo
```

When no cache root can be resolved at all, the text is still compiled and the source name falls back to the virtual form `<generated:#insert/<hash>.glo>`. Nothing is written, so nothing can be opened, and the same build repeats the work every time.

The third subtree is filled by the compile-time C archive builder. Importing a module that builds native code compiles its C sources for the output target. Verified inputs allow archive reuse under `native/v9/<name>/<target>/<key>/`; changed sources or headers rebuild it. When dependencies cannot be verified, each build uses a fresh `uncached-...` directory. Embedding Lua and Generating Bindings with Bindgen cover the modules that use this service.

CONSOLE / OUTPUT

```
C:\Users\almag\AppData\Local\glosso\cache\native\v9\lua55
C:\Users\almag\AppData\Local\glosso\cache\native\v9\glosso_rgfw
C:\Users\almag\AppData\Local\glosso\cache\native\v9\glosso_wasm_compiler_rt_builtins
C:\Users\almag\AppData\Local\glosso\cache\native\v9\glosso_wasm_compiler_rt_builtins\wasm32-wasm-freestanding
C:\Users\almag\AppData\Local\glosso\cache\native\v9\glosso_wasm_compiler_rt_builtins\wasm32-wasm-wasi
```

16 Reference

16.1 The Standard Library Map

The standard library is a directory of ordinary Glosso source files that ships beside the compiler. Nothing in it is privileged: every module is written in the language the manual describes, and the compiler finds it by searching for a directory that contains `Prelude.glo` or a `Prelude/` subdirectory. Where that search looks, and how `--std` overrides it, is covered in the chapter on the environment and toolchain.

This chapter is the map: what is in the tree, which names can be imported, and how to find the module that owns an operation. The import forms themselves belong to the chapter on source files, imports, and loads.

How a module name resolves

`#import "Name"` is resolved against the standard-library root by one rule, applied to the two candidate paths `std/Name.glo` and `std/Name/module.glo`.

- If `std/Name.glo` exists, the module is that one file.
- If `std/Name/module.glo` exists, that entry file is the module. It explicitly loads implementation dependencies and uses `#export` to expose its component modules. Other files in the directory are not automatically included.
- If both exist the name is ambiguous, and if neither exists the name is unknown. Both outcomes are hard errors.

CONSOLE / OUTPUT

```
error: cannot find module "Container" (searched: …\glosso\std)

error: ambiguous module "Ascii": both …\mystd\Ascii.glo and …\mystd\Ascii\module.glo exist
```

A module name is a relative path with `/` separators, so `Container/HashMap` names the file `std/Container/HashMap.glo` and the same rule applies one level down. The name is validated before the lookup: an absolute path, a backslash, a `..` component, or an empty component is rejected outright.

Namespaces are not modules

Because the rule keys on `module.glo`, a directory that does not have one is a namespace and not a module. Its components are still importable one at a time by path; the directory name itself is not. That is the whole difference between `#import "Memory"`, which works, and `#import "Container"`, which does not.

GLOSSO

```
#import "Io";
#import "Container/HashMap";
#import "Strings/Utf8";
#import "String_Builder";

main :: () {
    counts := HashMap(string, s64).with_seed(1, 2);
    defer counts.deinit();
    if counts.insert("glosso", 42) == {
    case .Ok(r);      print("inserted, len = %\n", counts.len());
    case .Err(error); print("insert failed\n");
    }
    upper := ("glosso").to_upper();
    defer upper.free();
    print("upper = %\n", upper);
}
```

CONSOLE / OUTPUT

```
inserted, len = 1
upper = GLOSSO
```

One name breaks the rule. `Math` is a namespace directory with no `module.glo`, but the driver rewrites the module name `Math` to `Math/Core` before the lookup runs, so scalar mathematics is reachable under its short name.

GLOSSO

```
#import "Io";
#import "Math";

main :: () {
    print("%\n", sqrt(2.0));
}
```

CONSOLE / OUTPUT
```
1.414214
```

> **NOTE**
> `Math` is the only rewritten name. No other namespace directory has a shorthand, and adding one would require a compiler change rather than a library change.

Modules that are one file

These fifteen names resolve to a single `.glo` file at the top of the tree, and each imports on its own.

Module	Purpose
`Ascii`	ASCII byte classification and case folding, all over `u8`.
`Bindgen`	Compile-time generation of bindings from a C header.
`Compiler`	The compile-time build API: nested compiles, C archives, build options.
`Context`	The allocator procedures and constructors behind the implicit `context`.
`Dynamic_Library`	Runtime shared-library loading, symbol lookup, and explicit handle release.
`Format`	String formatting entry points, including `value.format_int`.
`Html`	Lightweight HTML parsing, crawling, and extraction.
`Io`	The stream umbrella: `print`, writers, readers, scanning, files.
`Jump`	Explicit control transfer over resumable execution contexts.
`Process`	Process prelude: ids, executable path, exit status, abort.
`Simd`	Portable fixed-width vector values, `Simd(T, N)`.
`String_Builder`	Compile-time and runtime string building, with owned buffers, borrowed views, and allocation transfer.
`Terminal`	A dependency-free terminal and TUI toolkit.
`Thread`	Native threading primitives layered on `Os`.
`VarArg`	C ABI `va_list` support: `Va_List`, `start`, `arg`, `end`.

Terminal sessions and structured events

A `Terminal_Session` returned by `Terminal_Session.terminal_begin` owns every terminal setting it changes. Pair it with `Terminal_Session.end`, normally through `defer`, so input modes, the cursor, and the alternate screen are restored even when the program leaves early.

Call `Terminal_Session.read_event` for a `Terminal_Event` whose nonempty `Terminal_Event_Kind` is `Key`, `Mouse`, `Resize`, or `Focus`. `Terminal_Session.read_key` is the compatibility wrapper: while waiting for a key it consumes and discards mouse, resize, and focus events.

Windows reads `ReadConsoleInputW` records and uses native console controls where they preserve the same behavior; other terminals decode ANSI/UTF-8 keys, SGR mouse reports, and focus reports. Mouse coordinates are zero-based and relative to the current viewport, with horizontal wheel buttons and separate `moved`, `dragged`, and `double_click` flags. A classic Windows console cannot emulate every VT feature: alternate-screen sessions, canvas rendering, styling, and other ANSI-only controls can report `.Unsupported` when VT output is unavailable.

Modules that are a directory

These sixteen directories carry a `module.glo` entry point. An umbrella entry point uses `#export` to expose component modules; implementation fragments can be joined with `#load,embed`. The Lua entry point uses `#insert` to build its native runtime on demand.

Module	Purpose
`Archive`	ZIP central-directory reading and extraction with path and resource checks.
`Gui`	Native windows, input, 2D drawing, 3D scenes, and an immediate-mode UI on Windows x86_64 MSVC.
`Image`	Owned RGBA8 images and PNG, JPEG, and WebP codecs written in Glosso.

Module	Purpose
`Lua`	The Lua 5.5 C API, with a static runtime built from bundled sources.
`Math/Bignum`	Arbitrary-precision `BigInt` and `BigFloat`.
`Math/SpecialFunctions`	Airy, Bessel, gamma, hypergeometric, integrals, zeta and polylog.
`Memory`	Raw memory operations, generic slice algorithms, and `ManuallyDrop(T)`.
`Meta`	Compile-time type introspection and folded intrinsics.
`Os`	The target OS primitive boundary: threads, clocks, syscalls.
`Prelude`	Loaded automatically for every program; see the chapter on the implicit prelude.
`Random`	Deterministic engines, OS entropy, sampling, distributions.
`Sort`	Sorting, ordered search, heaps, deterministic permutations.
`Strings`	UTF-8 and UTF-16 views and conversions, plus owned UTF-8 buffer cleanup; exports `Memory`, `Utf8`, and `Utf16`.
`Time`	Clocks, durations, civil calendars, text formats, TZif zones.
`Typeclass`	`Functor`, `Applicative`, `Monad`, `Error`, and `Derive`; `Try` and `Termination` are in the Prelude.
`Unicode`	UTF-8 helpers plus the generated character tables.

Images and codecs

`#import "Image"` provides owned, tightly packed RGBA8 pixels and PNG, JPEG, and WebP codecs implemented in Glosso. `Image.image_read` detects the format from the bytes; `image_write` chooses it from the filename extension unless an explicit `Image_Format` is supplied. `Image.resize` takes output dimensions and uses nearest-neighbor sampling. Narrow imports such as `Image/Core` and `Image/Png` are also available.

GLOSSO
```
#import "Image";

main :: () -> Image_Error!void {
    photo := Image.image_read("photo.jpg")?;
    thumbnail := photo.resize(320, 240)?;
    _ := image_write("thumbnail.png", &thumbnail)?;
    _ := image_write("thumbnail.jpg", &thumbnail, .quality = 90)?;
    return .Ok;
}
```

`Image.image_decode(bytes)` borrows encoded bytes and returns independent pixels; `Image.encode` returns an owned dynamic byte array that cleans up at scope exit. PNG and WebP encoding preserve RGBA losslessly. JPEG encoding is lossy, accepts quality 1 through 100, and discards alpha. Decoding supports progressive JPEG and both lossy and lossless WebP stills; animated WebP is rejected. `Image_Limits` defaults to 67,108,864 pixels and 536,870,912 encoded bytes, with checks on codec working buffers too.

> **NOTE**
> An `Image` owns its dynamic pixel array and moves by value. Scope exit destroys the pixels automatically; `image.destroy()` releases them early. Use `image.clone()` for an independent copy and `*Image` for a borrowed receiver. Keep the creating `context.allocator` installed until cleanup. Pixels allocated from temporary storage expire at the next reset. File writes finish encoding before opening the destination, then create or truncate it; they are not atomic replacements.

Namespace directories

These six directories have no `module.glo` and no sibling file, so the directory name is not importable at all. Every component listed is importable by path, subject to the caveat below.

Directory	Components
`Container`	`BTreeMap`, `BTreeSet`, `BinaryHeap`, `Deque`, `LinkedList`, `HashMap`, `HashSet`, `Hash`, `Error`.
`Crypto`	`Aes`, `AesGcm`, `ChaCha20`, `ChaCha20Poly1305`, `Poly1305`, `Sha1`, `Sha2`, `Sha3`, `Hmac`, `Kdf`, `ConstantTime`, `Encoding`, `Random`, `Error`.
`File_System`	`Atomic`, `Copy`, `Directory`, `File`, `Map`, `Metadata`, `Native`, `Path`, `Sandbox`, `System`, `Temporary`, `Types`, `Walk`, `Watch`.

Directory	Components
Math	Core, Constants, Bit, Complex, Matrix, NumberTheory.
Net	Address, Socket, Poll, Async, Dns, Dns_Types, Resolver, Hosts, Interface, Platform, Url, Encoding, Http, Http_Types, Http_Auth, Http_Server, Https, Tls, Certificate, Trust_Store, WebSocket, Types.
Serialization	Core, Json, Toml, Yaml, MessagePack.

Filesystem path results can own text too. In File_System/System, a successful weakly_canonical(path) owns its string and must be released with string.free, including the fallback to absolute when the path is absent. relative(path, base) also returns an owned string, independent of temporary path components; even "." is an owned result rather than a borrowed literal.

In Net/Socket, Socket.socket_create defaults to kind = .Stream and protocol = .Default. It returns a Net_Error with kind .Unsupported when the target has no socket ABI or the family is .Unspecified. This is not a generic .System_Error with a zero native code. On success, the returned socket owns its descriptor and its fallible Drop closes it at scope exit. Socket.close remains available for an explicit early close.

Net/Tls verifies the peer chain and hostname by default. The Linux GNU SDK bundles static OpenSSL archives; other hosted targets resolve an OpenSSL-compatible provider at link time. Unless ca_file or ca_directory is supplied, Linux supplements the provider's default trust paths with a native CA bundle. SSL_CERT_FILE and SSL_CERT_DIR retain their provider-defined override behavior.

The Serialization components provide Serialization_Document.json_document_from_string, Serialization_Document.toml_document_from_string, Serialization_Document.yaml_document_from_string, and Serialization_Document.messagepack_document_from_bytes(bytes) for an owned parsed tree. The resulting Serialization_Document captures the current allocator; document.destroy() releases its arrays and decoded text. Views into document.root expire at destruction, and the allocator's storage must stay valid for the document's lifetime. For a raw model returned by a bundled value parser, value.destroy(.owned_text = true) also frees decoded text; the default leaves text borrowed for manually built models.

text.substring(start, end) and text.substring_view(start, end) from Strings/Utf8 borrow a clamped byte range without allocating. Their offsets need not be UTF-8 boundaries; both results last only as long as the input storage and must not be freed. Use view.clone() for an independent allocation. Trimming and split pieces also borrow the input; split owns only its array of views. Filesystem helpers such as parent_path(path), filename, and path_dirname use borrowed views where their contracts promise to borrow the input.

Two names are both a file and a directory. The file is what #import resolves to, because the directory has no module.glo; the directory holds the components that the file deliberately leaves out.

Module	Included by the file	Explicit only
Io	Core, Transfer, Adapters, Buffer, Scan, Vector, Binary, Hash, Format, Terminal	none
Process	the small prelude only	Arguments, Child, Command, Environment, Inspection, Limits, Pipe, Scheduling, Signals, System, User

> **NOTE**
> A component declares its dependencies explicitly. Importing its umbrella adds the public components selected by #export; an ordinary #load or #import inside that component does not expose those dependency names to the caller. Files intended only as shared source fragments are assembled explicitly with #load,embed.

Finding the module that owns an operation

Nothing outside the prelude is in scope until it is imported, so a missing import shows up as an unbound name rather than a link error. The compiler suggests the closest name it can already see, which is a hint about spelling and not about the module.

CONSOLE / OUTPUT

```
error: unknown function sqrt
  --> …\miss.glo:4:18
   |
4 |     print("%\n", sqrt(2.0));
   |                  ^^^^
help: did you mean 'seek'?
```

The direct way to see what a module exports is to ask for a name it does not have. A failed `#import,only` names the module's entire public surface in a note, which makes it a usable index.

CONSOLE / OUTPUT

```
error: #import,only names unknown public declaration 'is_ascii_hex' in module "Ascii"
note: public names: is_ascii_alpha, is_ascii_alphanumeric, is_ascii_digit,
      is_ascii_lowercase, is_ascii_uppercase,
      is_ascii_whitespace, to_ascii_lower_byte, to_ascii_upper_byte
```

Two things that list does not show. A typeclass method is not a module-level declaration, so `sqrt` does not appear in the public names of `Math/Core` even though `#import "Math"` is what brings it into scope. Visibility also matters: private and unit-public declarations are excluded from selective imports by other ownership units. An explicitly `#public` name is exported even when its spelling begins with `_`.

- Operations on scalars, including the numeric typeclasses, live in `Math/Core`; bit manipulation in `Math/Bit`; the named constants in `Math/Constants`.
- Operations on a borrowed slice live in `Memory`; operations that order or search one live in `Sort`; operations that own storage live under `Container`.
- Anything that touches the operating system has a portable face in a top-level module and a raw face in `Os`. Prefer the portable one: `Os` is the boundary the rest of the library is built on, and its shape changes with the target.

read_file(path, max_count) performs one bounded read and may return fewer bytes than requested. Use read_file_up_to(path, max_count) to repeat short reads up to the limit, or File.open(path) followed by file.read() to read through EOF. Raw returned strings need free() with the original allocator.

Default parameters and typed flags

The library uses one entry point when operations differ only by a setting, allocator, or fixed argument. Omit a setting to use its default; select an alternative with `.parameter = value`. Input-only option records have been replaced by direct parameters. Values with their own lifetime, such as a `File`, a `Dns_Resolver_Config`, and workspace `Build_Options`, remain records.

`open` in `File_System/File` takes a path followed by `access = .Read`, `disposition = .Open_Existing`, `permissions = File_Permissions.Default_File`, and `flags = Open_Flags.None`. Its path type must satisfy `AsView(P, Path_View)`. The returned `File` owns its handle, and its fallible `Drop` closes the handle automatically. `file.close()` can finish it early and consumes the handle even if closing reports an error.

GLOSSO

```
#import "File_System/File";

create_private_file :: (path: string) -> Fs_Error!void {
    file := File.open(path, .Write, .Create_New, .permissions = File_Permissions.Owner_Read |
File_Permissions.Owner_Write, .flags = Open_Flags.Exclusive | Open_Flags.Sync)?;
    _ := file.close()?;
}
```

Operation	Current call
Read an existing file	`File.open(path)`
Create or truncate a writable file	`File.open(path, .Write, .Create_Always)`
Append, creating when absent	`File.open(path, .Write, .Open_Or_Create, .flags = .Append)`
Open for reading and writing	`File.open(path, .access = .Read_Write)`
Synchronize data only	`file.sync(.data_only = true)`; omit the parameter for full synchronization

Independent Boolean choices are collected in an `enum_flags` type. Combine qualified members with `|`, test membership with `&`, and clear a member with `flags = flags & ~Open_Flags.Append`. Plain integers and members of an unrelated enum cannot satisfy a typed flags parameter. Filesystem permissions likewise use `File_Permissions` members instead of untyped integer constants.

API family	Direct parameter examples
Terminal	`Terminal_Session.terminal_begin(Terminal_Flags.Default | Terminal_Flags.Mouse, .title = "App")`; `session.clear(.Line)`
JSON	`json_serialize(value, .pretty = true, .indent = "    ")`; `json_deserialize(T, text, .max_depth = 64)`
Memory	`pointer.fill(byte_count)` zeros a range; `pointer.fill(byte_count, byte_value)` fills it; `pointer.copy` supports overlap

API family	Direct parameter examples
Strings and formatting	`text.index_of(needle, .start = offset)`; `text.replace(old, new, .limit = count)`; `format(fmt, args..., .allocator = allocator)`
Sorting	`values.sort()` is stable by default; `values.sort(.stable = false)` selects unstable sorting

Methods, constructors, and domain operations

Methods describe a value's own behavior: `values.push(item)`, `text.substring(start, end)`, and `file.close()`. Constructors belong to the type they create, as in `File.open(path)`, `Image.image_read(path)`, and `f64.from_bits(bits)`. Domain operations use free functions with named inputs, such as `image_write(path, &image)`, `json_serialize(value)`, and `add_build_file(path, workspace)`. Import the module that owns the operation; importing a domain module does not add all of its operations to every string or byte slice.

Native windows and drawing

`#import "Gui"` exports `Window`, `Input_Types`, `Input`, `Scene`, and `UI`. The current backend requires a Windows x86_64 MSVC target on a Windows host. It builds and caches its bundled native sources with the compiler's embedded Clang and uses Win32 and Direct3D 11; it has no raylib dependency. `create_window` creates a window, `update_window_events` pumps input once per frame, and `begin_render_frame` / `present_frame` bracket rendering. Querying input does not pump the event queue.

`Gui/Scene` supplies meshes, cameras, materials, scene submission, rectangles, and text labels. Window and mesh handles use explicit `destroy_window` and `release_mesh` cleanup. The UI layer keeps its state in a caller-owned `Ui_Context`; widgets need stable nonzero IDs, and input routing uses the preceding frame's widget layout. Keep the context across frames and submit each frame's complete event batch.

16.2 Grammar Reference

Glosso is parsed by a hand-written recursive-descent parser with a precedence-climbing expression loop. The grammar below is written to match that parser production by production rather than to be fed to a generator, so where the parser makes a decision by scanning ahead the grammar says so instead of pretending to be context-free.

Two things the notation cannot carry. Operator precedence and associativity are not in the grammar at all: they live in a table the parser fills in before parsing, from `#operator` declarations in the source dependency graph, which is why the expression production is one flat loop. That mechanism is described in the chapter on the operator prescan, and the standard tiers in the chapter on operator precedence. Token formation is likewise a separate pass; the chapter on punctuation and the token table owns it.

Notation

Form	Meaning
`=` and `;`	open and close a production
juxtaposition	sequence
`|`	alternatives
`[ x ]`	x is optional
`{ x }`	x repeats zero or more times
`( x )`	grouping
"x"	a literal token spelling
UPPERCASE	a token class produced by the lexer

Tokens

The lexer produces the classes below. `#char` is the one directive the lexer itself interprets. It yields a marked integer token whose origin is retained so the checker can infer `u8`, `s8`, or `rune` for ASCII literals and `rune` for non-ASCII literals. A `\\` at the start of a line yields `MLSTR`, which only `#string` and `#asm` consume.

GRAMMAR

```
IDENT     = ( letter | "_" ) { letter | digit | "_" } ;
INT       = decimal | based | charlit ;
decimal   = digit { digit | "_" } ;
based     = decimal "#" basedigit { basedigit | "_" } ;   (* base 2 to 36 *)
charlit   = "#char" quote ( codepoint | char-escape | "\x" hex hex | unicode-escape ) quote ;
FLOAT     = decimal "." decimal [ exponent ] | decimal exponent ;
exponent  = ( "e" | "E" ) [ "+" | "-" ] digit { digit } ;
```

```
STRING    = quote { any - quote - "\" | string-escape } quote ;
CSTRING   = "c" STRING ;   (* no whitespace between prefix and quote *)
BYTES     = "b" STRING ;   (* decoded bytes, no trailing zero *)
string-escape = "\" fixed-escape | "\x" hex hex
              | unicode-escape ;
unicode-escape = "\u{" hex [ hex [ hex [ hex [ hex [ hex ] ] ] ] ] "}" ;
MLSTR     = "\\" { any - newline } ;
DIRECTIVE = "#" ( letter | digit | "_" ) { letter | digit | "_" } ;
OPNAME    = "'" opchar { opchar } "'" ;                  (* no whitespace *)
LABEL     = "'" IDENT ;                                  (* no closing quote *)
BACKTICK  = "`" IDENT ;
DBACKTICK = "``" IDENT ;
TRIPLECOLON = ":::" ;                    (* instance separator *)
OP        = opchar { opchar } ;                          (* maximal munch *)
opchar    = "+" | "-" | "*" | "/" | "<" | ">" | "!" | "@" | "%"
          | "^" | "&" | "|" | "~" | "?" | "." | "," | ":" | "=" ;
keyword   = "if" | "ifx" | "else" | "while" | "for" | "return"
          | "break" | "continue" | "struct" | "union" | "case"
          | "enum" | "using" | "defer" | "context" | "cast"
          | "acast" | "where" | "noalias" ;
```

There are exactly nineteen keywords. `typeclass`, `instance`, `distinct`, `enum_flags`, `true`, `false`, `null`, `mut` and `volatile` are ordinary identifiers that the parser recognises by position, which is why each of them is still legal as a variable name.

Program and declarations

The `definition` alternative is deliberately shared by `::`, `:=`, `::=`, and a typed second `:` or `::`. A definition-shaped initializer therefore creates a named type, procedure, namespace, library, or macro rather than an ordinary constant or global. An explicit annotation describes that declaration value and is checked against its actual kind; without one, the same kind is inferred.

An opening parenthesis does not by itself make a procedure. `K :: (1 + 2) * 3;` and `K :: (42);` are ordinary constants, including inside a body. Parameter syntax and the tokens after the matching `)` distinguish a procedure from a grouped value.

GLOSSO

```
Point : type : struct { x: ssize; }
double : (ssize) -> ssize : (value: ssize) -> ssize { return value * 2; }
Io : Namespace : #import "Io";
kernel32 : Library : #library,system "kernel32";
Show_One : Typeclass(type) : typeclass(T: type) {
    show_one :: (value: T) -> string;
}
trace : Macro((ssize) -> void) : (value: ssize) #expand { _ := value; }
Auto_Point := struct { x: ssize; }
auto_main ::= () {}
```

Definition	Its explicit declaration annotation
aggregate, `distinct`, `#fn_ptr`	`type`
`#import`, `#load`	`Namespace`
`#library`	`Library`
typeclass with parameter kinds `K...`	`Typeclass(K...)`
macro with signature `F`	`Macro(F)`
ordinary procedure	its procedure type

> **NOTE**
> `Namespace` and `Library` are case-sensitive compile-time-only types. `Typeclass(K...)` and `Macro(F)` are parameterized forms recognized in an explicit declaration annotation: `Typeclass` needs at least one parameter kind, and `Macro` takes exactly one procedure signature. They are not generally callable type constructors in ordinary expressions—`row(Typeclass(type))` is an unresolved function call, for example. The old lowercase `namespace`, `library`, and `pattern` type spellings do not resolve.

GLOSSO

```
#import "Io";

LIMIT :: 10;
WIDTH : s32 : 4;
counter : s32 = 0;
#thread_local depth : s32 = 0;
Meters :: distinct f64 #derive(Add, Eq);
Callback :: #fn_ptr(a: s32) -> s32;
libc :: #library "c";
c_abs :: (v: s32) -> s32 #foreign libc "abs";
```

```
double :: (a: s32) -> s32 { return a * 2; }

main :: () {
    cb: Callback = double;
    d : Meters = 1.5;
    print("% % % % %\n", LIMIT, WIDTH, counter, depth, cb(21));
    print("% %\n", c_abs(-7), d + d);
}
```

CONSOLE / OUTPUT

```
10 4 0 0 42
7 3
```

Aggregates

GRAMMAR

```
struct      = "struct" { structdir } "{" { structmember } "}" ;
structdir   = "#c_call" | "#packed" | "#align" "(" INT ")" | "#magic" STRING
            | derive | "#modify" ( block | expression ) ;
structmember= [ "using" ] [ "#as" [ "using" ] ] IDENT ":" fieldtype
              [ "=" expression ] ";"
            | "#empty" ":" type { "," type } ";" ;

enum        = "enum" [ type ] "{" { IDENT [ "::" [ "-" ] INT ] ";" } "}" ;
enumflags   = "enum_flags" type "{" { IDENT [ "::" INT ] ";" } "}" ;

union       = "union" { uniondir } "{" { unionvariant } "}" ;
uniondir    = "#raw" | "#magic" STRING | derive ;
unionvariant= [ "#from" ] IDENT ":" fieldtype ";" ;

fieldtype   = type | struct | union | enum | enumflags ;
```

Every aggregate member ends with `;`, and the diagnostic names the member: a missing one is `error: struct member 'x' must end with ';'`. An enum variant fixes its integer with `::`, not `=`; writing `=` gives `error: enum variant integer values use '::', not '='`. An aggregate written in field position is hoisted to file scope under the name `Owner.field`, which is why `fieldtype` may open a new declaration.

An enum with signed backing may give a variant a negative integer literal, such as `Low :: -1;`. Each named field, variant, and parameter must be unique within its declaration; the blank name `_` may repeat. A struct has at most one `#as` field.

Procedures

The order is fixed and the parser enforces it: parameters, then the return type, then every directive, then every `where` clause, then the body. Reversing the last two is `error: function directives must appear before where clauses`. A body is required unless the declaration is `#magic`, `#foreign`, or a signature-only requirement inside a typeclass. `#operator` is the only operator directive. Its binary and assignment modes take a literal integer level from 0 to 1000, or reuse a level already registered for the spelling. Its unary modes never take a level. A suffix operator may use `#operator(suffix, try(Target))`, where `Target` names its leading `#empty: $Target` witness. It returns `Try_Branch(Try(Source).Output, Target)`: `Continue` supplies the expression's value and `Break` leaves the boundary with the completed carrier. The ordinary operator body performs any residual conversion. All anonymous `#empty:` witnesses must precede runtime parameters. Defaults are evaluated at the call site, so a receiver method cannot refer to `#self` in a parameter default.

A `#expand` procedure with an explicit result other than `void` is a value macro automatically. Its body may contain several statements, branches, and static recursion, provided every reachable path returns a value. The older `#expand,expression` form remains for a macro whose result is inferred; that form must still reduce to exactly one `return <expression>;` (or equivalent static branches).

> **NOTE**
> Control flow in a macro distinguishes its own expansion from its caller explicitly. Plain `return` supplies the macro's value, while prefixing `return` with a backtick leaves the procedure that invoked the macro. Plain `defer` runs inside the macro before its value returns; prefixing `defer` with a backtick registers the statement in the caller. A backtick likewise selects a caller name for reads and writes, not only for declarations.

Typeclasses and instances

GRAMMAR

```
typeclass   = "typeclass" "(" [ tcparams ] ")" { where }
              { derive | minimal } "{" { member } "}" ;
tcparams    = tcparam { "," tcparam } [ "," ] ;
tcparam     = [ "#comptime" ] IDENT ":" type ;
minimal     = "#minimal" "(" [ disjunction ] ")" ;
disjunction = conjunction { "|" conjunction } ;
conjunction = atom { "," atom } ;
```

```
atom        = name | "(" disjunction ")" ;
member      = name "::" ( classproc | type ";" ) ;

instance    = head ":::" "instance" [ "!" ] IDENT { where }
              "{" { member } "}" ;
head        = token { token } ;   (* every token up to the ":::" *)
```

An instance is found by scanning forward for a `:::` at bracket depth zero whose next token is the identifier `instance`; everything before that `:::` is the head, taken as raw tokens and re-parsed as a type. `:::` is a dedicated, reserved lexer token; the former `:: instance` spelling is rejected rather than interpreted as an instance. `#minimal` sits between the header and the body, so a `#minimal` written after the closing brace is read as a new declaration and fails with `error: bad decl name 'minimal'`. A member whose body is a type rather than a parameter list is an associated type.

GLOSSO

```
#import "Io";

Scale :: typeclass (L: type, R: type) #minimal(scale_by) {
    Output :: type;
    scale_by :: (a: L, b: R) -> Output;
}

Sized :: typeclass (C: type) {
    Item :: type;
    measure :: (c: C) -> ssize;
    is_empty :: (c: C) -> bool {
        return measure(c) == 0;
    }
}

(s32, s32) ::: instance Scale {
    Output :: s32;
    scale_by :: (a: s32, b: s32) -> s32 { return a * b; }
}

([]mut $T) ::: instance Sized where Show(T) {
    Item :: T;
    measure :: (c: []mut T) -> ssize { return c.count; }
}

main :: () {
    print("%\n", scale_by(6, 7));
    values : [4]s32 = .[1, 2, 3, 4];
    print("% %\n", measure(values[..]), is_empty(values[..]));
}
```

CONSOLE / OUTPUT

```
42
4 0
```

Types

GRAMMAR

```
type        = modified [ "!" type ] ;          (* right associative *)

modified    = "?" modified
            | "*" { "*" } quals modified
            | "[" "*" "]" quals modified
            | "[" "]" quals modified
            | "[" ".." "]" modified
            | "[" ( INT | IDENT | "_" ) "]" modified
            | ( "#aos" | "#soa" ) modified
            | "$" IDENT
            | DBACKTICK
            | "#Context" | "#Never" | "#Self_Type"
            | struct | union | enum | enumflags
            | parentype
            | named ;

quals       = { "mut" | "volatile" } ;
named       = IDENT { "." IDENT } [ "(" typeargs ")" ] { "." IDENT } ;
typeargs    = type { "," type }
            | type "," dimension "," dimension     (* Matrix *)
            | type "," INT ;                       (* Simd *)
dimension   = INT | IDENT ;
parentype   = "(" [ elem { "," elem } ] ")" [ "->" type ] ;
elem        = [ "noalias" ] [ IDENT ":" ] type ;
```

`E!T` binds looser than every prefix modifier and groups to the right, so `Io_Error!?*User` is one result whose payload is `?*User`, and `E1!E2!T` is `E1!(E2!T)`. A parenthesised list is a tuple type unless an `->` follows, in which case it is a procedure type; a single-element list with no name is the parenthesised type itself. An inline `struct`, `union` or `enum` is legal only where a field name is already in hand, and elsewhere gives `error: inline aggregate types are only allowed in aggregate fields`. `[_]T` infers a fixed array's length from its initializer, including a `b"..."` literal; it does not denote a many-pointer.

NOTE
Two shapes look like grammar but are not. `T.[ ... ]` does not exist: the postfix loop has an arm for `.{` and none for `.[`, so `s64.[1,2]` is the type `s64` followed by a separate array literal, and `for v: s64.[1, 2]` reports `error: 'type' is not iterable; it needs an Iterable instance`. And adjacent operator characters glue into one token, so `E!*T` lexes as the single operator `!*`; the result type of a pointer must be written `E! *T` or `E!(*T)`.

Expressions

The `binop` loop is precedence climbing over a table keyed by operator spelling. An operator with no entry in the table ends the expression, which is how a reserved spelling produces a plain unexpected-token error. Two operators that share a precedence level but disagree about associativity cannot be mixed without parentheses, and the parser says which two.

In value expressions, `&x` and `x.&` take the address of an addressable place; `*p` and `p.*` dereference a typed pointer. The prefix `*` still constructs a pointer when its operand is a type value. `c"..."` has inferred type `cstring` and includes a trailing zero in static storage. `b"..."` has inferred type `[N]u8`, where `N` is its decoded byte count, with no implicit terminator.

Bracket types can also appear as type values: `size_of([]u8)`, `Bytes :: [..]u8;`, and `Container([]u8)` are valid shapes. A generic struct type can qualify its literal, as in `Box(ssize).{ .value = 3 }`. Array and struct literal elements require commas; both forms allow a trailing comma. Chained tuple selection `tuple.0.1` is two member accesses, never a floating-point token.

NOTE
A `#code` quote may contain a braced statement list, one unbraced statement, or a parenthesised expression such as `#code (a < b)`. The single-statement form borrows its terminator from the surrounding delimiter, which is a contextual exception to the `statement` grammar. `#insert` is a primary expression as well as a statement, declaration, and pattern splice; `#insert,scope() code` resolves a `Code` value in the insertion environment.

CONSOLE / OUTPUT
```
error: operators '<+' and '+>' both have precedence 19 but different associativity, so '<+ +>' is ambiguous; add parentheses
```

An infix `!` is handled by the parser directly rather than through the table, so it works even though `!` carries no precedence, and it stops at `==` followed by `{` so that the switch head is not swallowed. A `..` whose right side is missing becomes a suffix operator, which is what makes `a[2..]` and the pattern range `100..` parse.

Statements

A procedure, struct, enum, union, distinct type, typeclass, or function-pointer alias declared inside a body has lexical scope. A name :: expression binding is a local constant. Ordinary local procedures do not capture runtime locals. A nested distinct and a bodyless nested #foreign or #magic procedure accept an optional semicolon; braced definitions do not. The terminator on a local declaration, assignment, expression statement, value-bearing return, or legacy #asm is waived immediately before } inside a #code quote. Expression-position #inline loops may use a comma after an expression in the generated body. #falling must be the last statement of its arm. An ifx conditional must produce a value on every path.

GLOSSO
```
#import "Io";

Shape :: union {
    Circle: f64;
    Square: f64;
}

main :: () {
    total := 0;
    for i: 0..4 total += i;
    for < i: 0..4 total += i;

    n := 2;
    if n == {
    case 1, 2;
        print("one or two\n");
        #falling;
    case 3;
        print("three\n");
    else;
        print("other\n");
    }

    grade := ifx n == { case 1; "I"; case 2; "II"; else; "?"; };
    s: Shape = .Square(3.0);
    #pattern .Square(side) := s else { return; }
    print("total=% grade=% area=%\n", total, grade, side * side);
}
```

CONSOLE / OUTPUT

```
one or two
three
total=12 grade=II area=9
```

Patterns

GRAMMAR

```
patternexpr = patternlet
            | "#pattern" ( "{" { arm [ ";" | "," ] } "}"
                         | arm
                         | pattern ) ;
arm         = pattern "=" expression ;
patternswitch = "#pattern" [ expression ] "==" "{"
                  { patterncase } [ "else" ";" casebody ] "}" ;
patterncase = "case" ( pattern | "{" { arm [ ";" | "," ] } "}" )
              [ "if" expression ] ";" casebody ;

pattern     = element { "|" element } ;
element     = rangeexpr | punary ;
punary      = "#insert" expression | "==" ppostfix | ppostfix ;
ppostfix    = pprimary { "." namelike
                       | "(" [ pattern { "," pattern } ] ")"
                       | ".{" pfields "}" } ;
pprimary    = "..."
            | BACKTICK | DBACKTICK
            | "*" IDENT
            | INT | FLOAT | STRING | "true" | "false" | "null"
            | IDENT [ "@" punary ]
            | "." IDENT
            | ".{" pfields "}"
            | ".[" [ pattern { "," pattern } ] "]"
            | "(" pattern ")" ;
pfields     = { ( "..." | "." namelike [ "=" pattern ] | pattern )
              [ "," ] } ;
rangeexpr   = expression ;   (* one containing ".." or "..=" *)
```

A bare identifier in pattern position binds, which is why matching a named constant needs the `==` marker and why a backticked name refers to the quoting environment instead. A range is not pattern grammar at all: an element that contains `..` or `..=` is parsed as an ordinary expression and converted by `ToPattern`, which is how `100..`, `..=9` and a bare `..` work without a production of their own.

Directives

A directive is lexed as one token class and given meaning by the position it appears in, so the same spelling can mean different things in a declaration, a statement and an expression. The table gives the positions each one is accepted in.

Directive	Position
`#enable`, `#disable`	file scope, between top-level declarations
`#import`, `#load`, `#library`, `#fn_ptr`	declaration position; `#import` and `#load` also bare
`#thread_local`	prefix of a global variable declaration
`#comptime`	top-level declaration, statement, expression prefix, parameter modifier
`#insert`	top-level declaration, statement, expression, pattern element; the declaration, statement, and expression forms accept optional `,scope(...)`
`#if`	declaration blocks and statements; `#assert`, `#compile_error`, `#asm`, `#bytes`, `#push_context`, `#push_allocator` appear as statements
`#inline`	before `while` or `for`, in statement or expression position, and as a function directive
`#pattern`, `#partial`, `#falling`	pattern test or switch, after `if`, and at the end of a case arm
`#try`	`#try { expression }` in expression position, and `#try defer`
`#code`, `#string`, `#simd`, `#undefined`, `#meaningful`	expression position; `#code` quotes a block or one statement
`#source_location`	primary expression, with no arguments; a `Source_Location` value for the directive's lexical `#` token, including in a constant, default, or quote
`#char`	a character literal recognised by the lexer; ASCII defaults to `u8`, non-ASCII to `rune`
`#derive`, `#minimal`	`#derive` on `struct`, `union`, `typeclass` and `distinct`; `#minimal` on `typeclass` only
`#empty`, `#as`, `#from`, `#raw`, `#packed`, `#align(N)`	aggregate declarations; `#empty` also in a parameter list

Directive	Position
`#c_call`, `#modify`, `#magic`	both aggregate and procedure declarations
`#operator`, `#expand`, `#foreign`, `#fresh_result`, `#unchecked_transfer`, `#no_context`, `#returns_twice`, `#dump`, `#must`, `#lazy`	procedure declarations; `#lazy` on a parameter. A diverging return type is `-> #Never`.
`#Context`, `#Never`, `#Self_Type`, `#aos`, `#soa`	type position; `#Self_Type` belongs to inherent blocks and instances
`#public`, `#public(unit)`, `#private`	one named declaration or a transparent group; also the visibility default after an inherent block's `:::`
`#self`, `*#self`, `*mut #self`	first parameter of an inherent method; `#self` names the receiver in its body
`#on_drop_error(owner, error) { ... }`	a local cleanup error handler registered immediately after its owning declaration

Decisions the grammar defers to lookahead

The parser uses lookahead in the cases below to choose between forms that share an opening token sequence.

- An instance is recognised by scanning for the dedicated `:::` token at bracket depth zero followed by the identifier `instance`. Until that token is found, the head is held as raw tokens.
- `#pattern` is a test when a `=` appears at depth zero before any `;`, `,` or opening `{`; a `:=` there makes it the bind-or-diverge form; otherwise it is a pattern value.
- `==` followed by `{` ends an expression, so `if subject == {` opens a switch instead of comparing against a struct literal.
- Inside a body, name :: followed by a procedure signature, struct, enum, union, distinct, typeclass, or #fn_ptr introduces a local declaration. A parenthesized value initializer remains a local constant.
- `..` becomes a suffix operator when the token after it can only end an expression, and a `#pattern` element treats `|` as an alternative separator rather than the bitwise operator.
- A named parameter list is distinguished from an expression by three tokens: `.`, an identifier, then `=`. Writing `f(name = value)` gives `error: unexpected token '='`.

> **NOTE**
> After a parse error the parser discards tokens until it reaches one in column 1 that can start a declaration: an identifier or operator name followed by `:` or `::`, or one of a small set of top-level directives. A construct indented past column 1 is therefore never a resynchronisation point, and one bad declaration can swallow the indented declarations that follow it.

Memory contract attachment

GRAMMAR

```
memory-prefix = "#memory" "{" [ memory-effect { "," memory-effect } [ "," ] ] "}" ;
contracted-decl = ( memory-prefix [ visibility ] | visibility memory-prefix ) function-or-fn-ptr-decl ;
memory-effect = effect-name [ "(" operand { "," operand } [ "," ] ")" ] ;
operand = place | IDENT ":" ( place | STRING ) ;
place = ( IDENT | "#self" | "arg" "(" INT ")" ) { "." ( IDENT | INT ) } ;
memory-overlay = "#memory" qualified-name "{" memory-effect { "," memory-effect } [ "," ] "}" [ ";" ] ;
trusted-block = "#memory" "trusted" "(" STRING ")" block ;
```

Effect names and operand shapes are constrained by the contract site. Leading contracts attach to one function or named function-pointer declaration. Calls can carry postfix assertions; historical trailing declaration contracts also accept a bare effect or a parenthesized effect list. The Memory Contracts and Storage Lifetimes chapter lists effects and analysis modes.

16.3 Diagnostic Reference

Glosso has no diagnostic numbers. The message text is the identifier, so the way to find an entry below is to search for the exact string the compiler printed. The entries use representative names and values; the compiler substitutes those from the program it is checking.

CONSOLE / OUTPUT

```
error: unknown function print
  --> …\hello.glo:2:5
   |
2 |     print("hello\n");
   |     ^^^^^
help: did you mean 'tprint'?
```

Internally a message may carry a `lex error line N:` or `parse error line N:` prefix; the renderer strips it, so the prefix never appears in output. Checker diagnostics are collected rather than fatal: several are reported in one run, separated

by a blank line, and the run ends with a count. Lexical errors are not collected, because a lexical failure aborts the whole file at once.

CONSOLE / OUTPUT

```
error: 300 does not fit s8, whose range is -128..=127
  --> …\two.glo:1:18
   |
1 | a :: () { x: s8 = 300; }
   |                   ^^^

error: 400 does not fit s8, whose range is -128..=127
  --> …\two.glo:2:18
   |
2 | b :: () { y: s8 = 400; }
   |                   ^^^

error: aborting due to 2 previous errors
```

The compiler's own exit status distinguishes a usage mistake from a rejected program, and `--run` reports the child process separately.

Exit status	Meaning
`0`	The requested artifact was produced.
`1`	The program was rejected, a source file could not be read, or a program run under `--run` failed.
`2`	The command line was wrong; the usage block is printed with the message.

A program run under `--run` that aborts is reported as `glosso: program exited with status exit code: 134` on Windows, and the compiler itself then exits `1`. A WASI module takes the equivalent path through `proc_exit(125)`. Overflow and bounds traps print a runtime-check category before terminating.

Driver messages

Driver messages are the only ones with no source frame. They are written to standard error, prefixed `glosso:` , and are produced before any file is read.

Message	Produced by
`unknown option --bogus-option`	Any unrecognised flag. The usage block follows.
`unknown backend 'nope' (choose llvm, object, static-lib, shared-lib, or llvm-ir)`	`--backend` with a name outside the set.
`--color must be auto, always, or never`	`--color` with any other value.
`--run is available only for --backend llvm`	`--run` combined with an object or library backend.
`--std directory must contain Prelude.glo or Prelude/`	`--std` pointing at a directory that is not a standard library.
`cannot read missing.glo`	The named source file does not exist. Exit status 1, not 2.

Lexical errors

A stray visible character is named, for example `unexpected character '🙂'`. Invisible characters use a code point and, when known, a descriptive name: a byte-order mark is `unexpected character U+FEFF (byte order mark)` and a non-breaking space is `unexpected character U+00A0 (no-break space)`. Only space, tab, CR, and LF count as source whitespace.

Message	Produced by
`reserved operator token '=' (operator '=' cannot be overloaded)`	A reserved spelling written as an operator name or in operator position, including `:::`, `.*`, and `.&`. The message names the operator twice.
`operator token cannot end with '.'`	An operator run whose last character is `.` and which is not `..`, as in `2 *. 3`.
`expected a directive name after '#'`	A bare `#` with no directive name.
`operator name must not be empty`	An empty quoted operator name. Whitespace in a quoted name and an unterminated name have their own diagnostics.
`float literal '5.' needs a digit after the '.'; write '5.0'`	A decimal token followed immediately by a dot with no fractional digit.
`stray '.': a member name must follow '.' directly, with no space between them`	A standalone dot that cannot start member access.
`unterminated block comment`	End of input while a nested `/* ... */` depth is still open.

Message	Produced by
`unterminated string literal`	A `"` with no closing quote before end of file.
`unknown string escape '\q'`	A quoted string uses a backslash escape outside the closed escape table.
`\x escape needs exactly two hexadecimal digits`	A quoted string's `\x` is not followed by exactly two hexadecimal digits.
`\u escape must use \u{hex-digits}`	A quoted string uses `\u` without braces; related messages reject an empty, over-six-digit, non-hexadecimal, or non-scalar value.
`unexpected token '=-'`	Adjacent operator characters form one token. If a valid split exists, the diagnostic explains the split and asks for spaces, as in `a = -1`.
`unexpected directive 'frobnicate'`	A `#name` the parser does not recognise in that position. There is no unknown-directive message.
`unexpected token 'line one'`	A `\\` continuation line outside a `#string` or `#asm` block.

Number literals

Message	Produced by
`legacy '0x' integer literals are not supported; use '16#digits'`	`0xff`. The `0b` form has its own message naming `2#digits`.
`integer literal base must be between 2 and 36, got 40`	`40#zz`. Base 1 and base 0 give the same message.
`digit '2' is not valid for base 2`	`2#102`.
`base-2 integer literal needs at least one digit`	`2#` with nothing after the marker.
`base-16 integer literal does not fit in 128 bits`	A based bit pattern wider than the lexer's 128-bit pattern payload. Decimal integer tokens have arbitrary precision and instead receive the ordinary destination-range check.
`300 does not fit s8, whose range is -128..=127`	A literal outside the annotated type. The message always states the range.
`a based literal is a bit pattern, so it may use every bit of s8 - this one is wider than that`	A `note` attached to the previous message when the literal was written `base#digits`, as in `16#1ff` at `s8`.
`invalid '_' separator in number literal`	A leading, trailing, or doubled `_`. Fractional parts, exponents, and base digits each have their own wording.
`scientific number literal needs exponent digits after 'e' or 'E'`	`1e` with no digits.
`float literal '1e400' is out of range for f64; float literals are read as f64 values, including those given an f80 or f128 type`	A float token overflows the initial binary64 reading, or a nonzero mantissa underflows to zero.
`number literals must use ASCII digits`	A non-ASCII digit anywhere in an integer, fractional part, or exponent.

`#char` and `#string`

Message	Produced by
`#char expects a string literal`	`#char 5`, or a newline between `#char` and the string.
`unterminated #char string literal`	`#char "a;`.
`#char expects one character`	`#char ""`.
`#char string literal must contain exactly one character`	More than one direct code point, as in `#char "ab"`.
`#char \x escape needs exactly two hexadecimal digits`	`#char "\x4"` or `#char "\xZZ"`; `#char "\x41"` is valid and has value 65.
`unknown #char escape '\q'`	A `#char` escape outside its fixed table, exact `\xHH` form, and `\u{...}` Unicode form.
`#char \u escape needs \u{...} with one to six hexadecimal digits`	A malformed Unicode escape in a character literal. A surrogate or value above U+10FFFF instead reports `#char \u escape is not a Unicode scalar value`.
`#char literal can infer only as u8, s8, or rune, not u32`	`c: u32 = #char "A"`. The message names the type that was asked for.
`#char literal can infer only as rune, not u8`	`c: u8 = #char "λ"`.

Message	Produced by
`#string expects one or more '\\' lines`	A `#string` immediately followed by `;`.
`unknown #string modifier 'wrapped'; expected 'oneline' or 'escape'`	Any modifier outside the pair.
`duplicate #string modifier 'oneline'`	The same modifier written twice.

Declarations, procedures, and parameters

Message	Produced by
`bad decl name '1'`	A top-level declaration whose name does not lex as an identifier, as in `1bad :: 3;`. Module-level `#if` declaration blocks are supported.
`declaration must end with ';'`	A top-level declaration that did not end in its own definition body and lacks its required terminator, such as `#import "Io"`, `K :: 5`, or `G: ssize = 7`. The frame points just after the declaration's last token.
`expected a declaration, found ';'`	A stray semicolon at file scope, including one after a function or aggregate body. After a braced definition the help says the declaration ended at `}` and to remove the semicolon.
`declaration 'Wrong' has type type, not declared type Library`	An explicit declaration-value annotation that does not match the named definition on its right.
`Typeclass expects the kind of each typeclass parameter`	`Typeclass` used without at least one parameter kind in an explicit declaration annotation.
`Macro expects one function signature, for example Macro((ssize) -> void)`	`Macro` with zero or several type arguments in an explicit declaration annotation; a non-procedure argument instead reports `Macro expects a function signature`.
`a top-level constant binding uses '::'; the second ':' is missing`	`Foo : struct { … }`.
`a global variable without an initializer needs a type`	`g: ;`.
`function 'render' is already declared`	Two ordinary procedures with one name. There is no ad-hoc overloading; the constant form is `constant 'K' is already declared`.
`constant cannot be modified`	Assigning to a local `::` binding. It carries `help: change '::' to ':=' to make the binding mutable`. Assigning to a top-level constant gives `unbound '<name>'` instead.
`#thread_local can only prefix a global variable declaration`	`#thread_local` on a constant or a procedure.
`main must not take parameters; use get_cmdline_args() for command-line arguments`	Any parameter on `main`.
`noalias parameter 'v' must have a pointer type, got ssize`	`noalias` on a non-pointer parameter.
`function directives must appear before where clauses`	A directive written after a `where`; the order in a signature is fixed.
`polymorph $T is already bound; use plain T after the first $T`	`(a: $T, b: $T)`.
`lambda captures local 'base', but captured closures are not supported`	A lambda reading an enclosing local. It carries `help: pass the value as an explicit callback or context argument`.
`discarded return value of #must function 'mkval'`	Calling a `#must` procedure as a statement.
`variadic parameter uses '...T'; '..' is the range operator`	`..T` written where `...T` was meant.
`#empty parameters must appear before runtime parameters`	An anonymous erased witness appears after a runtime parameter.
`#self cannot be used in a parameter default; defaults are evaluated at the call site, where the receiver is not in scope`	A method default referring to its receiver.
`parameter 'x' is already declared`	A repeated parameter name, with a note at the first declaration. A blank `_` parameter may repeat.
`argument for #comptime parameter 'n' must be known at compile time`	A runtime value at a `#comptime` parameter; the attached `note` names the binding.

Aggregates and built-in sugar

Message	Produced by
`struct member 'x' must end with ';'`	A missing member terminator. Unions and enums have the same message with their own noun.
`enum variant integer values use '::', not '='`	`A = 1;` in an enum body.
`bracket type expects [], [*], [..], [N], or [_]`	Any other bracket form in a type.
`the built-in option type is written '?T'; its variants are '.Some(x)' and '.None'`	Writing `Option(T)`. The result form has the matching message for `Result(E, T)`.
`the built-in Context type is written #Context`	Naming `Context` without the `#`.
`no field 'z' in 'V'`	An unknown member, with `help: did you mean 'x'?` when a near name exists.
`member of non-struct`	A member access whose left side is not a struct, including `p.field` on a `*T`: a single pointer does not auto-dereference, so write `p.*.field`.
`cannot take a pointer to a non-addressable expression`	`&42` or `42.&`: address-taking needs a storage location.
`operator '*' expects a typed pointer`	Prefix `*` applied to an ordinary non-pointer value; use `&value` to take its address.
`left side of assignment is not assignable`	Writing through `*p` or `p.*` when `p` points to read-only storage.
`fixed array literal has 2 element(s), expected 3`	An array literal whose length disagrees with the annotation.
`[]ssize has no field capacity`	Asking a view for a dynamic array's field.
`field 'x' is already declared`	A repeated struct field, with a note at the first declaration. Duplicate union and enum variants have corresponding messages; `void` fields are supported.
`struct 'A' already converts through its '#as' field 'x'; a struct has only one '#as' field`	A second conversion field in one struct.
`a bracket type has no struct literal; write the array as '.[...]'`	A bracket type followed by `.{...}`. Generic struct literals such as `Box(ssize).{ .value = 3 }` are supported.

Modules and feature gates

Message	Produced by
`cannot find module "NoSuchModule" (searched: …)`	`#import` of a name that is neither file nor directory on the search path. The message lists every root it looked in, the standard library first and then each `--module-path` directory.
`ambiguous module "X": both … and … exist`	A module that is both `std/X.glo` and `std/X/module.glo`.
`#load: cannot find file "mod/nope.glo" (looked for …)`	A `#load` path that does not resolve relative to the loading file.
`#import,only names unknown public declaration 'no_such_thing' in module "Io"`	A selector naming something the module does not export; a `note` lists the public names.
`#import modifier must be only or hide`	Any other modifier after `#import,`.
`#library modifier must be system, dyn, or static`	Any other link kind. Two kinds at once give `#library accepts only one link-kind modifier`.
`#foreign refers to unknown library 'nolib' (declare it with #library)`	A `#foreign` naming a library handle that was never declared.

Names and resolution

Message	Produced by
`unbound 'nosuchname'`	An identifier in value position with no binding, including a capture used after the `if` that bound it and a private name read from another file.
`unknown function nosuchfn`	A call to a name that is not a procedure, with `help: did you mean …?` when a near name exists.
`no overload 'f' accepts these arguments`	Every candidate failed. A `note` lists them, one `candidate:` line each.
`constraint failed for double`	Every candidate matched on types but a `where` clause evaluated false.

Message	Produced by
`cannot infer a type here; annotate it or give it a use that fixes one`	A binding with nothing to read a type from, such as a bare lambda.

Types and conversions

Coercion failures use one shape, `got A, expected B`, for every position: an annotated initialiser, an argument, a `return`, and a literal that has already been defaulted. The compile-time number kinds get their own wording, because the failure there is the conversion rule rather than a type mismatch.

Message	Produced by
`got u8, expected s8`	A conversion that is not a lossless widening. Only widening is implicit, and floats never convert implicitly in either direction.
`cannot implicitly convert comptime_float to ssize`	A fractional literal at an integer type. The integer form names `comptime_int`.
`cannot cast string to ssize`	A `cast` between families that have no conversion.
`cast requires a type in parentheses; use acast for contextual casts`	`cast T value`.
`acast needs a target type from context or an explicit cast type`	`acast` in a position that carries no expected type.
`'null' is not a value of *s64`	`null` at a plain `*s64`, with two notes naming `?*s64` and `.None`.
`cast to *s64 of an address that is zero at compile time`	`cast(*s64) 0`, with notes explaining the niche that makes it illegal.
`got ssize, expected Option_ssize`	`?T` does not wrap implicitly; write `.Some(x)`. The diagnostic uses the instantiated union's internal name.
`got []u8, expected string`	A byte view does not convert implicitly to a string view; an explicit `cast(string)bytes` borrows its storage.
`string has no field capacity`	Strings are views with read-only descriptors. Assigning to their `.count`, `.data`, or indexed code units reports `left side of assignment is not assignable`.

Operators

Message	Produced by
`no operator '+' for u8, u16`	Operators are homogeneous: there is no implicit promotion of one operand to the other's type.
`no operator '%' for f64, f64`	`%` is integer-only.
`no operator '*' for Meters and Meters: a distinct type gets only the classes it derives; add '#derive(...)' or '#derive,all' on the declaration of 'Meters'`	An operator on a `distinct` type that did not derive the class.
`shift count 8 is not less than the 8 bits of u8`	A compile-time shift count at or past the width, with `note: a shift by the width or more has no defined result`.
`operator '<+>' must be declared by a typeclass and implemented by its instances`	A free operator function at file scope. There are no free operator overloads.
`operator '+' is already owned by typeclass 'Add'`	A second class claiming a spelling at the same arity and fixity.
`binary operator '<+>' needs #operator(left|right|assign, n)`	A binary operator declared with no precedence anywhere.
`operator precedence level must be between 0 and 1000`	A level outside the accepted range.
`operators '**' and '~>' both have precedence 19 but different associativity, so '** ~>' is ambiguous; add parentheses`	Mixing two same-tier operators that disagree on associativity.

Typeclasses and instances

Message	Produced by
`typeclass 'Show' is already declared`	Re-declaring a class name. Class names live in one global namespace.

Message	Produced by
`function 'show' is already owned by typeclass 'Show'`	A second typeclass claiming a method name another class already owns. A plain declaration of the same name in a user file gives `function 'show' is owned by typeclass 'Show' and cannot be redeclared` instead.
`instance 'Dog :: Greet' does not satisfy #minimal(greet)`	An instance short of its minimal complete definition, with `note: minimal complete definition declared here`.
`ambiguous typeclass operation 'greet' for these argument types`	Two applicable instances scored equally. Identical semantic instance heads are rejected when declared.
`associated type 'K.Elem' is ambiguous; provide type arguments`	An argument-free projection with more than one supplying instance.
`associated type arguments must follow the typeclass name; write 'K(S).Elem' instead of 'K.Elem(S)'`	Arguments attached to the associated type rather than the class.
`typeclass 'Plain' does not declare a derive macro`	`#derive(C)` for a class with no `#derive` hook of its own.
`non-constant function call`	An evidence query in a position the `where` evaluator cannot fold, such as `where ! C(T)` or `where A(T) || B(T)`.

`Try`, `Termination`, and `Show`

Message	Produced by
`postfix '?' requires a Try-returning function or an enclosing #try block`	`?` with no boundary to break out to.
`no suffix operator '?' for ssize`	`?` on a value whose type has no `Try` instance. A type that has one but disagrees about the branch type gives `postfix '?' operand type T does not implement Try`.
`cannot infer the carrier type of #try expression; propagate at least one value with '?'`	A `#try` block with no `?` inside it.
`#try contains one expression; remove the semicolon`	A `#try` block whose single expression ends in `;`. Anything else in the block gives `#try requires one expression`, and a missing block gives `#try requires a block`.
`#try defer requires a Try-returning function or an enclosing #try block`	An error-only defer with no error path to run on.
`main return type Thing does not implement Termination`	A `main` result outside the integers, `E!void`, and types with a hand-written instance.
variadic arg type `Plain` does not implement `Show` in call `print`	A `print` argument with no `Show` instance. The message quotes the type, the class, and the callee in backticks. Expanding a pack gives `expanded variadic element type …` instead.

Control flow, switches, and patterns

Message	Produced by
`break outside loop`	`break` at function level, and also inside `#inline while` or a genuinely unrolled `#inline for`, which have no loop to leave.
`'for *x:' needs a PointerIterable instance for string`	A `for *x:` over a type that has an `Iterable` instance but no `PointerIterable` one. A string loop decodes and yields `rune` values, while direct indexing still reads a `u8` or `u16` code unit; decoded values cannot be borrowed.
`'P' is not iterable; it needs an Iterable instance`	A `for` over a type with no `Iterable` instance and no `for_expansion` macro.
`non-exhaustive switch`	A switch the coverage analysis cannot prove total, a tagged-union switch that leaves a variant uncovered included: `if s == { … }` is always a pattern switch. Up to six `note: missing case: …` witnesses follow, then `...and N more`, and last `note: cover them, or add 'else' or '#partial'`.
`cannot switch on a raw union (it has no tag)`	`if u == { … }` on a `union #raw`.
`tagged-union payload '.Circle' may only be bound by a pattern match`	Reading `value.Variant` directly, with a `note` naming `#enable(allow_union_unwrap)`.
`'Triangle' is not a variant of union U`	A `case` label naming a variant the subject's union does not declare.

Message	Produced by
`cannot decide a value: an 'ifx' needs an 'else' arm`	The conditional `ifx` without `else`. The switch forms give `cannot decide a value for this ifx` plus the missing cases.
`an ifx arm must end in an expression`	An `ifx` arm whose last statement produces nothing.
`a '#pattern' test cannot be negated`	`!(#pattern P = e)`, with a `note` pointing at the `#pattern P := e else { … }` form.
`the 'else' of a '#pattern ... :=' binding must not return normally`	A bind-or-diverge `else` that can fall through.
`#if condition must be bool, got comptime_int`	`#if` on a value that is truthy but not `bool`; `#if` is stricter than `if`.
`no label 'nowhere' in this procedure (define it with 'nowhere:)`	`goto` at a label the procedure does not declare.

Compile-time evaluation

Message	Produced by
`unknown const i`	A runtime binding read in a position that must fold, such as a `#if` condition or a range bound in a pattern.
`assertion failed: math is broken`	A `#assert` whose condition folds to false. Without a message the text is `assertion failed`.
`unsupported: widget`	`#compile_error`; the message is the expression's value, printed with no prefix of its own.
`#inline while condition must be compile-time evaluable`	A runtime condition on an unrolled loop.
`#inline while made no compile-time progress; update a compile-time variable used by the condition`	A body that never changes what the condition reads.
`#inline for expects a compile-time array`	`#inline for` over something other than a tuple or fixed array; a dynamic array gives its own message about runtime length.
`#insert generated invalid source: parse error line 1: unexpected token ';'`	Generated text that does not parse, with a `note` and a frame pointing into the cached generated file.
`#insert needs a Code value or a compile-time string`	A statement or declaration splice whose operand folds to any other value kind.
`#insert expects scope after ','`	A modifier other than `scope` after `#insert,`; the accepted form is `#insert,scope(...) value`.
`a 'Code' value is compile-time only; bind it with '::' rather than ':='`	`:=` on a `Code` value. `Pattern` and values produced by named definitions are compile-time-only too.
expression macro 'f' must contain exactly `return <expr>;`	The legacy inferred-result `#expand,expression` form has a body other than one return statement (or equivalent static branches). Explicitly typed value macros may use a multi-statement body when every path returns; a missing path reports `expanded value macro does not return a value on every path`.
`compile-time execution does not support u129, an integer wider than 128 bits`	A `#comptime { ... }` VM step that executes a value of an integer type wider than 128 bits. Arbitrary-precision constant folding and compiled code are not limited this way.

Memory contracts

Temporal memory findings use warning: by default and error: with --borrow-check=error. They name the memory rule and carry notes for the storage origin, release, or conflicting borrow. --borrow-check=off disables body analysis. Written contracts and memory_strict boundaries are checked separately from managed-value move errors.

Message	Produced by
`[memory.dangling_return] pointer to local storage escapes through return`	Returning a pointer into a local whose storage ends at function exit.
`[memory.double_release] double release of resource`	Releasing an already discharged acquisition obligation.
`[memory.use_after_release]`	Access through a tracked resource or derived view after release.
`[memory.stale_reference]`	Using a retained view after its storage was invalidated.

Message	Produced by
`[memory.X1] noalias argument overlaps another live reference`	A call violates the unique-access promise of a noalias parameter.
`body of 'modify' has undeclared writes effect on 'value'`	The implementation writes through a parameter that its contract did not permit it to write.
`exported function 'view' requires a #memory contract under memory_strict`	An exported memory boundary lacks a written contract in a strict module.
`resource must be released by ...`	An acquired raw resource reaches scope exit without release, ownership transfer, or an explicit permitted opt-out.

Warnings

Warnings use the identical layout with `warning:` in place of `error:`. Besides the temporal-analysis family above there are three, and all three point at code that compiles but probably does not mean what it says.

Message	Produced by
`this expression changes meaning under the 0-20 precedence table: bitwise operators now bind tighter than comparisons`	An unparenthesised bitwise expression as an operand of a comparison, or the reverse, as in `flags & mask == 4`. It carries `note: add parentheses to say which reading you mean`.
`unreachable pattern - already covered by an earlier case`	An arm of a `#pattern` switch that the arms above it already cover.
`no '+' for <T>: the where-clause only provides Addable`	A generic definition uses an operator whose owning typeclass is absent from the parameter's `where` evidence. It is reported at the definition and carries `note: reported at the definition; add the class to the 'where' clause`.

Runtime messages

A program can also print. Runtime messages come from the compiled program, not the compiler, are written to standard error, and are followed by the target's abort path. Array bounds, union-tag failures, and assertions name a source position when available; the shorter category reports do not.

CONSOLE / OUTPUT

```
…\k11.glo:5:46: union access: expected .Box, but the value holds one of .Circle | .Box
glosso: program exited with status exit code: 134
```

Message	Produced by
`union access: expected .Box, but the value holds one of .Circle | .Box`	An `#enable(allow_union_unwrap)` payload read whose tag check failed at `-O0` or `-O1`.
`assertion failed: x must be big`	A `#assert` whose condition was not compile-time known and was false at run time.
`cast to *s64 of an address that is zero`	A runtime `cast(*s64)` of a null address; another destination type is named when one was written.
`glosso debug allocator: leaked 1 allocation(s), 64 byte(s)`	The debug allocator, enabled by a nonzero `GLOSSO_DEBUG_ALLOCATOR`, at exit; each leak is listed with its allocation site.
`glosso debug allocator: double free`	The same allocator on a repeated release. Use-after-free, invalid free, and invalid realloc have matching lines.
`runtime check failed: arithmetic.overflow`	Checked integer addition, subtraction, multiplication, or negation overflowed.
`runtime check failed: divide.by.zero` or `divide.overflow`	A checked integer division or remainder used zero, or signed division evaluated `MIN / -1`.
`<path>:<line>:<column>: runtime check failed: array.bounds`	A checked index was outside an array, view, `string`, or `string16` count. C-string indexing does not scan for a bound.
`runtime check failed: slice.bounds`	A checked slice range was negative, inverted, or extended past the source count.
`integer formatting base is outside 2..=16`	`value.format_int` used an unsupported base in a checked build.

NOTE

These backend guards are emitted only at `-O0` and `-O1`. Hosted Windows and Linux x86-64 terminate them with status 134, WASI uses 125, and the remaining targets call `abort()`. At `-O2` and `-O3`, overflow and bounds guards are absent; bad division reaches LLVM and the target hardware instead of a Glosso report.

16.4 Style Guidelines for Glosso Code

The compiler enforces none of what follows. These are the conventions the standard library holds to, and the reason to hold to them is that a program spends most of its text calling that library: code written in the same shapes reads as one body of work rather than two.

Naming

There are four name shapes and the shape says what kind of thing the name binds. Nothing in the language reads a name's case, so the shape is carrying information the compiler does not: a reader who sees `Slot_Table` knows it is a type before finding the declaration.

Kind	Shape	Examples
Types: struct, union, enum, distinct	`Upper_Snake_Case`	`Slot_Table`, `Io_Error_Kind`, `Date_Time`
Typeclasses	`PascalCase`, no underscores	`Show`, `PartialEq`, `FromString`, `Indexable`
Enum variants	`Upper_Snake_Case`	`Read_At`, `Unexpected_Eof`, `Out_Of_Memory`
Flag-enum members	`Upper_Snake_Case`	`Append`, `No_Follow`, `Owner_Read`
Constants	`UPPER_SNAKE`	`MAX_SLOTS`, `DEFAULT_TEMP_ALLOCATOR_SIZE`
Procedures, parameters, fields, locals	`lower_snake_case`	`slot_table_new`, `capacity`, `it_index`
Module-internal declarations	a __ prefix on the usual shape	`__grid_index`, `__BTreeMap_Node`

The library holds this line exactly: of more than three thousand top-level procedures in `std`, not one is spelled in camel case. The typeclass exception is deliberate. A class name appears inside type expressions, in `where` clauses, and as an instance head, always beside real type names, and dropping the underscores is what keeps `where Eq(T) && Hash(T, DefaultHasher)` legible.

A procedure that operates on a type is named after that type: the module prefix comes first and the verb second, so every operation on a value sorts together in a listing and in a completion menu.

GLOSSO

```
MAX_SLOTS :: 8;
DEFAULT_NAME : string : "unnamed";

Slot_State :: enum { Free; Taken; Retired; }

Slot_Flags :: enum_flags u32 {
    Pinned :: 16#1;
    Dirty  :: 16#2;
}

Slot_Table :: struct {
    name: string;
    used: ssize;
}

Describable :: typeclass (T: type) #minimal(describe) {
    describe :: (value: T) -> string;
}

Slot_Table ::: instance Describable {
    describe :: (value: Slot_Table) -> string { return value.name; }
}

slot_table_new :: (name: string) -> Slot_Table {
    return .{ .name = name, .used = 0 };
}

slot_table_is_full :: (table: Slot_Table) -> bool {
    return table.used >= MAX_SLOTS;
}
```

NOTE

Names collide across the whole program, so the prefix habit is doing real work. A typeclass method name is reserved with no import at all: `show :: (x: ssize) -> ssize { … }` is `error: function 'show' is owned by typeclass 'Show' and cannot be redeclared`, and so are `next`, `from_string`, and every operator name. Anything an import brings in collides too: with `#import "Math";` in the file, `PI :: 3.14;` is `error: constant 'PI' is already declared` and `abs` is `error: function 'abs' is already declared`.

Layout

- Four spaces per level, never a tab. There is not one tab character in `std`.

- Lines wrap near 80 columns; nine lines in ten of the library are under 76.
- The opening brace stays on the line that opened the construct, and the closing brace sits at the owner's indentation.
- One blank line between top-level declarations; none inside a short body.
- Semicolons terminate top-level declarations that do not end in their own definition body, and ordinary simple statements inside bodies. A small set of statement forms accepts an optional `;`; switch case and `else` labels require one, and a label statement requires `;` or the legacy `:`. The library writes semicolons consistently.

A signature too long for one line breaks after the open parenthesis, puts one parameter per line, keeps a trailing comma on the last one, and returns the `)` to the declaration's indentation. Directives and `where` clauses then go on their own lines, in that order, before the body's brace.

GLOSSO

```
slot_table_reserve :: (
    table: *mut Slot_Table,
    count: ssize,
    label: string = "reserve",
) -> ssize  {
    table.*.used += count;
    return table.*.used;
}
```

A switch puts its `case` labels one level inside the braces, not at the brace's own indentation. A one-line arm follows its label on the same line; anything longer starts on the next line, indented once more. The pattern is worth keeping because a `case` label is not a block and a reader needs the indentation to see where an arm ends.

GLOSSO

```
stream_error_reason :: (kind: Stream_Error_Kind) -> string {
    if kind == {
        case .None; return "no error";
        case .Eof;  return "end of file";
        case .Short_Write;
            report_short_write();
            return "short write";
        else; return "unknown error";
    }
}
```

File and declaration order

Nothing in Glosso requires an order: `main` may call a procedure declared below it, and `#import` may sit at the bottom of the file. The library still fixes one, because a file read top to bottom should introduce a name before it is used.

Position	Contents
1	A header comment naming the file and stating in one line what it is for, then a short paragraph of what a caller must know.
2	`#import`, `#load`, and `#export` lines.
3	The module's error type, then its data types, then its typeclasses and instances.
4	Constructors, then queries, then mutators, then the destructor, grouped by the type they belong to.
5	Private helpers, optionally prefixed with `#private`; ordinary declarations are file-private by default.

GLOSSO

```
// grid.glo - a fixed grid with checked access.
//
// A Grid owns no memory; grid_new copies its stride and nothing else.

#import "Io";

#public Grid_Error :: enum { Out_Of_Range; }

#public Grid :: struct {
    cells: [9]s64;
    stride: ssize;
}

#public grid_new :: (stride: ssize) -> Grid {
    return .{ .stride = stride };
}

#public grid_get :: (grid: *mut Grid, row: ssize, col: ssize) -> Grid_Error!s64 {
    index := __grid_index(grid, row, col);
    if index < 0 return .Err(.Out_Of_Range);
    return .Ok(grid.*.cells[index]);
```

```
}

#private __grid_index :: (grid: *mut Grid, row: ssize, col: ssize) -> ssize {
    if row < 0 || col < 0 || col >= grid.*.stride return -1;
    return row * grid.*.stride + col;
}
```

Visibility prefixes apply to a declaration or a transparent braced group. `#public { ... }` exports that group's declarations; a nested `#private` overrides it and the enclosing default resumes afterward. Use `#public(unit)` when helpers should be shared across source files in one ownership unit. The removed `private_section` and `siblings` feature-gate spellings produce migration diagnostics.

> **NOTE**
> The entry file's `main` may keep its default private visibility. Language visibility and native symbol export are separate: `#c_call` still supplies library entry points, while another Glosso module needs an accessible declaration to use their Glosso names.

Documenting ownership

Document whether an API moves an owner, borrows storage, returns a fresh allocation, or transfers a raw buffer to the caller. Describe mutation that invalidates views and the allocator or foreign function needed to release raw results. Use Drop for automatic cleanup and pointers for helpers that leave ownership with their caller.

When to reach for a typeclass

A typeclass is the only overloading mechanism the language has, so the question is never "class or free function" but "class or generic procedure". A `where` clause on a generic procedure says what the caller's type must already be able to do; a typeclass says what a type must newly declare. Prefer the former until a second implementation exists.

- One behaviour, many types, each needing its own body: a typeclass. `Show` and `Hash` are this.
- One body that works for every type meeting a condition: a generic procedure with `where`. No class is needed.
- A new operator spelling: a typeclass, always. An operator must have exactly one owning class, and a free operator function is rejected.
- A name that must be callable before its implementations exist, such as a callback a library invokes: a typeclass, so the library can constrain on it.
- A marker with no methods: a typeclass with `#minimal()`, queried with `#if C(T)`.

Give every class parameter a position in every method's signature. A default method that omits one cannot be dispatched, because nothing at the call site decides it; an anonymous `#empty:` witness parameter is the standard way to give a type a position without giving it a value. State the minimal definition with `#minimal` even when it is a single name, because that is what turns an incomplete instance into one error at the instance rather than a missing overload at every call.

GLOSSO

```
Byte_Size :: typeclass (T: type) #minimal(byte_size) {
    byte_size :: (value: T) -> ssize;
}

Packet :: struct { payload: s64; }

Packet ::: instance Byte_Size {
    byte_size :: (value: Packet) -> ssize { return 8; }
}

$T ::: instance Byte_Size where Is_Integer(T) {
    byte_size :: (value: T) -> ssize { return size_of(T); }
}

// a class, because each type answers differently
total_size :: (a: $A, b: $B) -> ssize where Byte_Size(A) && Byte_Size(B) {
    return byte_size(a) + byte_size(b);
}

// no class, because one body serves every integer
doubled :: (value: $T) -> T where Is_Integer(T) {
    return value + value;
}
```

> **NOTE**
> Instance selection is ordinary overload resolution, so it runs through implicit coercions: a concrete `s64` instance will capture a `u32` argument, because `u32` widens losslessly and a concrete parameter scores higher than a generic one. When an instance must key on exactly one type, write a blanket instance guarded by `where T == s64` rather than a concrete head.

Errors

A procedure that can fail returns `E!T`; a procedure whose only failure is absence returns `?T`. The library gives each module one error type named after the module, propagates with `?` rather than hand-written branching, and widens between modules with `#from` variants so a caller's union absorbs a callee's error without a conversion function.

GLOSSO

```
#import "Io";
#import "Typeclass";
#import "Typeclass/Error";

Parse_Error :: struct { text: string; }

Parse_Error ::: instance Error {
    report_error :: (error: Parse_Error) -> void {
        ("bad number").error_report_line(error.text);
    }
}

App_Error :: union #derive(Error) {
    #from parse: Parse_Error;
}

parse_count :: (text: string) -> Parse_Error!s64 {
    value := from_string(s64, text);
    if #pattern .Some(n) = value return .Ok(n);
    return .Err(.{ .text = text });
}

main :: () -> App_Error!void {
    n := parse_count("17")?;
    print("n = %\n", n);
    bad := parse_count("nope")?;
    return .Ok();
}
```

CONSOLE / OUTPUT

```
n = 17
bad number: nope
```

That program exits with status 1 and prints its report to standard error, without a line of exit-code handling: `main` returning `App_Error!void` dispatches through `Termination`, which reports the error and returns 1. Reserve `#assert` for conditions a caller cannot cause and an error value for everything a caller can.

Comments

The library comments the contract, not the mechanism. A file header is a `//!` block stating what the module is for and what a caller must guarantee; a declaration carries a `///` block stating the invariant a reader could not derive from the signature, under `#` headings of which `# Errors`, `# Ownership` and `# Invalidation` are the common three. Line comments inside a body are rare, and `// ----` section rules divide long files.

GLOSSO

```
//! std/Container/HashSet.glo - hash set implemented as HashMap(T, u8).
//!
//! HashSet(T) has the same shallow single-owner and Eq/Hash requirements
//! as HashMap.  Iteration order is arbitrary, and structural mutation
//! invalidates pointers and iterators.

/// Creates a set with the explicit key `(k0, k1)`, presized for `capacity`.
///
/// # Ownership
/// The returned set owns its table; release it with `HashSet(T).deinit`.
```

- Say what a caller must guarantee, not what the next statement does.
- Put the ownership story in the header: who allocates, who releases, what invalidates what.
- Name a known limitation where a reader will hit it, rather than in a change log.
- A `/* */` comment nests, so it is safe to comment out a region that already contains one; an unterminated one is the lexical error `unterminated block comment`.

Source and reproduction

The language content comes from the 146 sections declared in `src/section_catalog.glo` and loaded by `main.glo`. Topic groups follow `manualSidebarGroups` in `app.js`. Content is reproduced from the repository sources without changing the language examples or claims.

The Typst file reads a checked-in content snapshot. Building this document needs Typst and the adjacent `reference-manual.json` refreshing the snapshot also needs Node.js. No external Typst packages, network access, or Glosso compiler are needed. The font lists fall back to Libertinus Serif / New Computer Modern, DejaVu Sans / Liberation Sans, and DejaVu Sans Mono / Liberation Mono.

```
node scripts/generate-typst-reference.mjs
typst compile docs/glosso-reference.typ docs/glosso-reference.pdf
node scripts/generate-typst-reference.mjs --check
```

Run these commands from the repository root. The check compares the snapshot with the current manual and reports an error if the sources have changed. To update the document, refresh the snapshot before compiling the PDF.

Manual sources

`src/sections_getting_started.glo`
`src/sections_lexical.glo`
`src/sections_declarations.glo`
`src/sections_types.glo`
`src/sections_aggregates.glo`
`src/sections_pointers.glo`
`src/sections_functions.glo`
`src/sections_typeclasses.glo`
`src/sections_operators.glo`
`src/sections_control_flow.glo`
`src/sections_comptime.glo`
`src/sections_memory.glo`
`src/sections_runtime.glo`
`src/sections_interop.glo`
`src/sections_targets.glo`
`src/sections_reference.glo`

Snapshot SHA-256

`6a0d91afec9e7f33acfb486c94fff03e230b4286f3dbb6dd31482ee84f03c80e`